Aspera Orchestrator v4.1.6 Plugins Manuals

2026/10/20

Contents

1 Asset Management **36**

1.1 Dalet Asset Management Plugin . . . 36
1.1.1 Saved Parameters Description . . . 36
1.1.2 Inputs description . . . 36
1.1.3 Outputs description . . . 37
1.1.4 Supported Actions . . . 37
1.1.5 Operating Instructions . . . 37
1.2 Evertz Mediator operation . . . 37
1.2.1 How It Works . . . 37
1.2.2 Configuration Instructions . . . 37
1.2.3 Example Configuration . . . 38
1.2.4 Saved Parameters Description . . . 38
1.2.5 Inputs Description . . . 39
1.2.6 Outputs Description . . . 39
1.2.7 Supported Actions . . . 39
1.2.8 Dependencies . . . 39
1.2.9 Operating Instructions . . . 39
1.3 ShowMgr . . . 40
1.3.1 Saved Parameters Description . . . 40
1.3.2 Inputs description . . . 41
1.3.3 Outputs description . . . 41
1.3.4 Supported Actions . . . 41
1.3.5 Dependencies . . . 41
1.3.6 Operating Instructions . . . 41
1.4 Sony Ci . . . 41
1.4.1 How It Works . . . 42
1.4.2 Configuration Instructions . . . 42
1.4.3 Example Configuration . . . 42
1.4.4 Saved Parameters Description . . . 42
1.4.5 Inputs Description . . . 43
1.4.6 Outputs Description . . . 43
1.4.7 Supported Operations . . . 43
1.4.8 Operating Instructions . . . 43
1.4.9 Dependencies . . . 44
1.4.10 Troubleshooting . . . 44
1.5 The Platform . . . 44
1.5.1 Saved Parameters Description . . . 45
1.5.2 Authentication Parameters Start . . . 45
1.5.3 Authentication Parameters End . . . 45
1.5.4 Inputs description . . . 46
1.5.5 Outputs description . . . 47
1.5.6 Operating Instructions . . . 47
1.5.7 Examples given below . . . 48
1.5.8 An example of Search . . . 48
1.5.9 An example of Create Media Object . . . 48
1.5.10 An example of Create Media File Object . . . 48
1.6 XytechMediaPulse . . . 48

1.6.1 Inputs description . . . 49
1.6.2 Outputs description . . . 49
1.6.3 Operating Instructions . . . 49

2 File Operations **50**
2.1 Archive manager . . . 50
2.1.1 How It Works . . . 50
2.1.2 Configuration Instructions . . . 51
2.1.3 Example Configuration . . . 51
2.1.4 Saved Parameters Description . . . 51
2.1.5 Inputs Description . . . 52
2.1.6 Outputs Description . . . 52
2.1.7 Supported Actions . . . 52
2.1.8 Dependencies . . . 52
2.1.9 Operating Instructions . . . 52
2.2 Aspera Node API . . . 53
2.2.1 How It Works . . . 53
2.2.2 Configuration Instructions . . . 53
2.2.3 Example Configuration . . . 54
2.2.4 Saved Parameters Description . . . 54
2.2.5 Inputs Description . . . 54
2.2.6 Outputs Description . . . 55
2.2.7 Supported Actions . . . 55
2.2.8 Dependencies . . . 55
2.2.9 Node API Usage Overview . . . 55
2.3 Path conversion . . . 56
2.3.1 How It Works . . . 56
2.3.2 Configuration Instructions . . . 56
2.3.3 Example Configuration . . . 56
2.3.4 Saved Parameters Description . . . 57
2.3.5 Inputs Description . . . 57
2.3.6 Outputs Description . . . 57
2.3.7 Supported Actions . . . 57
2.3.8 Dependencies . . . 57
2.3.9 Operating Instructions . . . 57
2.4 Dependency checker . . . 57
2.4.1 How It Works . . . 58
2.4.2 Configuration Instructions . . . 58
2.4.3 Example Configuration . . . 58
2.4.4 Saved Parameters Description . . . 59
2.4.5 Inputs Description . . . 59
2.4.6 Outputs Description . . . 59
2.4.7 Supported Actions . . . 59
2.4.8 Dependencies . . . 59
2.4.9 Operating Instructions . . . 59
2.4.10 Troubleshooting . . . 59
2.5 Diva archive . . . 60
2.5.1 How It Works . . . 60
2.5.2 Configuration Instructions . . . 60
2.5.3 Example Configuration . . . 61
2.5.4 Saved Parameters Description . . . 61
2.5.5 Inputs Description . . . 62
2.5.6 Outputs Description . . . 62
2.5.7 Supported Actions . . . 63
2.5.8 Dependencies . . . 63
2.5.9 Operating Instructions . . . 63
2.6 Exiftool xmp tagger . . . 66
2.6.1 Saved Parameters Description . . . 66
2.6.2 Inputs description . . . 67
2.6.3 Outputs description . . . 67

2.6.4 Supported Actions . . . 67
2.6.5 Dependencies . . . 67
2.6.6 Operating Instructions . . . 67
2.7 FFProbe Information . . . 68
2.7.1 How It Works . . . 68
2.7.2 Configuration Instructions . . . 68
2.7.3 Example Configuration . . . 68
2.7.4 Saved Parameters Description . . . 69
2.7.5 Inputs Description . . . 69
2.7.6 Outputs Description . . . 69
2.7.7 Supported Actions . . . 69
2.7.8 Dependencies . . . 70
2.7.9 Operating Instructions . . . 70
2.8 File archiving operation . . . 71
2.8.1 How It Works . . . 72
2.8.2 Configuration Instructions . . . 72
2.8.3 Example Configuration . . . 72
2.8.4 Saved Parameters Description . . . 72
2.8.5 Inputs Description . . . 73
2.8.6 Outputs Description . . . 73
2.8.7 Operating Instructions . . . 73
2.8.8 Dependencies . . . 73
2.9 File generator . . . 74
2.9.1 How It Works . . . 74
2.9.2 Configuration Instructions . . . 74
2.9.3 Example Usage . . . 75
2.9.4 *Same example with suppression marked linefeeds* . . . 75
2.9.5 Example Configuration . . . 75
2.9.6 Saved Parameters Description . . . 75
2.9.7 Inputs Description . . . 76
2.9.8 Outputs Description . . . 76
2.9.9 Supported Actions . . . 76
2.9.10 Dependencies . . . 76
2.9.11 Troubleshooting . . . 76
2.10 File information . . . 76
2.10.1 How It Works . . . 77
2.10.2 Configuration Instructions . . . 77
2.10.3 Example Configuration . . . 77
2.10.4 Saved Parameters Description . . . 77
2.10.5 Inputs Description . . . 77
2.10.6 Outputs Description . . . 77
2.10.7 Supported Actions . . . 78
2.10.8 Dependencies . . . 78
2.10.9 Operating Instructions . . . 78
2.11 ISM and M3U8 File Parser . . . 78
2.11.1 Saved Parameters Description . . . 78
2.11.2 Inputs description . . . 79
2.11.3 Outputs description . . . 79
2.11.4 Operating Instructions . . . 79
2.12 Local file operation . . . 79
2.12.1 How It Works . . . 80
2.12.2 Configuration Instructions . . . 80
2.12.3 Example Configuration . . . 81
2.12.4 Saved Parameters Description . . . 81
2.12.5 Inputs Description . . . 81
2.12.6 Outputs Description . . . 82
2.12.7 Supported Actions . . . 82
2.12.8 Operating Instructions . . . 82
2.12.9 Dependencies . . . 82
2.12.10 Troubleshooting . . . 82

2.13 Local file permission . . . 83
2.13.1 How It Works . . . 83
2.13.2 Configuration Instructions . . . 83
2.13.3 Example Configuration . . . 83
2.13.4 Saved Parameters Description . . . 84
2.13.5 Inputs Description . . . 84
2.13.6 Outputs Description . . . 84
2.13.7 Supported Actions . . . 84
2.13.8 Dependencies . . . 84
2.13.9 Operating Instructions . . . 85
2.14 MD5 checksum . . . 85
2.14.1 How It Works . . . 85
2.14.2 Configuration Instructions . . . 85
2.14.3 Example Configuration . . . 86
2.14.4 Saved Parameters Description . . . 86
2.14.5 Inputs Description . . . 86
2.14.6 Outputs Description . . . 86
2.14.7 Supported Actions . . . 86
2.14.8 Dependencies . . . 86
2.14.9 Operating Instructions . . . 86
2.14.10 Troubleshooting . . . 86
2.15 Media info . . . 87
2.15.1 How It Works . . . 87
2.15.2 Configuration Instructions . . . 87
2.15.3 Example Configuration . . . 88
2.15.4 Saved Parameters Description . . . 88
2.15.5 Inputs Description . . . 88
2.15.6 Outputs Description . . . 89
2.15.7 Operating Instructions . . . 89
2.15.8 Dependencies . . . 90
2.15.9 Troubleshooting . . . 90
2.16 Mediabin asset management . . . 90
2.16.1 Saved Parameters Description . . . 91
2.16.2 Inputs description . . . 91
2.16.3 Outputs description . . . 91
2.16.4 Operating Instructions . . . 91
2.17 Remote file operation . . . 91
2.17.1 How It Works . . . 92
2.17.2 Configuration Instructions . . . 92
2.17.3 Example Configuration . . . 92
2.17.4 Saved Parameters Description . . . 92
2.17.5 Inputs Description . . . 93
2.17.6 Outputs Description . . . 93
2.17.7 Supported Actions . . . 93
2.17.8 Dependencies . . . 93
2.17.9 Operating Instructions . . . 94
2.17.10 Aspera Configuration Requirements . . . 94
2.17.11 File and Directory Operations . . . 94
2.17.12 Base Directory Functionality . . . 94
2.17.13 Troubleshooting . . . 94
2.18 SglFlashnetArchive . . . 95
2.18.1 Inputs description . . . 95
2.18.2 Outputs description . . . 95
2.18.3 Supported Actions . . . 96
2.18.4 Dependencies . . . 96
2.18.5 Operating Instructions . . . 96
2.19 Spreadsheet parser . . . 96
2.19.1 How It Works . . . 97
2.19.2 Configuration Instructions . . . 97
2.19.3 Saved Parameters Description . . . 97

2.19.4 Inputs Description . . . 97
2.19.5 Outputs Description . . . 98
2.19.6 Supported Actions . . . 98
2.19.7 Dependencies . . . 98
2.19.8 Operating Instructions . . . 98
2.19.9 Example Output . . . 98
2.20 PGP encryption . . . 99
2.20.1 How It Works . . . 99
2.20.2 Configuration Instructions . . . 99
2.20.3 Example Configuration . . . 100
2.20.4 Saved Parameters Description . . . 100
2.20.5 Inputs Description . . . 101
2.20.6 Outputs Description . . . 101
2.20.7 Supported Actions . . . 101
2.20.8 Dependencies . . . 101
2.20.9 Operating Instructions . . . 101
2.21 Xml Parsing . . . 101
2.21.1 How It Works . . . 102
2.21.2 Configuration Instructions . . . 102
2.21.3 Example Configuration . . . 102
2.21.4 Saved Parameters Description . . . 102
2.21.5 Inputs Description . . . 103
2.21.6 Outputs Description . . . 103
2.21.7 Supported Actions . . . 103
2.21.8 Dependencies . . . 103
2.21.9 Operating Instructions . . . 103
2.21.10 Overview . . . 103
2.21.11 What is XPath? . . . 103
2.21.12 Basic Example . . . 103
2.21.13 Example with Namespaces . . . 104
2.21.14 Additional Resources . . . 104

3 File Transfer **105**
3.1 Ascp client . . . 105
3.1.1 How It Works . . . 106
3.1.2 Configuration Instructions . . . 106
3.1.3 Example Configuration . . . 106
3.1.4 Saved Parameters Description . . . 107
3.1.5 Source Files Format Options . . . 108
3.1.6 Inputs Description . . . 108
3.1.7 Outputs Description . . . 108
3.1.8 Supported Actions . . . 108
3.1.9 Operating Instructions . . . 108
3.1.10 Dependencies . . . 109
3.1.11 Troubleshooting . . . 109
3.2 Aspera Node API Transfer . . . 109
3.2.1 How It Works . . . 110
3.2.2 Configuration Instructions . . . 110
3.2.3 Example Configuration . . . 111
3.2.4 Saved Parameters Description . . . 111
3.2.5 Inputs Description . . . 112
3.2.6 Outputs Description . . . 112
3.2.7 Supported Actions . . . 112
3.2.8 Dependencies . . . 113
3.2.9 Operating Instructions . . . 113
3.2.10 Transfer Architecture . . . 113
3.2.11 Custom Payload Examples . . . 113
3.2.12 Node-to-Node Transfer with SSH Key . . . 113
3.2.13 ATS Upload with Basic Token . . . 113
3.2.14 Aspera On Cloud Package Upload with Bearer Token . . . 113

3.2.15 Authentication Methods . . . 113
3.2.16 Runtime Configuration . . . 113
3.3 Aspera Out-of-Transfer File Validation . . . 113
3.3.1 How It Works . . . 114
3.3.2 Configuration Instructions . . . 114
3.3.3 Example Configuration . . . 114
3.3.4 Saved Parameters Description . . . 115
3.3.5 Inputs Description . . . 115
3.3.6 Mandatory Inputs for 'Retrieve list of files' Operation . . . 115
3.3.7 Mandatory Inputs for 'Update file status' Operation . . . 115
3.3.8 Optional Inputs for 'Retrieve list of files' Operation . . . 116
3.3.9 Optional Inputs for 'Update file status' Operation . . . 116
3.3.10 Outputs Description . . . 116
3.3.11 Operating Instructions . . . 116
3.3.12 Dependencies . . . 116
3.4 Console Smart Transfer . . . 116
3.4.1 How It Works . . . 117
3.4.2 Configuration Instructions . . . 117
3.4.3 Example Configuration . . . 117
3.4.4 Saved Parameters Description . . . 117
3.4.5 Inputs Description . . . 118
3.4.6 Required Inputs: . . . 118
3.4.7 Optional Inputs: . . . 118
3.4.8 Outputs Description . . . 118
3.4.9 Supported Actions . . . 118
3.4.10 Dependencies . . . 118
3.4.11 Operating Instructions . . . 118
3.4.12 Important Limitations . . . 118
3.5 FASP controller . . . 118
3.5.1 How It Works . . . 119
3.5.2 Configuration Instructions . . . 120
3.5.3 Example Configuration . . . 120
3.5.4 Saved Parameters Description . . . 120
3.5.5 Inputs Description . . . 121
3.5.6 Outputs Description . . . 121
3.5.7 Supported Actions . . . 121
3.5.8 Dependencies . . . 121
3.5.9 Operating Instructions . . . 121
3.5.10 Plugin Overview . . . 121
3.5.11 Aspera Central API Setup . . . 121
3.5.12 Node API Setup . . . 122
3.5.13 Node API Account Configuration . . . 122
3.5.14 Transfer User Configuration . . . 122
3.5.15 Port Configuration . . . 122
3.5.16 Command Operations . . . 122
3.5.17 Session Management . . . 122
3.6 FASP transfer . . . 122
3.6.1 How It Works . . . 125
3.6.2 Configuration Instructions . . . 125
3.6.3 Example Configuration . . . 125
3.6.4 Saved Parameters Description . . . 125
3.6.5 Inputs Description . . . 125
3.6.6 Outputs Description . . . 126
3.6.7 Job Order Description (Parameters) . . . 126
3.6.8 Example Job Order . . . 126
3.6.9 Operating Instructions . . . 128
3.6.10 Dependencies . . . 128
3.6.11 Troubleshooting . . . 128
3.7 Faspex5 file processing . . . 129
3.7.1 How It Works . . . 129

3.7.2 Configuration Instructions . . . 129
3.7.3 Example Configuration . . . 129
3.7.4 Saved Parameters Description . . . 130
3.7.5 Inputs Description . . . 130
3.7.6 Outputs Description . . . 131
3.7.7 Supported Operations . . . 131
3.7.8 Operating Instructions . . . 131
3.7.9 Orchestrator Instance Setup . . . 131
3.7.10 Faspex Server Configuration . . . 131
3.7.11 Dependencies . . . 131
3.7.12 Troubleshooting . . . 132
3.8 FASPstream . . . 132
3.8.1 How It Works . . . 132
3.8.2 Configuration Instructions . . . 132
3.8.3 Example Configuration . . . 133
3.8.4 Saved Parameters Description . . . 133
3.8.5 Inputs Description . . . 134
3.8.6 Outputs Description . . . 135
3.8.7 Supported Actions . . . 135
3.8.8 Dependencies . . . 135
3.8.9 Operating Instructions . . . 135
3.9 FTP transfer . . . 136
3.9.1 How It Works . . . 137
3.9.2 Configuration Instructions . . . 138
3.9.3 Example Configuration . . . 138
3.9.4 Saved Parameters Description . . . 138
3.9.5 Inputs Description . . . 139
3.9.6 Outputs Description . . . 139
3.9.7 Operating Instructions . . . 139
3.9.8 Prerequisites . . . 139
3.9.9 Basic Workflow . . . 140
3.9.10 Dependencies . . . 140
3.10 Ftps transfer . . . 140
3.10.1 How It Works . . . 140
3.10.2 Configuration Instructions . . . 141
3.10.3 Example Configuration . . . 141
3.10.4 Saved Parameters Description . . . 141
3.10.5 Inputs Description . . . 142
3.10.6 Outputs Description . . . 142
3.10.7 Supported Actions . . . 142
3.10.8 Dependencies . . . 142
3.10.9 Operating Instructions . . . 142
3.11 FXP Transfer . . . 144
3.11.1 How It Works . . . 144
3.11.2 Configuration Instructions . . . 145
3.11.3 Example Configuration . . . 145
3.11.4 Saved Parameters Description . . . 145
3.11.5 Inputs Description . . . 146
3.11.6 Outputs Description . . . 147
3.11.7 Supported Actions . . . 147
3.11.8 Dependencies . . . 147
3.11.9 Operating Instructions . . . 147
3.11.10 Warnings and Best Practices . . . 147
3.12 HTTP Transfer . . . 147
3.12.1 How It Works . . . 148
3.12.2 Configuration Instructions . . . 148
3.12.3 Example Configuration . . . 148
3.12.4 Saved Parameters Description . . . 148
3.12.5 Inputs Description . . . 149
3.12.6 Outputs Description . . . 149

3.12.7 Supported Actions . . . 150
3.12.8 Dependencies . . . 150
3.12.9 Operating Instructions . . . 150
3.12.10 Server Configuration Requirements . . . 151
3.13 iTunes transporter . . . 152
3.13.1 How It Works . . . 153
3.13.2 Configuration Instructions . . . 153
3.13.3 Example Configuration . . . 153
3.13.4 Saved Parameters Description . . . 154
3.13.5 Inputs Description . . . 155
3.13.6 Outputs Description . . . 155
3.13.7 Operating Instructions . . . 155
3.13.8 Dependencies . . . 155
3.13.9 Troubleshooting . . . 156
3.14 Robocopy . . . 156
3.14.1 How It Works . . . 156
3.14.2 Configuration Instructions . . . 156
3.14.3 Example Configuration . . . 157
3.14.4 Saved Parameters Description . . . 157
3.14.5 Inputs Description . . . 158
3.14.6 Outputs Description . . . 158
3.14.7 Supported Actions . . . 158
3.14.8 Dependencies . . . 159
3.14.9 Operating Instructions . . . 159
3.15 SCP transfer . . . 160
3.15.1 How It Works . . . 161
3.15.2 Configuration Instructions . . . 161
3.15.3 Example Configuration . . . 161
3.15.4 Saved Parameters Description . . . 161
3.15.5 Inputs Description . . . 162
3.15.6 Outputs Description . . . 162
3.15.7 Supported Actions . . . 162
3.15.8 Dependencies . . . 162
3.15.9 Operating Instructions . . . 162
3.15.10 Troubleshooting . . . 162
3.16 SFTP transfer . . . 162
3.16.1 How It Works . . . 164
3.16.2 Configuration Instructions . . . 164
3.16.3 Example Configuration . . . 164
3.16.4 Saved Parameters Description . . . 164
3.16.5 Inputs Description . . . 165
3.16.6 Outputs Description . . . 165
3.16.7 Supported Operations . . . 165
3.16.8 Operating Instructions . . . 165
3.16.9 Dependencies . . . 166
3.16.10 Troubleshooting . . . 166
3.17 Shares transfer setup . . . 166
3.17.1 How It Works . . . 167
3.17.2 Configuration Instructions . . . 167
3.17.3 Example Configuration . . . 167
3.17.4 Saved Parameters Description . . . 167
3.17.5 Inputs Description . . . 168
3.17.6 Outputs Description . . . 168
3.17.7 Operating Instructions . . . 168
3.17.8 Dependencies . . . 168
3.17.9 Troubleshooting . . . 169
3.18 WebDAV Operation . . . 169
3.18.1 How It Works . . . 169
3.18.2 Configuration Instructions . . . 170
3.18.3 Example Configuration . . . 170

3.18.4 Saved Parameters Description . . . 170
3.18.5 Inputs Description . . . 171
3.18.6 Outputs Description . . . 171
3.18.7 Supported Actions . . . 171
3.18.8 Dependencies . . . 171
3.18.9 Operating Instructions . . . 171

4 File Transformations 173
4.1 ADI Transformation . . . 173
4.1.1 Saved Parameters Description . . . 173
4.1.2 Inputs description . . . 174
4.1.3 Outputs description . . . 174
4.1.4 Operating Instructions . . . 174
4.2 Imagemagick . . . 174
4.2.1 How It Works . . . 175
4.2.2 Configuration Instructions . . . 175
4.2.3 Example Configuration . . . 175
4.2.4 Saved Parameters Description . . . 176
4.2.5 Inputs Description . . . 176
4.2.6 Outputs Description . . . 176
4.2.7 Supported Actions . . . 177
4.2.8 Dependencies . . . 177
4.2.9 Operating Instructions . . . 177
4.3 MacCaption Operation . . . 179
4.3.1 Saved Parameters Description . . . 179
4.3.2 Inputs description . . . 180
4.3.3 Outputs description . . . 180
4.3.4 Supported Actions . . . 180
4.3.5 Dependencies . . . 180
4.3.6 Operating Instructions . . . 180
4.4 MXF Legalizer . . . 180
4.4.1 Saved Parameters Description . . . 181
4.4.2 Inputs description . . . 181
4.4.3 Outputs description . . . 181
4.4.4 Supported Actions . . . 181
4.4.5 Dependencies . . . 181
4.4.6 Operating Instructions . . . 181
4.5 Subler Operation . . . 181
4.5.1 Saved Parameters Description . . . 182
4.5.2 Inputs description . . . 182
4.5.3 Outputs description . . . 183
4.5.4 Supported Actions . . . 183
4.5.5 Dependencies . . . 183
4.5.6 Operating Instructions . . . 183
4.6 XSLT Transformation . . . 184
4.6.1 How It Works . . . 185
4.6.2 Configuration Instructions . . . 185
4.6.3 Example Configuration . . . 185
4.6.4 Saved Parameters Description . . . 185
4.6.5 Inputs Description . . . 185
4.6.6 Outputs Description . . . 186
4.6.7 Supported Actions . . . 186
4.6.8 Dependencies . . . 186
4.6.9 Operating Instructions . . . 186

5 Integration 187
5.1 Amazon DynamoDB Operation . . . 187
5.1.1 How It Works . . . 187
5.1.2 Configuration Instructions . . . 188
5.1.3 Example Configuration . . . 188
5.1.4 Saved Parameters Description . . . 188

5.1.5 Inputs Description . . . 189
5.1.6 Outputs Description . . . 189
5.1.7 Supported Actions . . . 189
5.1.8 Dependencies . . . 189
5.1.9 Operating Instructions . . . 189
5.2 Amazon Glacier Operation . . . 191
5.2.1 How It Works . . . 191
5.2.2 Configuration Instructions . . . 191
5.2.3 Example Configuration . . . 192
5.2.4 Saved Parameters Description . . . 192
5.2.5 Inputs Description . . . 193
5.2.6 Outputs Description . . . 193
5.2.7 Supported Actions . . . 193
5.2.8 Dependencies . . . 193
5.2.9 Operating Instructions . . . 194
5.3 Amazon Media Convert Operation . . . 195
5.3.1 How It Works . . . 195
5.3.2 Configuration Instructions . . . 195
5.3.3 Example Configuration . . . 196
5.3.4 Saved Parameters Description . . . 196
5.3.5 Inputs Description . . . 196
5.3.6 Outputs Description . . . 197
5.3.7 Supported Actions . . . 197
5.3.8 Dependencies . . . 197
5.3.9 Operating Instructions . . . 197
5.4 Amazon S3 Operation . . . 198
5.4.1 How It Works . . . 199
5.4.2 Configuration Instructions . . . 200
5.4.3 Example Configuration . . . 200
5.4.4 Saved Parameters Description . . . 200
5.4.5 Inputs Description . . . 201
5.4.6 Outputs Description . . . 202
5.4.7 Supported Actions . . . 202
5.4.8 Dependencies . . . 202
5.4.9 Operating Instructions . . . 202
5.4.10 Assume Role Configuration . . . 202
5.5 Amazon SimpleDB Operation . . . 203
5.5.1 Saved Parameters Description . . . 203
5.5.2 Inputs description . . . 204
5.5.3 Outputs description . . . 204
5.5.4 Supported Actions . . . 204
5.5.5 Dependencies . . . 204
5.5.6 Operating Instructions . . . 204
5.6 Amazon SNS Operation . . . 205
5.6.1 How It Works . . . 205
5.6.2 Configuration Instructions . . . 206
5.6.3 Example Configuration . . . 206
5.6.4 Saved Parameters Description . . . 206
5.6.5 Inputs Description . . . 206
5.6.6 Outputs Description . . . 207
5.6.7 Supported Actions . . . 207
5.6.8 Dependencies . . . 207
5.6.9 Operating Instructions . . . 207
5.7 Amazon SQS Operation . . . 209
5.7.1 How It Works . . . 209
5.7.2 Configuration Instructions . . . 209
5.7.3 Example Configuration . . . 210
5.7.4 Saved Parameters Description . . . 210
5.7.5 Inputs Description . . . 210
5.7.6 Outputs Description . . . 211

5.7.7 Supported Actions . . . 211
5.7.8 Dependencies . . . 211
5.7.9 Operating Instructions . . . 211
5.8 Amazon SWF Operation . . . 212
5.8.1 Saved Parameters Description . . . 212
5.8.2 Inputs description . . . 213
5.8.3 Outputs description . . . 213
5.8.4 Supported Actions . . . 213
5.8.5 Dependencies . . . 213
5.8.6 Operating Instructions . . . 213
5.9 Amazon Transcribe Operation . . . 214
5.9.1 Saved Parameters Description . . . 214
5.9.2 Inputs description . . . 215
5.9.3 Outputs description . . . 215
5.9.4 Supported Actions . . . 215
5.9.5 Operating Instructions . . . 215
5.10 AMQP message . . . 216
5.10.1 How It Works . . . 216
5.10.2 Configuration Instructions . . . 217
5.10.3 Example Configuration . . . 218
5.10.4 Saved Parameters Description . . . 218
5.10.5 Inputs Description . . . 219
5.10.6 Outputs Description . . . 220
5.10.7 Supported Actions . . . 220
5.10.8 Dependencies . . . 220
5.10.9 Operating Instructions . . . 220
5.11 Aspera On Cloud Management . . . 220
5.11.1 How It Works . . . 221
5.11.2 Configuration Instructions . . . 221
5.11.3 Example Configuration . . . 221
5.11.4 Saved Parameters Description . . . 222
5.11.5 Inputs Description . . . 222
5.11.6 Outputs Description . . . 222
5.11.7 Supported Actions . . . 223
5.11.8 Dependencies . . . 223
5.11.9 Operating Instructions . . . 223
5.11.10 OAuth Setup Requirements . . . 223
5.11.11 Shared Inbox Operations . . . 223
5.11.12 Member List Configuration . . . 223
5.11.13 Metadata Configuration . . . 223
5.11.14 User Management Operations . . . 223
5.12 Aspera Shares Management . . . 224
5.12.1 How It Works . . . 224
5.12.2 Configuration Instructions . . . 224
5.12.3 Example Configuration . . . 225
5.12.4 Saved Parameters Description . . . 225
5.12.5 Inputs Description . . . 225
5.12.6 Outputs Description . . . 225
5.12.7 Supported Operations . . . 225
5.12.8 Operating Instructions . . . 226
5.12.9 Dependencies . . . 226
5.12.10 Troubleshooting . . . 226
5.12.11 Version Upgrade Notes . . . 226
5.13 Async Remote Execution . . . 226
5.13.1 How It Works . . . 227
5.13.2 Configuration Instructions . . . 227
5.13.3 Example Configuration . . . 227
5.13.4 Saved Parameters Description . . . 228
5.13.5 Inputs Description . . . 228
5.13.6 Outputs Description . . . 228

5.13.7 Supported Actions . . . 229
5.13.8 Dependencies . . . 229
5.13.9 Operating Instructions . . . 229
5.13.10 Operating Modes . . . 229
5.13.11 Environment Variables . . . 229
5.13.12 Code Blocks . . . 229
5.13.13 Available Variables in Code Blocks . . . 229
5.13.14 SSH Authentication . . . 229
5.13.15 SSH Key Passphrase . . . 229
5.13.16 Connection Options . . . 230
5.13.17 Best Practices . . . 230
5.14 Azure Service Bus Operation . . . 230
5.14.1 How It Works . . . 230
5.14.2 Configuration Instructions . . . 230
5.14.3 Example Configuration . . . 230
5.14.4 Saved Parameters Description . . . 231
5.14.5 Inputs Description . . . 231
5.14.6 Outputs Description . . . 231
5.14.7 Supported Actions . . . 231
5.14.8 Dependencies . . . 232
5.14.9 Operating Instructions . . . 232
5.15 CatdvRestRequest . . . 232
5.15.1 How It Works . . . 232
5.15.2 Configuration Instructions . . . 232
5.15.3 Example Configuration . . . 233
5.15.4 Saved Parameters Description . . . 233
5.15.5 Inputs Description . . . 233
5.15.6 Endpoint Operation Support . . . 234
5.15.7 Outputs Description . . . 234
5.15.8 Operating Instructions . . . 234
5.15.9 Dependencies . . . 234
5.15.10 Troubleshooting . . . 234
5.16 Corba Operation . . . 235
5.16.1 How It Works . . . 235
5.16.2 Configuration Instructions . . . 236
5.16.3 Example Configuration . . . 236
5.16.4 Saved Parameters Description . . . 236
5.16.5 Inputs Description . . . 236
5.16.6 Outputs Description . . . 236
5.16.7 Operating Instructions . . . 237
5.16.8 Dependencies . . . 237
5.16.9 Troubleshooting . . . 237
5.17 Curl Operations . . . 237
5.17.1 How It Works . . . 238
5.17.2 Configuration Instructions . . . 238
5.17.3 Example Configuration . . . 238
5.17.4 Saved Parameters Description . . . 238
5.17.5 Operation Types . . . 239
5.17.6 Inputs Description . . . 239
5.17.7 Outputs Description . . . 239
5.17.8 Supported Actions . . . 239
5.17.9 Dependencies . . . 239
5.17.10 Operating Instructions . . . 239
5.17.11 Parameter Format Options . . . 240
5.17.12 Example 1: Direct Command Execution . . . 240
5.17.13 Example 2: Pre-configured with Parameters . . . 240
5.17.14 Example 3: Image Download . . . 240
5.17.15 Troubleshooting . . . 240
5.18 Custom progress notifier . . . 240
5.18.1 How It Works . . . 241

5.18.2 Configuration Instructions . . . 241
5.18.3 Example Configuration . . . 241
5.18.4 Saved Parameters Description . . . 241
5.18.5 Inputs Description . . . 242
5.18.6 Outputs Description . . . 242
5.18.7 Available Progress Variables . . . 242
5.18.8 Operating Instructions . . . 243
5.18.9 Example Output Log . . . 243
5.18.10 Dependencies . . . 243
5.18.11 Troubleshooting . . . 243
5.19 Facebook Video Upload Operation . . . 243
5.19.1 How It Works . . . 244
5.19.2 Configuration Instructions . . . 244
5.19.3 Example Configuration . . . 244
5.19.4 Saved Parameters Description . . . 244
5.19.5 Inputs Description . . . 245
5.19.6 Outputs Description . . . 245
5.19.7 Supported Actions . . . 245
5.19.8 Dependencies . . . 245
5.19.9 Operating Instructions . . . 245
5.20 Frameio Operation . . . 246
5.20.1 How It Works . . . 247
5.20.2 Configuration Instructions . . . 247
5.20.3 Example Configuration . . . 248
5.20.4 Saved Parameters Description . . . 248
5.20.5 Inputs Description . . . 248
5.20.6 Outputs Description . . . 248
5.20.7 Supported Actions . . . 248
5.20.8 Dependencies . . . 248
5.20.9 Operating Instructions . . . 248
5.21 Google Cloud Storage Operation . . . 249
5.21.1 Saved Parameters Description . . . 249
5.21.2 Inputs description . . . 249
5.21.3 Outputs description . . . 250
5.21.4 Supported Actions . . . 250
5.21.5 Dependencies . . . 250
5.21.6 Operating Instructions . . . 250
5.21.7 Before you can use the plugin, you need to perform these operations: . . . 250
5.22 Google Pubsub Operation . . . 250
5.22.1 Saved Parameters Description . . . 251
5.22.2 Inputs description . . . 251
5.22.3 Outputs description . . . 251
5.22.4 Supported Actions . . . 251
5.22.5 Dependencies . . . 252
5.22.6 Operating Instructions . . . 252
5.22.7 Before you can use the plugin, you need to perform these operations: . . . 252
5.23 Ibm mq integration . . . 252
5.23.1 How It Works . . . 253
5.23.2 Configuration Instructions . . . 253
5.23.3 Example Configuration . . . 253
5.23.4 Saved Parameters Description . . . 254
5.23.5 Inputs Description . . . 254
5.23.6 Outputs Description . . . 255
5.23.7 Supported Actions . . . 255
5.23.8 Dependencies . . . 255
5.23.9 Operating Instructions . . . 255
5.24 Ibm mq mft transfer . . . 257
5.24.1 How It Works . . . 257
5.24.2 Configuration Instructions . . . 258
5.24.3 Example Configuration . . . 258

5.24.4 Saved Parameters Description . . . 259
5.24.5 Inputs Description . . . 259
5.24.6 Outputs Description . . . 260
5.24.7 Supported Actions . . . 260
5.24.8 Dependencies . . . 260
5.24.9 Operating Instructions . . . 260
5.25 InfluxDB . . . 262
5.25.1 How It Works . . . 263
5.25.2 Configuration Instructions . . . 263
5.25.3 Example Configuration - Query Operation . . . 263
5.25.4 Example Configuration - Write Operation . . . 263
5.25.5 Saved Parameters Description . . . 264
5.25.6 Inputs Description . . . 264
5.25.7 Outputs Description . . . 264
5.25.8 Supported Actions . . . 265
5.25.9 Dependencies . . . 265
5.25.10 Operating Instructions . . . 265
5.25.11 Overview . . . 265
5.25.12 Database Connection Setup . . . 265
5.25.13 Query Operations . . . 265
5.25.14 Query Syntax . . . 265
5.25.15 Chunking and Precision . . . 265
5.25.16 Write Operations . . . 265
5.25.17 Data Format . . . 265
5.25.18 Write Examples . . . 266
5.25.19 Write Options . . . 266
5.25.20 Response Codes . . . 266
5.25.21 Output Processing . . . 266
5.25.22 Best Practices . . . 266
5.26 JIRA Ticket Creation . . . 267
5.26.1 Saved Parameters Description . . . 267
5.26.2 Inputs description . . . 267
5.26.3 Outputs description . . . 267
5.26.4 Supported Actions . . . 267
5.26.5 Dependencies . . . 268
5.26.6 Operating Instructions . . . 268
5.27 Kafka stream . . . 268
5.27.1 How It Works . . . 268
5.27.2 Configuration Instructions . . . 269
5.27.3 Example Configuration . . . 269
5.27.4 Saved Parameters Description . . . 269
5.27.5 Inputs Description . . . 270
5.27.6 Outputs Description . . . 270
5.27.7 Supported Actions . . . 270
5.27.8 Dependencies . . . 271
5.27.9 Operating Instructions . . . 271
5.28 Local execution . . . 273
5.28.1 How It Works . . . 273
5.28.2 Configuration Instructions . . . 274
5.28.3 Example Configuration . . . 274
5.28.4 Saved Parameters Description . . . 274
5.28.5 Inputs Description . . . 274
5.28.6 Outputs Description . . . 275
5.28.7 Supported Actions . . . 275
5.28.8 Dependencies . . . 275
5.28.9 Operating Instructions . . . 275
5.29 Media Mate Operation . . . 275
5.29.1 How It Works . . . 276
5.29.2 Configuration Instructions . . . 276
5.29.3 Example Configuration . . . 276

5.29.4 Saved Parameters Description . . . 277
5.29.5 Inputs Description . . . 277
5.29.6 Outputs Description . . . 277
5.29.7 Supported Actions . . . 278
5.29.8 Dependencies . . . 278
5.29.9 Operating Instructions . . . 278
5.30 Mongodb operation . . . 279
5.30.1 How It Works . . . 280
5.30.2 Configuration Instructions . . . 280
5.30.3 Example Configuration . . . 280
5.30.4 Saved Parameters Description . . . 281
5.30.5 Inputs Description . . . 281
5.30.6 Operation-Specific Inputs: . . . 281
5.30.7 Database_command . . . 281
5.30.8 Document_aggregate . . . 281
5.30.9 Document_find . . . 281
5.30.10 Document_find_one . . . 281
5.30.11 Document_find_and_modify . . . 281
5.30.12 Document_group . . . 282
5.30.13 Document_insert . . . 282
5.30.14 Document_map_reduce . . . 282
5.30.15 Document_remove . . . 282
5.30.16 Document_update . . . 282
5.30.17 Outputs Description . . . 282
5.30.18 Supported Operations . . . 282
5.30.19 Operating Instructions . . . 283
5.30.20 Connection URI Format . . . 283
5.30.21 Example Use Cases . . . 283
5.30.22 Case 1: Map-Reduce Analysis . . . 283
5.30.23 Case 2: Document Operations . . . 284
5.30.24 Case 3: Advanced Operations . . . 284
5.30.25 Dependencies . . . 284
5.30.26 Troubleshooting . . . 284
5.31 Remote execution . . . 284
5.31.1 How It Works . . . 286
5.31.2 Configuration Instructions . . . 286
5.31.3 Example Configuration . . . 286
5.31.4 Expected Test Results . . . 286
5.31.5 Saved Parameters Description . . . 286
5.31.6 Inputs Description . . . 287
5.31.7 Outputs Description . . . 287
5.31.8 Supported Actions . . . 287
5.31.9 Dependencies . . . 287
5.31.10 Operating Instructions . . . 287
5.31.11 Standard I/O Configuration . . . 287
5.31.12 Status Updates . . . 288
5.31.13 Output Variable Extraction . . . 288
5.31.14 Example Shell Script . . . 288
5.31.15 Environment Variables . . . 288
5.32 REST request . . . 288
5.32.1 How It Works . . . 290
5.32.2 Configuration Instructions . . . 290
5.32.3 Example Configuration . . . 290
5.32.4 Saved Parameters Description . . . 290
5.32.5 Inputs Description . . . 292
5.32.6 Mandatory Inputs: . . . 292
5.32.7 Optional Inputs: . . . 292
5.32.8 Outputs Description . . . 292
5.32.9 Supported Actions . . . 292
5.32.10 Dependencies . . . 292

5.32.11 Operating Instructions . . . 292
5.32.12 URL Parameters and Authentication . . . 293
5.32.13 Content Types and Response Handling . . . 293
5.32.14 Processed Outputs . . . 293
5.32.15 Test Results Example . . . 293
5.32.16 Polling Functionality . . . 293
5.32.17 Trigger Functionality . . . 293
5.32.18 Example XML Response Processing . . . 293
5.33 Soap request . . . 294
5.33.1 How It Works . . . 295
5.33.2 Configuration Instructions . . . 295
5.33.3 Example Configuration . . . 296
5.33.4 Saved Parameters Description . . . 296
5.33.5 Inputs Description . . . 297
5.33.6 Outputs Description . . . 298
5.33.7 Supported Actions . . . 298
5.33.8 Dependencies . . . 298
5.33.9 Operating Instructions . . . 298
5.33.10 Overview . . . 298
5.33.11 Service Configuration Methods . . . 298
5.33.12 Payload Configuration . . . 298
5.33.13 Explicit Payload Mode (Recommended) . . . 298
5.33.14 Parameter Mode (Alternative) . . . 299
5.33.15 Authentication . . . 299
5.33.16 XML Parsing Modes . . . 299
5.33.17 Modern Mode (Nokogiri - Default since v1.7.2) . . . 299
5.33.18 Legacy Mode (XmlSimple) . . . 299
5.33.19 Polling and Stop Conditions . . . 299
5.33.20 Trigger Mode and Record Processing . . . 299
5.33.21 Response Processing Examples . . . 300
5.33.22 Example SOAP Response . . . 300
5.33.23 Result Records Extracting Code . . . 300
5.33.24 Record Formatting Code . . . 300
5.33.25 Record Identification Code . . . 300
5.33.26 Simple Formatting Alternative . . . 300
5.33.27 Results Formatting Code . . . 300
5.33.28 Debugging Tips . . . 301
5.33.29 Session Management . . . 301
5.33.30 Best Practices . . . 301
5.34 Softlayer API . . . 301
5.34.1 How It Works . . . 302
5.34.2 Configuration Instructions . . . 302
5.34.3 Example Configuration . . . 302
5.34.4 Saved Parameters Description . . . 303
5.34.5 Inputs Description . . . 303
5.34.6 Outputs Description . . . 304
5.34.7 Supported Actions . . . 304
5.34.8 Dependencies . . . 304
5.34.9 Operating Instructions . . . 304
5.35 3PlayMedia Operation . . . 304
5.35.1 Saved Parameters Description . . . 305
5.35.2 Inputs description . . . 305
5.35.3 Outputs description . . . 305
5.35.4 Supported Actions . . . 305
5.35.5 Dependencies . . . 305
5.35.6 Operating Instructions . . . 306
5.36 Watson Language Translator . . . 306
5.36.1 Saved Parameters Description . . . 306
5.36.2 Inputs description . . . 307
5.36.3 Outputs description . . . 307

5.36.4 Supported Actions . . . 307
5.36.5 Dependencies . . . 307
5.36.6 Operating Instructions . . . 307

6 Other Utilities 308
6.1 Collection manager . . . 308
6.1.1 How It Works . . . 308
6.1.2 Configuration Instructions . . . 308
6.1.3 Example Configuration . . . 309
6.1.4 Saved Parameters Description . . . 309
6.1.5 Inputs Description . . . 309
6.1.6 Outputs Description . . . 309
6.1.7 Supported Actions . . . 309
6.1.8 Dependencies . . . 310
6.1.9 Operating Instructions . . . 310
6.1.10 Troubleshooting . . . 310
6.2 Custom Python . . . 310
6.2.1 How It Works . . . 310
6.2.2 Configuration Instructions . . . 310
6.2.3 Example Configuration . . . 311
6.2.4 Example 1: Sample Python code to return a file size. . . . 311
6.2.5 Example 2: Sample code to parse XML data. . . . 311
6.2.6 Saved Parameters Description . . . 312
6.2.7 Source Control Configuration . . . 312
6.2.8 GitHub . . . 312
6.2.9 Bitbucket . . . 312
6.2.10 Inputs Description . . . 313
6.2.11 Outputs Description . . . 313
6.2.12 Supported Actions . . . 313
6.2.13 Operating Instructions . . . 313
6.2.14 Dependencies . . . 313
6.2.15 Installing Flask and Dependencies . . . 313
6.3 Hello world . . . 313
6.3.1 How It Works . . . 314
6.3.2 Configuration Instructions . . . 314
6.3.3 Example Configuration . . . 314
6.3.4 Saved Parameters Description . . . 314
6.3.5 Inputs Description . . . 315
6.3.6 Outputs Description . . . 315
6.3.7 Supported Actions . . . 315
6.3.8 Dependencies . . . 315
6.3.9 Operating Instructions . . . 315
6.4 Input Manipulations . . . 316
6.4.1 How It Works . . . 317
6.4.2 Configuration Instructions . . . 317
6.4.3 Example Configuration . . . 317
6.4.4 Saved Parameters Description . . . 317
6.4.5 Inputs Description . . . 318
6.4.6 Outputs Description . . . 318
6.4.7 Supported Actions . . . 318
6.4.8 Dependencies . . . 318
6.4.9 Operating Instructions . . . 319
6.5 Stored parameter set . . . 319
6.5.1 How It Works . . . 320
6.5.2 Configuration Instructions . . . 320
6.5.3 Example Configuration . . . 321
6.5.4 Saved Parameters Description . . . 321
6.5.5 Inputs Description . . . 321
6.5.6 Outputs Description . . . 321
6.5.7 Supported Actions . . . 321

6.5.8 Dependencies . . . 322
6.5.9 Troubleshooting . . . 322
6.6 Password generator . . . 322
6.6.1 How It Works . . . 322
6.6.2 Configuration Instructions . . . 322
6.6.3 Example Configuration . . . 323
6.6.4 Saved Parameters Description . . . 323
6.6.5 Inputs Description . . . 323
6.6.6 Outputs Description . . . 323
6.6.7 Supported Actions . . . 323
6.6.8 Dependencies . . . 324
6.6.9 Operating Instructions . . . 324
6.6.10 Overview . . . 324
6.6.11 Character Set Configuration . . . 324
6.6.12 Password Length . . . 324
6.6.13 Constraint Configuration . . . 324
6.6.14 Encryption and Storage . . . 324
6.6.15 Password Generation Process . . . 324
6.6.16 Best Practices . . . 324
6.7 Regex matcher . . . 325
6.7.1 How It Works . . . 325
6.7.2 Configuration Instructions . . . 325
6.7.3 Example Configuration . . . 325
6.7.4 Saved Parameters Description . . . 326
6.7.5 Inputs Description . . . 326
6.7.6 Outputs Description . . . 326
6.7.7 Supported Actions . . . 326
6.7.8 Dependencies . . . 326
6.7.9 Operating Instructions . . . 326
6.8 Shared state operation . . . 327
6.8.1 How It Works . . . 327
6.8.2 Configuration Instructions . . . 327
6.8.3 Example Configuration . . . 328
6.8.4 Saved Parameters Description . . . 328
6.8.5 Inputs Description . . . 328
6.8.6 Outputs Description . . . 328
6.8.7 Supported Actions . . . 328
6.8.8 Dependencies . . . 329
6.8.9 Operating Instructions . . . 329
6.9 Look-up table . . . 329
6.9.1 How It Works . . . 330
6.9.2 Configuration Instructions . . . 330
6.9.3 Example Configuration . . . 330
6.9.4 Saved Parameters Description . . . 331
6.9.5 Dynamic Mapping Definition . . . 331
6.9.6 Inputs Description . . . 331
6.9.7 Outputs Description . . . 331
6.9.8 Operating Instructions . . . 331
6.9.9 Example of a Simple Mapping . . . 331
6.9.10 Example of Multi-values Mapping . . . 332
6.9.11 Example of Multi-inputs Mapping . . . 332
6.9.12 How to Define the Mapping Using a File . . . 332
6.9.13 How to Define the Mapping at Run Time . . . 332
6.9.14 How to Retrieve All Possible Values for a Column . . . 332
6.9.15 Example of Use of the KeyForTableIndexing . . . 333
6.9.16 Use of RegEx and GLOB Matching . . . 333
6.9.17 Dependencies . . . 333
6.10 Timecode manager . . . 333
6.10.1 How It Works . . . 334
6.10.2 Configuration Instructions . . . 334

6.10.3 Example Configuration . . . 334
6.10.4 Saved Parameters Description . . . 334
6.10.5 Inputs Description . . . 335
6.10.6 Outputs Description . . . 335
6.10.7 Supported Actions . . . 335
6.10.8 Dependencies . . . 335
6.10.9 Operating Instructions . . . 335
6.11 UUID Generator . . . 336
6.11.1 How It Works . . . 336
6.11.2 Configuration Instructions . . . 336
6.11.3 Example Configuration . . . 336
6.11.4 Saved Parameters Description . . . 337
6.11.5 Inputs Description . . . 337
6.11.6 Outputs Description . . . 337
6.11.7 Supported Actions . . . 337
6.11.8 Dependencies . . . 337
6.11.9 Operating Instructions . . . 337
6.12 Watch file movement . . . 337
6.12.1 How It Works . . . 338
6.12.2 Configuration Instructions . . . 338
6.12.3 Example Configuration . . . 338
6.12.4 Saved Parameters Description . . . 338
6.12.5 Inputs Description . . . 338
6.12.6 Outputs Description . . . 339
6.12.7 Supported Actions . . . 339
6.12.8 Dependencies . . . 339
6.12.9 Operating Instructions . . . 339

7 Quality Control **341**
7.1 ADI 3 Parser and Validator . . . 341
7.1.1 How It Works . . . 341
7.1.2 Configuration Instructions . . . 341
7.1.3 Example Configuration . . . 342
7.1.4 Saved Parameters Description . . . 342
7.1.5 Inputs Description . . . 343
7.1.6 Outputs Description . . . 343
7.1.7 Supported Actions . . . 344
7.1.8 Dependencies . . . 344
7.1.9 Operating Instructions . . . 344
7.2 ADI Parser and Validator . . . 344
7.2.1 How It Works . . . 345
7.2.2 Configuration Instructions . . . 345
7.2.3 Example Configuration . . . 346
7.2.4 Saved Parameters Description . . . 346
7.2.5 Inputs Description . . . 346
7.2.6 Outputs Description . . . 347
7.2.7 Supported Actions . . . 347
7.2.8 Dependencies . . . 347
7.2.9 Operating Instructions . . . 347
7.3 Aurora file verification . . . 348
7.3.1 Saved Parameters Description . . . 349
7.3.2 Inputs description . . . 349
7.3.3 Outputs description . . . 349
7.3.4 Supported Actions . . . 350
7.3.5 Dependencies . . . 350
7.3.6 Operating Instructions . . . 350
7.4 Baton file correction . . . 350
7.4.1 How It Works . . . 351
7.4.2 Configuration Instructions . . . 351
7.4.3 Example Configuration . . . 351

7.4.4 Saved Parameters Description . . . 352
7.4.5 Inputs Description . . . 352
7.4.6 Outputs Description . . . 353
7.4.7 Supported Actions . . . 353
7.4.8 Dependencies . . . 353
7.4.9 Operating Instructions . . . 353
7.5 Baton file verification . . . 355
7.5.1 How It Works . . . 356
7.5.2 Configuration Instructions . . . 356
7.5.3 Example Configuration . . . 357
7.5.4 Saved Parameters Description . . . 357
7.5.5 Inputs Description . . . 358
7.5.6 Outputs Description . . . 358
7.5.7 Supported Actions . . . 359
7.5.8 Dependencies . . . 359
7.5.9 Operating Instructions . . . 359
7.6 Baton xml report . . . 359
7.6.1 How It Works . . . 359
7.6.2 Configuration Instructions . . . 360
7.6.3 Example Configuration . . . 360
7.6.4 Saved Parameters Description . . . 360
7.6.5 Inputs Description . . . 360
7.6.6 Outputs Description . . . 361
7.6.7 Supported Actions . . . 361
7.6.8 Dependencies . . . 361
7.6.9 Operating Instructions . . . 361
7.7 Cerify File Verification . . . 362
7.7.1 Saved Parameters Description . . . 363
7.7.2 Inputs description . . . 363
7.7.3 Outputs description . . . 363
7.7.4 Operating Instructions . . . 363
7.7.5 An example of a QC workflow below . . . 364
7.8 Irt mxf analyser . . . 364
7.8.1 Saved Parameters Description . . . 364
7.8.2 Inputs description . . . 365
7.8.3 Outputs description . . . 365
7.8.4 Supported Actions . . . 365
7.8.5 Dependencies . . . 365
7.8.6 Operating Instructions . . . 365
7.9 Mx fixer . . . 365
7.9.1 Saved Parameters Description . . . 366
7.9.2 Inputs description . . . 366
7.9.3 Outputs description . . . 366
7.9.4 Supported Actions . . . 366
7.9.5 Dependencies . . . 366
7.9.6 Operating Instructions . . . 366
7.10 Orion OTT Monitor . . . 366
7.10.1 Saved Parameters Description . . . 367
7.10.2 Inputs description . . . 367
7.10.3 Outputs description . . . 367
7.10.4 Operating Instructions . . . 368
7.11 Pulsar content verification . . . 368
7.11.1 Saved Parameters Description . . . 368
7.11.2 Inputs description . . . 368
7.11.3 Outputs description . . . 369
7.11.4 Operating Instructions . . . 369
7.11.5 An example of a QC workflow below . . . 369
7.12 QScan file verification . . . 369
7.12.1 Saved Parameters Description . . . 370
7.12.2 Inputs description . . . 370

7.12.3 Outputs description . . . 370
7.12.4 Supported Actions . . . 370
7.12.5 Dependencies . . . 370
7.12.6 Operating Instructions . . . 370
7.13 Quasar Venera QC . . . 370
7.13.1 Saved Parameters Description . . . 371
7.13.2 Inputs description . . . 371
7.13.3 Outputs description . . . 371
7.13.4 Operating Instructions . . . 371
7.14 Vidchecker verification . . . 371
7.14.1 How It Works . . . 372
7.14.2 Configuration Instructions . . . 372
7.14.3 Example Configuration . . . 372
7.14.4 Saved Parameters Description . . . 373
7.14.5 Inputs Description . . . 373
7.14.6 Outputs Description . . . 373
7.14.7 Supported Actions . . . 373
7.14.8 Dependencies . . . 374
7.14.9 Operating Instructions . . . 374
7.15 VqbifAnalyzer . . . 374
7.15.1 Saved Parameters Description . . . 375
7.15.2 Inputs description . . . 375
7.15.3 Outputs description . . . 375
7.15.4 Supported Actions . . . 375
7.15.5 Dependencies . . . 375
7.15.6 Operating Instructions . . . 375
7.16 Watson speech to text . . . 376
7.16.1 How It Works . . . 377
7.16.2 Configuration Instructions . . . 377
7.16.3 Example Configuration . . . 377
7.16.4 Saved Parameters Description . . . 377
7.16.5 Inputs Description . . . 378
7.16.6 Outputs Description . . . 378
7.16.7 Supported Actions . . . 378
7.16.8 Dependencies . . . 378
7.16.9 Operating Instructions . . . 378
7.17 Watson video enrichment . . . 380
7.17.1 Saved Parameters Description . . . 381
7.17.2 Inputs description . . . 381
7.17.3 Outputs description . . . 381
7.17.4 Supported Actions . . . 381
7.17.5 Dependencies . . . 381
7.17.6 Operating Instructions . . . 381
7.18 XSD Validation . . . 382
7.18.1 How It Works . . . 382
7.18.2 Configuration Instructions . . . 382
7.18.3 Example Configuration . . . 382
7.18.4 Saved Parameters Description . . . 383
7.18.5 Inputs Description . . . 383
7.18.6 Outputs Description . . . 383
7.18.7 Supported Actions . . . 383
7.18.8 Dependencies . . . 384
7.18.9 Operating Instructions . . . 384

8 Resources **386**
8.1 Load balancer . . . 386
8.1.1 How It Works . . . 386
8.1.2 Configuration Instructions . . . 386
8.1.3 Example Configuration . . . 387
8.1.4 Expected Test Results . . . 387

8.1.5 Saved Parameters Description . . . 387
8.1.6 Inputs Description . . . 387
8.1.7 Outputs Description . . . 387
8.1.8 Supported Actions . . . 387
8.1.9 Dependencies . . . 387
8.1.10 Operating Instructions . . . 388
8.1.11 Distribution Algorithms . . . 388
8.1.12 Resource List Formats . . . 388
8.2 Load manager . . . 388
8.2.1 How It Works . . . 389
8.2.2 Configuration Instructions . . . 389
8.2.3 Example Configuration . . . 389
8.2.4 Saved Parameters Description . . . 390
8.2.5 Inputs Description . . . 390
8.2.6 Outputs Description . . . 391
8.2.7 Supported Actions . . . 391
8.2.8 Operating Instructions . . . 391
8.2.9 Queue Integration Examples . . . 392
8.2.10 Tag-Based Allocation Example . . . 392
8.2.11 Dependencies . . . 392
8.2.12 Troubleshooting . . . 392
8.3 Queue stager . . . 393
8.3.1 How It Works . . . 394
8.3.2 Configuration Instructions . . . 394
8.3.3 Example Configuration . . . 394
8.3.4 Saved Parameters Description . . . 394
8.3.5 Inputs Description . . . 395
8.3.6 Outputs Description . . . 395
8.3.7 Supported Actions . . . 395
8.3.8 Operating Instructions . . . 396
8.3.9 Priority and Weight Guidelines . . . 396
8.3.10 Dependencies . . . 396
8.3.11 Troubleshooting . . . 396
8.4 Resource manager . . . 397
8.4.1 How It Works . . . 398
8.4.2 Configuration Instructions . . . 398
8.4.3 Example Configuration . . . 398
8.4.4 Saved Parameters Description . . . 399
8.4.5 Inputs Description . . . 399
8.4.6 Outputs Description . . . 399
8.4.7 Supported Actions . . . 399
8.4.8 Dependencies . . . 399
8.4.9 Operating Instructions . . . 400
8.4.10 Overview . . . 400
8.4.11 Understanding the Example Configuration . . . 400
8.4.12 Resource Pool Types . . . 400
8.4.13 Unicity Scope Options . . . 400
8.4.14 Timeout Configuration . . . 400
8.4.15 Queue Integration . . . 400
8.4.16 Releasing Resources . . . 401
8.4.17 Method 1: Release by Token (Recommended) . . . 401
8.4.18 Method 2: Release by Resource ID . . . 401
8.4.19 Important Notes: . . . 401
8.4.20 Best Practices . . . 401

9 System **402**
9.1 Array fan-out . . . 402
9.1.1 How It Works . . . 403
9.1.2 Configuration Instructions . . . 403
9.1.3 Example Configuration . . . 403

9.1.4 Saved Parameters Description . . . 403
9.1.5 Inputs Description . . . 404
9.1.6 Outputs Description . . . 404
9.1.7 Operating Instructions . . . 404
9.1.8 Dependencies . . . 404
9.1.9 Troubleshooting . . . 404
9.2 Federated workflow . . . 404
9.2.1 How It Works . . . 405
9.2.2 Configuration Instructions . . . 405
9.2.3 Example Configuration . . . 406
9.2.4 Saved Parameters Description . . . 406
9.2.5 Inputs Description . . . 407
9.2.6 Outputs Description . . . 408
9.2.7 Supported Actions . . . 408
9.2.8 Dependencies . . . 408
9.2.9 Operating Instructions . . . 408
9.3 Filter . . . 408
9.3.1 How It Works . . . 409
9.3.2 Configuration Instructions . . . 409
9.3.3 Saved Parameters Description . . . 409
9.3.4 Inputs Description . . . 409
9.3.5 Outputs Description . . . 409
9.3.6 Supported Actions . . . 410
9.3.7 Operating Instructions . . . 410
9.3.8 Example Configuration . . . 410
9.4 Funnel-in . . . 410
9.4.1 How It Works . . . 411
9.4.2 Configuration Instructions . . . 411
9.4.3 Example Configuration . . . 411
9.4.4 Saved Parameters Description . . . 411
9.4.5 Inputs Description . . . 412
9.4.6 Outputs Description . . . 412
9.4.7 Operating Instructions . . . 412
9.4.8 Dependencies . . . 412
9.4.9 Troubleshooting . . . 412
9.5 Merge point . . . 413
9.5.1 How It Works . . . 413
9.5.2 Configuration Instructions . . . 413
9.5.3 Example Configuration . . . 413
9.5.4 Saved Parameters Description . . . 414
9.5.5 Inputs Description . . . 414
9.5.6 Outputs Description . . . 414
9.5.7 Operating Instructions . . . 414
9.5.8 Dependencies . . . 415
9.6 Snapshot . . . 415
9.6.1 How It Works . . . 415
9.6.2 Configuration Instructions . . . 415
9.6.3 Example Configuration . . . 416
9.6.4 Saved Parameters Description . . . 416
9.6.5 Inputs Description . . . 416
9.6.6 Outputs Description . . . 417
9.6.7 Supported Actions . . . 417
9.6.8 Dependencies . . . 417
9.6.9 Operating Instructions . . . 417
9.7 Sub-workflow . . . 417
9.7.1 How It Works . . . 418
9.7.2 Configuration Instructions . . . 419
9.7.3 Example Configuration . . . 419
9.7.4 Saved Parameters Description . . . 419
9.7.5 Inputs Description . . . 419

9.7.6 Outputs Description . . . 419
9.7.7 Supported Actions . . . 419
9.7.8 Dependencies . . . 420
9.7.9 Operating Instructions . . . 420
9.7.10 Troubleshooting . . . 420
9.8 User lookup . . . 420
9.8.1 How It Works . . . 420
9.8.2 Configuration Instructions . . . 420
9.8.3 Example Configuration . . . 421
9.8.4 Saved Parameters Description . . . 421
9.8.5 Inputs Description . . . 421
9.8.6 Outputs Description . . . 421
9.8.7 Supported Actions . . . 422
9.8.8 Dependencies . . . 422
9.8.9 Operating Instructions . . . 422
9.9 Work Order Metadata . . . 423
9.9.1 How It Works . . . 424
9.9.2 Configuration Instructions . . . 424
9.9.3 Example Configuration . . . 424
9.9.4 Saved Parameters Description . . . 424
9.9.5 Inputs Description . . . 425
9.9.6 Outputs Description . . . 425
9.9.7 Supported Actions . . . 425
9.9.8 Dependencies . . . 425
9.9.9 Operating Instructions . . . 425
9.9.10 Troubleshooting . . . 426
9.10 Workflow launcher . . . 426
9.10.1 How It Works . . . 427
9.10.2 Configuration Instructions . . . 427
9.10.3 Example Configuration . . . 428
9.10.4 Saved Parameters Description . . . 428
9.10.5 Inputs Description . . . 429
9.10.6 Outputs Description . . . 429
9.10.7 Supported Actions . . . 429
9.10.8 Dependencies . . . 429
9.10.9 Operating Instructions . . . 429
9.10.10 Overview . . . 429
9.10.11 Execution Modes . . . 429
9.10.12 Tracking Launched Workflows . . . 429
9.10.13 Via UI . . . 429
9.10.14 Via API . . . 429
9.10.15 Priority Levels . . . 429
9.10.16 Work Order Grouping . . . 430
9.10.17 Dedicated Execution Resources . . . 430
9.10.18 Important Notes . . . 430

10 Transcoding **431**
10.1 Akamai transcoding . . . 431
10.1.1 Saved Parameters Description . . . 431
10.1.2 Inputs description . . . 432
10.1.3 Outputs description . . . 432
10.1.4 Supported Actions . . . 432
10.1.5 Dependencies . . . 432
10.1.6 Operating Instructions . . . 432
10.2 Alchemist Snell OD operations . . . 432
10.2.1 How It Works . . . 433
10.2.2 Configuration Instructions . . . 433
10.2.3 Example Configuration . . . 433
10.2.4 Saved Parameters Description . . . 433
10.2.5 Inputs Description . . . 434

10.2.6 Outputs Description . . . 434
10.2.7 Supported Actions . . . 434
10.2.8 Dependencies . . . 434
10.2.9 Operating Instructions . . . 434
10.3 Amazon Elastic Transcoder . . . 436
10.3.1 Saved Parameters Description . . . 436
10.3.2 Inputs description . . . 436
10.3.3 Outputs description . . . 437
10.3.4 Supported Actions . . . 437
10.3.5 Dependencies . . . 437
10.3.6 Operating Instructions . . . 437
10.4 Amberfin transcoding . . . 437
10.4.1 Saved Parameters Description . . . 438
10.4.2 Inputs description . . . 438
10.4.3 Outputs description . . . 438
10.4.4 Operating Instructions . . . 439
10.4.5 An example of a transcode . . . 439
10.5 Ateme transcoding . . . 439
10.5.1 How It Works . . . 440
10.5.2 Configuration Instructions . . . 440
10.5.3 Example Configuration . . . 440
10.5.4 Saved Parameters Description . . . 440
10.5.5 Inputs Description . . . 440
10.5.6 Outputs Description . . . 441
10.5.7 Operating Instructions . . . 441
10.5.8 Dependencies . . . 441
10.5.9 Troubleshooting . . . 441
10.6 Bitmovin encoding . . . 442
10.6.1 Saved Parameters Description . . . 442
10.6.2 Inputs description . . . 442
10.6.3 Outputs description . . . 442
10.6.4 Supported Actions . . . 443
10.6.5 Dependencies . . . 443
10.6.6 Operating Instructions . . . 443
10.7 Cambria ftc transcoder . . . 444
10.7.1 Saved Parameters Description . . . 444
10.7.2 Inputs description . . . 444
10.7.3 Outputs description . . . 444
10.7.4 Supported Actions . . . 444
10.7.5 Dependencies . . . 444
10.7.6 Operating Instructions . . . 444
10.8 Carbon/WFS transcoding . . . 444
10.8.1 Saved Parameters Description . . . 446
10.8.2 Inputs description . . . 446
10.8.3 Outputs description . . . 447
10.8.4 Operating Instructions . . . 447
10.8.5 An example of a transcode . . . 447
10.9 Imagine StreamZ transcoding . . . 448
10.9.1 Saved Parameters Description . . . 448
10.9.2 Inputs description . . . 448
10.9.3 Outputs description . . . 449
10.9.4 Operating Instructions . . . 449
10.9.5 An example of a transcode . . . 449
10.10 Imagine TM transcoding . . . 449
10.10.1 Saved Parameters Description . . . 450
10.10.2 Inputs description . . . 450
10.10.3 Outputs description . . . 450
10.10.4 Operating Instructions . . . 451
10.10.5 An example of a transcode . . . 451
10.11 Dolby VM600/DP600 . . . 451

10.11.1 How It Works . . . 452
10.11.2 Configuration Instructions . . . 452
10.11.3 Example Configuration . . . 452
10.11.4 Saved Parameters Description . . . 452
10.11.5 Outputs Description . . . 453
10.11.6 Listing Workflow Profiles: . . . 453
10.11.7 Submitting a Work Order: . . . 453
10.11.8 Supported URI Formats . . . 453
10.11.9 Supported Work Order Types . . . 453
10.11.10 Operating Instructions . . . 453
10.11.11 Dependencies . . . 454
10.11.12 Troubleshooting . . . 454
10.12 Elemental Media Convert . . . 454
10.12.1 How It Works . . . 455
10.12.2 Configuration Instructions . . . 455
10.12.3 Example Configuration . . . 455
10.12.4 Saved Parameters Description . . . 455
10.12.5 Inputs Description . . . 456
10.12.6 Mandatory Inputs: . . . 456
10.12.7 Optional Inputs: . . . 456
10.12.8 Outputs Description . . . 456
10.12.9 Supported Output Group Types . . . 456
10.12.10 Operating Instructions . . . 457
10.12.11 Dependencies . . . 457
10.12.12 Troubleshooting . . . 457
10.13 Elemental transcoding . . . 457
10.13.1 How It Works . . . 458
10.13.2 Configuration Instructions . . . 458
10.13.3 Example Configuration . . . 459
10.13.4 Saved Parameters Description . . . 459
10.13.5 Inputs Description . . . 460
10.13.6 Mandatory Inputs: . . . 460
10.13.7 Mandatory Fields if Authentication is Needed: . . . 460
10.13.8 Optional Inputs: . . . 460
10.13.9 Outputs Description . . . 460
10.13.10 Operating Instructions . . . 461
10.13.11 Custom XML Payload Example . . . 461
10.13.12 Dependencies . . . 461
10.13.13 Troubleshooting . . . 461
10.14 Emotion operation . . . 461
10.14.1 Saved Parameters Description . . . 462
10.14.2 Inputs Description . . . 462
10.14.3 Outputs Description . . . 463
10.14.4 Operating Instructions . . . 463
10.15 Encoding.com transcoding . . . 463
10.15.1 Saved Parameters Description . . . 464
10.15.2 Inputs description . . . 464
10.15.3 Outputs description . . . 464
10.15.4 Operating Instructions . . . 465
10.15.5 An example of a transcode . . . 465
10.16 Envivio transcoding . . . 465
10.16.1 Saved Parameters Description . . . 466
10.16.2 Inputs description . . . 466
10.16.3 Outputs description . . . 466
10.16.4 Supported Actions . . . 466
10.16.5 Dependencies . . . 467
10.16.6 Operating Instructions . . . 467
10.17 EolementheWorkflowManager . . . 467
10.17.1 Saved Parameters Description . . . 467
10.17.2 Inputs description . . . 468

10.17.3 Outputs description . . . 468
10.17.4 Supported Actions . . . 468
10.17.5 Dependencies . . . 468
10.17.6 Operating Instructions . . . 468
10.18 Episode transcoding . . . 468
10.18.1 Saved Parameters Description . . . 469
10.18.2 Inputs description . . . 469
10.18.3 Outputs description . . . 470
10.18.4 Operating Instructions . . . 470
10.18.5 An example of a transcode . . . 470
10.19 FFMPEG transcoding . . . 470
10.19.1 How It Works . . . 471
10.19.2 Configuration Instructions . . . 471
10.19.3 Example Configuration . . . 471
10.19.4 Saved Parameters Description . . . 472
10.19.5 Inputs Description . . . 472
10.19.6 Outputs Description . . . 472
10.19.7 Command Construction . . . 472
10.19.8 Operating Instructions . . . 472
10.19.9 Dependencies . . . 473
10.19.10 Troubleshooting . . . 473
10.20 Flics transcoding . . . 473
10.20.1 How It Works . . . 473
10.20.2 Configuration Instructions . . . 473
10.20.3 Example Configuration . . . 474
10.20.4 Saved Parameters Description . . . 474
10.20.5 Inputs Description . . . 474
10.20.6 Outputs Description . . . 475
10.20.7 Supported Actions . . . 475
10.20.8 Dependencies . . . 475
10.20.9 Operating Instructions . . . 475
10.21 FlipFactory transcoding . . . 475
10.21.1 Saved Parameters Description . . . 476
10.21.2 Inputs description . . . 476
10.21.3 Outputs description . . . 476
10.21.4 Supported Actions . . . 477
10.21.5 Dependencies . . . 477
10.21.6 Operating Instructions . . . 477
10.22 HandBrake transcoding . . . 477
10.22.1 Saved Parameters Description . . . 477
10.22.2 Inputs description . . . 478
10.22.3 Outputs description . . . 478
10.22.4 Supported Actions . . . 478
10.22.5 Dependencies . . . 478
10.22.6 Operating Instructions . . . 478
10.23 Hybrik Transcoder . . . 479
10.23.1 How It Works . . . 479
10.23.2 Configuration Instructions . . . 479
10.23.3 Example Configuration . . . 480
10.23.4 Saved Parameters Description . . . 480
10.23.5 Inputs Description . . . 480
10.23.6 Outputs Description . . . 481
10.23.7 Supported Actions . . . 481
10.23.8 Dependencies . . . 481
10.23.9 Operating Instructions . . . 481
10.24 Marquis Mews Operation . . . 483
10.24.1 Saved Parameters Description . . . 483
10.24.2 Inputs description . . . 483
10.24.3 Outputs description . . . 484
10.24.4 Operating Instructions . . . 484

10.25 Minnetonka AudioTools Server . . . 484
10.25.1 Saved Parameters Description . . . 484
10.25.2 Inputs description . . . 485
10.25.3 Outputs description . . . 485
10.25.4 Supported Actions . . . 485
10.25.5 Dependencies . . . 485
10.25.6 Operating Instructions . . . 485
10.26 MOG MXF Speedrail . . . 485
10.26.1 Saved Parameters Description . . . 486
10.26.2 Inputs description . . . 486
10.26.3 Outputs description . . . 486
10.26.4 Supported Actions . . . 486
10.26.5 Dependencies . . . 486
10.26.6 Operating Instructions . . . 486
10.27 Unified packager . . . 486
10.27.1 Saved Parameters Description . . . 487
10.27.2 Inputs description . . . 487
10.27.3 Outputs description . . . 488
10.27.4 Supported Actions . . . 488
10.27.5 Dependencies . . . 488
10.27.6 Operating Instructions . . . 488
10.28 Vantage Transcoding . . . 488
10.28.1 How It Works . . . 489
10.28.2 Configuration Instructions . . . 490
10.28.3 Example Configuration . . . 490
10.28.4 Saved Parameters Description . . . 490
10.28.5 Inputs Description . . . 491
10.28.6 Mandatory Inputs for SubmitFile Method: . . . 491
10.28.7 Mandatory Inputs for SubmitItems Method: . . . 491
10.28.8 Optional Inputs: . . . 491
10.28.9 Outputs Description . . . 491
10.28.10 Supported API Methods . . . 492
10.28.11 Operating Instructions . . . 492
10.28.12 Step-by-Step Examples . . . 492
10.28.13 Example 1: Transcoding with SubmitFile API Method . . . 492
10.28.14 Example 2: Transcoding with SubmitItems API Method . . . 492
10.28.15 Example 3: Transcoding with Runtime Parameters (SubmitItems API Method) . . . 493
10.28.16 JSON Format Examples . . . 493
10.28.17 Vantage Variables Example: . . . 493
10.28.18 Vantage Items Example: . . . 493
10.28.19 Dependencies . . . 493
10.28.20 Troubleshooting . . . 493
10.28.21 Important Note - Custom Category Override . . . 493
10.29 VqmaAnalyzer . . . 494
10.29.1 Saved Parameters Description . . . 494
10.29.2 Inputs description . . . 494
10.29.3 Outputs description . . . 494
10.29.4 Supported Actions . . . 495
10.29.5 Dependencies . . . 495
10.29.6 Operating Instructions . . . 495
10.30 Xf converter . . . 495
10.30.1 Saved Parameters Description . . . 496
10.30.2 Inputs description . . . 496
10.30.3 Outputs description . . . 496
10.30.4 Supported Actions . . . 496
10.30.5 Dependencies . . . 496
10.30.6 Operating Instructions . . . 496
10.31 Zencoder transcoding . . . 497
10.31.1 How It Works . . . 497
10.31.2 Configuration Instructions . . . 498

10.31.3 Example Configuration . . . 498
10.31.4 Saved Parameters Description . . . 498
10.31.5 Inputs Description . . . 498
10.31.6 Outputs Description . . . 498
10.31.7 Supported Actions . . . 498
10.31.8 Operating Instructions . . . 499
10.31.9 Dependencies . . . 499
10.31.10 Troubleshooting . . . 499

11 Triggers **500**
11.1 Amazon SQS trigger . . . 500
11.1.1 How It Works . . . 500
11.1.2 Configuration Instructions . . . 501
11.1.3 Example Configuration . . . 501
11.1.4 Saved Parameters Description . . . 501
11.1.5 Inputs Description . . . 502
11.1.6 Outputs Description . . . 502
11.1.7 Supported Actions . . . 503
11.1.8 Dependencies . . . 503
11.1.9 Operating Instructions . . . 503
11.2 AMQP trigger . . . 503
11.2.1 How It Works . . . 504
11.2.2 Configuration Instructions . . . 504
11.2.3 Example Configuration . . . 505
11.2.4 Saved Parameters Description . . . 506
11.2.5 Inputs Description . . . 507
11.2.6 Outputs Description . . . 508
11.2.7 Supported Actions . . . 508
11.2.8 Dependencies . . . 508
11.2.9 Operating Instructions . . . 508
11.3 Aspera central watcher . . . 508
11.3.1 How It Works . . . 510
11.3.2 Configuration Instructions . . . 510
11.3.3 Example Configuration . . . 510
11.3.4 Saved Parameters Description . . . 511
11.3.5 Trigger Persistence Scope Details . . . 512
11.3.6 Trigger Types Details . . . 512
11.3.7 Inputs Description . . . 512
11.3.8 Outputs Description . . . 512
11.3.9 Supported Actions . . . 512
11.3.10 Operating Instructions . . . 512
11.3.11 Dependencies . . . 513
11.3.12 Troubleshooting . . . 513
11.4 AoC Package Watcher . . . 513
11.4.1 How It Works . . . 514
11.4.2 Configuration Instructions . . . 514
11.4.3 Example Configuration . . . 514
11.4.4 Saved Parameters Description . . . 515
11.4.5 Inputs Description . . . 516
11.4.6 Outputs Description . . . 516
11.4.7 Supported Actions . . . 516
11.4.8 Dependencies . . . 516
11.4.9 Operating Instructions . . . 517
11.4.10 Package Operations . . . 517
11.4.11 Filter Configuration . . . 517
11.4.12 Node API Integration . . . 517
11.4.13 Trigger Persistence Scopes . . . 517
11.5 Aspera Node API Transfer Monitor . . . 517
11.5.1 How It Works . . . 518
11.5.2 Configuration Instructions . . . 518

11.5.3 Advanced Configuration Options . . . 518
11.5.4 Connection Parameters . . . 518
11.5.5 Filtering Options . . . 518
11.5.6 Monitoring Behavior . . . 518
11.5.7 Example Configuration . . . 519
11.5.8 Saved Parameters Description . . . 519
11.5.9 Inputs Description . . . 519
11.5.10 Outputs Description . . . 519
11.5.11 Supported Actions . . . 520
11.5.12 Dependencies . . . 520
11.5.13 Operating Instructions . . . 520
11.5.14 Important Parameters . . . 520
11.6 Aspera node file watcher . . . 520
11.6.1 How It Works . . . 521
11.6.2 Configuration Instructions . . . 521
11.6.3 Example Configuration . . . 521
11.6.4 Saved Parameters Description . . . 522
11.6.5 Inputs Description . . . 523
11.6.6 Outputs Description . . . 523
11.6.7 Supported Actions . . . 523
11.6.8 Dependencies . . . 523
11.6.9 Operating Instructions . . . 523
11.6.10 Plugin Operation . . . 523
11.6.11 Configuration Guidelines . . . 523
11.6.12 Common Settings (File Search and File Pagination) . . . 523
11.6.13 File Search Specific (Default Mode) . . . 523
11.6.14 File Pagination Specific . . . 524
11.6.15 Node Configuration Requirements . . . 524
11.6.16 Node API Account Setup . . . 524
11.6.17 Path Configuration . . . 524
11.6.18 Port Configuration . . . 524
11.6.19 Trigger Persistence Scope Details . . . 525
11.6.20 Important Notes . . . 525
11.7 Azure Service Bus Trigger . . . 525
11.7.1 How It Works . . . 525
11.7.2 Configuration Instructions . . . 525
11.7.3 Example Configuration . . . 526
11.7.4 Saved Parameters Description . . . 526
11.7.5 Inputs Description . . . 526
11.7.6 Outputs Description . . . 527
11.7.7 Supported Actions . . . 527
11.7.8 Dependencies . . . 527
11.7.9 Operating Instructions . . . 527
11.8 Custom trigger . . . 527
11.8.1 How It Works . . . 528
11.8.2 Configuration Instructions . . . 528
11.8.3 Example Configuration . . . 528
11.8.4 Example: Sample code to check that a file transfer is complete based on the aspx file extension. . . . 528
11.8.5 Saved Parameters Description . . . 529
11.8.6 Inputs Description . . . 529
11.8.7 Outputs Description . . . 529
11.8.8 Supported Actions . . . 529
11.8.9 Operating Instructions . . . 529
11.8.10 Dependencies . . . 529
11.9 Database trigger . . . 530
11.9.1 How It Works . . . 530
11.9.2 Configuration Instructions . . . 530
11.9.3 Connection Modes . . . 531
11.9.4 Mode A — Configuration File / DSN Mode . . . 531
11.9.5 Mode B — Inline Connection String Mode . . . 531

11.9.6 Connection Precedence . . . 531
11.9.7 Example Configuration . . . 531
11.9.8 Query and Formatting Examples . . . 532
11.9.9 Trigger Persistence Scope Details . . . 532
11.9.10 Saved Parameters Description . . . 532
11.9.11 Inputs Description . . . 533
11.9.12 Outputs Description . . . 533
11.9.13 Supported Actions . . . 533
11.9.14 Dependencies . . . 533
11.9.15 Operating Instructions . . . 533
11.9.16 Driver Installation (Choose One) . . . 533
11.9.17 Oracle Configuration . . . 533
11.9.18 ODBC Configuration . . . 533
11.9.19 MySQL Configuration . . . 533
11.10 Email inbox watcher . . . 533
11.10.1 How It Works . . . 534
11.10.2 Configuration Instructions . . . 534
11.10.3 Example Configuration . . . 535
11.10.4 Saved Parameters Description . . . 536
11.10.5 Inputs Description . . . 536
11.10.6 Outputs Description . . . 537
11.10.7 Supported Actions . . . 537
11.10.8 Dependencies . . . 537
11.10.9 Operating Instructions . . . 537
11.11 Faspex5 inbox watcher . . . 538
11.11.1 How It Works . . . 538
11.11.2 Configuration Instructions . . . 538
11.11.3 Example Configuration . . . 539
11.11.4 Saved Parameters Description . . . 539
11.11.5 Inputs Description . . . 540
11.11.6 Outputs Description . . . 540
11.11.7 Trigger Persistence Scope Details . . . 541
11.11.8 Operating Instructions . . . 541
11.11.9 Orchestrator Instance Setup . . . 541
11.11.10 Faspex Server Configuration . . . 541
11.11.11 Required Configuration Parameters . . . 541
11.11.12 Testing and Validation . . . 541
11.11.13 Dependencies . . . 542
11.11.14 Troubleshooting . . . 542
11.12 Faspex5 package monitor . . . 542
11.12.1 How It Works . . . 542
11.12.2 Configuration Instructions . . . 543
11.12.3 Example Configuration . . . 543
11.12.4 Saved Parameters Description . . . 543
11.12.5 Inputs Description . . . 544
11.12.6 Outputs Description . . . 544
11.12.7 Supported Actions . . . 544
11.12.8 Dependencies . . . 544
11.12.9 Operating Instructions . . . 544
11.12.10 Prerequisites Setup . . . 544
11.12.11 Orchestrator Instance Requirements . . . 544
11.12.12 Faspex Server Configuration . . . 545
11.12.13 Required Configuration Parameters . . . 545
11.12.14 Operation Behavior . . . 545
11.12.15 Important Notes . . . 545
11.13 FTP Folder Trigger . . . 545
11.13.1 How It Works . . . 546
11.13.2 Configuration Instructions . . . 546
11.13.3 Example Configuration . . . 546
11.13.4 Saved Parameters Description . . . 547

11.13.5 Inputs Description . . . 547
11.13.6 Outputs Description . . . 548
11.13.7 Supported Actions . . . 548
11.13.8 Dependencies . . . 548
11.13.9 Operating Instructions . . . 548
11.14 FTP Trigger . . . 549
11.14.1 How It Works . . . 551
11.14.2 Configuration Instructions . . . 551
11.14.3 Example Configuration . . . 552
11.14.4 Saved Parameters Description . . . 552
11.14.5 Inputs Description . . . 553
11.14.6 Outputs Description . . . 553
11.14.7 Supported Actions . . . 554
11.14.8 Dependencies . . . 554
11.14.9 Operating Instructions . . . 554
11.15 Local file watcher . . . 554
11.15.1 How It Works . . . 556
11.15.2 Configuration Instructions . . . 556
11.15.3 Example Configuration . . . 556
11.15.4 Saved Parameters Description . . . 557
11.15.5 Inputs Description . . . 558
11.15.6 Outputs Description . . . 558
11.15.7 Supported Actions . . . 558
11.15.8 Dependencies . . . 558
11.15.9 Operating Instructions . . . 558
11.15.10 Overview . . . 558
11.15.11 Watch Directory Configuration . . . 559
11.15.12 Multi-Folder Search . . . 559
11.15.13 File Pattern Configuration . . . 559
11.15.14 Monitoring Behavior . . . 559
11.15.15 File Content and Output Options . . . 559
11.15.16 Archiving and Cleanup . . . 560
11.15.17 Trigger Persistence Scope Details . . . 560
11.16 Local folder watcher . . . 560
11.16.1 How It Works . . . 561
11.16.2 Configuration Instructions . . . 561
11.16.3 Example Configuration . . . 561
11.16.4 Saved Parameters Description . . . 561
11.16.5 Inputs Description . . . 562
11.16.6 Outputs Description . . . 562
11.16.7 Supported Actions . . . 562
11.16.8 Dependencies . . . 563
11.16.9 Operating Instructions . . . 563
11.17 Remote file watcher . . . 563
11.17.1 How It Works . . . 564
11.17.2 Configuration Instructions . . . 564
11.17.3 Example Configuration . . . 565
11.17.4 Saved Parameters Description . . . 565
11.17.5 Inputs Description . . . 566
11.17.6 Outputs Description . . . 567
11.17.7 Supported Actions . . . 567
11.17.8 Dependencies . . . 567
11.17.9 Operating Instructions . . . 567
11.18 Remote folder watcher . . . 568
11.18.1 How It Works . . . 568
11.18.2 Configuration Instructions . . . 568
11.18.3 Example Configuration . . . 569
11.18.4 Saved Parameters Description . . . 569
11.18.5 Inputs Description . . . 570
11.18.6 Outputs Description . . . 570

11.18.7 Supported Actions . . . 570
11.18.8 Dependencies . . . 570
11.18.9 Operating Instructions . . . 571
11.19 Schedule trigger . . . 571
11.19.1 How It Works . . . 572
11.19.2 Configuration Instructions . . . 572
11.19.3 Example Configuration . . . 573
11.19.4 Saved Parameters Description . . . 573
11.19.5 Inputs Description . . . 573
11.19.6 Outputs Description . . . 573
11.19.7 Supported Actions . . . 573
11.19.8 Dependencies . . . 573
11.19.9 Operating Instructions . . . 574
11.20 SFTP trigger . . . 575
11.20.1 How It Works . . . 575
11.20.2 Configuration Instructions . . . 575
11.20.3 Example Configuration . . . 576
11.20.4 Saved Parameters Description . . . 576
11.20.5 Inputs Description . . . 577
11.20.6 Outputs Description . . . 577
11.20.7 Supported Actions . . . 577
11.20.8 Dependencies . . . 577
11.20.9 Operating Instructions . . . 577
11.21 Soap request listener . . . 577
11.21.1 How It Works . . . 578
11.21.2 Configuration Instructions . . . 578
11.21.3 Example Configuration . . . 578
11.21.4 Saved Parameters Description . . . 579
11.21.5 Inputs Description . . . 579
11.21.6 Outputs Description . . . 579
11.21.7 Supported Actions . . . 579
11.21.8 Dependencies . . . 579
11.21.9 Operating Instructions . . . 579
11.22 SonyCi file watcher . . . 579
11.22.1 How It Works . . . 580
11.22.2 Configuration Instructions . . . 580
11.22.3 Example Configuration . . . 580
11.22.4 Saved Parameters Description . . . 580
11.22.5 Inputs Description . . . 581
11.22.6 Outputs Description . . . 581
11.22.7 Trigger Persistence Scope Details . . . 581
11.22.8 Operating Instructions . . . 581
11.22.9 Dependencies . . . 582
11.22.10 Troubleshooting . . . 582

12 User Interactions **583**
12.1 Console notification . . . 583
12.1.1 How It Works . . . 584
12.1.2 Configuration Instructions . . . 584
12.1.3 Example Configuration . . . 584
12.1.4 Saved Parameters Description . . . 584
12.1.5 Inputs Description . . . 585
12.1.6 Mandatory Inputs: . . . 585
12.1.7 Optional Runtime Inputs: . . . 585
12.1.8 Outputs Description . . . 585
12.1.9 Setup Instructions . . . 585
12.1.10 Operating Instructions . . . 585
12.1.11 Dependencies . . . 586
12.1.12 Troubleshooting . . . 586
12.2 Email notification . . . 586

12.2.1 How It Works . . . 587
12.2.2 Configuration Instructions . . . 587
12.2.3 Example Configuration . . . 588
12.2.4 Saved Parameters Description . . . 588
12.2.5 Inputs Description . . . 588
12.2.6 Outputs Description . . . 588
12.2.7 Operating Instructions . . . 589
12.2.8 *Defining the Message Core parameters* . . . 589
12.2.9 *Defining the HTML Message Template* . . . 589
12.2.10 *Attaching files* . . . 589
12.2.11 Supported Actions . . . 589
12.2.12 Dependencies . . . 589
12.2.13 Troubleshooting . . . 589
12.3 RSS Feed Reader . . . 589
12.3.1 How It Works . . . 590
12.3.2 Configuration Instructions . . . 590
12.3.3 Example Configuration . . . 590
12.3.4 Saved Parameters Description . . . 591
12.3.5 Inputs Description . . . 591
12.3.6 Outputs Description . . . 591
12.3.7 Supported Actions . . . 592
12.3.8 Dependencies . . . 592
12.3.9 Operating Instructions . . . 592
12.4 SCOM notification . . . 592
12.5 Slack notification . . . 593
12.5.1 How It Works . . . 593
12.5.2 Configuration Instructions . . . 593
12.5.3 Example Configuration . . . 593
12.5.4 Saved Parameters Description . . . 594
12.5.5 Inputs Description . . . 594
12.5.6 Outputs Description . . . 594
12.5.7 Supported Actions . . . 594
12.5.8 Dependencies . . . 595
12.5.9 Operating Instructions . . . 595
12.5.10 Overview . . . 595
12.5.11 Authentication Setup . . . 595
12.5.12 Channel Configuration . . . 595
12.5.13 Message Formatting . . . 595
12.5.14 Basic Text Messages . . . 595
12.5.15 Message Attachments . . . 595
12.5.16 Posting As User vs Bot . . . 595
12.5.17 Sample Request . . . 596
12.5.18 Output Processing . . . 596
12.5.19 Common Color Values . . . 596
12.5.20 Error Handling . . . 596
12.5.21 Best Practices . . . 596
12.6 User input . . . 596
12.6.1 How It Works . . . 597
12.6.2 Configuration Instructions . . . 597
12.6.3 Example Configuration . . . 598
12.6.4 Saved Parameters Description . . . 598
12.6.5 Basic Parameters . . . 598
12.6.6 Email Notification Parameters . . . 599
12.6.7 User Assignment Parameters . . . 599
12.6.8 Inputs Description . . . 599
12.6.9 Outputs Description . . . 599
12.6.10 Operating Instructions . . . 599
12.6.11 Dependencies . . . 600

13 Virus Scan **601**

13.1 ClamAV check . . . 601
13.1.1 How It Works . . . 602
13.1.2 Configuration Instructions . . . 602
13.1.3 Example Configuration . . . 602
13.1.4 Saved Parameters Description . . . 603
13.1.5 Inputs Description . . . 603
13.1.6 Outputs Description . . . 603
13.1.7 Supported Actions . . . 603
13.1.8 Dependencies . . . 604
13.1.9 Operating Instructions . . . 604
13.2 ICAP virus scan . . . 606
13.2.1 Saved Parameters Description . . . 606
13.2.2 Inputs description . . . 607
13.2.3 Outputs description . . . 607
13.2.4 Supported Actions . . . 607
13.2.5 Dependencies . . . 607
13.2.6 Operating Instructions . . . 607
13.3 McAfee virus check . . . 607
13.3.1 Saved Parameters Description . . . 608
13.3.2 Inputs description . . . 608
13.3.3 Outputs description . . . 608
13.3.4 Supported Actions . . . 608
13.3.5 Operating Instructions . . . 608
13.4 Opswat metadefender . . . 612
13.4.1 Saved Parameters Description . . . 613
13.4.2 Inputs description . . . 613
13.4.3 Outputs description . . . 613
13.4.4 Supported Actions . . . 613
13.4.5 Dependencies . . . 613
13.4.6 Operating Instructions . . . 613
13.5 Symantec Icap DLP Operations . . . 614
13.5.1 Saved Parameters Description . . . 614
13.5.2 Inputs description . . . 614
13.5.3 Outputs description . . . 614
13.5.4 Supported Actions . . . 614
13.5.5 Dependencies . . . 615
13.5.6 Operating Instructions . . . 615
13.6 Symantec Icap Virus Scan Operations . . . 615
13.6.1 Saved Parameters Description . . . 615
13.6.2 Inputs description . . . 616
13.6.3 Outputs description . . . 616
13.6.4 Supported Actions . . . 616
13.6.5 Dependencies . . . 616
13.6.6 Operating Instructions . . . 616

Chapter 1

Asset Management

1.1 Dalet Asset Management Plugin



Figure 1.1: Dalet Asset Management Plugin icon

This plug-in provides the ability to interact with Dalet Asset Management System.

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.5	Fixing reference in help file.
0.1.4	Setting SSL flag to true to handle https
0.1.3	Corrected usage of resp.body and resp_data
0.1.2	Added more payloads
0.1.1	Changed the category to Asset Management
0.1.0	adding payload columns
0.0.1	initial release

1.1.1 Saved Parameters Description

- Name: The name used to identify a saved DaletAssetManagement configured instance.
- Payload file: The file containing the API request to the Dalet Media Server. Click on 'refresh' to update the saved inputs.
- Saved inputs: The parameters to be sent in the API request. This is updated after the payload file is specified.
- Soap wsdl path: The URL for the wsdl file used to make the Dalet API calls.

1.1.2 Inputs description

There are three inputs available in this plugin: payload file, saved inputs, and soap wsdl path. All three inputs are required. Only the payload file and soap wsdl path may be specified at runtime.

1.1.3 Outputs description

- *response*: The processed response from the request

1.1.4 Supported Actions

None

1.1.5 Operating Instructions

This plugin allows for API calls to be made to a Dalet Media Server.

The API calls need to be created in the actions_config folder. API calls should be specified in the Payload file, then clicking 'refresh' should populate the saved inputs field. The response from the server is processed in an array format.

This plugin is compatible with Dalet version 4.0 or greater.

1.2 Evertz Mediator operation



Figure 1.2: Evertz Mediator operation icon

This plugin provides ability to interact with a Evertz Mediator system.

Version	Comment
0.1.2	Enhanced action template UI
0.1.1	Fix mediator status, add SSL support and help page, and clean up code
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.4	Fix for ruby 2.3 compatibility
0.0.3	Fix submission
0.0.1	First version

1.2.1 How It Works

- Connects to an Evertz Mediator web service endpoint using either a configured Mediator address or a selected Remote Node.
- Authenticates with the provided Mediator credentials and starts a Mediator session.
- Submits a workflow transition request to update the status of the specified placing.
- Supports credentials saved directly in the action, supplied at runtime, or resolved from Remote Node parameters.
- Supports SSL/TLS connections to Mediator when the Use SSL option is enabled.
- Returns the Mediator session key as an output and logs out after the status update attempt.

1.2.2 Configuration Instructions

1. Create a new action step using the EvertzMediatorOperation plugin.
2. The plugin logs in to Evertz Mediator and performs a workflow status transition for a placing.
3. Fill out the following fields:
    - Name: The name used to identify a saved EvertzMediatorOperation configured instance.

- Comments: Some comments about this saved EvertzMediatorOperation configured instance.
- Mediator Node: Optional Remote Node used to resolve the Mediator connection address and, if configured, the credentials.
- Mediator Address: The Mediator hostname, base URL, or service URL. If only a host is provided, the plugin automatically appends /mediator/main/ws.
- Operation: The Mediator action to perform. This plugin currently supports updating workflow status.
- Placing ID: The Mediator placing identifier to update.
- Mediator User: The Mediator username used to log in. This can also come from runtime inputs or the selected Remote Node parameter mediator_user.
- Mediator Password: The Mediator password used to log in. This can also come from runtime inputs or the selected Remote Node parameter mediator_password.
- Mediator Status: The target Mediator workflow status or requirement name that should be applied to the placing.
- Use SSL?: Check this box to connect to the Mediator service using HTTPS instead of HTTP.

1.2.3 Example Configuration

Example plugin configuration using a direct Mediator address.

This configuration connects directly to a Mediator server over HTTPS and updates the placing status.

Parameter	Value
Name	Direct Mediator Status Update
Comments	Sample configuration using a direct Mediator address
Mediator Address	mediator.example.com
Operation	Update status
Placing ID	123456
Mediator User	api_user
Mediator Password	password
Mediator Status	Approved
Use SSL?	Selected

Example plugin configuration using a Remote Node.

This configuration uses a saved Remote Node for connectivity and updates the placing status in Mediator.

Parameter	Value
Name	Node-based Mediator Status Update
Comments	Sample configuration using a Remote Node
Mediator Node	Mediator Node A
Operation	Update status
Placing ID	ABC-10025
Mediator Status	Ready for TX
Use SSL?	Selected

1.2.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved EvertzMediatorOperation configured instance.
Comments	Some comments about this saved EvertzMediatorOperation configured instance.
Mediator Node	Optional Remote Node used to resolve the Mediator connection address and, if configured, the credentials.
Mediator Address	The Mediator hostname, base URL, or service URL. If only a host is provided, the plugin automatically appends /mediator/main/ws.
Operation	The Mediator action to perform. This plugin currently supports updating workflow status.
Placing ID	The Mediator placing identifier to update.

Parameter	Description
Mediator User	The Mediator username used to log in. This can also come from runtime inputs or the selected Remote Node parameter mediator_user.
Mediator Password	The Mediator password used to log in. This can also come from runtime inputs or the selected Remote Node parameter mediator_password.
Mediator Status	The target Mediator workflow status or requirement name that should be applied to the placing.
Use SSL?	Check this box to connect to the Mediator service using HTTPS instead of HTTP.

1.2.5 Inputs Description

The list of inputs depends on the configuration of the EvertzMediatorOperation action template. Some values can be saved in the action, supplied at runtime, or resolved from the selected Remote Node.

Parameter	Description
*Placing_id **	The Mediator placing identifier to update.
*Mediator_status **	The target Mediator workflow status or requirement name.
Mediator_user	Required at runtime if no saved value exists and the selected Remote Node does not provide mediator_user.
Mediator_password	Required at runtime if no saved value exists and the selected Remote Node does not provide mediator_password.
Mediator_node	Required at runtime if the action is configured to use the runtime node option.
Mediator_address	Required at runtime if no Mediator Node or saved Mediator Address is configured.

1.2.6 Outputs Description

Parameter	Description
Session_ID	The session key returned by the Mediator login request.

1.2.7 Supported Actions

Update status

1.2.8 Dependencies

Network connectivity to the Evertz Mediator web service and valid credentials with permission to perform workflow transitions.

1.2.9 Operating Instructions

How to Configure an Evertz Mediator operation?

To update a placing status in Mediator, the following is required:

- Provide either a Mediator Address or a Mediator Node.
- If using a Remote Node, ensure it resolves to the correct Mediator host and any required node parameters are configured.
- Use valid Mediator credentials with permission to perform workflow transitions.
- Enable Use SSL when the Mediator endpoint requires HTTPS.

Connection Behavior

The plugin logs in to Mediator, performs the requested status transition, and then logs out. The returned session key is also exposed as an output.

Troubleshooting Tips

- If only a hostname is provided for Mediator Address, the plugin automatically appends /mediator/main/ws.
- If login fails, verify the username, password, endpoint address, and SSL setting.
- If the status change fails, verify that the placing ID exists and that the target Mediator status is valid for the workflow.

1.3 ShowMgr



Figure 1.3: ShowMgr icon

This action plug-ins is used to interact with ShowMgr (using API calls)

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.1	Changed the category to asset mgmt.
0.1.0	added service url
0.0.1	initial release

1.3.1 Saved Parameters Description

- ”This action plug-ins is used to interact with ShowMgr (using API calls)”
- Name: The name used to identify a saved ShowMgrOperation configured instance.
- Comments: Comments associated to this template or configuration.
- Subscription Key: The Access key used to access the API calls
- Service URL: The Service url of the service we are trying to reach to
- API Operation: The list of operations currently supported by the plugin. (Get Orders List, Get Orders Info, Put Orders Status, Get and Update Trigger)
- Order Class: The Class of the Order we are trying to access
- Requesting Order Status: The Order status of the Orders we are looking for.
- Updating Order Status: The Order status of the job we want it to be updated with.
- Order Id: Order Id of the job we want information about
- Keep Trigger Ongoing?: This is for the one API operation where we can keep looking for the orders at a given frequency

  Details:

  'Get Orders List' - To get the Orders list matching the given Order Class and Order Status.

  'Get Order Info' - Given an Order Id this operation will retrieve the information about the order.

  'Update Order Status' - Given the Order id it will update the job with the collected status information.

  'ShowMgr Trigger Operation' - This operation combines the Get Order and Update in to 1 step and has a trigger associated with it to support constant polling.

1.3.2 Inputs description

- Service URL And Subscription Key are the only mandatory parameter required for this plugin.

  Every Operation Requires some input while not the Other.

  Get Orders List - No Mandatory input. Order Class and Status can be provided to narrow down the result

  Get Order Info - Order Id must be provided to collect the information

  Update Order Status - Order Id and the status to be updated with must be provided

  ShowMgr Trigger Operation - Updating Order Status must be provided. Also, Order Status and Class if provided will narrow down the results better.

1.3.3 Outputs description

- This action executes an Subler transformation action on a remote node, via SSH.
- *Order_Info*: Hash of the information about the order id given
- *Orders_list*: List of Orders matching the given criteria
- *Order_Id*: Order Id of the Order that was taken in to execution
- *File Location*: File Location of the Order id provided
- *Job Status*: Job status of the Order Id provided
- *Execution Report*: Response received as is from the API call
- *Error*: If the request failed, this parameter will hold the reason for the failure.
- *Order Status*: Status of the Order Id Provided.

1.3.4 Supported Actions

None

1.3.5 Dependencies

None

1.3.6 Operating Instructions

Make sure that you have a valid subscription key to reach the ShowMgr API methods.

1.4 Sony Ci

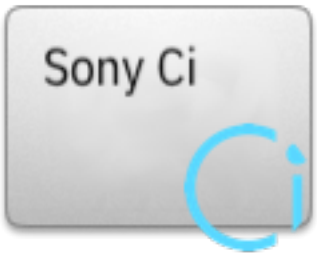


Figure 1.4: Sony Ci icon

This plugin enables management of media assets within Sony Ci cloud platform through upload, download, and metadata operations.

Version	Comment
0.2.2	Enhanced action template UI
0.2.1	Fix issues when storing certain string characters (Like colons)

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	add operations upload, get metadata, set metadata
0.0.2	add Content-Length header
0.0.1	initial development

1.4.1 How It Works

- Integrates with Sony Ci cloud platform to manage media assets through upload, download, and metadata operations.
- Connects to Sony Ci using API credentials (username, password, client id, and client secret) for secure authentication.
- Supports four main operations: UPLOAD (create assets), DOWNLOAD (retrieve assets), GET METADATA (read asset metadata), and SET METADATA (update asset metadata).
- Generates Aspera transfer specifications for high-speed file transfers in conjunction with Node API Transfer plugin.
- Handles workspace and folder management within Sony Ci for organized asset storage.

1.4.2 Configuration Instructions

1. Add a Sony Ci Media Asset Management step to your workflow.
2. Configure the Sony Ci authentication:
    - Username: Your Sony Ci account username.
    - Password: Your Sony Ci account password.
    - Client id: Sony API client id for authentication.
    - Client secret: Sony API client secret for authentication.
3. Set up the operation parameters:
    - Operation: Choose UPLOAD, DOWNLOAD, GET METADATA, or SET METADATA.
    - Workspace id: Target workspace (for UPLOAD operations).
    - Folder id: Target folder within workspace (for UPLOAD operations).
    - Asset id: Asset identifier (for DOWNLOAD, GET/SET METADATA operations).

1.4.3 Example Configuration

Parameter	Example Value
Username	user@company.com
Password	********
Client id	sony-api-client-123
Client secret	********
Operation	UPLOAD
Workspace id	ws-12345-abcde
Folder id	folder-67890-fghij
Source filepath	/path/to/video.mp4
Source filesize	1073741824
Asset id	asset-98765-klmno (for non-upload operations)

1.4.4 Saved Parameters Description

Parameter	Description
Comments	The comments of the Sony Ci Transcoder template.
Username	The Sony Ci account username for authentication.
Password	The Sony Ci account password for authentication.
Client id	The Sony API client id used for authentication.
Client secret	The Sony API client secret used for authentication.
Operation	The operation to execute: UPLOAD, DOWNLOAD, GET METADATA, or SET METADATA.
Workspace id	The target workspace id for UPLOAD operations. Optional - defaults to user's personal workspace if not specified.
Folder id	The target folder id for UPLOAD operations. Optional - defaults to workspace root folder if not specified.

Parameter	Description
Source filesize	The filesize in bytes of the source file to upload for UPLOAD operations. Optional - defaults to 1 byte.
Source filepath	The filepath of the source file to upload for UPLOAD operations. Can be filename only if in docroot.
Asset id	The asset id for DOWNLOAD, GET METADATA and SET METADATA operations.
Download folder	The download folder path for DOWNLOAD operations. Must be relative to docroot.
Add metadata	Metadata name-value pairs to add for SET METADATA operations.

1.4.5 Inputs Description

Authentication parameters (available for all operations):

- Username: The Sony Ci account username. Overrides configuration value.
- Password: The Sony Ci account password. Overrides configuration value.
- Client id: The Sony API client id used for authentication. Overrides configuration value.
- Client_secret: The Sony API client secret used for authentication. Overrides configuration value.

For UPLOAD operations:

- Workspace id: The target workspace id for asset creation.
- Folder id: The target folder id within the workspace.
- Filesize: The filesize in bytes of the source file to upload.
- Filepath: The filepath of the source file to upload.

For DOWNLOAD, GET METADATA and SET METADATA operations:

- Asset_id: The asset id for DOWNLOAD, GET METADATA and SET METADATA operations.
- Parameters: The metadata name-value pairs for SET METADATA operations. Format: { 'name1' => ['value1'], 'name2' => ['value2'] }

1.4.6 Outputs Description

For UPLOAD operations:

- asset_id: The created asset id.
- result_transfer_spec: The upload transfer specification.

For DOWNLOAD operations:

- result_transfer_spec: The download transfer specification.

For GET METADATA and SET METADATA operations:

- metadata: The array of collected or set name-value pairs.

1.4.7 Supported Operations

- UPLOAD - Create an asset in Sony Ci and get Aspera transfer specification
- DOWNLOAD - Get Aspera transfer specification to download an asset from Sony Ci
- GET METADATA - Return metadata name-value pairs for a given asset
- SET METADATA - Set metadata name-value pairs for a given asset

1.4.8 Operating Instructions

Prerequisites:

- The api.cimediacloud.com endpoint must be reachable on port TCP 443 from your Orchestrator system.
- The Sony Ci workspace must be Aspera enabled for the plugin to work.
- This plugin must be used in conjunction with the Node API Transfer plugin for actual file transfers.

Upload Configuration:

- UPLOAD Operation: Creates an asset in Sony Ci and returns an Aspera transfer specification for upload.
- Specify Sony Ci credentials (username, password, client id, client secret).

- Configure workspace id (optional - defaults to personal workspace), folder id (optional - defaults to root), source filesize (optional - defaults to 1 byte), and filepath.
- Source filepath must be relative to the docroot of the system user linked to the node API user. If in docroot, use filename only.
- Use the returned transfer spec with Node API Transfer plugin by selecting 'Submit Transfer Using Payload'.
- Set activity_logging to true in aspera.conf <server> section for completion.
- Files with the same name create new assets instead of overwriting existing ones.

Other Operations:

- DOWNLOAD Operation: Returns Aspera transfer specification for downloading assets from Sony Ci. Requires the Node API Transfer plugin to perform the actual download using the returned transfer specifications.
- Download folder must be specified relative to docroot of system user linked to node user.
- GET METADATA Operation: Returns metadata name-value pairs for specified asset.
- SET METADATA Operation: Sets metadata name-value pairs for specified asset. Metadata names must exist or error occurs.
- Metadata can be entered directly using the 'Add Metadata' interface by specifying Name and Value fields, or programmatically using the format: { 'name1' => ['value1'], 'name2' => ['value2'] }. For example: { 'test' => ['Test-val2'] }.

1.4.9 Dependencies

- Sony Ci workspace must be Aspera-enabled for transfer operations.
- Node API Transfer plugin is required for actual file upload/download operations.
- The aspweb user must have read permissions for source files and write permissions for download folders.
- Network connectivity to api.cimediacloud.com on port TCP 443 is required.

1.4.10 Troubleshooting

- Authentication failures: Verify Sony Ci credentials (username, password, client id, client secret) are correct and active.
- Upload failures: Ensure workspace is Aspera-enabled and source file path is correct relative to docroot.
- Download failures: Check that asset id exists and download folder path is correct relative to docroot.
- Metadata errors: Verify metadata names exist for the asset before attempting SET METADATA operations.
- Transfer spec issues: Ensure Node API Transfer plugin is properly configured with 'Submit Transfer Using Payload' option.
- File size mismatches: Transfers complete even if the specified file size is inaccurate, but avoid setting it to 0 bytes.
- Network connectivity: Test connectivity to api.cimediacloud.com on port 443 if operations fail.

1.5 The Platform



Figure 1.5: The Platform icon

This action plug-in provides the ability to create, read, and update thePlatform objects.

Version	Comment
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.2	verify ssl false

Version	Comment
0.3.1	Adding help file
0.3.0	Adding support for Program and Server Objects
0.2.0	token storage, support for all form types, more configuration options
0.1.8	Changing the category
0.1.7	Added styling to help text.
0.1.6	Added logic to return failed status and propagate exceptions from thePlatform when API calls go wrong.
0.1.5	Removing code to explicitly encoding query values. RestClient is doing it just before the call
0.1.4	Added new output for total result count
0.1.3	Improvements requested by Roger 4-11-2014: Make fields run-time variables, fix for count and pretty which are always true, added new output
0.1.2	Improvements requested by Roger 4-10-2014
0.1.1	Improvements and bug fixes 4-9-2014
0.1.0	Added columns needed for media obj create and media obj update
0.0.1	Initial Release

1.5.1 Saved Parameters Description

Name: The name used to identify a saved thePlatform configured instance.

Comments: The comments to identify the plugin template

1.5.2 Authentication Parameters Start

Account Name: Name of the account which contains the data that you are querying. The value can be found on the top right hand side when you are logged in to ThePlatform GUI. Usually Development, Staging and Production accounts are setup

ThePlatform User Name: User name used to log into ThePlatform GUI

ThePlatform Password: Password used to log into ThePlatform GUI

1.5.3 Authentication Parameters End

Object: Supported Objects to query are Media, MediaFile, Program, Server. Depending on the Object chosen the respective API call is made to ThePlatform

Schema: Choosen the schema that supports the latest API calls

Form Type: Choosen the format of the response. We recommend JSON

Store Token: If checked, Orchestrator will store the API authentication token until it expires instead of generating one every API call. Tokens could be limited and this option could be used to use tokens as needed.

ThePlatform URL: The URL to which the API call is made to. Parameterized for special cases. Default URL is https://data.media.theplatform.com/media/data/ which is valid for Media, MediaFile and Server Objects. The URL is usually appended by the object_id. Use http://data.entertainment.tv.theplatform.com/entertainment/data/Program for Program Object

Authentication URL: The URL to which the API call is made to for obtaining the token. Defaulted to https://identity.auth.theplatform.com/idm/web/Authentication/

Account URL: URL of the account under which the Media Object is stored. Used for Media Object creation, update and delete

Media Asset: Name of the Media Asset being created.

Source URL: URL for the source file used for creation of new media file within a media object

Media ID URL: URL for the media object which needs to be updated with a new media file

Fields: Every ThePlatform object contains many fields, including custom fields. Check the fields needed in the response. Not chosing any will result in a response that includes

Count: If checked, the API call requests for a count field in the response. Useful for cases like search when only a count is needed.

Entries: If checked, the API call requests for all entries. Unchecking will result in the API call being submitted with entries='false'. Check ThePlatform documentation for more information.

Owned: If checked, the API call will include /owned. Check ThePlatform documentation for more information.

Types: If checked, the API call is submitted with types='variable'. Check ThePlatform documentation for more information.

Valid feed: If checked, the API call is submitted with validFeed='true'. Check ThePlatform documentation for more information.

Feed: If checked, the API call will include /feed. Check ThePlatform documentation for more information.

Pretty: If checked, the API call requests for a formatted response adhering to the Format chosen above.

Sort By: Click 'Add Sort By' to build out the API call so that the response contains resulted sorted either in descending or ascending order of the fields chosen

Query By: Click 'Add Query By' to build out the API call so that the response includes only the data that matches with the query parameters and their values.

Range: Provide a range to limit or expand the number of entries. For example, enter range as 0 to 4 to obtain the top five results. Range is calculated after the Sort and Query operations.

1.5.4 Inputs description

The list of inputs depends on the configuration of the thePlatform action template.

- All required parameters are available as inputs at runtime if not specified by the template.

  This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

These inputs are mandatory for thePlatform API call:

- *Account Name*: Name of the account which contains the data that you are querying. The value can be found on the top right hand side when you are logged in to ThePlatform GUI. Usually Development, Staging and Production accounts are setup
- *ThePlatform User Name*: User name used to log into ThePlatform GUI
- *ThePlatform Password*: Password used to log into ThePlatform GUI
- *Object*: Supported Objects to query are Media, MediaFile, Program, Server. Depending on the Object chosen the respective API call is made to ThePlatform
- *Schema*: Choosen the schema that supports the latest API calls
- *Form Type*: Choosen the format of the response. We recommend JSON
- *Operation*: Supports Search operation for all Objects. Creation and Deletion of Media Objects. Creation of Media File Objects inside a Media Object (essentially updating a Media Object).

  For Media Object creation below inputs are needed:

- *Account URL*: URL of the account under which the Media Object is stored. Used for Media Object creation, update and delete
- *Media Asset*: Name of the Media Asset being created.

  For Media File Object creation below inputs are needed:

- *Account URL*: URL of the account under which the Media Object is stored. Used for Media Object creation, update and delete
- *Source URL*: URL for the source file used for creation of new media file within a media object
- *Media ID URL*: URL for the media object which needs to be updated with a new media file

These inputs are optional:

- *Store Token*: If checked, Orchestrator will store the API authentication token until it expires instead of generating one every API call. Tokens could be limited and this option could be used to use tokens as needed.

- *ThePlatform URL*: The URL to which the API call is made to. Parameterized for special cases. Default URL is https://data.media.theplatform.com/media/data/ which is valid for Media, MediaFile and Server Objects. The URL is usually appended by the object_id. Use http://data.entertainment.tv.theplatform.com/entertainment/data/Program for Program Object
- *Authentication URL*: The URL to which the API call is made to for obtaining the token. Defaulted to https://identity.auth.theplatform.com/idm/web/Authentication/
- *Fields*: Every ThePlatform object contains many fields, including custom fields. Check the fields needed in the response. Not chosing any will result in a response that includes
- *Count*: If checked, the API call requests for a count field in the response. Useful for cases like search when only a count is needed.
- *Entries*: If checked, the API call requests for all entries. Unchecking will result in the API call being submitted with entries='false'. Check ThePlatform documentation for more information.
- *Owned*: If checked, the API call will include /owned. Check ThePlatform documentation for more information.
- *Types*: If checked, the API call is submitted with types='variable'. Check ThePlatform documentation for more information.
- *Valid feed*: If checked, the API call is submitted with validFeed='true'. Check ThePlatform documentation for more information.
- *Feed*: If checked, the API call will include /feed. Check ThePlatform documentation for more information.
- *Pretty*: If checked, the API call requests for a formatted response adhering to the Format chosen above.
- *Sort By*: Click 'Add Sort By' to build out the API call so that the response contains resulted sorted either in descending or ascending order of the fields chosen
- *Query By*: Click 'Add Query By' to build out the API call so that the response includes only the data that matches with the query parameters and their values.
- *Range*: Provide a range to limit or expand the number of entries. For example, enter range as 0 to 4 to obtain the top five results. Range is calculated after the Sort and Query operations.

  In addition, every Query By value chosen becomes a optional runtime input

1.5.5 Outputs description

- *QueryResults*: Result of the Search operation. Depending on the Type chosen it is either going to be a Hash or a String.
- *ThePlatform_Token*: A String value representing the API token used to query thePlatform.
- *Is_Token_Released*: Result of the API call to release the API token. It is either true or false.
- *Create_result*: Result of the Create Media Object API call. Depending on the Type chosen it is either going to be a Hash or a String.
- *Media_Id_Url*: Media ID URL obtained as a result of the Media Object creation
- *Delete_result*: Result of the Delete Media Object API call. Depending on the Type chosen it is either going to be a Hash or a String.
- *Update_result*: Result of the Create New Media File object API call (to modify a Media Object). Depending on the Type chosen it is either going to be a Hash or a String.
- *Total_result_count*: Total count of the objects returned.

1.5.6 Operating Instructions

This action plug-in is used make API calls to ThePlatform (also known as MPX). The operations supported are:

- Search for Media Objects, Program Objects, MediaFile Objects, Server Objects
- Create New Media Object
- Create New File Object
- Delete Media Object

The mandatory inputs are listed above.

1.5.7 Examples given below

1.5.8 An example of Search

a) Provide the Authentication Parameters.

b) Select Object to Search, Form Type and Schema. Check ThePlatform API documentation for latest schema listings.

c) Select Operation as Search and click to store the token for optimal use of tokens

d) Check the Fields desired in the response.

e) Check the Count, Entries, Feed, Valid Feed, Owned and Pretty falgs if desired.

f) Add fields to sort the response by.

g) Add fields to Query by and supply the values. Limit the range or expand it depending on the need.

1.5.9 An example of Create Media Object

a) Provide the Authentication Parameters.

b) Select Object as Media, Form Type and Schema. Check ThePlatform API documentation for latest schema listings.

c) Select Operation as CreateNewMediaObject and click to store the token for optimal use of tokens

d) Supply the account url and asset name.

e) Check Pretty flag to format response.

1.5.10 An example of Create Media File Object

a) Provide the Authentication Parameters.

b) Select Object as MediaFile, Form Type and Schema. Check ThePlatform API documentation for latest schema listings.

c) Select Operation as CreateNewFileObject and click to store the token for optimal use of tokens

d) Supply the account url, media id url and soure url.

e) Check Pretty flag to format response.

1.6 XytechMediaPulse



Figure 1.6: XytechMediaPulse icon

This plug-in provides the ability to query the MediaPulse API and return MediaOrders and corresponding source file(s).

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.3	Changed the category to asset mgmt.
0.0.2	Removed the check_once input parameter.
0.0.1	Plugin release.

1.6.1 Inputs description

The list of inputs depends on the configuration of the XytechMediaPulse action template.

- Name: Name of the action step.
- Comments: Comments associated to this template or configuration.
- End point: The end point URL for the API.
- Payload file: The payload file of the operation that is to be performed.

Plugin currently supports Query and Retrieve operations.

End point and Payload file are Mandatory inputs. On selecting the payload file, the template shows the

Mandatory attributes are loaded from the payload file residing in the plugin payload directory.

Inputs Database name and its UserId and Password are mandatory for any operation on the MediaPulse API.

Query operation is achieved by performing query on date_added column. The Minimum and Maximum values for date added with return array of all the MediaOrder that were added between that duration.

Retrieve operation is achieved by querying on the database with MediaOrder Id. The plugin code returns the array of all the source files associated to that MediaOrder.

1.6.2 Outputs description

- *Status*: The response status can be 'true' is the call is successful and 'false' if it fails.
- *Row_count*: If the API call is successful, row count returns the number of rows that are returned by the API.
- *Response_message*: Message is NULL for successful API calls. For failed calls, Message displays the message sent by API, which typically contains the error that caused the failure.
- *Response_code*: Response code of the API. E.g. 200 for success.
- *MO_list*: Shows array of MO in case of Query request OR array of File(s) in case of Retrieve request.

1.6.3 Operating Instructions

In order to perform the API calls, provide all the mandatory inputs.

Select the payload file that needs to be performed and click refresh link.

Provide all the mandatory attributes for the operation selected.

Response is sent back as XML document and is converted to a hash.This hash is stored in the plugin output.

Chapter 2

File Operations

2.1 Archive manager



Figure 2.1: Archive manager icon

This action plug-in can be used to manage a file archive, by deleting or compressing aging entries.

Version	Comment
0.4.1	Enhanced action template UI
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.0	2015-04-22 ADM - Added do not delete flag
0.2.5	2015-03-02 ML - New icon
0.2.4	Modified inputs spec to have manifest_dir accept dynamic parameters
0.2.3	Modified inputs spec to have archive_root and kept_duration accept dynamic parameters
0.2.2	Added help
0.2.1	Layout change for kept duration
0.2.0	Added option to use mtime instead of ctime
0.1.1	Fixed optional use of runtime archive root
0.1.0	Made optional the recursive search in the file pattern
0.0.3	Added support for non-zip compression format
0.0.2	Added management of the manifest files
0.0.1	Created based on Trim Archive custom ruby code

2.1.1 How It Works

- This action manages file purging or archiving from a local folder on the orchestrator server.
- The folder can be on a local disk or on a network-mounted storage.
- If no archive path is specified, files older than the specified age will be deleted.
- Note:This action is not a trigger, the archiving is performed only once. To make the archiving periodic, you should link an Archive manager step with a Scheduled Trigger step.

2.1.2 Configuration Instructions

1. This action manages file purging or archiving from a local folder on the orchestrator server.
2. Fill out the following fields:
    - Archive root: The base path for the files to archive. Supports GLOB syntax.
    - Purge file regexp pattern: The pattern used to select archive source files.
    - Pattern matching descends in sub-directories: Allows you to specify if the selection has to descend into subfolders.
    - Keep duration: Allows to specify after how long a file is moved to the archive.
    - Use mtime: If set, uses mtime instead of ctime to determine the keep duration.
    - Do not delete files: If set, the archiving result is computed but no action is taken on the file systems for the real files.
    - Make a compressed copy of the files (zip) before deleting: If set, moves all matching files to an archive.
    - Compressed copy archive name: The name to use for the archive file (if specified). Default is the name of the directory if the entity being compressed is a directory or archive otherwise. Note: '.zip' will be appended as a suffix.
    - Compression format: The format for the archive file (if selected). Optional - default is 'zip'
    - Purge manifest directory: Optional - If specified, manifest files of the deletions will be created in the provided directory. By default no manifest file is generated.
    - Manifest retention duration: Optional - If a manifest directory is specified, it allows to specify the manifest retention period. 0 for unlimited.

2.1.3 Example Configuration

This is a minimum setup configuration for the plugin.

Parameter	Value
Archive root	/home
Purge file regexp pattern	.*.log$
Pattern matching descends in sub-directories	Selected
Make a compressed copy of the files (zip) before deleting	Selected
Keep duration	5 seconds

2.1.4 Saved Parameters Description

Parameter	Description
Archive root	The base path for the files to archive. Supports GLOB syntax.
Purge file regexp pattern	The pattern used to select archive source files.
Pattern matching descends in sub-directories	Allows you to specify if the selection has to descend into subfolders.
Keep duration	Allows to specify after how long a file is moved to the archive.
Use mtime	If set, uses mtime instead of ctime to determine the keep duration.
Do not delete files	If set, the archiving result is computed but no action is taken on the file systems for the real files.
Make a compressed copy of the files (zip) before deleting	If set, moves all matching files to an archive.
Compressed copy archive name	The name to use for the archive file (if specified). Default is the name of the directory if the entity being compressed is a directory or archive otherwise. Note: '.zip' will be appended as a suffix.

Parameter	Description
Compression format	The format for the archive file (if selected). Optional - default is 'zip'
Purge manifest directory	Optional - If specified, manifest files of the deletions will be created in the provided directory. By default no manifest file is generated.
Manifest retention duration	Optional - If a manifest directory is specified, it allows to specify the manifest retention period. 0 for unlimited.

2.1.5 Inputs Description

Parameter	Description
Archive_root *	The base path for the files to archive. Supports GLOB syntax.
Compressed_name	The name to use for the archive file (if specified). Default is the name of the directory if the entity being compressed is a directory or archive otherwise.
Kept_duration *	Allows you to specify after how long a file is moved to the archive.
Kept_duration_unit	Defines the time unit for Kept duration, such as seconds, minutes, or hours.
Manifest_directory	Allows specification of the directory to place the manifest
Purge_file_pattern	Allows to specify if the selection has to descend into sub folders.

2.1.6 Outputs Description

Parameter	Description
Files_purged	The list of purged (archived) files.
Purge_manifest	The full path of the manifest file, if any.

2.1.7 Supported Actions

None

2.1.8 Dependencies

None

2.1.9 Operating Instructions

- Configure the Archive root path to specify the directory containing files to manage.
- Define the Purge file regexp pattern to match the files you want to archive or delete.
- Set the Keep duration to specify how long files should remain before archiving/deletion.
- Optionally enable 'Make a compressed copy' to create a zip archive before deletion, or leave it disabled to simply delete files.
- For periodic archiving, combine this action with a Scheduled Trigger in your workflow.
- Use 'Do not delete files' option to perform a dry run and verify which files would be affected without making actual changes.
- Ensure that the aspweb user has the necessary permissions to access and modify files in the specified directory.



Figure 2.2: Aspera Node API icon

2.2 Aspera Node API

This action plug-in provides the ability to perform file system operations such as search, rename, and delete. It also supports retrieving node information (e.g., token, SSH user, and port) and checking node availability through status queries (ping).

Version	Comment
0.2.1	Enhanced action template UI
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.3	Returns failure if rename operation is unsuccessful
0.1.2	Support the form hostname:port in node address
0.1.1	fix search criteria analysis
0.1.0	fixed reading mapped inputs for server address
0.0.9	Added styling to help text.
0.0.8	RemoteNode Dependency
0.0.7	Uses double-entered password
0.0.6	Adding help file. Hiding polling frequency field from the view, as it is not used in the plugin yet.
0.0.5	Defaults port to 9092 is use_SSL is checked
0.0.4	Fixes delete operation
0.0.3	exception handling of RemoteNode.find_by_name()
0.0.2	Moved to file operation category
0.0.1	Initial release. Supports ping, info, search, rename, delete

2.2.1 How It Works

- Connects to a remote Aspera Node using the Aspera Node API over HTTPS (port 9092) or HTTP (port 9091, if enabled).
- Supports operations including Ping, Info, Search, Rename, and Delete on the remote node.
- Authenticates with a configured Node API username and password.
- Allows advanced search and filtering options for file operations.

2.2.2 Configuration Instructions

1. Create a new action step using the AsperaNodeApi plugin.
2. Fill out these fields with appropriate values:
   - Name: A label for the action step.
   - Comments: Optional notes or description.
   - Aspera node: Select a remote node or enter IP/domain manually.
   - Aspera node IP address or hostname: Remote node IP or domain name.
   - Node api port: Port for Node API (default 9092 for HTTPS).
   - Node user name: Node API username.
   - Node password: Password for Node API access.
   - Use ssl?: Check to use HTTPS (default 9092), otherwise HTTP on port 9091 if enabled.
   - Operation: Choose from Ping, Info, Search, Rename, Delete.
   - Source: For Search - a folder path; for Rename - one or more comma-separated file paths.
   - Target: For Rename - target filename or regex pattern.

- Overwrite during rename: Enable to overwrite existing files during rename.
- Search Parameter - Filter file type: Enter one or more of 'file', 'directory', 'symbolic link' as comma-separated values. If provided, the result set will include only those file types.
- Search Parameter - Filter basename: Enter one or more values comma-separated. If provided, the result set will include only files that match the file base name.
- Search Parameter - Max count: Default is 1000000. Total number of files to return after a search operation.
- Search Parameter - Minimum size: Enter an integer value (bytes) to search for files whose size is more than the value provided.
- Search Parameter - Filter mtime min: Filter for files modified after a specific time, in the format '2011-01-31T12:59:59Z' (T or space in the middle; Z optional at the end).
- Search Parameter - Filter mtime max: Filter for files modified before a specific time, in the format '2011-01-31T12:59:59Z' (T or space in the middle; Z optional at the end).

2.2.3 Example Configuration

Field	Value
Name	`New AsperaNodeApi Step`
Comments	Retrieve info from remote Aspera node
Aspera node	`-Select Node-`
Aspera node IP address	`aspera-node.example.com`
Node api port	`9092`
Node user name	`node_watchfolder`
Node password	`<password>`
Operation	`Info`
Use ssl?	Checked
Source	(e.g., `/tmp/` for Search, or comma-separated paths for Rename)

2.2.4 Saved Parameters Description

Parameter	Description
Name	Label identifying this action step.
Comments	Optional descriptive notes.
Aspera node	Remote node selected or specified.
Aspera node IP address	IP or domain for the remote node.
Node api port	Port for Node API; 9092 default for HTTPS.
Node user name	Node API username, distinct from OS user.
Node password	Password for Node API user access.
Use ssl?	Enables HTTPS on the Node API endpoint.
Operation	API operation to perform (Ping, Info, etc.).
Source	Path(s) for Search or Rename operations.
Target	Rename destination or regex pattern.
Overwrite during rename	Overwrite files on rename if checked.
Search Parameters	Filters limiting search results by type, size, and date.

2.2.5 Inputs Description

- Aspera_Node_Server_Address: Remote node name, IP address, domain, or address:port.

- node_user_name: Node API user name.
- node_password: Node API password.
- filter_basename: Filters files matching these base names.
- filter_file_type: Filters by file type (file, directory, symbolic link).
- filter_mtime_min: Minimum file modification date/time.
- filter_mtime_max: Maximum file modification date/time.
- max_count: Max number of files to retrieve.
- use_ssl: Use HTTPS (default 9092) if checked, otherwise HTTP on port 9091.

2.2.6 Outputs Description

Output	Description
NODE_API_RESULTS	Results as a hash/map: info, search results, rename/delete status.
Step_information	Execution metadata including operation, completion status, and timing.

2.2.7 Supported Actions

Ping, Info, Search, Rename, and Delete.

2.2.8 Dependencies

Remote nodes must run Aspera High Speed Transfer Server or Endpoint (Linux/Windows) with the asperanoded service and valid license.

2.2.9 Node API Usage Overview

This action can:

1. Ping: Check if the remote node responds to Node API requests.
2. Info: Retrieve primary information about the remote node.
3. Search: Detect files or folders matching patterns under watch directories. Supports globbing and multiple patterns (comma-separated).
4. Rename: Rename source files to specified targets or using regex patterns.
5. Delete: Delete specified files.

Globbing pattern examples:

- `*.txt`: Matches all files with a .txt extension.
- `foo*/\*.txt`: Matches files under directories starting with ”foo”.
- `foo/*/bar/*.txt`: Matches files under foo/bar subdirectories.
- `fo?`: Matches ”foo” or ”fo1”, but not ”foo1”.
- `fo[1-9]`: Matches ”fo1”, ”fo2”, etc., but not ”foo1”.

Note: All paths are relative to the transfer user’s docroot defined in `/opt/aspera/etc/aspera.conf` on the remote server.

Example excerpt from `aspera.conf`:

\ <user>\ <name>xfer</name>\ <file_system>\ <access>\ <paths>\ <path>\ <absolute>/home/xfer/data</absolute>\ </path>\ </paths>\ </access>\ </file_system>\ </user>\

To enable both HTTP (port 9091) and HTTPS (port 9092), ensure `aspera.conf` contains:

\ <server>\ <server_name>example.aspera.com</server_name>\ <http_port>9091</http_port>\ <https_port>9092</https
<enable_http>true</enable_http>\ <enable_https>true</enable_https>\ </server>\

Restart the `asperanoded` service as root after changes:

\ service asperanoded restart\

For more detailed API documentation, see the Aspera Developer Network.

2.3 Path conversion



Figure 2.3: Path conversion icon

Converts an absolute file path from one mount point to another based on configured input and output mappings. Supports one-to-one and multi-mount point conversions.

Version	Comment
0.2.1	Enhanced action template UI
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.2	2015-02-17 ML - New icon
0.1.1	Adds file path breakdown in outputs, changed error status to error
0.1.0	Adds option to run on an array of file path
0.0.4	Fixed problem with multiple network paths
0.0.3	Adds display name
0.0.2	support for multiple mount points
0.0.1	initial release

2.3.1 How It Works

- This plugin converts an absolute file path from one file system to another.
- If the same file is exposed via two different mount points, the plugin infers the corresponding path from the second mount point.
- If no match is found in the input mount points, the original path is returned.

2.3.2 Configuration Instructions

1. Create a new action step using the ConvertPath plugin.
2. Fill out the following fields:
    - Name: The label to identify this ConvertPath step.
    - input_mount_point *: The absolute path of the mount point to be matched for substitution.
    - output_mount_point *: The absolute path of the target mount point used in the substitution.
    - file_path *: The absolute file path to convert.

2.3.3 Example Configuration

Example 1: Two hosts mount the same shared storage using different names:

- Host1 mounts it as `/mnt/external`
- Host2 mounts it as `d:\`

If a file `file1.dat` exists at `/mnt/external/file1.dat` on Host1, set:

- input_mount_point: `/mnt/external`
- output_mount_point: `d:\`

The plugin converts the path to: `d:\file1.dat`

Example 2: Hosts use multiple mount mappings:

- Host1: `/mnt/external` (storage1), `/mnt/ext_storage` (storage2)
- Host2: `/ext/storage1` (storage1), `/ext/storage2` (storage2)

Configure:

- input_mount_point: `["/mnt/external", "/mnt/ext_storage"]`
- output_mount_point: `["/ext/storage1", "/ext/storage2"]`

If file path is `/mnt/ext_storage/file1.dat`, output will be `/ext/storage2/file1.dat`

If no input mount point matches, the output is the original `file_path`.

2.3.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved ConvertPath configured instance.
Comments	Some comments about this saved ConvertPath configured instance.
input_mount_poi	The absolute path of the mount point to be matched for the substitution.
output_mount_poi	The absolute path of the target mount point to use in the substitution.
file_path	The absolute file path to convert.

2.3.5 Inputs Description

Inputs depend on the ConvertPath configuration. All values are mandatory and can be provided in the template or at runtime.

2.3.6 Outputs Description

Output	Description
output_file_path	The converted absolute file path.

2.3.7 Supported Actions

None

2.3.8 Dependencies

None

2.3.9 Operating Instructions

This plugin is useful when the same storage is mounted in a different way across hosts. It helps to infer the correct path on the target system based on mappings.

- Use it when transferring files between systems with different mount point names.
- Simplifies cross-host file references in shared storage setups.

2.4 Dependency checker

The DependencyChecker plug-in monitors the presence of specific files in a defined location and allows workflows to pause until those files are detected. This ensures that dependent data is available before proceeding to the next step.



Figure 2.4: Dependency checker icon

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.6	Allow to cancel step
0.0.5	Fixes stability period
0.0.4	Added Pending files output
0.0.3	Fixes problem with stability detection when file is not available at first check
0.0.2	fix UI labels, add widtget for list of files, fix no-stability check option
0.0.1	Initial implementation

2.4.1 How It Works

- Checks for the presence of one or more files.
- If all specified files are found and stable, the step completes successfully.
- If not found within the specified timeout period, the step fails.
- If timeout is `0`, it waits indefinitely.

2.4.2 Configuration Instructions

1. Create a new action step using the DependencyChecker plugin.
2. Fill out the following fields:
   - Name: A label for the step (e.g., `Wait for Files`).
   - Dependency Paths *: Enter one file name per field. Use the Add Dependency Paths button to add more. (e.g., `test_dependency.txt`, `test_dependency2.txt`)
   - Polling Frequency: How often (in seconds) to check for files (e.g., `5`).
   - Stability Period: Time (in seconds) files must remain unchanged (e.g., `10`).
   - Base Path (optional): Directory path prefix for files (e.g., `/tmp`).
   - Detect Partial Files: If checked, considers a file found as soon as it appears.
   - Timeout: Total time (in seconds) to wait before failing (e.g., `60`). Use `0` to wait forever.
3. Click Update to save the step.

2.4.3 Example Configuration

Field	Value
Base Path	`/tmp` (if any)
Dependency Paths	`test_dependency2.txt`
Polling Frequency	`10`(default)
Stability Period	`10`(default)
Timeout	`0` (waits indefinitely)
Detect Partial Files	Unchecked

Behavior: The plugin will monitor `/tmp/test_dependency2.txt` and complete the step once it appears and remains stable for 10 seconds.

2.4.4 Saved Parameters Description

Parameter	Description
Name	Label to identify this DependencyChecker step
Dependency Paths	List of file names to monitor (comma-separated)
Polling Frequency	Time interval (in seconds) to check for files
Stability Period	Duration a file must remain unchanged to be valid
Base Path	Optional base directory path for dependency files
Detect Partial Files	Checks file availability immediately if enabled
Timeout	Maximum wait time before failure (0 = wait forever)

2.4.5 Inputs Description

Inputs are derived from the step configuration. Most important input:

- dependency_file_paths: Full file paths being monitored (e.g., `/tmp/test_dependency2.txt`)

2.4.6 Outputs Description

Output	Description
Dependency file paths	Array of successfully found files

2.4.7 Supported Actions

None

2.4.8 Dependencies

None

2.4.9 Operating Instructions

This plugin is ideal for workflows that must pause and wait until required input files are available. Common scenarios include:

- Waiting for files uploaded by external systems
- Triggering jobs only after prior steps generate required output files

2.4.10 Troubleshooting

- Ensure the file names and base path are correct.
- Verify that files appear in the expected location.
- Adjust stability period if files change too frequently.
- Set an appropriate timeout value to avoid indefinite waiting.



Figure 2.5: Diva archive icon

2.5 Diva archive

This action plug-in provides the ability to archive or restore files to/from the DIVArchive system. It implements the DIVA REST WEB services API 1.0, 2.1 and 2.2

Version	Comment
0.4.2	Enhanced action template UI
0.4.1	Improve error handling and code maintainability
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.2	Adding support for dynamic parameters in Object Category.
0.3.1	Set default priority to -1
0.3.0	Add support for basic auth
0.2.1	Add support for Diva version 2.2
0.2.0	Add Diva version
0.1.0	Add hash output
0.0.9	Add support for getObjectInfo operation
0.0.8	Run time support for polling frequency.
0.0.7	Added styling to parameter help text.
0.0.6	Adding Remote Node dependency
0.0.5	Runtime parameter option to server_address and port
0.0.4	2015-04-29 CT - Allow remote node in server address input
0.0.3	2015-02-17 ML - New icon
0.0.2	added handling of avid mobid
0.0.1	initial release

2.5.1 How It Works

- Provides integration with DIVArchive media asset management system via REST/SOAP web services.
- Supports five operations: archiveObject, restoreObject, partialRestore, deleteObject, and getObjectInfo.
- Supports DIVArchive API versions 1.0, 2.1, and 2.2 with version-specific authentication.
- Communicates via HTTP REST/SOAP on default port 9763.
- Registers a client session before each operation to obtain a session code.
- Monitors operation progress asynchronously via polling with configurable frequency.
- Extracts AVID mob IDs from response additional info when present.
- Version 2.2 requires HTTP Basic authentication for all operations.
- Returns XML, hash, status messages, and AVID mob ID as outputs.

2.5.2 Configuration Instructions

1. Create a new action step using the DivaArchive plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved DivaArchive configured instance.
   - Comments: Comments about this DivaArchive configured instance.
   - Diva Node: The Diva node name from configured Remote Nodes. Use this OR Diva server IP address.
   - Diva server IP address: The Diva node IP address or domain name. Use this OR Diva Node.

- Diva WEB service port: Override the default Diva WEB service port (default: 9763).
- Basic auth login: HTTP Basic authentication login. Required for API version 2.2 only.
- Basic auth password: HTTP Basic authentication password. Required for API version 2.2 only.
- Diva version: The DIVArchive WEB services API version: 1.0 (DIVArchive 6.5), 2.1 or 2.2 (Upto DIVArchive 8.2).
- Operation: The operation to execute: archiveObject, restoreObject, partialRestore, deleteObject, or getObjectInfo.
- Object name: Name of the object in DIVArchive.
- Object category: Category of the object. Required for archive and partialRestore. Optional for restore, delete, and getObjectInfo.
- Source/Dest: Source for archive or destination for restore (e.g. disk, video server). Must be defined in DIVA configuration.
- Media name: Group of Tapes or Array of Disks defined in DIVA where the instance is created (e.g. STORAGE-ON-DISK). Archive operation only.
- File path root: Root folder for files to archive/restore. Required for archive, optional for restore.
- Filename list: Comma-separated list of filenames to archive. Archive operation only.
- Quality of service: Quality of service value (0-6). Controls restore method (cache, direct, nearline). See QoS table.
- Priority: Priority level for the operation (0-100, 0 = lowest, 100 = highest, default -1).
- Options: Additional options for archive/restore operations.
- Polling frequency: Interval in seconds to poll for job completion (default: 10).
- Instance ID: (partialRestore only) ID of a non-spanned Tape Instance, or -1 for DIVA_ANY_INSTANCE.
- Format: (partialRestore only) Video format value (0-13). See Format table.
- Parameters: (partialRestore only) List of source/destination file names and offset data.

2.5.3 Example Configuration

This is an example configuration for the DivaArchive plugin archiving a media object using API version 2.2.

Parameter	Value
Name	Archive Media Object
Diva server IP address	diva.example.com
Diva WEB service port	9763
Diva version	2.2
Basic auth login	orchestrator
Basic auth password	********
Operation	archiveObject
Object name	my_media_object
Object category	VIDEO
Source/Dest	DISK_SOURCE
Media name	STORAGE-ON-DISK
File path root	/media/incoming
Filename list	video.mxf, audio.wav
Priority	50
Polling frequency	10

2.5.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved DivaArchive configured instance.
Comments	Comments about this DivaArchive configured instance.
Diva Node	The Diva node name from configured Remote Nodes. Use this OR Diva server IP address.
Diva server IP address	The Diva node IP address or domain name. Use this OR Diva Node.
Diva WEB service port	Override the default Diva WEB service port (default: 9763).
Basic auth login	HTTP Basic authentication login. Required for API version 2.2 only.
Basic auth password	HTTP Basic authentication password. Required for API version 2.2 only.

Parameter	Description
Diva version	The DIVArchive WEB services API version: 1.0 (DIVArchive 6.5), 2.1 or 2.2 (Upto DIVArchive 8.2).
Operation	The operation to execute: archiveObject, restoreObject, partialRestore, deleteObject, or getObjectInfo.
Object name	Name of the object in DIVArchive.
Object category	Category of the object. Required for archive and partialRestore. Optional for restore, delete, and getObjectInfo.
Source/Dest	Source for archive or destination for restore (e.g. disk, video server). Must be defined in DIVA configuration.
Media name	Group of Tapes or Array of Disks defined in DIVA where the instance is created (e.g. STORAGE-ON-DISK). Archive operation only.
File path root	Root folder for files to archive/restore. Required for archive, optional for restore.
Filename list	Comma-separated list of filenames to archive. Archive operation only.
Quality of service	Quality of service value (0-6). Controls restore method (cache, direct, nearline). See QoS table.
Priority	Priority level for the operation (0-100, 0 = lowest, 100 = highest, default -1).
Options	Additional options for archive/restore operations.
Polling frequency	Interval in seconds to poll for job completion (default: 10).
Instance ID	(partialRestore only) ID of a non-spanned Tape Instance, or -1 for DIVA_ANY_INSTANCE.
Format	(partialRestore only) Video format value (0-13). See Format table.
Parameters	(partialRestore only) List of source/destination file names and offset data.

2.5.5 Inputs Description

Changes in this list of inputs will override any values set in the action template.

Parameter	Description
server address *	Remote node name or remote server IP address or domain name. Required if Diva Node not configured.
object name *	Name of the object to archive/restore/delete.
Basic auth login	HTTP Basic authentication login. Required for API version 2.2 only.
Basic auth password	HTTP Basic authentication password. Required for API version 2.2 only.
object category	Category of the object. Optional for restore, delete and getObjectInfo operations.
source / dest	Source for archive or destination for restore. Must be defined in DIVA configuration.
media name	Group of Tapes or Array of Disks in DIVA where instance is created. Archive operation only.
filepath root	Root folder for files to archive/restore. Required for archive, optional for restore.
filename list	Comma-separated list of filenames to archive. Example: `JTM-140803.mxf, 'IMI_SONY test.MXF'`. Archive operation only.
format	Video format value (0-13). Partial restore operation only.
partial restore parameters	Array of source/destination file and offset data. Partial restore only. Use double-quotes. Example: `[["file.MXF", "_file.MXF", "", "", "00:00:00:00", "99:99:99:99", "1"]]`
polling_frequency	Polling interval in seconds (default: 10).

2.5.6 Outputs Description

Parameter	Description
XML_result	The DIVA output in XML format
HASH_result	The DIVA output in Ruby hash format
Diva_status_message	The DIVA status message (e.g. DIVA_OK, DIVA_ERR_OBJECT_DOESNT_EXIST)
Diva_request_status_message	The DIVA request status message from getRequestInfo (e.g. DIVA_COMPLETED, DIVA_ABORTED)

Parameter	Description
Avid mob id	AVID MOB ID if returned by getRequestInfo within the additionalInfo section

2.5.7 Supported Actions

None

2.5.8 Dependencies

None

2.5.9 Operating Instructions

This plugin integrates with DIVArchive via REST/SOAP web services to perform media asset archive, restore, partial restore, delete, and object info operations.

API Version Differences

- Version 1.0 - DIVArchive 6.5:
  - SOAP/XML envelope format
  - No HTTP Basic authentication required
  - API path: `services/DIVArchiveWS_REST_1.0`
- Version 2.1 - Upto DIVArchive 8.2:
  - SOAP/XML envelope format
  - No HTTP Basic authentication required
  - API path: `services/DIVArchiveWS_REST_2.1`
- Version 2.2 - Upto DIVArchive 8.2:
  - REST/XML format (no SOAP envelope)
  - HTTP Basic authentication required for all operations
  - Provide 'Basic auth login' and 'Basic auth password'
  - API path: `diva/service/rest/2.2/DIVArchiveWS_REST`

Supported Operations

- archiveObject:
  - Archives files from source media into DIVArchive
  - Required inputs: object name, object category, source/dest, media name, filepath root, filename list
  - Filename list: comma-separated string (e.g. `file1.mxf, file2.mxf`)
  - Monitors request state via polling until DIVA_COMPLETED
- restoreObject:
  - Restores a full object from DIVArchive to destination media
  - Required inputs: object name, source/dest
  - Optional inputs: object category, filepath root
  - Quality of service controls restore method (direct, cache, nearline)
- partialRestore:
  - Restores specific files or time ranges from a DIVArchive object
  - Required inputs: object name, object category, source/dest, parameters, format
  - Parameters define per-file source, destination, byte/timecode offsets
  - Parameters format: `[[source, dest, byte_begin, byte_end, tc_begin, tc_end, offset_type]]`
  - Use double-quotes in parameters, not single-quotes
- deleteObject:
  - Removes an object from DIVArchive
  - Required inputs: object name
  - Optional inputs: object category
  - Monitors request state via polling until DIVA_COMPLETED
- getObjectInfo:
  - Retrieves metadata and status information for an object
  - Required inputs: object name
  - Optional inputs: object category

- Synchronous - does not poll (returns result directly)

Quality of Service (QoS) Values

Value	Constant	Description
0	DIVA_QOS_DEFAULT	Default QoS (direct and cache for restore)
1	DIVA_QOS_CACHE_ONLY	Use Cache Restore only
2	DIVA_QOS_DIRECT_ONLY	Use Direct Restore only
3	DIVA_QOS_DI-RECT_AND_CACHE	Direct Restore if available, else Cache Restore
4	DIVA_QOS_CACHE_AND_DI-RECT	Cache Restore if available, else Direct Restore
5	DIVA_QOS_NEARLINE_ONLY	Use Nearline only
6	DIVA_QOS_NEARLINE_AND_DI-RECT	Nearline if available, else Direct

Video Format Values (partialRestore)

Value	Constant	Description
0	DIVA_FORMAT_BYTES	Raw byte partial restore
1	DIVA_FORMAT_BYTES_HEADER	Deprecated feature
2	DIVA_FORMAT_VIDEO_GXF	GXF video format (PDR video servers)
3	DIVA_FORMAT_VIDEO_SEA	Seachange video format
4	DIVA_FORMAT_VIDEO_AVI_MATROX	Matrox-specific AVI format + WAV files
5	DIVA_FORMAT_VIDEO_MPEG2_TS	MPEG2 Transport Stream
6	DIVA_FORMAT_VIDEO_MXF	MXF video format
7	DIVA_FORMAT_VIDEO_PINNACLE	Pinnacle video format
8	DIVA_FORMAT_VIDEO_OMNEON	Omneon video format
9	DIVA_FORMAT_VIDEO_LEITCH	Leitch video format
10	DIVA_FORMAT_VIDEO_QUANTEL	Quantel QCP video format
11	DIVA_FORMAT_AUTODETECT	Automatic detection of input format
12	DIVA_FORMAT_FOLDER_BASED	Fully restore specified folders
13	DIVA_FORMAT_DPX	DPX video format

DIVA Status Codes

Code	Constant	Description
1000	DIVA_OK	Success
1001	DIVA_ERR_UNKNOWN	Unknown error
1002	DIVA_ERR_INTERNAL	Internal error
1004	DIVA_ERR_BROKEN_CONNECTION	Connection broken
1008	DIVA_ERR_INVALID_PARAMETER	Invalid parameter
1009	DIVA_ERR_OBJECT_DOESNT_EXIST	Object does not exist
1010	DIVA_ERR_SEVERAL_OBJECTS	Multiple matching objects
1016	DIVA_ERR_OBJECT_ALREADY_EXISTS	Object already exists
1018	DIVA_ERR_SOURCE_OR_DESTINATION_DOESN'T_IST	Source or destination not found
1023	DIVA_ERR_OBJECT_OFFLINE	Object is offline
1024	DIVA_ERR_TIMEOUT	Request timed out
1031	DIVA_ERR_OBJECT_IN_USE	Object currently in use
1032	DIVA_ERR_CANNOT_ACCEPT_MORE_REQUESTS	Server at capacity
1039	DIVA_ERR_OBJECT_IS_LOCKED	Object is locked

Abort Reason Codes (when Diva_request_status_message = DIVA_ABORTED)

Code	Constant	Description
0	DIVA_AR_NONE	Request not aborted
1	DIVA_AR_DRIVE	Drive troubles
2	DIVA_AR_TAPE	Tape troubles
3	DIVA_AR_ACTOR	Actor troubles
4	DIVA_AR_DISK	Disk troubles
5	DIVA_AR_DISK_FULL	Disk is full
6	DIVA_AR_SOURCE_DEST	Source/destination troubles
7	DIVA_AR_RESOURCES	Resources attribution troubles
8	DIVA_AR_LIBRARY	Library troubles
9	DIVA_AR_PARAMETERS	Wrong request parameters
10	DIVA_AR_UNKNOWN	Unknown code
11	DIVA_AR_INTERNAL	Internal DIVArchive WS error

Partial Restore Parameters

- The parameters field defines a list of files to partially restore, each with source, destination, and offset information.
- Parameter format per file: `[source_file, dest_file, byte_begin, byte_end, tc_begin, tc_end, offset_type]`
- Field descriptions:
  - `source_file`: Filename as stored in DIVArchive
  - `dest_file`: Output filename at destination
  - `byte_begin / byte_end`: Byte range (leave empty for timecode mode)
  - `tc_begin / tc_end`: Timecode range (format: 00:00:00:00 to 99:99:99:99)
  - `offset_type`: 0 = OFFSET_FORMAT_BYTE, 1 = OFFSET_FORMAT_TIMECODE
- Single file example:

```
[["video.MXF", "_video.MXF", "", "", "00:00:00:00", "99:99:99:99", "1"]]
```

- Multi-file example:

```
[["file1.mxf", "_file1.mxf", "", "", "00:01:00:00", "00:02:00:00", "1"], ["file2.mxf", "_file2.mxf", "", "", "00:00:00:00", "99:99:99:99", "1"]]
```

- Important: Use double-quotes, not single-quotes, in parameters.

Common Use Cases

- Media archiving: Archive ingested video and audio files from disk into DIVArchive storage.
- Media restoration: Restore archived content to production servers for editing or broadcast.
- Clip extraction: Use partialRestore to retrieve specific timecode ranges from archived objects.
- Asset management: Query object metadata via getObjectInfo in automated workflows.
- Storage cleanup: Delete obsolete objects from DIVArchive after content expiry.
- AVID integration: Retrieve MOB IDs from restored content for AVID Media Composer workflows.
- Nearline retrieval: Use QoS settings to control restoration from nearline tape storage.

Best Practices

- Select the correct API version matching your DIVArchive installation (1.0, 2.1, or 2.2).
- For version 2.2, always provide Basic auth login and password.
- Use Remote Node configuration for server credentials and reusability.
- Set appropriate polling frequency - large archive/restore jobs may take minutes (30-60s recommended).
- Use priority levels to manage concurrent operations (0-100 range).
- For partialRestore, use double-quotes in parameters, never single-quotes.
- Use getObjectInfo to verify object existence before archive or restore operations.
- Monitor Diva_request_status_message output for detailed operation state.
- Check DIVA status codes in workflow error handling for precise failure diagnosis.
- For high-priority workflows, use QoS direct-only to avoid tape nearline latency.

Troubleshooting

- Connection refused: Verify Diva server address and port 9763 is accessible from Orchestrator.
- Authentication failed (v2.2): Verify Basic auth login and password are correct.
- DIVA_ERR_OBJECT_DOESNT_EXIST (1009): Object name doesn't exist in DIVArchive. Verify exact name.
- DIVA_ERR_OBJECT_ALREADY_EXISTS (1016): Object already archived. Use deleteObject first or use a different name.
- DIVA_ERR_OBJECT_IN_USE (1031): Another operation is already running on this object. Wait and retry.
- DIVA_ERR_CANNOT_ACCEPT_MORE_REQUESTS (1032): DIVA server at capacity. Reduce concurrent requests.
- DIVA_ABORTED: Check abort reason code in response. Common causes: disk full, tape errors, wrong parameters.
- registerClient returns no session code: DIVA server connectivity issue. Check server status and network.
- partialRestore parameter errors: Verify double-quotes used in parameters array. Check source file names match DIVArchive exactly.
- Source/Dest not found (1018): Verify the source/destination name is defined in DIVA configuration.

Important Notes

- Use Diva Node OR Diva server IP address - not both.
- API version 2.2 requires HTTP Basic authentication. Versions 1.0 and 2.1 do not.
- For delete and getObjectInfo, only the object name is required.
- The filename list is comma-separated. Filenames with spaces can be wrapped in single or double quotes.
- partialRestore parameters must use double-quotes, not single-quotes.
- Default port is 9763. Only override if DIVA is configured on a different port.
- Default polling frequency is 10 seconds.
- Default priority is -1 (system default).
- getObjectInfo is synchronous and does not poll.
- All other operations are asynchronous and poll via getRequestInfo until complete.
- The AVID mob ID is parsed from the additionalInfo section of getRequestInfo when present.

2.6 Exiftool xmp tagger



Figure 2.6: Exiftool xmp tagger icon

This action plug-in can be used to retrieve metadate information about an image, video or audio file using the exiftool application. Also supported is the ability to insert metadata tag in image file (but not videos).

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.4	2016-04-11 JG Adding dependency for execution node
0.0.3	2015-02-16 ML - New icon
0.0.2	2014-12-15 ML - Disable auto-complete and accept tags as hash, make options optional
0.0.1	initial release

2.6.1 Saved Parameters Description

- Name: The name used to identify a saved ExiftoolXmpTagger configured instance.
- Comments: Some comments about this saved ExiftoolXmpTagger configured instance.
- Media file path: The source file for the analysis operation.
- Tagging operation: The plugins can execute either to read or to write tags. In the case of write, 2 different operations are supported. One modifies the source file by setting the metadata as instructure. The other leaves the source file untouched, but create a copy of it in whihc the metadata is set as instructed.

- Tagged media file path: (only in mode Insert and copy) The full path to the output file consisting of a copy of the source file with metadata tags set as instructed.
- Tags to insert: (not in mode Read) The set of tags to be set. For each tag, a pair of field is provided, the first identifies the tag, the second is the value to set that tag to. Dynamic attributes are supported, so a value can be an input variable (or a combination).
- Tags to read: (only in mode Read) The set of tags to extract as named, singled-out outputs.
- Execution Node: The node where the analysis operation must be executed (i.e. the node where the ExifTool application is installed).
- Execution Login: The user account used to execute the ExifTool analysis operation. If not specified the default user accound for the node will be used.
- Execution Password: The password used to log in into the remote node. If not specified the default password for the node will be used.
- Path to the mediainfo binary: The path to locate the exiftool binary in the remote node. Not needed if the binary folder for ExifTool is in the sytem path for the account use to remote connect on the specified server.
- Any optional parameters to pass to the ExifTool process: Any optional parameters supported by the exiftool binary. Those options will be passed on the command line as they are.

2.6.2 Inputs description

The list of inputs depends on the configuration of the ExiftoolXmpTagger action template.

- *Source file path*: All the variables listed in Source file path field are mandatory and must be of type string.
- *Options*: All the variables listed in the optional parameters to pass to the exiftool process are mandatory and must be of type string.
- *Dynamic variables*: All the variables listed in the 'tags to insert' or 'tags to read sections'

2.6.3 Outputs description

- *Tagged_media_file*: The full path to the file being tagged. Points to the copied file in the case of Insert and copy and to the source file otherwise.
- *Metadata*: (only in mode Read) A hash containing all the metadata tags found for that file with their values. For example: { 'Epidose Title' => 'Morning light', 'Duration' => '2435s', 'Season' => '3' }
- *Tags read*: (only in mode Read) The specified singled-out tags, with their value returned as a string

2.6.4 Supported Actions

None

2.6.5 Dependencies

None

2.6.6 Operating Instructions

This action executes the specified metadata tagging operation on a remote node where ExifTool is available. The connection to the remote server is done over SSH, so make sure that the target server has SSH enabled to it. If the server is a Windows machine, OpenSSH can be installed.

It is typically a good idea to provide the explicit path to the exiftool binary to avoid configuration issues. On a Unix-flavored machine, running 'which exiftool' should provide the path information.

Important Note: this action plugins does not do any file movement. Hence, for it to run a file analysis, the specified file has to be accessible. Either the file is transferred to that server or it must be accessible from that server either on some type of network share.

Testing has been done against an install of ExifTool v9.08 and it is to be noted that tag insertion is only supported for images and documents at this point. For video files, only read is supported. Also, the insertion of tags only works for standard tags or for tags that are declared in the .ExifTool_config file, which is located in the executing user home directory. If that file does not exist, one can be created using the sample file provided with the plugin.

2.7 FFProbe Information



Figure 2.7: FFProbe Information icon

This action plug-in provides the ability to submit a file to FFProbe to grab media properties and technical Metadata

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.3	Correcting the plugin_name in metadata.yml
0.0.2	-> Adding Remote Node Dependency
0.0.1	-> Initial release

2.7.1 How It Works

- Executes FFProbe (FFmpeg's multimedia stream analysis tool) on a remote node via SSH.
- Analyzes audio and video files to extract metadata, codec information, stream details, and technical properties.
- Connects to a configured remote node via SSH to execute FFProbe commands.
- Generates a detailed report file with FFProbe output on the remote node.
- Returns both the report file path and report content as outputs.
- Supports custom FFProbe command options for flexible analysis configuration.
- Requires FFProbe to be installed on the remote node.

2.7.2 Configuration Instructions

1. This action allows to execute an FFProbe task on a remote node.
2. Create a new action step using the FfprobeInfo plugin.
3. Fill out the following fields:
   - Name: The name used to identify a saved FFProbe Info configured instance.
   - Comments: Comments associated to this template or configuration.
   - Execution node: The remote node to execute the FFProbe operation on. Select from the list of already configured Remote Nodes.
   - FFProbe Binary path: The path of the ffprobe executable on the remote node (e.g., /usr/bin/ffprobe or /opt/ffmpeg/bin/ffprobe).
   - Input file path: The path of the input file to analyze with FFProbe on the remote node.
   - Report file path: The path on the remote node where the report file with the FFProbe output will be saved.
   - Command options: Any FFProbe options or parameters. This string will be added to the FFProbe command BEFORE the input file path (e.g., -show_format -show_streams -print_format json).

2.7.3 Example Configuration

This is an example configuration for the FfprobeInfo plugin to analyze a video file and output JSON format.

Parameter	Value
Name	Analyze Video File

Parameter	Value
Comments	Extract metadata from video files
Execution node	MediaServer01
FFProbe Binary path	/usr/bin/ffprobe
Input file path	/media/incoming/video.mp4
Report file path	/media/reports/video_info.json
Command options	-show_format -show_streams -print_format json

2.7.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved FFProbe Info configured instance.
Comments	Comments associated to this template or configuration.
Execution node	The remote node to execute the FFProbe operation on. Select from the list of already configured Remote Nodes.
FFProbe Binary path	The path of the ffprobe executable on the remote node (e.g., /usr/bin/ffprobe or /opt/ffmpeg/bin/ffprobe).
Input file path	The path of the input file to analyze with FFProbe on the remote node.
Report file path	The path on the remote node where the report file with the FFProbe output will be saved.
Command options	Any FFProbe options or parameters. This string will be added to the FFProbe command BEFORE the input file path (e.g., -show_format -show_streams -print_format json).

2.7.5 Inputs Description

The run time inputs of this action depend on how the configuration is done. Variables exposed in the command line will be exposed as mandatory inputs.

Parameter	Description
Execution_node	The remote node identifier (optional if configured in template). Must be a configured Remote Node with SSH access.
FFProbe_Binary_path	The full path to the ffprobe executable on the remote node (optional if configured in template).
Input_file_path	The full path to the input media file on the remote node (optional if configured in template).
Report_file_path	The path where the FFProbe report will be saved on the remote node (optional if configured in template).
Command_options	FFProbe command line options and parameters (optional if configured in template).
Dynamic variables	Any variables used in the command options will be exposed as mandatory runtime inputs.

2.7.6 Outputs Description

This action executes an FFProbe analysis action on a remote node, via SSH.

Parameter	Description
Report_File_Path	The path of the output file on local Orchestrator server, with the output of FFProbe command
Report	A report containing the complete output of the FFProbe command execution

2.7.7 Supported Actions

None

2.7.8 Dependencies

FFProbe needs to be installed on the remote node. FFProbe is part of the FFmpeg project and can be installed via package managers or compiled from source.

2.7.9 Operating Instructions

This action plug-in provides the ability to submit a command to FFProbe tool. FFProbe command will be executed on a remote node via an SSH connection.

Command Line Construction

- The FFProbe command line is constructed in the following order:
- `<FFProbe Binary path> <Command options> <Input file path>`
- Example:

  `/usr/bin/ffprobe -show_format -show_streams -print_format json /media/input/video.mp4`
- The command is executed via SSH on the remote node specified in the configuration.

Common FFProbe Options

- Output Format Options:
  - `-print_format json` - Output in JSON format (recommended for parsing)
  - `-print_format xml` - Output in XML format
  - `-print_format csv` - Output in CSV format
  - `-print_format flat` - Output in flat key=value format
- Information Display Options:
  - `-show_format` - Show container format information (duration, bitrate, size)
  - `-show_streams` - Show stream information (codec, resolution, framerate)
  - `-show_chapters` - Show chapter information
  - `-show_packets` - Show packet-level information
  - `-show_frames` - Show frame-level information
- Stream Selection:
  - `-select_streams v:0` - Select first video stream only
  - `-select_streams a:0` - Select first audio stream only
  - `-select_streams v` - Select all video streams
  - `-select_streams a` - Select all audio streams
- Verbosity Control:
  - `-v quiet` - Suppress FFProbe logging output
  - `-v error` - Show only errors
  - `-v info` - Show informational messages (default)
- Common Combinations:
  - `-show_format -show_streams -print_format json` - Complete media info in JSON
  - `-show_format -show_streams -v quiet -print_format json` - Clean JSON output
  - `-select_streams v:0 -show_entries stream=width,height,codec_name -print_format json` - Video dimensions and codec only

Common Use Cases

- Media file validation: Verify codec compatibility, resolution, and format before processing.
- Quality control: Extract technical metadata to ensure media meets quality standards.
- Asset management: Catalog media files with detailed technical properties.
- Transcoding preparation: Analyze source files to determine optimal transcoding parameters.
- Compliance checking: Verify media meets technical specifications for broadcast or delivery.
- Debugging: Troubleshoot playback or processing issues by examining stream details.
- Metadata extraction: Extract embedded metadata, timecodes, and technical information.

Best Practices

- Use JSON output format (`-print_format json`) for easier parsing and processing in downstream steps.

- Add -v quiet to suppress FFProbe logging and get clean output.
- Specify absolute paths for FFProbe binary, input files, and report files.
- Ensure the report directory exists on the remote node before execution.
- Use -select_streams to limit analysis to specific streams and improve performance.
- Test FFProbe commands manually via SSH before configuring in Orchestrator.
- Configure appropriate timeouts for large media files that take longer to analyze.
- Store report files with descriptive names (include timestamp or file ID).
- Implement error handling for missing files, permission issues, or FFProbe failures.
- For sensitive data, ensure secure SSH connections and proper file permissions.

Troubleshooting

- FFProbe not found: Verify FFProbe is installed and the binary path is correct. Run `which ffprobe` on the remote node.
- Permission denied: Ensure SSH user has execute permissions on the FFProbe binary and read permissions on the input file.
- Input file not found: Verify the file path is correct as seen from the SSH session on the remote node.
- Report write failure: Ensure the report directory exists and the SSH user has write permissions.
- SSH connection failure: Verify Remote Node configuration and SSH connectivity.
- Invalid command options: Test the FFProbe command manually via SSH before configuring it in Orchestrator.
- Empty or corrupted report: Check FFProbe version compatibility and verify the input file is a valid media file.
- Timeout errors: Increase timeout settings for large media files that take longer to analyze.

2.8 File archiving operation



Figure 2.8: File archiving operation icon

This plugin provides the ability to execute file archiving operations on both Orchestrator locally-mounted file systems and remote Linux systems via SSH.

Version	Comment
0.4.1	Enhanced action template UI
0.4.0	Removing Windows support code, Updating to be compatible with Ruby 3.4 and Rails 8
0.3.1	Plugin includes remote node dependency on workflow export
0.3.0	Allow plugin to perform operations on remote nodes.
0.2.4	Fix compatibility issue on OS_untar with ruby 2.3 and command line options with multiple
0.2.3	Remove duplicate permissions and cmd line options
0.2.2	Added styling to parameter help text.
0.2.1	Bug fix for os_untar - Only fail when the error code is explicitly non-zero. Exposing more outputs which are useful for debugging errors/failures
0.2.0	Adding new field to allow customers to add flags to OS commands
0.1.1	Fixing issue where OS operations failures did not translate into step errors
0.1.0	Adding new field to support CHMOD and CHOWN on extracted files
0.0.4	2015/05/25 - PM -> Bug fix in inputs_spec, adding unzip to when statement
0.0.3	2015/03/04 - JG -> Enhancement to all functionalities and fixed bugs.
0.0.2	2015/01/20 - JG -> Fixed layouts and exception scenarios.

2.8.1 How It Works

- Provides comprehensive file archiving operations including tar, untar, tgz, untgz, zip, and unzip with both Ruby library and OS-based command line implementations.
- Supports both local execution and remote operations on Linux systems via SSH connections with configurable node selection options.
- Handles single files, directories, and arrays with glob pattern support for flexible source selection and recursive directory processing.
- Includes base directory functionality for simplified path management and target directory creation with configurable failure handling.
- Offers multiple node configuration methods including global remote nodes, runtime selection, and manual advanced parameter specification.

2.8.2 Configuration Instructions

1. Add a File Archiving Operation step to your workflow.
2. Configure the execution environment:
   - Use remote node?: Check to enable remote execution (Linux only).
   - Remote node: Select from global nodes, runtime, or manual configuration.
   - File operation: Choose archiving operation (tar, untar, zip, unzip, tgz, untgz).
3. Set up file paths:
   - Sources: Absolute paths to files or directories to archive.
   - Target: Absolute path for the archive destination.
   - Base: Optional base directory for path simplification.

2.8.3 Example Configuration

Parameter	Example Value
Use remote node?	unchecked
File operation	tar
Sources	/home/user/documents
Target	/backup/archive.tar
Base	/home/user
Is target a directory?	unchecked
Is base a directory?	checked
Fail on incomplete?	checked

2.8.4 Saved Parameters Description

Parameter	Description
Use remote node?	Check this box to show and use options for picking a remote node to execute on. Note that remote operation is currently only supported on remote Linux machines.
Remote node	A remote node is required if *Use remote node?* is checked. This is the remote node where the operations will be performed. The remote node can be selected in three ways: from the global remote nodes, at runtime, or by manually writing the details of the remote node in the advanced parameters. If runtime is selected, the plugin will use the runtime node regardless of whether the advanced parameters are set.
IP Address	Advanced - The IP address of the remote node.
Port	Advanced - The port to be used to connect to the remote node through SSH. By default, port 22 is used.
Node user	Advanced - The username associated with the remote node.
Node password	Advanced - The password associated with the remote node.
File operation	The operation that will be executed. The full set of options are: tar, untar, OS tar, OS untar, tgz, untgz, OS tgz, OS untgz, zip, and unzip. However, note that some operations may be unavailable if you are using a Windows machine or using the remote node functionality.
Sources	Source absolute directory or file paths.
Target	Target absolute directory path.

Parameter	Description
Is target a directory?	Check this box if path entered for *Target* is a directory that already exists. If it does not already exist, it will be created. Only used when executing OS tar or OS tgz on a remote node.
Base	The base directory. See the explanations in the operating instructions.
Is base a directory?	Check this box if path entered for *Base* is a directory that already exists. Only used when executing OS tar or OS tgz on a remote node.
Fail on incomplete?	Return a failure if the operation does not reach completion.

2.8.5 Inputs Description

Changes in this list of inputs will override any values set in the action template.

- source: Source absolute paths.
- base: The base directory.
- operation: The operation to execute.
- target: Target directory path.

2.8.6 Outputs Description

- Failed_list: The files or directories unsuccessfully handled as an array.
- File_list: The files or directories successfully handled as an array.

2.8.7 Operating Instructions

- This plugin is primarily used for archiving operations. It provides archiving operations using both Ruby libraries and standard command line utilities (OS-based).
- The plugin supports both file-based and directory-based operations. If a directory is specified, all included files and subfolders will be archived.
- Example: Setting Sources to `/root/foo/` and Target to `/root/bar/` will create the directory `/root/bar/foo/`.
- For any of the operations, either the source or the base directory should be provided.
- Sources is usually an array type that can accommodate a single path or an array of paths. However, when using tgz or zip operations, source is a string.
- Example: `Sources = /root/foo/abc.txt` or `Sources = ["/root/foo/abc.txt"]`
- Glob expressions are supported within Sources. Example: `Sources = /root/foo/*.txt` to process all files under /root/foo/ with a txt extension.
- Put a '/' at the end of the Target directory path. Otherwise, if the last directory name does not exist, it will be used as the file name to be created.
- Example: Setting Sources to `/root/foo/abc.txt` and Target to `/root/bar` when the bar directory does not exist will create a file `/root/bar` with the content of `/root/foo/abc.txt`. If instead Target is set to `/root/bar/` and the bar directory does not exist, the first source file's directory path will be considered as target path.
- The tgz operation doesn't work on multiple elements. It takes only a single source folder or file to archive. Use OS tgz if more than one folder needs to be processed.
- Base is the ”base directory” that the operation should refer to. When executing: `tar -cvf /root/result.tar / root/foo/bar.txt /root/bar/foo.txt`, the result is the archiving of both files. Instead of mentioning root twice, you could store `/root` as base and provide the rest under source files.

2.8.8 Dependencies

- For remote operations, SSH access must be available on the target Linux system with appropriate user credentials.
- The aspweb user must have proper filesystem permissions for all specified source and target paths when executing locally.
- Required archiving utilities (tar, zip, etc.) must be installed on the execution system (local or remote).



Figure 2.9: File generator icon

2.9 File generator

This action plug-in is used to create a file using dynamic content provided from parameters and/or the output from previous steps in the workflow. Although it can generate any text file, it is primarily used to generate XML files to be used by other steps or other applications. The user defines the template to be used, the variable content required and variable names, and maps the variable inputs.

Version	Comment
0.5.1	Enhanced action template UI
0.5.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.4.0	body template to longtext
0.3.0	Adds support for append mode
0.2.2	Adds support for implied types in target_file_path
0.2.1	Adds styling to parameter help text.
0.2.0	Adds support for changing status within file generation erb
0.1.9	New icon
0.1.8	Avoid crash when target_file_path is blank
0.1.7	Fixed bug introduced in 0.1.6 due to wrong variable name
0.1.6	Added file body path as an input if neither path nor body is configured
0.1.5	Moved to File Operations category
0.1.4	Adds help file
0.1.3	Makes template_name a payload
0.1.2	Adds location of generated file as output
0.1.1	fixes an edge case for method_missing
0.1.0	adds option to remove extra line feed added by ERB
0.0.4	adds auto creation of target folder
0.0.3	enables rich text edit, bug fix in method_missing
0.0.2	adds target file path as optional input
0.0.1	initial release

2.9.1 How It Works

- Generates files dynamically using a template written with ERB (Embedded Ruby).
- Templates can include variables such as `<%=filename%>` and embedded Ruby logic (`<% %>`).
- Both required and optional inputs can be declared. If required inputs aren't mapped to any value, then the workflow will fail.
- Target file paths can be defined directly or constructed dynamically using variables.
- Generated content can be appended to existing files if desired.
- File content can optionally be returned as workflow output for downstream steps.
- Linefeed suppression allows cleaner output by avoiding blank lines from embedded Ruby logic.

2.9.2 Configuration Instructions

1. Add a FileGenerator step to your workflow.
2. Provide the following parameters:

- File body template: Define the ERB template for the file. Supports variables and Ruby logic.
- Required inputs: Declare inputs that must be provided for rendering.
- Optional inputs: Declare optional variables that may be used in the template.
- Target file path: Path for the generated file (can use variables).
- Append to existing?: If enabled, appends content to an existing file instead of overwriting it.
- Return file content?: Makes file contents available as workflow outputs.
- Suppress marked linefeeds: Prevents blank lines caused by embedded Ruby.

2.9.3 Example Usage

Example using embedded Ruby code

If an input `recipients` contains *bob@ac.com; bill@ac.com; doug@ac.com*, the following template:

\ <destinations>\ <% destinations = recipients.split(/[;]+/) %>\ <% destinations.each do |destination| %>\ <to><%= destination %></to>\ <% end %>\ </destinations>\

Will be rendered as:

\ <destinations>\ <to>bob@ac.com</to>\ <to>bill@ac.com</to>\ <to>doug@ac.com</to>\ </destinations>\

2.9.4 *Same example with suppression marked linefeeds*

\ <destinations>\ <% destinations = recipients.split(/[;]+/) %>\\ <% destinations.each do |destination| %>\\ <to><%= destination %></to>\ <% end %>\\ </destinations>\

Will be rendered without extra blank lines:

\ <destinations>\ <to>bob@ac.com</to>\ <to>bill@ac.com</to>\ <to>doug@ac.com</to>\ </destinations>\

Behavior: With this configuration, the workflow generates the file using the provided template, required/optional inputs, and target file path. Outputs are available if enabled.

2.9.5 Example Configuration

Parameter	Value
File body template	XML template with embedded Ruby loops
Required inputs	recipients
Optional inputs	None
Target file path	/tmp/output.xml
Append to existing?	No
Return file content?	Yes
Suppress marked linefeeds	Yes

2.9.6 Saved Parameters Description

Parameter	Description
File body template	The ERB template text used to generate the file.
Required inputs	Declared mandatory variables for rendering.
Optional inputs	Declared optional variables that may or may not be used in the template.
Target file path	Path where the generated file is created.
Append to existing?	If enabled, appends content to the existing file.
Return file content?	If enabled, exposes generated content as an output.

Parameter	Description
Suppress marked linefeeds	Prevents blank lines caused by embedded Ruby.

2.9.7 Inputs Description

- Target file path: Requested if the 'Target file path' parameter is left blank.
- Declared inputs: All required and optional inputs referenced in the template or file path.

2.9.8 Outputs Description

- File path: The full path of the generated file.
- File content: The generated content (only if 'Return file content?' is checked).

2.9.9 Supported Actions

None

2.9.10 Dependencies

No external dependencies. Templates may use standard Ruby logic supported by ERB.

2.9.11 Troubleshooting

- Ensure all required inputs are defined; missing inputs will cause the workflow to fail.
- Verify that the target file path is valid and writable.
- Check ERB syntax in the template—unmatched tags will cause rendering errors.
- Use suppression linefeeds if strict formatting is required.

2.10 File information



Figure 2.10: File information icon

Extracts multiple information about a local file, given its full path.

Version	Comment
0.2.1	Enhanced action template UI
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.1	Fix compatibility issue with array mode (when set to nil)
0.1.0	Accept an array of files
0.0.6	Help file fix.
0.0.5	Output dates in unix format.
0.0.4	Fixed file size and added file size in Bytes.

Version	Comment
0.0.3	added file size in KB as integer to the outputs
0.0.2	added custom icon, and file type (folder, symlink), fixes help
0.0.1	initial release

2.10.1 How It Works

- Retrieves detailed information about a local file or folder, given its full path.

2.10.2 Configuration Instructions

1. Create a new action step using the FileInfo plugin.
2. Fill out the following fields:
   - Name: The name of this template or configuration.
   - Comments: Comments associated with this template or configuration.
   - File path*: The full path of the file to retrieve information for.
   - Fail if file does not exist: By default, the action supports non-existent paths and computes available path information, setting all other values to -1. When this option is enabled, the action will fail if the specified file does not exist.
   - Accept array of files: When enabled, allows the action to receive an array of file paths instead of a single file path. Each file path in the array will be processed independently, and the outputs will contain a corresponding list of results.

2.10.3 Example Configuration

NOTE: In this example, we are retrieving information for a single file path. When using array mode, replace this field with a list of paths.

Parameter	Value
File path	/home/product/test
Fail if file does not exist	Selected
Accept array of files	Not selected (example uses a single file)

2.10.4 Saved Parameters Description

Parameter	Description
Name	The name of this template or configuration.
Comments	Comments associated with this template or configuration.
File path	The full path of the file to retrieve information for.
Fail if file does not exist	By default, the action supports non-existent paths and computes available path information, setting all other values to -1. When this option is enabled, the action will fail if the specified file does not exist.
Accept array of files	When enabled, allows the action to receive an array of file paths instead of a single file path. Each file path in the array will be processed independently, and the outputs will contain a corresponding list of results.

2.10.5 Inputs Description

Parameter	Description
File path	The full path of the file to retrieve information for.

2.10.6 Outputs Description

Parameter	Description
file_name	The file or folder name.
naked_name	The file name without its extension, if any.
file_size	The file size as a string.
file_size_in_byte	The file size in Bytes as a number.
file_size_in_KB	The file size in KBytes as a number.
extension	The file extension.
parent_name	The name of the parent folder.
parent	The full path of the parent folder.
symlink?	Returns true if symlinks are supported by the OS and the specified path points to a symlink (not an actual file).
atime	The last access time, if the file exists. -1 otherwise.
mtime	The last modification time, if the file exists. -1 otherwise.
ctime	The change time, if the file exists. -1 otherwise.

2.10.7 Supported Actions

None

2.10.8 Dependencies

None

2.10.9 Operating Instructions

This action takes a full path as input and extracts detailed information about the file or folder. When array mode is enabled, provide a list of file paths and the action will return a corresponding list of results.

2.11 ISM and M3U8 File Parser



Figure 2.11: ISM and M3U8 File Parser icon

This plugin provides ability to parse an ISM file and a M3U8 file

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.4	Added styling to parameter help text.
0.0.3	Added help file.
0.0.2	Fetches content from the head tag as well.
0.0.1	First version

2.11.1 Saved Parameters Description

Name: The name used to identify a saved IsmM3u8Parser configured instance.

Comments: Some comments about this saved IsmM3u8Parser configured instance.

File Path: File path of the .ism or .m3u8 file.

Return output as array?: By default the .ism file or .m3u8 files are parsed and returned as Hash of file name(s). Checking this checkbox will also return the parsed result in form of an array.

2.11.2 Inputs description

The list of inputs depends on the configuration of the IsmM3u8Parser action template.

- File Path is a mandatory input.

- Return output as array? in optional. Defaults to 'false'.

2.11.3 Outputs description

- *Array_of_ISM_Filenames*: Array of the file name(s) from the .ism file.
- *Hash_of_ISM_Filenames*: Hash of the file name(s) from the .ism file.
- *Hash_of_M3u8_Filenames*: Hash of the file name(s) from the .m3u8 file.
- *Array_of_M3u8_Filenames*: Array of the file name(s) from the .m3u8 file.

2.11.4 Operating Instructions

- Operation of this plugin is simple and straightforward.

- The plugin strictly supports '.ism' and '.m3u8' file formats only.

- Provide a '.ism' or '.m3u8' file as input.

- Plugin returns Hash and Array(in case the 'Return output as array?' is checked.) of the parsed file.

E.g.

File Path: /Users/root/m3u8/stsp.m3u8

Return output as array? False (Unchecked)

Outputs:

Hash_of_M3u8_Filenames : { ”/Users/root/m3u8/stsp.m3u8” => { ”/Users/root/m3u8/pl.m3u8” => [”/Users/root/m3u8/playlist.
}}

2.12 Local file operation



Figure 2.12: Local file operation icon

This action plug-in executes local file operations on Orchestrator-mounted file systems. It supports actions such as copy, move, delete, makedir, sym-link, touch, listing, and archive handling, with options for validation, overwriting, timestamp preservation, and permissions control.

Version	Comment
1.2.1	Enhanced action template UI
1.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.1.0	Adds no glob on sources for delete
1.0.3	Updating help page
1.0.2	Adding troubleshooting tip
1.0.1	Fixes chmod and chown for mkdir
1.0.0	Adds option to fail when listed source files are missing
0.9.2	Removing terminate execution (not needed anymore)
0.9.1	Adding chown to touch and fixing it for link.
0.9.0	Adds option to fail when file listing is empty, hides irrelevant sections in editor
0.8.2	Adds chmod and chown functionality to symlink and overwrite flag overwrites symlink if it exists
0.8.1	Adds chmod and chown functionality to list operation also
0.8.0	Add option to create hard symlink
0.7.3	Fixes metadata
0.7.2	Fixes touch option to allow new file creation
0.7.1	New icon
0.7.0	Added new field to expose failed list an an array without error description
0.6.4	Added help file
0.6.3	Change default block size to 1MB
0.6.2	Making file permissions runtime
0.6.1	Making base optional for UnTAR and TAR
0.6.0	Added two enhancements a) Added logic to allow list operation using base folder (if provided) b) Added logic to CHMOD or CHOWN directories created or files copied
0.5.1	fixes problem with copy and move due to the overwrite flag
0.5.0	adds overwrite option for copy and move
0.4.2	fixes failed list output in mkdir
0.4.1	adds progress reporting to tar/untar and makes copy/move err when explicitly specified files are not moved
0.4.0	adds copy by block (to provide progress report)
0.3.5	removes un-needed dependencies
0.3.4	adding tar and untar
0.3.3	fix timestamping
0.3.2	adds directory listing
0.3.1	fix symlink typo
0.3.0	adds touch, symlink and stat operations. Adds option to fail on incomplete operation and failure list as output
0.2.4	fixes bug when multiple files are processed
0.2.3	returns list of files
0.2.2	Supports for payload in sources, target and base
0.2.1	Uses terminate instead of throw
0.2.0	Adds an option to preserve or not timestamp when moving or copying files
0.1.1	Returns failed and successful list as outputs

2.12.1 How It Works

- Provides file system actions such as copy, move, delete, makedir, stat, symlink, touch, directory listing, tar, and untar.
- Works on single files, directories, arrays, and supports glob expressions for source selection.
- Options include validating sources, overwriting target files, preserving timestamps, and exposing results/failed files.
- Includes permission configuration (chmod/chown) and supports recursive operations.

2.12.2 Configuration Instructions

1. Add a LocalFileOperation step to your workflow.
2. Set up the minimal required fields:
    - File operation: Choose the action (e.g., copy, symlink).
    - Sources: Absolute path(s) to source file(s) or directories.
    - Target: Absolute path for target location.
    - Overwrite?: Check to allow overwriting files in target.
    - Preserve timestamp?: Enable to try to retain source timestamps.

- Validate sources?: Enable to error if any source is missing.

2.12.3 Example Configuration

Parameter	Example Value
File operation	copy
Sources	/opt/aspera/orchestrator/lfo3
Target	/opt/aspera/orchestrator/lfo2
Validate sources?	checked
Overwrite?	checked
Preserve timestamp?	checked
Fail on incomplete?	checked
Copy block size	10
Make the symlink a hardlink?	checked (for symlink operation)

2.12.4 Saved Parameters Description

Parameter	Description
File operation	The operation to execute: copy, move, delete, makedir, stat, sym-link, touch, dir listing, tar, untar.
Sources	Source absolute file/directory paths; supports arrays or glob patterns.
Validate sources?	Errors if specified sources do not exist; required for safer operations.
Disable glob on sources?	Disables wildcard/glob expansion, using sources literally.
Target	Target absolute path for copy/move and related operations.
Base	Base directory prefix for copy/move path transformations.
Copy block size	Block size for copy/move, in bytes (default 1MB; e.g. 10).
Overwrite?	Overwrite target files if they exist.
Preserve timestamp?	Attempts to preserve source timestamps (depends on environment).
Fail on incomplete?	Marks step as failed if operation not complete.
Fail on empty?	Fails step if a list/match returns no results.
Expose failed files as an array?	Exposes failed file paths as output array.
Chmod	Octal permissions mode for created/copied files (e.g. 755).
Recursive	Enable recursive handling for directories.
User ID	User ID for file/directory ownership.
Group ID	Group ID for file/directory ownership.
Make the symlink a hardlink?	If enabled and using symlink, will create hardlink instead.

2.12.5 Inputs Description

- source: Absolute path(s) (array or glob supported) to operate on.
- target: Destination absolute path for relevant actions.
- operation: Operation type (copy, move, etc.).
- base: Base directory prefix; affects destination path in copy/move.
- preserve_timestamp: Try to preserve file timestamps (true/false).
- Chmod: Permissions mode (e.g. 755) for output.
- Uid: User ID for ownership.
- Guid: Group ID for ownership.

2.12.6 Outputs Description

- Failed_list: Array of files/dirs that failed to process.
- File_list: Array of successfully handled files/dirs.

2.12.7 Supported Actions

- copy
- move
- delete
- makedir
- stat
- symlink
- touch
- dir listing
- tar
- untar

2.12.8 Operating Instructions

- The plugin supports both file-based and directory-based operations. If a directory is specified, all included files and subfolders will be copied, moved, or deleted.
- When copying/moving a directory, the destination will include the directory itself unless configured otherwise. Example: `Sources = /root/foo/`, `Target = /root/bar/` will create `/root/bar/foo/`.
- Sources may be a single path or an array of paths.

  Example: `Sources = /root/foo/titi.txt`

  or

  `Sources = ["/root/foo/titi.txt"]`
- Glob patterns are supported in sources (e.g., `Sources = /root/foo/*.txt` to include all text files).
- Target path rules: If the target is missing a trailing slash, and the last directory does not exist, it will be treated as a filename.

  Example: `Sources = /root/foo/titi.txt` with `Target = /root/bar` (and bar does not exist) → result: `/root/bar` (a file).

  If instead `Target = /root/bar/` (and 'bar' does not exist) → an error will be displayed.
- The touch operation will not create a new file if it does not exist; instead, it updates the timestamps on an existing file.
- The Base parameter modifies how paths are replicated. Example:

  Command: `cp /root/foo/bar.txt /home/`

  Result: `/home/bar.txt`

  With Base = `/root`:

  Same copy gives `/home/foo/bar.txt`.

2.12.9 Dependencies

- The aspweb user must have proper filesystem permissions for all specified source and target paths.

2.12.10 Troubleshooting

- In Windows, if the 'move' operation is failing between two mapped drives, check that the source and target paths are configured properly in the plugin. UNC paths are always preferred.
- In Windows, the 'move' operation usually works between mapped drives if the paths are configured with backslashes. Example: ”\\10.0.154.185\sharefororch\”

- Sometimes Windows struggles with backslashes in paths. Using double backslashes like ”\\share\\folder\\” may resolve issues.
- If 'Preserve timestamp?' does not retain the original time, confirm filesystem and user permission capabilities.
- Hardlinks require both source and target to be on the same filesystem/partition. Verify this if hardlink operations fail.
- Ensure your OS and filesystem support symlinks/hardlinks, as not all filesystems (e.g., FAT32) allow them.

2.13 Local file permission



Figure 2.13: Local file permission icon

This action plug-in provides the ability to do the chmod and chown file operations after a file has been downloaded.

Version	Comment
0.1.2	Enhanced action template UI
0.1.1	Updating to allow aspweb user to perform file operations
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.5	Added styling to parameter help text.
0.0.4	2015-02-16 ML - New icon
0.0.3	Fix type of chmod_get where the plugin errors when there is no value
0.0.2	Fix type of source_files types to be array, with tolerance for string
0.0.1	First version of the permissions plugin allowing users to do CHOWN and CHMOD operations on file(s)

2.13.1 How It Works

- This action plug-in performs file permission and ownership operations on the local system.
- It supports both CHMOD and CHOWN operations on files and directories.
- GLOB patterns can be used to target multiple files or directories.
- When a directory is specified, all contained files and subdirectories can be processed recursively.
- Successful and failed operations are reported separately for CHMOD and CHOWN.

2.13.2 Configuration Instructions

1. Create a new action step using the LocalFilePermissions plug-in.
2. Complete the following configuration fields:
    - Name: The name used to identify a saved LocalFilePermissions configuration.
    - File(s): Files or directories to apply the operation on. GLOB wildcards are supported. Multiple entries can be separated by commas.
    - Permissions code: Octal mode value used for CHMOD operations (e.g. 755).
    - User UID or name: User ID or username used for CHOWN operations.
    - Group GID or name: Group ID or group name used for CHOWN operations.
    - Recursive: If enabled, operations are applied recursively to subdirectories and files.

2.13.3 Example Configuration

Below is an example configuration for the LocalFilePermissions plug-in.

Parameter	Value
File(s)	/data/output/*.mp4
Permissions code	755
User UID or name	aspera
Group GID or name	media
Recursive	Selected

2.13.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved LocalFilePermissions configuration.
File(s)	Files or directories to apply the operation on. GLOB wildcards are supported. Multiple entries can be separated by commas.
Permissions code	Octal mode value used for CHMOD operations (e.g. 755).
User UID or name	User ID or username used for CHOWN operations.
Group GID or name	Group ID or group name used for CHOWN operations.
Recursive	If enabled, operations are applied recursively to subdirectories and files.

2.13.5 Inputs Description

All parameters are available as runtime inputs if they are not specified in the template. This plug-in does not support value overwrites: parameters defined in the template are not exposed as runtime inputs.

Parameter	Description
file_list	Files or directories to operate on, if not specified in the template.
permis-sions_code	Octal permissions value for CHMOD operations.
user_uid_or_nar	User ID or name for CHOWN operations.
group_gid_or_na	Group ID or name for CHOWN operations.
recursive	Whether the operation should descend recursively.

2.13.6 Outputs Description

Parameter	Description
CHOWN_File_lis	List of files on which the CHOWN operation was successful.
CHOWN_Failed_	List of files on which the CHOWN operation failed.
CHMOD_File_lis	List of files on which the CHMOD operation was successful.
CHMOD_Failed_	List of files on which the CHMOD operation failed.

2.13.7 Supported Actions

CHMOD, CHOWN

2.13.8 Dependencies

Local file system access and sufficient permissions to execute CHMOD and CHOWN commands.

2.13.9 Operating Instructions

1. The plug-in executes CHMOD and/or CHOWN commands on the local file system.
2. Both files and directories can be specified as input.
3. When a directory is provided, all contained files and subdirectories are included.
4. Recursive processing is controlled using the Recursive option.
5. Results are separated into successful and failed file lists for each operation.

2.14 MD5 checksum



Figure 2.14: MD5 checksum icon

This action plug-in calculates MD5 checksums for specified files and optionally verifies them against previously calculated values. It supports memory-efficient processing with configurable block sizes and dynamic file path inputs for flexible file integrity verification workflows.

Version	Comment
0.3.1	Enhanced action template UI
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.2	Added styling to parameter help text.
0.2.1	2015-02-16 ML - New icon
0.2.0	Adds blocksize configuration
0.1.0	Adds progress reporting
0.0.3	Adds display name
0.0.2	fixes template and adds help
0.0.1	first tracked version

2.14.1 How It Works

- Calculates the MD5 checksum for a specified file.
- Optionally verifies the calculated checksum against a provided value.
- Uses configurable block size for memory-efficient processing of large files.
- Fails the step if verification mode is enabled and checksums don't match.

2.14.2 Configuration Instructions

1. Create a new action step using the Md5Checksum plugin.
2. Fill out the following fields:
   - Filter name: Label for the step (e.g., `File Checksum Validator`).
   - Comments: Notes for the template or configuration.
   - File path: Path to the file for MD5 calculation (e.g., `/path/to/example.txt`). If left blank, file path will be required as input.
   - Block size: Number of bytes read at a time for calculation (default: `131072`).

2.14.3 Example Configuration

Field	Value
Filter name	`New Md5Checksum Step`
File path	`/path/to/example.txt`
Block size	`131072`

Behavior: The plugin will calculate the MD5 checksum for `/path/to/example.txt`, reading 131,072 bytes at a time for optimal memory usage.

2.14.4 Saved Parameters Description

Parameter	Description
Comments	Notes and comments for this step or template configuration.
File path	Path to the file for which the MD5 checksum needs to be computed.
Block size	Number of bytes read at a time in memory to compute the calculation.

2.14.5 Inputs Description

- File_path: Required if 'File path' field is left blank, or when using dynamic parameters (must be string type).
- Check Against: Optional. If provided, enables verify mode - the computed checksum will be compared to this value.
- All dynamic variables listed in the 'File path' field are mandatory and must be of type string.

2.14.6 Outputs Description

Output	Description
MD5_checksum	The calculated MD5 checksum for the specified file.
Checked_filepath	The actual path of the file that was checked.
Step_information	Step execution details and metadata.

2.14.7 Supported Actions

None

2.14.8 Dependencies

None

2.14.9 Operating Instructions

This action performs the calculation of the MD5 checksum for a specified file.

- Use for file integrity verification and validation workflows.
- Enable verification mode by providing a checksum value in inputs.
- Adjust block size based on available memory and file sizes.

If an MD5 checksum is provided in inputs, the action will compare the calculated checksum against it and exit with 'Failed' status if the two do not match.

2.14.10 Troubleshooting

- Ensure that the file path is accessible and the file exists.
- Verify that file permissions allow read access.
- Check that dynamic input variables are properly formatted as strings.
- For large files, consider adjusting the block size to optimize performance.

2.15 Media info



Figure 2.15: Media info icon

This plugin extracts comprehensive technical and tag metadata from video and audio files using the MediaInfo application.

Version	Comment
0.7.1	Enhanced action template UI
0.7.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.6.0	Add XML output option
0.5.0	Add options to skip cover data from generic result
0.4.0	Adding Local execution mode
0.3.5	Adding Remote Node Dependency
0.3.4	Fixing Parsing in output
0.3.3	Improvement to help file to indicate when MediaInfo outputs might be blank
0.3.2	2015-02-16 ML - New icon
0.3.1	fixes migration name for additional sections
0.3.0	adds option to enter additional sections
0.2.0	exposing image information if available
0.1.9	preventing autocomplete of user info by browsers
0.1.8	exposes execution login and password as runtime params
0.1.7	fixes bug on removal of additional outputs
0.1.6	escapes binary path to avoid windows issues with space in binary path
0.1.5	adds ability to dynamically provide node and binary_path
0.1.3	strips \r when server is windows, avoids crash on badly formatted output
0.1.2	fails if no results are returned
0.1.1	adds support for space and other exotic characters in filename
0.1.0	adds support for singled out additional outputs
0.0.7	adds functionality to mask password
0.0.6	fixes default name in template editor
0.0.5	exposes detailed mode checkbox
0.0.4	help file provided
0.0.3	remove duplicate method process_outputs
0.0.2	bug fix (missing constant)
0.0.1	initial release

2.15.1 How It Works

- Executes MediaInfo analysis operations on remote nodes via SSH to extract comprehensive metadata from media files.
- Supports detailed mode equivalent to the -f (full) verbose option for extensive metadata extraction.
- Extracts metadata organized into General, Video, Audio, and Image sections with optional additional sections.
- Provides output in hash format by default or XML format when specified, with customizable additional data fields.
- Handles embedded cover data with options to ignore problematic outputs that might affect database storage.

2.15.2 Configuration Instructions

1. Add a MediaInfo step to your workflow.

2. Configure the essential parameters:
    - Media file path: Source file path for analysis (relative to remote node).
    - Execution Node: Remote node where MediaInfo application is installed.
    - Path to the mediainfo binary: Full path to MediaInfo executable on remote node.
    - Detailed Mode: Enable for comprehensive metadata extraction (-f option).
3. Optionally configure advanced settings:
    - Execution Login: User account for remote execution (if different from node default).
    - Execution Password: Password for remote login (if different from node default).
    - Additional Sections: Extract other sections beyond standard Audio, Video, General, Image.

2.15.3 Example Configuration

Parameter	Example Value
Media file path	/media/videos/sample.mp4
Detailed Mode	checked
Ignore Cover Data output	unchecked
Additional Sections	unchecked
Return xml output instead of hash?	unchecked
Execution Node	media-analysis-server
Execution Login	mediauser
Path to the mediainfo binary	/usr/bin/mediainfo
Any optional parameters	--Output=JSON

2.15.4 Saved Parameters Description

Parameter	Description
Media file path	The source file for the analysis operation. Must be accessible from the remote node.
Detailed Mode	Equivalent to the -f (full) verbose option for the mediainfo command line. When enabled, most parameter values are returned as arrays instead of strings.
Ignore Cover Data output	If the media has embedded cover data, it could prevent storage in the database. This option ignores cover data to avoid storage issues.
Additional Sections	Allows extraction of other sections besides Audio, Video, General and Image if present (e.g., Audio #1, Audio #2).
Additional Outputs	Provides the ability to extract specific data fields from MediaInfo results. Requires name, value type, and field identification using hash_name -> hash_value syntax.
Return xml output instead of hash?	Produces XML output instead of the default Hash output format.
Execution Node	The node where the analysis operation must be executed (where MediaInfo application is installed).
Execution Login	The user account used to execute the MediaInfo analysis operation. Uses node default if not specified.
Execution Password	The password for remote node login. Uses node default if not specified.
Path to the mediainfo binary	The path to locate the MediaInfo binary on the remote node. Not needed if binary folder is in system path.
Any optional parameters to pass to the MediaInfo process	Optional parameters supported by the mediainfo binary, passed directly to command line.

2.15.5 Inputs Description

- Source file path: All variables listed in Source file path field are mandatory and must be of type string.

- Command line options: All variables listed in the optional parameters to pass to the MediaInfo process are mandatory and must be of type string.
- Message: Optional message parameter for the analysis operation.

2.15.6 Outputs Description

- General: A hash containing all the metadata information that MediaInfo classifies as General.

  For example: *{ 'Writing library' => 'VirtualDub build 11636/release', 'Format' => 'AVI', 'File size' => '5.21 MiB', 'Format/Info' => 'Audio Video Interleave', 'Overall bit rate' => '798 Kbps', 'Complete name' => '/Users/mlusinchi/Movies/annie_iode 'Duration' => '54s 720ms' }*

- Video: Same as above for the Video metadata.

  For example: *{ 'Stream size' => '4.30 MiB (83%)', 'Codec ID/Info' => 'Intel Indeo Video 5.0 Wavelet', 'Format' => 'Indeo 4', 'ID' => '0', 'Codec ID' => 'IV50', 'Display aspect ratio' => '2.35:1', 'Width' => '320 pixels', 'Duration' => '54s 720ms', 'Height' => '136 pixels', 'Bits/(Pixel*Frame)' => '0.606', 'Bit rate' => '659 Kbps', 'Frame rate' => '25.000 fps' }*

- Audio: Same as above for the Audio metadata.

  For example: *{ 'Compression mode' => 'Lossy', 'Stream size' => '855 KiB (16%)', 'Format' => 'MPEG Audio', 'Mode' => 'Joint stereo', 'Mode extension' => 'MS Stereo', 'ID' => '1', 'Codec ID' => '55', 'Channel(s)' => '2 channels', 'Alignment' => 'Split across interleaves', 'Interleave, duration' => '40 ms (1.01 video frame)', 'Format version' => 'Version 1', 'Format profile' => 'Layer 3', 'Interleave, preload duration' => '451 ms', 'Sampling rate' => '48.0 KHz', 'Duration' => '54s 720ms', 'Bit rate mode' => 'Constant', 'Codec ID/Hint' => 'MP3', 'Bit rate' => '128 Kbps' }*

- XML output: The XML output if 'Return xml output instead of hash?' is selected.

Note: For each of those hash fields, if several values are provided by mediainfo for the same field, an array is returned instead. This is the case if the -f option is used.

For example, in default mode in the Video hash, a pair 'Bit rate'=>'638 Kbps' whereas in full mode (-f), the pair is 'Bit rate'=>['637520', '638 Kbps']

2.15.7 Operating Instructions

- This action executes the specified file analysis operation on a remote node where MediaInfo is available.
- The connection to the remote server is done over SSH, so make sure that the target server has SSH enabled. If the server is a Windows machine, OpenSSH can be installed. Aspera PointToPoint or Enterprise Server will provide SSH with it.
- It is typically a good idea to provide the explicit path to the mediainfo binary to avoid configuration issues. On a UNIX-flavored machine, running 'which mediainfo' should provide the path information.
- Important Note: This action plugin does not do any file movement. Hence, for it to run a file analysis, the specified file has to be accessible. Either the file is transferred to that server or it must be accessible from that server either on some type of network share.
- Important Note II: MediaInfo does not explicitly fail when the file is not found. The plugin provides empty information for General, Video, Image. So during development or testing, when you encounter this scenario check the file path (relative to the remote node) and also the permissions to the file.
- Note about the use on a Windows node:
- When the execution node is Windows, a couple of considerations need to be taken into account during the configuration:
- 1/ The mediainfo CLI needs to be installed. It can be downloaded separately from http://mediaarea.net/download/binary/mediainfo/0.7.64/MediaInfo_CLI_0.7.64_Windows_i386.zip (for 32 bit) or http://mediaarea.net/download/binary/mediainfo/0.7.64/MediaInfo_CLI_0.7.64_Windows_x64.zip (for 64 bit).
- 2/ The etc/passwd file needs to be set for the SSH user used by Orchestrator with a valid user root and to use the 'bin/sh' shell (the switch shell has bizarre escape rules that are best avoided). In the case below '/' is used as the doc root for the user Orchestrator.
- If Aspera Point to Point or Enterprise Server is used to provide remote access, the passwd file will be found in C:\Program Files\Aspera\Enterprise Server\etc\passwd
- 3/ When configuring the action, make sure the path to the mediainfo executable is provided. The path should be provided as a UNIX path since it is used through a UNIX Shell. Typically, the mediainfo command line should be installed at c:/mediainfo/mediainfo.exe or C:/Program Files/Mediainfo/mediainfo.exe
- The path can also use the cygwin preferred POSIX equivalent: /cygdrive/c/mediainfo/mediainfo.exe

- File path should also be provided in UNIX format. For example: c:/DATA/in/Violin Margo.mov or /cygdrive/c/DATA/in/Violin Margo.mov

Example passwd file configuration:

Administrator:unused_by_nt/2000/xp:500:513:U-EXPERIEN-F1FAFA\

\ Administrator,S-1-5-21-1801674531-1202660629-2067101283-500:/home/Administrator:/bin/switch

\ orchestrator:unused_by_nt/2000/xp:1003:513:orchestrator,U-EXPERIEN-F1FAFA\

\ orchestrator,S-1-5-21-1801674531-1202660629-2067101283-1003:/:/bin/sh

2.15.8 Dependencies

- MediaInfo application must be installed on the target execution node.
- SSH access must be enabled on the target server for remote execution.
- The aspweb user must have appropriate permissions to access source files when executing locally.
- Network connectivity between Orchestrator and execution node is required for SSH operations.

2.15.9 Troubleshooting

- Empty metadata results: Verify that the source file path is correct relative to the remote node and that file permissions allow access.
- SSH connection failures: Ensure SSH is enabled on target server and credentials are correct. For Windows, verify OpenSSH installation or Aspera server configuration.
- MediaInfo 'not found' errors: Provide explicit path to mediainfo binary or ensure the binary location is in the system PATH for the execution user.
- Path format issues on Windows: Use UNIX path format (e.g., /cygdrive/c/path) and ensure etc/passwd file is properly configured with /bin/sh shell.
- Cover data storage issues: Enable 'Ignore Cover Data output' option if embedded cover data prevents database storage of results.
- Detailed mode array handling: When using detailed mode, expect array values for fields with multiple entries instead of single string values.

2.16 Mediabin asset management



Figure 2.16: Mediabin asset management icon

This plugin provide the ability to perform some asset management on a Mediabin server. Supported operations include asset listing and retrieval (with download).

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.4	Added Remote Node Dependency
0.0.3	Added help file and corrected the label for Use SSL field.
0.0.2	exception handling of RemoteNode.find_by_name()
0.0.1	initial release

2.16.1 Saved Parameters Description

- Name: The name used to identify a saved MediabinAssetManagement configured instance.
- Comments: Some comments about this saved MediabinAssetManagement configured instance.
- Mediabin node: Select the node where Mediabin server is installed. The select option lists all the remote nodes that are configured on the orchestrator box.

  Advanced parameters section
- Mediabin address: The host address of the Mediabin server.
- Mediabin wsdl url: The URL for Mediabin API WSDL.
- Mediabin user: User name to access the API.
- Mediabin Password: Password for the user to use API.
- re-enter: Re-enter the password.
- Use ssl?: If checked the API call will be performed over HTTPS. Else, it is made on HTTP.
- Operation: Two operations are available for current version of the plugin. List assets and Download asset.
- Mediabin path: Path on the Mediabin server where the above mentioned operation is to be performed on.
- Local path: In case the operation selected is as Download asset, local path must be provided as the destination of the downloaded asset.

2.16.2 Inputs description

The list of inputs depends on the configuration of the MediabinAssetManagement action template.

Mediabin wsdl url, Mediabin user, Mediabin password are mandatory fields.

Either of Mediabin node OR Mediabin Address should be provided

If Operation is selected as 'List assets' Mediabin path is a mandatory field.

If Operation is selected as 'Download asset' Mediabin path and Local path are mandatory fields.

2.16.3 Outputs description

- *Assets*: An array of all Mediabin asset that are present on the Mediabin path.
- *Local_path*: The path of the local directory where the downloaded asset will reside. This is the same as the one given in the input. Will be populated when the asset is downloaded successfully. Asset name is appended in the path.

2.16.4 Operating Instructions

In order to operate the plugin, installation of Mediabin server is required.

Access to the Mediabin WSDL and credentials to consume the API should also be present

With this version of plugin, Listing of assets and downloading of an asset are the two operations that are supported.

Select the operation that needs to be performed and provide the required inputs for the selected operation.

Execution of the plugin is on synchronous mode.

2.17 Remote file operation

This action plug-in provides the ability to execute file operations (copy, move, delete, makedir, dir list) on remote nodes using Aspera transfer servers.

Version	Comment
0.3.2	Enhanced action template UI
0.3.1	Prevent input/output pre-processing code from being overwritten

Version	Comment
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.8	Mask password field.
0.2.7	Added styling to help text.
0.2.6	Adding Remote Node Dependency
0.2.5	Add error description to status details in case of delete error
0.2.4	Fixed the ordering in move and copy with overwrite
0.2.3	2015-02-17 ML - New icon
0.2.2	added a help file
0.2.1	exception handling of RemoteNode.find_by_name()
0.2.0	Adds flag to overwrite on cp or mv
0.1.0	0.0.10 is not a well supported version number. Switching to 0.1.0 instead
0.0.10	fixed regression introduced in 0.0.9 with deletions
0.0.9	continue deletion even if one fails
0.0.8	removed timeout
0.0.7	fix wrong variable name for remote password
0.0.6	removes need for target when using option 'list'
0.0.5	adds comments about trailing '/' in editor
0.0.4	returns target file list as output
0.0.3	allow payload string definition for source and target
0.0.2	adds list command
0.0.1	initial release

2.17.1 How It Works

- Executes file operations on remote nodes using the Aspera ascmd executable for file browsing and management.
- Supports copy, move, delete, makedir, and directory listing operations via SSH authentication.
- Works with remote nodes running Aspera transfer servers (e.g., point-to-point) and SSH servers.
- Handles both single files and directories with multiple source support.
- Uses relative paths based on remote user's docroot as defined in /opt/aspera/etc/aspera.conf.

2.17.2 Configuration Instructions

1. Add a RemoteFileOperation step to your workflow.
2. Configure the essential parameters:
    - Name: Enter a descriptive name for this step.
    - File operation: Select the operation (copy, move, delete, makedir, dir list).
    - Remote node: Choose the target remote node from the list of configured Remote Nodes.
    - Source: Add source file/directory paths (click 'Add Source' for multiple sources).
3. Set operation-specific options:
    - Target: Specify target directory path (for copy/move operations).
    - Base: Optional base directory for path transformations.
    - Overwrite?: Check to allow overwriting existing files in the target directory.

2.17.3 Example Configuration

Parameter	Example Value
Name	Remote File Copy Operation
File operation	copy
Remote node	aspera-server-01
Source	/data/source_file.txt
Target	/backup/
Overwrite?	checked

2.17.4 Saved Parameters Description



Figure 2.17: Remote file operation icon

Parameter	Description
Name	Display name for this remote file operation step.
Comments	Notes or description for this step.
File operation	Operation to execute: copy, move, delete, makedir, dir list.
Overwrite?	Allow overwriting files if already present in the target directory (for copy/move operations).
Remote node	Target remote node name from the list of defined Remote Nodes.
Source	Source directory or file paths. Multiple sources can be added by clicking 'Add Source'.
Target	Target directory path for copy and move operations.
Base	Base directory for copy/move operations affecting path replication (see Operating Instructions).

2.17.5 Inputs Description

Changes to the inputs listed below will override any values set in the action template.

- source: Source path for the operation.
- target: Target directory path for copy/move operations.
- base: Base directory affecting path replication.
- remote_node_password: Password for remote node authentication.
- remote_node_user: Username for remote node authentication.

2.17.6 Outputs Description

- Directory_list: An array of files or directories processed by the operation. For copy/move operations, returns the target paths.

2.17.7 Supported Actions

- copy
- move
- delete
- makedir
- dir list

2.17.8 Dependencies

- The remote node must have the Aspera transfer server installed (e.g., point-to-point).
- The remote node must run the SSH server for authentication.
- The remote user must be an operating system account defined in /opt/aspera/etc/aspera.conf.
- Referenced files/folders need appropriate permissions for the remote user.
- The Aspera ascmd executable must be available for file browsing operations.

2.17.9 Operating Instructions

2.17.10 Aspera Configuration Requirements

This plugin uses the Aspera ascmd executable to browse files on remote nodes with Aspera transfer servers. The remote user must be defined in `/opt/aspera/etc/aspera.conf` with appropriate docroot configuration:

```
\ <user>
\   <name>xfer</name>
\   <file_system>
\     <access>
\       <paths>
\         <path>
\           <absolute>/home/xfer/data</absolute>
\         </path>
\       </paths>
\     </access>
\   </file_system>
\ </user>
\
```

Path Resolution: A target directory like ”/foo” refers to ”/home/xfer/data/foo” on the remote server.

2.17.11 File and Directory Operations

- Supports both file-based and directory-based operations. If a directory is specified, all included files and subfolders will be copied/moved/deleted.
- Directory handling example: `Source = /foo/`, `Target = /bar/` will create the directory `/bar/foo/`.
- Source supports single path or array of paths:

  Example: `Source = /foo/titi.txt` or `Source = ["/foo/titi.txt", "/foo/toto.txt"]`
- Glob expressions (e.g., *.txt) are not supported within Source paths.
- Target path handling: Add '/' at the end of Target directory path. Without it, if the last directory doesn't exist, it will be treated as a filename.

  Example: `Source = /foo/titi.txt` with `Target = /bar` (bar doesn't exist) → creates file `/bar`

  With `Target = /bar/` (bar doesn't exist) → creates directory and results in `/bar/titi.txt`

2.17.12 Base Directory Functionality

The Base parameter controls path replication behavior:

- Without Base: `cp /foo1/foo2/titi.txt /bar/` → Result: `/bar/titi.txt`
- With Base = /foo1/: `cp /foo1/foo2/titi.txt /bar/ --base /foo1/` → Result: `/bar/foo2/titi.txt`

Base directory allows replicating the directory structure under a specified base path to the target location.

2.17.13 Troubleshooting

- Verify Aspera transfer server is properly installed and configured on the remote node.
- Ensure SSH connectivity and authentication work between Orchestrator and remote node.
- Check that the remote user account exists in both the operating system and /opt/aspera/etc/aspera.conf.
- Verify file/directory permissions allow the remote user to access specified paths.
- Confirm that paths are relative to the user's docroot as defined in aspera.conf.

- For permission issues, check that the remote user has read access for sources and write access for targets.

2.18 SglFlashnetArchive



Figure 2.18: SglFlashnetArchive icon

This plug-in proivdes capability to perform rest operation on CatDV server.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.4	Ajusting name of plugin.
0.0.3	Added styling to parameter help text.
0.0.2	Changed the category type to File Operations
0.0.1	Plugin release

2.18.1 Inputs description

The list of inputs depends on the configuration of the SglFlashnetArchive action template.

-->

- Name: Name of the action step.
- Comments: Comments associated to this template or configuration.
- Sgl ip address (or hostname): The hostname or the IP address of the SGL FalshNet archiving server.
- Sgl xml api port: SGL FalshNet archiving server port where the request XML will be printed.
- Api version: The API version of the server.
- Status polling frequency: The polling frequency in seconds to poll for the response.
- Data source server: Datasource server is the datasource on the server from where the context will be accesses / archived.
- Operator username: User name of the operator performing the operation.
- Calling application: Name of the system that is accessing the API.
- Operation: Select the operation that needs to be performed (e.g. Archive, Restore etc.). Click the refresh link next to the dropdown menu. This will load all the attributes that are need to for the selected operation.
- Mandatory attributes: These attribute(s) are mandatory for the selected operation.
- Optional attributes: These attribute(s) are optional for the selected operation.

Mandatory and Optional attributes are loaded from the payload file residing in the plugin payload directory

Plugin payload directory also contains attirbuts_mapping file. This is required because certain attributes in the payload, are difficult to interpret as in on the plugin template. E.g. PartialStart.QWD is mapped to FilePartialStart. So on the template user will see File Partial Start while the value of this attribute will go as PartialStart.QWD in the actual payload that is submitted.

Sgl ip address (or hostname), Sgl xml api port, Data source server, Operator username, Calling application and Operation are Mandatory inputs.

2.18.2 Outputs description

- *Errors*: Response error(s).

- *Status*: The status of the request that was submitted. Typically it will be Passed / Failed.
- *RequestId*: The request id of the API request that was made. Not returned in case of Search operation.
- *Output_file*: Returned in case of Search operation.It returns array of all files that matched the search criteria.
- *Job_status*: Status of the operation that was requestion.
- *Exit_Code*: The exit code of the operation that was performed.
- *Error_code*: In case error occurs this output field is populated with the error code that is returned.
- *Job_status_info*: Contains hash of the response on the operation performed. Job_status, Error_code, Exit_code are part of Job_status_info.

2.18.3 Supported Actions

None

2.18.4 Dependencies

None

2.18.5 Operating Instructions

In order to perform the API calls, provide all the mandatory inputs.

Select the operation that needs to be performed and click refresh link.

Provide all the mandatory attributes for the operation selected.

Response is sent back as XML document and the string. This JSON string is converted to a hash and is stored in the plugin output.

2.19 Spreadsheet parser



Figure 2.19: Spreadsheet parser icon

This plug-in allows workflows to read and process data from spreadsheet files such as Excel. It extracts the spreadsheet content into structured data that can be used in later steps.

Version	Comment
0.2.1	Enhanced action template UI
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.5	Fix Input validation would fail with no real reason
0.1.4	Fix spreadsheet rows missing cells
0.1.3	Fix for parsing small spreadsheets
0.1.2	Fixed multisheet parsing
0.1.1	Fixed bug in the mark_warnings code
0.1.0	warning marking as * made optional
0.0.7	Added styling to parameter help text.
0.0.6	Updating the code for nil fields as per Randy's diff to handle boolean and date fields
0.0.5	2015-03-02 ML - New icon

Version	Comment
0.0.4	reads large spreadsheets in chunks with reporting
0.0.3	modification of xlsx support to fix issues
0.0.2	fixes validation, corrects typo in GUI and fixes error handling of unknown formats
0.0.1	first tracked version

2.19.1 How It Works

The SpreadsheetParser plug-in reads the specified file, then organizes its contents by rows, columns, or individual cells. You can choose how the data is indexed (by headers, column names, etc.) and whether to include header rows/columns.

2.19.2 Configuration Instructions

Create a new action step using the SpreadsheetParser plug-in.

Fill out the following fields:

Name: A label for the step (e.g., `Parse Sheet`).

Comments: Optional notes about this configured instance.

File Path: The full path to the spreadsheet file. Dynamic attributes are supported. Optional if provided as input.

Return First Sheet Only: If checked, only the first sheet will be parsed.

Organize Data: Select how to organize data – by rows, columns, or cells.

Index By: Choose how to index data – by headers, header-row column names, header-column row names, or both rows and columns.

Drop Header Row: If checked, the top-most row will be excluded from results.

Drop Header Column: If checked, the left-most column will be excluded from results.

Mark Untyped Entries with a Warning?: Appends `*` to entries without a type if checked.

2.19.3 Saved Parameters Description

Parameter	Description
Name	The name used to identify this SpreadsheetParser instance.
Comments	Optional comments about this instance.
File Path	Full path to the spreadsheet file. Dynamic attributes are supported. Optional if provided as input.
Return First Sheet Only	If checked, only the first sheet will be parsed.
Organize Data	Choose to organize data by rows, columns, or cells.
Index By	Select how to index data: by headers, header-row column names, header-column row names, or rows and columns.
Drop Header Row	If checked, removes the first row from results.
Drop Header Column	If checked, removes the first column from results.
Mark Untyped Entries with a Warning?	If checked, appends `*` to values with no type.

2.19.4 Inputs Description

The list of inputs depends on how the SpreadsheetParser action template is configured.

- File_path: All variables listed in the File Path field must be provided as input and must be of type `string`.

2.19.5 Outputs Description

Output	Description
First Sheet	Only available if 'Return First Sheet Only' is checked. Contains data from the first sheet structured as a hash.
Sheet Name	Name of the first sheet (only if 'Return First Sheet Only' is checked).
Sheets	All sheet data indexed by sheet name (if 'Return First Sheet Only' is not checked).

Note: The structure of sheet data depends on the selected organization and indexing configuration.

2.19.6 Supported Actions

None

2.19.7 Dependencies

None

2.19.8 Operating Instructions

A parsed spreadsheet is returned as a hash. Its structure depends on the Organize By setting:

- By Rows: Hash keys are row identifiers; values are nested hashes (columns).
- By Columns: Hash keys are column identifiers; values are nested hashes (rows).
- By Cells: Flat hash with spreadsheet cell ID as keys (e.g., `A1`, `B3`).

Indexing also depends on the chosen Index By option.

2.19.9 Example Output

Given the following spreadsheet:

\ Date Time Time cost Expenses Total\ 11/01/2012 4 1200 450 1650\ 11/02/2012 3 900 0 900\ 11/03/2012 4.5 1350 550 1900\ TOTAL 11.5 3450 1000 4450\

Organize by rows - index by rows and columns

\ ”Sheets” => {\ ”Example” => {\ 5 => {”A” => ”TOTAL”, ”B” => 11.5, ”C” => 3450, ”D” => 1000, ”E” => 4450 },\ 1 => {”A” => ”Date”, ”B” => ”Time”, ”C” => ”Time cost”, ”D” => ”Expense”, ”E” => ”Total” },\ 2 => {”A” => Thu, 01 Nov 2012, ”B” => 4, ”C 3 => {”A” => Fri, 02 Nov 2012, ”B” => 3, ”C” => 900, ”D” => 0, ”E” => 900 },\ 4 => {”A” => Sat, 03 Nov 2012, ”B” => 4.5, ”C” => 1350, ”D }\ }\

Organize by rows - index by header-row columns name

\ ”Sheets” => {\ ”Example” => {\ 5 => { ”Expense” => 1000, ”Time cost” => 3450, ”Time” => 11.5, ”Date” => ”TOTAL”, ”Total” => 4450 },\ 1 => { ”Expense” => ”Expense”, ”Time cost” => ”Time cost”, ”Time” => ”Time”, ”Date” => ”Date”, ”Total” => ”Total” },\ 2 => { ”Expense” => 450, ”Time cost” => 1200, ”Time” => 4, ”Date” => Thu, 01 Nov 2012, ”Total” => 1650 },\ 3 => { ”Expense” => 0, ”Time cost” => 900, ”Time” => 3, ”Date” => Fri, 02 Nov 2012, ”Total” => 900 },\ 4 => { ”Expense” => 550, ”Time cost” => 1350, ”Time” => 4.5, ”Date” => Sat, 03 Nov 2012, ”Total” => 1900 }\ }\ }\

Organize by rows - index by header-column rows name

\ ”Sheets” => {\ ”Example” => {\ ”Date” => {...},\ ”Thu, 01 Nov 2012” => {...},\ ”Fri, 02 Nov 2012” => {...},\ ”Sat, 03 Nov 2012” => {...},\ ”TOTAL” => {...}\ }\ }\

Organize by rows - index by headers

\ ”Sheets” => {\ ”Example” => {\ ”Thu, 01 Nov 2012” => {...},\ ”Fri, 02 Nov 2012” => {...},\ ...\ }\ }\

Organize by cells - index by rows and columns

\ ”Sheets” => {\ ”Example” => {\ ”A1” => ”Date”, ”B1” => ”Time”, ”C1” => ”Time cost”, ...\ }\ }\

Organize by columns - index by rows and columns

\ ”Sheets” => {\ ”Example” => {\ ”A” => {1 => ”Date”, 2 => ..., ...},\ ”B” => {...}, ...\ }\ }\

Organize by columns - index by header-row columns name

\ ”Sheets” => {\ ”Example” => {\ ”Date” => {...},\ ”Time” => {...}, ...\ }\ }\

Organize by columns - index by header-column rows name

\ ”Sheets” => {\ ”Example” => {\ ”A” => {...},\ ”B” => {...}, ...\ }\ }\

Organize by columns - index by headers

\ ”Sheets” => {\ ”Example” => {\ ”Date” => {...},\ ”Expense” => {...}, ...\ }\ }\

2.20 PGP encryption



Figure 2.20: PGP encryption icon

This action plug-in can be used to encyrpt/decrypt programatically a file using the symantec pgp command line tool.

Version	Comment
0.1.2	Enhanced action template UI
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.2	Added Remote Node Dependency
0.3.1	Adds support for temporary licenses when in Symantec mode
0.3.0	New icon
0.2.9	Prevents auto-complete of user field
0.2.8	Adds display name
0.2.7	adds functionality to mask password
0.2.6	fix inputs_spec
0.2.5	change default naming in configurator
0.2.4	returns execution outputs
0.2.3	outputs edge case fix
0.2.2	accounted for special case of self extracting in the default extensions
0.2.1	added quotation to passphrase, tuned error reporting and made option a payload
0.2.0	added gpg mode
0.1.1	use of module_ssh_tools
0.1.0	added pgp_binary_path and options
0.0.1	initial release

2.20.1 How It Works

- This action plug-in allows you to encrypt or decrypt files programmatically.
- It uses the Symantec PGP Command Line tool for secure encryption and decryption operations.
- Simply provide the file path and required parameters to perform the desired action.
- Ideal for environments where data protection and compliance are critical.

2.20.2 Configuration Instructions

1. Create a new action step using the SymantecPgp plugin.

2. Fill out the following fields:
    - Name: The name used to identify a saved SymantecPgp configured instance.
    - Comments: Comments about this saved SymantecPgp configured instance.
    - Source file path: The source file(s) for the PGP operation. Supports multiple files, comma-separated.
    - Target file path: The destination file(s) for the PGP operation. Supports multiple files, comma-separated.
    - Passphrase: The passphrase to use for the PGP operation.
    - PGP Operation: Defines whether PGP should encrypt or decrypt the source file to the destination file.
    - Timeout: The default timeout for the remote PGP operation. If not specified, the default engine value will be used.
    - Gpg compatibility mode: In this mode, the action operates with behavior and output closely aligned to standard GNU PG (GnuPG) conventions, ensuring interoperability with systems and workflows that rely on traditional GPG.
    - Execution Node: The node where the PGP operation must be executed.
    - Execution Login: The user account used to execute the PGP operation. If not specified, the default user account for the node will be used.
    - Execution Password: The password used to log in to the remote node. If not specified, the default password for the node will be used.
    - Path to the PGP binary: The path to locate the Symantec PGP binary on the remote node. Not needed if Symantec PGP is in the system path.
    - Optional parameters: Any optional parameters supported by the Symantec PGP binary. These options will be passed as they are.

2.20.3 Example Configuration

This is a minimum setup configuration for the SymantecPgp plugin.

Parameter	Value
Source file path	/project/path/secret
Target file path	/home/backup
Passphrase	passphrase
PGP Operation	encrypt
Execution Node	Configured node

2.20.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved SymantecPgp configured instance.
Comments	Comments about this saved SymantecPgp configured instance.
Source file path	The source file(s) for the PGP operation. Supports multiple files, comma-separated.
Target file path	The destination file(s) for the PGP operation. Supports multiple files, comma-separated.
Passphrase	The passphrase to use for the PGP operation.
PGP Operation	Defines whether PGP should encrypt or decrypt the source file to the destination file.
Timeout	The default timeout for the remote PGP operation. If not specified, the default engine value will be used.
Gpg compatibility mode	In this mode, the action operates with behavior and output closely aligned to standard GNU PG (GnuPG) conventions, ensuring interoperability with systems and workflows that rely on traditional GPG.
Execution Node	The node where the PGP operation must be executed.
Execution Login	The user account used to execute the PGP operation. If not specified, the default user account for the node will be used.
Execution Password	The password used to log in to the remote node. If not specified, the default password for the node will be used.
Path to the PGP binary	The path to locate the Symantec PGP binary on the remote node. Not needed if Symantec PGP is in the system path.
Optional parameters	Any optional parameters supported by the Symantec PGP binary. These options will be passed as they are.

2.20.5 Inputs Description

The list of inputs depends on the configuration of the SymantecPgp action template. If dynamic attributes are used in the message definition, each attribute becomes a required input.

Parameter	Description
source_file_path *	The source file(s) for the PGP operation. Supports multiple files, comma-separated.
passphrase *	The passphrase to use for the PGP operation.
options	Any optional parameters supported by the Symantec PGP binary. These options will be passed as they are.
target_file_path	The destination file(s) for the PGP operation. Supports multiple files, comma-separated.

2.20.6 Outputs Description

Parameter	Description
processed_file_path	The path(s) of the file(s) produced by the PGP operation. If multiple files are produced, they are separated by commas.
passphrase	The passphrase to use for the PGP operation.

2.20.7 Supported Actions

None

2.20.8 Dependencies

None

2.20.9 Operating Instructions

This action executes the specified PGP operation on a remote node where Symantec PGP is installed. It enables secure encryption or decryption of a file located on that remote node using the provided passphrase. The process ensures that sensitive data remains protected during transmission and storage. By leveraging Symantec PGP's command-line capabilities, this action supports automated workflows and compliance with security standards.

2.21 Xml Parsing



Figure 2.21: Xml Parsing icon

This action plug-in extracts data from XML documents using XPath expressions. It supports XML input from text strings or file paths, allowing multiple configurable parameters to be exposed as workflow outputs.

Version	Comment
0.1.3	Enhanced action template UI
0.1.2	Fix issues when storing xpaths (Eg://root/node/subnode)
0.1.1	Fix issues when storing certain string characters (Like colons)
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.5	Added raw to parameters field
0.0.4	Ensure file is closed if used as input for the xml
0.0.3	Supports run time inputs for xml and xml file
0.0.2	Fixed bugs from initial version
0.0.1	initial version

2.21.1 How It Works

- Parses XML documents using XPath expressions to extract specific data elements.
- Supports both XML input as a text string or from an external file path.
- Allows configuring multiple parameters, each with its own XPath expression, name, and data type.
- Extracts values based on XPath queries and exposes them as workflow outputs.
- Handles XML documents with or without namespaces using appropriate XPath syntax.
- No limit on the number of output parameters that can be configured.

2.21.2 Configuration Instructions

1. Add an XML Parsing step to your workflow.
2. Configure the XML source:
    - Accept XML as String?: Check to enter XML directly in a text box, uncheck to specify a file path.
    - Xml file as string: Enter XML content directly (if checkbox is checked).
    - Xml file path: Specify path to XML file (if checkbox is unchecked).
3. Add parameters to extract data:
    - Add Parameter: Click to add new extraction parameters.
    - Name: Define the output parameter name for the workflow.
    - XPath: Specify the XPath expression to locate the data.
    - Type: Select the expected data type (String, Array, etc.).

2.21.3 Example Configuration

Parameter	Example Value
Accept XML as String?	unchecked (using file path)
Xml file path	/opt/aspera/orchestrator/xml_test.xml
Parameter 1 - Name	empName
Parameter 1 - XPath	//Employee/name
Parameter 1 - Type	String/Text
Parameter 2 - Name	empRole
Parameter 2 - XPath	//Employee/role
Parameter 2 - Type	String/Text

2.21.4 Saved Parameters Description

Parameter	Description
Accept XML as String?	Checkbox to choose input method: checked = enter XML in text box, unchecked = provide file path.
Xml file as string	Text area to enter XML content directly when 'Accept XML as String?' is checked.
Xml file path	File path to XML document when 'Accept XML as String?' is unchecked.
Add Parameter	Button to add new extraction parameters for defining outputs.

Parameter	Description
Type	Data type for each parameter: specifies expected format of extracted data (String, Array, etc.).
Name	Parameter name that will be exposed as an output in the workflow.
XPath	XPath expression used to parse and extract data from the XML document.

2.21.5 Inputs Description

Usually this plugin does not have runtime specified parameters, unless an external file is used for the configuration, in which case the file path can be specified at runtime.

- Xml file path: File path to XML document (only when external file configuration is used at runtime).

2.21.6 Outputs Description

- Configured Parameters: All specified parameters will be available as outputs based on configuration.
- Example: empName: Employee name extracted using XPath (e.g., 'Alice').
- Example: empRole: Employee role extracted using XPath (e.g., 'Developer').
- Step_information: Processing details including step status, retrieved parameter count, and execution metadata.

2.21.7 Supported Actions

None

2.21.8 Dependencies

- No external dependencies. Uses standard XPath processing capabilities.

2.21.9 Operating Instructions

2.21.10 Overview

This action provides parsed outputs from XML using XPath. There is no limit to the number of outputs and it can be requested by the user at the plugin configuration as Parameters. Each Parameter would need an XPath (the expression used to parse the XML provided), Name (to identify the parsed output), Type (the user should know the type of parsed output expected from the XPath expression).

2.21.11 What is XPath?

- XPath is a syntax for defining parts of an XML document. It uses path expressions to navigate in XML documents, and it contains a library of standard functions.

2.21.12 Basic Example

\<?xml version=”1.0” encoding=”UTF-8”?>\<Employees>\ <Employee id=”3”>\ <age>40</age>\ <name>Tom</name>\ <gender>Male</gender>\ <role>Manager</role>\ </Employee>\ <Employee id=”4”>\ <age>25</age>\ <name>Meghna</name>\ <gender>Female</gender>\ <role>Manager</role>\ </Employee>\</Employees>\

To retrieve the age of all employees in an Array from the above XML, add a Parameter with Type as Array and XPath as:

//age

To retrieve the details of Employee with an attribute of id = 4 in a String from the above XML, add a Parameter with Type as String and XPath as:

//Employee[@id = '4']

2.21.13 Example with Namespaces

\ <feed xmlns=”http://www.w3.org/2005/Atom”>\ <title>Example Feed</title>\ <link href=”http://example.org/”/>\ <updated>2003-12-13T18:30:02Z</updated>\ <author>\ <name>John Doe</name>\ </author>\ <id>urn:uuid:60a76c80-d399-11d9-b93C-0003939e0af6</id>\ </feed>\

To retrieve the name of the author from the above XML, add a Parameter with Type String and XPath as:

//xmlns:author/xmlns:name

2.21.14 Additional Resources

For more information on XPath:

Nokogiri XPath Tutorial

There are even tools to evaluate XPath from an XML. Example:

http://xmltoolbox.appspot.com/xpath_generator.html

Chapter 3

File Transfer

3.1 Ascp client



Figure 3.1: Ascp client icon

This action plugin performs a file transfer upload or download operation to/from a server using Aspera's FASP protocol. In order for it to work, an Aspera client or server product needs to be installed on the Orchestrator server.

Version	Comment
1.6.2	Enhanced action template UI
1.6.1	Removing legacy support
1.6.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.5.2	Removing windows support code
1.5.1	Fixed remote node specification as input
1.5.0	Added cookie
1.4.1	Added ascp error cause in status details
1.4.0	Added option to specify an authentication token
1.3.1	Set display name with correct capitalization
1.3.0	disable management port connection option added
1.2.4	ruby 2.3 converting array to string support
1.2.3	Support both remote node by name or by id (name has precedence)
1.2.2	Calculating percent complete and passing it to report_progress.
1.2.1	Support for a file-list as source for upload operation.
1.2.0	Support for a file-list as source.
1.1.9	Added styling to help text.
1.1.8	Adding RemoteNode Dependency.
1.1.7	fixed problem that was preventing passing the host as a parameter
1.1.6	prevents autocomplete of user in UI
1.1.5	fixed a missing end to a if-end statement which was causing SyntaxError
1.1.4	fixed problem when remote node is specified in the template
1.1.3	fixed run time host specification
1.1.2	run time host can either be an ip address or a RemoteHost name

Version	Comment
1.1.1	removes password from carried thru input
1.1.0	only carry thru inputs if explicitly specified. help file added
1.0.1	returns list of transferred files
1.0.0	uses Aspera process, instead of popen, to isolate environment variable used for the password
0.1.0	specifies the management port
0.0.10	protects against management socket creation failures, change defaults display in UI
0.0.9	improves file path escaping
0.0.8	fixes download command line
0.0.7	fixes issue when non-standard port were configured in the node
0.0.6	adds functionality to mask password
0.0.5	adds help file
0.0.4	fixes escaping, auto converts path on windows install, fix input_spec for ad-hoc nodes
0.0.3	fixes typo in template configurator and add clarifications
0.0.2	modification of failure handling
0.0.1	initial release

3.1.1 How It Works

- Performs high-speed Aspera FASP uploads and downloads between the local Aspera Orchestrator server and remote Aspera servers.
- Supports single and multiple file transfers with comprehensive source file specification options including arrays and file lists.
- Integrates with various Aspera server types including Enterprise Server, Connect Server, and Point-to-Point configurations.
- Provides flexible authentication methods including traditional password-based and token-based authentication for secure transfers.
- Utilizes ascp command-line functionality with full support for all ascp options and parameters for advanced transfer control.
- Includes management connection capabilities for real-time transfer monitoring and status reporting with configurable port ranges.

3.1.2 Configuration Instructions

1. Add an ASCP Client step to your workflow.
2. Configure the remote Aspera server connection:
    - Aspera server node: Select from defined remote nodes with Aspera Server installed.
    - Remote Node details - Server Address: Provide alternate IP address if needed.
    - Remote Node details - Aspera server user/password: Enter credentials if different from node definition.
3. Set up transfer parameters:
    - Operation: Choose Upload or Download from Orchestrator server's perspective.
    - Source files: Specify files using array format or semicolon-delimited list.
    - Destination path: Set target directory (must be directory for multiple files).
4. Configure transfer optimization:
    - Target Rate: Set maximum transfer rate (supports G/M/K suffixes and percentages).
    - Command line options: Add additional ascp options as needed.

3.1.3 Example Configuration

Parameter	Example Value
Aspera server node	aspera-remote-01
Remote Node details - Server Address	192.168.1.100
Remote Node details - ssh port	22
Remote Node details - Aspera server user	aspera_user
Operation	Upload
Source files	['/data/file1.mov', '/data/file2.mov']
Destination path	/remote/uploads/

Parameter	Example Value
Target Rate	100M
Use token authentication	unchecked
Management port base	33001
Management port range	100

3.1.4 Saved Parameters Description

Parameter	Description
Aspera server node	The node with which the local Aspera Orchestrator server will transfer. That node needs to have an Aspera Server installed on it (Aspera Enterprise Server, Aspera Connect or Aspera Point-to-Point).
Remote Node details - Server Address	Alternate remote Aspera Server IP address - will override the node address if provided.
Remote Node details - ssh port	If the remote Aspera Server is not listening on TCP/22 and the node is not configured with the target SSH port, enter the alternate port here.
Remote Node details - Aspera server user	The Aspera user account on the remote side used for the file transfer, if different from the one used in the node definition.
Remote Node details - Aspera server password	The password for the Aspera user account on the remote side used for the file transfer, if different from the one used in the node definition.
Token Authentication - Use token authentication	If selected, token authentication will be used instead of password authentication.
Token Authentication - Token	The token to use for authentication when token authentication is enabled.
Cookie	This is required for a transfer to Faspex to display it as complete within Faspex.
Operation	This is either Upload or Download from the Aspera Orchestrator server's perspective.
File-List as source	Check this if you want to use the ASCP File List option: the path of the file should be given as the source. The file-list is a file that contains the list of source filepaths. This is a good option to use when there are a lot of files to be downloaded or uploaded.
Source files	This input is an array of file names to be transferred. It can be in normal Ruby syntax ['/example/file_1', '/example/file_2'], semicolon delimited list '/example/file_1;/example/file_2', or if file list option is checked, it should be the path of the file list: example/list_file.txt.
Destination path	This is typically a directory. If multiple files are transferred, this must be a directory. Only if you are transferring a single file can this be a file name – keep in mind that the next transfer would overwrite the file.
Target Rate	The target (max) transfer rate. This is in the format used for the –l command-line option. It accepts suffixes 'G/g' for Giga, 'M/m' for Mega, 'K/k' for Kilo, and 'P/p/%' for percentage. Decimals are allowed, e.g. '.1G' for 100 Mbit/s.
Command line options	Other ascp command line options. For more information, see man ascp(1).
Disable Management Connection?	This option is provided to avoid management connection in case of any socket issues.
Management port - Management port base	The starting port number of the range of ports that can be used to open the management socket to monitor the transfer status.

Parameter	Description
Management port - Management port range	The range of ports that can be used starting at the base. The management port will be selected at random between base and base+range.
Legacy support - Carry thru all inputs	If checked, all inputs will also be provided as outputs.

3.1.5 Source Files Format Options

- Normal Ruby syntax: ['/example/file_1', '/example/file_2', '/example/file_3']
- Semicolon delimited list: /example/file_1;/example/file_2;/example/file_3
- File list (when File-List as source is checked): example/list_file.txt

3.1.6 Inputs Description

The list of inputs depends on the configuration of the AscpClient action template as the fields for the paths and options all support dynamic inputs. As well, if values are set in the template, they are not shown when you right-click the step and click Map Inputs. By default, when nothing is set, the following inputs are available:

- Destination_path: The path on the remote side.
- Files: An array of source files to be transferred - a string value would be accepted if the format matches the ones described above.
- Operation: Upload or Download operation type.
- Remote_host: The IP address of the remote system.
- Remote_user: The Aspera user to be used for the transfer.
- Remote_user_password: The password for the Aspera user used for the transfer (only if token authentication not selected).
- Token: The token to use for authentication if token authentication is selected.
- Target_rate: Target rate for the transfer.
- Ssh_port: The SSH port used on the remote system.
- Command_line_options: Any other ascp command line options.

Note: All the required and optional inputs defined in the template body and target file path will also be requested.

3.1.7 Outputs Description

- Session ID: The Aspera session ID for the transfer. This UUID is a string composed of series of hexadecimal characters (e.g., e1e1a704-9080-4a34-a3fa-33d250d90132).
- Transferred files: An array of the local path to the files transferred.
- Transferred bytes: The total number of bytes transferred (as an integer).

3.1.8 Supported Actions

- Upload files from local Orchestrator server to remote Aspera server
- Download files from remote Aspera server to local Orchestrator server
- Single and multiple file transfers
- File list-based transfers for bulk operations
- Transfer monitoring and status reporting

3.1.9 Operating Instructions

- Transfer Types: This action performs an Aspera FASP upload or download between the local Aspera Orchestrator server on which an Aspera client or server is also installed. Single or multiple files can be transferred.
- Command Line Integration: This action supports all ascp command line options, providing full flexibility for advanced transfer scenarios and optimization.

- Debugging Support: Since this action uses the ascp command line behind the scene, it is very useful to access the Aspera log for debugging. That log is typically located in /var/log/messages on the remote server.
- Doc Root Enforcement: Since the transfer is done using Aspera, the doc roots set in place for the Aspera user under which Orchestrator runs will be enforced on the local side. Same applies for the remote user on the remote side.
- Authentication Options: Choose between traditional password authentication or token-based authentication based on your security requirements and server configuration.
- File List Usage: When dealing with large numbers of files, use the File-List option to specify a file containing the list of source file paths rather than listing all files individually.
- Management Connection: The management connection can be disabled if socket issues occur, but this will prevent real-time transfer monitoring capabilities.

3.1.10 Dependencies

- Aspera client or server software installed on the local Orchestrator server.
- Remote Aspera Server (Enterprise Server, Connect, or Point-to-Point) installed and configured on the target node.
- Network connectivity between local and remote systems on the configured SSH port (default 22).
- Valid user accounts and authentication credentials on both local and remote Aspera servers.
- Proper doc root configurations and file system permissions for both source and destination paths.
- Available management ports in the specified range for transfer monitoring (if management connection is enabled).

3.1.11 Troubleshooting

- If transfers fail to start, verify that the remote Aspera server is running and accessible on the specified IP address and SSH port.
- For authentication failures, confirm that user credentials are correct and that the user has appropriate permissions on both local and remote systems.
- If file path errors occur, ensure that source files exist and are accessible, and that destination paths have proper write permissions.
- For token authentication issues, verify that the token is valid and that the remote server supports token-based authentication.
- If management connection fails, try disabling the management connection option or check that the specified port range is available and not blocked by firewalls.
- For transfer rate issues, verify that the target rate format is correct (use suffixes G/M/K for Giga/Mega/Kilo or % for percentage).
- Check Aspera logs at /var/log/messages on the remote server for detailed error information and transfer status.
- Ensure that doc root configurations allow access to both source and destination file paths on respective systems.

3.2 Aspera Node API Transfer



Figure 3.2: Aspera Node API Transfer icon

This action plugin provides the ability to initiate and monitor server-to-server transfers using Aspera's Node API cluster protocol.

Version	Comment
1.3.2	Enhanced action template UI

Version	Comment
1.3.1	Add retry logic for Node API connection failures and timeouts
1.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.2.7	Allows to cancel a transfer
1.2.6	Fixed problem with src_base wrongly spelled
1.2.5	Added completed and failed file list
1.2.4	Added percent_complete support
1.2.3	Support the form hostname:port in node address
1.2.2	Fixes error in computing state in some edge conditions
1.2.1	Detail reasons for error while pulling status
1.2.0	Add configurable path separator character
1.1.0	Add src base option for transfers
1.0.0	Add option for verbose logging
0.9.5	Set http proxy on instance level, not global level
0.9.4	Improved logging
0.9.3	Verify status when using bytes written
0.9.2	Added remote user only if username present
0.9.1	Added remote user in token based auth case
0.9.0	Added FASP and HTTP Proxy URL
0.8.2	Use bytes written to check if transfer complete
0.8.1	Add status details for debug logging
0.8.0	Fixed rate policy default value and exposed at runtime
0.7.0	Add encryption at rest support
0.6.3	Fix issue with remote key path
0.6.2	Added option to force rate policy at runtime
0.6.1	Work around for rate policy not exposed at runtime
0.6.0	Added rate policy to default transfer parameter
0.5.4	adding backward compatibility to partially completed status
0.5.3	Catch REST timeouts and force retry on next polling
0.5.2	Node status change from ”partially completed” to partially_completed fixed.
0.5.1	new output - files transferred
0.5.0	made response hash output optional
0.4.0	adding key based authentication
0.3.0	Changing remote_token column from boolean to type string
0.2.1	Changing code to read port/username/password from configuration instead of remote node.
0.2.0	Adding option to monitor a job's progress based on id, adding exception handling in progress calculation and default to verify ssl as false
0.1.2	Adding realtime info to status details - helpful during long transfers
0.1.1	Adding polling frequency get method definition
0.1.0	Adding Remote Token to the json structure
0.0.1	initial release

3.2.1 How It Works

- Initiates and monitors FASP transfers between Aspera nodes using the Node API for communication and control.
- Supports multiple operation modes: parameter-based transfers, custom JSON payload transfers, and transfer monitoring.
- Handles various authentication methods including SSH keys, access keys/secrets, and Bearer/Basic tokens.
- Works with different Aspera environments: node-to-node transfers, ATS (Aspera Transfer Service), and Aspera On Cloud.
- Provides comprehensive transfer monitoring with configurable polling frequency and detailed status reporting.

3.2.2 Configuration Instructions

1. Add an AsperaNodeApiTransferOperation step to your workflow.
2. Configure the service node (initiating node) parameters:
    - Name: Enter a descriptive name for the transfer operation.
    - Service node: Select configured node or leave empty for manual configuration.
    - Service node IP: IP address or domain name of the initiating node (if Service Node is not provided).

- User login: Node API username or access key for authentication.
- User password: Node API password or secret key for authentication.
- Use ssl: Check for HTTPS (port 9092), uncheck for HTTP (port 9091).

3. Set up transfer operation and remote host details:
   - Operation: Choose 'submit transfer using parameters', 'submit transfer using payload', or 'monitor transfer'.
   - Direction: Select 'Send' or 'Receive' for transfer direction.
   - Remote host: Hostname or IP address of the remote Aspera server.
   - Auth type: Choose authentication method (password, SSH key, or token).
   - Source: Source file paths (use path separator for multiple files).
   - Destination root: Destination directory path.

3.2.3 Example Configuration

Parameter	Example Value
Name	Node Transfer Operation
Service node	transfer-node-01
Use ssl	checked
Operation	submit transfer using parameters
Direction	Send
Remote host	destination.example.com
Auth type	SSH key
Remote user	xfer
Remote key path	/opt/aspera/keys/transfer_key
Source	/data/upload/file1.txt
Destination root	/incoming
Target rate Kbps	100000
Create dir	checked
Polling frequency	10

3.2.4 Saved Parameters Description

Parameter	Description
Name	Display name for this transfer operation step.
Service node	Initiating node configured with IP, port, username, and password for Node API access.
Service node IP	IP address, domain name, or address:port of initiating node (when Service Node not provided).
Service node port	Node API port on initiating node (when Service Node not provided).
User login	Node API user login or access key (when Service Node not provided). Use access key if restrictions configured in aspera.conf.
User password	Node API password or secret key (when Service Node not provided). Use secret key if restrictions configured in aspera.conf.
Use ssl	Enable HTTPS on port 9092, disable for HTTP on port 9091.
Operation	Operation type: 'submit transfer using parameters' (individual parameters), 'submit transfer using payload' (custom JSON), or 'monitor transfer'.
Response hash as output?	Return job status response data. Uncheck for multiple file transfers to avoid large outputs.
Direction	Transfer direction: Send or Receive.
Remote host	Remote host name or IP address (address:port format not supported here).
Auth type	Authentication method: password, SSH key, or token.
Remote access key	Access key for remote host authentication.
Remote secret key	Secret key for remote host authentication.
Remote user	Remote username (default: xfer).

Parameter	Description
Remote password	Password for remote user authentication.
Token	Authentication token for remote host access. Precede with 'Basic' or 'Bearer' based on token type.
Remote key path	Path to private SSH key file on Orchestrator server.
Remote key passphrase	Passphrase for SSH key if configured.
Source root id	ID of source root (only for transfers to Aspera On Cloud).
Destination root id	ID of destination root (only for transfers to Aspera On Cloud).
Destination root	Destination directory file path (not applicable for Aspera On Cloud transfers).
Source	Source file paths separated by path separator character for multiple files.
Path separator	Character used to separate multiple paths (default: semicolon).
Src Base	Source base path applied to all sources above.
Target rate Kbps	Transfer rate in kilobits per second.
Fasp port	FASP UDP port value (default: 33001).
SSH port	FASP SSH port value (default: 33001).
Multi session	Maximum number of simultaneous transfer sessions (default: 1).
Delete source	Delete source files after successful transfer.
Create dir	Create destination directory if it does not exist.
Rate policy	Rate policy for the transfer (e.g., high, fair, low).
EAR operation	Encryption at rest operation: none, encrypt, decrypt, or --Runtime--.
EAR passphrase	Passphrase for encryption at rest operations.
Payload	Custom JSON payload when 'submit transfer using payload' operation is chosen.
FASP Proxy	FASP Forward Proxy setting (format: dnats://node_user:@proxy_addr:9092/).
HTTP Proxy	HTTP proxy for Node API server calls (format: https://proxy:port/).
Verbose logging?	Enable detailed request/response logging to Node API (use only for debugging).
Polling frequency	Frequency in seconds for polling job status during monitoring.

3.2.5 Inputs Description

- Source: Required source file paths for transfer operations.
- Service Node information: Required node connection details (IP, port, credentials) when not using predefined service node.
- Remote Host: Required remote host connection information including authentication details.
- EAR operation: Optional encryption at rest operation when EAR operation is set to --Runtime--.
- Authentication parameters: Required authentication details based on chosen auth type (password, SSH key, or token).

3.2.6 Outputs Description

- Job id: Unique identifier of the initiated transfer.
- Job_Status_Response_Received: Raw response received for job status queries.
- Transfer_Upload_Response_Received: Raw response received for transfer upload operations.
- Transferred file count: Number of files successfully transferred.
- Transferred file list: List of successfully transferred files (destination paths relative to docroot).
- Failed file list: List of files that failed to transfer (destination paths relative to docroot).

3.2.7 Supported Actions

- Submit transfer using parameters
- Submit transfer using payload

- Monitor transfer

3.2.8 Dependencies

- Aspera Transfer Server or Transfer Endpoint with valid license on both service and remote nodes.
- Node API enabled and accessible on the service node.
- Network connectivity between Orchestrator and service node for API calls.
- FASP connectivity between service node and remote host for file transfers.
- The aspweb user must have proper permissions to read SSH key files when using key-based authentication.

3.2.9 Operating Instructions

3.2.10 Transfer Architecture

- Service node: The initiating node where Orchestrator uses Node API to start and monitor transfers.
- Remote host: The destination node serving the transfers using FASP protocol for actual file transfer.
- Protocol flow: Node API for control, FASP protocol for data transfer between nodes.

3.2.11 Custom Payload Examples

3.2.12 Node-to-Node Transfer with SSH Key

\{\ ”direction”:”send”,\ ”remote_user”:”root”,\ ”remote_host”:”testchris5.aspera.cloud”,\ ”ssh_private_key”:””,\ ”destination_root”:”/”,\ ”fasp_port”:33001,\ ”ssh_port”:33001,\ ”target_rate_kbps”:””,\ ”multi_session”:1,\ ”create_dir”:true,\ ”delete_source”:false,\ ”rate_policy”:”high”,\ ”paths”:[{ ”source”: ”/home/aspera_files/tutu.txt” }]\ }\

3.2.13 ATS Upload with Basic Token

\{\ ”direction”: ”send”,\ ”remote_host”: ”w.demo.aspera.org”,\ ”remote_access_key”: ”ABCDEFGH”,\ ”token”: ”Basic ”,\ ”target_rate_kbps”: ””,\ ”create_dir”: false,\ ”ssh_port”: ”33001”,\ ”fasp_port”: ”33001”,\ ”destination_root” : ”/”,\ ”remote_user”: ”xfer”,\ ”multi_session”: ”1”,\ ”delete_source”: ”false”,\ ”paths”: [{ ”source”: ”/temp.mpg” }]\ }\

3.2.14 Aspera On Cloud Package Upload with Bearer Token

\{\ ”direction”: ”send”,\ ”remote_host”: ”testchris5.aspera.cloud”,\ ”token”: ”Bearer ”,\ ”target_rate_kbps”: 1000000,\ ”create_dir”: true,\ ”ssh_port”: ”33001”,\ ”fasp_port”: ”33001”,\ ”delete_source”: ”false”,\ ”paths”: [{ ”source”: ”/data_to_files/sma
”tags”: {\ ”aspera”: {\ ”node”: {\ ”file_id”: ”72”,\ ”access_key”: ”jjgawSlKQfISWpdRnWFOuxArb_QoE4qDX_yXKEmy05k”\ },\ ”files”: {\ ”package_id”: ”BnfkYxtl8Q”,\ ”package_operation”: ”upload”\ }\ }\ }\ }\

3.2.15 Authentication Methods

- SSH Key: Use private key file path and optional passphrase for key-based authentication.
- Access Key/Secret: Use access key and secret key pairs for ATS and cloud environments.
- Token: Use Bearer or Basic tokens for API-based authentication (precede with token type).
- Password: Use username and password for basic authentication.

3.2.16 Runtime Configuration

- EAR operation can be specified as runtime input when set to --Runtime-- in template configuration.
- Source paths support multiple files using the configured path separator character.
- Enable verbose logging only for debugging to prevent crowding log files.
- Response hash output should be disabled for large multi-file transfers to prevent oversized outputs.

3.3 Aspera Out-of-Transfer File Validation

This plugin provides the ability to retrieve the list of files to validate and update the file status during validation.



Figure 3.3: Aspera Out-of-Transfer File Validation icon

Version	Comment
0.1.2	Enhanced action template UI
0.1.1	Updated the label from Keep ongoing to Continue Monitoring
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.1	Initial implementation

3.3.1 How It Works

- Provides Out-of-transfer file validation (OTFV) capabilities for Aspera file transfers by interfacing with Node API to manage file validation workflows.
- Supports two primary operations: retrieving lists of files pending validation and updating file status during validation processes.
- Enables continuous monitoring through configurable polling with validator ID management to prevent simultaneous validation conflicts.
- Integrates with High-Speed Transfer Server (HSTS) to validate files during transfer operations with real-time status updates.
- Provides flexible connection options including remote node selection, SSL support, and customizable API credentials for secure validation workflows.

3.3.2 Configuration Instructions

1. Add an Aspera OTFV step to your workflow.
2. Configure the Node API connection:
   - Node ip address (or hostname): IP address or hostname of the Node API server.
   - Remote node: Select configured remote node or use manual IP address entry.
   - Use remote node?: Check to use remote node instead of manual IP address.
   - Node user: Username for Node API authentication.
   - Node password: Password for Node API authentication.
3. Set up operation parameters:
   - Operation: Choose between 'Retrieve list of files' or 'Update file status'.
   - Validator id: Unique ID to prevent simultaneous validation conflicts.
   - Max result: Maximum number of files to collect per validation request.
   - Continue Monitoring: Enable continuous polling for file validation.

3.3.3 Example Configuration

Parameter	Example Value
Node ip address (or hostname)	192.168.1.100
Use remote node?	unchecked
Use ssl?	checked
Node port	9092
Node user	aspera_validator
Operation	Retrieve list of files

Parameter	Example Value
Validator id	validator_001
Max result	50
Continue Monitoring	checked
Polling frequency	30
File status	validating

3.3.4 Saved Parameters Description

Parameter	Description
Node ip address (or hostname)	The IP address (or hostname) of Node API.
Remote node	The remote node from which to get the IP address (or hostname) of NodeAPI.
Use remote node?	If selected, 'Remote node' is used instead of 'Node ip address'.
Use ssl?	If selected, SSL will be used to connect to Node API.
Node port	The port of Node API.
Node user	The user of Node API.
Node password	The password of Node API.
Operation	The operation to execute.
Validator id	The unique ID to prevent simultaneous validation of the same file by different validators.
Max result	The maximum number of collected files for validation by each request.
Continue Monitoring	If selected, trigger will be executed every time files will be detected for validation.
Polling frequency	The interval in seconds to check for files to be validated.
File status	The file status to set.
Session uuid	The FASP session ID.
File id	The file ID.
Bytes processed	The number of bytes processed.
Error code	The custom error code.
Error descrition	The custom error description.

3.3.5 Inputs Description

3.3.6 Mandatory Inputs for 'Retrieve list of files' Operation

- node_ip_address: The IP address (or hostname) of Node API if 'Use remote node?' is not selected in configuration.
- remote_node: The remote node from which to get the IP address (or hostname) of NodeAPI if 'Use remote node?' is selected in configuration.
- node_user: The user of Node API.
- node_password: The password of Node API.

3.3.7 Mandatory Inputs for 'Update file status' Operation

- node_ip_address: The IP address (or hostname) of Node API if 'Use remote node?' is not selected in configuration.
- remote_node: The remote node from which to get the IP address (or hostname) of NodeAPI if 'Use remote node?' is selected in configuration.
- node_user: The user of Node API.
- node_password: The password of Node API.
- File status: The file status to set if 'file status' value is '--runtime--'.
- session_uuid: The FASP session ID.
- file_id: The file ID.

3.3.8 Optional Inputs for 'Retrieve list of files' Operation

- node_port: The port of Node API (default is '9092' if 'Use ssl?' is selected or '9091' if 'Use ssl?' is not selected).
- validator_id: The unique ID to prevent simultaneous validation of the same file by different validators (default is 'orchestrator').
- max_result: The maximum number of collected files for validation by each request (default is '20').
- polling_frequency: The interval in seconds to check for files to be validated if 'Continue Monitoring' is selected (default is '10').

3.3.9 Optional Inputs for 'Update file status' Operation

- node_port: The port of Node API (default is '9092' if 'Use ssl?' is selected or '9091' if 'Use ssl?' is not selected).
- validator_id: The unique ID to prevent simultaneous validation of the same file by different validators (default is 'orchestrator').
- bytes_processed: The number of bytes processed if 'file status' value is 'validating' or '--runtime--' (default is '0').
- error_code: The custom error code if 'file status' value is 'error' or '--runtime--' (default is '1').
- error_decription: The custom error description if 'file status' value is 'error' or '--runtime--' (default is 'error').

3.3.10 Outputs Description

- result_count: The number of collected files for validation if 'Retrieve list of files' operation is selected.
- file_transfer_info: The list of files to be validated if 'Retrieve list of files' operation is selected.

3.3.11 Operating Instructions

- OTFV (Out-of-transfer file validation) needs HSTS (High-Speed Transfer Server) to be configured for proper operation. See the dedicated topic in the High-Speed Transfer Server admin guide.
- The plugin supports two main operations: 'Retrieve list of files' for getting files pending validation, and 'Update file status' for updating the validation status of specific files.
- When using 'Continue Monitoring' option, the plugin will continuously poll for files at the specified polling frequency interval.
- Validator ID is crucial for preventing conflicts when multiple validators are processing the same files simultaneously.
- SSL connection is recommended for secure communication with the Node API, which uses port 9092 by default (9091 for non-SSL).
- File status values include: 'to_be_validated', 'validating', 'error', and 'complete' depending on the validation workflow state.

3.3.12 Dependencies

- High-Speed Transfer Server (HSTS) must be properly configured for OTFV functionality.
- Node API service must be accessible and running on the specified server with appropriate credentials.
- Network connectivity between Orchestrator and the Node API server is required.
- The aspweb user must have proper permissions for API communication, as Orchestrator runs as aspweb instead of root.

3.4 Console Smart Transfer

This action plugin provides the ability to initiate Smart Transfer templates in Aspera Console or to list available templates.

Version	Comment
0.2.1	Enhanced action template UI
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.5	Verify SSL set to false on polling
0.1.4	Verify SSL set to false
0.1.3	Added new status of willretry to the check status method
0.1.2	Adding timestamp to the status details
0.1.1	Adding submitted status to the check_status check
0.1.0	Add Monitor status operation

Version	Comment
0.0.3	Fixing source path list from string to Array
0.0.2	Fixing password field type as password
0.0.1	Initial Commit

3.4.1 How It Works

- Connects to Aspera Console to invoke Smart Transfer templates with configurable transfer parameters.
- Supports two main operations: listing available Smart Transfer templates and submitting transfer requests.
- Provides asynchronous status monitoring with configurable polling frequency for submitted transfers.
- Handles source file specification with base path support for flexible file selection.
- Returns lists of templates, transfer IDs, and error information through structured output variables.

3.4.2 Configuration Instructions

1. Add a ConsoleSmartTransferOperation step to your workflow.
2. Configure the essential Console connection parameters:
    - Name: Enter a descriptive name for the step.
    - Console IP: IP address of the Aspera Console node (e.g., '192.168.1.100').
    - Console port: Console port number (default is 443).
    - Login: Username to access the console service.
    - Password: Password for console authentication.
3. Set up the operation and transfer details:
    - Operation: Choose either 'List the Templates' or 'Submit the Transfer'.
    - Smart Transfer ID: The ID of the template to invoke (required for Submit operation).
    - Transfer Name: Descriptive name for the transfer.
    - Monitor Status: Check to enable asynchronous status monitoring.

3.4.3 Example Configuration

Parameter	Example Value
Name	Smart Transfer Operation
Console IP	192.168.1.100
Console port	443
Login	console_user
Password	********
Transfer Name	Data Backup Transfer
Transfer Comments	Daily backup operation
Source Path List	/data/backup/*
Source Base	/data
Monitor Status	checked
Smart Transfer ID	template_001
Operation	Submit the Transfer

3.4.4 Saved Parameters Description

Parameter	Description
Name	Display name for this console smart transfer operation step.
Console IP	IP address of the Aspera Console node to connect to.
Console port	Port number for console connection (default: 443).
Login	Username for accessing the console service.
Password	Password for console service authentication.
Transfer Name	Descriptive name to assign to the transfer operation.

Parameter	Description
Transfer Comments	Optional comments to associate with the transfer.
Source Path List	List of source files/paths for the transfer operation.
Source Base	Common base path to fetch source files from.
Monitor Status	Enable asynchronous status monitoring for submitted transfers.
Smart Transfer ID	Unique identifier of the Smart Transfer template to invoke.
Operation	Operation type: 'List the Templates' to retrieve available templates, or 'Submit the Transfer' to initiate a transfer.

3.4.5 Inputs Description

3.4.6 Required Inputs:

- login: Console service username (required for both operations).
- password: Console service password (required for both operations).
- IP: Console IP address (required for both operations).
- smart transfer ID: Template ID to invoke (required for Submit Transfer operation).

3.4.7 Optional Inputs:

- polling frequency: Status monitoring frequency interval in seconds (default: 10). Used when Monitor Status option is enabled.

3.4.8 Outputs Description

- Smart_Template_List: Array containing the list of Smart Transfer templates available on the console.
- Transfer_Ids: Array containing transfer IDs after initiating transfers (when applicable).
- Error_Message: Error details and exception messages when operations fail.

3.4.9 Supported Actions

- List the Templates
- Submit the Transfer

3.4.10 Dependencies

- Access to Aspera Console with valid authentication credentials.
- Network connectivity to the specified Console IP address and port.
- Smart Transfer templates must be configured and available on the Console.
- Proper permissions to execute the selected Smart Transfer templates.

3.4.11 Operating Instructions

3.4.12 Important Limitations

NOTE: According to the Console guide, it is important to note that if a transfer does not allow the user to select the source items at start, then including a ”source” section will result in an error being returned when using this call.

Reference: Refer to the Aspera Developer Network documentation for more detailed information about Smart Transfers.

3.5 FASP controller

This action plug-in provides the ability to control and modify the parameters of running transfers using Aspera's FASP protocol.



Figure 3.4: Console Smart Transfer icon



Figure 3.5: FASP controller icon

Version	Comment
0.7.1	Enhanced action template UI
0.7.0	No longer support ruby 1.8, Updating to be compatible with Ruby 3.4 and Rails 8
0.6.1	Fix for queryParams variables type as array
0.6.0	Adding new columns for max transfer and to get only failed files. Fixing bug when central address is given at runtime
0.5.5	Fixing gem name in dependencies. Adding more logging for session monitoring
0.5.4	fixing WSDL files
0.5.3	removed soap4r dependency
0.5.2	Fixed adjust rate to continue when one adjust rate call fails
0.5.1	Fixed issue when no session found
0.5.0	Adding option to adjust rate
0.4.0	Adding option to return all inprogress session ids
0.3.4	Returning percent_complete from check_status method to be consumed by the activities screen.
0.3.3	Added mkdir for cached WSDL directory for monitor transfer session and monitor transfer job options.
0.3.2	Added Remote Node Dependency
0.3.1	Added help file
0.3.0	- ML - 2015/06/05 - Adds support for authenticated central requests thru node API
0.2.1	- PM - 2015/02/18 - Typo in the migration leading to column not being created.
0.2.0	- PM - 2015/02/18 - Making the plugin asynchronous when keep on going is checked.
0.1.4	- PM - 2015/02/18 - Defaulting MaxReturns to 100K for every poll
0.1.3	Improved exception handling and logging
0.1.2	Improvements to plugin - option to wait until the transfer is finished
0.1.1	Removing reference to ”fasp_controllers” in WSDL_CACHE path which was causing file not found errors
0.1.0	Adding session and job monitoring
0.0.1	Initial release, only supports cancel

3.5.1 How It Works

- Controls and monitors active Aspera transfer sessions through Central server API and Node API interfaces.
- Supports multiple control operations: cancel sessions, monitor transfers, find all sessions, and adjust transfer rates.

- Provides real-time session monitoring with configurable polling frequency and wait-until-completion options.
- Works with Enterprise Server, Connect Server, Faspex Server, and Point to Point environments.
- Returns comprehensive session information including file lists, transfer status, and rate details.

3.5.2 Configuration Instructions

1. Add a FaspControl step to your workflow.
2. Configure the basic connection parameters:
   - Name: Enter a descriptive name for the control operation.
   - Command: Choose operation (Cancel session, Monitor transfer session, Monitor transfer job, find all sessions, Adjust_rate set).
   - Controlled Aspera node: Select the remote node to monitor or control.
   - Central address: IP address or hostname of the Aspera Transfer server.
   - Node user: Node API username for authentication.
   - Node password: Node API password for authentication.
3. Set up monitoring and control options:
   - Wait until end: Check to keep step in progress until session/job completion.
   - Polling frequency: Interval in seconds for status polling (default: 5 seconds).
   - Session Ids: Specify session IDs to control (optional if using transfer conditions).
   - Transfer Rate: Set transfer rate in kbps for Adjust rate operations.

3.5.3 Example Configuration

Parameter	Example Value
Name	Transfer Session Monitor
Comments	Monitors active transfer sessions
Command	Monitor transfer session
Wait until end	checked
Polling frequency	5
Controlled Aspera node	transfer-server-01
Central address	192.168.1.100
Node user	node_api_user
Node password	********
Transfer Rate	100000
Shared Rate?	unchecked

3.5.4 Saved Parameters Description

Parameter	Description
Name	Display name for this FaspControl step.
Comments	Notes or description for the FaspControl template.
Command	Operation to execute: Cancel session, Monitor transfer session, Monitor transfer job, find all sessions, Adjust_rate set.
Wait until end	Keep step in progress as long as session and job are in progress.
Polling frequency	Interval in seconds for polling Central server for session and job status (default: 5 seconds).
Controlled Aspera node	Remote node to connect to for monitoring or canceling Aspera transfers.
Central address	IP address or hostname of the Aspera Transfer server (format: lala.com or 10.0.200.202).
Service uri root	Optional SOAP URI part of WSDL URL to Central server (defaults to expected value).
Node user	Node API username required for authentication.
Node password	Node API password required for authentication.
Transfer Rate	Transfer rate in kbps for Adjust rate operations.

Parameter	Description
Shared Rate?	Enable to share the rate across all sessions.
Session Ids	Specific session IDs to control (use instead of transfer conditions if specified).

3.5.5 Inputs Description

All parameters are available as runtime inputs if not specified in the template. This plugin does not support value overwrites: template values cannot be overridden at runtime.

- Command: Control operation to execute on transfer sessions.
- Central_address: IP address or hostname of Aspera Transfer server.
- Node_user: Node API username for authentication.
- Node_password: Node API password for authentication.
- Session_Ids: Specific session IDs to target for control operations.
- Transfer_Rate: Transfer rate in kbps for rate adjustment operations.
- Polling_frequency: Status polling interval in seconds.
- Wait_until_end: Boolean flag to wait for session/job completion.

3.5.6 Outputs Description

- Session_file_list: Array of files to be transferred in the session.
- Transferred_file_list: Array of files successfully transferred in the session.
- No_of_files_transferred: Number of files successfully transferred in the session.
- Growing_file_list: Array of files currently in progress during Orchestrator polling.
- Session_status: Session status obtained via API call: Complete, Failed, Error, or Stopped.
- Transfer_Rate_Set: Transfer rate set for each identified session.

3.5.7 Supported Actions

- Cancel session
- Monitor transfer session
- Monitor transfer job
- Find all sessions
- Adjust_rate set

3.5.8 Dependencies

- Aspera Enterprise Server, Connect Server, Faspex Server, or Point to Point with valid license.
- Aspera Central API must be enabled with persistent store on the target server.
- Node API must be enabled and accessible on the controlled Aspera node.
- Network connectivity to Central server (port 40001) and Node API ports.

3.5.9 Operating Instructions

3.5.10 Plugin Overview

This plugin enables monitoring and control of Aspera transfer sessions. Sessions can be monitored for file transfer progress or cancelled as needed. Supply session IDs at runtime and choose commands to perform on those sessions.

3.5.11 Aspera Central API Setup

Works with Enterprise Server, Connect Server, Faspex Server, and Point to Point as they all use Aspera Central to expose transfer statuses.

Required Configuration: Enable reliable API on the Aspera Server with IP address polling:

\ <central_server>\ <address>0.0.0.0</address>\ <port>40001</port>\ <persistent_store>enable</persistent_store>\ </central_server>\

3.5.12 Node API Setup

Remote nodes must run Aspera Enterprise Server, Connect Server, or Point to Point with valid license and asperanoded service.

3.5.13 Node API Account Configuration

Node username is a Node API account (not an OS account). Find the associated transfer user:

\ # /opt/aspera/bin/asnodeadmin -l\ List of node user(s):\ user system/transfer user acls\ ========== ========== ==========\ node_user xfer []\

3.5.14 Transfer User Configuration

The transfer user docroot is defined in `/opt/aspera/etc/aspera.conf`:

\ <user>\ <name>xfer</name>\ <file_system>\ <access>\ <paths>\ <path>\ <absolute>/home/xfer/data</absolute>\ </path>\ </paths>\ </access>\ </file_system>\ </user>\

3.5.15 Port Configuration

Enable both HTTP and HTTPS for Node API in `/opt/aspera/etc/aspera.conf`:

\ <server>\ <server_name>test.sl.dev.asperacloud.net</server_name>\ <http_port>9091</http_port>\ <https_port>9092</https_port>\ <enable_http>true</enable_http>\ <enable_https>true</enable_https>\ </server>\

Restart asperanoded service to apply the configuration changes:

\ service asperanoded restart\

3.5.16 Command Operations

- Cancel session: Terminates active transfer sessions using provided session IDs.
- Monitor transfer session: Tracks session progress and provides file transfer status.
- Monitor transfer job: Monitors job-level transfer status and completion.
- Find all sessions: Discovers and lists all active transfer sessions.
- Adjust_rate set: Modifies transfer rate for specified sessions, with optional rate sharing.

3.5.17 Session Management

- Session IDs can be provided at runtime to target specific transfers for control operations.
- The wait until end option keeps the workflow step active until monitored sessions are complete.
- Polling frequency determines how often the plugin checks session status (default: 5 seconds).
- Transfer rate adjustments can be applied to individual sessions or shared across all sessions.

Reference: More information about Aspera Node API is available on the Aspera Developer Network.

3.6 FASP transfer

This plugin provides the ability to initiate server-to-server transfers using Aspera's FASP protocol.

Version	Comment
4.3.5	Enhanced action template UI
4.3.4	Fix bug when passing no node credentials
4.3.3	Fix decryption bug when setting node password
4.3.2	Fix problem when checking transfer status
4.3.1	Parse faspex links before requesting the token to faspex
4.3.0	No longer supporting ruby 1.8, Updating to be compatible with Ruby 3.4 and Rails 8
4.2.2	Support for validating and validation states
4.2.1	Fixed wrong database migration file name
4.2.0	Fixed wrong payload formatting for pull and push from url payloads

Version	Comment
4.1.0	Added No Mgmt Port Files option to generic, pull from url default and push from url default payloads. Default value is No.
4.0.0	Added Proxy options for pull and push from url
3.9.0	Added Key Passphrase option to generic payload. Default is ''.
3.8.0	Fixing fasp_push_with_keys payload.
3.7.2	Update help page.
3.7.1	Changing default value of password to nil.
3.7.0	Add new payloads.
3.6.0	Added Tags option to generic payload. Default is {}.
3.5.0	Added Pre Calculate Job Size option to generic payload. Default is No.
3.4.0	Added File Checksum option to generic payload. Default is None.
3.3.1	Prevents crash when average bandwidth can not be calculated (due to 0.0 / 0 -> NaN)
3.3.0	Add split separator
3.2.4	Correct issue with interpretation of , or ; characters in faspex package title
3.2.3	Expose Average bandwidth as an additional output
3.2.2	Make sure the same ip address is used when the controlled node is clustered with round robin
3.2.1	Fixes default value for remove directory when Faspex download
3.2.0	Allows to download Faspex pacjage without parent folder
3.1.8	Verify ssl fix for getting transfer stats
3.1.7	Minor fix, correcting dependency gem
3.1.6	Fixing WSDL files
3.1.5	Verify SSL issue fix
3.1.4	Making plugin compatible with 3.0. Also checking if job_id is present in check_status
3.1.3	Removed soap4r dependency
3.1.2	Fixes a warning in Ruby code
3.1.1	Fixes a bug in the step configuration where overwrite dropdown values were not persisting. Adding more logging on transfers
3.1.0	Offers an explicit switch for the XML escaping mechanism, default is to try all 3 as previously done
3.0.1	Returns polling frequency on execute to avoid polling for status right away (if on 2.7.0)
3.0.0	Added remove after transfer on faspex upload
2.9.2	Loosen the validation of target variable to allow arrays
2.9.1	Increased http timeout value and fixed the looping in get token
2.9.0	Fixed crypt operation reference when uploading/downloading to URL. Added retry when getting token
2.8.6	Returning percent_complete from check_status method to be consumed by the activities screen.
2.8.5	Changes default minimum rate to 0 (min rate is a bad thing to set as it interferes with VLINKs)
2.8.4	Fixes status polling to avoid using iteration token, which hides session in some circumstances
2.8.3	Fixes status polling to avoid using iteration token, which hides session in some circumstances
2.8.2	Fixes formatting of status message to keep order of stats consistent
2.8.1	Modifies timeout logic to expire if same status is reported for longer than calculated timeout
2.8.0	Adds Node API call to fetch files statues and extract the file list that failed to transfer.
2.7.9	SS Corrected the proxy tag for fasp_generic_with_proxy payload.
2.7.8	Adding Remote Node Dependency
2.7.7	Exposes allowed values for overwrite field in a pull down selector
2.7.6	Supports run-time node user/password for authenticated requests
2.7.5	Removing widgetdata tags from Payloads
2.7.4	Updating help file with contents from ADN
2.7.3	Adding a new payload - Generic Fasp Transfers with proxy support.
2.7.2	Bug fix for change introduced in 2.6.1 - the exclusion pattern input must not be mandatory runtime input.
2.7.1	Fixed problem when specifing a target rate different than the one configured in faspex
2.7.0	Exposing faspex relative url (/aspera/faspex by default, it can be /faspex in same cases). Republishing all the wsdl files.
2.6.1	Exposing exclusion pattern (only one for now)
2.6.0	Exposing crypt operation and password on all url based payloads
2.5.6	Fix support for authenticated transfer via node
2.5.5	Including the Error code and Error message for the job to outputs and Jobstats
2.5.4	Adding the percentage complete (as one of the parameters for jobStatus) for in progress file transfers
2.5.3	Adding missing ERB templates for WSDL generation under wsdl directory
2.5.2	Adding MaxResults to getSessionFileList
2.5.1	Added payload to support node as a target, fix generation of WSDL (missing erb file)

Version	Comment
2.5.0	Fixes fasp push to url payload when referencing the private key
2.4.0	Adds support for authenticated request via Node
2.3.4	New icon
2.3.3	Fixes stats reporting to avoid reset at every check_status
2.3.2	Fixes XML escape to only apply to XML field values.”
2.3.1	Escape for XML safety the inputs before expanding them into XML payload to avoid malformed message when &, < or > are used
2.3.0	Fixing issue with private-key use when using URL
2.2.9	Adding a debug statement to inform sessionID for the jobId. Fixing a bug in the UI which caused the template to default to fasp_generic when payload is present.
2.2.8	Adds Token authentication methods
2.2.7	Protects again empty optional values. Added remove empty folder. Added options to faspex transfer.
2.2.6	Adds pull-down for encryption cipher field
2.2.5	Make support for old and new Faspex RSS APIs more generic
2.2.4	Fixed problem with new fasp url produced by Faspex RSS APIs
2.2.3	Fixed problem when files include &
2.2.2	Fixed regression introduced in 2.2.0 for correct connect server address in payload
2.2.1	Fixed faspex url parsing in case of embedded credentials having a '@' in their encoded form
2.2.0	Added faspex_host variable to all payloads interacting with Faspex
2.1.3	Fixes bug preventing the md5 checksum from being returned
2.1.2	Fixes typo in generic payload introduced in 2.1.1
2.1.1	Added support for source base in generic payload
2.1.0	Added Overwrite option to default payload. Default is Diff.
2.0.9	Make Manifest Policy configurable in fasp generic
2.0.8	Avoid exiting with Error in case of HTTP timeout when getting the session status info
2.0.7	Fixes 2.0.6 bug
2.0.6	Restoring ability to provide central IP at realtime (warning: bug - use localhost instead of specified node)
2.0.5	Add retry on http timeout during get status
2.0.4	Use selector for create_path values
2.0.3	Provide Job ID as an output
2.0.2	Attempt job submission with and without payload escaping
2.0.1	Adds some fasp url parsing function to support standalone fasp_url payloads
2.0.0	Makes asynchronous and adds ability to cancel transfers
1.5.0	Fixes expansion in pull payloads
1.4.9	Avoids special characters in generated path
1.4.7	Default operation to Push, fix remove after transfer default
1.4.6	Fixed nil when accessing outputs (introduced in 1.4.4) and default values for RemoveAfterTransfer and LocalUserName
1.4.5	Added RemoveAfterTransfer and Local User Name
1.4.4	Added Checksum as output param It could be nil
1.4.3	Escape file paths to avoid xml parsing errors in Central
1.4.2	Expose the central node as standard input
1.4.1	Bug fix for encryption at rest
1.4.0	Auto-migrate modified payload to support encryption from URL and generic, fixes protection tag in payloads
1.3.4	Clean up double defined methods
1.3.3	Change all runtime payload referennce to FASP_PAYLOADS_DIR
1.3.2	Supports fasp URL with encryption of rest specification
1.3.1	Uses defaulted values in the UI, prevents crash if payload not present
1.3.0	Adds option to lock-down saved inputs (i.e. not shown as optional)
1.2.0	Fixes issue with saved inputs
1.1.0	Splits config and runtime
1.0.2	Fixes issue in reporting of stats when a session is restarted
1.0.1	Adds default pull/push payload templates and modified multi-files splitting conditions.
1.0.0	Uses reliable API instead of legacy call-back based.
0.0.7	Adds retrieval of faspex account from fasp URL if provided (requires FaspexInboxWatcher >= 0.4.0)
0.0.6	Fixes typo in authentication error reporting
0.0.5	Updates to GUI's saved input features, generated wsdl is based on runtime address
0.0.4	Exposes central address as possible run-time parameter
0.0.3	Adds capability to generate faspex token

Version	Comment

3.6.1 How It Works

- Initiates and monitors high-speed file transfers using Aspera's FASP protocol through remote Aspera nodes with comprehensive transfer control and monitoring capabilities.
- Supports multiple Aspera server types including Enterprise Server, Connect Server, Faspex Server, and Point-to-Point configurations for flexible deployment scenarios.
- Utilizes Aspera Central's SOAP API for transfer management, requiring proper configuration to accept external node connections and reliable query API enablement.
- Provides customizable transfer payloads using embedded Ruby templates or predefined XML configurations for precise transfer parameter control.
- Enables real-time transfer monitoring with detailed statistics, file lists, MD5 checksums, and comprehensive error reporting for failed transfers.

3.6.2 Configuration Instructions

1. Add a Fasp Transfer step to your workflow.
2. Configure the payload settings:
   - Payload File: Select from available template files in the fasp_transfers directory.
   - Payload: Write custom embedded Ruby payload code (overrides file selection).
   - Split separator: Define file separator for file lists (comma by default).
3. Set up transfer parameters through payload configuration:
   - Define operation type (Push or Pull) for upload or download transfers.
   - Configure remote and local system details including authentication.
   - Specify file paths, transfer rates, and security parameters.
   - Set retry policies and monitoring preferences.

3.6.3 Example Configuration

Parameter	Example Value
Payload File	fasp_pull_default.xml.erb
Payload	(custom Ruby code if needed)
Split separator	,
Operation (in payload)	Pull
Target Rate (in payload)	45000
Remote Address (in payload)	10.20.13.102
Remote User (in payload)	username
Local Path (in payload)	/opt/localfiles/
Remote Path (in payload)	/home/user/files/

3.6.4 Saved Parameters Description

Parameter	Description
Payload File	The template payload file to be filled in and used. These files are stored in the /opt/aspera/orchestrator/config/orchestrator/actions/fasp_transfers directory. Files added to this directory can be seen in the dropdown after pressing the 'refresh' button next to the selector.
Payload	A custom payload can be written here. The payload should be written as embedded Ruby. The selected Payload file will be ignored if this field is filled in.
Split separator	The file separator when a file list as a string is passed to the plugin.

3.6.5 Inputs Description

The inputs depend on the configuration of the FaspTransfer action template and the payload being used.



Figure 3.6: FASP transfer icon

- payload_variables: Various runtime inputs based on the embedded Ruby payload configuration (e.g., remote_address, remote_user, local_paths, file_lists).
- operation: Transfer operation type: Push (upload) or Pull (download).
- file_paths: Source and destination file paths for the transfer operation.
- authentication: Remote system authentication credentials (username, password, or SSH keys).
- transfer_parameters: Rate control, retry policies, and other transfer-specific settings.

3.6.6 Outputs Description

- Job_ID: The ID of the Fasp Session.
- Transfer_Stats: The Fasp Session statistics including transfer rates, duration, and completion status.
- Transferred_File_list: The files transferred by this session with their status and details.
- Transferred_Files_md5: For each file, the computed MD5 checksum for integrity verification.

3.6.7 Job Order Description (Parameters)

3.6.8 Example Job Order

This example describes a Job Order (in this case a pull operation) in XML format.

XML Format

```
<Order>
    <Version>2</Version>
    <Agent>aspera.sdk.sample</Agent>
    <ApplicationData>
        <Cookie>Sample-Appl-jZUVvgLruAbKKebM4cJb_Ff6CSGqcW</Cookie>
        <Token>${ACCESS_TOKEN}</Token>
    </ApplicationData>
    <Operation>Pull</Operation>
    <!-- other options: Push -->
    <RemoteLocation>
        <System>
            <Address>10.20.13.102</Address>
            <Port>22</Port>
            <UserName>user</UserName>
            <Password>${PASSWORD}</Password>
        </System>
        <Authentication>
            <Methods>
                <Method>Password</Method>
                <Method>Public Key</Method>
            </Methods>
            <KeyPath>/home/userhome/.ssh/key</KeyPath>
            <KeyPassphrase>${PASSWORD}</KeyPassphrase>
        </Authentication>
        <Files>
```

```
        <Path>/home/user/file</Path>
    </Files>
    <Proxy>
        <Url>dnat://proxyaddr</Url>
        <!-- or dnat://proxyUser:<YOUR_PASSWORD>@proxyaddr -->
        <User>proxyUser</User>
        <Password>${PASSWORD}</Password>
    </Proxy>
</RemoteLocation>
<LocalLocation>
    <System>
        <UserName>localUser</UserName>
        <Password>${PASSWORD}</Password>
    </System>
    <Files>
        <Path>/opt/localfiles/file</Path>
    </Files>
</LocalLocation>
<FileParameters>
    <CreatePath>Yes</CreatePath>
    <!-- other options: No -->
    <ResumeCheck>Sparse Checksum</ResumeCheck>
    <!-- other options: None, Full Checksum, Attributes -->
    <ManifestPath>/opt/tmp/manifests</ManifestPath>
    <Protection>Encrypt</Protection>
    <!-- other options: None, Decrypt -->
    <Passphrase>${PASSWORD}</Passphrase>
    <PreCalculateJobSize>Yes</PreCalculateJobSize>
    <!-- other options: No -->
    <RemoveAfterTransfer>No</RemoveAfterTransfer>
    <!-- other options: Yes -->
    <RemoveEmptyDirectories>No</RemoveEmptyDirectories>
    <!-- other options: Yes -->
    <SymbolicLinks>skip</SymbolicLinks>
    <!-- other options: follow, copy, copy+force -->
    <PreserveTimestamps>Yes</PreserveTimestamps>
    <!-- other options: No -->
    <Overwrite>Diff</Overwrite>
    <!-- other options: Always, Never, Older, DiffAndOlder -->
    <Exclude>
        <Pattern>*.pdf</Pattern>
    </Exclude>
    <SourceBase>src/dir</SourceBase>
</FileParameters>
<ChannelParameters>
    <Port>33001</Port>
    <DatagramSize>1500</DatagramSize>
    <AutoDetectPathMTU>Yes</AutoDetectPathMTU>
    <!-- other options: No -->
</ChannelParameters>
<LinkParameters>
    <RemoteCapacity>2000000</RemoteCapacity>
    <LocalCapacity>2000000</LocalCapacity>
    <AutoDetectCapacity>No</AutoDetectCapacity>
    <!-- other options: Yes -->
</LinkParameters>
<RateParameters>
    <Policy>Fair</Policy>
    <!-- other options: Fixed, Trickle -->
    <TargetRate>45000</TargetRate>
```

```
        <TargetRateAsPercent>No</TargetRateAsPercent>
        <!-- other options: Yes -->
        <MinimumRate>0</MinimumRate>
        <MinimumRateAsPercent>No</MinimumRateAsPercent>
        <!-- other options: Yes -->
        <BandwidthCap>2000000</BandwidthCap>
        <Priority>2</Priority>
        <!-- regular priority. Other option: 1 (higher priority)-->
    </RateParameters>
    <SecurityParameters>
        <EncryptionCipher>AES-128</EncryptionCipher>
        <!-- other options: None -->
    </SecurityParameters>
    <RetryParameters>
        <Count>3</Count>
        <BaseInterval>5</BaseInterval>
        <MaximumInterval>10</MaximumInterval>
    </RetryParameters>
</Order>
```

3.6.9 Operating Instructions

- This action allows starting and monitoring Fasp Transfers using remote Aspera nodes. The plugin uses Aspera Central to initiate and control transfers, so Aspera Central must be configured to listen and accept connections from external nodes.
- For Aspera releases < 3.4, the Aspera Node needs to have Reliable Query API enabled in aspera.conf with proper central_server configuration including address (0.0.0.0), port (40001), and persistent_store enabled.
- The plugin works with Enterprise Server, Connect Server, Faspex Server, and Point-to-Point configurations. For complete setup information and Central SOAP API understanding, visit the Aspera Developer Network.
- To retrieve failed transfer file lists, node user and password must be provided, and ascp version should be 3.6.1 or above for full functionality.
- Configure the split separator to 'None' when passing single files or directories with special characters to avoid confusion with file list separators (comma by default).
- Payload files are stored in `/opt/aspera/orchestrator/config/orchestrator/actions/fasp_transfers` and can be customized using embedded Ruby for dynamic transfer configurations.

3.6.10 Dependencies

- Aspera Central SOAP API must be accessible and configured to accept external node connections.
- Reliable Query API must be enabled on Aspera servers for transfer monitoring (required for versions < 3.4).
- Network connectivity between Orchestrator and Aspera transfer nodes with appropriate firewall configurations.
- Valid authentication credentials with sufficient permissions for transfer operations on target Aspera systems.
- ASCP version 3.6.1 or above for comprehensive failed file reporting capabilities.
- The aspweb user must have proper file permissions for payload files and any local transfer paths, as Orchestrator runs as aspweb instead of root.

3.6.11 Troubleshooting

- If workflows break after plugin upgrades/downgrades, copy all files from `/opt/aspera/orchestrator/config/orchestrator/actions/fasp_transfers/payloads` to `/opt/aspera/orchestrator/config/orchestrator/actions/fasp_transfers` to correct payload issues.
- When downloading from Faspex servers connected to HSTS on different machines, map the `faspex_host` input parameter to the Faspex server address to avoid authorization token request issues.
- For transfers with special characters in file paths, set the split separator to 'None' to prevent misinterpretation as file list separators.
- Verify Aspera Central configuration accepts external connections if transfer initiation fails - check central_server settings in aspera.conf.
- Ensure proper authentication credentials and verify network connectivity if connection errors occur during transfer setup.

- Check ASCP version compatibility (3.6.1+) if failed file reporting is not working as expected.

3.7 Faspex5 file processing



Figure 3.7: Faspex5 file processing icon

This plugin integrates with Faspex5 servers to manage file processing workflows by retrieving lists of files awaiting validation and updating their processing status throughout the validation lifecycle.

Version	Comment
0.0.6	Enhanced action template UI
0.0.5	Improved error handling during reauthentication and API operations
0.0.4	Updated the label from Keep ongoing to Continue Monitoring
0.0.3	Updated the label from Keep ongoing to Continue Monitoring
0.0.2	Fix issues with error message as optional input and node saving
0.0.1	This is based on AsperaOTFV version 0.0.1

3.7.1 How It Works

- Integrates with the Faspex 5 server to process files requiring validation or status updates.
- Supports two main operations: retrieves lists of files awaiting processing, and updates file processing status.
- Authenticates with Faspex 5 using SSH private keys and API client credentials.
- Provides continuous monitoring capabilities with configurable polling intervals.
- Handles file processing workflows by managing file states (processing, completed, failed) within Faspex 5 packages.

3.7.2 Configuration Instructions

1. Add a Faspex5FileProcessing step to your workflow.
2. Configure the Faspex 5 connection settings:
   - Faspex Node: Select the remote node where Faspex server is installed.
   - Faspex Server Address: The full URL or hostname if not using a remote node.
   - Ssh Private key: Private key in PEM format for authentication.
   - Faspex User: Username of the package sender.
   - Faspex API Client ID: Client ID from the Faspex server configuration.
3. Set up the operation parameters:
   - Operation: Choose 'Retrieve the list of files' or 'Update file status'.
   - Max Result: Maximum number of files collected per request.
   - Polling frequency: Interval in seconds for checking files.
   - Continue Monitoring: Enable for continuous monitoring.

3.7.3 Example Configuration

Parameter	Example Value
Faspex Node	faspex5-server

Parameter	Example Value
Faspex Server Address	https://faspex5.company.com
Ssh Private key	
Faspex User	orchestrator@company.com
Faspex API Client ID	orchestrator-client-12345
Operation	Retrieve the list of files
Max Result	50
Continue Monitoring	checked
Polling frequency	30
File Status	processing (for update operation)
Error Message	File validation failed

3.7.4 Saved Parameters Description

Parameter	Description
Faspex Node	The node where Aspera Faspex server is installed. If no remote node is configured, use the Advanced parameters section.
Faspex Server Address	Advanced - The full URL or the host name of the Faspex server, if not configured as Remote Node.
Ssh Private key	Private key in PEM format for authenticating with the Faspex server.
Faspex User	The Faspex user for creating the Package. This user will be the sender of the package.
Faspex API Client ID	The client id provided in the Faspex server Configurations.
Operation	The operation to execute: 'Retrieve the list of files' to get files to be processed, or 'Update file status' to set the status of a processing file.
Max Result	The maximum number of collected files for validation by each request.
Continue Monitoring	If selected, a trigger will be executed every time files are detected for validation.
Polling frequency	The interval in seconds to check for files to be validated.
File ID	The id of the file to be validated when the operation selected is 'Update file status'.
File Status	The file status to set when updating file status.
Error Message	The custom error message when the file validation fails.

3.7.5 Inputs Description

Mandatory inputs for each operation:

- Faspex Server Address: Faspex5 server address, hostname or remote node name. Required for all operations.
- Ssh Private Key: Required to authenticate Orchestrator with Faspex5 server. Must be in PEM format.
- Faspex API Client ID: Faspex5 API client id must be provided for authentication.
- Faspex_sender_user: Faspex5 user's login username, used as the package sender.

Mandatory inputs for 'Update file status' operation:

- File status: The file status to set when operation is 'Update file status'.
- file_id: The file ID to update when operation is 'Update file status'.

Optional inputs for 'Retrieve list of files' operation:

- max_result: The maximum number of collected files for validation by each request.
- polling_frequency: The interval in seconds to check for files to be validated if 'Continue Monitoring' is selected.

Optional inputs for 'Update file status' operation:

- error_message: The custom error description if 'file status' value is 'failed'.

3.7.6 Outputs Description

- step_information: Provides a hash with the step details like step_id, step_name, step_status, step_status_details, etc.
- result_count: The number of collected files for validation if 'Retrieve list of files' operation is selected.
- file_transfer_info: The list of files to be validated if 'Retrieve list of files' operation is selected.

3.7.7 Supported Operations

- Retrieve the list of files - Get the list of files awaiting processing validation
- Update file status - Set the status of a processing file (processing, completed, failed)

3.7.8 Operating Instructions

3.7.9 Orchestrator Instance Setup

- Generate a new SSH key exclusively for Faspex interaction in PEM format:

  `ssh-keygen -t rsa -b 4096 -m PEM -C "your_email@example.com"` (save as key_for_faspex.pub)

- Convert to PEM format:

  `ssh-keygen -f key_for_faspex.pub -m 'PEM' -e > key_for_faspex.pub.pem`

- In HA environments, share the new private key among all Orchestrator instances.
- For Faspex 5.0.4 and later, enable the parameter 'allow_global_key': 'true' or 'Allow signing with global key'.

Note: Newer versions of OpenSSH default to generating ED25519 keys rather than RSA. If you encounter the error *do_convert_to_pem: unsupported key type ED25519* when exporting the public key to PEM format, it means an ED25519 key was generated, which is not supported for PEM conversion. Make sure to use the `-t rsa` flag as shown above to explicitly generate an RSA key.

3.7.10 Faspex Server Configuration

API Client Configuration:

- Enable JWT grant type: On
- Allow refresh token: Off
- All users can use this client to access the API
- Public key in PEM format: Paste the SSH public key generated exclusively for Faspex interaction

User Account Configuration:

- Log into Faspex 5 with the user account that will be used by Orchestrator
- Navigate to Account settings > Preferences
- Scroll down to the 'Public key in PEM format (optional)' field
- Paste the same SSH public key generated exclusively for Faspex interaction (same key used in API Client Configuration)

Package Processing Configuration:

- Enable File processing: On

Transfer Options Configuration:

- Disable Downloads during transfers

3.7.11 Dependencies

- The Faspex5 server must be accessible from the Orchestrator system.
- A valid SSH private key in PEM format is required for authentication.
- The API client must be properly configured in Faspex 5 with JWT grant type enabled.
- File processing must be enabled in the Faspex 5 server configuration.
- The aspweb user must have read permissions for the SSH private key file used for Faspex 5 authentication.

3.7.12 Troubleshooting

- Authentication errors: Verify that all credentials (Faspex server address, SSH private key, Faspex user, Faspex API client ID) are valid and properly configured.
- Ensure the SSH private key is in the correct PEM format and matches the public key configured in the Faspex API client.
- Check that the Faspex API client has JWT grant type enabled and refresh token disabled.
- Verify that file processing is enabled in the Faspex5 server configuration.
- For Faspex 5.0.4+, confirm that 'allow_global_key' is set to true in the server settings.
- If continuous monitoring is not working, check the polling frequency setting and ensure 'Continue Monitoring' is enabled.
- For file status update failures, verify that the file ID exists and is in a state that allows status updates.

3.8 FASPstream



Figure 3.8: FASPstream icon

This action plug-in provides the ability to start, monitor and cancel a stream to stream between 2 Aspera transfer servers using faspstream.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	Add @ssh_args = nil and @remote_node = nil to avoid an execution error
0.0.1	initial release

3.8.1 How It Works

- Enables high-speed streaming of live video/audio content using Aspera FASP technology.
- Supports both sending and receiving transfer modes between nodes.
- Streams can be UDP to UDP (multicast/unicast), TCP to file, file to stream, or other combinations.
- Monitors stream status via Node API with configurable polling frequency.
- Supports stream cancellation via the workorder or step cancel button.
- Requires:
  - Faspstream license on both initiating and serving nodes
  - Faspstream hub license on the receiving side for specific bandwidth
  - asperanoded and asperacentral services running on monitored nodes
- Uses ascp4 binary for stream execution on the initiating node.

3.8.2 Configuration Instructions

1. Create a new action step using the Faspstream plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved Faspstream configured instance.
   - Comments: Comments about this saved Faspstream configured instance.
   - Initiating node: Remote node name where the stream is initiated from.
   - Initiating node ip address: Initiating node IP address or domain name.
   - Initiating node ssh port: Initiating node ssh port (33001 by default).

- Initiating node ssh keypath: Initiating node ssh private key path.
- Initiating node username: Initiating node ssh login.
- Initiating node password: Initiating node ssh password.
- Serving node: Remote node name where the stream is served from.
- Serving node ip address: Serving node IP address or domain name.
- Serving node ssh port: Serving node ssh port (33001 by default).
- Serving node ssh keypath: Serving node ssh private key path.
- Serving node username: Serving node ssh login.
- Serving node password: Serving node ssh password.
- Node api target: A reference to the initiating or serving node for the stream monitoring.
- Node api port: Port to access the monitoring node (9092 by default).
- Node api username: Node api user to access the monitoring node.
- Node api password: Node api password to access the monitoring node.
- Target rate: Stream target rate in Kbps.
- Minimum rate: Stream minimum rate in Kbps.
- Chunk size: Chunk size (64k by default).
- Source stream url: Source stream URL.
- Destination stream url: Destination stream URL.
- Ascp4 binary path: File path to the ascp4 binary on the initiating node.
- Ascp4 command line options: Extra ascp4 command line options.
- Transfer mode: Sending (send) or receiving (recv) mode.
- Monitoring polling frequency: Polling frequency in seconds for the stream monitoring.

3.8.3 Example Configuration

This is a minimum setup configuration for the Faspstream plugin.

Parameter	Value
Initiating node	StreamNode1
Initiating node ip address	192.168.1.100
Initiating node ssh port	33001
Initiating node username	aspera
Serving node	StreamNode2
Serving node ip address	192.168.1.200
Serving node ssh port	33001
Serving node username	aspera
Node api target	serving_node
Node api port	9092
Target rate	10000
Minimum rate	8000
Chunk size	64k
Source stream url	udp://localhost:2001
Destination stream url	udp://233.3.3.3:2000?ttl=2&pkt_size=1316
Ascp4 binary path	/usr/bin/ascp4
Transfer mode	send

3.8.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved Faspstream configured instance.
Comments	Comments about this saved Faspstream configured instance.
Initiating node	Remote node name where the stream is initiated from.
Initiating node ip address	Initiating node IP address or domain name.
Initiating node ssh port	Initiating node ssh port (33001 by default).

Parameter	Description
Initiating node ssh keypath	Initiating node ssh private key path.
Initiating node username	Initiating node ssh login.
Initiating node password	Initiating node ssh password.
Serving node	Remote node name where the stream is served from.
Serving node ip address	Serving node IP address or domain name.
Serving node ssh port	Serving node ssh port (33001 by default).
Serving node ssh keypath	Serving node ssh private key path.
Serving node username	Serving node ssh login.
Serving node password	Serving node ssh password.
Node api target	A reference to the initiating or serving node for the stream monitoring.
Node api port	Port to access the monitoring node (9092 by default).
Node api username	Node api user to access the monitoring node.
Node api password	Node api password to access the monitoring node.
Target rate	Stream target rate in Kbps.
Minimum rate	Stream minimum rate in Kbps.
Chunk size	Chunk size (64k by default).
Source stream url	Source stream URL.
Destination stream url	Destination stream URL.
Ascp4 binary path	File path to the ascp4 binary on the initiating node.
Ascp4 command line options	Extra ascp4 command line options.
Transfer mode	Sending (send) or receiving (recv) mode.
Monitoring polling frequency	Polling frequency in seconds for the stream monitoring.

3.8.5 Inputs Description

The list of inputs depends on the configuration of the Faspstream action template. Changes in this list of inputs will override any value set in the action configuration/template.

Parameter	Description
Initiating node or address	Remote node name or address/domain name where the stream is initiated from
Initiating node ssh key path	Initiating node ssh private key path
Initiating node ssh username	Initiating node ssh login
Initiating node ssh password	Initiating node ssh password
Initiating node ssh port	Initiating node ssh port (optional, 33001 by default)

Parameter	Description
Serving node or address	Remote node name or address/domain name where the stream is served from
Serving node ssh key path	Serving node ssh private key path
Serving node ssh username	Serving node ssh login
Serving node ssh password	Serving node ssh password
Serving node ssh port	Serving node ssh port (optional, 33001 by default)
Node API target	initiating_node or serving_node
Node API port	Port to access the monitoring node (optional, 9092 by default)
Node API user name	Node api user to access the monitoring node
Node API user password	Node api password to access the monitoring node
Target rate	Stream target rate in Kbps
Minimum rate	Stream minimum rate in Kbps
Chunk size	Chunk size (optional, 64k by default)
Source stream URL	Source stream URL
Destination stream URL	Destination stream URL
Ascp4 binary path	File path to the ascp4 binary on the initiating node (optional, default ascp4)
Command line options	Extra ascp4 command line options (optional)
Transfer mode	Sending (send) or receiving (recv) mode

3.8.6 Outputs Description

Parameter	Description
statistics	The statistics as returned by the monitoring API when the session is completed

3.8.7 Supported Actions

None

3.8.8 Dependencies

A faspstream license is needed on both initiating and serving sides. The data receiving side (defined by the transfer mode) must have a faspstream hub license defined for a specific bandwidth.

3.8.9 Operating Instructions

This plug-in enables high-speed streaming of live video/audio content using Aspera FASP technology.

Monitoring Configuration

- Specify a node api target (initiating or serving node) for stream monitoring.
- A node api https request is sent each polling period to the target node to check stream status.
- It can be addressed to either node provided that asperanoded and asperacentral services are running on them.
- This is always the case on the serving node, not necessarily on the initiating node.
- Specify the node api https port (9092 by default), username and password.
- The node api username can be obtained with the `/opt/aspera/bin/asnodeadmin -l` command on the target node.

Stream Parameters Configuration

- Specify the expected stream target and minimum rate in Kbps.
- To guarantee good video quality:
  - Minimum rate must be equal to the video bitrate + packet loss (about 1 Mbps faster than the encoded rate)
  - Target rate must be 10 to 20% higher than the video encoded rate
- Specify the chunk size. It's the faspstream buffer size used for read/write operations as well as an internal transmission and compression unit (must be at least 4K and limited to 128M).
- Specify the source (where to listen from) and destination (where to write to) stream URLs.
- URL examples:
  - `udp://localhost:2001` for a unicast UDP
  - `udp://233.3.3.3:2000?ttl=2&pkt_size=1316` for a multicast UDP
  - `tcp://127.0.0.1:5000` for a unicast TCP
  - `file:///opt/work/test/pp3` for a file URL
- Typically a UDP provider is used for stream to stream use cases and a TCP provider is used for stream to file or file to stream use cases.
- On send transfer mode, the source URL is on the initiating node and the destination URL is on the serving node.
- On recv transfer mode, the source URL is on the serving node and the destination URL is on the initiating node.

Ascp4 and Transfer Mode Configuration

- Specify the ascp4 binary path on the initiating node. Examples: `ascp4` or `/usr/bin/ascp4`
- If ascp4 is used instead of the full filepath, make sure that the path to the binary name is included in the OS PATH variable.
- On a Linux initiating node, check that the user shell is bash rather than aspshell in /etc/passwd.
- On a Windows initiating node installed with Aspera Point-to-Point (providing the faspstream ascp4 binary), the delivered OpenSSH software is able to serve the generated command line if the user shell is configured to be switch.exe or sh.exe (check in the OpenSSH `etc\passwd` file).
- On a MAC initiating node, make sure to enable Sharing + Remote Login for the addressed user.
- Specify extra command line arguments. See ascp4 man page for a list of them.
- Specify the transfer mode (send or recv):
  - On sending, the data flows from initiating to serving node
  - On receiving, the data flows from serving to initiating node

Troubleshooting

- If you keep seeing the 'Empty faspstream session id' status message:
  - Make sure that the command line is correct by checking it in the worker logs.
  - Ensure that both nodes have active licenses.
  - Check the node logs for more details (Typically stored at `/var/log/messages`).
  - Verify on the initiating/serving node whether the ascp4 process is started
- The node api target is also used to cancel a stream when pressing on the workorder or step cancel button.
- This may be necessary if the stream ascp4 processes (on initiating and serving nodes) must be stopped for any reason.
- Deleting a workorder running an active faspstream session will not stop the underlying ascp4 processes.
- Instead, you can cancel and then delete the workorder.

3.9 FTP transfer



Figure 3.9: FTP transfer icon

This plugin provides a configurable interface to a remote FTP server. It supports sending, retrieving and deleting files within server directories. Username and password authentication are supported when required by the server.

Version	Comment
1.3.1	Enhanced action template UI
1.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.2.0	Enable binary and passive modes by default
1.1.4	Make upload folder recursively an optional input
1.1.3	Added log to trace delete after transfers operations
1.1.2	Fix edge case when ignore invalid source files is selected
1.1.1	Fix edge case when ignore invalid source files is selected
1.1.0	Add option to ignore invalid source files
1.0.0	Add option to control FTP debug level
0.9.4	Resolved hostname to IP before using ftp component to connect
0.9.3	Fix for when remote directory does not exist on Filezilla FTP server.
0.9.2	Fix for when remote directory does not exist on FTP.
0.9.1	Fix for when mtime command fails on ftp
0.9.0	Adding Ignore File Listing for speeding up upload.
0.8.2	Adding Remote Node Dependency
0.8.1	Change display of progress output
0.8.0	2015-08-11 JY - Add option to show true byte count. Progress percentage added. Displays modification time.
0.7.3	2015-06-02 ML - Fix validation to actually validate AND to support source_file array. Adds default port
0.7.2	2015-02-26 JG - Fixing issues with variable names
0.7.1	2015-02-27 ML - New icon
0.7.0	2015-02-20 ADM - Add do archive option for uploads
0.6.0	2014-12-29 JG - Validate File Sizes
0.5.8	- 2014/12/18 - PM -> Bug fix in inputs spec. A typo from previous commits. local_directory must be optional if already provided
0.5.7	2014-12-10 ADM - Support Array as input for FTP UPLOAD ONLY
0.5.6	Fixed duplicated uploads when using GLOB
0.5.5	Added protection in case a remote directory does not exist and ftp server returns an unknown error
0.5.4	Fix to single file upload and improvements to help file
0.5.3	Prevents autocomplete of user field
0.5.2	Modified help file to explain the new fields added.
0.5.1	Folder download processes single file download correctly when download folder flag is checked
0.5.0	Added functionality to recursively download folders and their contents
0.4.1	Fixes a bug for single file upload
0.4.0	Correctly execute the migration
0.3.1	Adds functionality to recursively upload folders and their contents
0.3.0	Adds functionality to accept wilcard operators '*' and '?' in source file names
0.2.3	Adds logging
0.2.2	Adds display name
0.2.1	Hides password characters while entering in edit gui.
0.2.0	Adds passive mode support
0.1.5	Adds local and remote path as outputs
0.1.4	Change reporting to be time-based
0.1.3	fix bug with self contained remote server
0.1.2	use node user if not explicitly provided
0.1.1	Removed the infinite timeout and modified parameterized inputs.
0.1.0	Adds customizable port and node selector
0.0.1	functional release with no proxy -supports get, put, delete. Use default port

3.9.1 How It Works

- Provides a configurable interface to remote FTP servers for file transfer operations including upload (put), download (get), and delete actions.
- Supports both individual file transfers and recursive folder operations with configurable transfer modes (binary/ASCII, active/passive).

- Enables file renaming during transfer, source file removal after successful transfer, and archiving capabilities for uploaded files.
- Includes authentication support with username/password credentials and configurable connection parameters like port and directories.
- Offers performance optimization features such as configurable transfer block sizes and options to ignore file listings before uploads.

3.9.2 Configuration Instructions

1. Add an FTP Transfers step to your workflow.
2. Configure the server connection:
   - Remote Node: Select configured remote system or specify server details manually.
   - Remote Server: FTP server hostname or IP address.
   - Remote Server Port: FTP port (typically 21).
   - Remote User: Username for FTP authentication.
   - Remote Password: Password for FTP authentication.
3. Set up the transfer operation:
   - Action: Choose put (upload), get (download), or delete.
   - Local Directory: Local directory path for file operations.
   - Source file name: Name of file to transfer (supports wildcards).
   - Remote Directory: Target directory on FTP server.

3.9.3 Example Configuration

Parameter	Example Value
Remote Server	ftp.example.com
Remote Server Port	21
Remote User	ftpuser
Remote Password	********
Remote Directory	/uploads
Action	put
Local Directory	/local/files
Source file name	document.pdf
Target file name	uploaded_document.pdf
Transfer in binary mode	checked
Transfer in Passive Mode	checked
Remove source file after transfer	unchecked

3.9.4 Saved Parameters Description

Parameter	Description
Remote Node	Selector for the remote system containing the FTP server.
Remote Server	Remote server name containing the FTP server, if not using the ”Remote node” selector.
Remote Server Port	The FTP command port normally ”21” on the FTP server system.
Remote User	User name required on the FTP server for authentication.
Remote Password	FTP server password associated with the user name. Note this is shown in clear text.
Remote Directory	Directory off of the FTP root directory on the FTP server changed to upon FTP login.
Action	The FTP action to be performed: ”put”, ”get”, or ”delete”.
Local Directory	Local directory setting for the client side.
Source file name	Name of the file being transferred via FTP.

Parameter	Description
Target file name	Destination name of file being transferred, if renaming is to be performed.
Transfer in binary mode	Transfer the file exactly from the source, preserving file byte count and checksum, performing no translation. This is the default. Disable binary mode (enable ASCII mode) only if you need to transfer text files between Linux and Windows and require translation of LF to CR-LF.
Transfer in Passive Mode	Transfer the file in FTP passive mode, requiring no local port to be open on the sending side. This is the default. Disable passive mode (enable active mode) only if your remote FTP server requires it.
Remove source file after transfer	Remove the source file upon successful completion of the FTP transfer.
Ignore file listing before upload?	Check this box if file listing needs to be ignored before the upload operation. This might help speed up the operation.
Upload folder(s) recursively	Check this box if the source file provided is a folder, and this needs to be uploaded to the FTP server.
Download folder(s) recursively	Check this box if the remote FTP folder needs to be downloaded to local server.
Transfer block size	Amount of data copied from application memory space to kernel memory space at one time during FTP transfers.
Archive source file after transfer	If selected, after a successful FTP upload, the original file is locally moved to an archive folder.
Archive target folder	If archive after transfer is selected, this field specifies the local target folder for the archive.
Ignore invalid files	If selected, files that cannot be accessed for upload will be skipped without failing the step.

3.9.5 Inputs Description

Changes in this list of inputs will override any values set in the action template.

- action: The FTP actions available: ”put”, ”get”, or ”delete”.
- binary: Transfer flag, binary or ASCII.
- local_directory: Local directory on the client side.
- remote_directory: Directory on the FTP server below the default FTP directory.
- remote_password: FTP password on the remote server.
- remote_user: FTP username on the remote server.
- remove_after_transfer: Remove the source file upon successful completion of the transfer.
- upload_folder: Uploads a local folder to remote FTP server.
- allow_download_folder: Downloads folder from remote FTP server.
- source_filename: Source file name.
- target_filename: Destination name of the file being transferred if renaming is to be performed.

3.9.6 Outputs Description

- size: The size of the FTP transfer.
- target_file: The name of the target file.
- ignored_files: The list of files that have been ignored during the upload because not available or not accessible. Only available if 'Ignore invalid files' is selected.

3.9.7 Operating Instructions

3.9.8 Prerequisites

FTP servers require an open port 21; ensure that this port is available on the system containing the Orchestrator instance.

3.9.9 Basic Workflow

- Identify the remote FTP server and configure a remote node for it, or place the details of the FTP server in the advanced parameters section. If a particular user name and password is required to access the FTP server it can be placed in either the remote node, or within the advanced parameters settings.
- The FTP server will normally provide a directory that will be the ”root” directory for a particular connection. If there are directories that will be below this location, they should be entered in the ”remote directory” field.
- If files are going to be delivered to the FTP server, select ”put” from the ”Action” pull down menu. If files are to be pulled from the FTP server, select ”get”. To remove a file from the FTP server, select ”delete”.
- Insert the location that the files on the local system will be uploaded from, or downloaded to in the ”Local directory”.
- Enter the ”Source file name” that is to be acted upon during the transfer. Note that wild card characters - * and ? - are accepted in source file name.
- When renaming the file at the destination is required, supply the new name in the ”Target file name” field.
- If the file being transferred is a binary file, select the ”Transfer in binary mode” checkbox.
- If the file being uploaded is a folder, select the ”Upload Folder(s) Recursively?” checkbox to upload the folder and its contents.
- If the file being downloaded is a folder, select the ”Download Folder(s) Recursively?” checkbox to download the folder and its contents.
- To delete the source file post successful transfer, the ”Remove source file after transfer” checkbox must be selected.

3.9.10 Dependencies

- FTP server must be accessible on the configured port (typically port 21) from the Orchestrator system.
- Valid FTP credentials (username/password) must be available if the FTP server requires authentication.
- The aspweb user must have appropriate local filesystem permissions for source and destination directories.

3.10 Ftps transfer



Figure 3.10: Ftps transfer icon

This action plugin can be used to perform secure file transfer operations on FTPS servers with SSL/TLS encryption and file renaming support.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.1	Initial implementation

3.10.1 How It Works

- Provides secure file transfer operations to/from remote FTPS (FTP over SSL/TLS) servers.
- Supports Upload (Put) and Download (Get) operations.
- Uses SSL/TLS encryption for secure data transmission over FTP protocol.
- Supports username/password authentication for server access.
- Allows file renaming during transfer operations.
- Optionally removes source files after successful transfer (move operation).

- Connects to FTPS servers via Remote Node configuration or direct server credentials.
- Works with configurable remote directories relative to the FTPS root.
- Returns list of files transferred as output.

3.10.2 Configuration Instructions

1. Create a new action step using the FtpsTransfer plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved FTPS Transfer configured instance.
   - Remote Node: Selector for the remote system containing the FTPS server. Choose from configured Remote Nodes.
   - Remote Server: Remote server hostname or IP address containing the FTPS server, if not using the 'Remote node' selector.
   - Remote User: Username required on the FTPS server for authentication.
   - Remote Password: FTPS server password associated with the user name.
   - Remote Directory: Directory path relative to the FTPS root directory on the server, changed to upon FTPS login.
   - Action: The FTPS action to be performed: put (upload) or get (download).
   - Local Directory: Local directory path on the Orchestrator server for client-side operations.
   - Source file name: Name of the file being transferred via FTPS. For put operations, this is the local file; for get operations, this is the remote file.
   - Target file name: Destination name of file being transferred, if renaming is to be performed. If blank, original filename is used.
   - Remove source file after transfer: If selected, removes the source file upon successful completion of the FTPS transfer.

3.10.3 Example Configuration

This is an example configuration for the FtpsTransfer plugin uploading and renaming a media file.

Parameter	Value
Name	Upload Media to FTPS
Remote Node	ProductionFTPS
Remote Directory	/uploads/media
Action	put
Local Directory	/local/processed
Source file name	video.mp4
Target file name	video_final.mp4
Remove source file after transfer	Not selected

3.10.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved FTPS Transfer configured instance.
Remote Node	Selector for the remote system containing the FTPS server. Choose from configured Remote Nodes.
Remote Server	Remote server hostname or IP address containing the FTPS server, if not using the 'Remote node' selector.
Remote User	Username required on the FTPS server for authentication.
Remote Password	FTPS server password associated with the user name.
Remote Directory	Directory path relative to the FTPS root directory on the server, changed to upon FTPS login.
Action	The FTPS action to be performed: put (upload) or get (download).
Local Directory	Local directory path on the Orchestrator server for client-side operations.
Source file name	Name of the file being transferred via FTPS. For put operations, this is the local file; for get operations, this is the remote file.
Target file name	Destination name of file being transferred, if renaming is to be performed. If blank, original filename is used.

Parameter	Description
Remove source file after transfer	If selected, removes the source file upon successful completion of the FTPS transfer.

3.10.5 Inputs Description

Parameter	Description
Remote_Node	The remote node identifier containing the FTPS server configuration. Use this OR Remote_Server (not both).
Remote_Server	The hostname or IP address of the FTPS server. Use this OR Remote_Node (not both).
Remote_User	Username for FTPS server authentication (required if server requires authentication).
Remote_Password	Password for FTPS server authentication (required if server requires authentication).
Remote_Directory	Path relative to FTPS root directory (e.g., '/incoming/files', 'subfolder/data'). Leave blank for root directory.
Action	The operation to perform: 'put' (upload) or 'get' (download).
Local_Directory	Local directory path on Orchestrator server.
Source_file_nam	Name of the file to transfer. Local file for put, remote file for get operations.
Target_file_name	New name for the file at destination (optional). If blank, uses original source filename.
Remove_source_file_transfer	Boolean flag to delete source file after successful transfer.

3.10.6 Outputs Description

Parameter	Description
files_list	Array containing the names of the files transferred or listed during the operation

3.10.7 Supported Actions

None

3.10.8 Dependencies

None

3.10.9 Operating Instructions

This plugin provides a configurable interface to a remote FTPS server. The actions permitted include the ability to send and retrieve files from directory locations exposed by the server. Renaming files during transfer is supported. Username and password authentication is required for server access.

Supported Actions

- put (Upload):
  - Transfers files from local Orchestrator server to remote FTPS server
  - Source: Local Directory + Source file name
  - Destination: Remote Directory + Target file name (or Source file name if Target is blank)
  - If 'Remove source file after transfer' is selected, local file is deleted after successful upload
  - Example: Upload processed media files to FTPS distribution server
- get (Download):
  - Transfers files from remote FTPS server to local Orchestrator server

  - Source: Remote Directory + Source file name
  - Destination: Local Directory + Target file name (or Source file name if Target is blank)
  - If 'Remove source file after transfer' is selected, remote file is deleted after successful download
  - Example: Download reports from FTPS server for processing

Connection Methods

- Using Remote Node (Recommended):
  - Configure FTPS server details in Remote Node settings in Orchestrator
  - Select the Remote Node in the 'Remote Node' dropdown
  - Server, credentials, and connection settings are managed centrally
  - More secure - credentials stored in Remote Node configuration
  - Reusable across multiple workflow steps
- Using Direct Server Credentials:
  - Leave 'Remote Node' blank
  - Specify 'Remote Server' (hostname or IP)
  - Provide 'Remote User' and 'Remote Password' directly
  - Useful for one-off or dynamic server connections

Remote Directory Concept

- The FTPS server typically provides a root directory for each connection based on the user account.
- The 'Remote Directory' field specifies a path relative to this root.
- Examples:
  - FTPS root: `/ftps/user123`
  - Remote Directory: `/incoming/media`
  - Actual server path: `/ftps/user123/incoming/media`
- Leave 'Remote Directory' blank to work in the FTPS root directory.
- Use forward slashes (/) for directory separators regardless of server OS.

Common Use Cases

- Media distribution: Upload processed video or audio files to FTPS servers for delivery.
- Report retrieval: Download generated reports from FTPS servers for analysis or distribution.
- Data exchange: Secure file exchange with partners or external systems using FTPS.
- Automated backups: Upload backup files to remote FTPS storage servers.
- File cleanup: Delete processed files from FTPS servers to free up space.
- File inventory: List directory contents to discover available files before processing.
- Archive retrieval: Download archived files from FTPS servers for restoration or review.

Best Practices

- Use Remote Node configuration for credentials rather than direct server parameters for better security.
- Use FTPS (FTP over SSL/TLS) instead of plain FTP for encrypted transmission.
- Enable 'Remove source file after transfer' only when certain the transfer completed successfully.
- Use absolute paths for local directories to avoid ambiguity.
- Implement error handling in workflows for network failures or authentication issues.
- Monitor the files_list output to track which files were processed.
- Test FTPS connectivity and authentication before deploying workflows to production.
- Use Target file name for renaming when naming conventions differ between systems.
- Ensure FTPS server firewall rules allow connections from Orchestrator (typically ports 21 and passive data ports).

Troubleshooting

- Connection timeout: Verify FTPS server is reachable, firewall allows connections, and server is running.
- Authentication failed: Verify username and password are correct. Check if account is locked or expired.
- File not found: Verify file exists in the specified directory on the server.
- Permission denied: Ensure FTPS user has appropriate permissions (read for get/list, write for put, delete for remove).
- Passive mode issues: FTPS uses passive mode for data connections. Ensure firewall allows passive data port range.
- SSL/TLS errors: Verify server certificate is valid or configure to accept self-signed certificates.
- Directory not found: Verify 'Remote Directory' path is correct relative to FTPS root.
- Local file missing: For put operations, ensure source file exists in Local Directory.
- Disk space issues: For get operations, ensure local filesystem has sufficient space.

Important Notes

- FTPS uses SSL/TLS encryption for secure file transfers, unlike plain FTP.
- The 'Remote Directory' is relative to the FTPS root directory for the authenticated user.
- FTPS typically uses port 21 for control connection and a range of ports for passive data connections.
- If both 'Remote Node' and 'Remote Server' are specified, Remote Node takes precedence.
- The 'Remove source file after transfer' option deletes the source only after successful transfer.
- If 'Target file name' is blank, the original source filename is used at the destination.
- Ensure FTPS server and network support passive mode for data transfers.

Security Considerations

- Store credentials in Remote Node configuration rather than direct plugin parameters.
- Use complex passwords for FTPS accounts.
- Ensure FTPS server uses valid SSL/TLS certificates.
- Grant FTPS users only necessary permissions (read-only for downloads, write-only for uploads).
- Configure FTPS server to allow connections only from known Orchestrator IP addresses.
- Enable FTPS server logging to track file transfers and access attempts.
- Keep FTPS server software updated with security patches.

3.11 FXP Transfer



Figure 3.11: FXP Transfer icon

This action provides the ability to move files using the FXP protocol.

Version	Comment
0.3.1	Enhanced action template UI
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.0	Added timeout hard coded value of 84600 (24 hours) to read_timeout
0.1.9	Adding Remote Node Dependency
0.1.8	add fixes for FTPFXP library to support Filezilla as remote node when no TLS
0.1.7	add fixes for FTPFXP library to support Filezilla as remote node
0.1.6	2015-03-27 ML - New icon
0.1.5	prevents autocomplete of user field
0.1.4	after code review
0.1.3	fix problem when origin_user/password remote_user/password as specified as runtime input
0.1.2	retrieve node user and password if they are not specified
0.1.1	fix need to specify the remote file path
0.1.0	add the possibility to specify the remote file name
0.0.4	fix problem with remote directory
0.0.3	fix socket closing when all is ok; use the correct remote_path
0.0.2	fix problem in setting users and passwords
0.0.1	functional release

3.11.1 How It Works

- FXP (File eXchange Protocol) enables direct server-to-server data transfers. Unlike traditional FTP transfers, FXP does not route data through the client's network connection. The client acts only as a coordinator, while data flows directly between the two servers.

- When an FXP session is initiated, the client establishes and maintains separate FTP control connections to both servers. Using these control connections, the client instructs one server to initiate a direct data transfer with the other server.
- Once the FXP session is established, data flows directly between the servers without passing through the client. This allows efficient transfers even when the client has limited bandwidth.
- To support long-running transfers, a 24-hour timeout is configured in the FTPFXP library. This extended timeout allows large server-to-server transfers to complete without interruption. Transfers exceeding this duration may fail with a timeout error.

3.11.2 Configuration Instructions

1. Create a new action step using the FxpTransfer plug-in.
2. This action transfers data directly between two FTP servers using FXP.
3. Fill out the following configuration fields:
    - Remote node: The remote node used for the FXP transfer.
    - Advanced Parameters - Remote Server: The remote server IP address. This value overrides the node address if provided.
    - Advanced Parameters - Remote Server port: The remote server port. This value overrides the node port if provided.
    - Advanced Parameters - Remote user: The FTP user account on the remote server used for the transfer, if different from the node definition.
    - Advanced Parameters - Remote Password: The password for the remote FTP user account, if different from the node definition.
    - Advanced Parameters - Confirm Password: Re-enter the remote password to confirm.
    - Origin node: The origin node used for the FXP transfer.
    - Advanced Parameters - Origin Server: The origin server IP address. This value overrides the node address if provided.
    - Advanced Parameters - Origin Server port: The origin server port. This value overrides the node port if provided.
    - Advanced Parameters - Origin user: The FTP user account on the origin server used for the transfer, if different from the node definition.
    - Advanced Parameters - Origin Server password: The password for the origin FTP user account, if different from the node definition.
    - Advanced Parameters - Confirm Password: Re-enter the origin password to confirm.
    - Remote directory: The target directory on the remote server.
    - Remote file name: The target file name on the remote server.
    - Handshake protocol: The handshake protocol to use. Supported values are SSCN (Set Secured Client Negotiation) or the CPSV command.

3.11.3 Example Configuration

This is a minimal setup configuration for the FxpTransfer plug-in.

Parameter	Value
Remote node	node_2
Origin node	node_1
Remote directory	/root/src
Remote file name	test.txt
Handshake protocol	SSCN (Set Secured Client Negotiation)

3.11.4 Saved Parameters Description

Parameter	Description
Remote node	The remote node used for the FXP transfer.
Advanced Parameters - Remote Server	The remote server IP address. This value overrides the node address if provided.

Parameter	Description
Advanced Parameters - Remote Server port	The remote server port. This value overrides the node port if provided.
Advanced Parameters - Remote user	The FTP user account on the remote server used for the transfer, if different from the node definition.
Advanced Parameters - Remote Password	The password for the remote FTP user account, if different from the node definition.
Advanced Parameters - Confirm Password	Re-enter the remote password to confirm.
Origin node	The origin node used for the FXP transfer.
Advanced Parameters - Origin Server	The origin server IP address. This value overrides the node address if provided.
Advanced Parameters - Origin Server port	The origin server port. This value overrides the node port if provided.
Advanced Parameters - Origin user	The FTP user account on the origin server used for the transfer, if different from the node definition.
Advanced Parameters - Origin Server password	The password for the origin FTP user account, if different from the node definition.
Advanced Parameters - Confirm Password	Re-enter the origin password to confirm.
Remote directory	The target directory on the remote server.
Remote file name	The target file name on the remote server.
Handshake protocol	The handshake protocol to use. Supported values are SSCN (Set Secured Client Negotiation) or the CPSV command.

3.11.5 Inputs Description

Required inputs depend on how the FxpTransfer template is configured. If dynamic attributes are used, each attribute becomes a runtime input.

Parameter	Description
Origin_file_path *	The directory path on the origin server used for the transfer.
FXP_debug_flag	Enable debug logging for the FXP transfer.
Origin_password	The password for the FTP user account on the origin server.
Origin_server	The origin server IP address.
Origin_server_port	The origin server port.
Origin_user	The FTP user account on the origin server.

Parameter	Description
Remote_directory	The target directory on the remote server.
Remote_file_name	The target file name on the remote server.
Remote_password	The password for the FTP user account on the remote server.
Remote_server	The remote server IP address.
Remote_server_por	The remote server port.
Remote_user	The FTP user account on the remote server.
Target_file_path	The full target file path on the remote server.

3.11.6 Outputs Description

None

3.11.7 Supported Actions

None

3.11.8 Dependencies

None

3.11.9 Operating Instructions

1. FXP enables direct server-to-server file transfers without routing data through the client.
2. Select and configure the origin and remote FTP servers that will participate in the FXP transfer.
3. The client establishes separate FTP control connections to both servers to coordinate the transfer.
4. Once initiated, data flows directly between the origin and remote servers.
5. Transfer performance depends on the bandwidth available between the two servers rather than the client.
6. A 24-hour timeout is configured in the FTPFXP library to support large file transfers.
7. Transfers that exceed the configured timeout may fail with a timeout error.

3.11.10 Warnings and Best Practices

- Ensure both FTP servers explicitly allow FXP (server-to-server) transfers, as FXP is often disabled by default for security reasons.
- Verify that firewalls and network security rules allow the required data connections between the two servers.
- Use FXP only between trusted servers to avoid unauthorized access or data exposure.
- For large transfers, ensure sufficient disk space is available on the destination server and monitor transfer progress.

3.12 HTTP Transfer

This action provides the ability to upload, download and delete files from a remote host using HTTP or HTTPS (Hypertext Transfer Protocol or Hypertext Transfer Protocol over SSL).

Version	Comment
0.2.2	Enhanced action template UI
0.2.1	Add SSL/HTTPS support, redirect following, and response validation
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.4	Add some elements in the help
0.1.3	Adding support for HTTP Redirect
0.1.2	Adding Remote Node Dependency

Version	Comment
0.1.1	Expose Status details as output
0.1.0	Added option to fail if http transfer does not return a 2XX code
0.0.6	New icon
0.0.5	exception handling of RemoteNode.find_by_name()
0.0.4	Prevents autocomplete of user field
0.0.3	Adds display name
0.0.2	Adds functionality which allows users to input the chunk/block size for data upload and download
0.0.1	Adds DELETE method to the http plugin functionality and the remove source after operation functionality
0.0.0	First version of the http plugin which supports upload and download

3.12.1 How It Works

- Performs file transfer operations (upload, download, delete) on remote HTTP servers using the HTTP protocol.
- Supports Basic HTTP authentication via Remote Node configuration or direct plugin parameters.
- Supports HTTPS/SSL connections automatically when using port 443.
- Automatically follows HTTP redirects (301, 302) for all operations.
- Operates on both individual files and directories.
- Optionally removes source files after successful transfer (move operation).
- Configurable failure behavior on HTTP error codes (codes > 300).
- Reports transfer progress at configurable intervals.
- Returns target file paths, total size processed, and status details after operation.

3.12.2 Configuration Instructions

1. Create a new action step using the HttpTransfer plugin.
2. Fill out the following fields:
    - Name: The name used to identify a saved HTTP Transfer configured instance.
    - Local path: The file or directory absolute path on the local filesystem (source for uploads, destination for downloads).
    - Remote path: The file or directory path on the remote HTTP server (destination for uploads, source for downloads).
    - Remote key location: The file path on Orchestrator of the private SSH key used to access the remote server.
    - Operation: The operation to perform: Upload (local to remote), Download (remote to local), or Delete (remove from remote).
    - Remote node: The remote HTTP node to connect to. Must be a configured Remote Node with HTTP credentials.
    - Remove source after transfer: If set, the source file (local for uploads, remote for downloads) will be deleted after successful transfer.
    - Fail if http error (http return code > 300): By default this action will complete no matter what HTTP return code the server responds. Check this option if the action should fail in case of an HTTP error return code.
    - Reporting frequency: How often transfer progress will be reported in seconds. Default is 5 seconds.

3.12.3 Example Configuration

This is an example configuration for the HttpTransfer plugin uploading a media file to an HTTP server.

Parameter	Value
Name	Upload Media to HTTP Server
Local path	/local/media/video.mp4
Remote path	/uploads/media/video.mp4
Operation	Upload
Remote node	MediaHTTPServer
Remove source after transfer	Not selected
Fail if http error	Selected
Reporting frequency	5

3.12.4 Saved Parameters Description



Figure 3.12: HTTP Transfer icon

Parameter	Description
Name	The name used to identify a saved HTTP Transfer configured instance.
Local path	The file or directory absolute path on the local filesystem (source for uploads, destination for downloads).
Remote path	The file or directory path on the remote HTTP server (destination for uploads, source for downloads).
Remote key location	The file path on Orchestrator of the private SSH key used to access the remote server.
Operation	The operation to perform: Upload (local to remote), Download (remote to local), or Delete (remove from remote).
Remote node	The remote HTTP node to connect to. Must be a configured Remote Node with HTTP credentials.
Remove source after transfer	If set, the source file (local for uploads, remote for downloads) will be deleted after successful transfer.
Fail if http error (http return code > 300)	By default this action will complete no matter what HTTP return code the server responds. Check this option if the action should fail in case of an HTTP error return code.
Reporting frequency	How often transfer progress will be reported in seconds. Default is 5 seconds.

3.12.5 Inputs Description

All parameters are available as inputs at runtime if not specified by the template. This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

Parameter	Description
Local_path	The absolute path of the file or directory on the local filesystem. Required for upload (source) and download (destination) operations.
Remote_path	The path of the file or directory on the remote HTTP server. Required for upload (destination), download (source), and delete operations.
Remote_key_location	The file path on Orchestrator of the private SSH key used to access the remote server (optional if credentials are in Remote Node configuration).
Operation	The operation to perform: Upload, Download, or Delete.
Remote_node	The remote HTTP node identifier. Must be a configured Remote Node.
Remove_source_after_transfer	Boolean flag to delete the source file after successful transfer.
Fail_if_http_error	Boolean flag to fail the action on HTTP error codes greater than 300.
Reporting_frequency	Progress reporting interval in seconds (default: 5 seconds).

3.12.6 Outputs Description

Parameter	Description
Transferred_files	The list of target file paths involved in the operation
Transferred_bytes	The total size in bytes processed by the operation
Status details	The final status message of the operation including success or error details

3.12.7 Supported Actions

None

3.12.8 Dependencies

None

3.12.9 Operating Instructions

This plug-in allows upload, download and delete operations on a remote HTTP server using the HTTP protocol. It supports file-based operations with Basic HTTP authentication.

Supported Operations

- Upload:
  - Transfers files from local filesystem to remote HTTP server
  - Local path = source file/directory
  - Remote path = destination on HTTP server
  - If 'Remove source after transfer' is set, local file is deleted after successful upload
  - Example: Upload processed media files to distribution server
- Download:
  - Transfers files from remote HTTP server to local filesystem
  - Remote path = source file on HTTP server
  - Local path = destination on local filesystem
  - If 'Remove source after transfer' is set, remote file is deleted after successful download
  - Example: Download reports or assets from HTTP server for local processing
- Delete:
  - Removes files from the remote HTTP server
  - Remote path = file to delete on HTTP server
  - Local path is not required for delete operations
  - Example: Clean up temporary files on HTTP server after processing

HTTP Response Code Handling

- 2xx Success codes: Operation completed successfully (200 OK, 201 Created, 204 No Content).
- 3xx Redirection codes: Automatically followed (301 Moved Permanently, 302 Found). The plugin follows up to 10 redirects automatically.
- 4xx Client error codes: Request error (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found). Always causes failure when 'Fail if http error' is checked.
- 5xx Server error codes: Server-side error (500 Internal Server Error, 503 Service Unavailable). Always causes failure when 'Fail if http error' is checked.
- Default behavior: Without 'Fail if http error', the action completes regardless of HTTP return code. Enable this option for strict error handling in production workflows.

Common Use Cases

- Media distribution: Upload processed video or audio files to HTTP delivery servers.
- Report retrieval: Download generated reports from HTTP servers for email distribution or archiving.
- Content management: Upload web assets (images, CSS, JS) to HTTP-based content servers.
- Data exchange: Transfer data files between systems via HTTP endpoints.
- Cleanup operations: Delete temporary or processed files from HTTP servers after use.
- File staging: Download files from HTTP staging servers for local processing, then delete from staging.
- Backup/restore: Upload backups to HTTP-based storage servers.

Best Practices

- Use Remote Node configuration for credentials rather than hardcoding in plugin parameters.
- Enable 'Fail if http error' in production workflows to ensure proper error handling.
- Use 'Remove source after transfer' only when you are certain the transfer completed successfully.
- Set appropriate reporting frequency based on file size - lower values for large files, higher for small frequent transfers.
- Use absolute paths for local file paths to avoid ambiguity.
- Implement error handling in workflows for network failures or server unavailability.
- Monitor the OUT_SIZE output to verify expected data was transferred.
- Test HTTP connectivity and authentication before deploying workflows to production.
- Use OUT_STATUS_DETAILS output in notifications or logs for transfer audit trails.

Troubleshooting

- Authentication failures (401): Verify the HTTP credentials in the Remote Node configuration. Only Basic authentication is supported.
- Forbidden errors (403): Check that the authenticated user has sufficient permissions for the operation on the specified path. Verify server-side directory permissions.
- File not found (404): Verify the remote path is correct and the file exists on the HTTP server. For uploads, ensure the server is configured to accept PUT requests.
- Connection timeout: Check network connectivity to the HTTP server and verify the Remote Node configuration.
- Source file missing: Ensure the local path is absolute and the file exists before initiating the upload.
- Destination directory missing: Verify the remote destination directory exists or that the server supports directory creation.
- Large file timeouts: Increase step timeout settings and use a higher reporting frequency for large files.
- Transfer verification: Use the `OUT_SIZE` output to verify that the expected number of bytes were transferred.
- SSL/HTTPS issues: The plugin automatically uses SSL for port 443. Ensure the server has a valid certificate or accepts self-signed certificates.
- Server not configured for WebDAV: Upload and delete operations require WebDAV support (PUT and DELETE HTTP methods). See Server Configuration Requirements below.

Important Notes

- Only Basic HTTP authentication is supported.
- This plugin does not support value overwrites. If a value is specified in the template, it will not be available as a runtime input.
- The Remove source after transfer option deletes the source only after a successful transfer.
- By default, the action completes regardless of HTTP response code. Enable Fail if HTTP error for strict error handling.
- HTTP response codes above 300 are treated as errors when Fail if HTTP error is enabled.
- Progress is reported every 5 seconds by default. Adjust the reporting frequency based on file size and monitoring requirements.
- HTTPS/SSL is automatically enabled when using port 443. Self-signed certificates are accepted.
- HTTP redirects (301, 302) are automatically followed up to 10 times for all operations.
- The server must support WebDAV (PUT and DELETE methods) for upload and delete operations.

3.12.10 Server Configuration Requirements

The HTTP Transfer plugin requires the remote HTTP server to support WebDAV (Web Distributed Authoring and Versioning) for upload and delete operations. Download operations work with standard HTTP servers. Below are configuration instructions for common web servers.

Nginx Configuration

- Enable WebDAV module: Ensure the `ngx_http_dav_module` is compiled into nginx. Check with `nginx -V 2>&1 | grep -o with-http_dav_module`.
- Configure location block: Add a location block in your nginx configuration file (typically `/etc/nginx/nginx.conf` or `/etc/nginx/sites-available/default`).
- Set DAV methods: Enable PUT and DELETE methods using the `dav_methods` directive.
- Configure permissions: Set `create_full_put_path on` to allow directory creation and `dav_access` for file permissions.
- Set directory ownership: Ensure the nginx user (typically `nginx` or `www-data`) owns the upload directory.

Example Nginx Configuration:

```
location /uploads {
    alias /var/www/uploads;

    # Enable WebDAV methods
    dav_methods PUT DELETE;

    # Allow directory creation
    create_full_put_path on;

    # Set file permissions (user:group:others)
    dav_access user:rw group:r all:r;

    # Optional: Basic authentication
    auth_basic "Restricted Upload Area";
    auth_basic_user_file /etc/nginx/.htpasswd;

    # Optional: Limit upload size
    client_max_body_size 1000M;
}
```

Directory Setup (Nginx):

```
# Create upload directory
sudo mkdir -p /var/www/uploads

# Set ownership to nginx user
sudo chown -R nginx:nginx /var/www/uploads

# Set permissions
sudo chmod -R 755 /var/www/uploads
```

Security Considerations:

- Always use authentication: Configure Basic authentication to prevent unauthorized access.
- Use HTTPS in production: Configure SSL/TLS certificates to encrypt credentials and data in transit.
- Limit upload size: Set `client_max_body_size` (nginx) or `LimitRequestBody` (Apache) to prevent disk exhaustion.
- Restrict file types: Use server configuration to limit allowed file extensions if needed.
- Monitor disk space: Implement monitoring for the upload directory to prevent disk full conditions.
- Set proper permissions: Use minimal required permissions (755 for directories, 644 for files).
- Regular cleanup: Implement automated cleanup of old files to manage disk space.

Common Server Configuration Issues:

- 405 Method Not Allowed: WebDAV methods (PUT, DELETE) are not enabled. Check `dav_methods` directive.
- 403 Forbidden: Nginx user doesn't have write permissions. Check directory ownership and permissions.
- 500 Internal Server Error: SELinux is blocking writes. Check SELinux context with `ls -Z` and set proper context.
- 413 Request Entity Too Large: File exceeds `client_max_body_size`. Increase the limit in nginx configuration.
- Connection refused: Firewall blocking the port. Check firewall rules with `firewall-cmd --list-all`.

3.13 iTunes transporter

This action plugin provides the ability to deliver audio and video content, in a pre-generated iTunes Store package, to the iTunes Store.

Version	Comment
0.2.2	Enhanced action template UI
0.2.1	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.0	Remove hardcoded aspera directory reference
0.1.3	Fix for autofill username/password
0.1.2	Adding remote node dependency
0.1.1	Adding help file

Version	Comment
0.1.0	Fixing issue with rate not being accepted as KB
0.0.9	Adds option to report transfers to console
0.0.8	Version was not bumped when last change was made, only that correction
0.0.7	Prevents autocomplete of login field
0.0.6	Adds parsing of validation error from the result check
0.0.5	IMPORTANT: fix target rate parsing to properly get m and g
0.0.4	Moves successful file to the right folder
0.0.3	fixes JVM heap calculation
0.0.2	adds ouput log to default payload, returns outputs as hashes
0.0.1	Initial draft release, only supports upload. Tested only on simulated executable

3.13.1 How It Works

- Delivers audio and video content from pre-generated iTunes Store packages to the iTunes Store using the iTunesTransporter binary.
- Supports multiple transport modes including Aspera, DAV, and Signiant for flexible content delivery options.
- Provides comprehensive package management with options for post-upload handling such as moving files to success/failure directories.
- Integrates with remote sender nodes to retrieve packages from different locations, supporting both Unix and Windows environments.
- Includes advanced command-line customization capabilities for complex delivery scenarios and custom payload configurations.
- Offers detailed logging and monitoring with configurable output paths for tracking upload status and troubleshooting issues.

3.13.2 Configuration Instructions

1. Add an iTunes Transporter step to your workflow.
2. Configure iTunes Store credentials:
    - Provider shortname: Enter your iTunes content provider shortname.
    - iTunes user name: Provide your iTunes Store username.
    - iTunes user password: Enter your iTunes Store password.
3. Set up transport and package configuration:
    - Transport mode: Select transport method (Aspera, DAV, or Signiant).
    - Package path: Specify path to pre-generated iTunes Store packages.
    - iTunesTransporter binary path: Provide path to the iTunesTransporter executable.
4. Configure optional settings:
    - Sender node: Select remote node for package retrieval if needed.
    - Output log path: Set directory for log file storage.
    - Move after success/failure paths: Define post-upload file handling.

3.13.3 Example Configuration

Parameter	Example Value
Provider shortname	test_provider_id
iTunes user name	itunes_provider_id
iTunes user password	**********
Transport mode	Aspera
Checksum validation	false
Package path	/opt/iTunesPre/packages/
Delete after upload	false
Move after success path	/opt/iTunes/upload_success/
Output log path	/opt/aspera/logs/iTunes/
Target rate	10000
iTunesTransporter binary path	/usr/local/itms/bin/iTMSTransporter



Figure 3.13: iTunes transporter icon

3.13.4 Saved Parameters Description

Parameter	Description
Provider shortname	Shortname of the content provider for iTunes Store identification.
iTunes user name	User name for iTunes Store authentication and access.
iTunes user password	Password for iTunes Store authentication and access.
Transport mode	Transport mode of the content. Available options are 'Aspera', 'DAV', 'Signiant'. Defaulted to 'Aspera'.
Checksum validation	Select true if checksum validation is to be applied during transfer. Defaulted to false.
Proxy host	Proxy host address for network connections requiring proxy access.
Proxy port	Proxy port number for network connections requiring proxy access.
Http proxy host	HTTP proxy host address for HTTP-specific proxy requirements.
Http proxy port	HTTP proxy port number for HTTP-specific proxy requirements.
Target rate	Transfer target rate for upload speed control. Defaulted to 10000.
JVM heap size	Size of the JVM heap. Can be specified in kb, mb or gb (e.g., Xmx1024m is 1024 MB).
Package path	Path of pre-generated iTunes Store package for upload processing.
Delete after upload	Select true to delete the source package after successfully uploading it to the iTunes store.
Move after success path	The folder path where the content is to be moved after it is successfully uploaded to the iTunes Store.
Move after failure path	The folder path where the content is to be moved after it fails to upload to the iTunes Store.
Output log path	Path for the log file after the upload process completion.
Management port	Management port number for service communication. Defaulted to 40001.
Sender node	Remote node from where the pre-generated iTunes Store packages are obtained.
Sender node is windows?	Check this checkbox if the sender node is a Windows node (paths in Unix and Windows are treated differently).
iTunesTrans-porter binary path	The path for the iTunesTransporter binary (e.g., '/usr/local/itms/bin/iTMSTransporter').
Command line options	Used to provide command line payload for the transporter. Runtime variables can be defined with Ruby embedded tags. This will replace the above parameters with the ones provided in this field (except Sender node and iTunesTransporter binary path).
Execution login	Execution login for the sender node authentication.
Execution Password	Password for the execution login on the sender node.

Parameter	Description
re-enter	Re-enter password confirmation for the execution login on the sender node.

3.13.5 Inputs Description

Mandatory fields with default payload:

- Provider shortname: Content provider shortname for iTunes Store identification (required).
- iTunes user name: iTunes Store username for authentication (required).
- iTunes user password: iTunes Store password for authentication (required).

Optional runtime inputs:

- transport_mode: Transport method selection (Aspera, DAV, or Signiant).
- checksum_validation: Enable or disable checksum validation during transfer.
- package_path: Path to pre-generated iTunes Store packages.
- delete_after_upload: Whether to delete source packages after successful upload.
- target_rate: Transfer rate configuration for upload speed control.
- output_log_path: Directory path for log file storage.
- move_after_success_path: Directory for moving successfully uploaded packages.
- move_after_failure_path: Directory for moving failed upload packages.

3.13.6 Outputs Description

- Uploaded_packages: Hash of all the files successfully uploaded to the iTunes store with details and status information.
- Failed_packages: Hash of all the files that failed to upload to the iTunes store with error details and failure reasons.

3.13.7 Operating Instructions

- Prerequisites: This plugin provides the ability to deliver audio and video content from pre-generated iTunes Store packages to the iTunes Store. The iTunesTransporter binary must be installed and can be downloaded from iTunes Connect at https://itunesconnect.apple.com.
- Basic Configuration: Provide input parameters from the default payload OR custom command line inputs along with the binary path and login credentials. Essential parameters include Provider shortname, iTunes username/password, transport mode (Aspera recommended), and package paths.
- Package Processing: Packages are retrieved from the specified package path (e.g., /opt/iTunesPre/packages/) and uploaded to iTunes via the iTMSTransporter. The system supports automatic checksum validation and configurable transfer rates for optimized delivery.
- Post-Upload Handling: After successful upload, packages can be automatically moved to a success directory (e.g., /opt/iTunes/upload_success/). Failed uploads can be redirected to a separate failure directory for manual review and reprocessing.
- Logging and Monitoring: All upload activities are captured and stored in the specified log path (e.g., /opt/aspera/logs/iTunes/) for detailed tracking, troubleshooting, and audit purposes.
- Advanced Customization: For complex scenarios, use the Command line options field to provide custom payload with Ruby embedded tags for runtime variables. This overrides most standard parameters while preserving Sender node and binary path settings.
- Remote Node Support: When using remote sender nodes, ensure proper authentication is configured and select the appropriate Windows/Unix setting based on the sender node operating system for correct path handling.

3.13.8 Dependencies

- iTunesTransporter binary must be installed and accessible at the specified binary path location.
- Valid iTunes Connect account with appropriate permissions for content delivery to the iTunes Store.
- Network connectivity to Apple's iTunes Store servers, including any required proxy configurations.
- Java Virtual Machine (JVM) with sufficient heap size configuration for package processing.
- Proper file system permissions for accessing package directories, log paths, and post-processing locations.
- For remote sender nodes: SSH/network access and appropriate authentication credentials.

3.13.9 Troubleshooting

- If iTunesTransporter binary cannot be found, verify the binary path is correct and the executable has proper permissions. Download the latest version from iTunes Connect if necessary.
- For authentication failures, confirm iTunes username and password are correct and the account has appropriate permissions for content delivery. Check for any account restrictions or expired credentials.
- If transport mode issues occur, try different transport methods (Aspera, DAV, Signiant) based on your network configuration and available protocols. Aspera is generally recommended for large files.
- For package path errors, ensure the directory exists, contains valid iTunes Store packages, and has appropriate read/write permissions for the executing user.
- If proxy configuration is causing connection issues, verify proxy host/port settings are correct and test network connectivity through the proxy to Apple's servers.
- For JVM heap size errors, increase the heap size parameter (e.g., Xmx2048m) to handle larger packages or multiple simultaneous uploads.
- When using remote sender nodes, verify network connectivity, authentication credentials, and ensure the 'Sender node is windows?' setting matches the actual operating system.

3.14 Robocopy



Figure 3.14: Robocopy icon

This action plugin provides file copy operation on a Windows OS

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.3	Adding Remote Node Dependency
0.0.2	- 2015/02/09 - JG -> Parse Code Commented since its not used.
0.0.1	-> Initial release.

3.14.1 How It Works

- Provides robust file copy operations on Windows OS using the Robocopy (Robust File Copy) utility.
- Executes Robocopy commands on remote Windows nodes via SSH connection.
- Supports selective file copying, subdirectory transfers, and move operations.
- Can copy specific files or all files from source to destination directories.
- Optionally includes subdirectories and empty directories in the copy operation.
- Supports move operation (optionally delete from source after successful copy).
- Returns detailed transfer statistics: copied files, speed, report, and created directories.
- Automatically creates destination folders if they don't exist.

3.14.2 Configuration Instructions

1. Create a new action step using the RobocopyOperation plugin.
2. Fill out the following fields:
    - Name: The name used to identify a saved Robocopy Operation configured instance.

- Comments: Comments associated to this template or configuration.
- Execution node: The remote Windows node to execute the Robocopy operation on. Select from the list of already configured Remote Nodes.
- Robocopy Binary path: The path to the Robocopy executable as seen in the command prompt (e.g., C:/Windows/System32/Robocopy.exe).
- Input folder path: The source path of the input folder from where the files are to be copied.
- Output folder path: The destination path where the files have to be transferred. If not present, Robocopy will create the folder.
- Input Files: Names of the files from source path that need to be copied. If left empty, *.* (all files) will be considered. Separate multiple files with spaces.
- Copy Sub Directories?: Check this option to copy all subdirectories from the given source path (excludes empty directories).
- Copy Empty Directories?: Check this option to copy all subdirectories including empty directories from the given source path.
- Delete files from source?: Check this option to move files from source to destination and delete them from source after successful copy.
- Command options (after the file names): Any Robocopy option or parameter. This string will be added to the Robocopy command after the input files list.

3.14.3 Example Configuration

This is an example configuration for the RobocopyOperation plugin to copy media files with subdirectories.

Parameter	Value
Name	Copy Media Files
Comments	Copy video files to archive location
Execution node	WindowsFileServer01
Robocopy Binary path	C:/Windows/System32/Robocopy.exe
Input folder path	C:/media/incoming
Output folder path	D:/archive/media
Input Files	*.mp4 *.avi
Copy Sub Directories?	Selected
Copy Empty Directories?	Not selected
Delete files from source?	Not selected
Command options	/R:3 /W:5 /MT:8

3.14.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved Robocopy Operation configured instance.
Comments	Comments associated to this template or configuration.
Execution node	The remote Windows node to execute the Robocopy operation on. Select from the list of already configured Remote Nodes.
Robocopy Binary path	The path to the Robocopy executable as seen in the command prompt (e.g., C:/Windows/System32/Robocopy.exe).
Input folder path	The source path of the input folder from where the files are to be copied.
Output folder path	The destination path where the files have to be transferred. If not present, Robocopy will create the folder.
Input Files	Names of the files from source path that need to be copied. If left empty, *.* (all files) will be considered. Separate multiple files with spaces.
Copy Sub Directories?	Check this option to copy all subdirectories from the given source path (excludes empty directories).
Copy Empty Directories?	Check this option to copy all subdirectories including empty directories from the given source path.
Delete files from source?	Check this option to move files from source to destination and delete them from source after successful copy.

Parameter	Description
Command options (after the file names)	Any Robocopy option or parameter. This string will be added to the Robocopy command after the input files list.

3.14.5 Inputs Description

Input folder path, Output folder path, Execution node, and Robocopy binary path are mandatory parameters for the execution of this plugin. All of them support both design time and runtime specification. They also support dynamic entry by introducing new variables. The run time inputs of this action depend on how the configuration is done. Variables exposed in the command line will be exposed as mandatory inputs.

Parameter	Description
Execution_node *	The remote Windows node identifier. Must be a configured Remote Node with SSH access to a Windows system.
Robocopy_Binary_path *	The full path to the Robocopy executable on the remote node. Default location: C:/Windows/System32/Robocopy.exe
Input_folder_path *	The source directory path from which files will be copied.
Output_folder_path *	The destination directory path where files will be copied. Created automatically if it doesn't exist.
Input_Files	Space-separated list of file names or patterns to copy (e.g., 'file1.txt file2.doc' or '*.xml *.json'). If empty, all files (*.*) are copied.
Copy_Sub_Directories	Boolean flag to include subdirectories in the copy operation (excludes empty directories).
Copy_Empty_Directories	Boolean flag to include empty directories along with subdirectories in the copy operation.
Delete_files_fror	Boolean flag to delete source files after successful copy (move operation).
Command_options	Additional Robocopy command-line options (e.g., '/R:3 /W:5 /MT:8' for retry, wait, and multi-threading).
Dynamic variables	Any variables used in the configuration will be exposed as mandatory runtime inputs.

3.14.6 Outputs Description

This action executes a Robocopy transfer action on a remote Windows node, via SSH.

Parameter	Description
files_transferred	The list of files successfully copied to the destination
transfer_speed	The speed at which the transfer was performed (e.g., MB/sec)
transfer_report	A detailed report of the Robocopy operation including statistics, errors, and warnings
new_directories_transferred	List of new directories created during the copy operation

3.14.7 Supported Actions

None

3.14.8 Dependencies

Robocopy comes installed by default on Windows from 2003 Server onwards. Check the folder C:/Windows/System32/ for Robocopy.exe before assuming it needs installation.

3.14.9 Operating Instructions

This action plug-in provides the ability to submit and control a file transfer operation using the Windows Robocopy toolset. Robocopy will be executed on a remote node via an SSH connection.

Command Syntax

- The Robocopy command syntax is:
- `robocopy <Source> <Destination> [<File>[ ...]] [<Options>]`
- Example:

  `C:/Windows/System32/Robocopy.exe C:/source D:/destination *.txt /S /R:3`
- The command is executed via SSH on the specified Windows remote node.

Common Robocopy Options

- Copy Options:
  - `/S` - Copy subdirectories (excludes empty ones)
  - `/E` - Copy subdirectories including empty ones
  - `/COPYALL` - Copy all file information (timestamps, attributes, security, owner)
  - `/COPY:DAT` - Copy data, attributes, and timestamps
  - `/DCOPY:T` - Copy directory timestamps
  - `/SEC` - Copy files with security (ACLs)
  - `/MOV` - Move files (delete from source after copy)
  - `/MOVE` - Move files and directories (delete from source after copy)
- Performance Options:
  - `/MT[:n]` - Multi-threaded copying with n threads (default 8, max 128)
  - `/R:n` - Number of retries on failed copies (default 1 million)
  - `/W:n` - Wait time between retries in seconds (default 30)
  - `/IPG:n` - Inter-packet gap in milliseconds to free bandwidth
  - `/NFL` - No file list (don't log file names)
  - `/NDL` - No directory list (don't log directory names)
  - `/NP` - No progress (don't display percentage copied)
- File Selection:
  - `/A` - Copy only files with Archive attribute set
  - `/M` - Copy only files with Archive attribute and reset it
  - `/XF file [file]...` - Exclude files matching given names/patterns
  - `/XD dirs [dirs]...` - Exclude directories matching given names/patterns
  - `/MAX:n` - Maximum file size (exclude files bigger than n bytes)
  - `/MIN:n` - Minimum file size (exclude files smaller than n bytes)
  - `/MAXAGE:n` - Maximum file age (exclude files older than n days)
  - `/MINAGE:n` - Minimum file age (exclude files newer than n days)
- Logging Options:
  - `/V` - Produce verbose output
  - `/NJS` - No job summary
  - `/TEE` - Output to console window and log file
  - `/UNICODE` - Output log in Unicode format

Common Use Cases

- Backup operations: Copy files to backup or archive locations with subdirectories.
- Media file management: Transfer media files between storage systems.
- Data migration: Move data from old servers to new infrastructure.

- Synchronization: Keep directories in sync between locations (with appropriate options).
- Selective file copying: Copy only specific file types or patterns.
- Network transfers: Transfer files between network shares with retry logic.
- Archive operations: Move processed files to archive storage (with delete option).

Best Practices

- Always use absolute paths for source and destination directories.
- Test Robocopy commands manually via command prompt before configuring in Orchestrator.
- Be cautious with delete options (/MOV, /MOVE) - test thoroughly before production use.
- Use file patterns (*.xml, *.mp4) for selective copying instead of copying all files.
- Ensure destination drive has sufficient space before large copy operations.
- Configure appropriate Remote Node timeout for large transfers.

Troubleshooting

- Access denied errors: Verify the SSH user has read permissions on the source and write permissions on the destination.
- Source path not found: Verify the source path is correct as seen from the Windows command prompt.
- Destination cannot be created: Check that the parent directory exists and the user has write permissions.
- Robocopy not found: Verify `Robocopy.exe` exists at the specified path (default: `C:/Windows/System32/Robocopy.exe`).
- SSH connection failure: Verify the Remote Node configuration and ensure the SSH service is running on Windows.
- Timeout on large transfers: Increase the Remote Node timeout or use `/IPG` to throttle the transfer rate.
- Files skipped: Check the exit code and report. This may be due to `/XF`, `/XD` exclusions, or file age/size filters.
- Slow performance: Use the `/MT` option for multi-threading or `/IPG` to manage bandwidth.

3.15 SCP transfer



Figure 3.15: SCP transfer icon

This action enables uploading or downloading files and directories to or from a remote host using SCP (secure copy protocol over SSH).

Version	Comment
0.2.1	Enhanced action template UI
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.1	Adding Remote Node Dependency
0.1.0	2016-02-17 - Added help file
0.0.9	2015-02-26 ML - New icon, fixes dependency on FTP, fixes display name for consistency
0.0.8	exception handling of RemoteNode.find_by_name()
0.0.7	Prevents autocomplete of user field
0.0.6	Adds display name
0.0.5	Hides password characters while entering in edit gui.
0.0.4	Fix naming mismatch between reporting_frequency and reporting_increment
0.0.3	Fix Typo in OPERATION_LIST
0.0.2	Output files changed to an array
0.0.1	Original release, does not support file base and file removal after transfer

3.15.1 How It Works

- Transfers files or directories between your local system and a remote server using the SCP protocol.
- Supports upload (local to remote), download (remote to local), and recursive directory transfers.
- Uses either password-based or SSH key-based authentication, configurable via plugin fields or settings.

3.15.2 Configuration Instructions

1. Create a new action step using the ScpTransfer plugin.
2. Fill out the following fields:
    - Name: A label for this SCP transfer step.
    - Comments: Notes or a description of this transfer step.
    - Remote node: The target server (select from configured nodes or enter details manually).
    - Remote address: IP address or DNS name of the remote server (required if not preconfigured).
    - Remote port: SSH port for the remote server (default: 22).
    - Remote user: Username for connecting to the remote server.
    - Remote password: Password for authentication (leave blank if using key-based authentication).
    - Remote key location: Path to the SSH key for key-based authentication (leave blank if using password).
    - Operation: Choose upload (local to remote) or download (remote to local).
    - Local path: Absolute path to the file or directory on the local system.
    - Remote path: Destination path on the remote server for uploads, or source path for downloads.
    - Recursive transfer: Enable to recursively transfer directories and their contents.
    - Preserve timestamp: If enabled, attempts to preserve file timestamps during transfer (server-dependent).
    - Reporting frequency (in seconds): Interval for transfer status updates (default: 5 seconds).

3.15.3 Example Configuration

Field	Value
Name	`New ScpTransfer Step`
Comments	Upload `releaseversion.txt` to remote FTP directory
Remote node	`-Select Node-`
Remote address	`9.46.236.152`
Remote port	`22`
Remote user	`ftpuser`
Remote password	`<password>`
Remote key location	(empty)
Operation	`upload`
Local path	`/opt/aspera/orchestrator/releaseversion.txt`
Remote path	`/home/ftpuser/ftp_test/test_scp_output.txt`
Recursive transfer	Unchecked
Preserve timestamp	Checked
Reporting frequency	`5`

3.15.4 Saved Parameters Description

Parameter	Description
Name	Display name for this SCP transfer step.
Comments	Notes or details for this transfer step.
Remote node	The remote node to connect to (if configured in Orchestrator).
Remote address	IP or hostname of the remote server (required if not selecting a node).
Remote port	SSH port for the remote connection (default: 22).
Remote user	Username for remote authentication.
Remote password	Password for authenticating to the remote server.

Parameter	Description
Remote key location	Filesystem path to SSH key for key-based authentication.
Operation	Upload or download direction.
Local path	Full path to the source file or directory on the local system.
Remote path	Destination or source path on the remote server.
Recursive transfer	Enables recursive transfer of directories and subfolders.
Preserve timestamp	Attempts to preserve original timestamps during transfer.
Reporting frequency	Interval for transfer status updates.

3.15.5 Inputs Description

- Any field not specified in the template can be provided as a runtime input.
- Fields set in the template cannot be overridden at runtime.

3.15.6 Outputs Description

Output	Description
Transferred files	List of file paths processed during the transfer.
Transferred bytes	Total amount of data transferred (in bytes).
Step information	Status and details for each completed operation, including source/destination and result.

3.15.7 Supported Actions

None

3.15.8 Dependencies

None

3.15.9 Operating Instructions

This plugin enables uploading or downloading files or directories to or from a remote server using SCP.

- If a directory is specified, all files and subfolders will be included when recursive transfer is enabled.
- Authentication methods include password and SSH key, configurable via plugin fields.
- The remote user must have appropriate permissions for the target directory.
- For directory transfers, the target directory must not already exist—otherwise, the operation will fail.

3.15.10 Troubleshooting

- Verify remote user permissions before starting the transfer.
- Ensure that the remote server is accessible via SSH on the specified port.
- Check that authentication credentials (password or SSH key) are correct.
- For directory operations, confirm that the target directory does not already exist.

3.16 SFTP transfer

This plugin provides secure file transfer functionality to SFTP servers, supporting uploads, downloads, and deletions with flexible authentication options, proxy support, and specialized operations for handling large files and directory structures.



Figure 3.16: SFTP transfer icon

Version	Comment
0.7.1	Enhanced action template UI
0.7.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.6.0	Add keepalive option
0.5.7	Fix Remote Node log info
0.5.6	Fix default port value and improve 'OS' transfer operations
0.5.5	Avoid printing sensible information in log
0.5.4	Correctly close connection on completion
0.5.3	Fix undefined error reported in some cases even if the transfer is complete
0.5.2	Support for Key Authentication
0.5.1	Exposes source base path in the designer configuration. Updated help file with multi source instructions.
0.5.0	Adds OS Upload option
0.4.9	Adding Remote Node Dependency
0.4.8	Rescues channel closing to avoid issue introdiced in 0.4.7
0.4.7	Forces the closing of channel to avoid leaving open file handles on upload
0.4.6	Fixes looping issue in 0.4.5
0.4.5	Fixes issue in 0.4.4
0.4.4	In the case of upload of large files, ignore errors if reported after transfer was completed
0.4.3	OS download target file path was providing a directory instead of files
0.4.2	OS download needs to be outside of Net SFTP block or else it times out
0.4.1	Handle special characters and spaces. No quotes needed around filename.
0.4.0	Still need to escape spaces, no need for quotes, and now throws error if file is not downloaded
0.3.9	Removing escapes as they are no longer necessary
0.3.8	Add quotes around download to handle escapes. Note - OpenSSH must be version 4.9p1 or greater.
0.3.7	Handle filenames with special characters. Uses Shellwords library.
0.3.6	Adds support for filenames with spaces in them
0.3.5	Fixes issues where logging output is incorrect; also fixes bug where output incorrectly displays success
0.3.4	Adds notes about OS download and sshpass prerequisite
0.3.3	Adds support for OS download to circumvent the 4GB limitation of the ruby sftp library
0.3.2	Removes local_path from required inputs on delete operation
0.3.1	New icon
0.3.0	Adds overwrite and file-base
0.2.3	Forces transfer file by file
0.2.2	Adds ability to provide transfer direction at runtime
0.2.1	Prevents autocomplete of user field
0.2.0	Makes transfer recursive, and makes proxy user and password optional
0.1.0	Adds support for sftp thru an http proxy
0.0.4	Adds display name
0.0.3	Hides password characters while entering in edit gui.
0.0.2	name fix in edit gui
0.0.1	initial release

3.16.1 How It Works

- Enables secure file transfers using the SFTP (Secure File Transfer Protocol) for uploads, downloads, and deletions to/from remote servers.
- Supports both file-based and directory-based operations with recursive processing of subdirectories and multiple file handling.
- Provides flexible authentication methods including password-based and SSH key-based authentication for secure connections.
- Offers multiple operation types including OS Upload, Upload, Download, Delete, Transfer, and OS Download for various transfer scenarios.
- Includes proxy support for environments requiring HTTP proxy connections to reach SFTP servers.
- Features progress reporting, timestamp preservation, and source file removal options for comprehensive transfer management.

3.16.2 Configuration Instructions

1. Add an SFTP Transfer step to your workflow.
2. Configure the remote server connection:
   - Remote node: Select a configured remote node, or
   - Remote address: Enter the SFTP server IP or hostname.
   - Remote port: Specify the SSH port (default: 22).
   - Remote user: Enter the username for SFTP authentication.
3. Set up authentication:
   - Remote Password: Enter password for password-based authentication, or
   - Remote key location: Provide path to private key file for key-based authentication.
4. Configure transfer parameters:
   - Operation: Select transfer type (Upload, Download, Delete, etc.).
   - Local path: Specify local file or directory path.
   - Remote path: Set remote file or directory path.

3.16.3 Example Configuration

Parameter	Example Value
Remote address	192.168.1.100
Remote port	22
Remote user	ftpuser
Remote Password	<configured password>
Operation	Upload
Local path	/opt/aspera/orchestrator/releaseversion.txt
Remote path	/home/ftpuser/sftp_test.txt
Source base path	(empty)
Preserve timestamp	unchecked
Remove source after transfer	unchecked

3.16.4 Saved Parameters Description

Parameter	Description
Local path	The file or directory absolute path on the local file system.
Remote path	The file or directory path on the remote server.
Source base path	The directory on the source server to use as base path.
Operation	The supported operations are OS Upload, Upload, Download, Delete, Transfer, OS Download.
Remote node	The remote node to connect to (alternative to specifying address directly).
Remote key location	Absolute path to your private key whose public key you have shared with the SFTP server.
Preserve timestamp	If set, tries to preserve the source file timestamps when performing the operation. This behavior is server dependent.

Parameter	Description
Remove source after transfer	If set, the source file (local or remote) will be deleted after successful transfer.
Reporting frequency	How often progress will be reported. Default is 5 seconds.
Proxy host	Optional: if an HTTP proxy is used to get to the SFTP server, the host name (or IP address) needs to be provided for the proxy. The host name is entered as proxy.lala.com or 10.0.200.202.
Proxy port	Optional: if an HTTP proxy is used, the port the proxy server is listening for requests on.
Proxy user	Optional: if an HTTP proxy is used and the proxy server requires authentication, the logon ID for the user.
Proxy password	Optional: if an HTTP proxy is used and the proxy server requires authentication, the password for the proxy user.
Keepalive?	If selected, a keepalive packet is sent to the SSH server at a regular interval.

3.16.5 Inputs Description

All parameters are available as inputs at runtime if not specified by the template. This plugin does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

- local_path: Absolute path to local file or directory for the transfer operation.
- remote_path: Path to remote file or directory on the SFTP server.
- operation: Transfer operation type (Upload, Download, Delete, Transfer, OS Upload, OS Download).
- remote_host: SFTP server hostname or IP address.
- remote_port: SFTP server port number (typically 22).
- remote_user: Username for SFTP server authentication.
- remote_password: Password for SFTP server authentication.
- remote_key_location: Path to private key file for key-based authentication.
- source_base_path: Base directory path for source operations.
- preserve_timestamp: Boolean flag to preserve file timestamps during transfer.
- remove_source: Boolean flag to delete source files after successful transfer.
- proxy_host: HTTP proxy server hostname or IP address.
- proxy_port: HTTP proxy server port number.
- proxy_user: Username for proxy server authentication.
- proxy_password: Password for proxy server authentication.

3.16.6 Outputs Description

- Transferred_files: The list of target file paths involved in the operation.
- Transferred_bytes: The total size (in bytes) processed by the operation.

3.16.7 Supported Operations

- Upload - Transfer files from local to remote server
- Download - Transfer files from remote server to local
- OS Upload - Upload using native SFTP command (supports large files, including files over 4 GB)
- OS Download - Download using native SFTP command (supports large files, including files over 4 GB)
- Delete - Remove files from remote server
- Transfer - Generic transfer operation

3.16.8 Operating Instructions

- File and Directory Support: This plugin allows uploads, downloads and deletions from/to a remote server using the SFTP protocol. It supports both file-based and directory-based operations: if a directory is specified, all included files and subfolders will be uploaded/downloaded.
- Multiple Input Files: Source path (either remote or local) can support a list of file paths using either an Array or a comma-separated list format for batch operations.
- Authentication Methods: Both password and key authentication schemas are supported, and they can either be specified in the Remote Node configuration or directly as inputs/parameters for the plugin.

- OS Transfer Operations: 'OS' transfer operations support files as sources only and use the native SFTP command-line tools available on the Orchestrator server.
- Directory Path Limitation: In case of path-based operations, the target directory must not exist! If it exists, the execution will fail with an error code that is server dependent.
- Large File Support: There is a known limitation of the Ruby SFTP library that causes the download of files larger than 4GB to fail. To circumvent this limitation, this plugin offers the 'OS Download' option. However, this option requires both that SFTP be available natively on the Orchestrator server AND that the sshpass utility be installed.
- Proxy Configuration: When using HTTP proxy, configure all proxy-related parameters including host, port, and authentication credentials if required by your proxy server.

3.16.9 Dependencies

- SSH/SFTP server running on the target remote server with proper user accounts and permissions.
- Network connectivity to the remote SFTP server on the specified port (typically port 22).
- For key-based authentication: properly configured SSH key pairs with the public key installed on the remote server.
- For OS operations: native SFTP client installed and available in the system PATH on the Orchestrator server.
- For OS Download of large files (>4GB): sshpass utility installed on the Orchestrator server (install with 'yum install sshpass' on Linux).
- For proxy connections: HTTP proxy server configured and accessible with appropriate authentication credentials.

3.16.10 Troubleshooting

- If connection fails, verify that the SFTP server is running and accessible on the specified port, and check network connectivity and firewall settings.
- For authentication failures, confirm that username/password are correct or that SSH keys are properly configured with the public key installed on the remote server.
- If directory operations fail, ensure that the target directory does not already exist, as SFTP requires the target directory to be created during the operation.
- For large file downloads (>4GB) that fail, use the 'OS Download' operation and ensure that both native SFTP and sshpass utilities are installed on the Orchestrator server.
- To verify sshpass installation, run 'which sshpass' from the command line - it should return the path to the utility.
- For proxy-related issues, verify proxy server settings, connectivity, and authentication credentials. Test proxy connectivity independently if possible.
- If timestamp preservation doesn't work as expected, note that this behavior is server-dependent and may not be supported by all SFTP server implementations.
- For permission errors, ensure that the remote user account has appropriate read/write permissions for the target directories and files on the SFTP server.

3.17 Shares transfer setup



Figure 3.17: Shares transfer setup icon

This plugin returns the transfer specification to be used for Node API-based transfers to/from a Share defined with Aspera Shares.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	Correctly detect transfer errors in some edge cases.
0.0.1	Initial implementation

3.17.1 How It Works

- Generates transfer specifications for Shares file transfers without performing the actual transfer operation.
- Supports both send (upload to Shares) and receive (download from Shares) transfer directions with flexible configuration options.
- Integrates with remote Aspera nodes or directly connects to Shares servers using IP addresses, ports, and authentication credentials.
- Provides comprehensive path handling for source files and destination folders, including Share name integration and relative path resolution.

3.17.2 Configuration Instructions

1. Add a Shares Transfer Setup step to your workflow.
2. Configure the connection method (choose one):
   - Remote Node: Select a configured Shares node from the Remote Nodes list.
   - Direct connection: Specify IP address, port, username, and password directly.
3. Set up transfer parameters:
   - Source filepath: Path to the source file for the transfer.
   - Destination folder: Target folder path for the transferred file.
   - Direction: Choose 'send' (upload) or 'receive' (download).
   - Destination filename: Optional custom filename if different from source.

3.17.3 Example Configuration

Parameter	Example Value
Remote Node	shares-server-01
Remote server ip address	192.168.1.100
Remote service port	443
Username	shareuser
Password	********
Source filepath	/vodmanager/john/source_video.mxf
Destination folder	/vodmanager/peter/
Destination filename	processed_video.mxf
Direction	send

3.17.4 Saved Parameters Description

Parameter	Description
Remote Node	Shares node name from the list of Remote Nodes. Use this for pre-configured connection settings.
Remote server ip address	Shares server IP address or domain name. Required when not using a remote node.
Remote service port	Shares service port (443 by default). Specify the port for Shares server communication.
Username	Shares user name for authentication. Required for accessing the Shares server.
Password	Shares user password for authentication. Required for accessing the Shares server.
Source filepath	The source file path. Format depends on transfer direction - see Operating Instructions for details.
Destination folder	The destination folder path. Format depends on transfer direction - see Operating Instructions for details.

Parameter	Description
Destination filename	The destination filename. To be filled only if different from the source filename.
Direction	The transfer direction: send or receive. Can be left empty if you want to fill it as run-time input.

3.17.5 Inputs Description

Changes in this list of inputs will override any value set in the action configuration/template.

- remote node or server address: Shares remote node name or node IP address or domain name.
- user name: Shares user name for authentication.
- password: Shares user password for authentication.
- source: Source filepath for the transfer operation.
- destination folder: Destination folder path for the transfer.
- destination filename: Destination filename if different from source.
- direction: Transfer direction (send or receive).

3.17.6 Outputs Description

- transfer spec: The transfer specification JSON content to be used as input for the node API transfer plugin (payload field).

3.17.7 Operating Instructions

- Activity Logging Setup: Make sure to have activity_logging configured on the Aspera initiating node. It can be set with these commands:

  ```
  asconfigurator -x "set_server_data;activity_logging,true"
  ```

  ```
  service asperanoded restart
  ```

- Connection Configuration: Specify a remote node pointing to your Shares server. In the remote node config, enter Node type IP, Shares IP address/domain name in Address, Shares username in Ssh/fasp login, Shares password in Ssh/fasp password, and optionally the Shares port in Tcp port. Alternatively, specify Shares connection details directly in the plugin Advanced parameters section.
- Source File Path Rules: For a send operation, specify the source filepath on the initiating node (e.g., /dir1/afile.mov), relative to the system user docroot linked to the initiating node API user. For a receive operation, specify the source filepath on Shares, including the Share name (e.g., /vodmanager/john/afile.mxf where vodmanager is the Share name).
- Destination Folder Rules: For a send operation, specify the folder on Shares including the Share name (e.g., /vodmanager/peter/ where vodmanager is the Share name). For a receive operation, specify the folder path on the initiating node (e.g., /dir2/) relative to the system user docroot linked to the initiating node API user.
- File Naming: Add the destination filename if you want a target filename different from the source filename. If not specified, the original filename will be preserved.
- Transfer Direction: Select 'send' for upload to Shares or 'receive' for download from Shares. This parameter can be configured at setup time or provided as a runtime input.
- Output Usage: In output, you'll get the transfer specification JSON content to be used in the payload field of the node API transfer plugin for executing the actual transfer.

3.17.8 Dependencies

- Activity logging must be configured and enabled on the Aspera initiating node for proper transfer tracking.
- Network connectivity between the Orchestrator and the specified Shares server IP address and port.
- Valid Shares user credentials with appropriate permissions for the specified Share and folder paths.
- Proper configuration of remote nodes (if using the Remote Node method) with correct Shares server details.

3.17.9 Troubleshooting

- If connection fails, verify the Shares server IP address and port are correct and accessible from the Orchestrator network.
- Check that the specified Share name exists and the user has appropriate permissions to access the specified paths.
- Verify that activity logging is properly configured on the initiating node if transfer tracking is not working correctly.
- Ensure source file paths are correctly specified relative to the appropriate docroot (initiating node for send, Shares for receive).
- For remote node connections, verify that the node configuration matches the actual Shares server settings and credentials.
- Check firewall rules and network policies that might block communication between Orchestrator and Shares servers on the specified port.

3.18 WebDAV Operation



Figure 3.18: WebDAV Operation icon

This action provides the ability to upload, download, post commands, make directory, gather properties, copy, move, delete or check files on a remote host which supports WedDAV protocol (extended version of he HTTP protocol)

Version	Comment
0.3.1	Enhanced action template UI
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.0	Adding Verify SSL option
0.1.1	Adding Remote Node Dependency
0.1.0	exception handling of RemoteNode.find_by_name()
0.0.9	Deletes orphan line causing XmlSimple to break
0.0.8	Graciously handles unauthorized access (code 401)
0.0.7	Provides ability to use dynamic attributes for user and password even if address line is provided
0.0.6	Prevents autocomplete of user field
0.0.5	Fails upload if http result is not in 200 range
0.0.4	Adds progress reporting during uploads
0.0.3	Adds icon
0.0.2	Updates code with a bug fix (escaping special characters in URLs) and correction in default input spec in which source and target were added twice
0.0.1	First version of the Webdav transfer plugin which supports Upload, Download, Copy, Delete, Head, Move, Post, Propfind, MakeDir

3.18.1 How It Works

- This action provides the ability to:
  - Upload a file
  - Download a file
  - Send a POST request with parameters
  - Create a directory
  - Retrieve properties of files and folders

  - Perform copy or move operations on the remote server
  - Delete a remote file
  - Check if a file or folder exists on a remote host
- For these commands to work, the remote server must support the WebDAV protocol (an extended version of the HTTP protocol).
- All operations are heavily dependent on the source path and target path input parameters.

3.18.2 Configuration Instructions

1. Create a new action step using the WebdavTransfer plug-in.
2. Fill out the following configuration fields:
   - Name: The name used to identify a saved WebdavTransfer configuration.
   - Comments: Optional comments about this saved WebdavTransfer configuration.
   - Remote node: The remote node to connect to. Nodes are entered on the Remote Nodes page.
   - Remote address: The hostname or IP address of the remote server.
   - Remote user: The username to use when connecting to the remote server.
   - Remote password: The password to use when connecting to the remote server.
   - Operation: The operation to perform on the remote server.
   - Use SSL?: Select if a secure connection is required.
   - Verify SSL?: Validate if a secure connection is available.
   - Source path: The file or directory (absolute path) on the local file system (for upload) or remote server (for download).
   - Target path: The file or directory path on the remote server (for all operations except download). For download, it is an absolute path to a local file.
   - Parameters for POST request: A hash of parameters to be sent via the POST request.
   - Remove source after transfer: If selected, the source file (local or remote) will be deleted after transfer.
   - Select user agent: The user agent to use in the HTTP request object.
   - Reporting frequency: How often progress will be reported. Default is 5 seconds.

3.18.3 Example Configuration

This is a minimal setup configuration for the WebdavTransfer plug-in.

Parameter	Value
Remote Address	192.168.1.10
Operation	Upload
Source Path	/home/user/file.txt
Target Path	/remote/folder/file.txt

3.18.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved WebdavTransfer configuration.
Comments	Optional comments about this saved WebdavTransfer configuration.
Remote node	The remote node to connect to. Nodes are entered on the Remote Nodes page.
Remote address	The hostname or IP address of the remote server.
Remote user	The username to use when connecting to the remote server.
Remote password	The password to use when connecting to the remote server.
Operation	The operation to perform on the remote server.
Use SSL?	Select if a secure connection is required.
Verify SSL?	Validate if a secure connection is available.
Source path	The file or directory (absolute path) on the local file system (for upload) or remote server (for download).
Target path	The file or directory path on the remote server (for all operations except download). For download, it is an absolute path to a local file.

Parameter	Description
Parameters for POST request	A hash of parameters to be sent via the POST request.
Remove source after transfer	If selected, the source file (local or remote) will be deleted after transfer.
Select user agent	The user agent to use in the HTTP request object.
Reporting frequency	How often progress will be reported. Default is 5 seconds.

3.18.5 Inputs Description

Required inputs depend on how the WebdavTransfer template is configured. If dynamic attributes are used, each attribute becomes a runtime input.

Parameter	Description
*Remote Node or Remote Address **	Either select a remote node or enter a remote address and its port.
*Source path **	The source path is mandatory for Upload, Download, Copy, and Move operations. Use this input to specify the source for the operation.
*Target path **	The target path is mandatory for all operations. Use this input to specify the target for the operation.
Remote password	If the remote server requires authentication, enter the password here.
Remote port	The port of the remote server.
Remote user	If the remote server requires authentication, enter the username here.

3.18.6 Outputs Description

Parameter	Description
Response_Code	The HTTP response code indicating the result of the operation.
Transferred_bytes	The total size (in bytes) processed by the operation.
Transferred_files	The list of target file paths involved in the operation.

3.18.7 Supported Actions

None

3.18.8 Dependencies

None

3.18.9 Operating Instructions

- This action allows you to perform the following operations on a remote server: Upload a file, download a file, send a POST request with parameters, create a directory, retrieve properties of files and folders, copy or move files on the remote server, delete a remote file, and check whether a file or folder exists on a remote host.
- Prerequisites:

  The remote server must support the WebDAV protocol (an extended version of the HTTP protocol). All operations depend heavily on the source path and target path input parameters.

- Path Requirements by Operation: | Operation(s) | Source Path | Target Path | |----|----|----| | Upload | Path to the local file (must be a valid file) | Path to the remote target for upload (must be a valid file or folder) | | Download | Path to the remote source file (must be a valid file) | Path to the local file or folder (must be a valid file or folder) | | Copy and Move | Path to the remote source file to copy or move (must be a valid file) | Path to the remote destination (must be a valid file) | | Delete, Propfind, MakeDir, Post, Head | N/A | Path to the remote target (must be a valid file or folder) |
- Important Notes
    1. Authentication: Only basic HTTP/HTTPS authentication is currently supported. Parameters can be specified either in the Remote Node configuration or directly as inputs/parameters for the plug-in via the template.
    2. Hostname/IP: Do not prefix the hostname or IP address with http or https. Instead, use the port and SSL input parameter to complete the specification.

Chapter 4

File Transformations

4.1 ADI Transformation



Figure 4.1: ADI Transformation icon

This plugin provides ability to convert ADI 1.1 version compliant XML files to ADI 3.0 version and vice versa

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.7	Commenting out error messge polluting the log
0.0.6	Added help file content.
0.0.5	Fixing issue with too much Max HEAP size is defaulted. Setting to 512M
0.0.4	- 2015/07/22 - PM - Fixing issue with ADI XML as output - adding it to outputs spec so that it can show up in the step outputs
0.0.3	- 2015/05/21 - PM - Exposing ADI XML as an extra output
0.0.2	- 2015/05/21 - PM - Exposing ADI Hash as an extra output
0.0.1	First version

4.1.1 Saved Parameters Description

Name: The name used to identify a saved ADI Transformation configured instance.

Comments: The comments to identify the Adi Parser template.

ADI File path: The file path to the ADI file.

XSL File path: The file path to the XSL file.

Target path: Target file is the file location where the transformed file will be placed.

Conversion: Conversion type, choose from 'Convert from 1.1 to 3.0' OR 'Convert from 3.0 to 1.1'. Default is 'Convert from 1.1 to 3.0'.

Jar path: For the 'Convert from 3.0 to 1.1' option in the above option, conversion requires external Java jar library. Currently this parameter is optional and the JAR file is already shipped with the plugin.

4.1.2 Inputs description

The list of inputs depends on the configuration of the ADI Transformation action template.

- All required parameters are available as inputs at runtime if not specified by the template.

  This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

These inputs are mandatory for ADI Parsing and validation:

- *ADI File path*: The ADI file path must be provided, for the conversion to happen.
- *XSL File path*: The XSL file path must be provided, for the conversion to happen.

These inputs are optional:

- *Name*: It is advisable that a name is entered to identify the template.
- *Comments*: Information about the template which will help identify the template.
- *Target path*: Target file is not mandatory. If not given, by default it will be written in the same directory.
- *Jar path*: Currently this parameter is optional and the JAR file is already shipped with the plugin.

4.1.3 Outputs description

- *Output File Path*: File name after conversion.
- *Is_Valid_ADI*: This is a flag and is 'true' if XML file is valid after the conversion is successful. 'false' otherwise.
- *Is_Valid_xsl*: This is a flag and is 'true' of XSL file is valid after the conversion is successful. 'false' otherwise.
- *ADI_Hash*: Content of converted file in for of hash.
- *ADI_XML*: Content of the converted XML file.

4.1.4 Operating Instructions

This plug-in allows users convert ADI XML with version 1.1 to version 3 and vice versa.

Providing the mandatory inputs listed above, the plugin converts ADI XML in selected version format and is placed in the provided target path.

4.2 Imagemagick



Figure 4.2: Imagemagick icon

This action plug-in can be used to retrieve media information about an image or apply transformation to that image using the popular imagemagick application.

Version	Comment
0.1.3	Enhanced action template UI
0.1.2	Cleanup and remove unused code
0.1.1	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	Remove hardcoded aspera directory reference
0.0.9	Adding Remote Node Dependency
0.0.8	2015-02-24 ML - New icon

Version	Comment
0.0.7	2015-02-17 ML - ignores i/o line that are not parsed correctly
0.0.6	fix runtime specification of execution node
0.0.5	preventing autocomplete of user info by browsers
0.0.4	adds support for composite command (used for watermarking)
0.0.3	adds support for spaces in filenames
0.0.2	adds functionality to mask password
0.0.1	initial release, only supports convert and identify

4.2.1 How It Works

- Executes ImageMagick command line operations on a remote node where ImageMagick is installed, connecting via SSH.
- Supports the identify command for extracting image metadata, the convert command for image manipulation and conversion, and the composite command for overlaying images.
- Accepts a media file path as the source file for analysis or conversion, with support for dynamic runtime inputs via ERB-style template variables.
- Accepts command options passed before the file name and command line arguments passed after, giving full control over the ImageMagick command line.
- Defaults to -verbose for command options if none are specified.
- Supports an optional path to the ImageMagick binaries for environments where the binary folder is not in the system path.
- Supports optional execution credentials (login and password) for connecting to the remote node, falling back to the node's default account if not specified.
- Returns a hash of metadata or conversion result information extracted from the ImageMagick output.

4.2.2 Configuration Instructions

1. Create a new action step using the Imagemagick plugin.
2. Fill out the following fields:
    - Name: The name used to identify a saved Imagemagick configured instance.
    - Comments: Comments about this saved Imagemagick configured instance.
    - Media file path: The source file for the analysis or conversion operation. Supports dynamic ERB-style template variables.
    - Execution Node: The remote node where ImageMagick is installed and where the operation will be executed.
    - Execution Login: The user account used to connect to the remote node. Defaults to the node's default account if not specified.
    - Execution Password: The password for the remote node login. Defaults to the node's default password if not specified.
    - Path to the folder containing the binaries: Path to the ImageMagick binary folder on the remote node. Not required if the binary folder is already in the system path for the remote account.
    - Command options: Optional parameters passed to the ImageMagick binary before the media file name. Defaults to '-verbose' if left blank.
    - Command line arguments: Optional parameters passed after the media file name. Used with 'convert' to specify a target file path, or with 'composite' to specify overlay and output images.

4.2.3 Example Configuration

This is an example configuration for the Imagemagick plugin extracting metadata from an image file using the identify command.

Parameter	Value
Name	Identify Image Metadata
Comments	Extract metadata from source image using ImageMagick identify
Media file path	/data/ingest/<%= Image_name %>.jpg
Execution Node	imagemagick-server

Parameter	Value
Execution Login	(blank - use node default)
Execution Password	(blank - use node default)
Path to the folder containing the binaries	/opt/local/bin
Command options	-verbose
Command line arguments	(blank)

4.2.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved Imagemagick configured instance.
Comments	Comments about this saved Imagemagick configured instance.
Media file path	The source file for the analysis or conversion operation. Supports dynamic ERB-style template variables.
Execution Node	The remote node where ImageMagick is installed and where the operation will be executed.
Execution Login	The user account used to connect to the remote node. Defaults to the node's default account if not specified.
Execution Password	The password for the remote node login. Defaults to the node's default password if not specified.
Path to the folder containing the binaries	Path to the ImageMagick binary folder on the remote node. Not required if the binary folder is already in the system path for the remote account.
Command options	Optional parameters passed to the ImageMagick binary before the media file name. Defaults to '-verbose' if left blank.
Command line arguments	Optional parameters passed after the media file name. Used with 'convert' to specify a target file path, or with 'composite' to specify overlay and output images.

4.2.5 Inputs Description

The available inputs depend on the template configuration. Fields that use dynamic ERB-style variables expose those variables as mandatory runtime inputs of type string.

Parameter	Description
Media file path	By default, a file name must be provided as an input for identify, convert, and composite commands. If the 'Media file path' field is populated in the template, all dynamic variables referenced in it become mandatory runtime inputs of type string.
Options	If 'Command options' is left blank in the template, an optional Options input is available at runtime. If the field is populated, all dynamic variables referenced in it are mandatory runtime inputs of type string.
Command arguments	Follows the same pattern as Options but applies to the 'Command line arguments' template parameter. Dynamic variables referenced in it are mandatory runtime inputs of type string.

4.2.6 Outputs Description

The output is a hash of information returned by ImageMagick. The structure varies by command and options used. Sample output hashes are provided below.

Parameter	Description
Image_info	A hash containing metadata or conversion result information returned by ImageMagick. The fields in the hash depend on which command and options are used. If a custom output format option is used, the output is returned as-is and parsing is the responsibility of the next workflow step.

4.2.7 Supported Actions

None

4.2.8 Dependencies

Requires ImageMagick to be installed on the remote Execution Node. The remote node must have SSH enabled. For Windows nodes, OpenSSH must be installed.

4.2.9 Operating Instructions

This action executes an ImageMagick operation on a remote node over SSH. The identify, convert, and composite commands are supported, with full control over options and arguments via template fields and dynamic runtime inputs.

Using the identify Command

- The identify command line is structured as: `[binary path]/identify [options] <Media file path> [command arguments]`
- Example: `/opt/local/bin/identify -verbose /data/ingest/bozo.jpg`
- Command options default to `-verbose` if left blank. Command arguments are typically left blank for identify.
- Run `which identify` on the remote server to find the binary path if not known.

Using the convert Command

- The convert command line is structured as: `[binary path]/convert [options] <Media file path> [command arguments]`
- Example — resize: `/opt/local/bin/convert -verbose /data/ingest/bozo.jpg -resize 50% /data/ingest/bozo_small.png`
  - Path to binaries: `/opt/local/bin`
  - Command options: `-verbose`
  - Media file path: `/data/ingest/<%= Image_name %>.jpg`
  - Command arguments: `-resize 50% /data/ingest/<%= Image_name %>_small.png`
  - Dynamic input `Image_name` mapped to the string `bozo`
- Example — label generation: `/opt/local/bin/convert -background lightblue -fill blue -font Candice -size 165x70 -pointsize 24 label:Maurice /data/labels/Maurice.gif`
  - Command options: `-background lightblue -fill <%= Color %> -font Candice -size 165x70 -pointsize 24 label:<%= Label_name %>`
  - Media file path: `/data/label/<%= Label_name %>.gif`
  - Dynamic inputs `Color` and `Label_name` mapped to `blue` and `Maurice`

Using the composite Command

- The composite command line is structured as: `[binary path]/composite [options] <Media file path> [command arguments]`
- The Media file path field specifies the overlay (foreground) image — the image to be composited on top of another.
- The Command line arguments field specifies the base (background) image and the output file path.
- Example — watermark overlay: `/opt/local/bin/composite -dissolve 50% /data/watermarks/logo.png /data/ingest/photo.jpg /data/output/photo_watermarked.jpg`
  - Path to binaries: `/opt/local/bin`
  - Command options: `-dissolve 50%`
  - Media file path (overlay): `/data/watermarks/logo.png`

  - Command arguments (base image + output): `/data/ingest/photo.jpg /data/output/photo_watermarked.jpg`
- Run `which composite` on the remote server to find the binary path if not known.

Best Practices

- Always provide the explicit path to the ImageMagick binaries to avoid configuration issues caused by differences in the system path between accounts. Use `which convert`, `which identify`, or `which composite` on the remote server to find the path.
- Ensure the source file is accessible from the remote node before the workflow runs — either transferred to the server or accessible via a network share. This plugin does not perform file movement.
- Use dynamic ERB-style template variables in the Media file path, Command options, and Command line arguments fields to build flexible, reusable workflow steps.
- For convert and composite operations, always specify the target output file path in the Command line arguments field to define where the output file will be written.
- For composite operations, ensure both the overlay image (Media file path) and the base image (Command line arguments) are accessible on the remote node.
- Ensure the remote node has SSH enabled. For Windows nodes, install OpenSSH to enable SSH connectivity.
- Test ImageMagick commands directly on the remote server before configuring them in the workflow to validate syntax and options.
- If using custom output format options (e.g., `-format`), be aware that the Results output will be returned as raw text — implement parsing logic in a subsequent workflow step.

Troubleshooting

- Binary not found: Verify the path to the ImageMagick binary folder is correct on the remote node. Run `which convert`, `which identify`, or `which composite` on the remote server to confirm the path.
- SSH connection failure: Ensure the remote node has SSH enabled and is accessible from the Orchestrator server. Verify the Execution Login and Password are correct.
- File not found: Confirm the media file path is correct and the file is accessible from the remote node. This plugin does not transfer files. For composite, verify both the overlay and base image paths are accessible.
- Permission denied: Ensure the Execution Login account has read access to all source files and write access to the target output directory.
- Empty or unexpected Results output: Check that the Command options and Command line arguments are correctly formatted. Test the full command directly on the remote server.
- Dynamic variable not resolved: Ensure all ERB-style variables referenced in the template fields are provided as runtime inputs of type string.
- Windows node issues: Ensure OpenSSH is installed and running on the Windows node if using a Windows remote server.

Important Notes

- This plugin does not perform file movement — all source files must already be accessible from the remote node before the workflow step runs.
- The connection to the remote node is made over SSH. Ensure SSH is enabled on the target server. For Windows, OpenSSH must be installed.
- Supported commands are identify, convert, and composite.
- Command options default to -verbose if left blank.
- For the convert command, verbose output is written to stderr and is parsed automatically by the plugin.
- If a custom output format option is used, the Results output is returned as raw text and must be parsed by a subsequent workflow step.
- All dynamic variables referenced in template fields are mandatory runtime inputs of type string when those fields are populated.
- The Results hash structure varies depending on the command used (identify, convert, or composite) and the options applied.

Sample Results — convert Command

The following is a sample Results hash returned by a convert operation:

\ {\ ”Converted_Image” => ”/rails_root/Boots_small.png”,\ ”Image” => ”/rails_root/Boots.jpg”,\ ”Properties” => ”JPEG 3072x2304=>1536x1152 1536x1152+0+0 8-bit DirectClass 3.637MB 1.740u 0:01.720”\ }\

Sample Results — identify Command

The following is a sample Results hash returned by an identify -verbose operation:

\ {\ 'Image' => 'public/images/success.png',\ 'Format' => 'PNG (Portable Network Graphics)',\ 'Class' => 'DirectClass',\ 'Geometry' => '48x48+0+0',\ 'Resolution' => '28.35x28.35',\ 'Print size' => '1.69312x1.69312',\ 'Units' => 'PixelsPerCentimeter',\ 'Type' => 'TrueColorMatte',\ 'Colorspace' => 'sRGB',\ 'Depth' => '8-bit',\ 'Channel depth' => { 'red' => '8-bit', 'green' => '8-bit', 'blue' => '8-bit', 'alpha' => '8-bit' },\ 'Channel statistics' => {\ 'Red' => { 'min' => '0 (0)', 'max' => '255 (1)', 'mean' => '142.447 (0.558618)',\ 'standard deviation' => '114.972 (0.450871)', 'kurtosis' => '-1.89177', 'skewness' => '-0.204789' },\ 'Green' => { 'min' => '0 (0)', 'max' 'standard deviation' => '74.0432 (0.290365)', 'kurtosis' => '0.715435', 'skewness' => '-1.20533' },\ 'Blue' => { 'min' => '0 (0)', 'max' => 'standard deviation' => '117.118 (0.459284)', 'kurtosis' => '-1.88585', 'skewness' => '-0.209334' },\ 'Alpha' => { 'min' => '0 (0)', 'max' 'standard deviation' => '122.454 (0.480214)', 'kurtosis' => '-1.90476', 'skewness' => '0.239299' }\ },\ 'Image statistics' => {\ 'Overall' => { 'min' => '0 (0)', 'max' => '255 (1)', 'mean' => '146.484 (0.574449)',\ 'standard deviation' => '108.872 (0.42695)', 'kurtosis' => '-1.55176', 'skewness' => '-0.368109' }\ },\ 'Colors' => '631',\ 'Rendering intent' => 'Perceptual',\ 'Gamma' => '0.45455',\ 'Chromaticity' => {\ 'red primary' => '(0.64,0.33)',\ 'green primary' => '(0.3,0.6)',\ 'blue primary' => '(0.15,0.06)',\ 'white point' => '(0.3127,0.329)'\ },\ 'Interlace' => 'None',\ 'Background color' => 'white',\ 'Compression' => 'Zip',\ 'Properties' => {\ 'date:create' => '2011-12-22T15:32:51-08:00',\ 'date:modify' => '2011-12-22T15:32:51-08:00',\ 'signature' => '33f7a9e4cd2a4 },\ 'Profiles' => {\ 'Profile-icc' => '3144 bytes',\ 'Description' => 'sRGB IEC61966-2.1',\ 'Manufacturer' => 'IEC http://www.iec.c 'Model' => 'IEC 61966-2.1 Default RGB colour space - sRGB',\ 'Copyright' => 'Copyright (c) 1998 Hewlett-Packard Company'\ },\ 'Artifacts' => { 'filename' => 'public/images/success.png', 'verbose' => 'true' },\ 'Filesize' => '6.25KB',\ 'Number pixels' => '2.3K',\ 'Version' => 'ImageMagick 6.7.6-7 2012-05-01 Q16 http://www.imagemagick.org'\ }\

4.3 MacCaption Operation



Figure 4.3: MacCaption Operation icon

This action plug-in provides the ability to perform captioning operation using the Telestream MacCaption application.

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	First version
0.0.3	Adding Remote Node Dependency
0.0.2	Code fix
0.0.1	First version

4.3.1 Saved Parameters Description

- This action allows to execute an MacCaption task on a remote node.
- Name: The name used to identify a saved MacCaptionOperation configured instance.
- Comments: Comments associated to this template or configuration.
- Execution node: The remote node to execute the MacCaption operation from. List of already configured Remote Nodes.
- MacCaption Binary path: The path (as seen in the SSH session) of the MacCaption executable.
- Operation: Select the operation to be performed by MacCaption application on the selected file Some of the following inputs can vary based on the operation selected.
- Input caption source file path: The path (as seen in the SSH session) of the input file to MacCaption.
- Input caption format: The format of the input file to MacCaption.
- Input options: The options to be given in the command for the input file to MacCaption.

- Output media file path: The path (as seen in the SSH session) of the output media file from MacCaption.
- Output caption file path: The path (as seen in the SSH session) of the output caption file from MacCaption.
- Output caption format: The format of the output caption file from MacCaption.
- Output options: The options to be given in the command for the output file to MacCaption.
- Job options: The options to be given in the command for the operation performed on the file.
- Options: Any other options to be given in the command for the MacCaption operation to be performed on the file.

4.3.2 Inputs description

- The run time inputs of this action depend on how the configuration is done. Variables exposed in the command line will be exposed as mandatory inputs.

4.3.3 Outputs description

- This action executes an MacCaption action on a remote node, via SSH.
- *Raw_report*: A report of the MacCaption operation There can be other outputs for some operations.

4.3.4 Supported Actions

None

4.3.5 Dependencies

None

4.3.6 Operating Instructions

This action plug-in provides the ability to submit and control a captioning operation using the MacCaption application.

MacCaption will be executed on a remote node via an SSH connection.

NOTE: MacCaption needs to be installed in the remote node.

The result file will be stored on the specified path in the remote node.

4.4 MXF Legalizer



Figure 4.4: MXF Legalizer icon

This plug-in provides the ability to make compliant a MXF file. It implements the MXF Legalizer v2.5.2 SOAP API.

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	Added option to output the result as Hash
0.0.1	initial release

4.4.1 Saved Parameters Description

- Name: The name used to identify a saved MxfLegalizer configured instance.
- Comments: Some comments about this saved MxfLegalizer configured instance.
- Remote Node: The MXF Legalizer server node name from the list of Remote Nodes.
- Remote server ip address: The MXF Legalizer server node IP address or domain name.
- Remote soap service port : The MXF Legalizer SOAP API service port (8080 by default).
- Source: The path to the source file.
- Priority: The priority for the transform request.
- Resource ID: The resource id for the transform request.
- Expose result as Hash: If selected, the operation result is exposed as an Hash.
- Polling frequency: Interval in seconds to check for the job submission status.

4.4.2 Inputs description

The list of inputs depends on the configuration of the MxfLegalizer action template.

Changes in this list of inputs will override any value set in the action configuration/template.

- *remote node or server address*: MXF Legalizer remote node name or node IP address or domain name
- *source*: The path to the source file
- *priority*: The priority for the transform request
- *resource_id*: The resource id for the transform request

4.4.3 Outputs description

- *XML_result*: The job submission XML output

4.4.4 Supported Actions

None

4.4.5 Dependencies

None

4.4.6 Operating Instructions

Specify a remote node or an IP address/domain name for the MXF Legalizer server.

Specify the full source file path (e.g. file:///C:/cwf/data/media/test-files/MxfLegalizer_Testfile_XDCAM-HD422.mxf).

Select the priority from the drop down list.

Specify the resource id. It can be found in the Settings Window of the MXF Legalizer GUI, when logged in as the user that starts the job. If no GUI is available, the default resource id 14b2303b-f7cd-4164-af42-e91f6dc3f5ef can be used.

In output, you'll get the XML content returned from the job submission.

Note: upon wrong source file path or resource id, a 500 Internal Server Error is returned by the plugin.

4.5 Subler Operation

This action plug-in provides the ability to perform file transformation operations on a MAC System using the Subler toolset.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.4	Removing Windows support code
0.0.3	Adding Remote Node Dependency
0.0.2	- 2015/02/09 - JG -> Parse Code Commented since its not used.

Version	Comment
0.0.1	-> Initial release.

4.5.1 Saved Parameters Description

- This action plug-in provides the ability to perform file transformation operations on a MAC System using the Subler toolset.
- Name: The name used to identify a saved SublerOperation configured instance.
- Comments: Comments associated to this template or configuration.
- Execution node: The remote node to execute the Subler operation from. List of already configured Remote Nodes.
- Subler Binary path: The path (as seen in the SSH session) of the SublerCLI executable.
- Source Path: The path (as seen in the SSH session) of the input file to Subler(Optional).
- Destination path: The path (as seen in the SSH session) of the output file of Subler (Optional).
- Chapters File path: The path (as seen in the SSH session) of the file (Only .txt accepted) that provides Chapter information(Optional).
- Source options: Any Subler Source option or parameter. This string will be added to the Subler command after the Source path.
- Destination options: Any Subler Destination option or parameter. This string will be added to the Subler command after the Destination path.
- Command Line: In Case, where predefined operation is not available, this command line text allows user to enter their entire command line Subler command and executed as it is.
- Command options: Any Subler option or parameter for both and source and destination. This string will be added to the Subler command after the source and the destination path if present
- Operation: Populates a list of predefined operations available on Subler.

  Details:

  'Optimize' - On Entering the destination path this operation will automatically perform the Subler Optimize operation.

  'List Tracks' - Given a source path, this option will automatically fetch the Tracks present inside the source file and list them in the execution outputs.

  'Remove Metadata' - Removes the Metadata (If present) from the given Destination file.

  'List Metadata' - Lists the Metadata present along with the Source file provided.

  'MUX Operation' - This option helps to perform the Mux operation. Given the path to the Chapters file along with the Source path this option performs muxing and stores the result in the given destination path.

  'Remove Existing Subtitles' - Given the Source path, this option removes all the subtitles associated with it.

  'Command Line' – Gives User the ability to enter their own command line Subler command to be executed as it is.

4.5.2 Inputs description

- Execution Node is the only mandatory parameter required for this plugin.

  Subler plugin works on different modes which makes every other parameter optional. Source path is generally needed only incase of Remux operation. Similarly, some options are available for just the source file (Example listtracks) in which case the destination path is not needed.

  Hence, use this plugin by either providing the source or the destination using appropriate options.

  To provide better usability some of the operations of the Subler Plugin are predetermined in this plugin.

  Hence, the Design time inputs of this plugin depends on the operation chosen.

  The run time inputs of this action depend on how the configuration is done. Variables exposed in the command line will be exposed as mandatory inputs.



Figure 4.5: Subler Operation icon

4.5.3 Outputs description

- This action executes an Subler transformation action on a remote node, via SSH.
- *Result_file_path*: The path of the output file
- *transformation_report*: A report of the Subler operation
- *execution_output*: A execution output as produced by the Subler execution

4.5.4 Supported Actions

None

4.5.5 Dependencies

None

4.5.6 Operating Instructions

This action plug-in provides the ability to perform file transformation operations using the Subler toolset.

Subler will be executed on a remote node(MAC OS) via an SSH connection.

NOTE: SublerCLI needs to be installed in the remote node.

The transformation result file will be stored on the specified path in the remote node.

Below are some examples showing different ways the plugin can be used.

Note: Execution node is not included below since the examples are all run on a SSH client.

Example 1 (Simple Subler workflow with Source and Destination along with Chapters path provided):

In this example, we show the basic working of Subler

CONFIGURATION:

Operation : ”MUX Operation”

Source path : ”~/Desktop/bababa.mp3”

Destination path : ”~/Desktop/bababa.mp4”

Chapters File path : ”~/Desktop/chapters.txt”

Subler binary path : ”/usr/local/bin/SublerCLI”

OUTPUTS:

Performs the muxing operation and Shows the progress report along with the status of the tranformation

Example 2 (Subler Workflow with only source provided):

In this scenario, we are providing only Source path with extra options.

CONFIGURATION:

Operation : ”List Tracks”

Source path : ”~/Desktop/bababa.mp4”

Subler binary path : ”/usr/local/bin/SublerCLI”

OUTPUTS:

Shows the progress report along with the status of the tranformation

Shows the execution output which would have the track information.

Example Tracking Info

Track: 1, Video Track, 0:04:06:73, 216 kbit/s, H.264, 426 x 240

Track: 2, Chapter Track, 0:04:06:73, 0 kbit/s, Text

Track: 3, Video Track, 0:04:06:73, 0 kbit/s, Photo-JPEG, 426 x 240

Example 3 (Subler Workflow with destination provided):

In this example, we use only the destination file and options to execute subler.

CONFIGURATION:

Operation : ”Optimize”

Destination path : ”~/Desktop/bababa.mp4”

Subler binary path : ”/usr/local/bin/SublerCLI”

OUTPUTS:

Shows the progress report along with the status of the transformation

Shows the execution output saying ”Optimizing done”

4.6 XSLT Transformation



Figure 4.6: XSLT Transformation icon

This action plug-in is used to transform XML file using XSLT.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.5	saving to target file bug fix
0.0.4	Implemented by xslt transformation using nokogiri - removing xslt4r
0.0.3	adds inline help file
0.0.2	adds display name and enable rich text editing
0.0.1	initial release

4.6.1 How It Works

- This action performs a file transformation on XML content using a provided XSLT template.
- XSLT (Extensible Stylesheet Language Transformations) transforms XML into other XML formats or into outputs such as HTML, plain text, or XSL-FO (which can be converted to PDF, PostScript, or PNG).
- XSLT is a standards-based, Turing-complete language; all transformation logic must be defined in the XSLT template.
- XSLT templates are independent of Aspera and Aspera Orchestrator.
- For examples and guidance, consult external XSLT documentation (for example, Wikipedia).

4.6.2 Configuration Instructions

1. Create a new action step using the XsltTransformation plugin.
2. This action executes an XSLT transformation on provided XML content.
3. Fill out the following configuration fields:
   - Name: The name used to identify a saved XsltTransformation configuration.
   - Comments: Optional comments about this saved XsltTransformation configuration.
   - XML body: Optional. Paste XML content to transform. Dynamic attributes are supported. Typically left blank when XML is provided at runtime.
   - XML file path: Optional. Path to a local XML file; may include dynamic attributes (e.g. /data/<%=provider%>/<%=format%>_template.xml).
   - XSLT body: Optional. Paste XSLT code here. Dynamic attributes are supported.
   - XSLT file path: Optional. Path to a local XSLT file; may include dynamic attributes.
   - Target file path: Optional. If provided, the transformed output is written to this path.
   - Return file as output: If checked, the transformed content is returned as an action output. If both this option and a target file path are provided, both outputs are produced.

4.6.3 Example Configuration

This provides a minimal configuration example for the XsltTransformation plugin.

4.6.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved XsltTransformation configuration.
Comments	Optional comments about this saved XsltTransformation configuration.
XML body	Optional. Paste XML content to transform. Dynamic attributes are supported. Typically left blank when XML is provided at runtime.
XML file path	Optional. Path to a local XML file; may include dynamic attributes (e.g. /data/<%=provider%>/<%=format%>_template.xml).
XSLT body	Optional. Paste XSLT code here. Dynamic attributes are supported.
XSLT file path	Optional. Path to a local XSLT file; may include dynamic attributes.
Target file path	Optional. If provided, the transformed output is written to this path.
Return file as output	If checked, the transformed content is returned as an action output. If both this option and a target file path are provided, both outputs are produced.

4.6.5 Inputs Description

Required inputs depend on how the XsltTransformation template is configured. If dynamic attributes are used, each attribute becomes a required runtime input.

Parameter	Description
XML_content_or_path *	Required only when both 'XML body' and 'XML file path' are left blank. Provide raw XML text or a local file path.
XSLT_content_or_path *	Required only when both 'XSLT body' and 'XSLT file path' are left blank. Provide XSLT text or a local file path.

Parameter	Description
Target_file_path	Optional. Only required if the target path was not specified in the configuration.

4.6.6 Outputs Description

Parameter	Description
Generated_file_path	If the action writes the transformed file, this output contains the file path.
Generated_file_content	If the action returns the transformed content, this output contains the generated text.

4.6.7 Supported Actions

None

4.6.8 Dependencies

None

4.6.9 Operating Instructions

1. Provide XML content (or a path to an XML file) to be transformed.
2. Provide XSLT code (or a path to an XSLT file) that defines the transformation logic.
3. Ensure the XSLT conforms to standard XSLT rules; the action relies on a standard XSLT processor.
4. Execute the action to produce output (XML, HTML, plain text, or XSL-FO). If a target path is set, the file is written there; if 'Return file as output' is enabled, the content is returned as an output.
5. For help authoring XSLT templates, consult external documentation such as Wikipedia.

Chapter 5

Integration

5.1 Amazon DynamoDB Operation

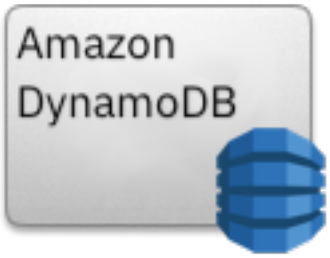


Figure 5.1: Amazon DynamoDB Operation icon

This action plug-ins can be used to interact with Amazon DynamoDB.

Version	Comment
0.4.1	Enhanced action template UI
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.0	Add assume role
0.2.0	Add http proxy support
0.1.3	Allows dynamic parameters in Input Hash.
0.1.2	Avoid autofill of key and secret
0.1.1	Masking of AWS Secret
0.1.0	Allows for SSL to not be used
0.0.1	Initial release

5.1.1 How It Works

- Performs Amazon DynamoDB item-level operations including Get, Put, Delete, Batch Get, and Batch Write.
- Supports single-item operations: Get Item, Put Item, and Delete Item on a specified table.
- Supports batch operations: Batch Get Item (up to 100 items) and Batch Write Item (put/delete) across one or more tables.
- Accepts input as a Ruby hash representing keys, items, or batch request structures depending on the operation.
- Supports both simple primary keys (partition key only) and composite primary keys (partition key + sort key).
- Optionally supports IAM role assumption via AWS STS for temporary credential generation.
- Returns item data as attribute maps, with pre-operation state for Put and Delete operations.
- Returns Item Collection Metrics for Batch Write operations where applicable.

5.1.2 Configuration Instructions

1. Create a new action step using the AmazonDynamoDBOperations plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved AmazonDynamoDBOperations configured instance.
   - Comments: Comments about this saved AmazonDynamoDBOperations configured instance.
   - Aws key: The access key credentials provided by Amazon.
   - Aws secret: The secret key credentials provided by Amazon.
   - Aws region: The region under which DynamoDB is hosted.
   - Verify SSL?: Check this box to use SSL.
   - Proxy URL: The URL of the proxy server through which AWS will be accessed. Leave blank if not using a proxy.
   - External id: External id when assuming a role (optional).
   - Policy: IAM policy in JSON format when assuming a role (optional).
   - Role arn: Amazon Resource Name (ARN) of the role. Required to assume a role.
   - Token duration: Duration of the generated token when assuming a role, in seconds. Default is 3600s (1 hour).
   - Operations: The operation to be performed on DynamoDB. Supported: Get Item, Put Item, Delete Item, Batch Get Item, Batch Write Item.
   - Input hash: A Ruby hash representing the key(s) or item(s) for the selected operation. Structure varies by operation — see Operating Instructions.
   - Table name: The name of the DynamoDB table on which the operation is to be performed.

5.1.3 Example Configuration

This is an example configuration for the AmazonDynamoDBOperations plugin retrieving a single item from a DynamoDB table.

Parameter	Value
Name	Get Customer Record
Comments	Retrieve a single customer item from DynamoDB by partition key
Aws key	
Aws secret	(stored securely)
Aws region	us-east-1
Verify SSL?	Checked
Proxy URL	(blank)
Role arn	(blank - not assuming a role)
Operations	Get Item
Table name	Customers
Input hash	{ ”CustomerId” => ”C-10293” }

5.1.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved AmazonDynamoDBOperations configured instance.
Comments	Comments about this saved AmazonDynamoDBOperations configured instance.
Aws key	The access key credentials provided by Amazon.
Aws secret	The secret key credentials provided by Amazon.
Aws region	The region under which DynamoDB is hosted.
Verify SSL?	Check this box to use SSL.
Proxy URL	The URL of the proxy server through which AWS will be accessed. Leave blank if not using a proxy.
External id	External id when assuming a role (optional).
Policy	IAM policy in JSON format when assuming a role (optional).
Role arn	Amazon Resource Name (ARN) of the role. Required to assume a role.
Token duration	Duration of the generated token when assuming a role, in seconds. Default is 3600s (1 hour).
Operations	The operation to be performed on DynamoDB. Supported: Get Item, Put Item, Delete Item, Batch Get Item, Batch Write Item.
Input hash	A Ruby hash representing the key(s) or item(s) for the selected operation. Structure varies by operation — see Operating Instructions.

Parameter	Description
Table name	The name of the DynamoDB table on which the operation is to be performed.

5.1.5 Inputs Description

Usually this plugin does not have runtime specified parameters. Inputs such as the table name, operation type, or input hash may be specified at runtime when working with dynamic data.

Parameter	Description
Aws_key	The AWS access key used for authentication.
Aws_secret	The AWS secret key used for authentication.
Aws_region	The AWS region where DynamoDB is hosted.
Operations	The DynamoDB operation to perform. Can be specified at runtime if not set in the template.
Table_name	The name of the DynamoDB table. Can be specified at runtime for dynamic table targeting.
Input_hash	The operation-specific hash structure. Can be specified at runtime when item keys or data are generated dynamically.
Role_arn	ARN of the IAM role to assume. Required only when using role assumption.
External_id	External ID for role assumption, if defined in the role trust relationship.
Policy	IAM policy in JSON format scoping permissions for the assumed role credentials.
Token_duration	Duration in seconds for temporary credentials generated during role assumption.

5.1.6 Outputs Description

Outputs vary by operation. Single-item operations return Item; batch operations return Items. Batch Write also returns Item Collection Metrics. Temporary credentials are returned when role assumption is used.

Parameter	Description
Item	A map of attribute names to AttributeValue objects for the retrieved or affected item. Returned by Get Item, Put Item, and Delete Item. For Put Item and Delete Item, reflects the item state *before* the operation. Returns an empty hash if no item was found or no change occurred.
Items	A map of table names to lists of items retrieved. Returned by Batch Get Item and Batch Write Item.
Item_Collection_Metrics	A map of table names to item collection size estimates (in GB) for affected items. Returned by Batch Write Item. May be empty if too few items were affected.
Temporary_access_key	Temporary access key generated if assume role is called.
Temporary_secret	Temporary secret generated if assume role is called.
Temporary_token	Temporary token generated if assume role is called.

5.1.7 Supported Actions

None

5.1.8 Dependencies

Requires valid AWS credentials (access key and secret) with appropriate DynamoDB permissions. Role assumption requires AWS STS access.

5.1.9 Operating Instructions

This action performs DynamoDB item-level operations using the configured AWS credentials and operation-specific input hash. The structure of the input hash varies by operation.

Operation Types and Input Hash Formats

- Get Item: Retrieves a single item by primary key. Input hash format: `{ "AttributeName" => "value" }`. Provide the partition key for simple primary keys, or both partition and sort keys for composite primary keys.
- Put Item: Writes a new item or overwrites an existing one. Input hash format: `{ "AttributeName" => "value" }`. All primary key attributes are required; additional attributes are optional. Returns the prior item state only if an existing item was overwritten and changed.
- Delete Item: Removes an item by primary key. Input hash format: `{ "AttributeName" => "value" }`. Provide all primary key attributes. Returns the item state as it was before deletion.
- Batch Get Item: Retrieves up to 100 items across one or more tables. Input hash format: `{ "TableName" => { keys: [{ "AttributeName" => "value" }], consistent_read: true } }`. Set `consistent_read` to `true` for strongly consistent reads or `false` for eventually consistent reads.
- Batch Write Item: Adds or removes items across one or more tables. Cannot be used to update existing items. Input hash format: `{ "TableName" => [{ put_request: { item: { "AttributeName" => "value" } } }, { delete_request: { key: { "AttributeName" => "value" } } }] }`. Use `put_request` to add items and `delete_request` to remove items.

Assume Role

- The assume role operation is only triggered when a Role ARN is specified.
- The IAM user must be listed as a principal in the role trust relationship with the `sts:AssumeRole` action allowed.
- Identify the IAM user with its access key, secret, and region.
- Specify the External ID if one is defined in the role trust relationship.
- Specify the IAM policy in JSON format without backslashes. Example: `{ "Version": "2012-10-17", "Statement": [{ "Sid": "Stmt1", "Effect": "Allow", "Action": "dynamodb:*", "Resource": "*" }] }`
- Specify the Role ARN. Example: `arn:aws:iam::921583127875:role/db_access`
- Temporary credentials (access key, secret, token) are generated, used for the operation, and returned as outputs.
- Token duration must be greater than 900 seconds (15 minutes) and no more than the role's Maximum Session Duration (up to 12 hours) as configured in the AWS Console.

Best Practices

- Use IAM roles with least-privilege policies scoped to only the DynamoDB actions and tables required.
- Store AWS credentials securely using Orchestrator's credential management rather than plaintext configuration.
- Use composite primary keys (partition + sort key) where possible to improve query precision.
- For Batch Get Item, keep requests under 100 items to stay within DynamoDB limits.
- Use Batch Write Item for bulk inserts or deletes rather than looping single-item operations for better performance.
- Note that Put Item and Delete Item return the pre-operation item state — use this for audit logging or rollback logic.
- Use consistent_read: true in Batch Get Item when your workflow requires the latest committed data.
- Implement error handling workflows to manage unprocessed items that may be returned in batch operations.
- Keep XSD schemas version controlled and synchronized with DynamoDB table schemas and access patterns.
- Use role assumption for cross-account DynamoDB access or when elevated permissions are required for specific operations.

Troubleshooting

- Authentication failure: Verify AWS Key and Secret are correct and the IAM user has DynamoDB permissions.
- Access denied: Ensure the IAM policy grants the required DynamoDB action (e.g., `dynamodb:GetItem`, `dynamodb:PutItem`).
- Table not found: Verify the table name is correct and exists in the specified AWS region.
- Invalid input hash: Check that the hash structure matches the expected format for the selected operation. Refer to the operation-specific formats in Operating Instructions.
- Empty Item output: For Get Item, the item may not exist. For Put Item, an empty result indicates a new item was written (not an overwrite). For Delete Item, confirm the key was correct.
- Batch Get Item limit exceeded: Ensure no more than 100 items are requested in a single Batch Get Item call.
- Batch Write Item update attempt: Batch Write Item does not support item updates — use Put Item for overwriting with new attribute values.
- Role assumption failure: Verify the IAM user is a principal in the role trust policy and `sts:AssumeRole` is permitted.
- Token duration error: Ensure duration is between 900 seconds and the role's configured Maximum Session Duration.
- Proxy connection issues: Verify the proxy URL is reachable from the Orchestrator server and correctly formatted.

Important Notes

- Batch Get Item cannot retrieve more than 100 items per request.
- Batch Write Item supports only `put_request` and `delete_request` — it cannot update existing item attributes.
- For Put Item and Delete Item, the Item output reflects the state *before* the operation was executed.
- Put Item returns an empty hash if the item is new or unchanged; it only returns data when an existing item is overwritten with different values.
- All primary key attributes must be fully specified in the input hash. For composite primary keys, both partition key and sort key are required.
- Role assumption is only triggered when a Role ARN is provided.
- Temporary credentials generated during role assumption are returned as outputs and can be used downstream.
- Token duration must be between 900 and the role's Maximum Session Duration (max 12 hours).
- All operations are synchronous — the workflow waits for the DynamoDB operation to complete before proceeding.

5.2 Amazon Glacier Operation



Figure 5.2: Amazon Glacier Operation icon

This action plug-in provides the ability to integrate with Amazon Glacier Service.

Version	Comment
0.7.1	Enhanced action template UI
0.7.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.6.0	Add assume role
0.5.1	Fix for secret masking
0.5.0	Add support for http proxy
0.4.2	Avoid autofill of key and secret
0.4.1	Masking of AWS Secret
0.4.0	Provide an option to upload a file instead of the payload
0.3.0	fix for ssl issue
0.2.0	change Archive id type
0.1.0	Adding delete Archive operation
0.0.1	Initial release.

5.2.1 How It Works

- Integrates with Amazon Glacier to archive content to a specified vault.
- Accepts archive content as either a direct string body or a file path reference.
- Supports specifying a vault name and account ID for precise archive targeting.
- Accepts an optional archive description and checksum for metadata and integrity verification.
- Optionally supports IAM role assumption via AWS STS for temporary credential generation.
- Returns the Archive ID and Location of the successfully stored archive.
- Returns the full Amazon response and any error messages for downstream handling.
- Supports proxy configuration for environments requiring indirect AWS access.

5.2.2 Configuration Instructions

1. Create a new action step using the AmazonGlacierOperation plugin.

2. Fill out the following fields:
    - Name: The name used to identify a saved AmazonGlacierOperation configured instance.
    - Comments: Comments about this saved AmazonGlacierOperation configured instance.
    - Amazon access Key: The access key credentials provided by Amazon.
    - Amazon secret key: The secret key credentials provided by Amazon.
    - Region: The region under which Glacier is hosted.
    - Proxy url: The URL of the proxy server through which AWS will be accessed. Leave blank if not using a proxy.
    - External id: External id when assuming a role (optional).
    - Policy: IAM policy in JSON format when assuming a role (optional).
    - Role arn: Amazon Resource Name (ARN) of the role. Required to assume a role.
    - Token duration: Duration of the generated token when assuming a role, in seconds. Default is 3600s (1 hour).
    - Vault name: Name of the vault where the content is to be archived.
    - Account: Account ID of the vault owner. Use '-' to default to the credential account ID.
    - Upload payload?: Specify whether the archive content is provided as a string body or a file path.
    - Upload file path: File path of the content to be archived. Used when 'Upload payload?' is unchecked.
    - Body: Content as a string to be archived. Used when 'Upload payload?' is checked.
    - Archive description: Optional description associated with the archive.
    - Checksum: Checksum of the message body for integrity verification.

5.2.3 Example Configuration

This is an example configuration for the AmazonGlacierOperation plugin archiving a report file to an Amazon Glacier vault.

Parameter	Value
Name	Archive Monthly Report
Comments	Archive end-of-month report file to Glacier vault
Amazon access Key	
Amazon secret key	(stored securely)
Region	us-east-1
Proxy url	(blank)
Role arn	(blank - not assuming a role)
Vault name	monthly-reports
Account	-
Upload payload?	File path
Upload file path	/data/reports/report_2025_01.pdf
Archive description	Monthly report for January 2025
Checksum	(blank - auto-calculated)

5.2.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved AmazonGlacierOperation configured instance.
Comments	Comments about this saved AmazonGlacierOperation configured instance.
Amazon access Key	The access key credentials provided by Amazon.
Amazon secret key	The secret key credentials provided by Amazon.
Region	The region under which Glacier is hosted.
Proxy url	The URL of the proxy server through which AWS will be accessed. Leave blank if not using a proxy.
External id	External id when assuming a role (optional).
Policy	IAM policy in JSON format when assuming a role (optional).
Role arn	Amazon Resource Name (ARN) of the role. Required to assume a role.
Token duration	Duration of the generated token when assuming a role, in seconds. Default is 3600s (1 hour).
Vault name	Name of the vault where the content is to be archived.
Account	Account ID of the vault owner. Use '-' to default to the credential account ID.

Parameter	Description
Upload payload?	Specify whether the archive content is provided as a string body or a file path.
Upload file path	File path of the content to be archived. Used when 'Upload payload?' is unchecked.
Body	Content as a string to be archived. Used when 'Upload payload?' is checked.
Archive description	Optional description associated with the archive.
Checksum	Checksum of the message body for integrity verification.

5.2.5 Inputs Description

Usually this plugin does not have runtime specified parameters. Inputs such as the archive body or file path may be specified at runtime when content is generated dynamically within the workflow.

Parameter	Description
Aws_key	The Amazon access key ID used for authentication.
Aws_secret	The Amazon secret access key used for authentication.
Aws_session_token	The Amazon session token (optional). Used with temporary credentials.
Aws_region	The Amazon region where Glacier is hosted.
Proxy_url	The HTTPS proxy URL. Leave blank if not using a proxy.
External_id	External ID for role assumption, if defined in the role trust relationship.
Policy	IAM policy in JSON format scoping permissions for the assumed role credentials.
Role_arn	ARN of the IAM role to assume. Required only when using role assumption.
Token_duration	Duration in seconds for temporary credentials generated during role assumption.

5.2.6 Outputs Description

The output includes the Archive ID and Location for the stored archive, the full Amazon response, and any error messages. Temporary credentials are returned when role assumption is used.

Parameter	Description
Response	Full response received from Amazon Glacier after the archive operation.
Error_Message	Error message returned from Amazon, if any. Present on failure.
Archive_Id	Unique identifier assigned to the archived content by Amazon Glacier.
Location	URI location of the archived file within the Glacier vault.
Temporary_access_key	Temporary access key generated if assume role is called.
Temporary_secret	Temporary secret generated if assume role is called.
Temporary_token	Temporary token generated if assume role is called.

5.2.7 Supported Actions

None

5.2.8 Dependencies

Requires valid AWS credentials (access key and secret) with appropriate Amazon Glacier permissions. Role assumption requires AWS STS access.

5.2.9 Operating Instructions

This action archives content to an Amazon Glacier vault using the configured AWS credentials. Content can be provided as a string body or referenced via file path.

Upload Payload Options

- String Body: Check 'Upload payload?' and provide the archive content directly in the Body field. Suitable for small text-based content generated at runtime.
- File Path: Uncheck 'Upload payload?' and provide the path in Upload file path. Suitable for binary files, large content, or pre-existing files on the Orchestrator server.

Assume Role

- The assume role operation is only triggered when a Role ARN is specified.
- The IAM user must be listed as a principal in the role trust relationship with the `sts:AssumeRole` action allowed.
- Identify the IAM user with its access key, secret, and region.
- Specify the External ID if one is defined in the role trust relationship.
- Specify the IAM policy in JSON format without backslashes. Example: `{ "Version": "2012-10-17", "Statement": [{ "Sid": "Stmt1", "Effect": "Allow", "Action": "glacier:*", "Resource": "*" }] }`
- Specify the Role ARN. Example: `arn:aws:iam::921583127875:role/gl_access`
- Temporary credentials (access key, secret, token) are generated, used for the operation, and returned as outputs.
- Token duration must be greater than 900 seconds (15 minutes) and no more than the role's Maximum Session Duration (up to 12 hours) as configured in the AWS Console.

Best Practices

- Use IAM roles with least-privilege policies scoped to only the Glacier actions and vaults required.
- Store AWS credentials securely using Orchestrator's credential management rather than plaintext configuration.
- Use file path instead of body for large or binary content to avoid memory constraints.
- Provide a meaningful Archive description to make retrieval and auditing easier — Glacier does not support renaming archives after upload.
- Store the returned Archive ID and Location outputs in your workflow or a database — these are required to retrieve or delete the archive later.
- Use checksums to verify data integrity, especially for large or critical archives.
- Use '-' as the Account ID unless specifically targeting a vault owned by a different account.
- Implement error handling workflows that check the Error_Message output to manage upload failures gracefully.
- Use role assumption for cross-account Glacier access or when elevated permissions are required.

Troubleshooting

- Authentication failure: Verify the Amazon access key and secret are correct and the IAM user has Glacier permissions.
- Access denied: Ensure the IAM policy grants the required Glacier action (e.g., `glacier:UploadArchive`).
- Vault not found: Verify the vault name is correct and exists in the specified AWS region.
- File not found: Verify the upload file path is correct and accessible from the Orchestrator server.
- Checksum mismatch: Verify the provided checksum matches the content being uploaded, or leave blank to allow Amazon to calculate it automatically.
- Empty Archive ID: Check the Error_Message output for details. A missing Archive ID typically indicates the upload did not complete successfully.
- Role assumption failure: Verify the IAM user is a principal in the role trust policy and `sts:AssumeRole` is permitted.
- Token duration error: Ensure duration is between 900 seconds and the role's configured Maximum Session Duration.
- Proxy connection issues: Verify the proxy URL is reachable from the Orchestrator server and correctly formatted.

Important Notes

- Archive content can be provided as a string body or file path, but not both simultaneously.
- When using file paths, ensure that the aspweb user is able to access the file on the Orchestrator server.
- The Archive ID and Location returned by this operation are the only references to the archive — store them securely as they are required for future retrieval or deletion.
- Using '-' as the Account ID instructs Amazon to use the default account ID associated with the provided credentials.
- Amazon Glacier is designed for infrequent access — retrieval jobs can take hours; this plugin covers upload only.
- Role assumption is only triggered when a Role ARN is provided.
- Temporary credentials generated during role assumption are returned as outputs and can be used downstream.
- Token duration must be between 900 and the role's Maximum Session Duration (max 12 hours).

- The operation is synchronous — the workflow waits for the Glacier upload to complete before proceeding.

5.3 Amazon Media Convert Operation



Figure 5.3: Amazon Media Convert Operation icon

This action plug-in provides the ability to integrate with Amazon Media Convert Service.

Version	Comment
0.2.2	Enhanced action template UI
0.2.1	Cleanup code & remove beta flag
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.3	Add session token
0.1.2	Add icon
0.1.1	Fix for secret masking
0.1.0	Add http proxy support
0.0.2	Avoid autofill of key and secret
0.0.1	Initial release.

5.3.1 How It Works

- Integrates with AWS Elemental MediaConvert, a file-based video transcoding service with broadcast-grade features for creating video-on-demand (VOD) content at scale.
- Supports Start Job operations to submit a new transcoding job using a JSON settings file, and Cancel Job operations to stop an in-progress job by ID.
- Accepts a JSON settings file path defining the full video and audio transcoding configuration for the job.
- Supports accelerated transcoding mode to significantly speed up jobs with long or visually complex content.
- Uses a configurable IAM role to define the permissions under which the MediaConvert job is created and executed.
- Accepts a unique API endpoint associated with the AWS account, as required by the MediaConvert service.
- Supports an optional polling frequency to control how often the plugin checks job status during execution.
- Supports proxy configuration for environments requiring indirect AWS access.

5.3.2 Configuration Instructions

1. Create a new action step using the AmazonMediaConvertOperation plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved AmazonMediaConvertOperation configured instance.
   - Comments: Comments about this saved AmazonMediaConvertOperation configured instance.
   - Aws key: The access key credentials provided by Amazon.
   - Aws secret: The secret key credentials provided by Amazon.
   - Aws region: The AWS region under which the MediaConvert service is hosted.
   - API Endpoint: The unique MediaConvert endpoint associated with the current AWS account.
   - Proxy URL: The URL of the proxy server through which AWS will be accessed. Leave blank if not using a proxy.
   - Operation: The operation to perform. Supported values: Start Job, Cancel Job.
   - Role: The IAM role used for creating the MediaConvert job.

- Mode: Whether accelerated transcoding is enabled or disabled. Accelerated mode can significantly speed up jobs with long or visually complex content.
- Settings Path: Path to the JSON file containing the video settings for the job. Must be correctly formatted for the MediaConvert API.
- Polling Frequency: How often (in seconds) the plugin polls the MediaConvert API for job status updates (optional).
- Job ID: The ID of the job to be cancelled. Required for Cancel Job operations.

5.3.3 Example Configuration

This is an example configuration for the AmazonMediaConvertOperation plugin submitting a new MediaConvert transcoding job.

Parameter	Value
Name	Transcode VOD Asset
Comments	Start a MediaConvert transcoding job for a new VOD asset
Aws key	
Aws secret	(stored securely)
Aws region	us-east-1
API Endpoint	https://abc123.mediaconvert.us-east-1.amazonaws.com
Proxy URL	(blank)
Operation	Start Job
Role	arn:aws:iam::123456789012:role/MediaConvertRole
Mode	Enabled
Settings Path	/data/jobs/transcode_settings.json
Polling Frequency	30
Job ID	(blank - not cancelling)

5.3.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved AmazonMediaConvertOperation configured instance.
Comments	Comments about this saved AmazonMediaConvertOperation configured instance.
Aws key	The access key credentials provided by Amazon.
Aws secret	The secret key credentials provided by Amazon.
Aws region	The AWS region under which the MediaConvert service is hosted.
API Endpoint	The unique MediaConvert endpoint associated with the current AWS account.
Proxy URL	The URL of the proxy server through which AWS will be accessed. Leave blank if not using a proxy.
Operation	The operation to perform. Supported values: Start Job, Cancel Job.
Role	The IAM role used for creating the MediaConvert job.
Mode	Whether accelerated transcoding is enabled or disabled. Accelerated mode can significantly speed up jobs with long or visually complex content.
Settings Path	Path to the JSON file containing the video settings for the job. Must be correctly formatted for the MediaConvert API.
Polling Frequency	How often (in seconds) the plugin polls the MediaConvert API for job status updates (optional).
Job ID	The ID of the job to be cancelled. Required for Cancel Job operations.

5.3.5 Inputs Description

Runtime inputs override any values set in the action template. Credentials and settings path can be specified at runtime for dynamic workflows.

Parameter	Description
Aws_key	The Amazon access key ID used for authentication.
Aws_secret	The Amazon secret access key used for authentication.

Parameter	Description
Aws_region	The AWS region where the MediaConvert service is hosted.
Role	The IAM role ARN used for creating the MediaConvert job.
Settings_path	Path to the JSON file containing the video settings for the job. Must be accessible from the Orchestrator server.
Proxy_url	The HTTPS proxy URL. Leave blank if not using a proxy (optional).
Aws_session_token	The Amazon session token. Used with temporary credentials (optional).

5.3.6 Outputs Description

The output contains error details if the operation fails. Check this output in error handling workflows to diagnose and respond to failures.

Parameter	Description
Error_Message	Contains error details if the operation fails. Empty on success.

5.3.7 Supported Actions

None

5.3.8 Dependencies

Requires valid AWS credentials with appropriate MediaConvert and IAM permissions. The IAM role must have a trust relationship permitting MediaConvert to assume it. The Settings Path JSON file must be accessible from the Orchestrator server.

5.3.9 Operating Instructions

This action integrates with AWS Elemental MediaConvert to submit or cancel video transcoding jobs. MediaConvert is a broadcast-grade, file-based transcoding service for creating VOD content at scale, combining advanced video and audio processing capabilities.

Supported Operations

- Start Job: Submits a new transcoding job to MediaConvert using the IAM role, settings JSON file, and accelerated transcoding mode specified in the configuration. The plugin polls for job completion at the configured polling frequency.
- Cancel Job: Cancels an in-progress MediaConvert job using the provided Job ID. Use the Job ID returned by a previous Start Job operation or retrieved from the AWS Console.

Accelerated Transcoding Mode

- When Mode is set to Enabled, MediaConvert uses accelerated transcoding to process jobs faster by parallelising the workload.
- Accelerated transcoding is most effective for long-duration or visually complex content such as high-bitrate, high-resolution video.
- Accelerated transcoding incurs additional AWS costs, evaluate the trade-off between speed and cost for your use case.
- Not all output configurations support accelerated transcoding. Refer to the AWS MediaConvert documentation for supported configurations.

Job Settings JSON File

- The Settings Path must point to a valid JSON file formatted according to the MediaConvert Job Settings API schema.
- The JSON file defines all aspects of the transcoding job including input files, output groups, video codec settings, audio settings, and captions.
- The settings file must be accessible from the Orchestrator server at the time the workflow step executes.
- Use the AWS MediaConvert Console to build and export a valid settings JSON file before referencing it in the plugin.

Finding Your API Endpoint

- AWS Elemental MediaConvert uses a per-account unique API endpoint rather than a shared regional endpoint.
- To find your endpoint, run the following AWS CLI command: `aws mediaconvert describe-endpoints --region <your-region>`
- Alternatively, the endpoint is visible in the AWS MediaConvert Console under Account > API endpoint.
- The endpoint format is: `https://<account-id>.mediaconvert.<region>.amazonaws.com`

Best Practices

- Store AWS credentials securely using Orchestrator's credential management rather than plaintext configuration.
- Use an IAM role with least-privilege permissions scoped to only the MediaConvert actions and S3 buckets required for the job.
- Validate the settings JSON file against the MediaConvert API schema before using it in a workflow, invalid settings will cause the job to fail at submission.
- Use the AWS MediaConvert Console to test job settings manually before automating them via this plugin.
- Store the Job ID from a Start Job step as a workflow variable so it can be used in a subsequent Cancel Job step if needed.
- Set an appropriate Polling Frequency - too frequent polling may increase API call costs; too infrequent may delay downstream workflow steps.
- Implement error handling workflows that check the Error_Message output to manage job submission failures or cancellation errors.
- Use accelerated transcoding selectively for long or complex content where the speed benefit justifies the additional cost.

Troubleshooting

- Authentication failure: Verify the AWS key and secret are correct and the IAM user or role has the required MediaConvert permissions (e.g., `mediaconvert:CreateJob`, `mediaconvert:CancelJob`).
- Invalid API endpoint: Confirm the API endpoint is the account-specific MediaConvert endpoint, not a generic regional endpoint. Retrieve it via the AWS CLI or Console.
- Settings file not found: Verify the Settings Path is correct and the file is accessible from the Orchestrator server at runtime.
- Invalid settings JSON: Validate the JSON file structure against the MediaConvert Job Settings API schema. Use the AWS Console to test the settings manually.
- Job not found for cancellation: Confirm the Job ID is correct and the job is still in a cancellable state (SUBMITTED or PROGRESSING).
- Accelerated transcoding failure: Verify the output configuration supports accelerated transcoding. Not all codecs and output groups are compatible, refer to the AWS documentation.
- IAM role error: Ensure the specified IAM role exists, has a trust relationship allowing MediaConvert to assume it, and has the required permissions for S3 input/output buckets.
- Proxy connection issues: Verify the proxy URL is reachable from the Orchestrator server and correctly formatted.

Important Notes

- The API Endpoint is account-specific and must be retrieved from the AWS MediaConvert Console or via the AWS CLI before configuring this plugin.
- The Settings Path must reference a valid, correctly formatted MediaConvert Job Settings JSON file accessible from the Orchestrator server.
- The Job ID field is only required for Cancel Job operations.
- Runtime inputs override values set in the template when provided.
- Accelerated transcoding incurs additional AWS costs and is not supported by all output configurations.
- The IAM role must have a trust relationship permitting MediaConvert to assume it, in addition to the necessary S3 and MediaConvert action permissions.
- The operation is synchronous from the workflow perspective, the plugin polls for job completion before the workflow step returns, using the configured polling frequency.

5.4 Amazon S3 Operation

This action plug-in provides the ability to execute file and bucket operations (such as upload, download, copy, list, and delete) on Amazon S3 storage.



Figure 5.4: Amazon S3 Operation icon

Version	Comment
1.3.1	Enhanced action template UI
1.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.2.1	Add support for copy_object acl
1.2.0	Add support for session token
1.1.1	File is no longer corrupted on multipart copy
1.1.0	Adds list objects operation
1.0.3	Fixes shared AWS config issue
1.0.2	Restoring the masking of AWS Secret with Autocomplete off
1.0.1	Avoid autocomplete in AWS credentials
1.0.0	Adds storage_class for upload and copy object options
0.9.0	Adds force_path_style option
0.8.0	Allows custom endpoint to be set
0.7.9	Patch for multi-threaded uploads.
0.7.8	Fixes syntax compatibility with Ruby 1.8.7.
0.7.7	Reconcile encoding on Delete.
0.7.6	Masking of AWS Secret
0.7.5	Restore Tier error checking.
0.7.4	Increased internal timeout.
0.7.3	Fix UI issue.
0.7.2	Fix failure on download issue.
0.7.1	Add accelerate functionality to all applicable operations.
0.7.0	Allow URI encoding to be turned on or off.
0.6.6	Adding multipart copy functionality.
0.6.5	URI encode the bucket and object to handle special characters.
0.6.4	Cancel option added for upload
0.6.3	Adds created bucket as an output
0.6.2	Fixed corruption of uploaded files larger than 52428800000 (due to limitation of number of possible chunks to 10000)
0.6.1	Fixed issue with presigned url
0.6.0	Adds support for pre-signed urls
0.5.0	Adds support for proxy
0.4.1	Stream progress report during multipart upload
0.4.0	add s3 copy operation
0.3.0	adding multipart upload feature
0.2.2	Change from upload_file to put_object for upload option
0.2.0	fix for ssl issue
0.1.0	Added Restore Object Operation
0.0.3	Added Head Object Operation
0.0.2	Added Download Operation
0.0.1	Initial release.

5.4.1 How It Works

- Performs various operations on Amazon S3 including bucket management, object operations, and file transfers.

- Supports List Bucket, Create Bucket, Upload to Bucket, Copy Object, Download Object, Delete Object, Head Object, Restore Object, Delete Bucket, and Generate Pre-Signed URL operations.
- Provides AWS credential management with support for role assumption and temporary credentials.
- Offers multipart upload capabilities for large files with configurable threading and chunk sizes.
- Includes SSL verification, transfer acceleration, proxy support, and custom endpoint configuration.

5.4.2 Configuration Instructions

1. Add an AmazonS3Operation step to your workflow.
2. Configure the essential AWS connection fields:
   - Operations: Select the S3 operation to perform (e.g., Upload to a Bucket, List Bucket).
   - AWS key: Your Amazon access key credentials.
   - AWS secret: Your Amazon secret key credentials.
   - AWS region: The AWS region where S3 is hosted.
   - Bucket: Name of the S3 bucket for the operation.
3. Set operation-specific parameters:
   - Object key: S3 object name/path (for object operations).
   - Local file path: Local file path (for upload/download operations).
   - Verify SSL?: Enable SSL verification (recommended).

5.4.3 Example Configuration

Parameter	Example Value
Operations	Upload to a Bucket
AWS key	
AWS secret	**********
AWS region	us-east-1
Verify SSL?	checked
Bucket	my-s3-bucket
Object key	uploads/myfile.txt
Local file path	/tmp/myfile.txt
Storage class	STANDARD
Allow multipart upload	checked
Overwrite existing file	checked

5.4.4 Saved Parameters Description

Parameter	Description
Name	Display name for this S3 operation step.
Comments	Notes or description for this step.
Operations	The S3 operation to perform: List Bucket, Create Bucket, Upload to a Bucket, Copy Object, Download Object, Delete Object, Head Object, Restore Object, Delete Bucket, Generate Pre-Signed URL.
AWS key	Amazon access key credentials for authentication.
AWS secret	Amazon secret key credentials for authentication.
AWS region	AWS region where S3 is hosted. For copy operations, use the destination bucket's region.
Verify SSL?	Enable SSL verification for secure connections.
Use transfer acceleration?	Enable Amazon S3 Transfer Acceleration (must be enabled for the bucket first).
Proxy URL	URL of proxy server for AWS access (optional).
Endpoint URL	Custom endpoint URL for S3 access (optional).
Force path style?	Keep bucket name in request URI instead of moving to host subdomain.
External id	External ID when assuming a role (optional).

Parameter	Description
Policy	IAM policy in JSON format when assuming a role (optional).
Role ARN	Amazon Resource Name (ARN) of the role to assume (required for role assumption).
Token duration	Duration of generated token when assuming role, in seconds (default: 3600s).
Encode S3 Paths?	Apply URL encoding to S3 path parameters (Bucket, Object Key, Source S3 File Path). Enabled by default.
Bucket	S3 bucket name. For copy operations, this is the destination bucket.
Object key	S3 object name including folder path (e.g., foo/bar.txt). Do not include bucket name.
Source Region	AWS region of source bucket for copy operations using multipart (required for multipart copy).
Source S3 File Path	Source file path for copy operations, including bucket name and object key (e.g., my_bucket/foo/bar.txt).
Local file path	Local path of file to upload or download location.
Storage class	S3 storage type for the object (available for upload and copy operations).
Allow multipart upload	Enable multipart upload/copy for large files (automatically used for files >5GB in copy operations).
Multipart thread limit	Number of threads for simultaneous multipart transfers (default: 5).
Multipart upload threshold	Minimum file size in bytes for multipart upload (5MB-5GB range, default: 62914560 bytes).
Multipart chunk size	Size in bytes of chunks for multipart transfer (default: 5242880 bytes).
If enabled, overwrites the existing object in the bucket with the same key (upload operation only).	
Acl	Access control list permissions for the object (upload operation only, optional).
Restore days	Lifetime of active copy in days for restore object operation.
Restore tier	Glacier retrieval tier for restore operation: Standard, Bulk, or Expedited.
Restore version	Object version for restore operation (optional).
Request payer	Confirmation string that requester accepts charges for restore operation (optional).
Prefix	Limit response to keys beginning with specified prefix (list objects operation only).
Delimiter	Character used to group keys (list objects operation only).
Start after	Key to start listing from (list objects operation only).

5.4.5 Inputs Description

Any values provided in this input list will override those defined in the action template.

- AWS_key: Amazon access key ID.
- AWS_secret: Amazon secret access key.
- AWS_session_token: Amazon session token (optional).
- AWS_region: Amazon AWS region.
- Proxy_url: HTTPS proxy URL.
- Endpoint_url: Custom endpoint URL for S3 access.
- External_id: External ID when assuming a role (optional).
- Policy: IAM policy in JSON format when assuming a role (optional).
- Role_ARN: Role ARN to assume.
- Token_duration: Generated token duration in seconds when assuming a role.
- Storage_class: Storage type for the object (for upload and copy operations).
- Acl: Access control list for the object in upload operation (optional).

5.4.6 Outputs Description

- Location: Location URL of the S3 bucket or key.
- Error Message: Error details if operation fails.
- bucket list: List of buckets present in the specified region.
- etag_value: ETag value from S3 for upload or copy operations.
- version_id: Version ID for upload operations.
- objects list: List of up to 1000 objects in the bucket.
- objects count: Number of returned objects (maximum 1000).
- Temporary_access_key: Temporary access key generated if assume role is used.
- Temporary_secret: Temporary secret generated if assume role is used.
- Temporary_token: Temporary token generated if assume role is used.

5.4.7 Supported Actions

- List Bucket
- Create Bucket
- Upload to a Bucket
- Copy Object
- Download Object
- Delete Object
- Head Object
- Restore Object
- Delete Bucket
- Generate Pre-Signed URL

5.4.8 Dependencies

None

5.4.9 Operating Instructions

- This plugin performs various operations on Amazon S3 with a default timeout of 600 seconds.
- Large File Handling: For large files using multipart uploads/copies, operations may take significant time. For files of several gigabytes, set timeout to infinite (value: 0) to prevent timeouts.
- Multipart operations: Set upload threshold and chunk size in bytes. For copy operations, multipart is automatically used for files larger than 5GB.
- Special characters in paths may cause issues due to AWS API restrictions, regardless of the Encode S3 Paths setting.
- Access Control Lists (ACL): Can be set for upload or copy operations. ACL permissions are defined in AWS documentation under 'Canned ACL'. Invalid ACL values are ignored.

5.4.10 Assume Role Configuration

Assume role: Allows an IAM user to assume a role for temporary elevated permissions.

- Prerequisites: The assume role operation is called only if a role ARN is specified. The IAM user must be allowed as a principal in the role trust relationship (sts:AssumeRole action).
- Setup: First identify the IAM user with access key, secret, and region.
- External ID: Specify external ID if defined in the role trust relationship.
- IAM Policy: Specify IAM policy in JSON format without backslashes. Example: `{ "Version": "2012-10-17", "Statement": [{ "Sid": "Stmt1", "Effect": "Allow", "Action": "s3:ListAllMyBuckets", "Resource": "*" }] }`
- Role ARN: Specify the role ARN (example: `arn:aws:iam::921583127875:role/s3_access`).
- Token Duration: Must be at least 900 seconds (15 minutes) and no more than the role's maximum session duration (up to 12 hours).
- Output: Temporary credentials (access key, secret, token) are generated, used, and returned when a role is specified.



Figure 5.5: Amazon SimpleDB Operation icon

5.5 Amazon SimpleDB Operation

This action plug-in provides the ability to execute SimpleDB operations

Version	Comment
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.0	Add assume role
0.2.1	Fix for secret masking
0.2.0	Add http proxy support
0.1.4	Avoid autofill of key and secret
0.1.3	Enable variables in inputs
0.1.2	Masking of AWS Secret as a password field
0.1.1	UI fix
0.1.0	Allows for SSL to not be used
0.0.1	Initial release.

5.5.1 Saved Parameters Description

- Name: The name used to identify a saved AmazonSimpledbOperation configured instance.
- Comments: Some comments about this saved AmazonSimpledbOperation configured instance.
- Amazon access Key: The access key credentials provided by Amazon
- Amazon secret key: The secret key credentials provided by Amazon
- Region: The region under which SimpleDB is hosted
- Verify SSL?: Check this box to use SSL.
- Proxy URL: The URL of the proxy server through which AWS will be accessed. Leave this field blank if you do not wish to use a proxy server.
- External id: External id when assuming a role (optional).
- Policy: Identity and Access Management (IAM) policy in JSON format when assuming a role (optional).
- Role arn: Amazon Resource Name (ARN) of the role. Required to assume a role.
- Token duration: Duration of the generated token. When assuming a role. In seconds. Default is 3600s (1 hour).
- Operation: The name of the operation you want to perform on the database
- Domain name: The name of the table you want to create or perform functions on
- Item name: Name of a particular item (Entry/Row) in the domain (table)
- Select Expression: Query that you want to run on the database
- Attributes put list: Array of attributes to be inserted. Each element of the array should be of the following syntax {(required)name: , (required)value: , (optional)replace: false} where required indicates the parameter is required and optional indicates that the field is optional
- Attributes list: Array of attributes to be deleted. Each element of the array should be of the following syntax {(required)name: , (optional)alternate_name_encoding: , (required)value: , (optional)alternate_value_encoding: } where required indicates the parameter is required and optional indicates that the field is optional
- Items put list: Array of Items to be inserted as a batch. Each element of the array should be of the following syntax {(required)name: , (required) attributes: [{(required)name: , (required)value: , (optional)replace: false },] } where required indicates the parameter is required and optional indicates that the field is optional
- Items delete list: Array of Items to be deleted as a batch. Each element of the array should be of the following syntax {(required)name: , (required) attributes: [{(required)name: , (optional)alternate_name_encoding: , (required)value: ,

(optional)alternate_value_encoding: },] } where required indicates the parameter is required and optional indicates that the field is optional

- Attributes names list: It is the optional parameter while trying to fetch attributes from the table which gives you an option to specify the list of attributes only whose values you want to fetch
- Next token: It is an optional parameter when using the Select operation where it is a string informing Amazon SimpleDB where to start the next list of ItemNames. It is an optional parameter when using list domains operation as well there it is a string informing Amazon SimpleDB where to start the next list of domain names.
- Strongly_Consistent read: An optional Boolean parameter Determines whether or not strong consistency should be enforced when data is read from SimpleDB. If true, any data previously written to SimpleDB will be returned. Otherwise, results will be consistent eventually, and the client may not see data that was written immediately before your read.
- Max number of domains: An optional parameter when executing the list number of domains operation using which you can specify the maximum number of domain names you want returned
- Proxy URL: The URL of the proxy server through which AWS will be accessed. Leave this field blank if you do not wish to use a proxy server.

5.5.2 Inputs description

The list of inputs depends on the configuration of the AmazonSimpledbOperation action template.

Changes in this list of inputs will override any values set in the action template.

Aws_key: The Amazon access key id.

Aws_secret: The Amazon secret access key.

Aws_session_token: The Amazon session token (optional).

Aws_region: The Amazon region.

Proxy_url: The HTTPS Proxy URL.

External_id: External id when assuming a role (optional).

Policy: IAM policy in JSON format when assuming a role (optional).

Role_arn: Role arn to assume.

Token_duration: Generated token duration in seconds, when assuming a role.

5.5.3 Outputs description

- *Error Message*: If there is an error, this variable shows the errors
- *Result*: Generally displays the Hash result for your query
- *Domain List*: Displays the Array of Domain names
- *Temporary_access_key*: Temporary access key generated if assume role is called
- *Temporary_secret*: Temporary secret generated if assume role is called
- *Temporary_token*: Temporary token generated if assume role is called

5.5.4 Supported Actions

None

5.5.5 Dependencies

None

5.5.6 Operating Instructions

This action plugin is used to perform actions on Amazon SimpleDB

For certain operations like put attributes, batch put attributes, delete attributes, batch delete attributes the plugin requires the input to be of a specific format (See Parameter Description for the expected/required input format for each input).

Assume role: for an IAM user to assume a role.

Pre-requisites:

The assume role operation is called only if there is a specified role arn.

The IAM user must be allowed as a principal in the role trust relationship (sts:AssumeRole action).

Instructions:

The IAM user must first be identified with its access key, secret and region.

Specify the external id if any defined in the role trust relationship.

Specify the IAM policy in JSON format. Don't put backslashes. For example, use something like { ”Version”: ”2012-10-17”, ”Statement”: [{ ”Sid”: ”Stmt1”, ”Effect”: ”Allow”, ”Action”: ”sdb:*”, ”Resource”: ”*” }] }. This policy will be effective for all operations realized with the generated temporary credentials.

Specify the role arn. For example arn:aws:iam::921583127875:role/sdb_access

Temporary credentials (access key, secret, token) are generated, used and returned if a role is specified.

Specify the token duration in seconds. The value must be more than 900 seconds (15 minutes) and be less than the role Maximum session duration (check in the AWS Console). The Maximum session duration can be set to 12 hours maximum.

5.6 Amazon SNS Operation



Figure 5.6: Amazon SNS Operation icon

This action plug-in provides the ability to use Amazon SNS service.

Version	Comment
0.4.1	Enhanced action template UI
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.0	Add assume role
0.2.1	Fix for secret masking
0.2.0	Add http proxy support
0.1.2	Avoid autofill of key and secret
0.1.1	Masking of AWS Secret
0.1.0	Allows for SSL to not be used
0.0.2	Display error message on failure.
0.0.1	Initial release.

5.6.1 How It Works

- Performs Amazon SNS (Simple Notification Service) operations including Publish, Subscribe, and Unsubscribe.
- Supports publishing messages to a topic ARN, target ARN, or phone number.
- Supports subscribing endpoints to SNS topics across multiple protocols (HTTP, HTTPS, email, SMS, SQS, Lambda, etc.).
- Supports unsubscribing from SNS topics using a subscription ARN.
- Optionally supports IAM role assumption via AWS STS for temporary credential generation.
- Accepts AWS access key, secret, and region for authentication.
- Supports proxy configuration for environments requiring indirect AWS access.
- Returns operation-specific outputs such as Message ID, Subscription ARN, or success status.

5.6.2 Configuration Instructions

1. Create a new action step using the AmazonSNSOperation plugin.
2. Fill out the following fields:
    - Name: The name used to identify a saved AmazonSNSOperation configured instance.
    - Comments: Comments about this saved AmazonSNSOperation configured instance.
    - AWS Key: The access key credentials provided by Amazon.
    - AWS Secret: The secret key credentials provided by Amazon.
    - AWS Region: The region under which SNS is hosted.
    - Verify SSL?: Check this box to use SSL.
    - Proxy URL: The URL of the proxy server through which AWS will be accessed. Leave blank if not using a proxy.
    - External id: External id when assuming a role (optional).
    - Policy: IAM policy in JSON format when assuming a role (optional).
    - Role arn: Amazon Resource Name (ARN) of the role. Required to assume a role.
    - Token duration: Duration of the generated token when assuming a role, in seconds. Default is 3600s (1 hour).
    - API Operation: Operation to be performed. Supported values: Publish, Subscribe, Unsubscribe.

5.6.3 Example Configuration

This is an example configuration for the AmazonSNSOperation plugin publishing a message to an SNS topic.

Parameter	Value
Name	Publish Order Notification
Comments	Publish order confirmation message to SNS topic
AWS Key	
AWS Secret	(stored securely)
AWS Region	us-east-1
Verify SSL?	Checked
Proxy URL	(blank)
Role arn	(blank - not assuming a role)
API Operation	Publish
TopicARN	arn:aws:sns:us-east-1:123456789012:OrderNotifications
Message	Order #889923 has been confirmed.
Subject	Order Confirmation

5.6.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved AmazonSNSOperation configured instance.
Comments	Comments about this saved AmazonSNSOperation configured instance.
AWS Key	The access key credentials provided by Amazon.
AWS Secret	The secret key credentials provided by Amazon.
AWS Region	The region under which SNS is hosted.
Verify SSL?	Check this box to use SSL.
Proxy URL	The URL of the proxy server through which AWS will be accessed. Leave blank if not using a proxy.
External id	External id when assuming a role (optional).
Policy	IAM policy in JSON format when assuming a role (optional).
Role arn	Amazon Resource Name (ARN) of the role. Required to assume a role.
Token duration	Duration of the generated token when assuming a role, in seconds. Default is 3600s (1 hour).
API Operation	Operation to be performed. Supported values: Publish, Subscribe, Unsubscribe.

5.6.5 Inputs Description

Usually this plugin does not have runtime specified parameters. Inputs such as ARNs or message content may be specified at runtime depending on workflow design.

Parameter	Description
AWS_Key	The AWS access key used for authentication.
AWS_Secret	The AWS secret key used for authentication.
AWS_Region	The AWS region where the SNS service is hosted.
Role_arn	ARN of the IAM role to assume. Required only when using role assumption.
External_id	External ID for role assumption, if defined in the role trust relationship.
Policy	IAM policy in JSON format scoping permissions for assumed role credentials.
Token_duration	Duration in seconds for the temporary credentials generated during role assumption.
TopicARN	ARN of the SNS topic. Used in Publish and Subscribe operations.
TargetARN	Target ARN for Publish. Used only if TopicARN is not specified.
Phone_Number	E.164-formatted phone number for SMS publish. Used only if both TopicARN and TargetARN are not specified.
Message	The message content to publish to the topic or endpoint.
Subject	Optional subject line for email endpoint deliveries.
Is_the_structure_of_the_message_JSON	Set message structure to JSON to send different messages per protocol.
Protocol	The protocol to use for the subscription (e.g., http, https, email, sms, sqs, lambda).
Endpoint	The endpoint that receives notifications. Format depends on the chosen protocol.
SubscriptionARN	ARN of the subscription to remove. Used in Unsubscribe operations.

5.6.6 Outputs Description

Outputs vary by API operation. Temporary credentials are returned when role assumption is used.

Parameter	Description
Temporary_access_key	Temporary access key generated if assume role is called.
Temporary_secret	Temporary secret generated if assume role is called.
Temporary_token	Temporary token generated if assume role is called.
Message_ID	Unique identifier assigned to the published message. Returned on Publish.
SubscriptionARN	The Amazon Resource Name of the subscription. Returned on Subscribe.
Successful?	True if the response returned a 200-level HTTP status code, false otherwise. Returned on Unsubscribe.

5.6.7 Supported Actions

None

5.6.8 Dependencies

Requires valid AWS credentials (access key and secret) with appropriate SNS permissions. Role assumption requires AWS STS access.

5.6.9 Operating Instructions

This action performs Amazon SNS operations — Publish, Subscribe, or Unsubscribe — using the configured AWS credentials and operation-specific parameters.

Publish Operation

- A message can be published to only one of the following: a topic ARN, a target ARN, or a phone number.
- TopicARN takes priority if multiple destination parameters are provided.
- TargetARN is used only if TopicARN is not specified.

- Phone Number is used only if both TopicARN and TargetARN are not specified. Must be in E.164 format.
- When Is the structure of the message JSON? is checked, the message must be valid JSON with at least a `"default"` top-level key.

Subscribe Operation

- Provide the TopicARN of the topic to subscribe to.
- Select the appropriate Protocol for the endpoint type.
- Endpoint formatting depends on the chosen protocol:
  - `http` — URL beginning with `http://`
  - `https` — URL beginning with `https://`
  - `email` / `email-json` — An email address
  - `sms` — Phone number of an SMS-enabled device
  - `sqs` — The ARN of an Amazon SQS queue
  - `application` — The EndpointARN of a mobile app and device
  - `lambda` — The ARN of an AWS Lambda function

Unsubscribe Operation

- Provide the SubscriptionARN of the subscription to remove.
- Returns Successful? as true if the HTTP response is a 200-level status code.

Assume Role

- The assume role operation is only triggered when a Role ARN is specified.
- The IAM user must be listed as a principal in the role trust relationship with the `sts:AssumeRole` action allowed.
- Identify the IAM user with its access key, secret, and region.
- Specify the External ID if one is defined in the role trust relationship.
- Specify the IAM policy in JSON format without backslashes. Example: `{ "Version": "2012-10-17", "Statement": [{ "Sid": "Stmt1", "Effect": "Allow", "Action": "SNS:*", "Resource": "*" }] }`
- Specify the Role ARN. Example: `arn:aws:iam::921583127875:role/sns_access`
- Temporary credentials (access key, secret, token) are generated, used for the operation, and returned as outputs.
- Token duration must be greater than 900 seconds (15 minutes) and no more than the role's Maximum Session Duration (up to 12 hours) as configured in the AWS Console.

Best Practices

- Use IAM roles with least-privilege policies scoped to only the SNS actions required.
- Store AWS credentials securely using Orchestrator's credential management rather than plaintext configuration.
- Use role assumption for cross-account or elevated-privilege SNS operations.
- Validate TopicARN and SubscriptionARN values before use to avoid runtime errors.
- Use JSON message structure when targeting multi-protocol topics that require tailored payloads per endpoint type.
- Use E.164 format for all phone numbers to ensure correct SMS delivery.
- Implement error handling workflows for publish failures or subscription confirmation timeouts.
- Monitor Message ID outputs to correlate published messages with SNS delivery logs.
- Set appropriate token durations when assuming roles — avoid excessively long durations for security.

Troubleshooting

- Authentication failure: Verify AWS Key and Secret are correct and the IAM user has SNS permissions.
- Access denied: Check that the IAM policy grants the required SNS action (e.g., `sns:Publish`, `sns:Subscribe`).
- Invalid TopicARN: Ensure the ARN format is correct and the topic exists in the specified region.
- Role assumption failure: Verify the IAM user is a principal in the role trust policy and that `sts:AssumeRole` is permitted.
- Token duration error: Ensure duration is between 900 seconds and the role's configured Maximum Session Duration.
- SMS not delivered: Verify the phone number is in E.164 format and SMS is supported in the target region.
- JSON message invalid: Ensure the message is valid JSON and includes a `"default"` top-level key when JSON structure is enabled.
- Proxy connection issues: Verify the proxy URL is reachable from the Orchestrator server and correctly formatted.
- SSL errors: Ensure Verify SSL is checked and the Orchestrator server trusts the AWS certificate chain.

Important Notes

- For Publish, only one destination is used: TopicARN takes priority over TargetARN, which takes priority over Phone Number.
- JSON message structure requires a valid JSON object with at least a ”default” key.
- Role assumption is only triggered when a Role ARN is provided.
- Temporary credentials generated during role assumption are returned as outputs and can be used downstream.
- Token duration must be between 900 and the role's Maximum Session Duration (max 12 hours).
- The Unsubscribe operation returns Successful? = true for any 200-level HTTP response.
- Endpoint format for Subscribe varies by protocol — refer to AWS documentation for protocol-specific requirements.
- All operations are synchronous — the workflow waits for the SNS operation to complete before proceeding.

5.7 Amazon SQS Operation



Figure 5.7: Amazon SQS Operation icon

This action plug-in provides the ability to perform an action onto an Amazon SQS queue.

Version	Comment
0.5.1	Enhanced action template UI
0.5.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.4.0	Add assume role
0.3.2	Adding support for session token
0.3.1	Fix for secret masking
0.3.0	Add http proxy support
0.2.1	Avoid autofill of key and secret
0.2.0	Adding message group id for FIFO queues
0.1.1	Masking of AWS Secret
0.1.0	Allows for SSL to not be used
0.0.1	Initial release.

5.7.1 How It Works

- This action plug-in performs operations on an Amazon Simple Queue Service (SQS) queue.
- The operations that can be performed on the Amazon SQS queue are as follows:
  - Queues_list: List all Amazon SQS queues.
  - Message_delete: Delete a message from an Amazon SQS queue.
  - Message_get: Get a message from an Amazon SQS queue.
  - Message_send: Send a message to an Amazon SQS queue.

5.7.2 Configuration Instructions

1. Create a new action step using the AmazonSqsOperation plug-in.
2. This action performs operations on an Amazon SQS queue using the configured AWS credentials.
3. Fill out the following configuration fields:
   - Name: The name used to identify a saved AmazonSqsOperation configuration.
   - Comments: Optional comments about this AmazonSqsOperation configuration.
   - AWS key: The AWS access key ID.

- AWS secret: The AWS secret access key.
- AWS region: The AWS region in which the Amazon SQS queue is hosted.
- Verify SSL?: When selected, SSL certificates are verified for AWS connections.
- External ID: Optional external ID used when assuming an IAM role.
- Policy: Optional Identity and Access Management (IAM) policy, in JSON format, used when assuming a role.
- Role ARN: The Amazon Resource Name (ARN) of the role to assume. Required when assuming a role.
- Token duration: The duration, in seconds, of the temporary security token generated when assuming a role. The default is 3600 seconds (1 hour).
- Operation: The operation to perform on the Amazon SQS queue.
  - Queues_list: List all Amazon SQS queues.
  - Message_delete: Delete a message from an Amazon SQS queue.
  - Message_get: Get a message from an Amazon SQS queue.
  - Message_send: Send a message to an Amazon SQS queue.
- Queue URL: The URL of the Amazon SQS queue.
- Message group ID: Required when sending messages to a FIFO queue. Identifies the message group.
- Message deduplication ID: Required when sending messages to a FIFO queue. Used for message deduplication.
- Message body: The message payload to send to the queue.
- Receipt handle: The unique identifier associated with a received message. Required when deleting a message.
- Max number of messages: The maximum number of messages to retrieve when receiving messages. The default is 1.
- Proxy URL: Optional proxy server URL used to access AWS. Leave this field blank if a proxy is not required.

5.7.3 Example Configuration

This is a minimal setup configuration for the AmazonSqsOperation plug-in.

Parameter	Value
Name	Amazon SQS Operation
Comments	Send a message to an Amazon SQS queue
AWS key	
AWS secret	**********
AWS region	us-east-1
Verify SSL?	Selected
Operation	SendMessage
Queue URL	https://sqs.us-east-1.amazonaws.com/123456789012/my-queue
Message body	Example message payload
Message group ID	order-processing
Message deduplication ID	msg-0001
Proxy URL	

5.7.4 Saved Parameters Description

5.7.5 Inputs Description

Required inputs depend on how the AmazonSqsOperation template is configured. If dynamic attributes are used, each attribute becomes a runtime input.

Parameter	Description
Aws_key *	The AWS access key ID.
Aws_secret *	The AWS secret access key.
Aws_session_token	The AWS session token (optional).
Aws_region	The AWS region.
External_id	The external ID used when assuming an IAM role (optional).
Policy	An IAM policy in JSON format used when assuming a role (optional).

Parameter	Description
Role_arn	The Amazon Resource Name (ARN) of the role to assume.
Token_duration	The duration, in seconds, of the temporary credentials when assuming a role.
Queue_url	The Amazon SQS queue URL.
Receipt_handle	The receipt handle associated with a message.
Message_group_id	The message group ID for FIFO queues.
Message_deduplication_id	The message deduplication ID for FIFO queues.
Message_body	The message body.
Max_number_of_messages	The maximum number of messages to retrieve (optional).

5.7.6 Outputs Description

Parameter	Description
Response	The raw AWS SDK response object returned by the Amazon SQS API.
Messages	The Messages collection from the AWS SDK response.
Messages_body	An array of message bodies extracted from the Messages collection.
Error_message	The error message returned by the AWS SDK if the operation fails.
Temporary_access_key	The temporary AccessKeyId returned by AWS STS when assuming a role.
Temporary_secret	The temporary SecretAccessKey returned by AWS STS when assuming a role.
Temporary_token	The temporary SessionToken returned by AWS STS when assuming a role.

5.7.7 Supported Actions

None

5.7.8 Dependencies

None

5.7.9 Operating Instructions

1. This action plug-in performs operations on an Amazon SQS queue. Specify the AWS access key, secret access key, and AWS region.
2. The Queue URL is required for all SQS operations except ListQueues. For receive operations, the maximum number of messages defaults to 1 and can be set between 1 and 10.
3. When sending messages to a FIFO queue, a Message Group ID is mandatory. A Message Deduplication ID is also required unless ContentBasedDeduplication is enabled on the queue.
4. Do not specify Message Group ID or Message Deduplication ID when sending messages to a standard queue, as this results in an error.
5. This plug-in supports assuming an IAM role. The AssumeRole operation is performed only when a Role ARN is specified.
6. Prerequisites: The IAM user associated with the provided AWS credentials must be trusted by the role and permitted to perform the sts:AssumeRole action.
7. To assume a role, identify the IAM user using AWS credentials and region, and optionally specify an External ID if required by the role trust policy.

8. Optionally specify an IAM policy in JSON format to restrict permissions for the assumed role. Do not escape JSON characters. This policy applies to all operations performed using the temporary credentials.
9. Specify the Role ARN to assume (for example: `arn:aws:iam::921583127875:role/sqs_access`). Temporary credentials are generated, used for the operation, and returned as outputs.
10. Specify the token duration in seconds. The value must be greater than 900 seconds (15 minutes) and less than or equal to the role's maximum session duration (up to 12 hours).

5.8 Amazon SWF Operation



Figure 5.8: Amazon SWF Operation icon

This action plug-in provides the ability to execute Amazon Simple Workflow API calls

Version	Comment
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.0	Add assume role
0.2.1	Fix for secret masking
0.2.0	Add http proxy support
0.1.2	Avoid autofill of key and secret
0.1.1	Masking of AWS Secret
0.1.0	Allows for SSL to not be used
0.0.1	Initial release.

5.8.1 Saved Parameters Description

- Name: The name used to identify a saved AmazonSwfOperation configured instance.
- Comments: Some comments about this saved AmazonSwfOperation configured instance.
- Amazon access Key: The access key credentials provided by amazon
- Amazon secret key: The secret key credentials provided by amazon
- Region: The region under which SWF is hosted
- Verify SSL?: Check this box to use SSL.
- Proxy URL: The URL of the proxy server through which AWS will be accessed. Leave this field blank if you do not wish to use a proxy server.
- External id: External id when assuming a role (optional).
- Policy: Identity and Access Management (IAM) policy in JSON format when assuming a role (optional).
- Role arn: Amazon Resource Name (ARN) of the role. Required to assume a role.
- Token duration: Duration of the generated token. When assuming a role. In seconds. Default is 3600s (1 hour).
- Domain: The name of the domain in which the workflow execution is created.
- Workflow id: Unique identifier given to the workflow execution
- Workflow type name: Name of the workflow to be executed
- Workflow type version: Version of the workflow to be executed
- Task list: The task list to use for the decision tasks generated for this workflow execution
- Task priority: Priority for the task. If not set, amazon workflow priority would be used
- Swf inputs: Inputs required to the workflow
- Execution start to close timeout: Timeout value in seconds
- Tags: Tags used to identify execution

- Task start to close timeout: Timeout value in seconds
- Child policy: Defaulted to TERMINATE. Other options are REQUEST_CANCEL, ABANDON

5.8.2 Inputs description

The list of inputs depends on the configuration of the AmazonSwfOperation action template.

Changes in this list of inputs will override any values set in the action template.

- *Aws_key*: The Amazon access key id.
- *Aws_secret*: The Amazon secret access key.
- *Aws_session_token*: The Amazon session token (optional).
- *Aws_region*: The Amazon region.
- *Proxy_url*: The HTTPS Proxy URL.
- *External_id*: External id when assuming a role (optional).
- *Policy*: IAM policy in JSON format when assuming a role (optional).
- *Role_arn*: Role arn to assume.
- *Token_duration*: Generated token duration in seconds, when assuming a role.

5.8.3 Outputs description

- *Response*: Response received from Amazon
- *Error_Message*: Error message if any received from Amazon
- *Run_Id*: Run id of the instance executed
- *Temporary_access_key*: Temporary access key generated if assume role is called
- *Temporary_secret*: Temporary secret generated if assume role is called
- *Temporary_token*: Temporary token generated if assume role is called

5.8.4 Supported Actions

None

5.8.5 Dependencies

None

5.8.6 Operating Instructions

This action plugin is used to perform queuing action to Amazon SQS.

Refer to the link below to know more about the SWF api calls and its parameter types

http://docs.aws.amazon.com/sdkforruby/api/Aws/SWF/Client.html#start_workflow_execution-instance_method

Assume role: for an IAM user to assume a role.

Pre-requisites:

The assume role operation is called only if there is a specified role arn.

The IAM user must be allowed as a principal in the role trust relationship (sts:AssumeRole action).

Instructions:

The IAM user must first be identified with its access key, secret and region.

Specify the external id if any defined in the role trust relationship.

Specify the IAM policy in JSON format. Don't put backslashes. For example, use something like { ”Version”: ”2012-10-17”, ”Statement”: [{ ”Sid”: ”Stmt1”, ”Effect”: ”Allow”, ”Action”: ”swf:*”, ”Resource”: ”*” }] }. This policy will be effective for all operations realized with the generated temporary credentials.

Specify the role arn. For example arn:aws:iam::921583127875:role/swf_access

Temporary credentials (access key, secret, token) are generated, used and returned if a role is specified.

Specify the token duration in seconds. The value must be more than 900 seconds (15 minutes) and be less than the role Maximum session duration (check in the AWS Console). The Maximum session duration can be set to 12 hours maximum.

5.9 Amazon Transcribe Operation



Figure 5.9: Amazon Transcribe Operation icon

This action plug-in provides the ability to execute Amazon Transcribe API calls

Version	Comment
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.0	Add assume role
0.1.1	Fix for secret masking
0.1.0	Add http proxy support
0.0.3	Avoid autofill of key and secret
0.0.2	Adding icon
0.0.1	Initial release.

5.9.1 Saved Parameters Description

- Name: The name used to identify a saved AmazonTranscribeOperation configured instance.
- Comments: Some comments about this saved AmazonTranscribeOperation configured instance.
- Amazon access Key: The access key id credential provided by Amazon.
- Amazon secret key: The secret access key credential provided by Amazon.
- Region: The region under which Transcribe is hosted (e.g. us-east-2 for US East Ohio).
- Proxy URL: The URL of the proxy server through which AWS will be accessed. Leave this field blank if you do not wish to use a proxy server.
- External id: External id when assuming a role (optional).
- Policy: Identity and Access Management (IAM) policy in JSON format when assuming a role (optional).
- Role arn: Amazon Resource Name (ARN) of the role. Required to assume a role.
- Token duration: Duration of the generated token. When assuming a role. In seconds. Default is 3600s (1 hour).
- Vocabulary name: The name of the vocabulary. The name must be unique within an AWS account. The name is case-sensitive.
- Language Code: The language code of the vocabulary entries.
- Polling frequency: By default, the queue is probed every 5 seconds for new messages. That polling frequency can be changed to any number of seconds.
- Transcription Job Name: The name of the transcription job.
- Media Sample Rate: The sample rate, in Hertz, of the audio track in the input media file.
- Media Format: The format of the input media file.
- Vocabulary File URI: The S3 location of the text file that contains the definition of the custom vocabulary. The URI must be in the same region as the API endpoint that you are calling. The general form is https://s3-.amazonaws.com///
- Output Bucket Name: The location where the transcription is stored.
- Show Speaker Labels?: Check in order to show speaker labels.
- Channel Identification?: Check in order to enable channel identification.
- Max Speaker Labels: Maximum number of speaker labels.
- Proxy URL: The URL of the proxy server through which AWS will be accessed. Leave this field blank if you do not wish to use a proxy server.

5.9.2 Inputs description

The list of inputs depends on the configuration of the AmazonTranscribeOperation action template.

- *Aws_key*: The Amazon access key id.
- *Aws_secret*: The Amazon secret access key.
- *Aws_region*: The Amazon region.
- *Proxy_url*: The HTTPS Proxy URL.
- *External_id*: External id when assuming a role (optional).
- *Policy*: IAM policy in JSON format when assuming a role (optional).
- *Role_arn*: Role arn to assume.
- *Token_duration*: Generated token duration in seconds, when assuming a role.
- *transcription_job_name*: The name of the transcription job.
- *lang_code*: The language code of the vocabulary entries.
- *file_uri*: The S3 location of the text file that contains the definition of the custom vocabulary. The URI must be in the same region as the API endpoint that you are calling. The general form is https://s3-.amazonaws.com///
- *media_format*: The format of the input media file.
- *media_sample_rate*: The sample rate, in Hertz, of the audio track in the input media file. Optional.
- *output_bucket_name*: The location where the transcription is stored. Optional.
- *show_speaker_labels*: Check in order to show speaker labels. Optional.
- *vocab_name*: The name of the vocabulary. The name must be unique within an AWS account. The name is case-sensitive. Optional.
- *channel_id*: Check in order to enable channel identification. Optional.

5.9.3 Outputs description

- *Response*: Response received from Amazon
- *Messages_body*: The array of messages body
- *Temporary_access_key*: Temporary access key generated if assume role is called
- *Temporary_secret*: Temporary secret generated if assume role is called
- *Temporary_token*: Temporary token generated if assume role is called

5.9.4 Supported Actions

None

5.9.5 Operating Instructions

This plugin provides the ability to use Amazon's Transcribe feature. Amazon

Transcribe makes it easy for developers to add speech-to-text capability to their applications.

Specify the AWS access key + secret, the AWS region and the queue URL.

Assume role: for an IAM user to assume a role.

Pre-requisites:

The assume role operation is called only if there is a specified role arn.

The IAM user must be allowed as a principal in the role trust relationship (sts:AssumeRole action).

Instructions:

The IAM user must first be identified with its access key, secret and region.

Specify the external id if any defined in the role trust relationship.

Specify the IAM policy in JSON format. Don't put backslashes. For example, use something like { ”Version”: ”2012-10-17”, ”Statement”: [{ ”Sid”: ”Stmt1”, ”Effect”: ”Allow”, ”Action”: ”transcribe:*”, ”Resource”: ”*” }] }. This policy will be effective for all operations realized with the generated temporary credentials.

Specify the role arn. For example arn:aws:iam::921583127875:role/tr_access

Temporary credentials (access key, secret, token) are generated, used and returned if a role is specified.

Specify the token duration in seconds. The value must be more than 900 seconds (15 minutes) and be less than the role Maximum session duration (check in the AWS Console). The Maximum session duration can be set to 12 hours maximum.

5.10 AMQP message



Figure 5.10: AMQP message icon

This action plug-in provides the ability to insert messages into an AMQP queue.

Version	Comment
0.9.1	Enhanced action template UI
0.9.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.8.3	Fixed problem when specifing optional parameters at runtime
0.8.2	Fixed problem when specifying header options
0.8.1	Fixed problem with routing key and multiple messages
0.8.0	Added configuration option to accept multiple messages as input
0.7.0	Adding option to set passive mode (false by default)
0.6.0	Adding additional message attributes to facilitate in configuring the header
0.5.1	Patches carrot gem
0.5.0	Extends message column
0.4.1	Adding RemoteNode Dependency.
0.4.0	Added support for AMQP versions higher than 0.8.0
0.3.2	Added content to help file about improvements in v0.3.0 and v0.3.1
0.3.1	When sending to an exchange, queue name should be optional
0.3.0	Added support to send to an exchange
0.2.0	Added direct routing option
0.1.7	New icon
0.1.6	Correctly use the vhost specified value
0.1.5	exception handling of RemoteNode.identify()
0.1.4	prevents auto-complete of user
0.1.3	fix - edit.html file has a typo which make username and password fields default to 0
0.1.2	fix dynamic node retrieval
0.1.1	adjustment to configurator, adding help file
0.1.0	adds support for vhost
0.0.2	marks message as persistent
0.0.1	first tracked version

5.10.1 How It Works

- This action sends text-based messages into a specified AMQP queue.
- Requires an AMQP server (e.g., RabbitMQ or Apache Qpid) reachable by Orchestrator on the configured port.
- Supports inserting structured messages (XML, JSON, or custom formats), as long as the receiving client can parse the content.
- Inserts all messages as persistent, ensuring they are stored by the AMQP broker until delivered.

5.10.2 Configuration Instructions

1. Create a new action step using the AmqpMessage plugin.
2. This action sends text-based messages into a specified AMQP queue.
3. Fill out the following fields:
    - Name: The name used to identify a saved AmqpMessage configured instance.
    - AMQP queue name: The name identifying the queue on the AMQP server. If the queue does not exist, it will be created when the first message is inserted.
    - AMQP Exchange name: The name defining the exchange on the AMQP server.
    - AMQP Version > 0.8.0 ?: Checks whether the AMQP server version is greater than 0.8.0.
    - AMQP Exchange type: The type of the exchange on the AMQP server. Defaults to Direct, but you may choose from Direct, Topic, Fanout, or Headers.
    - Queue connection arguments - AMQP server node: If the server where AMQP runs was defined as a remote node, it can be selected here. Alternatively, its address can be provided using the fields below.
    - Queue connection arguments - Queue host: The host name, if a node was not provided.
    - Queue connection arguments - Queue vhost: Specify a virtual host if one is used for that queue on the AMQP server.
    - Queue connection arguments - Queue port: A specific port can be specified. If not specified, the default port 15672 will be used.
    - Queue connection arguments - Queue user: A specific user can be specified if the default guest account is not used.
    - Queue connection arguments - Queue password: The password for the user, if the default is not used.
    - Queue connection arguments - Use SSL?: Indicates whether SSL is to be used to communicate with the AMQP server.
    - Message to queue: The text of the message to insert into the queue. Dynamic attributes can be used. If not provided at design time, the message must be supplied at runtime.
    - Process multiple messages: Allows the action to process multiple messages in the inputs. When enabled, a new runtime input is exposed to map an array of messages. If messages are specified at design time, they must be comma-separated.
    - Routing key: (Optional) Used to support direct message exchange.
    - Hide message?: By default, the inserted message is returned as an output. Enable this option to hide the message from the reporting interface.
    - Configure additional Queue/Exchange and Message Headers?: Enables configuration of additional queue, exchange, and message headers.
    - Durable?: Indicates whether the queue should survive a broker restart.
    - Auto delete?: Indicates whether the exchange should be automatically deleted when it is no longer in use.
    - Exclusive?: Indicates whether this queue is exclusive (usable only by this connection and removed when the connection closes).
    - X-max-priority: Supports specifying the maximum number of priorities.
    - X-message-ttl: Supports per-queue Message Time-to-Live (TTL).
    - X-expires: Specifies how long the queue may remain unused, in milliseconds.
    - Persistent?: Determines whether the message should be persisted to disk.
    - Mandatory?: Indicates whether the message should be returned if it cannot be routed to any queue.
    - Timestamp: A time value represented as an integer.
    - Expiration: The time (in milliseconds) after which the message will be deleted.
    - Message type: The type of event or command represented by the message; can be any string.
    - Reply to Queue: The queue name to which other applications should send the response.
    - Content type: The message content type (e.g., application/json).
    - Content Encoding: The encoding used for the message content (e.g., utf-8).
    - Correlation Id: The identifier linking this message to another, such as the request it is replying to.
    - Priority: The priority level of the message.
    - Message Id: An identifier for the message.
    - User Id: Optional user ID, verified by RabbitMQ against the connection username.
    - App Id: Optional application ID.
    - Header: A Ruby hash of additional attributes required by the application. Nested hashes are supported.
    - Passive Flag: If true, the exchange will be checked for existence, and an error will be raised if it does not exist. False by default.

5.10.3 Example Configuration

This is a minimum setup configuration for the plugin.

Parameter	Value
AMQP queue name	amqp_home
Message to queue	Test message
AMQP Exchange name	amqp_origin
AMQP Version > 0.8.0 ?	Selected
AMQP Exchange type	Direct
Queue connection arguments - AMQP server node	AMQP node

5.10.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved AmqpMessage configured instance.
AMQP queue name	The name identifying the queue on the AMQP server. If the queue does not exist, it will be created when the first message is inserted.
AMQP Exchange name	The name defining the exchange on the AMQP server.
AMQP Version > 0.8.0 ?	Checks whether the AMQP server version is greater than 0.8.0.
AMQP Exchange type	The type of the exchange on the AMQP server. Defaults to Direct, but you may choose from Direct, Topic, Fanout, or Headers.
Queue connection arguments - AMQP server node	If the server where AMQP runs was defined as a remote node, it can be selected here. Alternatively, its address can be provided using the fields below.
Queue connection arguments - Queue host	The host name, if a node was not provided.
Queue connection arguments - Queue vhost	Specify a virtual host if one is used for that queue on the AMQP server.
Queue connection arguments - Queue port	A specific port can be specified. If not specified, the default port 15672 will be used.
Queue connection arguments - Queue user	A specific user can be specified if the default guest account is not used.
Queue connection arguments - Queue password	The password for the user, if the default is not used.
Queue connection arguments - Use SSL?	Indicates whether SSL is to be used to communicate with the AMQP server.
Message to queue	The text of the message to insert into the queue. Dynamic attributes can be used. If not provided at design time, the message must be supplied at runtime.

Parameter	Description
Process multiple messages	Allows the action to process multiple messages in the inputs. When enabled, a new runtime input is exposed to map an array of messages. If messages are specified at design time, they must be comma-separated.
Routing key	(Optional) Used to support direct message exchange.
Hide message?	By default, the inserted message is returned as an output. Enable this option to hide the message from the reporting interface.
Configure additional Queue/Exchange and Message Headers?	Enables configuration of additional queue, exchange, and message headers.
Durable?	Indicates whether the queue should survive a broker restart.
Auto delete?	Indicates whether the exchange should be automatically deleted when it is no longer in use.
Exclusive?	Indicates whether this queue is exclusive (usable only by this connection and removed when the connection closes).
X-max-priority	Supports specifying the maximum number of priorities.
X-message-ttl	Supports per-queue Message Time-to-Live (TTL).
X-expires	Specifies how long the queue may remain unused, in milliseconds.
Persistent?	Determines whether the message should be persisted to disk.
Mandatory?	Indicates whether the message should be returned if it cannot be routed to any queue.
Timestamp	A time value represented as an integer.
Expiration	The time (in milliseconds) after which the message will be deleted.
Message type	The type of event or command represented by the message; can be any string.
Reply to Queue	The queue name to which other applications should send the response.
Content type	The message content type (e.g., application/json).
Content Encoding	The encoding used for the message content (e.g., utf-8).
Correlation Id	The identifier linking this message to another, such as the request it is replying to.
Priority	The priority level of the message.
Message Id	An identifier for the message.
User Id	Optional user ID, verified by RabbitMQ against the connection username.
App Id	Optional application ID.
Header	A Ruby hash of additional attributes required by the application. Nested hashes are supported.
Passive Flag	If true, the exchange will be checked for existence, and an error will be raised if it does not exist. False by default.

5.10.5 Inputs Description

The list of inputs depends on the configuration of the AmqpMessage action template. If dynamic attributes are used in the message definition, each attribute becomes a required input.

Parameter	Description
AMQP_Queue_nc	Mandatory if not provided at design time — the name of the queue into which the message will be inserted.
Message	Mandatory if not provided at design time — the message to insert into the specified queue.
Exchange_name	Optional - if provided, Orchestrator will attempt to send the message to a queue.
Queue_host	Optional - the address of the node where the queue is hosted.
Queue_node	Optional - the node where the queue is hosted.
Queue_password	Optional - the password for the user posting the message, if different from the default guest account.
Queue_port	Optional - the port used to access the queue.
Queue_user	Optional - the username used for posting the message, if different from the default guest account.
Queue_vhost	Optional - the virtual host for the queue, if one is used.

Parameter	Description
Routing_key	Used to support direct message exchange.
additional_queue_message_parameters	Enables configuration of additional queue, exchange, and message headers.
app_id	Optional application ID.
auto_delete	Indicates whether the exchange should be automatically deleted when it is no longer in use.
content_encoding	The encoding used for the message content (e.g., utf-8).
content_type	The message content type (e.g., application/json).
correlation_id	The identifier linking this message to another, such as the request it is replying to.
exclusive	Indicates whether this queue is exclusive (usable only by this connection and removed when the connection closes).
expiration	The time (in milliseconds) after which the message will be deleted.
headers	A Ruby hash of additional attributes required by the application. Nested hashes are supported.
mandatory	Indicates whether the message should be returned if it cannot be routed to any queue.
message_id	An identifier for the message.
priority	The priority level of the message.
reply_to	The queue name to which other applications should send the response.
timestamp	A time value represented as an integer.
type	The type of the exchange on the AMQP server. Defaults to Direct, but you may choose from Direct, Topic, Fanout, or Headers.
user_id	Optional user ID, verified by RabbitMQ against the connection username.
x_expires	Specifies how long the queue may remain unused, in milliseconds.
x_max_priority	Supports specifying the maximum number of priorities.
x_message_ttl	Supports per-queue Message Time-to-Live (TTL).

5.10.6 Outputs Description

Parameter	Description
Inserted_message	(Unless Hide message? is selected) — the actual message inserted into the queue.
Queue_name	The name of the queue into which the message was inserted.

5.10.7 Supported Actions

None

5.10.8 Dependencies

None

5.10.9 Operating Instructions

This plugin allows you to insert text messages into an AMQP queue. To function correctly, it requires an AMQP server—such as RabbitMQ or Apache Qpid—configured on a machine that Orchestrator can reach on the specified port.

Because the messages are text-based, you can insert structured content such as XML, JSON, or any custom format, provided the receiving client knows how to parse the structure from the text. All messages inserted through this plugin are marked as persistent, ensuring they are retained by the AMQP broker until successfully delivered.

5.11 Aspera On Cloud Management

A plugin to perform Aspera On Cloud management operations including shared inbox and user account management.



Figure 5.11: Aspera On Cloud Management icon

Version	Comment
0.3.1	Enhanced action template UI
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.1	Add get user operation
0.2.0	Add supports for http proxy
0.1.1	Use updated AoC API URL
0.1.0	Add description to shared inbox. Added possibility to use shared inbox id for deletion.
0.0.2	add shared inbox invite email for users with send permission
0.0.1	Initial implementation

5.11.1 How It Works

- Connects to Aspera On Cloud (AoC) to manage organizational resources including shared inboxes and user accounts.
- Supports OAuth authentication with client credentials and SSH key-based user authentication.
- Provides workspace management capabilities for organizing users and shared resources.
- Enables shared inbox operations with member management and configurable permissions.
- Returns structured output in both Ruby and JSON formats with HTTP status codes for operation tracking.

5.11.2 Configuration Instructions

1. Add an AsperaOnCloudManagement step to your workflow.
2. Configure the essential AoC connection parameters:
   - Name: Enter a descriptive name for the step.
   - AoC client id: Client ID from Organization → Integrations in AoC GUI.
   - AoC client secret: Client secret for OAuth authentication.
   - AoC organization name: Organization name (e.g., 'sedemo' for sedemo.ibmaspera.com).
   - AoC user name: User email address with organization admin privileges.
3. Set up authentication and operation details:
   - Private key filepath: Path to SSH private key on Orchestrator machine.
   - Operation: Choose from Create Shared Inbox, Delete Shared Inbox, Create User, Delete User, and Get User.
   - Workspace name: Target workspace for the operation.
   - Http proxy: Optional proxy URL for API access (requires Orchestrator 4.0.1+ or capsule).

5.11.3 Example Configuration

Parameter	Example Value
Name	AoC Management Operation
AoC client id	12345678-1234-1234-1234-123456789abc
AoC client secret	********
AoC organization name	sedemo
AoC user name	admin@company.com
Private key filepath	/opt/aspera/keys/aoc_private_key
Workspace name	Marketing Team

Parameter	Example Value
Operation	Create Shared Inbox
Shared inbox name	Project Files
Description	Shared space for project collaboration
User type	user

5.11.4 Saved Parameters Description

Parameter	Description
Name	Display name for this AoC management step.
Comments	Notes or description for this step.
AoC client id	Aspera On Cloud client ID obtained from Organization → Integrations → select an integration in the AoC GUI.
AoC client secret	Aspera On Cloud client secret for OAuth authentication.
AoC organization name	AoC organization name. Extract from login domain (e.g., 'sedemo' from sedemo.ibmaspera.com).
AoC user name	AoC user email address for authentication. Must be organization admin with SSH public key configured in Users → select user → Public key.
Private key filepath	File path to SSH private key on Orchestrator machine, corresponding to the public key configured for the AoC user.
Http proxy	HTTP proxy URL for accessing Aspera On Cloud APIs (requires Orchestrator 4.0.1+ or capsule).
Workspace name	Target workspace name as defined in the AoC GUI.
Operation	Operation to execute: Create Shared Inbox, Delete Shared Inbox, Create User, Delete User, and Get User.
Shared inbox name	Name of shared inbox. For Delete operation, can specify either name or ID.
Member list	Newline-separated list of user email addresses for shared inbox members, or JSON structure with permissions.
Metadata	Shared inbox metadata as JSON structure (optional).
Description	Descriptive text for the shared inbox (optional).
Email address	User email address for user operations.
First name	User's first name for user creation.
Last name	User's last name for user creation.
User type	User type: user, admin, or __RUNTIME__ for runtime specification.

5.11.5 Inputs Description

- Aoc_client_id: Aspera On Cloud client ID.
- Aoc_client_secret: Aspera On Cloud client secret.
- Aoc_organization_name: Aspera On Cloud organization name.
- Aoc_user_name: Aspera On Cloud user name (email address).
- Private_key_filepath: SSH private key file path on Orchestrator machine.
- Workspacename: Target workspace name.
- Shared inbox name: Shared inbox name or ID (for Delete operation, either value is accepted).
- Member list: Shared inbox member list (newline-separated emails or JSON structure).
- Metadata: Shared inbox metadata in JSON format.
- Description: Shared inbox description text.
- Email_address: User email address for user operations.
- First_name: User's first name.
- Last_name: User's last name.
- User type: User type (present as input when __RUNTIME__ is selected).

5.11.6 Outputs Description

- result: Operation output as Ruby data structure.

- result_json: Operation output as JSON-formatted structure.
- HTTP_return_code: HTTP status code returned by the AoC API operation.

5.11.7 Supported Actions

- Create Shared Inbox
- Delete Shared Inbox
- Create User
- Delete User
- Get User

5.11.8 Dependencies

- Orchestrator version 3.x or above required.
- Valid Aspera On Cloud organization with admin privileges.
- SSH key pair configured (public key in AoC user profile, private key on Orchestrator).
- Network access to Aspera On Cloud APIs (with proxy support if needed).
- The aspweb user should have permissions to read the private key file.

5.11.9 Operating Instructions

5.11.10 OAuth Setup Requirements

All OAuth token parameters are required for API authentication. Enter the workspace name for create/delete shared inbox operations.

5.11.11 Shared Inbox Operations

5.11.12 Member List Configuration

The shared inbox member list can be specified in two formats:

Newline-separated email list:

\ john@corp.com\ alex@corp.com\

For runtime input with newlines, use \n format:

\ john@corp.com\nalex@corp.com\

JSON structure with permissions:

\ [\ {\ ”email_address”: ”john@corp.com”,\ ”permissions”: ”send,receive”\ },\ {\ ”email_address”: ”alex@corp.com”, \ ”permissions”: ”send”\ }\ ]\

- Members with newline format are assigned send and receive permissions automatically.
- All members must be registered users in Aspera On Cloud.
- Member list processing stops on first error (e.g., unregistered user).
- Users with send permission receive invitation emails to contribute.
- If shared inbox already exists, operation returns '409 Conflict' error.

5.11.13 Metadata Configuration

JSON metadata structure example:

\ [\ {\ ”default_values”: [”john”],\ ”input_type”: ”single-text”,\ ”name”: ”first name”,\ ”required”: true\ },\ {\ ”default_values”: [],\ ”input_type”: ”single-text”, \ ”name”: ”last name”\ }\ ]\

5.11.14 User Management Operations

- User Creation: New users are assigned default IBMid authentication type (not Google or SAML).
- Limited Users: Created without workspace specification. Send package 'To' field shows only associated shared inbox(es).

- Standard Users: Created with workspace specification, becoming workspace members. Send package 'To' field shows shared inbox(es) and workspace user members.
- Duplicate Handling: If user already exists, operation returns '409 Conflict' error.

5.12 Aspera Shares Management



Figure 5.12: Aspera Shares Management icon

This plugin is used to utilize Aspera Shares command line utilities to perform shares management tasks such as creating, modifying, and administering shared directories on Aspera Shares servers.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.1	Initial implementation

5.12.1 How It Works

- Manages Aspera Shares operations including creating, modifying, and managing shared directories on Aspera Shares servers.
- Supports multiple Aspera Shares versions, including versions below 1.10.1 and versions 1.10.1 or higher, with cross-platform compatibility for Linux and other operating systems.
- Provides comprehensive share management capabilities through various operation types for different administrative tasks.
- Integrates with remote Aspera Shares nodes to perform operations on shared directories and user access management.
- Returns detailed operation results including login information, error messages, and raw response data for troubleshooting.
- Supports runtime parameter configuration for dynamic share creation and management based on workflow requirements.

5.12.2 Configuration Instructions

1. Add an Aspera Shares Management step to your workflow.
2. Configure the Aspera Shares server connection:
   - Aspera shares node: Select the remote node with Aspera Shares installed.
   - OS type: Choose the operating system type (Linux, Windows, etc.).
   - Aspera Shares version: Specify the Aspera Shares version for compatibility. Both versions below 1.10.1 and versions 1.10.1 or greater are supported.
3. Set up the share operation:
   - Operation type: Select the desired operation (e.g., Create Share).
   - Share Name: Enter the name for the share to be managed.
   - Node Name: Specify the target node name for the operation.
   - Directory: Provide the directory path for the share.

5.12.3 Example Configuration

Parameter	Example Value
Aspera shares node	shares
OS type	Linux
Aspera shares version	1.10.1 Or Greater
Operation type	Create Share
Share Name	Deborah
Node Name	node1
Directory	/Test1

5.12.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify this Aspera Shares Management step configuration.
Comments	Optional comments to describe the purpose and configuration details of this template.
Aspera shares node	The remote node where Aspera Shares server is installed and running.
OS type	The operating system type of the Aspera Shares server (Linux, Windows, etc.).
Aspera shares version	The Aspera Shares version for compatibility. Supports multiple versions, including those below 1.10.1 and versions 1.10.1 or higher.
Operation type	The type of shares management operation to perform (Create Share, Delete Share, Modify Share, etc.).
Share Name	The name of the share to be created, modified, or managed.
Node Name	The specific node name where the share operation will be executed.
Directory	The directory path on the server that will be associated with the share.

5.12.5 Inputs Description

Required Runtime Inputs:

- Directory: String value specifying the directory path for the share operation (required).
- Node_name: String value specifying the target node name for the operation (required).
- Share_name: String value specifying the name of the share to be managed (required).

Optional Runtime Inputs:

- operation_type: Override the configured operation type if needed.
- aspera_node: Specify alternate Aspera shares node if not configured in template.
- os_type: Override the operating system type if required.
- shares_version: Specify version requirements if different from template configuration.

5.12.6 Outputs Description

- Error_Message: Any error messages returned from the Aspera Shares operation, empty if operation completed successfully.
- Login_Id: The login identifier or session ID associated with the Aspera Shares operation.
- Raw_Information: Complete raw response data from the Aspera Shares server containing detailed operation results and metadata.

5.12.7 Supported Operations

- Create Share - Create new shared directories with specified access permissions
- Delete Share - Remove existing shares and associated configurations
- Modify Share - Update share properties, permissions, or directory associations
- List Shares - Retrieve information about existing shares on the server
- Add User to Share - Grant user access to specific shares
- Remove User from Share - Revoke user access from shares

5.12.8 Operating Instructions

- Version Compatibility: This plugin supports multiple Aspera Shares versions, including those below 1.10.1 and versions 1.10.1 or higher. Customers upgrading from version 1.8 should save and republish existing workflows with the updated action template to ensure optimal compatibility.
- Share Creation Process: When creating a new share, specify the share name, target node name, and directory path. The plugin will create the share configuration on the specified Aspera Shares server and return operation results.
- Directory Requirements: The specified directory must exist on the target server and be accessible by the Aspera Shares service. Ensure proper permissions are set for the directory before creating the share.
- Node Configuration: The Aspera shares node must be properly configured in Orchestrator with valid connection details and authentication credentials for the Aspera Shares server.
- Operation Results: Monitor the outputs for successful completion. The Error_Message field will be empty for successful operations, while Login_Id and Raw_Information provide detailed operation results.
- Runtime Configuration: All required parameters (Directory, Node_name, Share_name) must be provided either through template configuration or runtime inputs. Template values take precedence over runtime inputs if both are specified.

5.12.9 Dependencies

- Aspera Shares server (any supported version) installed and configured on the target node.
- Valid Aspera Shares node configuration in Orchestrator with proper authentication credentials.
- Network connectivity between Orchestrator and the Aspera Shares server on required ports.
- Proper file system permissions on the target directory for share creation and management.
- Valid user accounts and permissions on the Aspera Shares server for the specified operations.

5.12.10 Troubleshooting

- If the operation fails with version compatibility errors, verify that the Aspera Shares server is running a compatible version and that the action template has been updated and republished if upgrading from an older version.
- For directory access errors, verify that the specified directory exists on the target server and has appropriate permissions for the Aspera Shares service account.
- If node connection fails, check the Aspera shares node configuration in Orchestrator and verify network connectivity and authentication credentials.
- When share creation fails, examine the Raw_Information output for detailed error messages and ensure the share name is unique and follows naming conventions.
- For authentication issues, verify that the user account has appropriate permissions to perform the requested operation on the Aspera Shares server.
- If runtime inputs are not being recognized, ensure they match the exact parameter names (Directory, Node_name, Share_name) and check that required inputs are properly mapped.
- For OS-specific issues, verify that the OS type setting matches the actual operating system of the Aspera Shares server and that platform-specific configurations are correct.

5.12.11 Version Upgrade Notes

- Customers upgrading from Aspera Shares 1.8 to 1.10.1 must update their existing workflows.
- All existing action templates must be saved and republished to ensure compatibility with version 1.10.1.
- Review and test all share management workflows after upgrading to verify continued functionality.
- New features and improvements in version 1.10.1 may require configuration updates for optimal performance.

5.13 Async Remote Execution

This plugin executes commands remotely on target servers via SSH with support for synchronous and asynchronous execution modes. It offers flexible authentication options, custom output processing, and configurable polling for background process monitoring.

Version	Comment
0.6.4	Enhanced action template UI
0.6.3	Updated the label from Keep ongoing to Continue Monitoring
0.6.2	Fixes issues when using tcsh shell

Version	Comment
0.6.1	Handle cases where processed output, polling stop condition or connection options are empty
0.6.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.5.5	Fixes possible issue when remote temporary storage is full
0.5.4	Fixes possible issue when pid is equal to the year
0.5.3	Better debugging with PIDs
0.5.2	Added retry when checking for status
0.5.1	Allow failing on cancel
0.5.0	Add tcsh option
0.4.2	Cancel now SIGINT child processes
0.4.1	fixed remote_node mapped input reading
0.4.0	added SSH key data section for key content
0.3.1	Limit the size of stdout/stderr ”cat” at the end of a command
0.3.0	Valid SSH keys can be saved
0.2.1	Cancel now SIGKILL the script
0.2.0	Add extra connection options
0.1.1	Limited stdout and stderr to 16k to fit the database
0.1.0	Added environment variables
0.0.3	UI enhancements
0.0.2	Fixed ongoing trigger, cancel and stop condition code exec
0.0.1	Initial version

5.13.1 How It Works

- Executes commands remotely on target servers via SSH connections with flexible authentication methods.
- Supports multiple execution modes: synchronous, asynchronous fire-and-forget, and asynchronous with status monitoring.
- Provides configurable polling mechanisms for long-running background processes with custom status checking.
- Enables environment variable injection and custom output processing through Ruby code blocks.
- Offers comprehensive connection options and supports multiple SSH authentication methods including keys and passwords.

5.13.2 Configuration Instructions

1. Add an AsyncRemoteExecution step to your workflow.
2. Configure the remote server connection:
    - Name: Enter a descriptive name for the remote execution step.
    - Command: Enter the command to execute on the remote server.
    - Remote Node: Select from predefined remote nodes or leave empty for manual configuration.
    - Remote Address: IP address or hostname (if not using Remote Node).
    - Username: SSH login username for authentication.
    - Password or SSH key: Choose authentication method and provide credentials.
3. Set up execution parameters:
    - Shell: Specify the shell for command execution (e.g., /bin/bash).
    - Timeout: Maximum execution time in seconds before timeout.
    - Polling Period: Set to 0 for synchronous, -1 for fire-and-forget, or positive value for async monitoring.
    - Environment Variables: Optional list of variables to set as environment variables.

5.13.3 Example Configuration

Parameter	Example Value
Name	Remote Script Execution
Comments	Executes deployment script on remote server
Command	/opt/scripts/deploy.sh
Shell	/bin/bash
Timeout	300
Remote Node	production-server-01

Parameter	Example Value
Username	deploy_user
SSH key	/opt/aspera/keys/deploy_key
Polling Period	0
Environment Variables	DEPLOY_ENV,APP_VERSION

5.13.4 Saved Parameters Description

Parameter	Description
Name	Display name for this remote execution step.
Comments	Notes or description for this step.
Command	Command to execute remotely on the target server.
Shell	Shell environment for remote user (e.g., /bin/bash, /bin/sh).
Timeout	Maximum time in seconds to wait for execution before timing out.
Remote Node	Predefined remote node from Workflows → Remote Nodes section.
Remote Address	IP address or hostname of target server (when not using predefined Remote Node).
SSH Port	SSH port for connection (default: 22).
Username	SSH login username for remote server authentication.
Password	SSH login password for authentication.
SSH key	List of private key file paths separated by | character for key-based authentication.
SSH key data	List of raw private keys in PEM format separated by | character.
SSH key passphrase	Passphrase for private keys that require one.
Environment Variables	List of variables to set as environment variables on remote target.
Continue Monitoring	Enable to make this plugin act as a continuous trigger.
Polling command	Command to execute during each polling period to check execution status.
Polling Period	Interval for executing polling command (0=synchronous, -1=fire-and-forget, >0=async monitoring).
Polling stop condition	Ruby code executed after each poll to determine if polling should stop.
Processed outputs	Custom output variables to be made available in the workflow.
Processed outputs code	Ruby code executed to provide values for custom processed output variables.
Connection Options	Ruby hash containing extra options for Net::SSH.start connection.

5.13.5 Inputs Description

- Command: Remote command to execute (overrides template value if provided).
- Remote_Address: Target server address for SSH connection.
- Username: SSH username for authentication.
- Password: SSH password for authentication.
- Environment_Variables: Variables from Environment Variables list become mandatory mapped inputs.
- Polling_command: Status checking command for asynchronous monitoring.
- Timeout: Execution timeout in seconds.
- SSH_key: SSH private key file path for authentication.

5.13.6 Outputs Description

- exit_code: Exit code of command execution or last poll command execution.
- Std_out: Standard output of command execution or last poll command (limited to last 16MB).
- Std_err: Standard error output of command execution or last poll command.
- PID: Process ID of the executed command (when available).



Figure 5.13: Async Remote Execution icon

5.13.7 Supported Actions

None

5.13.8 Dependencies

- SSH access to target remote servers with appropriate user permissions.
- Network connectivity from Orchestrator to target servers on configured SSH ports.
- The aspweb user must have proper permissions to read SSH private key files.
- For environment variables: SSH daemon must be configured to accept environment variables (AcceptEnv in /etc/ssh/sshd_config).
- Net::SSH Ruby library for SSH connectivity.

5.13.9 Operating Instructions

5.13.10 Operating Modes

5.13.11 Environment Variables

- Environment variables listed in configuration become mandatory mapped inputs.
- SSH daemon configuration (/etc/ssh/sshd_config) must allow environment variables for security.
- Add variable names to AcceptEnv in sshd_config or use 'AcceptEnv *' to allow all variables.
- Variables are set on the remote target before command execution.

5.13.12 Code Blocks

Two Ruby code blocks are available for customization:

- Processed outputs: Executed on command results and/or last status check results. Can override command outputs.
- Polling stop condition: Executed after each poll to determine if polling should stop (return true to stop).

5.13.13 Available Variables in Code Blocks

- `@stdout`: Standard output from command execution
- `@stderr`: Standard error from command execution
- `@exitcode`: Exit code from command execution
- `@pid`: Process ID of executed command
- `@status`: Step result (STATUS_COMPLETE, STATUS_FAILED, STATUS_ERROR, STATUS_INPROGRESS)
- `@status_details`: Message shown in Orchestrator work order UI

5.13.14 SSH Authentication

5.13.15 SSH Key Passphrase

Net::SSH library supports only one passphrase for all keys requiring passphrases. Avoid mixing keys with and without passphrases.

5.13.16 Connection Options

Additional connection options can be specified as Ruby hash following Net::SSH documentation.

- Disable host key verification: `{ verify_host_key: false }`
- For Net::SSH 6.x with older SSH servers: `{ append_all_supported_algorithms: true }`

5.13.17 Best Practices

- Use longer polling periods for async operations as each poll creates a new SSH connection.
- For fire-and-forget mode, ensure proper cleanup of background processes on target systems.
- Test custom polling commands and stop conditions thoroughly before production use.
- Consider timeout values appropriate for your command execution requirements.
- Use SSH key authentication for better security and automation compatibility.

5.14 Azure Service Bus Operation



Figure 5.14: Azure Service Bus Operation icon

This action plug-in provides the ability to perform an action onto an Azure Service Bus queue.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.1	Initial release.

5.14.1 How It Works

- This action plug-in performs operations on an Azure Service Bus queue or topic.

5.14.2 Configuration Instructions

1. Create a new action step using the AzureServiceBusOperation plug-in.
2. This action performs the configured Azure Service Bus operation and emits the appropriate outputs.
3. Fill out the following configuration fields:
   - Name: The name used to identify a saved AzureServiceBusOperation configuration.
   - Comments: Optional comments about this AzureServiceBusOperation configuration.
   - Azure namespace: The Azure Service Bus namespace.
   - Azure key name: The Azure Service Bus access key name.
   - Azure key: The Azure Service Bus access key.
   - Queue or Topic name: The Azure Service Bus queue or topic name.
   - Subscription_name: For the Message_get_from_topic operation, enter the Azure Service Bus subscription name.
   - Proxy url: The proxy URL used to access Azure Service Bus. Leave this field blank if a proxy is not required.
   - Operation: The operation to execute.
   - Message body: For the Message_send operation, enter the message body.

5.14.3 Example Configuration

This is a minimal configuration example for the AzureServiceBusOperation plug-in.

Parameter	Value
Azure namespace	<sample_namespace>
Azure key name	<sample_key_name>
Azure key	<sample_key>
Queue or Topic name	topic1
Proxy url	https://test-source.azure-bus.com/bus1
Operation	Message_send
Message body	<contents to send>

5.14.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved AzureServiceBusOperation configuration.
Comments	Optional comments about this AzureServiceBusOperation configuration.
Azure namespace	The Azure Service Bus namespace.
Azure key name	The Azure Service Bus access key name.
Azure key	The Azure Service Bus access key.
Queue or Topic name	The Azure Service Bus queue or topic name.
Subscription_name	For the Message_get_from_topic operation, enter the Azure Service Bus subscription name.
Proxy url	The proxy URL used to access Azure Service Bus. Leave this field blank if a proxy is not required.
Operation	The operation to execute.
Message body	For the Message_send operation, enter the message body.

5.14.5 Inputs Description

Required inputs depend on how the AzureServiceBusOperation template is configured. If dynamic attributes are used, each attribute becomes a runtime input.

Parameter	Description
*Azure_namespace **	The Azure Service Bus namespace.
*Azure_key_name **	The Azure Service Bus access key name.
*Azure_key **	The Azure Service Bus access key.
*Queue or Topic name **	The queue or topic name.
Proxy_url	The proxy URL (optional).

5.14.6 Outputs Description

Parameter	Description
Message_body	For Message_get operations, the retrieved message body.
Error_message	The error message returned by Azure Service Bus, if an error occurs.

5.14.7 Supported Actions

None

5.14.8 Dependencies

None

5.14.9 Operating Instructions

1. This action plug-in performs operations on an Azure Service Bus queue or topic.
2. Specify the Azure Service Bus namespace, access key name, and access key.
3. Specify the queue or topic name.
4. Specify the subscription name only when retrieving messages from a topic.
5. Specify a proxy URL (e.g., https://myproxy.com:8443) if direct Internet access is not available. Without a proxy, Orchestrator must be able to reach .servicebus.windows.net on port 443.
6. Enter the operation to execute (Message_get_from_queue, Message_get_from_topic, or Message_send). Only one message can be retrieved using Message_get_from_queue or Message_get_from_topic. Only one message can be sent using Message_send. Enter the message body when using the Message_send operation.
7. Message_get_from_queue and Message_get_from_topic are get-and-delete operations. If no message is available, the step information will report ”no message in the queue” and the Message_body output will be empty.

5.15 CatdvRestRequest



Figure 5.15: CatdvRestRequest icon

This plugin provides capability to perform REST operations on CatDV server.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	Added styling to parameter help text.
0.0.1	Plugin release

5.15.1 How It Works

- Provides RESTful API integration with CatDV servers for performing operations on media content and metadata.
- Supports multiple HTTP operations including GET, PUT, POST, and DELETE for comprehensive media management workflows.
- Handles authentication through session management, automatically obtaining and managing session IDs for secure API access.
- Provides flexible endpoint support for various CatDV resources including catalogs, media, clips, smartfolders, and thumbnails.
- Returns structured JSON responses converted to hash format for easy integration with downstream workflow steps.
- Includes query parameter support for filtering and advanced search operations on CatDV content.

5.15.2 Configuration Instructions

1. Add a CatDV REST Request step to your workflow.
2. Configure CatDV server connection:

- Service url: Enter the CatDV server API URL (e.g., http://serverip:port/api/4/).
- Login: Provide the CatDV username for authentication.
- Basic Auth Password: Enter the password for the CatDV user account.

3. Set up API operation parameters:
   - Endpoint: Select the CatDV resource endpoint (clips, catalogs, media, etc.).
   - Operation: Choose the HTTP method (get, put, post, delete).
   - Selector: Specify resource ID or identifier for targeted operations.
4. Configure optional parameters:
   - Query parameters: Add filtering or search parameters as needed.
   - Post request body: Provide JSON payload for PUT/POST operations.

5.15.3 Example Configuration

Parameter	Example Value
Service url	http://catdvserverone.aspera.com:8080/api/4/
Login	catdv_user
Basic Auth Password	**********
Endpoint	clips
Operation	get
Selector	12
Query parameters	filter=and((clip.userFields[U23])eq(12345))
Post request body	(empty for GET operations)

5.15.4 Saved Parameters Description

Parameter	Description
Comments	Comments associated with this template or configuration for documentation purposes.
Service url	The CatDV server API base URL including protocol, server address, port, and API version path.
Login	Username for CatDV server authentication and API access authorization.
Basic Auth Password	Password for the specified CatDV user account for secure authentication.
Endpoint	The CatDV resource endpoint to target for the API operation (clips, catalogs, media, sourcemedia, thumbnail, smartfolders).
Operation	The HTTP method to perform on the selected endpoint (get, put, post, delete).
Selector	Resource identifier, ID, or path selector for targeting specific CatDV objects or collections.
Post request body	JSON payload content for PUT and POST operations. Required when operation is set to 'put' or 'post'.
Query parameters	Optional URL query parameters for filtering, searching, or modifying API requests (e.g., filter expressions).
Framewise Separation	BIF(s) Framewise Separation parameter for specialized CatDV operations (legacy parameter).

5.15.5 Inputs Description

Mandatory Parameters:

- Service Url: CatDV server API base URL (required for server connection).
- Login: CatDV username for authentication (required for API access).
- Basic Auth Password: CatDV user password (required for authentication).
- Endpoint: Target CatDV resource endpoint (required to specify API target).
- Selector: Resource identifier or selector (required for targeted operations).
- Operation: HTTP method for the API request (required to specify action type).

Conditional Parameters:

- Post request body: JSON payload content (mandatory only when operation is 'put' or 'post').

Optional Parameters:

- Query parameters: URL query parameters for filtering and search operations.

5.15.6 Endpoint Operation Support

Endpoints	Supported Operations
catalogs, media, sourcemedia, thumbnail	GET operations only
clips	GET, PUT, POST operations
smartfolders	GET, PUT, POST, DELETE operations
session	GET, POST, DELETE operations (used internally for authentication)

5.15.7 Outputs Description

- response_code: HTTP response code returned by the CatDV server API indicating the success or failure status of the operation.
- session_hash: Response hash received during the session authentication response containing session details and authentication information.
- session_status: Status of the authentication session establishment and management process.
- status: Overall status of the response (Can be OK, ERROR, AUTH, or MODIFIED) depending on the operation selected and execution result.
- result_hash: Response hash received during the operation response containing the actual API response data and results.

5.15.8 Operating Instructions

- Plugin Purpose: This action provides capability for performing RESTful operations like GET, PUT, POST, and DELETE for media content on CatDV servers. This plugin requires access to a CatDV server and valid login credentials for accessing the same.
- Response Processing: Responses are returned as JSON strings which are automatically converted to hash format and stored in the plugin outputs for easy access by downstream workflow steps.
- Authentication Handling: The plugin automatically handles session authentication by creating a session with the provided credentials and managing the session ID (jsessionid) for subsequent API calls.
- Example Request Formation: Using the sample configuration (Service URL: http://catdvserverone.aspera.com:8080/api/4/, Endpoint: clips, Operation: get, Selector: 12, Query parameters: filter=and((clip.userFields[U23])eq(12345))), the plugin forms a GET request URL like: http://catdvserverone.aspera.com:8080/api/4/clips;jsessionid=9F441C9E62E68812BEAA64F ter=and((clip.userFields[U23])eq(12345))
- Session Management: The jsessionid is automatically obtained after successful authentication with the credentials provided in the login and password fields, and is appended to subsequent API requests for authorization.
- API Documentation: Complete REST API documentation is available at: http://www.squarebox.com/server7-rest-api-introduction/ for detailed endpoint specifications and advanced usage patterns.

5.15.9 Dependencies

- Active CatDV server installation with REST API enabled and accessible on the specified service URL.
- Valid CatDV user account with appropriate permissions for the intended operations on target endpoints.
- Network connectivity between Orchestrator and the CatDV server including any required firewall configurations.
- CatDV server version compatibility with the API version specified in the service URL path.
- Proper CatDV database and media catalog configuration for the endpoints being accessed.

5.15.10 Troubleshooting

- If authentication fails, verify the CatDV server URL is accessible and the login credentials are correct. Check that the user account has appropriate permissions for API access.
- For connection timeout issues, confirm the CatDV server is running and the service URL includes the correct protocol, hostname, port, and API path.

- If specific endpoints return 'not found' errors, verify the endpoint name is spelled correctly and that the CatDV server supports the requested API version.
- For operation permission errors, check that the selected operation (GET, PUT, POST, DELETE) is supported for the target endpoint as specified in the endpoint operation support table.
- If POST/PUT operations fail, ensure the 'Post request body' contains valid JSON format and includes all required fields for the target CatDV resource.
- When query parameter filtering doesn't work as expected, verify the filter syntax matches CatDV's query specification and that the referenced fields exist in the target resource schema.
- For session-related errors, check that the CatDV server session management is functioning properly and that there are no session timeout or concurrent session limit issues.

5.16 Corba Operation



Figure 5.16: Corba Operation icon

This plugin is used to make a CORBA call to the Arris CMM (Content Management and Metadata server) using the JacORB utility.

Version	Comment
0.1.2	Enhanced action template UI
0.1.1	Update help docs to specify the Java version compatibility
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.10	Removing windows support code
0.0.9	Changed category to integration
0.0.8	Java jar updated to support logger library for logging
0.0.7	Increasing plugin timeout to 60 minutes
0.0.6	Updating Corba jar to do random port selection (5 times)
0.0.5	Updated Jar with IBM licensing annotation
0.0.4	Fixing the directory path for supporting windows
0.0.3	Changed Category to Asset Management
0.0.2	Adding support for aspera java
0.0.1	initial release

5.16.1 How It Works

- Enables CORBA-based communication with CMM (Content Management and Metadata) servers to notify about package arrivals and trigger distribution workflows.
- Establishes connection to CMM servers using configurable IP addresses, ports, and authentication parameters for reliable communication.
- Supports FTP URL integration for CMM machines to fetch data, enabling seamless content retrieval and processing workflows.
- Provides comprehensive logging capabilities with configurable log file paths for execution tracking and debugging purposes.
- Utilizes CORBA naming services and factory patterns to ensure proper service discovery and object instantiation in distributed environments.

5.16.2 Configuration Instructions

1. Add a CORBA Operation step to your workflow.
2. Configure the CMM server connection:
    - Corba Ip: IP address of the target CMM server.
    - Port: Port number for CMM server communication.
    - Name Service: CORBA naming service identifier (typically 'NameService').
    - Factory Name: Factory service name (commonly 'Default').
3. Set up data access and logging:
    - FTP Url: Complete FTP URL with credentials for CMM data access.
    - Package Name: ADI package name used as servant identifier.
    - Log file path: Destination path for execution logs and output tracking.

5.16.3 Example Configuration

Parameter	Example Value
Corba Ip	124.142.220.1
Port	5000
FTP Url	ftp://username:password@16.231.12.8/test/avc/ADI_1234.xml
Name Service	NameService
Factory Name	Default
Package Name	ADI_1234
Log file path	/tmp/log1.log

5.16.4 Saved Parameters Description

Parameter	Description
Corba Ip	Provide the IP address of the CMM server for establishing CORBA communication.
Port	Provide the port number of the CMM server for network connection.
FTP Url	Provide the complete FTP URL along with username and password for the CMM machine to fetch data from the specified location.
Name Service	Provide the name service for the CORBA call. Usually set to 'NameService' for standard CORBA implementations.
Factory Name	Name of the Factory service. The example configuration uses 'Default' as the standard factory name.
Package Name	Name of the ADI package. This is used as a servant name internally within the CORBA architecture.
Log file path	Provide a path to the log file where the execution logs will be stored. The logs are appended to the file for continuous tracking.

5.16.5 Inputs Description

Note: All inputs are mandatory for the CORBA call to be executed successfully.

- corba_ip: IP address of the CMM server (mandatory for CORBA connection establishment).
- port: Port number for CMM server communication (mandatory for network connection).
- ftp_url: Complete FTP URL with authentication credentials (mandatory for data retrieval).
- name_service: CORBA naming service identifier (mandatory for service discovery).
- factory_name: Factory service name for object instantiation (mandatory for CORBA operations).
- package_name: ADI package name used as servant identifier (mandatory for package processing).
- log_file_path: Destination path for execution logs and output tracking (mandatory for logging).

5.16.6 Outputs Description

- Corba_Call_Validation: Shows the validation output of the execution, indicating success or failure of the CORBA operation.

- Raw_Execution_Output: Shows the raw execution output of the call, providing detailed response information from the CMM server.
- Log_File_Path: Shows the path to the log file where the outputs are saved, enabling access to detailed execution logs.

5.16.7 Operating Instructions

- This plugin is used to make a CORBA call to the CMM (Content Management and Metadata) server to inform about package arrival so that it can be distributed to appropriate destinations.
- The plugin establishes a CORBA connection using the specified IP address and port, then uses the naming service to locate the appropriate factory for processing the package information.
- The FTP URL parameter allows the CMM machine to fetch data from the specified location, enabling automated content retrieval as part of the distribution workflow.
- All execution details are logged to the specified log file path, providing comprehensive tracking of operations for debugging and auditing purposes.
- The package name serves as a unique identifier within the CORBA servant architecture, ensuring proper package handling and distribution routing.

5.16.8 Dependencies

- Java Runtime Environment (JRE) must be installed and accessible via the system PATH, or Aspera's bundled Java at /opt/aspera/lib/java/jre8/bin/java must be available. The plugin executes a JAR file for CORBA operations.
- Important: When performing system security updates (e.g., using dnf update or dnf-automatic), be cautious not to accidentally upgrade Java, as this may break CORBA functionality. Consider locking the Java version using package manager version pinning to prevent unintended upgrades.
- CORBA runtime environment must be properly installed and configured on the Orchestrator server.
- Network connectivity between Orchestrator and the specified CMM server IP address and port.
- Valid authentication credentials for FTP access to the specified data location.
- Proper permissions for writing to the specified log file path location.
- CMM server must be running and accessible with the configured CORBA naming service.

5.16.9 Troubleshooting

- If connection fails, verify the CMM server IP address and port are correct and the server is accessible from the Orchestrator network.
- Check that the CORBA naming service is running on the CMM server and the Name Service parameter matches the expected service name.
- Validate FTP URL credentials and ensure the specified path exists and is accessible with the provided authentication.
- Verify the log file path directory exists and the Orchestrator process has write permissions to create and append to the log file.
- Ensure the Factory Name matches the expected factory service available on the CMM server for proper object instantiation.
- Check network firewall rules and security policies that might block CORBA communication between Orchestrator and CMM servers.

5.17 Curl Operations

This action plug-in executes curl command operations on remote nodes via SSH, supporting GET and POST requests with authentication, flexible parameter handling, image downloads, and configurable result storage for comprehensive HTTP operations.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	Adding Remote Node Dependency
0.0.1	Initial release



Figure 5.17: Curl Operations icon

5.17.1 How It Works

- Executes curl command operations on remote nodes via SSH.
- Supports both GET and POST request types with authentication.
- Offers two operation modes: direct command execution or pre-configured templates.
- Can download images and store results in specified file locations.

5.17.2 Configuration Instructions

1. Create a new action step using the CurlOperation plugin.
2. Fill out the following fields:
    - Name: Label for the step (e.g., `API Data Fetch`).
    - Comments: Notes for the template or configuration.
    - Service URL: The URL from where the fetch operation is performed.
    - Remote Node: The node to SSH into to perform the curl operation.
    - Login: Username for authentication (if required).
    - Password: Password for authentication (if required).
    - Request Type: Select GET or POST.
    - Operation: Choose between ”Enter command to Execute” or ”Execute the precompiled command”.
    - Parameters: Request parameters or file path containing parameters.
    - Result Storage Location: File path where execution results will be stored.

5.17.3 Example Configuration

Field	Value
Name	`API Data Fetch`
Service URL	`https://api.example.com/data`
Remote Node	`Linux Host`
Request Type	`POST`
Operation	`Execute the precompiled command`
Parameters	`fname=John&lname=Doe`
Result Storage Location	`/tmp/api_results.txt`

5.17.4 Saved Parameters Description

Parameter	Description
Comments	Notes and comments for this step or template configuration.
Service URL	The URL from where the fetch operation is performed.
Remote Node	The node to SSH into to perform the curl operation.
Login	Username for authentication (if required).
Password	Password for authentication (if required).
Request Type	Supports GET and POST calls.

Parameter	Description
Detailed Execution Result	Choose this option if detailed command execution information is needed.
Is Parameters a file path?	Check if parameters should be fetched from a file.
Parameters	Request parameters or path to parameter file.
Redirect Automatically	Automatically follow redirect URLs if the service responds with one.
Download Image?	Check this option if downloading an image.
Image Location	System location where the image should be downloaded.
Result Storage Location	File path where execution results will be stored.
Operation	Choose execution mode: direct command or pre-configured template.

5.17.5 Operation Types

- Enter command to Execute: Provides flexibility to run commands as-is without processing.
- Execute the precompiled command: Uses pre-built configurations within the plugin.

5.17.6 Inputs Description

- Required inputs depend on the operation chosen.
- For command entry mode: the command text area is mandatory.
- For pre-configured mode: service_url and result_file_path are mandatory.
- If download image option is chosen, Image Location must be provided.
- All parameters can be set in template or provided as runtime input.

5.17.7 Outputs Description

Output	Description
Image Location	The location where the image is stored.
Execution Message	The execution output displayed if not redirected to file.
File Path	The location where execution output is stored.
Error	Exception details if any errors occur during execution.

5.17.8 Supported Actions

None

5.17.9 Dependencies

None

5.17.10 Operating Instructions

This plugin provides basic curl functionality with two operation modes.

NOTE: In Execute command operation, ensure you pass arguments to store execution results (e.g., `> filename.txt`) to prevent database exceptions with large results.

5.17.11 Parameter Format Options

- Append to URL directly
- URL format: `fname=Mark&lname=Bates`
- JSON format: `{"name":"Mark"}`
- File path containing parameters

5.17.12 Example 1: Direct Command Execution

Configuration:

- Remote Node: Linux host
- Operation: Enter Command to Execute
- Command: `curl -X POST -u "user1:password1" quiet-waters-1228.herokuapp.com/login > /tmp/results.txt`

5.17.13 Example 2: Pre-configured with Parameters

Configuration:

- Remote Node: Linux host
- Operation: Execute the Preconfigured command
- Request Type: POST
- Service URL: `quiet-waters-1228.herokuapp.com/echo`
- Parameters: `fname=Mark&lname=Bates`
- Result Storage Location: `/usr/bin/output/test.txt`

5.17.14 Example 3: Image Download

Configuration:

- Remote Node: Linux host
- Operation: Execute the Preconfigured command
- Request Type: GET
- Service URL: `http://quiet-waters-1228.herokuapp.com/assets/image.jpg`
- Download Image: enabled
- Image Location: `/tmp/myimage.png`
- Result Storage Location: `/usr/bin/output/test.txt`

5.17.15 Troubleshooting

- Ensure remote node is accessible via SSH.
- Verify authentication credentials are correct.
- Check that result storage paths are writable.
- For large responses, always redirect output to files to avoid database limitations.

5.18 Custom progress notifier

This plugin provides the ability to intercept progress notifications from a step and add custom behavior to it. For example, logging to an external log file or RSS feed.

Version	Comment
0.2.2	Enhanced action template UI
0.2.1	Remove explicit require for progress_observer to fix Rails 8 Zeitwerk autoloading
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.6	Adding exception handling to check status
0.1.5	Including 'Percent complete' reporting functionality.
0.1.4	Adds help file, sorts action type
0.1.3	Fix non-persistence of instance variable for asynch mode

Version	Comment
0.1.2	New icon
0.1.1	Reset progress notified and logger to avoid singleton serialization issues
0.1.0	Adds ability to add I/Os
0.0.1	Initial release

5.18.1 How It Works

- Acts as a pass-through wrapper for existing action templates, allowing the proxied action to execute normally while adding custom progress notification capabilities.
- Enables execution of custom Ruby code every time the wrapped action reports a status change, providing real-time monitoring and logging functionality.
- Supports adding additional required and optional inputs to the proxied action, extending its functionality without modifying the original template.
- Provides access to progress details including status messages, state changes, and completion percentages through predefined variables.
- Offers the option to bypass standard system notifications while maintaining custom notification logic for specialized monitoring requirements.

5.18.2 Configuration Instructions

1. Add a Custom Progress Notifier step to your workflow.
2. Select the action template to wrap:
   - Action plugin: Choose the type of action template to proxy.
   - Action template: Select the specific common template to wrap (private templates cannot be selected).
3. Configure additional inputs and notification behavior:
   - Additional required inputs: Define extra mandatory inputs for custom logic (e.g., log file paths).
   - Additional optional inputs: Define extra optional inputs for enhanced functionality.
   - Report progress code: Write Ruby code to execute on each progress report.
   - By-pass standard notifications?: Check to disable standard system notifications.

5.18.3 Example Configuration

Parameter	Example Value
Action plugin	LocalFileWatcher
Action template	File Watcher Template
Additional required inputs	log_file_path
Additional optional inputs	notification_email
Report progress code	File.open(@inputs['log_file_path'], 'a') { |f| f.puts ”2026-08-07T09:39:54+02:00: -- --” }
Keep alive code	(not used in current version)
By-pass standard notifications?	unchecked

5.18.4 Saved Parameters Description

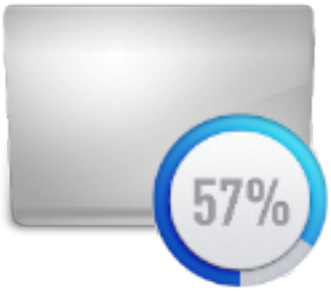


Figure 5.18: Custom progress notifier icon

Parameter	Description
Action plugin	Select the type of the action template to be proxied. This determines the category of actions available for wrapping.
Action template	The action template to be proxied - only common templates can be selected, not private ones. This is the actual action that will execute with custom progress monitoring.
Additional required inputs	Additional inputs required by the custom progress notifying logic that will be added to the inputs expected by the proxied plugin. One example could be the path to the external log file to log to.
Additional optional inputs	Same as above, but for optional inputs. These provide extra functionality without being mandatory for execution.
Report progress code	This block of code will be called every time the proxied action template reports progress. The code can reference inputs using the '@inputs' hash and access progress variables.
Keep alive code	This block of code would be used whenever the keep alive function is called, but it is not in the current version of the engine. Best is to ignore this for now.
By-pass standard notifications?	By default, the custom progress notification is called, then the standard system progress notification is called as well. If this is checked, only the custom notification code will be called.

5.18.5 Inputs Description

The inputs for the action will be the ones of the chosen proxied action template, augmented with those declared in the 'Additional required inputs' and 'Additional optional inputs' sections.

- proxied_action_inputs: All inputs required by the selected action template being proxied.
- additional_required_inputs: Custom mandatory inputs defined in the configuration for progress notification logic.
- additional_optional_inputs: Custom optional inputs defined in the configuration for enhanced functionality.

5.18.6 Outputs Description

The outputs for the action will be the ones of the chosen proxied action template.

- proxied_action_outputs: All outputs produced by the selected action template being proxied, unchanged from the original action.

5.18.7 Available Progress Variables

- @status_details: Contains the status message being reported by the proxied action (string)
- @status: Contains the status state change if applicable - nil indicates no changes (string)
- @percent: Contains the percent complete if provided (numerical)
- @inputs: Hash containing all inputs (both from proxied action and additional ones)

5.18.8 Operating Instructions

- This plugin is a pass-through to the chosen proxied action. The chosen proxied action will still be executed in lieu of this action, but it provides the ability to execute Ruby code every time the status of the executed proxied action reports a status change.
- In the progress reporting code, inputs (combining both the inputs from the proxied action and the additional ones requested by the custom progress notifier wrapper) can be accessed through the @inputs hash.
- The status message being reported by the proxied action is available in 3 parts: the status message in @status_details (string), the status state change in @status (string, nil indicates no changes), and the percent complete in @percent (numerical, if provided).
- Use the progress reporting code to implement custom logging, notifications, or monitoring logic that executes alongside the wrapped action without interfering with its normal operation.
- When 'By-pass standard notifications' is enabled, only your custom notification code will execute, giving you complete control over progress reporting behavior.

5.18.9 Example Output Log

The following shows an example custom log output from a File Watcher being monitored:

\ 2016-02-26T11:21:19-08:00: In Progress -- File Watcher started --\ 2016-02-26T11:21:19-08:00: In Progress -- Scanning ”/tmp” for files matching *.txt... --\ 2016-02-26T11:21:19-08:00: In Progress -- Identified 9 candidate files at Fri Feb 26 11:21:19 \ 2016-02-26T11:21:22-08:00: In Progress -- Scanning ”/tmp” for files matching *.mxf... --\ 2016-02-26T11:21:23-08:00: In Progress -- Scanning ”/tmp” for files matching *.txt... --\ 2016-02-26T11:21:23-08:00: Complete -- Triggered by file ”/tmp/la1.txt” --\ 2016-02-26T11:21:25-08:00: In Progress -- File Watcher started --\ 2016-02-26T11:21:25-08:00: In Progress -- Identified 8 candidate files at Fri Feb 26 11:21:25 PST 2016 --\ 2016-02-26T11:21:28-08:00: Complete -- Triggered by file ”/tmp/la2.txt” --\

5.18.10 Dependencies

- The selected action template to be proxied must be available and properly configured in the system.
- Ruby runtime must be available for executing custom progress notification code.
- Proper file system permissions for any log files or external resources referenced in the custom code.
- Network access if the custom progress code involves external notifications or API calls.

5.18.11 Troubleshooting

- If the proxied action fails to execute, verify that the selected action template is valid and all its required inputs are properly configured.
- Check Ruby syntax in the 'Report progress code' block if custom notifications are not working - syntax errors will prevent execution.
- Verify that paths referenced in @inputs hash exist and are accessible if file operations are used in the custom code.
- If progress notifications are not appearing, ensure 'By-pass standard notifications' is unchecked unless you want only custom notifications.
- Test the wrapped action independently first to ensure it works correctly before adding custom progress monitoring.
- Check log file permissions and disk space if custom logging is configured but not producing output.

5.19 Facebook Video Upload Operation

This action plugin is used to perform a resumable video upload via Facebook Graph API.

Version	Comment
0.3.0	Enhanced action template UI
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	Adds reference_only option and video_id output.
0.0.3	Adding to help page.
0.0.2	Correcting revision history.
0.0.1	Initial release.



Figure 5.19: Facebook Video Upload Operation icon

5.19.1 How It Works

- Uploads video files to Facebook on behalf of a User, Event, Page, or Group.
- Uses the Facebook Graph API with an access token for authenticated communication.
- Accepts the video as a file path on the Orchestrator server.
- Uses a configurable temporary folder to stage video chunks during the upload process.
- Automatically resets the upload if the video file size or modification time changes during transfer.
- Supports controlling whether the video is published to the feed or uploaded as a reference video.
- Returns the Upload Session ID and Video ID upon successful transfer.
- Returns the full server response from the publish call for downstream handling.

5.19.2 Configuration Instructions

1. Create a new action step using the FacebookUploadOperation plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved FacebookUploadOperation configured instance.
   - Comments: Comments about this saved FacebookUploadOperation configured instance.
   - Object id: User, Event, Page, or Group ID that serves as the endpoint for the video upload.
   - Access token: Token needed to authenticate and communicate with Facebook.
   - Video file: Path of the video file to upload.
   - Folder: Path of the folder where temporary video files are stored during transfer. Defaults to the plugin default folder if not specified.
   - Published: Enable to publish the video to the feed (default: true). Disable to upload as a reference video only.

5.19.3 Example Configuration

This is an example configuration for the FacebookUploadOperation plugin uploading a product demo video to a Facebook Page.

Parameter	Value
Name	Upload Product Demo Video
Comments	Upload weekly product demo video to company Facebook Page
Object id	123456789012345
Access token	(stored securely)
Video file	/data/videos/product_demo_2025_01.mp4
Folder	/tmp/facebook_uploads
Published	Selected (default)

5.19.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved FacebookUploadOperation configured instance.
Comments	Comments about this saved FacebookUploadOperation configured instance.

Parameter	Description
Object id	User, Event, Page, or Group ID that serves as the endpoint for the video upload.
Access token	Token needed to authenticate and communicate with Facebook.
Video file	Path of the video file to upload.
Folder	Path of the folder where temporary video files are stored during transfer. Defaults to the plugin default folder if not specified.
Published	Enable to publish the video to the feed (default: true). Disable to upload as a reference video only.

5.19.5 Inputs Description

Usually this plugin does not have runtime specified parameters. Inputs such as the video file path or object ID may be specified at runtime when these values are generated dynamically within the workflow.

Parameter	Description
object_id	User, Event, Page, or Group ID that serves as the endpoint for the video upload.
access_token	Token needed to authenticate and communicate with Facebook.
video_file	Path of the video file to upload. Must be accessible from the Orchestrator server.
folder	Path of the folder where temporary video files are stored during transfer. Defaults to the plugin default folder if not specified.

5.19.6 Outputs Description

The output includes the Upload Session ID, the Video ID assigned by Facebook, and the full server response from the publish call.

Parameter	Description
upload_id	ID of the upload session created by Facebook for this transfer.
video_id	ID assigned to the uploaded video by Facebook.
response	Full server response returned by Facebook after the publish video call.

5.19.7 Supported Actions

None

5.19.8 Dependencies

Requires a valid Facebook access token with appropriate permissions for video upload to the target User, Page, Event, or Group.

5.19.9 Operating Instructions

This action uploads a video file to Facebook using the Graph API, targeting the specified User, Page, Event, or Group object. The upload is staged via a temporary folder before being published.

Reference Video Option

- When Published is disabled, the video is uploaded as a reference video rather than being published to the object's feed.
- Reference videos can be used for rights management and content matching purposes on Facebook.
- Leave Published checked (default) for standard video publishing to a User, Page, Event, or Group.

Best Practices

- Store the Access token securely using Orchestrator's credential management rather than plaintext configuration.
- Ensure the access token has the appropriate Facebook permissions for video upload to the target object (e.g., `publish_video`).
- Verify the video file path is accessible by the aspweb user.

- Use a dedicated temporary folder with sufficient disk space to stage video chunks during upload, especially for large files.
- Store the returned Video ID output for use in downstream steps, such as setting video metadata or verifying publish status.
- Avoid modifying the video file during upload — any change to file size or modification time will reset the upload process.
- Implement error handling workflows that check the response output to manage upload failures or Facebook API errors gracefully.
- Refresh the Access token before expiry to prevent authentication failures mid-workflow.

Troubleshooting

- Authentication failure: Verify the access token is valid, not expired, and has the required Facebook permissions for the target object.
- Object not found: Confirm the Object ID is correct and the access token has permission to post to that User, Page, Event, or Group.
- Video file not found: Verify the video file path is correct and the file is accessible from the Orchestrator server.
- Upload reset unexpectedly: Ensure the video file is not being modified by another process during upload. Any change to file size or modification time will trigger a reset.
- Temporary folder errors: Verify the specified folder exists and the Orchestrator process has read/write permissions to it.
- Empty Video ID output: Check the response output for Facebook API error details. An absent Video ID typically indicates the upload did not complete successfully.
- Large file timeouts: For large video files, ensure the workflow timeout is configured to accommodate longer upload durations.

Important Notes

- The upload process is automatically reset if the video file's size or modification time changes during transfer. Ensure the file is stable before the workflow runs.
- The Object ID must correspond to a User, Event, Page, or Group that the access token has permission to upload video to.
- The temporary folder is used to stage video data during transfer — ensure it has sufficient disk space for the video being uploaded.
- The Access token must have appropriate Facebook API permissions granted for video upload to succeed.
- The Video ID returned is the only reference to the uploaded video — store it if downstream steps require it.
- The operation is synchronous — the workflow waits for the upload and publish call to complete before proceeding.

5.20 Frameio Operation



Figure 5.20: Frameio Operation icon

This action plug-in provides the ability to perform operations on Frame.io.

Version	Comment
0.3.2	Enhanced action template UI
0.3.1	Add support for using v4 Enterprise accounts (with developer tokens)
0.3.0	Add support for Frame.io v4 API
0.2.1	Fix threading logic to prevent workers stopping due to memory issues

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.4	Fixes the Upload Failed issue.
0.1.3	Fixes file closing issue.
0.1.2	Limiting the number of active threads.
0.1.1	Fix for upload urls data type.
0.1.0	Allows for SSL to not be used.
0.0.4	Adding compatibility for Ruby 1.8.7.
0.0.3	Fix for inputs page crash.
0.0.2	Additional changes to the initial release.
0.0.1	Initial release.

5.20.1 How It Works

- This action plug-in enables you to perform various operations on Frame.io using v2 or v4 API.

5.20.2 Configuration Instructions

1. Create a new action step using the FrameioOperation plug-in.
2. Complete the following configuration fields:
    - Name: The name used to identify a saved FrameioOperation configuration.
    - Comments: Optional comments for this saved FrameioOperation configuration.
    - Verify SSL?: Select this option to enable SSL verification.
    - API Version: Select the Frame.io API version to use (v2 or v4). v4 is recommended for new integrations and migrated accounts.
    - Authentication Type: Select the authentication method: 'Developer Token (Legacy)' adds x-frameio-legacy-token-auth header (v4 non-enterprise accounts); 'OAuth (Adobe IMS)' uses a plain Bearer token for Adobe IMS access tokens; 'Enterprise (Service Client)' adds x-frameio-service-client header for Adobe Admin Console Enterprise accounts.
    - Client ID: Required when Authentication Type is 'Enterprise (Service Client)'. The Adobe service client ID provided by your Frame.io administrator. Sent as the x-frameio-service-client header.
    - Operation: The operation to perform on Frame.io. Available options:
        - Get Account Id - Retrieve the Account ID of the authenticated user.
        - Get Team List - Retrieve all teams (v2) or workspaces (v4) associated with an account.
        - Get Project List - Retrieve all projects under a specific team (v2) or workspace (v4).
        - Get Assets List - Retrieve child assets (v2) or folder children (v4) within a folder or version stack. Returns an empty list for single assets.
        - Create Asset - Create a new file or folder asset. Returns upload URLs for file assets.
        - Upload Asset - Upload a file to Frame.io using pre-signed upload URLs obtained from Create Asset.
        - Create And Upload Asset - Create a file or folder asset and upload the file in a single step.
    - Token: Your Frame.io access token. Use a developer token for legacy_token auth, or an Adobe IMS access token for OAuth auth.
    - Account ID: The unique ID of your Frame.io account. Required for all v4 API operations. Can be retrieved using Get Account Id.
    - Team ID / Workspace ID: The ID of a specific team (v2) or workspace (v4). Required for Get Project List.
    - Asset ID / Folder ID: The ID of a specific asset (v2) or folder (v4). Required for Get Assets List, Create Asset, and Create And Upload Asset.
    - Asset type: The type of asset to create. Acceptable values: 'file' or 'folder'.
    - Asset name: The name to give the asset being created.
    - File type: v2 only. The MIME type of the file (e.g., video/mp4). Not used in v4, media type is inferred automatically from the file.
    - File size: Optional. The size of the file in bytes. Only used for Create Asset (v2 and v4). If local file path is provided, file size is resolved automatically and this field can be left blank.
    - Upload URLs list: Array of pre-signed S3 upload URLs obtained from the Create Asset operation output. Used by Upload Asset.
    - Local file path: The server-side path of the file to upload (e.g., /opt/aspera/orchestrator/output.mp4). Used to determine file size automatically and to read file content for upload.
    - Content type: The MIME type of the file being uploaded to S3 (e.g., video/mp4, video/quicktime). Required for Upload Asset and Create And Upload Asset.

5.20.3 Example Configuration

Below is an example configuration for uploading a file using the v4 API.

Parameter	Value
Verify SSL?	Selected
API Version	v4
Authentication Type	OAuth (Adobe IMS)
Operation	Create And Upload Asset
Token	**********
Account ID	account_id from Get Account Id
Asset ID / Folder ID	folder_id
Asset name	my_video.mp4
Asset type	file
Local file path	/opt/aspera/orchestrator/output.mp4
Content type	video/mp4

5.20.4 Saved Parameters Description

5.20.5 Inputs Description

Required inputs depend on how the FrameioOperation template is configured. If dynamic attributes are used, each attribute becomes a runtime input.

Parameter	Description
*Token **	Frame.io access token (developer token for legacy auth, or Adobe IMS access token for OAuth).

5.20.6 Outputs Description

Parameter	Description
Account_Id	Get Account Id: The unique ID of the Frame.io account.
Team_List	Get Team List: Array of teams (v2) or workspaces (v4) for the account.
Project_List	Get Project List: Array of projects under the specified team or workspace.
Assets_List	Get Assets List: Array of child assets or folder contents.
Upload_Urls	Create Asset / Create And Upload Asset: Array of pre-signed S3 upload URLs. In v4, each entry is a hash with 'url' and 'size' keys.
Number_of_threads_cessfully_executed	Upload Asset / Create And Upload Asset: Number of file chunks successfully uploaded to S3.
Result	All operations: The full raw API response.
Error_Message	All operations: Any error message generated during execution.

5.20.7 Supported Actions

None

5.20.8 Dependencies

None

5.20.9 Operating Instructions

1. Use this plug-in to execute Frame.io operations via v2 or v4 API.

2. For v4 Enterprise accounts (Adobe Admin Console): use v4 API with 'Enterprise (Service Client)' authentication. Provide the Client ID supplied by your Frame.io admin. The x-frameio-service-client header is added automatically.
3. For v4 non-enterprise accounts (Adobe-managed but not Enterprise): use v4 API with 'Developer Token (Legacy)' authentication. The x-frameio-legacy-token-auth header is added automatically.
4. For v4 accounts using Adobe IMS OAuth tokens: use v4 API with 'OAuth (Adobe IMS)' authentication. Only the Bearer token is sent.
5. v2 API continues to work for unmigrated accounts using Developer Token authentication.
6. Typical v4 upload workflow: (1) Get Account Id → (2) Get Team List to find workspace ID → (3) Get Project List to find root folder ID → (4) Get Assets List to browse folders → (5) Create And Upload Asset to upload a file.
7. Available operations:
    - Get Account Id - Retrieve the Account ID of the authenticated user.
    - Get Team List - Retrieve all teams (v2) or workspaces (v4) associated with an account.
    - Get Project List - Retrieve all projects under a specific team (v2) or workspace (v4).
    - Get Assets List - Retrieve child assets (v2) or folder children (v4) within a folder or version stack. Returns an empty list for single assets.
    - Create Asset - Create a new file or folder asset. Returns upload URLs for file assets.
    - Upload Asset - Upload a file to Frame.io using pre-signed upload URLs obtained from Create Asset.
    - Create And Upload Asset - Create a file or folder asset and upload the file in a single step.

5.21 Google Cloud Storage Operation



Figure 5.21: Google Cloud Storage Operation icon

This action plug-in provides the ability to perform an action on the Google Cloud Storage service.

Version	Comment
0.1.1	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	Remove hardcoded aspera directory reference
0.0.2	Update path to Ruby / SSL cert file
0.0.1	Initial release.

5.21.1 Saved Parameters Description

- Name: The name used to identify a saved GoogleCloudstorageOperation configured instance.
- Comments: Some comments about this saved GoogleCloudstorageOperation configured instance.
- Operation: The operation to perform.
- Credentials filepath: The file path on Orchestrator of the Google service account JSON key file.
- Project id: The project id.
- Bucket name: The storage bucket name.
- Local filepath: The local filepath. For upload and download operations.
- Remote filepath: The remote filepath. For upload and download operations.

5.21.2 Inputs description

The list of inputs depends on the configuration of the GoogleCloudstorageOperation action template.

- *Operation*: The operation to perform (only visible in input if -- select operation -- is left in the plugin configuration).
- *Credentials_filepath*: The file path on Orchestrator of the Google service account JSON key file.
- *Project_id*: The project id.
- *Bucket_name*: The bucket name.
- *Local_filepath*: The local filepath.
- *Remote_filepath*: The remote filepath.

5.21.3 Outputs description

- *Error_message*: The error message in case of failure.

5.21.4 Supported Actions

None

5.21.5 Dependencies

None

5.21.6 Operating Instructions

This action plugin is used to perform an action on the Google Cloud Storage service.

5.21.7 Before you can use the plugin, you need to perform these operations:

On Orchestrator, install the google-cloud-storage gem by adding this line in your Gemfile with your google-cloud-storage gem version:

gem 'google-cloud-storage', '1.21.0'

Run the following command:

bundle install

Log in the Google Cloud console at https://console.cloud.google.com/

Create a project.

In Billing, enable billing for the project and create a billing account.

In APIs & Services, under Credentials, create a service account JSON key.

Keep the json key credential file and copy it somewhere on the Orchestrator machine.

Upload/download operations:

To upload or download a file, specify the full local file path on Orchestrator with the filename (e.g. /tmp/file1.txt) and the remote filepath on Google Cloud Storage with the filename for the selected bucket (e.g. folder1/file1.txt).

If the source filepath (local for upload and remote for download) is not existing, an error is returned.

If the remote file is already existing in an upload operation, it will be overwritten by the local file.

If you specify a remote folder that is not existing in an upload operation, it will be created on the bucket. For example if you specify folder2/file2.txt and folder2 is not existing in the bucket, it will be created and file2.txt will be put in this folder.

If the local file is already existing in a download operation, it will be overwritten by the remote file.

If you specify a local folder that is not existing in a download operation, an error is retuned.

5.22 Google Pubsub Operation

This action plug-in provides the ability to perform an action on the Google Pub/Sub service.



Figure 5.22: Google Pubsub Operation icon

Version	Comment
0.1.1	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	Remove hardcoded aspera directory reference
0.0.2	Update path to ruby / SSL certs
0.0.1	Initial release.

5.22.1 Saved Parameters Description

- Name: The name used to identify a saved GooglePubsubOperation configured instance.
- Comments: Some comments about this saved GooglePubsubOperation configured instance.
- Operation: The operation to perform.
- Credentials filepath: The file path on Orchestrator of the Google service account JSON key file.
- Project id: The project id.
- Topic name: The topic name.
- Subscription name: The subscription name.
- Message body: The message content for the publish operation.
- Max number of messages: The maximum number of messages to be retrieved. For the read messages operation.
- Keep ongoing?: If checked, the containing workflow will be cloned after each trigger event, ensuring that the workflow execute everytime messages are returned. For the read messages operation.
- Polling frequency: By default, the Pub/Sub service is probed every 5 seconds for new messages. That polling frequency can be changed to any number of seconds. For the read messages operation.

5.22.2 Inputs description

The list of inputs depends on the configuration of the GooglePubsubOperation action template.

Changes in this list of inputs will override any values set in the action template.

- *Operation*: The operation to perform (only visible in input if -- select operation -- is left in the plugin configuration).
- *Credentials_filepath*: The file path on Orchestrator of the Google service account JSON key file.
- *Project_id*: The project id.
- *Topic_name*: The topic name.
- *Subscription_name*: The subscription name.
- *Message_body*: The message content for the publish operation.
- *Max_number_of_messages*: The maximum number of messages to be retrieved for the read messages operation (optional).

5.22.3 Outputs description

- *Messages*: The messages array received from the Pub/Sub service for the read messages operation.
- *Error_message*: The error message in case of failure.

5.22.4 Supported Actions

None

5.22.5 Dependencies

None

5.22.6 Operating Instructions

This action plugin is used to perform an action on the Google Pub/Sub service.

5.22.7 Before you can use the plugin, you need to perform these operations:

On Orchestrator, install the google-cloud-pubsub gem by adding this line in your Gemfile with your google-cloud-pubsub gem version:

gem 'google-cloud-pubsub', '0.39.1'

Run the following command:

bundle install

Log in the Google Cloud console at https://console.cloud.google.com/

Create a project.

In Billing, enable billing for the project and create a billing account.

In APIs & Services, under Credentials, create a service account JSON key.

Keep the json key credential file and copy it somewhere on the Orchestrator machine.

Regarding Google Pub/Sub:

Any message published to a topic without an existing subscription will be lost.

Published messages to a topic are delivered to all subscribers.

By default, subscriptions retain unacknowledged messages for up to 7 days from the moment they are published.

By default, subscriptions expire after 31 days of inactivity (if no read messages activity).

Read messages are immediately acknowledged and deleted.

If you read concurrently messages from 2 subscribers (via 2 workorders) on the same topic, each subscriber will get a message published to this topic.

The max number of returned messages on a read messages operation operation is by default 1.

5.23 Ibm mq integration



Figure 5.23: Ibm mq integration icon

This plugin is used to send/receive MQ messages to/from an IBM MQ system.

Version	Comment
0.5.4	Enhanced action template UI

Version	Comment
0.5.3	Updated the label from Keep ongoing to Continue Monitoring
0.5.2	Updating to be compatible with Orchestrator 4.1, which isn't shipped with a Java App and Tomcat server
0.5.1	Updating to be compatible with Ruby 3.4 and Rails 8
0.5.0	Remove hardcoded aspera directory reference
0.4.0	Added SSL support
0.3.2	Better handling of exceptions
0.3.1	added tomcat part to the help file
0.3.0	add user and password support
0.2.0	add message Id to response
0.1.4	accept json changed to xml
0.1.3	Help file update
0.1.2	Removing log_file_path code since we are introducing logging at MQ level
0.1.1	Added response code to the checks
0.1.0	Fixing layouts and added execution node
0.0.1	Initial implementation

5.23.1 How It Works

- Integrates with an IBM MQ server to send (put) and receive (get) messages via configured queues and channels.
- Supports Put Message operations to publish a payload to a specified MQ queue.
- Supports Get Message operations to retrieve messages from a specified MQ queue.
- Optionally supports SSL/TLS encryption and authentication using a configurable cipher and Java trust store.
- Can target an MQ server via a configured remote Execution Node or directly via IP address and port.
- Uses a Python-based MQ Bridge service that wraps the IBM MQ Java SDK JAR file to communicate with IBM MQ servers.
- For Get operations, supports a Keep Trigger Ongoing option to continuously listen for new incoming messages.
- Returns the received message content, Message ID, and JSON-formatted message for Get operations, and a delivery confirmation for Put operations.

5.23.2 Configuration Instructions

1. Create a new action step using the IbmMqIntegration plugin.
2. Fill out the following fields:
    - Name: The name used to identify a saved IbmMqIntegration configured instance.
    - Comments: Comments about this saved IbmMqIntegration configured instance.
    - Execution Node: Remote node on which the IBM MQ server is installed. Used when a remote node is configured for MQ.
    - MQ IP Address: IP address of the MQ server. Used when no remote Execution Node is configured.
    - MQ Port: Port the MQ server listens on. Default is 1414.
    - Use SSL?: Check this box to enable SSL/TLS encryption and authentication.
    - Cipher: SSL/TLS cipher suite to use (e.g., TLS_RSA_WITH_AES_128_CBC_SHA256). Must use the Oracle JRE TLS_ prefix.
    - Trust store filename: Filename of the Java application SSL/TLS trust store (e.g., user.jks). Must be placed in the Orchestrator JRE security directory.
    - Trust store password: Password for the trust store file (optional).
    - MQ Operation Type: The operation to perform. Supported values: Put Message, Get Message.
    - MQ Channel Name: Name of the MQ channel to connect through.
    - MQ Queue Name: Name of the MQ queue to send to or receive from.
    - MQ Queue Manager Name: Name of the MQ Queue Manager.
    - MQ User Name: Username for authenticating with the MQ Queue Manager.
    - MQ Password: Password for authenticating with the MQ Queue Manager.
    - MQ Put Request (XML file path): Path to the XML file containing the message payload to send. Used for Put Message operations.
    - Keep Trigger Ongoing: When enabled for Get Message operations, the plugin continues triggering on new incoming messages.

5.23.3 Example Configuration

This is an example configuration for the IbmMqIntegration plugin sending an order payload to an IBM MQ queue.

Parameter	Value
Name	Send Order Message to MQ
Comments	Put order payload onto the Orders queue
Execution Node	(blank - using IP address instead)
MQ IP Address	192.168.1.50
MQ Port	1414
Use SSL?	Not checked
MQ Operation Type	Put Message
MQ Channel Name	ORDER.CHANNEL
MQ Queue Name	ORDER.QUEUE
MQ Queue Manager Name	QM_ORDERS
MQ User Name	mquser
MQ Password	(stored securely)
MQ Put Request (XML file path)	/path/to/order_payload.xml
Keep Trigger Ongoing	Not checked

5.23.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved IbmMqIntegration configured instance.
Comments	Comments about this saved IbmMqIntegration configured instance.
Execution Node	Remote node on which the IBM MQ server is installed. Used when a remote node is configured for MQ.
MQ IP Address	IP address of the MQ server. Used when no remote Execution Node is configured.
MQ Port	Port the MQ server listens on. Default is 1414.
Use SSL?	Check this box to enable SSL/TLS encryption and authentication.
Cipher	SSL/TLS cipher suite to use (e.g., TLS_RSA_WITH_AES_128_CBC_SHA256). Must use the Oracle JRE TLS_ prefix.
Trust store filename	Filename of the Java application SSL/TLS trust store (e.g., user.jks). Must be placed in the Orchestrator JRE security directory.
Trust store password	Password for the trust store file (optional).
MQ Operation Type	The operation to perform. Supported values: Put Message, Get Message.
MQ Channel Name	Name of the MQ channel to connect through.
MQ Queue Name	Name of the MQ queue to send to or receive from.
MQ Queue Manager Name	Name of the MQ Queue Manager.
MQ User Name	Username for authenticating with the MQ Queue Manager.
MQ Password	Password for authenticating with the MQ Queue Manager.
MQ Put Request (XML file path)	Path to the XML file containing the message payload to send. Used for Put Message operations.
Keep Trigger Ongoing	When enabled for Get Message operations, the plugin continues triggering on new incoming messages.

5.23.5 Inputs Description

Runtime inputs can override configured values. Queue Manager Name, Queue Name, Channel Name, Body (file path), and IP address are the core required parameters for sending a message.

Parameter	Description
Queue Manager Name	Name of the MQ Queue Manager. Required for all operations.
Queue Name	Name of the MQ queue to send to or receive from. Required for all operations.
Channel Name	Name of the MQ channel. Required for all operations.
Body	Path to the XML file containing the message payload. Required for Put Message operations.
Ip	IP address of the MQ server. Required when no Execution Node is configured.
MQ_cipher	SSL/TLS cipher suite. Required when SSL is enabled.
MQ_trust_store	Trust store filename. Required when SSL is enabled.
MQ_trust_store_word	Trust store password. Required when SSL is enabled and the trust store is password-protected.

5.23.6 Outputs Description

For Put operations, the output confirms message delivery. For Get operations, the output includes the message content, its ID, and a JSON-formatted version where applicable.

Parameter	Description
Message_Received	Content of the received message for a Get operation. For a Put operation, returns the value 'Message Delivered'.
Message_ID	ID of the received message. Available for Get operations only.
Message_Received_JSON	Content of the received message formatted as JSON, if applicable. Available for Get operations only.

5.23.7 Supported Actions

None

5.23.8 Dependencies

Requires Python 3, Flask library, Java Runtime Environment, IBM MQ client libraries, and the MQ Bridge service running on port 8080. The MQ server must be accessible from the Orchestrator host.

5.23.9 Operating Instructions

This action integrates with an IBM MQ server to put or get messages via a configured queue and channel. A Python-based MQ Bridge service wraps the IBM MQ Java SDK JAR file to handle communication with IBM MQ.

IBM MQ Server Setup

- All commands below are run as the `mqm` user on the MQ server.
- Include MQ binaries in the path: `./opt/mqm/bin/setmqenv -s`
- Queue Manager commands:
  - Create a Queue Manager: `crtmqm queue_manager_name`
  - Start a Queue Manager: `strmqm queue_manager_name`
  - Open the command line for a Queue Manager: `runmqsc queue_manager_name`
- Queue creation (run inside `runmqsc`): `define qlocal (Queue_Name) usage (xmitq)`
- Channel creation and listener (run inside `runmqsc`):
  - Define a channel: `define channel (Channel_Name) chltype (SVRCONN) trptype (TCP)`
  - Start the listener: `start listener(system.default.listener.tcp)`

MQ Bridge Service Setup

- The plugin requires a Python-based MQ Bridge service that wraps the IBM MQ Java SDK JAR file to communicate with IBM MQ servers.
- The MQ Bridge service is located at: `/opt/aspera/orchestrator/script/ibm_mq_plugin_scripts/mq_bridge.py`

- Prerequisites:
  - Python 3 must be installed on the system
  - Flask Python library: `pip3 install flask`
  - Java Runtime Environment (JRE) to run the IBM MQ JAR file
  - IBM MQ client libraries installed at `/opt/mqm/bin`
- Starting the MQ Bridge:
  - Navigate to the scripts directory: `cd /opt/aspera/orchestrator/script/ibm_mq_plugin_scripts`
  - Start the bridge: `./start_mq_bridge.sh`
  - The script will automatically install Flask if not present and start the service in the background
- Stopping the MQ Bridge: `./stop_mq_bridge.sh`
- The MQ Bridge listens on port 8080 by default. Ensure no other service is using this port before starting.
- Service logs are located at: `/opt/aspera/orchestrator/log/mq_debug.log`
- Health check endpoint: `curl http://localhost:8080/health`
- MQ API message and reason codes reference: IBM MQ API Codes

SSL/TLS Configuration

- The cipher must use the Oracle JRE `TLS_` prefix (e.g., `TLS_RSA_WITH_AES_128_CBC_SHA256`), not the IBM `SSL_` prefix.
- The cipher must match the cipher defined on the MQ server side.
- The trust store file must be placed in: `/lib/java/jre/lib/security/`
- Supported trust store formats: JKS or PKCS. Specify the filename including its extension in the Trust store filename field.
- The trust store file must contain the full chain CA certificate from the MQ server side.
- Trust store files can be created with tools such as keytool, ikeycmd (bundled with Java), or runmqckm (bundled with IBM MQ).
- The trust store password is optional. Enter only if one was defined when the trust store was created.

Best Practices

- Store the MQ Password and trust store password securely using Orchestrator's credential management rather than plaintext configuration.
- Use an Execution Node when the MQ server is configured as a managed remote node in Orchestrator, and use IP address only for direct connectivity.
- Verify the MQ Bridge service is running on port 8080 before executing any MQ workflow steps.
- Check `mq_debug.log` first when troubleshooting — it contains detailed Python bridge and MQ SDK error output.
- Use Keep Trigger Ongoing judiciously — continuous message listening can hold workflow resources; ensure appropriate timeout and termination logic is in place.
- For SSL environments, test the cipher and trust store configuration manually before integrating into a workflow to isolate connectivity issues.
- Implement error handling workflows to manage scenarios where no message is available on the queue during a Get operation.
- Use Message_Received_JSON output for downstream JSON parsing when the expected message format is JSON.
- Ensure the XML payload file path is accessible from the Orchestrator server when using Put Message operations.

Troubleshooting

- MQ Bridge not running: Ensure the MQ Bridge service has been started using `./start_mq_bridge.sh` and is listening on port 8080. Check for port conflicts.
- Flask not installed: Install Flask using `pip3 install flask` or let the start script install it automatically.
- Connection refused: Verify the MQ IP address and port are correct and reachable from the Orchestrator server. Confirm the MQ listener is running.
- Authentication failure: Check the MQ User Name and Password. Ensure the user has the required permissions on the Queue Manager.
- Queue or channel not found: Verify the Queue Name, Channel Name, and Queue Manager Name are correctly specified and exist on the MQ server.
- SSL handshake failure: Confirm the cipher uses the Oracle `TLS_` prefix and matches the MQ server cipher. Verify the trust store file is in the correct directory and contains the MQ server CA certificate.
- Trust store errors: Ensure the trust store file format is JKS or PKCS and the password (if set) matches the one provided in the configuration.
- Empty Message_Received output: For Get operations, confirm a message is available on the queue. Check `mq_debug.log` for API error codes.

- File not found errors: Verify the XML payload file path is correct and accessible by the aspweb user.
- MQ API message and reason codes: Refer to the IBM MQ API code references for detailed diagnostics.
- Port 8080 conflict: If another service is using port 8080, stop it before starting the MQ Bridge, or reconfigure the bridge to use a different port in `mq_bridge.py`.

Important Notes

- The MQ Bridge service must be running on the Orchestrator host before any MQ workflow step is executed.
- Use either Execution Node or MQ IP Address to specify the MQ server location — not both simultaneously.
- The default MQ port is 1414. Only change this if your MQ server is configured on a different port.
- SSL cipher values must use the Oracle JRE `TLS_` prefix — IBM `SSL_` prefixed values are not supported.
- The trust store file must be placed in the designated Orchestrator JRE security directory before the workflow runs.
- Keep Trigger Ongoing is only applicable to Get Message operations and will cause the plugin to continuously trigger on new messages until stopped.
- The MQ Put Request Body field now accepts a file path to an XML file containing the payload, not the payload itself.
- For IBM MQ server setup and advanced configuration, refer to the IBM MQ Knowledge Center.

5.24 Ibm mq mft transfer



Figure 5.24: Ibm mq mft transfer icon

This plugin is used to send a file via IBM MQ MFT.

Version	Comment
0.5.3	Enhanced action template UI
0.5.2	Updating to be compatible with Orchestrator 4.1, which isn't shipped with a Java App and Tomcat server
0.3.1	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.0	Remove hardcoded aspera directory reference
0.2.0	Added SSL support
0.1.2	help file adding tomcat part
0.1.1	status part addition
0.1.0	status part addition
0.0.1	Initial implementation

5.24.1 How It Works

- Integrates with IBM MQ Managed File Transfer (MFT) to submit file transfer requests via a configured MQ queue and MFT agent.
- Accepts the transfer specification as an XML payload file path defining source, destination, agent, and transfer options.
- Submits the transfer by placing the XML message onto the MFT agent's configured queue, which the agent monitors for incoming transfer requests.
- Monitors transfer status by reading from a configured subscription queue — transfer submission and status monitoring are performed in a single plugin call.
- Supports SSL/TLS encryption and authentication using a configurable cipher and Java trust store.
- Can target an MQ server via a configured remote Execution Node or directly via IP address and port.
- Uses a Python-based MQ Bridge service that wraps the IBM MQ Java SDK JAR file to communicate with IBM MQ servers.
- Returns the Message ID of the submitted transfer for downstream tracking and correlation.

5.24.2 Configuration Instructions

1. Create a new action step using the IbmMqMftTransfer plugin.
2. Fill out the following fields:
    - Name: The name used to identify a saved IbmMqMftTransfer configured instance.
    - Comments: Comments about this saved IbmMqMftTransfer configured instance.
    - Execution Node: Node on which the IBM MQ server is installed. Used when a remote node is configured for MQ.
    - MQ IP Address: IP address of the MQ server. Used when no remote Execution Node is configured.
    - MQ Port: Port the MQ server listens on. Default is 1414.
    - Use SSL?: Check this box to enable SSL/TLS encryption and authentication.
    - Cipher: SSL/TLS cipher suite to use (e.g., TLS_RSA_WITH_AES_128_CBC_SHA256). Must use the Oracle JRE TLS_ prefix.
    - Trust store filename: Filename of the Java application SSL/TLS trust store (e.g., user.jks). Must be placed in the Orchestrator JRE security directory.
    - Trust store password: Password for the trust store file (optional).
    - MQ Operation Type: The operation to perform. Currently supported: Submit Transfer.
    - MQ Channel Name: Name of the MQ channel to connect through.
    - MQ Queue Name: Name of the MQ queue configured with the MFT agent to receive transfer messages.
    - MQ Queue Manager Name: Name of the MQ Queue Manager.
    - MQ Put Request (XML file path): Path to the XML file containing the transfer specification. Must be valid XML conforming to the MFT transfer request schema.
    - MQ Agent Name: Name of the MFT agent to listen to for transfer status monitoring.
    - MQ Subscription Queue Name: Name of the subscription queue used to read transfer status updates.
    - MQ Subscription Name: Name of the MQ subscription configured for transfer status monitoring.
    - MQ Message Id: Message ID of the submitted transfer, used to retrieve the status of that specific transfer.
    - MQ User Name: Username for authenticating with the MQ Queue Manager.
    - MQ Password: Password for authenticating with the MQ Queue Manager.

5.24.3 Example Configuration

This is an example configuration for the IbmMqMftTransfer plugin submitting a file transfer request via IBM MQ MFT.

Parameter	Value
Name	Transfer Archive File via MFT
Comments	Submit MFT transfer request to move file to archive directory
Execution Node	(blank - using IP address instead)
MQ IP Address	192.168.1.50
MQ Port	1414
Use SSL?	Not checked
MQ Operation Type	Submit Transfer
MQ Channel Name	SYSTEM.DEF.SVRCONN
MQ Queue Name	SYSTEM.FTE.COMMAND.ASPERA_AGENT
MQ Queue Manager Name	Aspera_Demo
MQ Agent Name	ASPERA_AGENT
MQ Subscription Queue Name	TRANSFER.STATUS.QUEUE
MQ Subscription Name	TRANSFER.STATUS.SUB
MQ User Name	mquser

Parameter	Value
MQ Password	(stored securely)
MQ Put Request (XML file path)	/path/to/mft_transfer_spec.xml

5.24.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved IbmMqMftTransfer configured instance.
Comments	Comments about this saved IbmMqMftTransfer configured instance.
Execution Node	Node on which the IBM MQ server is installed. Used when a remote node is configured for MQ.
MQ IP Address	IP address of the MQ server. Used when no remote Execution Node is configured.
MQ Port	Port the MQ server listens on. Default is 1414.
Use SSL?	Check this box to enable SSL/TLS encryption and authentication.
Cipher	SSL/TLS cipher suite to use (e.g., TLS_RSA_WITH_AES_128_CBC_SHA256). Must use the Oracle JRE TLS_ prefix.
Trust store filename	Filename of the Java application SSL/TLS trust store (e.g., user.jks). Must be placed in the Orchestrator JRE security directory.
Trust store password	Password for the trust store file (optional).
MQ Operation Type	The operation to perform. Currently supported: Submit Transfer.
MQ Channel Name	Name of the MQ channel to connect through.
MQ Queue Name	Name of the MQ queue configured with the MFT agent to receive transfer messages.
MQ Queue Manager Name	Name of the MQ Queue Manager.
MQ Put Request (XML file path)	Path to the XML file containing the transfer specification. Must be valid XML conforming to the MFT transfer request schema.
MQ Agent Name	Name of the MFT agent to listen to for transfer status monitoring.
MQ Subscription Queue Name	Name of the subscription queue used to read transfer status updates.
MQ Subscription Name	Name of the MQ subscription configured for transfer status monitoring.
MQ Message Id	Message ID of the submitted transfer, used to retrieve the status of that specific transfer.
MQ User Name	Username for authenticating with the MQ Queue Manager.
MQ Password	Password for authenticating with the MQ Queue Manager.

5.24.5 Inputs Description

Runtime inputs can override configured values. Queue Manager Name, Queue Name, Channel Name, Body (file path), and IP address are the core required parameters for submitting a transfer.

Parameter	Description
Queue Manager Name	Name of the MQ Queue Manager. Required for all operations.
Queue Name	Name of the MQ queue configured with the MFT agent. Required for transfer submission.
Channel Name	Name of the MQ channel to connect through. Required for all operations.
Body	Path to the XML file containing the transfer specification. Must be valid XML. Required for Submit Transfer operations.
Ip	IP address of the MQ server. Required when no Execution Node is configured.
Agent Name	Name of the MFT agent used for transfer status monitoring.
Subscription Name	Name of the MQ subscription configured for transfer status updates.
Subscription Queue Name	Name of the queue used to read transfer status messages.
MQ_cipher	SSL/TLS cipher suite. Required when SSL is enabled.
MQ_trust_store	Trust store filename. Required when SSL is enabled.
MQ_trust_store_word	Trust store password. Required when SSL is enabled and the trust store is password-protected.

5.24.6 Outputs Description

The output is the Message ID returned by MQ upon successful transfer submission. Use this ID to track or correlate the transfer in downstream workflow steps.

Parameter	Description
Message_Id	The Message ID returned by MQ upon successful submission of the transfer request. Use this ID to correlate and track the transfer status.

5.24.7 Supported Actions

None

5.24.8 Dependencies

Requires IBM MQ to be installed and configured with an MFT agent. Requires Python 3, Flask library, Java Runtime Environment, IBM MQ client libraries, and the MQ Bridge service running on port 8080. The MQ server must be accessible from the Orchestrator host.

5.24.9 Operating Instructions

This action submits a file transfer request to IBM MQ MFT by placing an XML payload (read from a file) onto the MFT agent's configured queue, then monitors the transfer status via a subscription queue — all within a single plugin call.

Transfer XML Payload

- The MQ Put Request must be a path to a valid XML file conforming to the MFT FileTransfer.xsd schema.
- The XML must define the originator, source agent, destination agent, and transfer set including source and destination file paths.
- The Queue Name must be the queue that the MFT agent is configured to monitor — the agent listens on this queue for incoming transfer requests.
- Transfer submission and status monitoring are performed in a single plugin call. Provide both the Queue Name and Subscription Name to enable status tracking.

Example Transfer XML

The following is a sample XML payload for submitting an MFT transfer request (save this to a file and provide the file path):

\ <?xml version="1.0" encoding="UTF-8"?>\ <request version="6.00" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"\ xsi:noNamespaceSchemaLocation="FileTransfer.xsd">\ <managedTransfer>\ <originator>\ <host-Name>127.0.0.1</hostName>\ <userID>mqm</userID>\ </originator>\ <sourceAgent QMgr="As-pera_Demo" agent="ASPERA_AGENT"/>\ <destinationAgent QMgr="Aspera_Demo" agent="ASPERA_AGENT"\ channel="SYSTEM.DEF.SVRCONN" portNumber="1414"/>\ <transferSet>\ <item checksumMethod="MD5" mode="bi-nary">\ <source disposition="leave" recursive="false" type="file">\ <file>/tmp/myfile.txt</file>\ </source>\ <destination exist="overwrite" type="directory">\ <file>/tmp/archive/</file>\ </destination>\ </item>\ </transferSet>\ </managedTransfer>\ </request>\

MQ Bridge Service Setup

- Important: If the MQ Bridge service was already set up as part of the IbmMqIntegration plugin, do not repeat this setup.
- The plugin requires a Python-based MQ Bridge service that wraps the IBM MQ Java SDK JAR file to communicate with IBM MQ servers.
- The MQ Bridge service is located at: `/opt/aspera/orchestrator/script/ibm_mq_plugin_scripts/mq_bridge.py`
- Prerequisites:
  - Python 3 must be installed on the system
  - Flask Python library: `pip3 install flask`
  - Java Runtime Environment (JRE) to run the IBM MQ JAR file
  - IBM MQ client libraries installed at `/opt/mqm/bin`
- Starting the MQ Bridge:
  - Navigate to the scripts directory: `cd /script/ibm_mq_plugin_scripts`
  - Start the bridge: `./start_mq_bridge.sh`
  - The script will automatically install Flask if not present and start the service in the background
- Stopping the MQ Bridge: `./stop_mq_bridge.sh`
- The MQ Bridge listens on port 8080 by default. Ensure no other service is using this port before starting.
- Service logs are located at: `/opt/aspera/orchestrator/log/mq_debug.log`
- Health check endpoint: `curl http://localhost:8080/health`
- MQ API message and reason codes reference: IBM MQ API Codes

SSL/TLS Configuration

- The cipher must use the Oracle JRE `TLS_` prefix (e.g., `TLS_RSA_WITH_AES_128_CBC_SHA256`), not the IBM `SSL_` prefix.
- The cipher must match the cipher defined on the MQ server side.
- The trust store file must be placed in: `/lib/java/jre/lib/security/`
- Supported trust store formats: JKS or PKCS. Specify the filename including its extension in the Trust store filename field.
- The trust store file must contain the full chain CA certificate from the MQ server side.
- Trust store files can be created with tools such as keytool, ikeycmd (bundled with Java), or runmqckm (bundled with IBM MQ).
- The trust store password is optional. Enter only if one was defined when the trust store was created.

Best Practices

- Store the MQ Password and trust store password securely using Orchestrator's credential management rather than plaintext configuration.
- Validate the XML transfer specification against the MFT FileTransfer.xsd schema before using it in a workflow to avoid runtime failures.
- Ensure the MQ Queue Name matches the queue the MFT agent is monitoring — using an incorrect queue will result in the transfer never being processed.
- Verify the MQ Bridge service is running on port 8080 before executing any MFT workflow steps. Check `mq_debug.log` for startup issues.
- Store the returned Message ID output for downstream correlation, auditing, or retry logic.
- For SSL environments, test the cipher and trust store configuration manually before integrating into a workflow to isolate connectivity issues.
- Implement error handling workflows to manage transfer failures or timeouts when waiting for status updates from the subscription queue.
- If this plugin is used alongside IbmMqIntegration, the MQ Bridge service setup only needs to be performed once.
- Ensure the XML file path is accessible by the aspweb when submitting transfer requests.

Troubleshooting

- MQ Bridge not running: Ensure the MQ Bridge service has been started using `./start_mq_bridge.sh` and is listening on port 8080. Check for port conflicts.
- Flask not installed: Install Flask using `pip3 install flask` or let the start script install it automatically.
- Transfer not initiated: Verify the Queue Name is the queue the MFT agent is configured to monitor. Confirm the MFT agent is running and connected to the Queue Manager.
- Invalid XML payload: Ensure the XML file contains well-formed XML and conforms to the MFT FileTransfer.xsd schema. Test the XML independently before use.
- File not found errors: Verify the XML file path is correct and accessible from the Orchestrator server.
- Authentication failure: Check the MQ User Name and Password. Ensure the user has the required permissions on the Queue Manager.
- Queue or channel not found: Verify the Queue Name, Channel Name, and Queue Manager Name are correctly specified and exist on the MQ server.
- Status not returned: Confirm the Subscription Name and Subscription Queue Name are correctly configured and the subscription is active on the MQ server.
- SSL handshake failure: Confirm the cipher uses the Oracle `TLS_` prefix and matches the MQ server cipher. Verify the trust store file is in the correct directory and contains the MQ server CA certificate.
- MQ API message and reason codes: Refer to the IBM MQ API code references for detailed diagnostics.
- Port 8080 conflict: If another service is using port 8080, stop it before starting the MQ Bridge, or reconfigure the bridge to use a different port in `mq_bridge.py`.

Important Notes

- The MQ Put Request Body field now accepts a file path to an XML file containing the transfer specification, not the XML content itself.
- The XML file must be valid XML — the plugin does not accept non-XML payloads for transfer specifications.
- The Queue Name must be the queue the MFT agent is configured to monitor for transfer requests, not a general-purpose queue.
- Transfer submission and status monitoring are a single call — provide both the Queue Name and Subscription Name to enable end-to-end tracking.
- The MQ Bridge service must be running on the Orchestrator host before any MFT workflow step is executed.
- If the MQ Bridge service setup was already completed for the IbmMqIntegration plugin, do not repeat the setup.
- SSL cipher values must use the Oracle JRE `TLS_` prefix — IBM `SSL_` prefixed values are not supported.
- The trust store file must be placed in the designated Orchestrator JRE security directory before the workflow runs.
- Use either Execution Node or MQ IP Address to specify the MQ server location — not both simultaneously.

5.25 InfluxDB



Figure 5.25: InfluxDB icon

This action plugin provides the ability to interact with InfluxDB

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.3	Fixes authentication issue

Version	Comment
0.0.2	Adding compatibility for Ruby 1.8.7.
0.0.1	Initial release.

5.25.1 How It Works

- Connects to InfluxDB time-series database to perform query and write operations.
- Supports authentication with username and password for secure database access.
- Executes queries with configurable precision and chunking for optimal performance.
- Writes data points with support for measurements, tags, fields, and timestamps.
- Returns responses in both raw format and structured hash format for flexible data processing.

5.25.2 Configuration Instructions

1. Add an InfluxdbOperation step to your workflow.
2. Configure the database connection parameters:
   - Name: Enter a descriptive name for the InfluxDB operation.
   - Comments: Add notes or description for this step.
   - Name of the database: The name of the database that is to be queried or written to.
   - URL: The URL to the instance of your InfluxDB. Include the port here (e.g., http://localhost:8086).
   - User name: The user name of the account being used to access the database. (Required if authentication is enabled)
   - Password: The password for the account being used to access the database. (Required if authentication is enabled)
3. Select the operation type:
   - Operation: Choose between Query or Write operation.
4. For Query operations, configure:
   - Query: The query to be executed on the database. Multiple commands can be delimited with semicolons.
   - Chunk size: Optional integer that determines how InfluxDB will chunk its responses. If left blank, responses will not be chunked.
   - Precision: Determines with how much precision the epoch timestamps will be returned (e.g., ns, u, ms, s, m, h).
5. For Write operations, configure:
   - Data: The data to be written to the database. Format: <measurement>,<tag_key>=<tag_value> <field_key>=<field_value> <timestamp>
   - Consistency: Determines the write consistency for the point. Only used for InfluxDB Enterprise clusters.
   - Retention Policy: Optional parameter that sets the target retention policy for the write. If blank, InfluxDB writes to the DEFAULT retention policy.
   - Precision: Determines the precision for the supplied Unix time values (e.g., ns, u, ms, s, m, h).

5.25.3 Example Configuration - Query Operation

Parameter	Example Value
Name	Get Temperature Metrics
Comments	Retrieves Temperature metrics from the DB
Name of the database	system_metrics
URL	http://influxdb-server:8086
User name	admin
Password	********
Operation	Query
Query	SELECT * FROM temperature
Chunk size	10000
Precision	ms

5.25.4 Example Configuration - Write Operation

Parameter	Example Value
Name	Write Temperature Data
Comments	Writes temperature readings to the DB
Name of the database	system_metrics
URL	http://influxdb-server:8086
User name	admin
Password	********
Operation	Write
Data	temperature,location=office value=25.3
Consistency	one
Retention Policy	Blank
Precision	ns

5.25.5 Saved Parameters Description

Parameter	Description
Name	Display name for this InfluxDB operation instance.
Comments	Notes or description for this step.
Name of the database	The name of the database that is to be queried or written to.
URL	The URL to the instance of your InfluxDB. Include the port here (e.g., http://localhost:8086).
User name	The user name of the account being used to access the database. (Required if authentication is enabled)
Password	The password for the account being used to access the database. (Required if authentication is enabled)
Operation	The operation to be performed. Available operations are Query and Write.
Query	(Query only) The query to be executed on the database. Multiple commands can be delimited with semicolons.
Chunk size	(Query only) Integer that determines how InfluxDB will chunk its responses. Optional - if left blank, responses will not be chunked.
Precision	(Query/Write) Determines the precision for epoch timestamps (Query) or Unix time values (Write). Values: ns, u, ms, s, m, h.
Data	(Write only) The data to be written to the database. Format: <measurement>,<tag_key>=<tag_value> <field_key>=<field_value> <timestamp>. Multiple points separated by newlines.
Consistency	(Write only) Determines the write consistency for the point. Only used for InfluxDB Enterprise clusters.
Retention Policy	(Write only) Optional parameter that sets the target retention policy for the write. If blank, writes to DEFAULT retention policy.

5.25.6 Inputs Description

Changes in this list of inputs will override any values set in the action template.

- database_name: The name of the database to query or write to.
- url: The URL to the InfluxDB instance including port.
- username: Username for database authentication.
- password: Password for database authentication.
- operation: The operation to perform (Query or Write).
- query: (Query only) The query to execute on the database.
- chunk_size: (Query only) How InfluxDB will chunk responses.
- data: (Write only) The data points to write to the database.
- consistency: (Write only) Write consistency level for Enterprise clusters.
- retention_policy: (Write only) Target retention policy for the write.
- precision: Timestamp precision (ns, u, ms, s, m, h).

5.25.7 Outputs Description

- Response_Code: The HTTP status code received from InfluxDB. 200 = successful query, 204 = successful write, 400 = bad request (syntax error), 401 = authorization failed (invalid credentials).
- Raw_Output: (Query only) The raw output as returned by InfluxDB.

- Hash_Output: (Query only) The response from InfluxDB in a hash/structured format.

5.25.8 Supported Actions

None

5.25.9 Dependencies

- Access to an InfluxDB instance (version 1.x) with network connectivity.
- Valid database credentials if authentication is enabled on the InfluxDB instance.
- Appropriate user permissions (read access for queries, write access for writes).

5.25.10 Operating Instructions

5.25.11 Overview

The InfluxdbOperation plugin provides integration with InfluxDB time-series databases, allowing workflows to query data and write data points. The plugin supports both authenticated and non-authenticated connections and provides flexible options for data precision and chunking.

5.25.12 Database Connection Setup

- URL format: Include the protocol and port in the URL (e.g., http://influxdb-server:8086 or https://influxdb-server:8086 for SSL).
- Authentication: If your InfluxDB instance has authentication enabled, provide both username and password. These parameters are optional for instances without authentication.
- Database name: Specify the exact name of the database you want to query or write to. The database must exist before performing operations.
- User permissions: Ensure the user account has appropriate permissions - read access for queries, write access for writes.

5.25.13 Query Operations

5.25.14 Query Syntax

- Use InfluxQL syntax for queries (SELECT statements, WHERE clauses, GROUP BY, etc.).
- Multiple queries can be executed in a single operation by separating them with semicolons.
- Common query patterns: `SELECT * FROM "measurement" WHERE time > now() - 1h`
- Use aggregation functions: `SELECT mean("field") FROM "measurement" GROUP BY time(5m)`

5.25.15 Chunking and Precision

- Chunk size: Controls how InfluxDB streams large result sets. Specify an integer to chunk by number of points, or leave blank for no chunking. Useful for large datasets to manage memory usage.
- Precision options: Controls timestamp format in query results:
- - `ns`: Nanoseconds (default)
- - `u`: Microseconds
- - `ms`: Milliseconds
- - `s`: Seconds
- - `m`: Minutes
- - `h`: Hours

5.25.16 Write Operations

5.25.17 Data Format

Data must be formatted in InfluxDB line protocol:

`<measurement>[,<tag_key>=<tag_value>...] <field_key>=<field_value>[,<field_key>=<field_value>...] [<timestamp>]`

- Measurement (required): The name of the measurement (like a table name).
- Tags (optional): Key-value pairs for indexed metadata. Format: `tag1=value1,tag2=value2`
- Fields (required): At least one field key-value pair. Format: `field1=value1,field2=value2`
- Timestamp (optional): Unix timestamp. If omitted, InfluxDB uses server time. Precision specified by the Precision parameter.
- Multiple points: Separate each point with a newline character to write multiple data points in a single request.

5.25.18 Write Examples

- Basic write: `temperature,location=room1 value=22.5`
- With timestamp: `temperature,location=room1 value=22.5 1609459200000000000`
- Multiple tags and fields: `cpu,host=server01,region=us-west usage_idle=20,usage_system=10 1609459200`
- Multiple points (separated by newline):

  `temperature,location=room1 value=22.5`

  temperature,location=room2 value=23.1

5.25.19 Write Options

- Retention Policy: Determines how long InfluxDB keeps the data. If blank, InfluxDB uses the DEFAULT retention policy.
- Consistency: Only applies to InfluxDB Enterprise clusters. Options: one, quorum, all, any. Determines how many data nodes must confirm the write.
- Precision: Specifies the format of the timestamp in your data. Must match the precision of your supplied timestamps.

5.25.20 Response Codes

- 200 OK: Query operation completed successfully.
- 204 No Content: Write operation completed successfully.
- 400 Bad Request: Syntax error in query or malformed data in write operation. Check the query/data format.
- 401 Unauthorized: Authentication failed. Verify username and password are correct.
- 404 Not Found: Database does not exist. Verify the database name.
- 500 Internal Server Error: InfluxDB encountered an error. Check InfluxDB logs for details.

5.25.21 Output Processing

- Response_Code: Always check this output first to verify the operation succeeded (200 for queries, 204 for writes).
- Raw_Output: Contains the complete JSON response from InfluxDB for queries. Use this when you need the exact response format.
- Hash_Output: Provides a structured Ruby hash format of the query response, making it easier to parse and process results in subsequent workflow steps.
- Error handling: For non-success response codes, the raw output will contain error details from InfluxDB.

5.25.22 Best Practices

- Always validate Response_Code output before processing query results or assuming write success.
- Use appropriate precision settings to balance between timestamp accuracy and performance.
- For large queries, implement chunking to manage memory usage and improve response times.
- Include error handling in your workflows to manage authentication failures and syntax errors.
- Use tags for indexed metadata that you'll filter on, and fields for actual measured values.
- When writing multiple points, batch them in a single write operation for better performance.
- Test queries and data formats using the InfluxDB CLI or web interface before implementing in workflows.
- Consider setting up retention policies appropriate for your data storage requirements.
- For production environments, use authentication and secure connections (HTTPS).

- Monitor the InfluxDB instance performance when implementing high-frequency write operations.

5.26 JIRA Ticket Creation



Figure 5.26: JIRA Ticket Creation icon

This action plug-in provides the ability to create JIRA tickets.

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	Add component and ssl vefification
0.0.2	Changing plugin name.
0.0.1	Initial release.

5.26.1 Saved Parameters Description

- Name: The name used to identify a saved JiraCreate configured instance.
- Comments: Some comments about this saved JiraCreate configured instance.
- Title: Title of the ticket to be created.
- Description: Description of the issue to be created
- Jira url: Your company's JIRA url.
- Username: Your JIRA username.
- Password: Your JIRA password.
- Ssl verification: To verify the validity of the SSL certificate presented by the Jira server.
- Ticket Type: Type of ticket describing the issue.
- Priority: Priority of the issue being created.
- Assignee: This parameter is optional. If left blank the default assignee will be used.
- Project key: Project key that the issue should be filed under.
- Component: Component name. Optional.

5.26.2 Inputs description

The list of inputs depends on the configuration of the JiraCreate action template.

5.26.3 Outputs description

- *Ticket_Id*: ID of the created ticket.
- *Create_Ticket_Response*: Response of the ticket creation such as: { ”id” => ”275075”, ”key” => ”VODT-3688”, ”self” => ”https://jira.xxx.net/rest/api/2/issue/275075” }
- *Ticket_JSON_Info*: URL of the created ticket such as: https://jira.xxx.net/rest/api/2/issue/275075
- *Error_Message*: Error message returned by the Jira server

5.26.4 Supported Actions

None

5.26.5 Dependencies

None

5.26.6 Operating Instructions

The JIRA URL must be like https://jira.xxx.net

Don't add a forward slash at the end of the URL.

HTTP is not supported.

Caution: you must enter the project key (e.g. VODT), not the project id (e.g. 11852).

'400 Bad Request' is returned if the project key value is unknown.

'400 Bad Request' is returned if the component is required by the Jira server and the value is unknown.

The ticket type and priority can be set in input of the plugin to enter a custom value (e.g. type 'Task' instead of the pre-defined 'Testing Task').

5.27 Kafka stream



Figure 5.27: Kafka stream icon

This plugin is used to send/receive messages to/from an Apache Kafka system.

Version	Comment
0.1.3	Enhanced action template UI
0.1.2	Updated the label from Keep ongoing to Continue Monitoring
0.1.1	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	Remove hardcoded aspera directory reference
0.0.2	Update path to ruby
0.0.1	Initial implementation

5.27.1 How It Works

- Provides integration with Apache Kafka for message streaming operations.
- Supports PUT (write/produce) and GET (read/consume) operations.
- Enables publishing messages to Kafka topics from Orchestrator workflows.
- Enables consuming messages from Kafka topics to trigger or feed workflow processing.
- Supports pattern matching with regular expressions to filter consumed messages.
- Can operate in trigger mode to continuously monitor topics for new messages.
- Supports consumer groups for parallel message processing across multiple consumers.
- Provides partition control for targeted message placement and retrieval.
- Supports message keys for log compaction and partition assignment.
- Optional SSL/TLS encryption for secure message transmission.
- Returns message body, key, offset, and partition information.

5.27.2 Configuration Instructions

1. Create a new action step using the KafkaStream plugin.
2. Fill out the following fields:
   - Name: Name of the Action plugin in the workflow.
   - Comments: Comments given to the step.
   - Execution node: Remote Node addressing the Kafka system. Specify the Kafka port in the TCP port field, not in the SSH port field, within the Remote node configuration.
   - Remote address: If no selected remote node, enter here the Kafka system IP address or domain name.
   - Remote port: If no selected remote node, enter the Kafka system port. Default is 9092.
   - Operation Type: PUT (write to) and GET (read from) operations are supported.
   - Topic: Topic to subscribe to. Can be a comma-separated list of topics.
   - Group id: Consumer group identifier. GET operation only. Used for parallel processing and checkpointing.
   - Pattern matching: Regular expression for pattern matching on the message body. GET operation only.
   - Read from beginning?: Read from the first message in the topic instead of reading only new messages. GET operation only. Applies only for new group IDs.
   - Return all?: Return all matching messages as a list. If not selected, return a single matching message. GET operation only.
   - Continue Monitoring: Keep triggering on new messages for continuous monitoring. GET operation only.
   - Polling frequency: Polling period in seconds. Default is 10 seconds. GET operation only.
   - Partition id: Partition number for targeted read/write operations.
   - Partition key: Partition key ensuring messages with the same key go to the same partition. PUT operation only.
   - Message key: Message key stored alongside message value. Used for log compaction and state restoration.
   - Sync producer?: Set synchronous (blocking) or asynchronous (non-blocking) producer. PUT operation only.
   - Message body: The message payload to send. PUT operation only.
   - Use SSL?: Enable SSL/TLS message encryption.
   - Certificate filepath: File path on Orchestrator of your SSL/TLS certificate. Must be in X509 format (e.g., ca_cert.pem), not JKS format.

5.27.3 Example Configuration

This is an example configuration for the KafkaStream plugin consuming messages from Kafka topics.

Parameter	Value
Name	Consume Order Messages
Execution node	KafkaCluster01
Operation Type	GET
Topic	orders, returns
Group id	orchestrator_processor
Pattern matching	”status”:”pending”
Read from beginning?	Not selected
Return all?	Selected
Continue Monitoring	Selected
Polling frequency	10

5.27.4 Saved Parameters Description

Parameter	Description
Name	Name of the Action plugin in the workflow.
Comments	Comments given to the step.
Execution node	Remote Node addressing the Kafka system. Specify the Kafka port in the TCP port field, not in the SSH port field, within the Remote node configuration.
Remote address	If no selected remote node, enter here the Kafka system IP address or domain name.
Remote port	If no selected remote node, enter the Kafka system port. Default is 9092.
Operation Type	PUT (write to) and GET (read from) operations are supported.

Parameter	Description
Topic	Topic to subscribe to. Can be a comma-separated list of topics.
Group id	Consumer group identifier. GET operation only. Used for parallel processing and checkpointing.
Pattern matching	Regular expression for pattern matching on the message body. GET operation only.
Read from beginning?	Read from the first message in the topic instead of reading only new messages. GET operation only. Applies only for new group IDs.
Return all?	Return all matching messages as a list. If not selected, return a single matching message. GET operation only.
Continue Monitoring	Keep triggering on new messages for continuous monitoring. GET operation only.
Polling frequency	Polling period in seconds. Default is 10 seconds. GET operation only.
Partition id	Partition number for targeted read/write operations.
Partition key	Partition key ensuring messages with the same key go to the same partition. PUT operation only.
Message key	Message key stored alongside message value. Used for log compaction and state restoration.
Sync producer?	Set synchronous (blocking) or asynchronous (non-blocking) producer. PUT operation only.
Message body	The message payload to send. PUT operation only.
Use SSL?	Enable SSL/TLS message encryption.
Certificate filepath	File path on Orchestrator of your SSL/TLS certificate. Must be in X509 format (e.g., ca_cert.pem), not JKS format.

5.27.5 Inputs Description

Parameter	Description
Remote_address	Kafka remote system IP address or domain name (e.g., kafka.example.com or 192.168.1.100).
Remote_port	Kafka remote system port. Default is 9092.
Topic	Topic to subscribe to. Can be a comma-separated list (e.g., 'topic1, topic2, topic3').
Group_id	Consumer group identifier. GET operation only. Required for GET operations.
Pattern	Regular expression for message body pattern matching. GET operation only. Optional.
Message_body	Message payload to send. PUT operation only. Required for PUT operations.
Partition_id	Partition number for targeted operations. Optional.
Partition_key	Partition key for consistent partition assignment. PUT operation only. Optional.
Message_key	Message key stored with message value. Optional.
Certificate_path	File path on Orchestrator of SSL/TLS certificate in X509 format. Required if Use SSL? is selected.

5.27.6 Outputs Description

If 'Return all?' is selected, each of the parameters below will be an array, with one element per matching message.

Parameter	Description
Message_Body	The sent or received message body (payload). If 'Return all?' is selected, returns an array of message bodies.
Message_Key	The received message key. GET operation only. If 'Return all?' is selected, returns an array of message keys.
Message_Offset	The received message offset within the partition. GET operation only. If 'Return all?' is selected, returns an array of offsets.
Partition_ID	The received message partition ID. GET operation only. If 'Return all?' is selected, returns an array of partition IDs.

5.27.7 Supported Actions

None

5.27.8 Dependencies

None

5.27.9 Operating Instructions

This plugin provides integration with Apache Kafka for message streaming. Both PUT (produce) and GET (consume) operations are supported.

PUT Operation (Produce Messages)

- Purpose: Produce/write messages to Kafka topics.
- Topics:
  - Specify one or more topics as comma-separated list
  - Example: `orders, notifications, audit_log`
  - Spaces before/after topic names are automatically removed
- Message Body: The payload to send. Can be plain text, JSON, XML, or any string format.
- Partition ID:
  - Optional: Write message to a specific partition identified by numeric ID
  - If not specified, Kafka uses partition key or default partitioner
- Partition Key:
  - Optional: Alphanumeric key ensuring messages with same key go to same partition
  - Useful for maintaining message order for related events
  - Example: Use customer_id as partition key to keep all customer messages in same partition
- Message Key:
  - Optional: Stored alongside message value
  - Used for log compaction to restore state after crashes
  - Different from partition key - partition key determines partition, message key is metadata
- Synchronous vs Asynchronous:
  - Synchronous (Sync producer? is selected): Blocking - waits for message delivery confirmation. Guarantees delivery but slower.
  - Asynchronous (Sync producer? is not selected): Non-blocking - returns immediately. Better performance but no delivery guarantee.

GET Operation (Consume Messages)

- Purpose: Consume/read messages from Kafka topics.
- Topics: Specify one or more topics as comma-separated list.
- Group ID (Required):
  - Used for consumer parallelism and checkpointing
  - Messages published to a topic are delivered to only one consumer instance within a subscribing consumer group
  - Consumer instances in different groups receive the same messages if subscribed to same topic
  - Example: Use group_id='processor_A' and group_id='processor_B' to have both process all messages
- Partition ID:
  - Optional: Return only messages from specific partition
  - Useful for targeted consumption from known partition
- Message Key:
  - Optional: Return only messages with this specific key
  - Filters messages based on message key metadata
- Pattern Matching:
  - Optional: Regular expression matched against message body
  - Returns only messages where body matches the pattern
  - Example: `"status":"pending"` matches JSON messages with pending status
- Read from Beginning:
  - Returns all messages in topic starting from first message
  - Important: Only applies when starting plugin with a new group ID
  - Kafka checkpointing preserves position - subsequent runs return only new messages
  - Checkpointing enables resuming after consumer crashes
- Return All:
  - If selected: Returns all matching messages as arrays
  - If not selected: Returns single matching message
  - When enabled, outputs become arrays (Message_Body[], Message_Key[], etc.)

- Continue Monitoring:
  - Continuously polls for new messages at specified polling frequency
  - Creates new workflow instance for each batch of matching messages
  - WARNING: Do not run 2 instances with same topic + group simultaneously

Consumer Groups and Checkpointing

- Purpose: Enable parallel message processing and fault tolerance.
- Single Group, Multiple Consumers:
  - Messages distributed across consumers in the same group
  - Each message delivered to only one consumer instance
  - Enables parallel processing of high-volume topics
  - Example: 3 consumers in group 'processors' share message load
- Multiple Groups:
  - Each group receives all messages from subscribed topics
  - Enables multiple independent processing pipelines
  - Example: group 'analytics' and group 'storage' both receive all messages
- Checkpointing:
  - Kafka tracks each consumer's position (offset) within each partition
  - Enables resuming from last processed message after crash/restart
  - Automatic - no manual intervention required
  - Persists across consumer restarts with same group ID

Important Limitations

- Maximum message size: 1MB by default (configurable on Kafka cluster).
- Message deletion: Controlled by Kafka based on retention policy (age or size), not by consumer reads.
- Concurrent triggers warning: Do NOT run 2 'Continue Monitoring' instances with same topic + group simultaneously.
- Certificate format: Must use X509 format (e.g., .pem), not JKS format.

Common Use Cases

- Event-driven workflows: Trigger workflows when events published to Kafka topics (orders, notifications, alerts).
- Data pipeline integration: Consume messages from data streams for processing, transformation, or storage.
- Microservices communication: Publish workflow results to Kafka for other services to consume.
- Audit logging: Write workflow events and actions to Kafka audit topics.
- Real-time analytics: Feed Kafka messages into analytics workflows for real-time processing.
- Multi-system coordination: Use Kafka as message bus for coordinating across multiple systems.
- Retry/DLQ patterns: Publish failed processing results to dead-letter queue topics.

Best Practices

- Use consumer groups for parallel processing and fault tolerance.
- Set unique group IDs for different processing pipelines to ensure all receive messages.
- Use partition keys when message order matters for related events.
- Use synchronous producers for critical messages requiring delivery guarantees.
- Use asynchronous producers for high-throughput scenarios where occasional loss is acceptable.
- Set appropriate polling frequency - balance between latency and overhead.
- Avoid running multiple 'Continue Monitoring' instances with same topic + group.
- Keep messages under 1MB - use external storage for large payloads and reference in message.
- Use pattern matching to filter messages and reduce processing overhead.
- Monitor consumer lag on Kafka cluster to ensure workflows keep up with message volume.

Troubleshooting

- Connection timeout: Verify Kafka broker is reachable, port 9092 is accessible, and firewall allows connections.
- Authentication failed: Verify Kafka cluster authentication settings match plugin configuration.
- Topic not found: Verify topic exists on Kafka cluster and has been created.
- No messages consumed: Check if messages exist in topic, group ID is correct, and offsets haven't passed available messages.
- 'Timed out while waiting for response' error: Multiple 'Continue Monitoring' instances with same topic + group running. Stop all instances and restart one with different group ID.
- SSL connection failure: Verify certificate path is correct and certificate is in X509 format.
- Message too large: Reduce message size below 1MB or increase Kafka cluster's max message size configuration.
- Pattern not matching: Test regex pattern separately, remember pattern matches against message body only.

Important Notes

- The Kafka port must be specified in TCP port field, not SSH port field, in Remote Node configuration.
- Default Kafka port is 9092 (broker port), not 2181 (Zookeeper port).
- Topics can be comma-separated lists. Spaces before/after names are automatically removed.
- 'Read from beginning' only applies for new group IDs. Existing groups resume from checkpointed offset.
- When 'Return all?' is enabled, all outputs become arrays (Message_Body[], Message_Key[], etc.).
- Do not run multiple 'Continue Monitoring' instances with same topic + group simultaneously.
- Messages are not deleted after consumption. Deletion controlled by Kafka retention policy.
- Maximum message size is typically 1MB (configurable on Kafka cluster).
- Partition key determines partition assignment. Message key is separate metadata for log compaction.
- Synchronous producers guarantee delivery but have lower throughput than asynchronous.
- SSL/TLS requires X509 certificate format (.pem), not JKS format.

5.28 Local execution



Figure 5.28: Local execution icon

This action plug-in provides the ability to invoke a third-party application or a script running locally on the workflow server. The integration mechanism conforms to the CGI-BIN protocol used by web and application servers (i.e. input parameters are provided as environment variable, output is provided on the stdio).

Version	Comment
0.4.1	Enhanced action template UI
0.4.0	Removing Windows support code, Updating to be compatible with Ruby 3.4 and Rails 8
0.3.4	Removed explicit html escaping since not needed with new Ruby
0.3.3	Added styling to parameter help text.
0.3.2	Allow user to specify their own status code when error/failure occurs
0.3.1	Display output from progress read at the end of process execution
0.3.0	Add escaping for single quotes
0.2.9	Fix UI glitch preventing display of empty commands
0.2.8	Restores compatibility with version of Orchestrator pre 2.2.1
0.2.7	New icon, fixes executor script in Linux to support parameters with quote, fixes configurator to allow quotes
0.2.6	Changes the execution to be made from the /opt/aspera/scripts (EXECUTABLE_DIRECTORY)
0.2.5	Bad - upgrade to 0.2.6
0.2.4	Fixes mis-matched between LOAD and ROOT of actions
0.2.3	Minor clean-up with no functionality changes
0.2.2	Moves script directory to /avoid install over-write/opt/aspera/scripts to avoid upgrade overwrite
0.2.1	Adds clean-up of temporary files from var run
0.2.0	Adds parallel execution mode, configurable outputs. DB_stored_scripts field was added to DB but is unused.

5.28.1 How It Works

- This action plug-in executes a third-party application or script locally on the workflow server.
- The integration mechanism follows a CGI-style protocol commonly used by web and application servers, where input parameters are provided as environment variables and output is returned via standard input/output (stdio).

5.28.2 Configuration Instructions

1. Create a new action step using the LocalExecution plug-in.
2. This action executes a local script or executable and captures its outputs.
3. Fill out the following configuration fields:
    - Name: The name used to identify a saved LocalExecution configuration.
    - Comments: Optional comments about this LocalExecution configuration.
    - Command: The command line to execute. This string may include variables that are replaced with their runtime values during execution. The command is constructed by appending the selected executable to the command arguments defined in the configuration interface. The executable must be located in the /tmp directory.
    - Multiline: When selected, the standard output (stdio) is returned as an array in the Multi output variable instead of a single string in the stdio output variable.
    - Mandatory Inputs: Defines required inputs that are passed to the script or executable as environment variables.
    - Optional Inputs: Defines optional inputs that are passed to the script or executable as environment variables.
    - Parallel Execution: When selected, the script or executable can report intermediate status information while it is running. The status reporting API is described in the Operating Instructions section.
    - Formatted Outputs: Defines the list of typed outputs expected from the script or executable. The output extraction API is described in the Operating Instructions section.

5.28.3 Example Configuration

This is a minimal setup configuration for the LocalExecution plug-in.

Parameter	Value
Command	./process_files.sh
Multiline	Selected
Mandatory Inputs	file_path, destination_dir
Optional Inputs	log_level
Parallel Execution	Selected
Formatted Outputs	Result, StatusMessage

5.28.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved LocalExecution configuration.
Comments	Optional comments about this LocalExecution configuration.
Command	The command line to execute. This string may include variables that are replaced with their runtime values during execution. The command is constructed by appending the selected executable to the command arguments defined in the configuration interface. The executable must be located in the /tmp directory.
Multiline	When selected, the standard output (stdio) is returned as an array in the Multi output variable instead of a single string in the stdio output variable.
Mandatory Inputs	Defines required inputs that are passed to the script or executable as environment variables.
Optional Inputs	Defines optional inputs that are passed to the script or executable as environment variables.
Parallel Execution	When selected, the script or executable can report intermediate status information while it is running. The status reporting API is described in the Operating Instructions section.
Formatted Outputs	Defines the list of typed outputs expected from the script or executable. The output extraction API is described in the Operating Instructions section.

5.28.5 Inputs Description

Required inputs depend on how the LocalExecution template is configured. If dynamic attributes are used, each attribute becomes a runtime input.

Parameter	Description
*Command line inputs **	All variables referenced in the command line are mandatory and must be of type String.
Environment variable inputs	Variables listed in the Mandatory or Optional Inputs fields may be typed, but are converted to strings before being passed as environment variables.

5.28.6 Outputs Description

Parameter	Description
Multi	Returned only when Multiline is selected. Contains stdio output split into an array, with one entry per line.
stdio	Returned only when Multiline is not selected. Contains the full stdio output as a single string.
Formatted Outputs	If configured, contains the values extracted from stdio based on the definitions in the Formatted Outputs configuration field.

5.28.7 Supported Actions

None

5.28.8 Dependencies

None

5.28.9 Operating Instructions

1. This action executes the configured command line. The executable or script receives some inputs as command-line arguments and others as environment variables. The command-line string may include static text as well as variables that are expanded at runtime. Variables are indicated using the syntax <%=variable_name%> embedded within the command string.

   For example: ”file_scan.sh <%=file_name%> -s <%=icap_server_name%>”. The executable or script must be located in `/tmp`, be accessible to the `aspweb` user, and have the appropriate permissions to execute. Additionally, scripts must include a valid shebang line.

2. When Parallel Execution is enabled, the executable can report status updates using the following format:

   '-#STATUS--#--STATUS MESSAGE#-'

   Either STATUS or STATUS MESSAGE may be omitted.

   For example: '-#Error--#--Action was stopped#-' updates both the status and message, while '-#--#--File tester.txt was scanned successfully#-' updates only the status message.

3. If outputs are defined in the Formatted Outputs field, values are extracted from stdio when they follow this format:

   '-#OUTPUT:VARIABLE_NAME--#--VARIABLE_VALUE#-'

   For example: '-#OUTPUT:size--#--120568#-' populates @outputs['size'] = 120568 when an output of type TYPE_INT is defined.

4. Note: Stdio lines containing formatted output or status markers are removed from the stdio or multiline output when Formatted Outputs are configured.

5.29 Media Mate Operation

This action plugin can be used to interact with the MediaMate subtitling service.

Version	Comment
0.3.2	Enhanced action template UI
0.3.1	Fix error handling & cleanup code

Version	Comment
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.5	Fix issue with submit operation timing
0.2.4	Make Submit call Open rather than Run
0.2.3	Correct error handling
0.2.2	Use model_path for Run and Submit
0.2.1	Correct synchronousness of plugin operations
0.2.0	Provide better error handling
0.1.0	Adding Submit operation and configuration for Open/Run
0.0.1	Initial release

5.29.1 How It Works

- Integrates with a MediaMate server via REST API calls to control MediaMate workflow model execution.
- Supports five operations: Open, Run, Stop, Close, and Submit, covering the full lifecycle of a MediaMate workflow model.
- Constructs REST call URLs dynamically from the configured server, port, and URL path, automatically appending the operation-specific path segment.
- The Submit operation combines Open, Run, Stop, and Close into a single plugin call, running the model to completion before returning.
- The Run and Submit operations poll the MediaMate server periodically for job status at a configurable polling frequency.
- The Open, Stop, and Close operations execute synchronously and return immediately upon completion.
- Accepts an XML configuration (XMLCFG) for Open and Submit operations, defining component settings as in MediaMate FLOXE.
- Returns the full MediaMate response, job state, FLX player state, and any error details for downstream workflow processing.

5.29.2 Configuration Instructions

1. Create a new action step using the MediaMateOperation plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved MediaMateOperation configured instance.
   - Comments: Comments about this saved MediaMateOperation configured instance.
   - Server: The server on which MediaMate is running. Used to build the REST call URL in the format: `http://<server>:<port>/<url_path>/`.
   - Port: The port on which MediaMate is running. Default is 8080. Used to build the REST call URL.
   - URL path: The path component of the REST call URL. The operation segment (e.g. open, run, stop, close) is appended automatically based on the Operation parameter. Use a short name in the URL path to distinguish concurrent job requests (e.g. `mediamate/mm01/interface`).
   - Operation: The operation to perform. Supported values: Open, Run, Stop, Close, Submit.
   - Xmlcfg: The XML configuration for Open and Submit operations, as used in MediaMate FLOXE. Must follow the structure: `<XMLCFG><ComponentList>...</ComponentList></XMLCFG>`.
   - Model path: The FLX file path of the MediaMate workflow model. Required for Open and Submit operations.
   - Polling frequency: The number of seconds Orchestrator waits between polls for job status. Used for Run and Submit operations. Default is 5 seconds.

5.29.3 Example Configuration

This is an example configuration for the MediaMateOperation plugin executing a complete MediaMate workflow model using the Submit operation.

Parameter	Value
Name	Submit MediaMate Workflow
Comments	Run a complete MediaMate workflow model via Submit operation
Server	mediamate-server.example.com
Port	8080
URL path	mediamate/mm01/interface

Parameter	Value
Operation	Submit
Xmlcfg	...
Model path	/data/models/workflow.flx
Polling frequency	5

5.29.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved MediaMateOperation configured instance.
Comments	Comments about this saved MediaMateOperation configured instance.
Server	The server on which MediaMate is running. Used to build the REST call URL in the format: `http://<server>:<port>/<url_path>/`.
Port	The port on which MediaMate is running. Default is 8080. Used to build the REST call URL.
URL path	The path component of the REST call URL. The operation segment (e.g. open, run, stop, close) is appended automatically based on the Operation parameter. Use a short name in the URL path to distinguish concurrent job requests (e.g. `mediamate/mm01/interface`).
Operation	The operation to perform. Supported values: Open, Run, Stop, Close, Submit.
Xmlcfg	The XML configuration for Open and Submit operations, as used in MediaMate FLOXE. Must follow the structure: `<XMLCFG><ComponentList>...</ComponentList></XMLCFG>`.
Model path	The FLX file path of the MediaMate workflow model. Required for Open and Submit operations.
Polling frequency	The number of seconds Orchestrator waits between polls for job status. Used for Run and Submit operations. Default is 5 seconds.

5.29.5 Inputs Description

All parameters are available as runtime inputs. Runtime values override any values set in the action template.

Parameter	Description
Server	The MediaMate server hostname or IP address.
Port	The port on which MediaMate is listening. Default is 8080.
URL_path	The path component of the REST call URL. Include a short name to distinguish concurrent requests.
Operation	The operation to perform: Open, Run, Stop, Close, or Submit.
Xmlcfg	XML configuration for the Open and Submit operations.
Model_path	FLX file path of the MediaMate workflow model. Required for Open and Submit.
Polling_frequency	Seconds between status polls for Run and Submit operations. Default is 5 seconds.

5.29.6 Outputs Description

Outputs include the full MediaMate server response, job state details, FLX player state (for Run and Submit), and error information from both the MediaMate server and the plugin itself.

Parameter	Description
Full_Response	A hash of the entire response received from the MediaMate server.
Short_Name	The MediaMate short name returned in the response.
Job_State	The state value of the running model as reported by MediaMate.
Job_State_Description	An explanation of the Job State as provided by MediaMate.
MediaMate_Error	An explanation of any error returned by the MediaMate server. Returns 'Success' if no error occurred.
FLX_Player_Stat	The state value of the FLX player. Available for Run and Submit operations.

Parameter	Description
Error_Message	An explanation of any error encountered by the plugin itself during execution.

5.29.7 Supported Actions

None

5.29.8 Dependencies

Requires a running and accessible MediaMate server. The MediaMate REST API must be reachable from the Orchestrator server on the configured port.

5.29.9 Operating Instructions

This action controls MediaMate workflow model execution via REST API calls. The plugin constructs the target URL from the configured server, port, and URL path, then appends the appropriate operation segment automatically.

Supported Operations

- Open: Loads the FLX file from the specified Model path and creates all required objects in MediaMate. Returns a model ID of 1, which is always used by subsequent Run, Stop, and Close operations. Requires both the Model path and Xmlcfg parameters.
- Run: Starts executing the loaded MediaMate workflow model. Polls the MediaMate server periodically for job status using the configured polling frequency. Always operates on model ID 1.
- Stop: Stops the running MediaMate workflow model. Executes synchronously. Always operates on model ID 1.
- Close: Closes the MediaMate workflow model and releases associated objects. Executes synchronously. Always operates on model ID 1.
- Submit: Combines Open, Run, Stop, and Close into a single operation. Loads the model, runs it to completion, then closes it. Polls for status during execution. Use this for end-to-end workflow execution in a single step.

URL Construction

- The REST call URL is constructed as: `http://<server>:<port>/<url_path>/<operation>/`
- The operation segment (`open`, `run`, `stop`, `close`) is appended automatically based on the Operation parameter, do not include it in the URL path field.
- Use a short name in the URL path to distinguish between concurrent MediaMate job requests. Example: `mediamate/mm01/interface` and `mediamate/mm02/interface` for two concurrent jobs.
- Example full URL for a Run operation: `http://mediamate-server.example.com:8080/mediamate/mm01/interface/run/`

Best Practices

- Use unique short names in the URL path for each concurrent MediaMate job to prevent conflicts between simultaneous workflow executions.
- Prefer the Submit operation for straightforward end-to-end model execution rather than chaining Open, Run, Stop, and Close steps individually.
- Use the Open + Run + Stop + Close sequence when you need to perform actions or checks between individual lifecycle stages.
- Set an appropriate Polling frequency, too frequent polling increases load on the MediaMate server; too infrequent delays downstream steps. The default of 5 seconds is suitable for most workflows.
- Store the Job State and MediaMate Error outputs and implement error handling workflows that evaluate these values after Run or Submit operations.
- Validate the Xmlcfg structure before use, it must follow the `<XMLCFG><ComponentList>...</ComponentList></XMLCFG>` format.
- Verify the FLX file path is accessible on the MediaMate server before the workflow runs.

Troubleshooting

- Connection refused: Verify the server hostname/IP and port are correct and that the MediaMate server is running and reachable from the Orchestrator server.



Figure 5.29: Media Mate Operation icon

- Operation fails immediately: Check the Error_Message output for plugin-level errors and the MediaMate_Error output for server-side errors.
- Model not found: Confirm the FLX file path is correct and accessible on the MediaMate server.
- Invalid Xmlcfg: Verify the XML configuration follows the required structure and is well-formed XML. Test it directly in MediaMate FLOXE before using it in the plugin.
- Job stuck in running state: Check the MediaMate server logs and use a Stop operation step to halt the model if needed.
- Concurrent job conflicts: Ensure each concurrent workflow uses a unique short name in the URL path to target separate MediaMate model instances.
- Unexpected Job State: Use the Job_State_Description output to get a human-readable explanation of the state returned by MediaMate.

Important Notes

- Run, Stop, and Close operations always act on model ID 1, the same ID that the Open operation always returns. Only one model can be active per URL path at a time.
- The Submit operation is equivalent to Open + Run + Stop + Close executed in sequence as a single plugin call.
- Only Run and Submit operations poll for job status. Open, Stop, and Close execute synchronously.
- The operation path segment (open, run, stop, close) is appended automatically to the URL, do not include it in the URL path parameter.
- Use a unique short name in the URL path to safely run concurrent MediaMate jobs from the same Orchestrator workflow or across multiple workflows.
- The Xmlcfg must use the exact structure `<XMLCFG><ComponentList>...</ComponentList></XMLCFG>`, deviations may cause the Open or Submit operation to fail.
- The MediaMate_Error output returns 'Success' when no error is present, making it safe to evaluate in downstream conditional logic.

5.30 Mongodb operation



Figure 5.30: Mongodb operation icon

This action plugin enables interaction with MongoDB databases, supporting document operations (insert, find, update, remove), advanced queries, aggregation pipelines, map-reduce processing, and database administration commands for comprehensive data management and analysis.

Version	Comment
1.3.2	Enhanced action template UI
1.3.1	Update mongo gem, updating to be compatible with Ruby 3.4 and Rails 8
1.3.0	Remove hardcoded aspera directory reference, Updating to standard coding conventions
1.2.3	Removing Windows support code
1.2.2	Make database operation an optional input
1.2.1	Added description of how to encode urls with special chars.
1.2.0	fixed raw HTML being displayed
1.1.9	Fixed typo for elsif condition.
1.1.8	Fixes for multiple host in URL and client options not being accepted
1.1.7	Added styling to parameter help text.
1.1.6	Accepts JSON or Hash
1.1.5	Inserts DB name into URI if not provided
1.1.4	Fixes document to translate into hash
1.1.3	Added help file
1.1.2	Removes GEM check. Reduces logging
1.1.1	Fixes logging reset
1.1.0	Added Logging Option
1.0.0	Initial Version

5.30.1 How It Works

- Provides comprehensive MongoDB database operations including document queries, insertions, updates, deletions, aggregations, map-reduce operations, and database commands.
- Supports connection to MongoDB instances using standard connection URIs with authentication, host, port, and database specifications.
- Enables flexible operation execution with configurable client options, database options, and collection-specific parameters for customized database interactions.
- Offers multiple document manipulation operations including find, find_one, find_and_modify, insert, update, remove, save, group, aggregate, and advanced map-reduce operations.
- Returns structured output data including document counts, individual documents, document IDs, operation responses, and result arrays based on the operation type.
- Includes optional logging capabilities for monitoring and debugging MongoDB operations within workflow executions.

5.30.2 Configuration Instructions

1. Add a MongoDB Operation step to your workflow.
2. Configure the database connection:
   - Connection uri: Enter MongoDB connection string (e.g., mongodb://203.0.113.100:27017).
   - Database name: Specify the target database name (e.g., inventory_demo).
   - Client options: Add MongoDB client configuration options if needed.
3. Set up the operation:
   - Operation: Select the MongoDB operation type (Document_find, Document_insert, Document_aggregate, etc.).
   - Collection name: Specify the target collection for document operations.
   - Database options: Configure database-specific parameters.
   - Logging enabled: Check to enable MongoDB server logging.

5.30.3 Example Configuration

Parameter	Example Value
Connection uri	mongodb://dbuser:@203.0.113.100:27017
Database name	inventory_demo
Operation	Document_find
Collection name	products
Client options	{ssl: true, connectTimeoutMS: 30000}
Database options	{readPreference: 'secondary'}
Logging enabled	checked

5.30.4 Saved Parameters Description

Parameter	Description
Connection uri	The connection URI containing the host, port, credentials, and other connection parameters. Format: mongodb://[username:password@]host1[:port1][,host2[:port2],...[,hostN[:portN]]][/[database][?options]]
Client options	Option parameters that can be applied to the MongoDB client for connection configuration and behavior customization.
Database name	Name of the database that needs to be connected to for the operation.
Database options	Option parameters that can be applied to the MongoDB Database for specific database-level configurations.
Logging enabled	This checkbox should be checked if logging needs to be enabled on the MongoDB server for operation monitoring.
Operation	Operation that needs to be performed on the MongoDB server. Available options are Database_command, Document_aggregate, Document_count, Document_distinct, Document_find, Document_find_and_modify, Document_find_one, Document_group, Document_insert, Document_map_reduce, Document_remove, Document_save and Document_update.

5.30.5 Inputs Description

Input parameters are not mandatory. URI defaults to 'mongodb://localhost:27017' and database name defaults to 'test'. The MongoDB client documentation lists all valid options that can be used with the MongoDB client.

5.30.6 Operation-Specific Inputs:

5.30.7 Database_command

- Selector: Database command specification. Example: {count: ”products”, query: {name: ”sample_item”}}.
- Options: Options are parameters that are used with the database command.

5.30.8 Document_aggregate

- Collection name: Name of the collection in the MongoDB database.
- Collection options: Option parameters that can be applied to the MongoDB collection.
- Pipeline: Aggregation pipeline array. Example: [{ ”$match”: { ”company”: { ”$exists”: true } } }, { ”$group”: { ”_id”: { ”name”: ”$name”, ”company”: ”$company” }, ”count”: { ”$sum”: 1 } } }].
- Options: Aggregation operation options.

5.30.9 Document_find

- Collection name: Name of the collection in the MongoDB database.
- Collection options: Option parameters that can be applied to the MongoDB collection.
- Selector: Query selectors that are used to query the collection. Example: {”name” => ”sample_product”}.
- Options: Query options such as limit, sort, projection. Example: {limit: 10, sort: {name: 1}}.

5.30.10 Document_find_one

- Collection name: Name of the collection in the MongoDB database.
- Collection options: Option parameters that can be applied to the MongoDB collection.
- Selector: Query selectors for finding a single document. Example: {”name” => ”specific_item”}.
- Options: Query options for the find operation.

5.30.11 Document_find_and_modify

- Collection name: Name of the collection in the MongoDB database.
- Collection options: Option parameters that can be applied to the MongoDB collection.
- Selector: Query criteria for selecting the document. Example: {”name” => ”item_to_modify”}.
- Document: The modification to apply. Example: {”name” => ”updated_item”}.

- Options: Options for the find and modify operation.

5.30.12 Document_group

- Collection name: Name of the collection in the MongoDB database.
- Collection options: Option parameters that can be applied to the MongoDB collection.
- Selector: Query selectors that are used to query the collection for grouping operation.
- Options: Group operation options. Example: {”key”: ”name”, ”cond”: {”name”: {”$exists”: true}}, ”aggregations”: {”count”: {”op”: ”$sum”, ”field”: 1}}}.

5.30.13 Document_insert

- Collection name: Name of the collection in the MongoDB database.
- Collection options: Option parameters that can be applied to the MongoDB collection.
- Documents: Document object(s) that need to be inserted. Example: {”name”:”sample_product”} or for multiple: [{”name”: ”product1”}, {”name”: ”product2”}].
- Options: Insert operation options.

5.30.14 Document_map_reduce

- Collection name: Name of the collection in the MongoDB database.
- Collection options: Option parameters that can be applied to the MongoDB collection.
- Map_function: JavaScript map function. Example: function() { if (this.name && this.qty) { emit(this.name, this.qty); } }.
- Reduce_function: JavaScript reduce function. Example: function(key, values) { return Array.sum(values); }.
- Options: Map-reduce operation options.

5.30.15 Document_remove

- Collection name: Name of the collection in the MongoDB database.
- Collection options: Option parameters that can be applied to the MongoDB collection.
- Selector: Query selectors for selecting documents to remove. Example: {”name”: ”item_to_remove”}.
- Allow Empty Selector?: Check this checkbox if the selector is required to be empty. NOTE: This will remove all documents in the collection.
- Options: Remove operation options.

5.30.16 Document_update

- Collection name: Name of the collection in the MongoDB database.
- Collection options: Option parameters that can be applied to the MongoDB collection.
- Selector: Query criteria for selecting documents to update. Example: {”name” => ”old_value”}.
- Documents: The update document that specifies modifications. Example: {”name” => ”new_value”}.
- Options: Update operation options such as upsert, multi.

5.30.17 Outputs Description

- Count: Count of all the documents that are returned from the selector query on the collection. Populated only for Document_count operation.
- Document: Hash of the document that is returned upon querying the database. Populated only for Document_find_and_modify and Document_find_one operations.
- Document_id: ID of the document returned. Populated only for Document_insert and Document_save operations.
- Response: Response hash of the operation. Populated only for Database_command, Document_map_reduce, Document_remove and Document_update operations.
- Results: Array of the documents that are returned performing operation. Populated only for Document_aggregate, Document_distinct, Document_find and Document_group operations.

5.30.18 Supported Operations

- Database_command - Execute administrative commands on the database
- Document_aggregate - Perform aggregation pipeline operations on collections

- Document_count - Count documents matching query criteria
- Document_distinct - Get distinct values for a field across documents
- Document_find - Query and retrieve multiple documents from collections
- Document_find_and_modify - Find and modify documents in a single atomic operation
- Document_find_one - Query and retrieve a single document from collections
- Document_group - Group documents by specified criteria with aggregation functions
- Document_insert - Insert new documents into collections
- Document_map_reduce - Perform map-reduce operations on collections
- Document_remove - Delete documents from collections
- Document_save - Save documents (insert or update based on ID)
- Document_update - Modify existing documents in collections

5.30.19 Operating Instructions

- MongoDB Server Requirement: This plugin is used to interact with MongoDB. A MongoDB database server must be installed and running, and credentials to access it should be available.
- Connection Configuration: Use the standard MongoDB connection URI format to specify host, port, database, and authentication details. The plugin defaults to 'mongodb://localhost:27017' if no URI is provided.
- Documentation Reference: For detailed information on options and selectors, refer to the MongoDB Ruby driver documentation at http://www.rubydoc.info/github/mongodb/mongo-ruby-driver/Mongo.
- Query Limitations: Some complex selector formats are not supported. For example, this selector for a Document_find operation is not supported: { ”product.id”: 12345 }, { ”product.id”: 1 }. It's supposed to retrieve only the id field in the document with id 12345. Use this form instead: { ”product.id”: 12345 } and then use a custom Ruby plugin to extract only the fields you need from the Results output.
- Map-Reduce Operations: When using Document_map_reduce, provide JavaScript functions for both Map_function and Reduce_function. Example Map function: function() { if (this.name && this.qty) { emit(this.name, this.qty); } }. Example Reduce function: function(key, values) { return Array.sum(values); }.
- Multiple Document Insertion: For Document_insert operations, you can create multiple inserts by providing an array format: [{”name”: ”item1”}, {”name”: ”item2”}]. After insertion, verify the records have been created in the collection.
- Find and Modify Operations: Document_find_and_modify performs atomic find and update operations. After entering details, select find_one_modify and update it to execute the operation.
- Database Commands: For Database_command operations, use the Selector field to specify the command. Example for counting documents: {count: ”collection_name”, query: {name: ”specific_value”}}.
- Aggregation Pipeline: For Document_aggregate operations, provide the aggregation pipeline in the Pipeline field. Example: [{ ”$match”: { ”company”: { ”$exists”: true } } }, { ”$group”: { ”_id”: { ”name”: ”$name”, ”company”: ”$company” }, ”count”: { ”$sum”: 1 } } }].
- Group Operations: For Document_group operations, configure the Options field with grouping criteria. Example: {”key”: ”name”, ”cond”: {”name”: {”$exists”: true}}, ”aggregations”: {”count”: {”op”: ”$sum”, ”field”: 1}}}.
- Data Safety: When using Document_remove with 'Allow Empty Selector?', be extremely cautious as this will remove ALL documents in the collection. Always verify your selector criteria before execution.

5.30.20 Connection URI Format

- Basic Format: `mongodb://[username:password@]host1[:port1][,host2[:port2],...[,hostN[:portN]]][/[database][?options]]`
- Simple Connection: `mongodb://203.0.113.100:27017`
- With Authentication: `mongodb://dbuser:<YOUR_PASSWORD>@203.0.113.100:27017/inventory_demo`
- Replica Set: `mongodb://203.0.113.100:27017,203.0.113.101:27017/inventory_demo?replicaSet=myReplSet`
- With Options: `mongodb://203.0.113.100:27017/inventory_demo?ssl=true&connectTimeoutMS=30000`
- Character Escaping: The characters ':' and '@' in a username or password must be escaped (RFC 2396).

5.30.21 Example Use Cases

5.30.22 Case 1: Map-Reduce Analysis

- Purpose: Aggregate quantity data by product name across a collection
- Map Function: `function() { if (this.name && this.qty) { emit(this.name, this.qty); } }`
- Reduce Function: `function(key, values) { return Array.sum(values); }`

- Result: Returns aggregated quantities for each product name

5.30.23 Case 2: Document Operations

- Insert: Add new products with `{"name":"sample_product"}`
- Update: Modify existing records using selector `{"name" => "old_product"}` and document `{"name" => "updated_product"}`
- Find: Query documents with `{"name" => "specific_product"}`
- Remove: Delete documents matching `{"name": "product_to_remove"}`

5.30.24 Case 3: Advanced Operations

- Aggregation: Use pipeline `[{ "$match": { "company": { "$exists": true } } }, { "$group": { "_id": { "name": "$name", "company": "$company" }, "count": { "$sum": 1 } } }]`
- Group: Configure options `{"key": "name", "cond": {"name": {"$exists": true}}, "aggregations": {"count": {"op": "$sum", "field": 1}}}`
- Find and Modify: Atomic operations combining find with update in a single transaction

5.30.25 Dependencies

- MongoDB server installed and running on the target host or accessible via network connection.
- Valid MongoDB database credentials with appropriate permissions for the intended operations.
- Network connectivity between Orchestrator and the MongoDB server on the specified port (default: 27017).
- MongoDB Ruby driver dependencies installed on the Orchestrator server.
- Proper firewall configurations allowing connections to MongoDB server ports.

5.30.26 Troubleshooting

- If connection fails, verify that the MongoDB server is running and accessible at the specified host and port in the connection URI.
- For authentication errors, confirm that the username and password in the connection URI are correct and that the user has appropriate permissions for the target database.
- If operations timeout, check network connectivity and consider adjusting client options like connectTimeoutMS and socketTimeoutMS in the connection URI.
- For collection not found errors, ensure the collection exists or use operations that can create collections implicitly (such as insert operations).
- If selector queries return unexpected results, verify the query syntax matches MongoDB query format and test queries directly in MongoDB shell first.
- When Document_remove operations fail with empty selector, ensure 'Allow Empty Selector?' is checked if you intend to remove all documents, otherwise provide specific query criteria.
- For Map-Reduce operations, ensure JavaScript functions are syntactically correct and test them in MongoDB shell before using in the plugin.
- If Document_find_and_modify operations don't work as expected, verify that both selector and document fields contain valid MongoDB query and update specifications.
- For aggregation pipeline issues, validate the pipeline array syntax and ensure each stage follows MongoDB aggregation framework specifications.
- For group operations, verify the Options JSON structure includes proper key, condition, and aggregation specifications.
- For large result sets, consider using options like limit and skip to manage memory usage and improve performance.
- When using Database_command operations, ensure the command syntax follows MongoDB's database command format and includes all required parameters.

5.31 Remote execution

This action plugin provides the ability to execute scripts or executables on remote nodes via SSH, with support for dynamic input variables and custom output extraction.



Figure 5.31: Remote execution icon

Version	Comment
1.2.2	Enhanced action template UI
1.2.1	Fix issues when using ERB template tags in command field
1.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.1.2	Avoid to set stdio in the detailed status if report stdio is set
1.1.1	Updating Edit panel and Help page
1.1.0	Add an option to remove the keep alive handling, causing issues on a Windows remote node
1.0.0	When a report stdio frequency is provided and return stdio option is selected, stdio is streamed to status and used as output.
0.9.1	Fixes help page
0.9.0	Implements cancel
0.8.5	Fixed raise condition on exit signal
0.8.4	Sets environment variable on the keep alive channel too
0.8.3	SSH close logic modified since it was taking more time.
0.8.2	Adding Remote Node Dependency
0.8.1	Clarifies the use of the 'use server address' features, by toggling sections.
0.8.0	Avoids closing before exit signal is received and ensures keep-alive channel is closed upon exit
0.7.2	Allows status changes in process output code block
0.7.1	New icon
0.7.0	Make server address as an optional field also. Requested by Turner 2014/11/14
0.6.4	Split outputs processing into lines, in case buffer is not read line by line
0.6.3	Adds a default status exit message on success
0.6.2	-> Improve help file
0.6.1	-> exception handling of RemoteNode.find_by_name()
0.6.0	-> Converted command type to text to allow for larger commands
0.5.4	-> Avoids the exit race condition (channel closed before exit signal is processed)
0.5.3	-> Fixing the version as it was still 0.5.1. Made the execution_node, execution_login and execution_password variables available for runtime.
0.5.2	-> Prevents autocomplete of login field
0.5.1	-> Added help
0.5.0	-> Adds code execution for the formatting of outputs
0.4.1	-> Fixes glitch with missing stdio
0.4.0	-> Adds capture of exit code and user defined outputs
0.3.1	-> Adds functionality to mask password
0.3.0	-> When a report stderr frequency is provided, stderr is streamed to status instead of log file.
0.2.0	-> When a report stdio frequency is provided, stdio is streamed to status instead of used as output.
0.1.5	-> modified view to collapse optional execution account info
0.1.4	-> Fixes erroneous return code
0.1.3	-> Uses termination API for early exit
0.1.2	-> Adds support for clustered nodes
0.1.1	-> Forces out of the SSH.wait when a status is returned
0.1.0	-> Makes the timeout a template parameter, SSH timeout is used only in the case of infinite template-defined timeout
0.0.2	-> Adds keep alive and set timeout to ensure the action times-out if the client SSHD becomes unreachable.
0.0.1	-> Working. Used for both executable and remote daemons

5.31.1 How It Works

- Executes scripts or executables on remote nodes via SSH connections with configurable timeout and cancellation support.
- Supports dynamic command line construction using input variables accessible via template syntax (<%=variable_name%>).
- Captures standard output, standard error, and exit codes from remote command execution.
- Provides multiline output processing and custom variable extraction from command output.
- Offers real-time status updates and progress reporting during remote execution.

5.31.2 Configuration Instructions

1. Add a RemoteExecution step to your workflow.
2. Configure the basic execution parameters:
   - Name: Enter a descriptive name for the step (e.g., 'New RemoteExecution Step').
   - Command: Enter the command line to execute (e.g., 'echo ”Hello from Orchestrator” > /tmp/orch_test_hello.txt').
   - Keep alive?: Check to enable SSH keep-alive mechanism (recommended).
3. Set timeout and cancellation options:
   - Is cancelable?: Check to allow cancellation of the remote execution.
   - Timeout: Leave empty to use the default of 60 seconds, or specify custom timeout.
4. Configure advanced features (optional):
   - Standard I/O: Click to expand and configure stdout/stderr handling.
   - Mandatory inputs: Click 'Add Mandatory inputs' to define required variables.
   - Optional inputs: Click 'Add Optional inputs' for optional variables.
   - Formatted Outputs: Click 'Add Formatted Outputs' to extract custom variables.

5.31.3 Example Configuration

Parameter	Example Value
Name	New RemoteExecution Step
Command	echo ”Hello from Orchestrator” > /tmp/orch_test_hello.txt
Is cancelable?	unchecked
Keep alive?	checked
Standard I/O	(collapsed - using defaults)

5.31.4 Expected Test Results

Based on the above configuration, the plugin successfully:

- Executes the echo command on the remote node (RPM Build 1037)
- Creates the file /tmp/orch_test_hello.txt with content 'Hello from Orchestrator'
- Returns successful execution status
- Provides standard outputs: exit_code, stdio, and any configured custom outputs

5.31.5 Saved Parameters Description

Parameter	Description
Name	Display name for this remote execution step.
Comments	Notes or description associated with this template or configuration.
Command	Command line to execute on the remote node. Use <%=variable_name%> syntax for dynamic variables.
Is cancelable?	When enabled, canceling will shut down the SSH connection, potentially stopping remote command execution.
Keep alive?	Enable internal keep-alive mechanism. Recommended ON for Linux; may be disabled for Windows to avoid extra processes.
Timeout	Maximum execution time in seconds before remote command is terminated. Leave empty to use 60-second default.
Standard I/O	Configure stdout/stderr reporting frequency and multiline output handling.

Parameter	Description
Mandatory inputs	Define required input variables accessible in the command line via <%=variable_name%> notation.
Optional inputs	Define optional input variables accessible in the command line via <%=variable_name%> notation.
Formatted outputs	Specify custom output variables that the step will produce.
Formatted outputs processing code	Ruby code executed after command completion. Access outputs via @outputs[VARIABLE_STDIO], @outputs[VARIABLE_MULTILINE], @outputs[VARIABLE_EXITCODE].
Remote execution node	Target remote node from the list of configured Remote Nodes.
Execution server address	Alternative remote node address to use instead of configured node address.
Use server address	Use the provided server address instead of the remote node's configured address.
Execution account	SSH username and password for remote connection. If empty, uses credentials specified in the remote node configuration.

5.31.6 Inputs Description

Runtime inputs depend on the configuration. Variables exposed in the command line become mandatory inputs.

- Execution_node: Remote node from which the command is executed.
- Execution_login: Login name to access the remote node (optional).
- Execution_password: Password to access the remote node (optional).
- Custom variables: Any variables defined in Mandatory inputs or Optional inputs sections.

5.31.7 Outputs Description

- exit_code: Exit code of the remote execution. Non-zero values cause plugin to return an error.
- stdio: Standard output from the remote execution when multiline is set to false.
- Multi: Standard output returned as an array when multiline is enabled (one entry per output line).
- Additional outputs: Custom outputs defined in ”Formatted outputs” section are also available.

5.31.8 Supported Actions

None

5.31.9 Dependencies

- SSH access must be configured and functional on target remote nodes.
- For environment variables over SSH, /etc/ssh/sshd_config must include AcceptEnv directives.
- Remote node must have required scripts/executables and proper permissions.

5.31.10 Operating Instructions

5.31.11 Standard I/O Configuration

- Empty stdio frequency: Standard output is streamed to stdio output variable and plugin status message when execution completes.
- With stdio frequency: Standard output streams to plugin status message during execution. Final output goes to stdio variable if enabled.
- With stderr frequency: Standard error streams to plugin status message. Otherwise goes to Orchestrator log file.

5.31.12 Status Updates

Remote executables can provide status updates using this format:

'-#STATUS--#--STATUS MESSAGE#-'

- '-#Paused--#--Remote action was paused#-' → updates status and message
- '-#--#--File tester.txt was scanned successfully#-' → updates message only
- Shell script example: `echo "-#Paused--#--Remote action was paused#-"`

5.31.13 Output Variable Extraction

Extract custom variables from standard output using this format:

'-#OUTPUT:VARIABLE NAME--#--VARIABLE VALUE#-'

Example: '-#OUTPUT:size--#--120568#-' → populates @outputs['size']=120568

5.31.14 Example Shell Script

Shell script returning error 2, output variable 'size', multiline true:

\ md5sum $1 | cut -d' ' -f1\ echo ”-#OUTPUT:size--#--120568#-”\ echo ”hello”\ exit 2\

Plugin outputs (multiline true):

\ size 120568\ exit_code 2\ Multi [”45c2d08009be4a2307eddd4f0bd10a8f”, ”hello”]\

Plugin outputs (multiline false):

\ size 120568\ exit_code 2\ stdio 45c2d08009be4a2307eddd4f0bd10a8f\ hello\

5.31.15 Environment Variables

To enable environment variables over SSH, modify /etc/ssh/sshd_config:

1. Add line: AcceptEnv [env_variable_1] [env_variable_2] [...]
2. Restart sshd service for changes to take effect
3. On Isilon systems, file location is /etc/mcp/templates/sshd_config

5.32 REST request



Figure 5.32: REST request icon

This plugin allows you to make REST requests to web services, either as a one-time operation or with repeated polling at specified intervals.

Version	Comment
1.2.4	Enhanced action template UI
1.2.3	Fix post_parameters_get to convert String to Hash
1.2.2	Updated the label from Keep ongoing to Continue Monitoring

Version	Comment
1.2.1	Updated the label from Keep ongoing to Continue Monitoring
1.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.1.6	Fix type conversion when headers parameters are passed at runtime
1.1.5	Fix result in case of a Json array
1.1.4	Added option for patch method
1.1.3	Fixed error in case of get request with empty headers parameters
1.1.2	Added protection against null headers parameters
1.1.1	Updating configuration and help pages.
1.1.0	Adding flexibility to specify ssl version.
1.0.9	Add some json_result examples in the help
1.0.8	Proxy is applied per request and not globally.
1.0.7	Ensure proxy is applied when polling.
1.0.6	Timeout to only be infinite if explicitly requested.
1.0.5	Prevent autofill from filling in username/password.
1.0.4	Fix for xml_output parsing.
1.0.3	Verify SSL option in each rest method
1.0.2	Allows use of run-time variable with quote value in header or post parameters
1.0.1	Set custom status and status details for execute once
1.0.0	Provide ability to give a file path on disk to be read into Post body
0.9.4	Avoid exceptions if responses contain nil values.
0.9.3	Set custom status details for polling.
0.9.2	Bug fix in polling stop code.
0.9.1	Return code for network failure field changed from ”-” to ”000” to maintain int for nil response.
0.9.0	Return code for network failure field added to maintain backward compatibility.
0.8.0	Ability to set status and status details in polling stop contition code.
0.7.10	Added styling to help text.
0.7.9	Added help section for SSL verification parameters.
0.7.8	204 code added to ACCEPTABLE_RESPONSE
0.7.7	Response type is now a required input
0.7.6	Allows the user to set status in process output code.
0.7.5	Fixed the uneditable rich text editor bug on the polling stop condition field
0.7.4	adding HTTP CODE to the save_output_to_file option
0.7.3	adding pdf to response type
0.7.2	Handle well an authentication error (was calling result.headers even if result was nil)
0.7.1	Expose http headers in output
0.7.0	Expose timeout of request, Fix no-timeout to use large constant instead of -1, Fix bug in file upload mode via PUT
0.6.1	Fixing option to avoid SSL verification
0.6.0	Adding support for key authentication
0.5.3	Fix PUT file upload other bug
0.5.2	Fix PUT file upload bug
0.5.1	Fix no-timeout to use nil instead of -1
0.5.0	Adding a flag and a field to write response data to directly. Helpful when large text is returned as response.
0.4.3	Catching RestClient exception and returning http code and http response as output. Still raising the exception to be backward compatible.
0.4.2	Making Rest method available at runtime.
0.4.1	Fixing Proxy Request
0.4.0	Adding Proxy field to Database.
0.3.3	Adds HTTP timeout set to same value as step timeout (0 for unlimited)
0.3.2	Adds rich text editor to polling condition text block
0.3.1	Improved help file
0.3.0	Fixed version for creation of migration _add_polling.rb
0.2.5	Add polling
0.2.4	Uses defaulted response type when not specified
0.2.3	Assigns header parameters to header attribute of RestClient request
0.2.2	Prevents auto complete of user auth field
0.2.1	Fix request type in PUT request
0.2.0	Adds support for PUT method
0.1.2	Adds support for DELETE method

Version	Comment
0.1.1	Adds support for GET method
0.1.0	Adds switch for http post mode, adds header parameters
0.0.2	Accepts 201 and 202 return status
0.0.1	Only supports POST command

5.32.1 How It Works

- Generates REST requests to remote services and handles responses with support for GET, POST, DELETE, and PUT methods.
- Supports multiple authentication methods including basic authentication and SSL certificate authentication.
- Provides polling capabilities to repeatedly execute requests until conditions are met.
- Offers trigger functionality to spawn new work orders when new records are detected in responses.
- Handles various content types (XML, JSON, text) with automatic parsing and formatting capabilities.

5.32.2 Configuration Instructions

1. Add a RestRequest step to your workflow.
2. Configure the basic request parameters:
    - Name: Enter a descriptive name for the step (e.g., 'New RestRequest Step').
    - Service URL: HTTP URL to reach the remote service (e.g., https://jsonplaceholder.typicode.com/posts/1).
    - REST method: Select GET, POST, DELETE, or PUT from the dropdown.
    - Response type: Choose json, xml, or text based on expected response format.
3. Configure SSL and security settings:
    - SSL verification: Check to enable SSL verification (recommended for HTTPS URLs).
    - Ssl version: Select appropriate SSL version or leave as default.
    - Basic Auth login/password: Leave empty if authentication is not required.
4. Set optional parameters:
    - Request timeout: Leave empty for no timeout or specify seconds.
    - Return Code for Network failure: Leave empty to use default -1.
    - Save output to file: Check only if you need to save response to a file.

5.32.3 Example Configuration

Parameter	Example Value
Name	New RestRequest Step
Comments	(empty)
Service URL	https://jsonplaceholder.typicode.com/posts/1
Rest method	GET
Basic Auth login	(empty)
Basic Auth password	
SSL verification	checked
SSL Version	-Select-
Response type	json
Request timeout	(default: no timeout)
Save output to file	unchecked
Processed outputs code	(empty)

5.32.4 Saved Parameters Description

Parameter	Description
Name	Display name for this REST request step.
Comments	Notes or description for this step.
Service URL	HTTP URL to reach the remote service.
Rest method	REST method: GET, POST, DELETE, or PUT.

Parameter	Description
Basic Auth login	Account name for basic authentication (optional).
Basic Auth password	Account password for basic authentication (optional).
SSL Client Certificate File Path	Path to SSL client certificate file.
SSL Client Key File Path	Path to SSL client key file.
SSL Client Key Passphrase	Passphrase for SSL client certificate.
SSL CA Certificate File Path	Path to SSL CA certificate file.
SSL Verification	Enable SSL verification instead of login/password authentication.
SSL Version	SSL version to be used.
Polling frequency	Polling interval in seconds. If not set, no polling is performed.
Polling stop condition code	Ruby code condition to stop polling (default: return true).
Continue Monitoring	Use plugin as trigger. Requires non-null polling frequency.
Maximum number of records returned	Maximum records in response for Formatted_result output. Each spawned work order handles this maximum.
Result records extracting code	Ruby code to extract array from response.
Record formatting code	Ruby code to format individual records from extracted array.
Record identification code	Ruby code to identify unique records from formatted records (for trigger mode).
Header parameters	HTTP request header name/value pairs in Ruby Hash format: { 'key' => 'value' }.
Content type	Request content type: jpg, xml, gif, text, json, pdf (for POST/PUT requests).
Read POST body from file?	Read POST body content from file instead of text field.
POST body file path	Full path to file containing POST body content.
HTTP form POST?	Use HTTP form to build POST request body.
POST parameters	Name/value pairs for HTTP form parameters.
POST request body	POST body content matching defined content type.
Response type	Expected response content type: json, xml, or text.
Request timeout	Timeout duration in seconds before giving up on request (optional).
Return Code for Network failure	Custom return code for network failures (default: -1, empty: -100).

Parameter	Description
Processed outputs	Name/type pairs for additional plugin outputs.
Processed outputs code	Ruby code executed after response processing to populate processed outputs.

5.32.5 Inputs Description

Changes in this list of inputs will override any values set in the action template.

5.32.6 Mandatory Inputs:

- Service_url: HTTP URL to reach the remote service.
- POST_parameters: Name/value pairs for HTTP form parameters (if 'HTTP form POST?' is selected).
- POST_body: POST request body content (if 'HTTP form POST?' is not selected).

5.32.7 Optional Inputs:

- Basic_auth_login: Account name for basic authentication.
- Basic_auth_password: Account password for basic authentication.
- Header_parameters: HTTP request header parameters.
- Processed_outputs_code: Ruby code executed after response processing.
- Polling_stop_condition: Ruby code condition to stop polling.
- Records_extracting_code: Ruby code to extract array from response.
- Record_formatting_code: Ruby code to format records from extracted array.
- PUT_file_path: File path for PUT method when POST request body is not filled.
- Save Output to file: Check to save output to file.
- Write response to this file: File path where response will be written.

5.32.8 Outputs Description

- Service_result: Raw response as a string.
- Response_header: HTTP header of the response as a hash.
- XML_result: XML parsed response as a hash (only present if response type is xml).
- JSON_result: JSON parsed response as a hash (only present if response type is json).
- Formatted_result: Formatted response as an array after extracting and formatting code have been applied.
- HTTP_return_code: HTTP status code returned by the service (e.g., 200 for success).
- Processed outputs: Each name/value pair defined in processed outputs creates a new output of the selected type.

5.32.9 Supported Actions

- GET
- POST
- DELETE
- PUT

5.32.10 Dependencies

None

5.32.11 Operating Instructions

Overview: This action generates REST requests to remote services and handles responses.

5.32.12 URL Parameters and Authentication

- Parameters can be passed in the HTTP URL using ? and & notation: `http://localhost/api/endpoint?param1=value1&param2=value2`
- Parameters can alternatively be passed in the HTTP request body using POST method with either HTML form format or custom content.
- Authentication can be done directly in the URL or by specifying basic authentication credentials in dedicated fields.
- Basic authentication combines username:password, Base64 encodes it, and puts it in the HTTP Authorization header.

5.32.13 Content Types and Response Handling

- Specify content type for request body (xml, json, text, etc.) and expected response type.
- If xml response type is selected, response is parsed and 'XML_result' variable populated with hash.
- If json response type is selected, response is parsed and 'JSON_result' variable populated with hash.
- If text response type is selected, no parsing occurs and 'Service_result' variable holds raw response string.

5.32.14 Processed Outputs

Define name/type pairs for processed outputs and use ”Processed outputs code” to populate them:

- Use syntax: `outputs['<variable_name>'] = <value>`
- XML example: `outputs['name'] = xml_result['name'][0]`
- JSON example: `outputs['name'] = json_result['name']`

5.32.15 Test Results Example

When calling a public REST API like JSONPlaceholder (https://jsonplaceholder.typicode.com/posts/1) with GET method:

- HTTP_return_code: 200 (indicating successful request)
- Service_result: Raw JSON response as string
- JSON_result: Parsed JSON response as hash with fields like ”userId”, ”id”, ”title”, ”body”
- Formatted_result: Empty array when no formatting code is specified

5.32.16 Polling Functionality

- Set the polling frequency (e.g., 5 seconds) to repeat requests until a condition is met.
- Define stop condition in Ruby using 'Polling stop condition code' field.
- Example XML stop condition: `xml_result['status'][0] == 'complete' rescue nil`
- Example JSON stop condition: `json_result['status'] == 'complete' rescue nil`

5.32.17 Trigger Functionality

- Select 'Continue Monitoring' to enable trigger mode that spawns new work orders for new records.
- Use 'Result records extracting code' to extract array from response (e.g., `return xml_result['workflow']`).
- Use 'Record formatting code' to format each record (often `record = record.to_s` for readability).
- Use 'Record identification code' to identify unique records (e.g., `return record['id']`).
- Records already detected are stored and won't be re-detected unless the workflow is restarted.
- Extracting and formatting code can be used without trigger mode for response formatting.

5.32.18 Example XML Response Processing

Given XML response:

\ <?xml version=”1.0”?>\ <workflows>\ <workflow id=”1” name=”create file”/>\ <workflow id=”2” name=”custom ruby”/>\ </workflows>\

After XML parsing, the `xml_result` variable contains the following hash:

\ { 'workflow' => [{ ”name” => ”create file”, ”id” => ”1” }, { ”name” => ”custom ruby”, ”id” => ”2” }] }\

- Result records extracting code: `return xml_result['workflow']`

- Record formatting code: `return { "id" => record["id"], "name" => record["name"] }`
- Record identification code: `return record["id"]`

5.33 Soap request



Figure 5.33: Soap request icon

This plug-in provides the ability to periodically execute a SOAP query on a web service. It can be used for a one-off request or as a polling mechanism.

Version	Comment
2.1.6	Enhanced action template UI
2.1.5	Wrap SOAP response to display service_results like older SOAP4R format
2.1.4	Updated the label from Keep ongoing to Continue Monitoring
2.1.3	Updated help documentation
2.1.2	Mask HTTP basic auth password field
2.1.1	Fix service action issue when using a local WSDL file
2.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
2.0.1	Fixed request file purging code
2.0.0	Added possibility to use the legacy XML parsing library (XmlSimple)
1.9.0	Delete the wire dump file once read unless soap debug mode selected
1.8.0	Removes debug mode
1.7.2	Upgrading XML parsing
1.7.1	setting verify ssl to false
1.7.0	Debug mode added
1.6.4	Set custom status details for polling.
1.6.3	Set status and status details in polling stop contition code (incomplete in 1.6.0).
1.6.2	Added styling to help text.
1.6.1	Fix utf-8 encoding in response
1.6.0	Adds the possibility to set the return status
1.5.1	Resolve conflict between 1.4.6 and 1.5.0
1.5.1	Resolve conflict between 1.4.6 and 1.5.0
1.5.0	Add http timeout parameter when using payload mode
1.4.6	Improve help file with formatting examples
1.4.3	Add help file and update pause, resume, cancel handling
1.4.2	Removes need for wsdl if endpoint and actions are explicitly provided
1.4.1	Protect in case the answer does not use a ”Body” container
1.4.0	Adds support for Basic auth in explicit mode only
1.3.1	Fix glitch on create in UI
1.3.0	Fix glitch on create in UI (does not work)
1.2.9	Correctly report the check_status frequency
1.2.8	Fix UI endpoint when using realtime WSDL input
1.2.7	Adds support for legacy wsdl path
1.2.6	Clean up after adding cookie export
1.2.5	Enable Cookie export and support (only with old soap)
1.2.4	Fixes polling to continue if @results is [] instead of just nil

Version	Comment
1.2.3	Fixes wiredump path when action starts with capital
1.2.2	Explicit return in the outputs_spec (bug fix for 1.2.1)
1.2.1	Avoids voiding result, adds raw outputs to trigger mode
1.2.0	Adds raw outputs options
1.1.0	Adds explicit soap action field to remove reliance on SOAP4R parsing of wsdl
1.0.1	Calls format_result even if no results row are present
1.0.0	Adds explicit payload mode (a-la SOAPUI)
0.2.5	Adds rpc_result variable in polling condition test for consistency
0.2.4	Fix bug in service_uri_get
0.2.3	Move SOAP StreamHandler modification to module_soap_tools
0.2.2	Expose polling stop code in GUI
0.2.1	Adds display name
0.2.0	Split config and runtime
0.1.2	Disambiguate rpc_driver
0.1.1	Bug fix when not used in trigger mode
0.1.0	Add customization of the identification of uniq record
0.0.3	Add patch to SOAP4R to handle field attributes, and support for complex inputs
0.0.2	Bug fix in inputs
0.0.1	Initial release

5.33.1 How It Works

- Generates SOAP requests to remote web services and processes their responses.
- Supports WSDL-based service discovery (remote URI or local file) or manual endpoint configuration.
- Provides polling capabilities for continuous monitoring and trigger-based workflow execution.
- Offers flexible response parsing with Ruby code for extracting, formatting, and processing data.
- Supports HTTP basic authentication and session management with cookies.

5.33.2 Configuration Instructions

1. Add a SoapRequest step to your workflow.
2. Configure the basic parameters:
   - Name: Enter a descriptive name for the SOAP request.
   - Comments: Add notes or a description for this step.
3. Set up service connection (choose one method):
   - Method 1 - WSDL URI: Provide the WSDL URI to describe the remote service (e.g., http://example.com/service?wsdl).
   - Method 2 - WSDL File: Use a local WSDL file stored in /opt/aspera/orchestrator/config/orchestrator/actions/soap_requests
   - Method 3 - Manual Configuration: Specify both the endpoint and SOAP action directly (requires 'Provide explicit payload?' to be checked).
4. Configure service operation:
   - Service: Select the SOAP operation from the drop-down list (populated from the WSDL).
   - Provide explicit payload?: Check to provide an explicit SOAP XML payload in the 'Service payload' box.
   - Service payload: Enter the complete SOAP XML payload (use <%= variable %> notation for dynamic values).
   - Service action parameters: Alternatively, provide operation parameters (if explicit payload is not used).
   - Session header parameters: Provide SOAP header parameters (if explicit payload not used).
5. Configure authentication (optional):
   - HTTP basic auth user: Account name for HTTP basic authentication.
   - HTTP basic auth password: Account password for HTTP basic authentication.
6. Set up polling and trigger behavior (optional):
   - Polling frequency: Interval in seconds for repeated requests. If not set, no polling is done.
   - Continue Monitoring: Enable trigger mode to spawn new workorders for each detected record.
   - Maximum number of records returned: Maximum records per response for Formatted_result output.
   - Polling stop condition code: Ruby code to determine when to stop polling.
   - Record identification code: Ruby code to identify unique records (required with 'Continue Monitoring').
7. Configure response processing:
   - Use legacy XML parsing?: Use XmlSimple library instead of Nokogiri (default since v1.7.2).

- Result records extracting code: Ruby code to extract an array from the response.
- Record formatting code: Ruby code to format each extracted record.
- Results formatting code: Ruby code executed after response processing.
- Result type: Expected type of Service_results output (array, string, int, hash, flag).
- Return raw response?: Include full XML response with SOAP envelope.
- Return structured response body?: Include XML response body as structured data.
- Return real status: Allow modifying return status in formatting/extraction code.

8. Enable debugging (optional):
   - Enable wsdl debug log file?: Store SOAP message exchange copy under run directory.

5.33.3 Example Configuration

Parameter	Example Value
Name	Get Database Records
Comments	Retrieves records from remote database service
Wsdl uri	http://www.example.com/ws/services/DBService?wsdl
Service	fetchRecords
Provide explicit payload?	checked
Service payload	<fetchRecords><query><%= query_param %></query></fetchRecords>
HTTP basic auth user	api_user
HTTP basic auth password	********
Polling frequency	30
Continue Monitoring	checked
Maximum number of records returned	100
Use legacy XML parsing?	unchecked
Result type	array
Return structured response body?	checked

5.33.4 Saved Parameters Description

Parameter	Description
Name	Display name for this SOAP request instance.
Comments	Notes or description for this step.
Wsdl uri	The WSDL URI to describe the remote service (e.g., http://example.com/service?wsdl).
Wsdl file	Local WSDL file stored in /opt/aspera/orchestrator/config/orchestrator/actions/soap_requests.
Enable wsdl debug log file?	If selected, copies of SOAP message exchanges are stored in the run directory.
Service	The SOAP operation from the defined operations for the remote service.
Provide explicit payload?	Check to provide an explicit SOAP XML payload in the 'Service payload' box.
End point	The URL for the remote node providing the service. Can be used if no WSDL is specified.

Parameter	Description
SOAP action	Explicit SOAP operation. Can be used if no WSDL is specified.
HTTP basic auth user	Account name for HTTP basic authentication. Optional.
HTTP basic auth password	Account password for HTTP basic authentication. Optional.
Service payload	The SOAP request XML payload. Supports variable notation <%= variable %>.
Service action parameters	SOAP request operation parameters if explicit payload is not provided.
Session header parameters	SOAP request header parameters if explicit payload is not provided.
Polling frequency	Polling interval in seconds. If not set, no polling is done.
Continue Monitoring	Enable trigger mode. Requires non-null polling frequency.
Maximum number of records returned	Maximum records in a response for the Formatted_result output.
Use legacy XML parsing?	Use XmlSimple library instead of Nokogiri (default since v1.7.2).
Polling stop condition code	Ruby code used to determine when to stop polling. Default: return true (stop now).
Record identification code	Ruby code to identify unique records from formatted records. Required with 'Continue Monitoring'.
Return real status	Allow modifying return status and status_details in formatting/extraction code. By default, step returns COMPLETE unless critical error detected.
Result records extracting code	Ruby code to extract an array from the response.
Record formatting code	Ruby code to format records from the extracted array.
Results formatting code	Ruby code executed after response processing.
Result type	Expected type of Service_results output: array, string, int, hash, or flag.
Return raw response?	Include full XML response with SOAP envelope in Raw_response output.
Return structured response body?	Include XML response body as structured data in Response_body output.

5.33.5 Inputs Description

Changes in this list of inputs will override any values set in the action template.

Mandatory Inputs:

- Wsdl_uri: The WSDL URI to describe the remote service. Not needed if End point and SOAP action are specified.
- Service_action: The SOAP operation from the defined operations. Not needed if End point and SOAP action are specified.

Optional Inputs:

- Http_basic_auth_user: Account name for HTTP basic authentication.

- Http_basic_auth_password: Account password for HTTP basic authentication.
- Result_formatting_code: Ruby code executed after the response is treated.
- Polling_stop_condition: Ruby code condition to stop the polling.
- Records_extracting_code: Ruby code to extract an array from the response.
- Record_formatting_code: Ruby code to format records from the extracted array.
- Cookie: HTTP header cookie to maintain session information between multiple SOAP requests.

5.33.6 Outputs Description

- Raw_response: The raw response as a string. Only present if 'Return raw response?' is selected.
- Response_body: The SOAP response body as a hash. Only present if 'Return structured response body?' is selected.
- Service_results: The results after the code in 'Results formatting code' has been applied. Type is determined by 'Result type'. By default an array.
- Cookie: The cookie response as a string for session management.

Note: Each processed output name/value pair will introduce a new output of the selected type.

5.33.7 Supported Actions

None

5.33.8 Dependencies

- Network connectivity to the remote SOAP service endpoint.
- Valid WSDL file (either remote URI or local file) or manual endpoint/action configuration.
- For local WSDL files: File must be stored in /opt/aspera/orchestrator/config/orchestrator/actions/soap_requests
- Ruby environment for custom code execution (extracting, formatting, stop conditions).

5.33.9 Operating Instructions

5.33.10 Overview

The SoapRequest plugin generates SOAP requests to remote web services and handles their responses. It supports WSDL-based service discovery, manual endpoint configuration, polling for continuous monitoring, and flexible response processing with Ruby code.

5.33.11 Service Configuration Methods

- Method 1 - Remote WSDL URI: Provide an HTTP URI to the WSDL (e.g., http://www.ebi.ac.uk/ws/services/WS-Dbfetch?wsdl). The plugin will parse the WSDL and populate available operations in the Service drop-down.
- Method 2 - Local WSDL File: Store the WSDL file in /opt/aspera/orchestrator/config/orchestrator/actions/soap_requests and select it from the drop-down. Useful for offline development or when the remote WSDL is unstable.
- Method 3 - Manual Configuration: Specify both End point and SOAP action directly. This requires checking 'Provide explicit payload?' and entering the complete XML payload manually. Use this when no WSDL is available.

5.33.12 Payload Configuration

5.33.13 Explicit Payload Mode (Recommended)

- Check the 'Provide explicit payload?' box to enter XML manually.
- Test your SOAP request with tools like SoapUI first, then copy and paste the XML into the 'Service payload' field.
- Use variable notation <%= variable_name %> for dynamic values.
- This method is more predictable than relying on parameter parsing from WSDL.

Example Payload:

\ <fetchRecords>\ <database><%= database_name %></database>\ <query><%= search_query %></query>\ <maxResults>100</maxResults>\ </fetchRecords>\

5.33.14 Parameter Mode (Alternative)

- Leave 'Provide explicit payload?' unchecked and use 'Service action parameters' or 'Session header parameters'.
- Not recommended as it relies on WSDL parsing which can be unpredictable.
- The plugin attempts to build the SOAP request from the parameters and WSDL structure.
- Use explicit payload mode for more control and reliability.

5.33.15 Authentication

- HTTP Basic Authentication is supported via username and password fields.
- Credentials are combined as "username:password", Base64 encoded, and sent in the Authorization header.
- Leave fields empty if authentication is not required.
- For more complex authentication (e.g., WS-Security), include authentication elements in your explicit payload.

5.33.16 XML Parsing Modes

IMPORTANT: For all configurations defined before version 2.0 of the plugin, legacy mode is the default. For new configurations, legacy mode will not be selected as the default.

5.33.17 Modern Mode (Nokogiri - Default since v1.7.2)

Example response structure:

\{"GetSessionsResponse"=>{"xmlns:ns1"=>"urn:Aspera:XML:FASPSessionNET:2009/11:Types",\ "SessionKeysResult"=>{"ResultCount"=>"2", "RemainingResultCount"=>"0",\ "IterationToken"=>"dd549761f97954417d2abf3c47ec877b000000
"SessionKey"=>[{"UUID"=>"fe77385c-3d17-4613-b839-1fc04bc5d423"},\ {"UUID"=>"678357a5-cbf1-490a-8417-3d1f17390ea6"}]}}}\

5.33.18 Legacy Mode (XmlSimple)

Example response structure:

\{"GetSessionsResponse"=>[{"SessionKeysResult"=>[{"ResultCount"=>["2"],\ "SessionKey"=>[{"UUID"=>["fe77385c-3d17-4613-b839-1fc04bc5d423"]},\ {"UUID"=>["678357a5-cbf1-490a-8417-3d1f17390ea6"]}],\ "RemainingResultCount"=>["0"],\ "IterationToken"=>["dd549761f97954417d2abf3c47ec877b0000000000001229"]}],\ "xmlns:ns1"=>"urn:Aspera:XML:FASPSessionNET:2009/11:Types"}]}\

Note: The structure differs significantly between modes. Adjust your Ruby code accordingly.

5.33.19 Polling and Stop Conditions

- Set a 'Polling frequency' (in seconds) to enable repeated requests.
- The SOAP request will repeat until the stop condition is met.
- Define stop condition in 'Polling stop condition code' using Ruby.
- The `xml_result` variable contains the parsed XML body.

Example Stop Condition (Modern Mode):

\ xml_result["GetSessionsResponse"]["SessionKeysResult"]["RemainingResultCount"] == '0' rescue nil\

Example Stop Condition (Legacy Mode):

\ xml_result["GetSessionsResponse"][0]["SessionKeysResult"][0]["RemainingResultCount"][0] == '0' rescue nil\

5.33.20 Trigger Mode and Record Processing

Enable 'Continue Monitoring' to spawn new workorders for each detected record.

- Requires 'Polling frequency' to be set.
- Use 'Result records extracting code' to extract an array from the response.
- Use 'Record formatting code' to format each extracted record.
- Use 'Record identification code' to identify unique records.
- Each detected record spawns a new workorder instance.
- Previously detected records are memorized and won't re-trigger (unless workflow is restarted).

5.33.21 Response Processing Examples

Note: The following examples assume legacy XML parsing mode. Adapt for modern mode by adjusting array indices.

5.33.22 Example SOAP Response

\ <soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">\ <soapenv:Body>\ <ns1:GetSessionsResponse xmlns:ns1="urn:Aspera:XML:FASPSessionNET:2009/11:Types">\ <SessionKeysResult>\ <ResultCount>2</ResultCount>\ <RemainingResultCount>0</RemainingResultCount>\ <IterationToken>dd549761f97954417d2abf3c47ec877b0000000000001229</IterationToken>\ <SessionKey>\ <UUID>fe77385c-3d17-4613-b839-1fc04bc5d423</UUID>\ </SessionKey>\ <SessionKey>\ <UUID>678357a5-cbf1-490a-8417-3d1f17390ea6</UUID>\ </SessionKey>\ </SessionKeysResult>\ </ns1:GetSessionsResponse>\ </soapenv:Body>\ </soapenv:Envelope>\

5.33.23 Result Records Extracting Code

Modern Mode:

\ return xml_result["GetSessionsResponse"]["SessionKeysResult"]["SessionKey"]\

This extracts:

\ [{"UUID"=>"fe77385c-3d17-4613-b839-1fc04bc5d423"},\ {"UUID"=>"678357a5-cbf1-490a-8417-3d1f17390ea6"}]\

5.33.24 Record Formatting Code

\ return { "id" => record["UUID"] } rescue nil\

Service_results output:

\ [{"id" => "fe77385c-3d17-4613-b839-1fc04bc5d423"},\ {"id" => "678357a5-cbf1-490a-8417-3d1f17390ea6"}]\

5.33.25 Record Identification Code

\ return "#{record["id"]}"\

This identifies each record by its UUID for trigger tracking.

5.33.26 Simple Formatting Alternative

If you only need the UUID values:

\ return record["UUID"]\

Service_results output:

\ ["fe77385c-3d17-4613-b839-1fc04bc5d423",\ "678357a5-cbf1-490a-8417-3d1f17390ea6"]\

In this case, 'Record identification code' can be left empty as each record is already a unique identifier.

5.33.27 Results Formatting Code

For post-processing after extraction and formatting:

\ token = xml_result["GetSessionsResponse"]["SessionKeysResult"]["IterationToken"]\ results = { "iteration_token" => token }\

Change 'Result type' to 'hash'. Service_results output:

\ {"iteration_token"=>"91286e2557483993505661edef5435690000000000001234"}\

5.33.28 Debugging Tips

- Inspect xml_result structure: Use `return xml_result.inspect` in 'Result records extracting code' to see the full structure.
- Enable debug logging: Check 'Enable wsdl debug log file?' to save SOAP exchanges for inspection.
- Test with SoapUI: Validate your SOAP requests externally before implementing in the plugin.
- Use rescue clauses: Always include `rescue nil` in Ruby code to prevent errors from stopping workflows.
- Check Response_body output: Enable 'Return structured response body?' to see the parsed SOAP body structure.

5.33.29 Session Management

- Use the Cookie input and output for maintaining sessions across multiple SOAP requests.
- The Cookie output from one request can be mapped to the Cookie input of subsequent requests.
- This allows stateful interactions with services that require session management.
- Cookies are returned as strings and should be passed without modification.

5.33.30 Best Practices

- Use explicit payload mode for better control and reliability.
- Test SOAP requests with SoapUI or similar tools before implementing in workflows.
- For new configurations, use modern XML parsing mode (Nokogiri) for cleaner data structures.
- Always include `rescue nil` or proper error handling in Ruby code.
- Use 'Enable wsdl debug log file?' during development to troubleshoot request/response issues.
- Keep polling frequencies reasonable to avoid overwhelming the remote service.
- When using trigger mode, ensure 'Record identification code' uniquely identifies records.
- Store sensitive credentials securely and use variable notation in payloads.
- Document your Ruby extraction and formatting code for future maintenance.
- Consider using 'Maximum number of records returned' to limit memory usage with large responses.

5.34 Softlayer API



Figure 5.34: Softlayer API icon

Integration with Softlayer API

Version	Comment
0.2.3	Enhanced action template UI
0.2.2	Updated the label from Keep ongoing to Continue Monitoring
0.2.1	Updated the label from Keep ongoing to Continue Monitoring
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.2	added SoftLayer_Network_StorageAdded
0.1.1	Fixed ORC-2546 where payload is not read from mapped input
0.1.0	Added result limit and result offset
0.0.1	Initial version

5.34.1 How It Works

- This action plug-in integrates with the Softlayer API.
- SoftLayer uses a fixed API endpoint; it cannot be changed.
- You must provide a SoftLayer username and API key (called the Classic Infrastructure API Key in IBM Cloud).
- The list of available SoftLayer services is documented in the Softlayer Developer Network.
- Trigger and Polling behaviour:
  - If the polling frequency is greater than 0, the API URL is called periodically based on the polling interval.
  - After each call, the Polling Stop Condition code is evaluated. If it returns `true`, polling stops.
  - The polling logic can also update the action status:
    * `@status` — accepts `STATUS_COMPLETE`, `STATUS_INPROGRESS`, or `STATUS_FAILED`.
    * `@status_details` — adds a message to the Work Order view.
- Processed Outputs:
  - This code runs once at the end of execution or after the final poll.
  - It processes the final API response.
  - Use it to populate output variables, for example:
    `@outputs['variable_name'] = value`

5.34.2 Configuration Instructions

1. Create a new action step using the SoftlayerApi plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved SoftlayerApi configured instance.
   - Comments: Some comments about this saved SoftlayerApi configured instance.
   - Softlayer URL: Choose between the public or private Softlayer URL.
   - Proxy Address: HTTP Proxy to reach Softlayer URL.
   - SSL verification: Verify SSL certificate presented by Softlayer URL.
   - Username: Softlayer API username.
   - APIKey: Softlayer API key.
   - Softlayer Service: One of the Softlayer services.
   - Softlayer init param: Softlayer initialisation parameter, which usually refers to a specific object ID.
   - Softlayer Method: A method applicable for Softlayer service selected
   - Rest method: HTTP REST method.
   - Response type: Softlayer response format, either XML, JSON or TXT.
   - Object Mask: A list of attributes to pass as objectMask to Softlayer API.
   - Object Filter: A Json string containing a Softlayer objectFilter.
   - Post body: HTTP POST body.
   - Request Timeout: Timeout given to execute the request. 0 applies a 60 second default.
   - Continue Monitoring: Makes the plugin act as a continuous trigger.
   - Polling Frequency: How often to call the URL. 0 disables polling.
   - Polling stop condition: Ruby code that will be evaluated at every poll. Returning true stops the poll.
   - Processed outputs: A list of variables to make available as outputs.
   - Processed outputs code: Ruby code that is executed at the end to fill processed outputs with values
   - Result limit: Softlayer API result limit
   - Result offset: Softlayer API result offset

5.34.3 Example Configuration

This is a minimum setup configuration for the SoftlayerApi plugin.

Parameter	Value
Softlayer URL	https://api.softlayer.com/rest/v3.1
SSL verification	Selected
Username	admin
APIKey	password
Softlayer Service	SoftLayer_Account
Softlayer init param	123456
Softlayer Method	getObject
Rest method	get

Parameter	Value
Response type	json
Continue Monitoring	Selected

5.34.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved SoftlayerApi configured instance.
Comments	Some comments about this saved SoftlayerApi configured instance.
Softlayer URL	Choose between the public or private Softlayer URL.
Proxy Address	HTTP Proxy to reach Softlayer URL.
SSL verification	Verify SSL certificate presented by Softlayer URL.
Username	Softlayer API username.
APIKey	Softlayer API key.
Softlayer Service	One of the Softlayer services.
Softlayer init param	Softlayer initialisation parameter, which usually refers to a specific object ID.
Softlayer Method	A method applicable for Softlayer service selected
Rest method	HTTP REST method.
Response type	Softlayer response format, either XML, JSON or TXT.
Object Mask	A list of attributes to pass as objectMask to Softlayer API.
Object Filter	A Json string containing a Softlayer objectFilter.
Post body	HTTP POST body.
Request Timeout	Timeout given to execute the request. 0 applies a 60 second default.
Continue Monitoring	Makes the plugin act as a continuous trigger.
Polling Frequency	How often to call the URL. 0 disables polling.
Polling stop condition	Ruby code that will be evaluated at every poll. Returning true stops the poll.
Processed outputs	A list of variables to make available as outputs.
Processed outputs code	Ruby code that is executed at the end to fill processed outputs with values
Result limit	Softlayer API result limit
Result offset	Softlayer API result offset

5.34.5 Inputs Description

The list of inputs depends on the configuration of the SoftlayerApi action template. If dynamic attributes are used in the message definition, each attribute becomes a required input.

Parameter	Description
*Softlayer_method**	A method applicable for Softlayer service selected.
Softlayer_init-param	Softlayer initialisation parameter, which usually refers to a specific object ID.
Object_filter	A Json string containing a Softlayer objectFilter.
Object_mask	A list of attributes to pass as objectMask to Softlayer API.

Parameter	Description
Post_body	HTTP Post.
Proxy_address	HTTP Proxy to reach Softlayer URL.
Request_time-out	Timeout given to execute the request. 0 applies a 60 second default.
Result_limit	Softlayer API result limit.
Result_offset	Softlayer API result offset.

5.34.6 Outputs Description

Parameter	Description
HTTP_return_code	The HTTP status code of the response (e.g., 200).
Service_result	The result returned by the API call.

5.34.7 Supported Actions

None

5.34.8 Dependencies

None

5.34.9 Operating Instructions

The best place to begin is the Softlayer API documentation. The API uses a single, fixed endpoint that cannot be changed. To authenticate, you must provide a SoftLayer username and API key, referred to in IBM Cloud as the ”Classic Infrastructure API Key.” A full list of available services is available in the Softlayer Developer Network.

If the polling frequency is set above zero, the API URL is called repeatedly at the specified interval. After each call, the polling stop condition code is evaluated, and polling stops if the condition returns `true`. During polling, the step’s outcome can be controlled by updating `@status`—which may be set to `STATUS_COMPLETE`, `STATUS_INPROGRESS`, or `STATUS_FAILED`—and `@status_details`, which adds informational text to the Work Order view.

At the end of execution, or after the final poll, the processed outputs code is run. This code operates on the final API response and should be used to populate your defined output variables, for example: `@outputs['variable_name'] = value`.

5.35 3PlayMedia Operation



Figure 5.35: 3PlayMedia Operation icon

Add description here

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	Fix autofill bug that displays username/password
0.0.1	Initial implementation

5.35.1 Saved Parameters Description

- Name: The name used to identify a saved ThreePlayOperation configured instance.
- Name: Name of the plugin instance
- Comments: comments associated with the instance
- Service URL: URL of the 3PlayMedia service
- API Key: The api key to access 3PlayMedia service
- Trigger Operation: 1.Submit to 3Play, 2. Check Status, 3.Download Caption are the 3 operations supported in the plugin as of now
- API secret key: Required for Submit operation. The secret key to initiate a job
- Link: Link of the url used to trigger a job
- Name 3 Play: Name to be associated with the new job
- Description: Description for the job
- File: The file for triggering a job
- Video: video id for the job
- Batch name: Folder name for the job
- Batch id: Folder id associated for the job
- Project Name: Name of the project under which the job has to be created
- Account Name: Name of the account under which the job has to be created
- Attribute1:
- Attribute2:
- Attribute3:
- Monitor Status for completion?: On a submit job, once the job is submitted, check this option if you want orchestrator to monitor the status of the job for completion
- Check for downloading caption: Once the job is completed, if this option is checked, we initiate the download caption option
- Polling Frequency: For async jobs, the interval of polling
- Starting timecode sample: For download caption, use this option if needed
- Frames per second: For download caption, enter the frames per second
- Drop Frame:
- Format: Enter the format of the caption file

5.35.2 Inputs description

- *service_url, apikey, spi secret key, link*: the mandatory parameters for submit job operation
- *internal_id and apikey*: the mandatory parameters for the poll for status operation
- *internal_id, apikey and format*: the mandatory parameters for the download caption operation

5.35.3 Outputs description

- Document all available outputs using add_output_help
- *Status*: Status of the job
- *Internal_id*: InternalId of the job
- *Raw Output*: Raw output received from the server
- *Error*: If any error found, this variable will hold the message

5.35.4 Supported Actions

None

5.35.5 Dependencies

None

5.35.6 Operating Instructions

5.36 Watson Language Translator

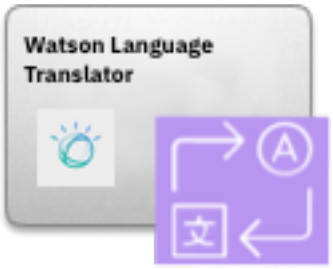


Figure 5.36: Watson Language Translator icon

This action plug-in provides the ability to translate from one language to another using IBM Watson Language Translator service.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.1	Initial implementation

5.36.1 Saved Parameters Description

- Name: The name used to identify a saved WatsonLanguageTranslator configured instance.
- Comments: Some comments about this saved WatsonLanguageTranslator configured instance.
- Operation: The operation you want to perform using IBM Watson Language Translator service, which are:
  - Translate - Translates text or an array of texts from one language to another.
  - List identifiable languages - Get the list of all the languages which IBM Watson Language Translator service can recognize.
  - Identify Language - Identify the language of the input text.
  - List Models - Get the list of all the language translation models provided by the IBM Watson Language Translator service.
  - Translate Document - Translate a given document from one language to another.
  - Get Document Status - Get the status of the document submitted for translation.
  - Get Transalated Document - Retrieve the translated document.
- Url: The URL of IBM Watson Language Translator service.By default it is https://gateway.watsonplatform.net/language-translator/api
- Use api key: If selected, the API key is used instead of username and password.
- Api key: The API key of IBM Watson Language Translator service.
- Username: The username of IBM Watson Language Translator service.
- Password: The password of IBM Watson Language Translator service.
- Version: The version of IBM Watson Language Translator service to be used for making API requests.By default it is 2018-05-01
- Source Language: Translation source language code.
- Target Language: Translation target language code.
- Input text: For Translation operation: It is the Input text in UTF-8 encoding which is required to be transalated.It can take array of inupts, Multiple entries will result in multiple translations in the response. For multiple entries the input should be in this format: [”text1”,”text2”]. For Identify Language operation: It is just input text in UTF-8 format.
- Model id: The identifier of the model to be used for the translation request. Example the model id for translation from English to Spanish is en-es
- File path: The path to the file that needs to be transalated.
- Filename: The name of the file that needs to be transalated.
- Document id: The document ID of the document whose details we are trying to fetch.
- Accept: The type of response we want to get when trying to fetch the Translated Document. Example : text/html,text/json,text/plain,text/richtext,text/rtf,text/xml etc.

5.36.2 Inputs description

The list of inputs depends on the configuration of the WatsonLanguageTranslator action template.

5.36.3 Outputs description

- *Error Message*: If there is an error, this variable shows the error
- *Result*: Generally displays the Hash result for your query
- *Translations*: Displays the list of translation output in UTF-8, corresponding to the input text entries.
- *Document_id*: System generated ID identifying a document being translated using one specific translation model.
- *Filename*: Displays the filename from the submission
- *Status*: Displays the status of the translation job associated with a submitted document. Possible values: [processing, available, failed]

5.36.4 Supported Actions

None

5.36.5 Dependencies

None

5.36.6 Operating Instructions

The IBM Watson Language Translator service is an IBM Cloud service. You get the credentials (URL, username, password, API key) with an online subscription. Using those credentials you can exceute the IBM Watson Language Translator service operations through the plugin.

Chapter 6

Other Utilities

6.1 Collection manager

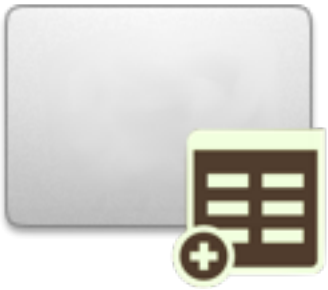


Figure 6.1: Collection manager icon

This action plug-in enables creating a name-value pair collection (Hash) or adding fields to an existing one.

Version	Comment
0.3.2	Enhanced action template UI
0.3.1	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.0	Remove hardcoded aspera directory reference
0.2.2	Updated path to runtime version of ruby
0.2.1	fixed plugin description
0.2.0	added pretty JSON and TOML outputs
0.1.0	- Marc - 2015/05/06 - Adds option to return as json and to extract specific fields
0.0.1	- Marc - 2015/05/05 - initial release

6.1.1 How It Works

- Creates or updates a name–value pair collection (`Hash`).
- If no source is provided, a new collection is created.
- If a Hash or JSON collection is provided, elements are updated when matching keys exist or added when missing.
- Elements are always typed as defined at design-time.
- Collections can also be exported in multiple formats: JSON, Pretty JSON, or TOML.
- Specific elements can be extracted from the collection and returned as their own typed outputs.

6.1.2 Configuration Instructions

1. Create a new action step using the CollectionManager plug-in.
2. Fill out the following fields:
    - Required elements: Mandatory elements to be added to the collection. Must be typed.

- Optional elements: Optional elements that may be included, also typed.
- Extracted outputs: Specific elements to return as separate typed outputs.
- Use JSON?: If checked, collection is also returned serialized in JSON.
- Use pretty JSON?: If checked, collection is serialized as pretty JSON.
- Use TOML?: If checked, collection is serialized in TOML format.

3. Optional: You can provide a `Source_collection` (Hash) or `Source_JSON_collection` (JSON text) to extend or modify an existing collection.

6.1.3 Example Configuration

Field	Value
Required elements	`Name` (String), `Age` (Integer), `DOB` (Date)
Optional elements	`Working` (Boolean), `Skills` (Array)
Use JSON?	Checked
Use pretty JSON?	Checked
Use TOML?	Checked

Behavior: This configuration builds a collection with required and optional elements. The resulting collection is returned as a Hash, JSON, Pretty JSON, and TOML string. Extracted outputs are also returned in their declared types.

6.1.4 Saved Parameters Description

Parameter	Description
Required elements	Mandatory typed elements to include in the collection
Optional elements	Optional typed elements that may be added
Extracted outputs	Elements extracted from the collection as individual outputs
Use JSON?	Returns the collection serialized in JSON format
Use pretty JSON?	Returns the collection serialized in human-readable JSON
Use TOML?	Returns the collection serialized in TOML format

6.1.5 Inputs Description

- Source_collection: Optional Hash collection to use as a base. If not provided, a new collection is created.
- Source_JSON_collection: Optional JSON collection to use as a base. If both Hash and JSON are provided, they are merged.

All declared elements are then added or updated in the collection. Element types are enforced at design-time.

6.1.6 Outputs Description

- Modified_collection: The new collection that is either based on the one provided or created, with the addition of all the elements defined with their provided values.
- Modified_JSON_collection: (Only provided if 'Use JSON?' is checked. The new collection above, but serialized in JSON and provided as plain text.)
- Modified_pretty_JSON_collection: (Only provided if 'Use pretty JSON?' is checked. The new collection above, but serialized in pretty JSON and provided as plain text.)
- Modified_TOML_collection: (Only provided if 'Use TOML?' is checked. The new collection above, but serialized in TOML and provided as plain text.)

6.1.7 Supported Actions

None

6.1.8 Dependencies

For Orchestrator versions earlier than 4.0.0, install the following gems for TOML support:

\ gem 'citrus', '3.0.2'\ gem 'toml-rb', '2.0.1'\ bundle install\

6.1.9 Operating Instructions

The CollectionManager plug-in is used to build or update typed collections. You can merge collections, enforce typing, and output in multiple formats. It is especially useful for workflows where structured data needs to be passed across steps.

6.1.10 Troubleshooting

- Ensure all element types match the declared type; no implicit conversion occurs.
- Verify that provided JSON is valid when using `Source_JSON_collection`.
- If TOML output fails on Orchestrator < 4.0.0, confirm that `citrus` and `toml-rb` gems are installed.
- Check that the extracted output names match the elements in the collection.

6.2 Custom Python



Figure 6.2: Custom Python icon

This action plug-in can be used to script in python for some customized behaviour.

Version	Comment
0.1.3	Enhanced action template UI
0.1.2	Log errors to stderr instead of displaying stacktrace in work order output
0.1.1	Add Bitbucket integration
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	Fix Help
0.0.1	Initial Release.

6.2.1 How It Works

- This plugin allows execution of Python code within the workflows.
- It enables dynamic logic, data transformation, and integration with external systems.
- The plugin uses declared inputs and outputs to interact with the workflow and must explicitly set the workflow status.

6.2.2 Configuration Instructions

1. Add a Custom Python step to your workflow.
2. Configure the execution environment:
   - Name: The label to identify this ConvertPython step.
   - Required inputs: Define the inputs your Python code needs.
   - Outputs: Define the expected outputs from the script.
   - Execute code: Write or paste the Python logic to be executed.

6.2.3 Example Configuration

6.2.4 Example 1: Sample Python code to return a file size.

Field	Parameter	Type
Required inputs	file_path	String
Outputs	file_size	Integer

```
import os
\ file_path = inputs[”file_path”]
\ if os.path.isfile(file_path):
\   outputs[”file_size”]  = os.path.getsize(file_path)
\   outputs[”status”] = ”Complete”
\ else:
\   outputs[”status”] = ”Failed”
```

6.2.5 Example 2: Sample code to parse XML data.

Field	Parameter	Type
Required inputs	file_path	String
Outputs	media_file_name	String

The XML file is:

```
<Asset>
\   <File>The-Hobbit-VO.mxf</File>
\   <Profile>XDCAM HD422</Profile>
\   <Target>VOD Distrib</Target>
\   <Source>Internal Demo</Source>
\ </Asset>
```

The Python code is:

```
import os
\ import xmltodict
\ xml_file_path = inputs['file_path']
\ with open(xml_file_path) as fd:
\     data = xmltodict.parse(fd.read())
\ try:
\     outputs['media_file_name'] = data['Asset']['File']
\ except:
\     pass
```

Note: inputs['var_name'] can also be abbreviated var_name. Example:

```
xml_file_path = file_path
```

To specify a custom status message, you can set the status_details key of the outputs hash with the customized message using this syntax:

```
outputs['status_details'] = 'Customized status details message'
```

6.2.6 Saved Parameters Description

Parameter	Description
Name	The label to identify this ConvertPython step.
Required inputs	Define the inputs your Python code needs.
Optional inputs	Add any additional inputs that may or may not be used.
Outputs	Define the expected outputs from the script.
Execute code	Write or paste the Python logic to be executed.
Use code from source control?	If you do not want to enter the code inline, enable this option to fetch it from a source control system. Use the dropdown to select either GitHub or Bitbucket. The available fields below will update based on your selection. See the *Source Control Configuration* section for details.
Use cached code?	If checked, the plugin uses the last saved (cached) version of the Python code instead of downloading the latest version from source control every time the step runs. This helps keep workflow runs consistent, especially when the remote code is being updated. If the cached code is missing, the plugin will automatically fetch the latest code and update the cache.
Verify SSL	Check this box to use SSL when connecting to the source control host.

6.2.7 Source Control Configuration

When Use code from source control? is enabled, select a provider from the dropdown. The fields displayed will vary depending on the selected provider.

6.2.8 GitHub

Parameter	Description
GitHub host name	If you are using GitHub Enterprise, enter the host name used to access your GitHub Enterprise instance.
GitHub token	For private repositories, enter the personal access token of the GitHub user for authentication.
GitHub repo	The repository name where the Python code file resides.
GitHub file path	The path to the file from the root of the repository (exclude the repository name itself).
GitHub branch name	The branch to fetch the code from.
GitHub user	The GitHub username under which the repository is present.

The GitHub API URL format used is: *https://api.github.com/repos/Username/RepositoryName/contents/FilePath?ref=Branch-Name*

6.2.9 Bitbucket

Parameter	Description
Bitbucket host name	Enter the Bitbucket host name in API URL format (e.g. *api.bitbucket.org* for standard Bitbucket Cloud, or your self-hosted equivalent)
Bitbucket token	For private repositories, enter the access token or app password for authentication.
Bitbucket repo	The repository name where the Python code file resides.
Bitbucket file path	The path to the file from the root of the repository (exclude the repository name itself).
Bitbucket branch name	The branch to fetch the code from.
Workspace	The Bitbucket workspace under which the repository is present.

If the source control host is unreachable, the plugin falls back to a cached version of the code stored at: */opt/aspera/orchestrator/config/orchestrator/custom_pythons*

6.2.10 Inputs Description

- The list of inputs depends on the configuration of the CustomPython action template.
- The inputs for the action will be the ones declared in the 'Required inputs' and 'Optional inputs' section.

6.2.11 Outputs Description

- The outputs optionally declared in the 'Outputs' section will be returned with their specified names and types.

6.2.12 Supported Actions

None

6.2.13 Operating Instructions

- The plugin allows running Python code.
- Inputs are accessed using the inputs dictionary.
- The Flask server will run on port 3200 on enabling the plugin (by default). To customize that port, customers can change the port specified */opt/aspera/orchestrator/actions/custom_pythons/orchestrator-custom-pythons_flask_server.service*. After that, they will need to disable the plugin and then re-enable it.
- The Flask server will stop on disabling the plugin.
- Inputs must be declared under Required or Optional inputs.
- Outputs are accessed using the outputs dictionary.
- Two reserved output keys are status and status_details.
- To update workflow status, use two reserved output keys: status and status_details.
- Both status and status_details must be part of the outputs dictionary.
- The code must set outputs['status'] to one of the following values: '*Complete*' – when the workflow finishes successfully, '*Failed*' – when the workflow fails and '*Error*' – when there is an execution error
- The plugin includes source control integration to fetch code directly from GitHub or Bitbucket repositories.
- When GitHub/Bitbucket parameters are provided as mapped inputs, the UI will display them as GitHub parameters (for example, Github_token), even if Bitbucket is configured. Bitbucket authentication will still function correctly.
- If the source control host is unreachable, the plugin uses a cached version of the code.
- Cached code is stored at: */opt/aspera/orchestrator/config/orchestrator/custom_pythons*

6.2.14 Dependencies

- Python3, Flask and flask_cors must be installed in the same environment as the Orchestrator server.
- A Flask server file is available at: */opt/aspera/orchestrator/actions/custom_pythons/flask_server.py*
- This file can be used to run the Flask server required for the plugin if the server is not running already.

6.2.15 Installing Flask and Dependencies

For RHEL 9 and Rocky Linux 9:

\ yum install pip\ pip install flask\ pip install flask_cors

For RHEL 8 and Rocky Linux 8:

\ dnf install python3-pip\ pip3 install flask\ pip3 install flask_cors

6.3 Hello world

This is a reference implementation for a simple Action plugin illustrating the API.



Figure 6.3: Hello world icon

Version	Comment
1.3.1	Enhanced action template UI
1.3.0	Made fixes to ensure that the plugin runs to completion
1.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.1.0	Initial release

6.3.1 How It Works

- Constructs a customizable message by parsing a prompt string at execution time.
- Supports variable substitution using the syntax `<%=variable_name%>` embedded in static text.
- Displays the message letter by letter with a configurable pause between each character.
- Default tempo is 2 seconds per letter, overridable at runtime.
- Prompt can be stored in the database or provided at runtime via optional input.
- Runtime prompt takes precedence over stored prompt if both are provided.
- Supports workflow actions: Pause, Resume, Cancel, and Rollback.

6.3.2 Configuration Instructions

1. Create a new action step using the HelloWorld plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved HelloWorld configured instance.
   - Comments: Comments about this saved HelloWorld configured instance.
   - Prompt: The string to use as a prompt. That string can include variables that get replaced during execution by their run-time values.

6.3.3 Example Configuration

This is an example configuration for the HelloWorld plugin with variable substitution.

Parameter	Value
Name	Welcome Message
Comments	Displays a personalized greeting message
Prompt	Hello <%=name%>, welcome to <%=system_name%>!

6.3.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved HelloWorld configured instance.
Comments	Comments about this saved HelloWorld configured instance.
Prompt	The string to use as a prompt. That string can include variables that get replaced during execution by their run-time values.

6.3.5 Inputs Description

Both inputs are optional. The action can execute using only the stored prompt with default tempo settings.

Parameter	Description
prompt_override	Provides the ability to use a run-time variable to set the prompt pattern instead of using a value stored in the database. If both are provided, the run-time one takes precedence.
tempo	Provides the ability to use a different value for the duration of the pause between each letter during execution. Expressed in seconds. Default is 2 seconds.

6.3.6 Outputs Description

Parameter	Description
message	A string representing the prompt provided with the variables replaced by their values at run-time

6.3.7 Supported Actions

- Pause: Temporarily halts the letter-by-letter display. The message remains partially displayed.
- Resume: Continues the letter-by-letter display from where it was paused.
- Cancel: Stops the action execution immediately. The partial message is not saved.
- Rollback: Reverses the action execution. Can be used to undo the message display in workflow rollback scenarios.

6.3.8 Dependencies

None

6.3.9 Operating Instructions

This action returns a string that represents a message constructed at execution time by parsing the prompt (either stored in the database, or provided at execution time using the prompt_override optional input variable).

Variable Syntax and Substitution

The prompt string can contain static text and variables expanded at run-time. Variables are indicated using the syntax <%=variable_name%> embedded into static text.

- Basic variable substitution: `"Hello <%=name%>."`
- If `name = "John"`, the output becomes: `"Hello John."`
- Multiple variables: `"Welcome <%=first_name%> <%=last_name%> to <%=company%>!"`
- If `first_name = "Jane"`, `last_name = "Doe"`, `company = "Acme Corp"`, the output becomes: `"Welcome Jane Doe to Acme Corp!"`
- System variables: `"Processing started at <%=timestamp%> on <%=hostname%>"`
- Variables can reference any input, output, or system variable available in the workflow context.

Prompt Configuration Options

- Stored Prompt (Database):
  - Configure the prompt in the 'Prompt' parameter during action template creation
  - Prompt is saved in the database and reused for all executions
  - Best for static messages that rarely change
  - Example: `"System backup completed successfully."`
- Runtime Prompt Override:
  - Provide the prompt using the `prompt_override` optional input at execution time
  - Runtime prompt takes precedence over stored prompt
  - Best for dynamic messages that vary per execution
  - Example: Different messages based on workflow conditions or external data
- Hybrid Approach:
  - Configure a default prompt in the database

  - Override with runtime prompt only when needed
  - Provides flexibility while maintaining a fallback default

Tempo Configuration

- The tempo controls the speed at which the message is displayed letter by letter.
- Default tempo: 2 seconds per letter
- Fast display: Set tempo to 0.1 or 0.5 seconds for quick message rendering
- Slow display: Set tempo to 5 or 10 seconds for dramatic effect or demonstration purposes
- Instant display: Set tempo to 0 to display the entire message immediately
- Tempo can be overridden at runtime using the `tempo` optional input (expressed in seconds).

Common Use Cases

- Welcome message: Greet users by name when they log in or start a workflow.
- Status notification: Display dynamic status messages with variable information (e.g., file count, processing time).
- Demonstration/Training: Show slow letter-by-letter display for educational purposes.
- Workflow logging: Generate formatted log messages with runtime variable values.
- Conditional messaging: Use runtime prompt override to display different messages based on workflow conditions.

Best Practices

- Use descriptive variable names in prompts for better readability and maintainability.
- Test prompts with sample data to ensure variables are replaced correctly.
- Set appropriate tempo values based on use case: fast for production, slow for demonstrations.
- Use stored prompts for static messages and runtime override for dynamic content.
- Ensure all referenced variables are available in the workflow context before execution.
- Consider using Pause/Resume actions for long messages that may need interruption.
- Document variable requirements in the Comments field for easier template maintenance.

Example Execution Flow

Configuration:

Prompt (stored): `"Processing file <%=filename%> for user <%=username%>."`

Tempo: 2 seconds (default)

Runtime Variables:

```
filename = "report.pdf"
```

```
username = "admin"
```

Execution:

The action displays the message letter by letter:

`P` (wait 2s) `r` (wait 2s) `o` (wait 2s) `c` (wait 2s) ... `Processing file report.pdf for user admin.`

Output:

```
message = "Processing file report.pdf for user admin."
```

6.4 Input Manipulations

This action plug-in can be used to manipulate inputs using data operations based on the input type.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	Bug fix. Allows array and hash as run time inputs
0.0.1	initial release



Figure 6.4: Input Manipulations icon

6.4.1 How It Works

- Provides a comprehensive set of data manipulation operations for strings, arrays, and hashes.
- Supports string operations: split, compare, substitute (sub), global substitute (gsub), and more.
- Supports array operations: concatenation and type conversion.
- Supports hash operations: key-value retrieval and type conversion.
- Allows pattern matching using both strings and regular expressions.
- Returns processed output based on the selected operation and input type.
- Enables dynamic data transformation within workflows without custom scripting.

6.4.2 Configuration Instructions

1. Create a new action step using the InputManipulation plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved Input Manipulation configured instance.
   - Comments: Comments about this saved Input Manipulation configured instance.
   - Input type: Select the type of input - String, Array, or Hash.
   - Operation: The operation to be performed on the input based on the selected input type.
   - Required inputs: Inputs that must be provided for the selected operation to execute.
   - Optional inputs: Additional inputs that may be required depending on the specific operation chosen.
   - Outputs: The output variables that will be returned after the operation is performed.

6.4.3 Example Configuration

This is an example configuration for the InputManipulation plugin performing a string split operation.

Parameter	Value
Name	String Split Operation
Comments	Split email address by @ symbol
Input type	String
Operation	Split
Input	user@example.com
Pattern	'@'

6.4.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved Input Manipulation configured instance.
Comments	Comments about this saved Input Manipulation configured instance.
Input type	Select the type of input - String, Array, or Hash.
Operation	The operation to be performed on the input based on the selected input type.
Required inputs	Inputs that must be provided for the selected operation to execute.

Parameter	Description
Optional inputs	Additional inputs that may be required depending on the specific operation chosen.
Outputs	The output variables that will be returned after the operation is performed.

6.4.5 Inputs Description

The inputs for the action will be the ones declared in the 'Required inputs' and 'Optional inputs' section. The specific inputs required depend on the selected input type and operation.

General Inputs

Parameter	Description
Input type	Select the type of input - String, Array, Hash
Operation	The operation to be performed on the input

String Operations - Additional Inputs

Parameter	Description
Input	The input string on which the selected operation is performed
Pattern(String/Regexp)	This input is required only for Split, Compare, Sub and Gsub operations. If string is used, enter it within quotes. Pattern can also be given as Regexp
Replace by	This input is required only for Sub and Gsub operations

Array Operations - Additional Inputs

Parameter	Description
Array1	The input array on which the selected operation is performed
Array2	This input is required only for Concat operation

Hash Operations - Additional Inputs

Parameter	Description
Input	The input hash on which the selected operation is performed
Key	This input is required only for 'Get Value for a Key' operation

6.4.6 Outputs Description

The outputs optionally declared in the 'Outputs' section will be returned with their specified names and types.

Parameter	Description
output	The processed output based on the operation performed. The type and content depend on the input type and operation selected

6.4.7 Supported Actions

None

6.4.8 Dependencies

None

6.4.9 Operating Instructions

This plug-in provides comprehensive data manipulation operations for strings, arrays, and hashes.

Available String Operations

- Split - Divides a string into an array based on a pattern (string or regexp). Requires: Input, Pattern.
- Compare - Compares a string with a pattern and returns true/false. Requires: Input, Pattern.
- Sub - Substitutes the first occurrence of a pattern with a replacement string. Requires: Input, Pattern, Replace by.
- Gsub - Globally substitutes all occurrences of a pattern with a replacement string. Requires: Input, Pattern, Replace by.
- Upcase - Converts all characters in the string to uppercase. Requires: Input.
- Downcase - Converts all characters in the string to lowercase. Requires: Input.
- Capitalize - Capitalizes the first character of the string. Requires: Input.

Available Array Operations

- Concat - Concatenates two arrays into a single array. Requires: Array1, Array2.
- ConvertToString - Converts the given array into a string. Requires: Array1.
- ConvertToHash - Converts the given array into a hash. Requires: Array1.

Available Hash Operations

- Get Value for a Key - Retrieves the value associated with a specific key. Requires: Input, Key.
- Get Keys - Returns an array of all keys in the hash. Requires: Input.
- Get Values - Returns an array of all values in the hash. Requires: Input.
- Hash to Array - Converts the given hash into an array. Requires: Input.

Common Use Cases and Examples

- Email domain extraction: Use Split operation with pattern `"@"` on input `"user@example.com"` to get `["user", "example.com"]`
- Text normalization: Use Gsub operation to replace multiple spaces with single space, pattern `/\s+/`, replace by `" "`
- Array merging: Use Concat operation to combine two arrays, e.g., `[1,2,3]` + `[4,5,6]` = `[1,2,3,4,5,6]`
- Configuration lookup: Use Get Value for a Key operation to retrieve values from configuration hashes
- Data validation: Use Compare operation to validate string patterns, e.g., checking if an email contains `"@"`

Best Practices

- Choose the appropriate input type (String, Array, Hash) before selecting operations.
- Use regular expressions for complex pattern matching instead of simple strings when needed.
- Test patterns with sample data to ensure correct matching behavior.
- Use Gsub instead of Sub when you need to replace all occurrences of a pattern.
- Validate that required inputs are provided based on the selected operation.
- Chain multiple Input Manipulation steps for complex transformations.
- Use descriptive output variable names for better workflow readability.

6.5 Stored parameter set



Figure 6.5: Stored parameter set icon

This action plug-in provides the ability to set the values of multiple parameters within a workflow step. It can be used to store default values or apply a predefined set of values for reuse across workflows. This helps reduce duplication and

ensures consistency when configuring actions.

Version	Comment
0.6.5	Enhanced action template UI
0.6.4	Fix issues when storing strings with single/double quotes
0.6.3	Make parameter field columns expandable
0.6.2	Add support for password type parameters
0.6.1	Fix issues when storing certain string characters (Like colons)
0.6.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.5.6	Fixes issue with gsub in input
0.5.5	Fixes issue when removing a single parameter
0.5.4	Fixes dynamic parameters with single quotes
0.5.3	Fixes configurator UI affecting Orchestrator 3+ deployment using ruby 2.3 preventing the use of dynamic attributes
0.5.2	Ensures that stored YML can be viewed even if escaped characters are present
0.5.1	Added backward capability to view external stored YML
0.5.0	Added capability to view external stored YML
0.4.3	Fixes odd issue for windows path finishing with \\
0.4.2	Enable safe export/import when using external data source
0.4.1	Fixes typo in editor, rename JS function to avoid collisions
0.4.0	New icon
0.3.9	Added examples for the two YAML formats when using an external configuration file
0.3.8	Fixes naming pop-up triggering
0.3.7	Added support for Array and Hash
0.3.6	Matching outputs hash to the outputs spec for external parameters file
0.3.5	Added extra condition for when the YAML file is not Marshalled
0.3.4	Added more verbiage in help file for usage of external configuration file
0.3.3	Prevents the use of special characters for parameter names in the configurator screen
0.3.2	Fixed problems with windows paths with / as last character
0.3.1	Fixed problems with windows paths and single quote characters. Added support for dynamic inputs.
0.3.0	Increase the DB size for the parameter length. Added Help file
0.2.1	Fixes non-external storage output definition
0.2.0	Adds external storage option
0.1.3	Extracts read and write from the execute method.
0.1.2	Adds display name
0.1.1	fixed template layout
0.1.0	initial version

6.5.1 How It Works

- Centralizes workflow configuration in one place, similar to a configuration file.
- Allows defining any number of parameters that will be exposed in the workflow using their names.
- At run time, the specified values for each parameter are used.
- It is a best practice to create one or more Stored parameter set templates containing values specific to the runtime environment (e.g., IP addresses, directory paths, usernames).
- Templates can be reused across workflows; changing the template updates all workflows consistently.
- Optional: Users can enable *Use external storage* to read parameters from a YAML configuration file stored under:

  /tmp/mass_parameter_setters.

6.5.2 Configuration Instructions

1. Add a Stored parameter set step to your workflow.
2. Choose one of the following configuration options:
   - Internal parameters: Add parameters directly in the step (Name, Value, Type).
   - External storage: Enable 'Use external storage?' and specify a definition file path located in `/tmp/mass_parameter_setters`.

3. If using internal parameters, provide for each:
   - Name: The parameter name exposed in the workflow.
   - Value: The default runtime value.
   - Type: The parameter type (string, integer, etc.).
4. If using external storage, the config file must be in YAML format:
   - `key: value`
   - `key1: value1`
   - `key2: value2`
5. Advanced usage: YAML can be generated using Ruby's `to_yaml` on objects:
   - `h = { key1: "value1", key2: "value3", key: "value", key4: [1,2,3] }`
   - produces:
   - ---\ :key1: value1\ :key2: value3\ :key: value\ :key4:\ - 1\ - 2\ - 3

6.5.3 Example Configuration

Type	Field	Value
String/Text	Username	`SampleUser`
Date/Time	StartDate	`2025-01-01`
Flag/Boolean	IsActive	`True`
Numeric/Integer	Age	`30`
Hash	Regions	`{"NewYork":"HQ","London":"Branch","Tokyo":"APAC"}`
Array	AllValues	`["SampleUser","2025-01-01","True","30","HQ"]`

Behavior: With the example above, the workflow exposes multiple parameters (Username, StartDate, IsActive, Age, Regions, and AllValues) with their respective values. Any step in the workflow can reference these parameters at run time.

6.5.4 Saved Parameters Description

Parameter	Description
Use external storage?	Allows parameters to be read from an external YAML file located in `/tmp/mass_parameter_setters`.
Name	Path of the parameters definition file (when using external storage).
Add Parameter	Adds a new parameter directly to the list (used when external storage is not selected).
Name	For any new parameter: the parameter name exposed in the workflow.
Value	For any new parameter: the parameter value.
Type	For any new parameter: the parameter type (string, integer, etc.).

6.5.5 Inputs Description

Inputs depend on the configuration:

- External storage enabled
  - Config name: File path of the parameters definition file (overrides the saved file path).
- Internal parameters — No additional inputs at run time; values are taken from the saved configuration.

6.5.6 Outputs Description

- All configured parameters: Every parameter defined (internally or from external storage) is exposed as a workflow output.

6.5.7 Supported Actions

None

6.5.8 Dependencies

No external dependencies, except when using external storage (requires access to /tmp/mass_parameter_setters).

6.5.9 Troubleshooting

- If the step cannot find the external file, ensure it is placed in `/tmp/mass_parameter_setters` and accessible to Orchestrator.
- YAML syntax errors will prevent parameters from loading; validate the file with a YAML linter.
- When using internal parameters, verify that names are unique to avoid conflicts.
- If workflows do not reflect changes, confirm that the template was updated and re-saved.

6.6 Password generator



Figure 6.6: Password generator icon

This plugin provides the ability to generate random strings that can be used as passwords. The length and constraint of the generated random strings can be configured. Generated passwords can optionally be encrypted for secure storage.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.7	Added styling to parameter help text.
0.0.6	Added Help File
0.0.5	2015-03-09 ML - New icon
0.0.4	Changed output type to string
0.0.3	fix typo in edit screen
0.0.2	avoids constraints change collisions
0.0.1	initial release - basic with no import, no recurring

6.6.1 How It Works

- Generates secure passwords based on user-defined character sets and constraints.
- Supports customizable password length and character requirements.
- Applies optional constraints to ensure password complexity and security requirements.
- Provides encrypted password storage option for enhanced security.

6.6.2 Configuration Instructions

1. Add a PasswordGenerator step to your workflow.
2. Configure the basic parameters:
   - Name: Enter a descriptive name for the password generator.
   - Comments: Add notes or description for this step.
3. Set up character and length requirements:
   - Valid Characters: Optional field to choose the list of valid characters that the password can contain.
   - Password Length: Optional field to specify the length of the password to be generated.

4. Configure constraints and security options:
    - Constraints: Any constraints on the password can be given here. It will be considered while creating passwords.
    - No Constraints: Check this option if you do not want any constraints applied while generating the password.
    - Keep Encrypted: On choosing this, the password is kept encrypted and stored in an encrypted format in the DB.

6.6.3 Example Configuration

Parameter	Example Value
Name	Secure Password Generator
Comments	Generates passwords for user accounts
Valid Characters	ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!@#$%
Password Length	16
Constraints	ABCDEFGHIJKLMNOPQRSTUVWXYZ (Note: This means that the password must contain at least one uppercase letter.)
No Constraints	unchecked
Keep Encrypted	checked

6.6.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved PasswordGenerator configured instance.
Comments	Some comments about this saved PasswordGenerator configured instance.
Valid Characters	An optional field to choose the list of valid characters that the password can contain.
Constraints	An optional field to specify characters that the generated password must contain.
No Constraints	Check this option if you do not want any constraints applied while generating the password.
Password Length	This optional field allows the user to decide the length of the password to be generated.
Keep Encrypted	On choosing this, password is kept encrypted and stored in an encrypted format in the DB.

6.6.5 Inputs Description

All inputs are optional. Some of the inputs are pre-populated with generic values which can be updated or removed if needed.

- Valid Characters: List of valid characters that the password can contain.
- Password Length: Desired length of the generated password.
- Constraints: Constraints to apply during password generation.
- No Constraints: Boolean flag to disable all constraints.
- Keep Encrypted: Boolean flag to store password in encrypted format.

6.6.6 Outputs Description

- Generated Password: The password generated by the plugin based on the specified parameters and constraints if encryption is not enabled.
- Encrypted Generated Password: The password generated by the plugin based on the specified parameters and constraints if encryption is enabled.

6.6.7 Supported Actions

None

6.6.8 Dependencies

- Database access if 'Keep Encrypted' option is enabled for storing encrypted passwords.

6.6.9 Operating Instructions

6.6.10 Overview

This is a simple Password Generator which generates passwords based on the list of valid characters provided. If the generated password has characters that meet the constraints listed, then further processing is done to ensure compliance and randomize the password further.

6.6.11 Character Set Configuration

- Default character set: If no valid characters are specified, the plugin uses a default set of alphanumeric characters.
- Custom character set: Specify any combination of letters, numbers, and special characters to create passwords from a specific character pool.
- Character exclusion: You can exclude ambiguous characters (like 0, O, l, 1) by simply not including them in the valid characters list.

6.6.12 Password Length

- Specify the desired password length in the Password Length field.
- If no length is specified, a default length of 8 characters will be used.
- Longer passwords (16+ characters) are recommended for better security.
- Consider your application's password requirements when setting the length.

6.6.13 Constraint Configuration

- No Constraints mode: When 'No Constraints' is checked, the password is generated purely from the valid character set without any complexity requirements.
- With Constraints: When a constraint is added, the password is generated to include at least one character from the constraint string. For example, if the constraint is 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', the password will contain at least one uppercase letter.
- Constraint validation: The plugin validates the generated password against the specified constraints and regenerates if necessary.
- Example constraints: 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', '0123456789', '!@#$%'.

6.6.14 Encryption and Storage

- Keep Encrypted enabled: When checked, the generated password is encrypted before being stored in the database.
- Encryption benefits: Provides an additional layer of security for password storage and retrieval.
- Use case: Enable encryption when passwords need to be stored for later retrieval or when compliance requirements mandate encrypted storage.
- Output format: The output will still be the encrypted version of the password. For further use in subsequent steps, the password can be decrypted using the `OrchCipher.decrypt` command.

6.6.15 Password Generation Process

- The plugin first generates a random password using the specified valid characters.
- If constraints are defined, the password is validated against those constraints.
- If validation fails, the password is regenerated until it meets all specified constraints.
- Once a valid password is generated, it is optionally encrypted if 'Keep Encrypted' is enabled.

6.6.16 Best Practices

- Use a minimum password length of 12-16 characters for better security.
- Include a diverse character set with uppercase, lowercase, numbers, and special characters.
- Define constraints that match your application's security requirements.

- Enable encryption when passwords need to be stored in the database.
- Avoid using easily guessable patterns in your constraint definitions.
- Test the generated passwords to ensure they meet your password policy.
- Consider excluding ambiguous characters (0, O, l, 1, I) to prevent confusion.

6.7 Regex matcher



Figure 6.7: Regex matcher icon

This action plug-in provides the ability to parse a regex expression and extract parts of it

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.4	Exposes RegEx Pattern as runtime input
0.0.3	New icon
0.0.2	Fix issues with additional outputs
0.0.1	Initial release

6.7.1 How It Works

- This action evaluates a regular expression against an input expression and extracts matching values.

6.7.2 Configuration Instructions

1. Create a new action step using the RegexMatcher plugin.
2. This action evaluates an input expression with a specified regular expression.
3. Fill out the following configuration fields:
   - Name: The name used to identify a saved RegexMatcher configuration.
   - Comments: Optional comments about this saved RegexMatcher configuration.
   - Expression: The input string or expression to be evaluated by the regex.
   - Regex Pattern: The regex pattern used to evaluate the expression.
   - Additional Outputs: By default, an output for the full match is provided, plus one output per regex capture group. You may define named outputs that are constructed from capture groups using placeholders (e.g., '\1' for group 1). Note: only up to 9 capture groups are supported.

6.7.3 Example Configuration

This is a minimal setup configuration for the RegexMatcher plugin.

Parameter	Value
Expression	123456789/AB1234/expenseReport_1234.txt
Regex Pattern	([0-9]*)/[A-Z]{2}([0-9]*)/(.*)\..*
Additional outputs - Asset_ID	A\1-\3

Parameter	Value
Additional outputs - User_ID	\2

6.7.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved RegexMatcher configuration.
Comments	Optional comments about this saved RegexMatcher configuration.
Expression	The input string or expression to be evaluated by the regex.
Regex Pattern	The regex pattern used to evaluate the expression.
Additional Outputs	By default, an output for the full match is provided, plus one output per regex capture group. You may define named outputs that are constructed from capture groups using placeholders (e.g., '\1' for group 1). Note: only up to 9 capture groups are supported.

6.7.5 Inputs Description

Required inputs depend on the action template configuration. If dynamic attributes are used, each attribute becomes a runtime input.

Parameter	Description
Expression *	The expression to be evaluated by the regex.
Regex_pattern *	The regex pattern used to evaluate the expression.

You can build the expression from multiple runtime inputs using dynamic variables. For example: `"<%= root_folder %>/ingest/<%= File.basename(file_path) %>"` - in that case, 'root_folder' and 'file_path' become inputs.

6.7.6 Outputs Description

Parameter	Description
Match	The substring matching the regex pattern.

By default, an output named 'Group_N' is provided for each capture group. If you define custom outputs, those replace the default group outputs. All outputs are returned as strings.

6.7.7 Supported Actions

None

6.7.8 Dependencies

None

6.7.9 Operating Instructions

1. Provide the input expression to evaluate.
2. Provide the regex pattern to apply to the expression.
3. Optionally define additional named outputs constructed from capture groups.
4. Run the action. If the expression matches the pattern, outputs are created for the match and capture groups; otherwise the action fails.



Figure 6.8: Shared state operation icon

6.8 Shared state operation

This action plug-in can be used to create and retrive shared state objects.

Version	Comment
0.5.1	Enhanced action template UI
0.5.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.4.0	adding delete operation
0.3.0	adding comments field
0.2.4	Fixes to accept object inputs and not just string
0.2.3	Fixes internal code to provide correct error message
0.2.2	Fixes rendering of entry type in configuration UI
0.2.1	Allows runtime typed input for creation
0.2.0	Allows retrieval of non-hash entries
0.1.0	Fixes logic to use run-time inputs when present
0.0.2	Entry returns the entry, not a hash with single entry containing the hash
0.0.2	fixing plugin file name
0.0.1	initial release

6.8.1 How It Works

- This action plug-in can be used to create, delete and retrieve shared state objects.
- Based on the selected operation, the available input and output options will change

6.8.2 Configuration Instructions

1. Create a new action step using the SharedStateOperation plugin.
2. The plugin uses different inputs based on the operation.
3. Fill out the following fields for Create Shared State:
   - Name: The name used to identify a saved SharedStateOperation configured instance.
   - Comments: Comments describing this configured instance.
   - Entry: State info of any type (Hash, Int, Array, String...etc) that would be stored as a BLOB in the database.
     - NOTE: String entries should be within quotes.
   - Full Path: Full path of the state info - Path + Entry Name.
     - If this input is given, the Path and Entry Name inputs will not be considered for searching the shared states for a match.
   - Aggregate Type: A value used to group shared states.
   - Aggregate ID: An identifier used to group shared states.
4. Fill out the following fields for Delete Shared State or Retrieve Shared State:
   - Name: The name used to identify a saved SharedStateOperation configured instance.
   - Comments: Comments describing this configured instance.
   - Retrieve Multiple: Enables retrieval of multiple shared states based on the given inputs.
   - Shared State ID: The database ID of the shared state record.
   - Path: The path of the shared state.
   - Entry Name: The name of the shared state entry.

- Full Path: Full path of the state info - Path + Entry Name.
  - If this input is given, the Path and Entry Name inputs will not be considered for searching the shared states for a match.
- Aggregate Type: A value used to group shared states.
- Aggregate ID: An identifier used to group shared states.

6.8.3 Example Configuration

This is a minimum setup configuration for the SharedStateOperation plugin.

Parameter	Value
Operation	Create Shared State
Entry	300
Full path	/home/state

6.8.4 Saved Parameters Description

6.8.5 Inputs Description

The list of inputs depends on the configuration of the SharedStateOperation action template. If dynamic attributes are used in the message definition, each attribute becomes a required input.

Create Shared State

Retrieve Shared State

Delete Shared State

6.8.6 Outputs Description

Single/Default Retrieve

Parameter	Description
aggregate_id	Aggregate id if present in the record created, deleted or retrieved.
aggregate_type	Aggregate type if present in the record created, deleted or retrieved.
entry	The shared State.
entry_type	The data type of the shared State created, deleted or retrieved.
id	The ID for the record created, deleted or retrieved.
name	The name of the shared state.
path	The path of the directory which contains the shared state info.

Multiple Retrieve

Parameter	Description
Shared State(s)	Array of hashes containing the shared states.
Number of shared states retrieved	Count of the Shared State(s) output.

6.8.7 Supported Actions

None

6.8.8 Dependencies

None

6.8.9 Operating Instructions

- Select the Create operation to create a new shared state. Specify the entry data and full path where the state information should be saved.
- Select the Retrieve operation to retrieve an existing shared state. Provide the shared state ID, full path, or other identifying information to locate the state.
- Select the Delete operation to remove a shared state. Provide the shared state ID, full path, or other identifying information to locate and delete the state.
- Use aggregate_type and aggregate_id to group related shared states for easier management and retrieval.

6.9 Look-up table



Figure 6.9: Look-up table icon

This plugin provides a framework to associate miscellaneous values to parameters at design time and retrieve these associated values at runtime based on input parameter values.

Version	Comment
0.8.8	Enhanced action template UI and added default value for return_array_of_rows column
0.8.7	Ensure column width is restored on switching tabs
0.8.6	Fixes bug where first row with any matching parameter is returned instead of requiring all parameters to match
0.8.5	Fixes bug preventing wildcard patterns from matching in glob expressions
0.8.4	Make columns expandable
0.8.3	Fixes CSV import issue with different encoding formats
0.8.2	Fixes CSV import routing
0.8.1	Fixes issues with partially empty row or column
0.8.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.7.0	Adds option to return array of rows
0.6.0	Fixes problem when importing CSV causing logout
0.5.9	Fixed problem when having empty cells
0.5.8	Fixed output when specifing KeyForTableIndexing
0.5.7	Fixed double-quote escaping issue
0.5.6	Fixed edge case for CVS export due to Ruby migration
0.5.5	Fixed to CVS export due to Ruby migration
0.5.4	added raw to parameters
0.5.3	Tweaked so that it works in testing utility
0.5.2	Adding support for typed inputs
0.5.1	Fixes missing string outputs issue introduced in 0.5.0
0.5.0	Adding support for typed outputs
0.4.0	adding comments section
0.3.7	Fixed the 'new empty lines' bug. Some csv formats add a new empty line for each line present in the csv.
0.3.6	Added support for run time defined variables

Version	Comment
0.3.5	Fix import problem from CSV with special characters
0.3.4	New icon
0.3.2	Fixes offset bug when removing multiple rows or columns
0.3.1	Removes glob and regex switch from inputs_spec
0.3.0	Adds explicit switch for glob and regex
0.2.5	Fixed problem with Glob expressions
0.2.4	Adds option to use RegEx matching
0.2.3	Adds option to list entire table as a hash
0.2.2	Fixes validation for edge case of dynamic param file
0.2.1	Changes status message for lookup failure
0.2.0	Increased DB size for associations
0.1.5	Only allow import/export if configuration is saved (an ID is needed)
0.1.4	Fixes treatment of single quote and windows slashes in field values
0.1.3	Disables input grid in when in view only mode
0.1.2	Bug fix on used with external data sheet
0.1.1	Bug fix, making strip spaces optional
0.1.0	Adds support for CSV and option to strip lead and trailing spaces
0.0.7	Adds display name
0.0.6	added ability to set parameters to use for the lookup when file is an input
0.0.5	added outputs as hash when file path is a runtime parameter
0.0.4	added parameters file path as input value
0.0.3	image folder changed to support multi-tenancy
0.0.2	fix issue with loading null parameter tables

6.9.1 How It Works

- Performs parameter lookups using configurable input/output mapping tables to retrieve associated values based on input criteria.
- Supports multiple mapping definition methods including dynamic table editor, external files, and runtime specifications.
- Provides flexible matching capabilities including case sensitivity, GLOB patterns, and regular expressions for enhanced lookup functionality.
- Enables both single and multi-value mappings with support for complex primary keys defined by multiple input values.
- Offers specialized features like typed outputs, array returns, column value enumeration, and table indexing for advanced use cases.

6.9.2 Configuration Instructions

1. Add a SimpleParameterLookup step to your workflow.
2. Set up the basic parameters:
   - Param Filename: Optional absolute path to external mapping file.
   - Case Sensitive Lookup: Enable for case-sensitive parameter matching.
   - Support typed outputs: Enable to convert column types (e.g., expiration_date → expiration as date type).
3. Configure the mapping table using the dynamic editor:
   - Use Add Row and Add Column to define input/output associations.
   - First row defines column names (input and output variable names).
   - Subsequent rows contain the actual mapping values.

6.9.3 Example Configuration

Parameter	Example Value
Param Filename	/opt/aspera/config/user_mapping.txt
Case Sensitive Lookup	unchecked
Disable GLOB matching	unchecked
Disable REGEX matching	unchecked
Support typed outputs	checked
Return array of rows	unchecked

6.9.4 Saved Parameters Description

Parameter	Description
Param Filename	The absolute path to the file defining the parameter mapping, if available.
Case Sensitive Lookup	Specifies if the input parameter lookup should be case sensitive.
Disable GLOB matching	By default, the characters * and ? are interpreted with their GLOB meaning, that behavior is disabled by this flag.
Disable REGEX matching	By default, values surrounded by / are interpreted as REGEX, that behavior is disabled by this flag.
Support typed outputs	If selected, then any outputs from a column that is suffixed by a type name will be converted to that type and the output name will lose the suffix. For example, if the column is called expiration_date, the output will be name 'expiration' and will be of type 'date'.
Return array of rows	If selected, then all rows will be returned into array of hashes. Simple or multi-values mappings can also be used to get only matching rows.

6.9.5 Dynamic Mapping Definition

A dynamic table editor is available to define input/output mapping rules using the *Add Row*, *Add Column* buttons. See Operating Instructions for more details.

6.9.6 Inputs Description

- ParametersAssociation: Specification of the input/output associations. If set by a previous step it overrules both the dynamic definition in the template and the one in the definition file.
- ColumnToListValueFrom: The name of the column to retrieve all available values for. If set the step returns as output all values in the given column.
- KeyForTableIndexing: The name of the column to be used to index the table by rows. If provided, no lookup will be performed, instead the entire table will be returned as a Hash of rows, with each row represented as a hash.

Both inputs are optional, and usually they can be left unset (see examples in Operating Instructions for specific use cases).

6.9.7 Outputs Description

- ColumnValues: The list of available values for the column specified in ColumnToListValueFrom (if any).
- Mapped outputs: Based on the input/output definition all associated values are exported as output. The output name is the name of the column.

6.9.8 Operating Instructions

6.9.9 Example of a Simple Mapping

The mapping between a user name (input) and his email address (output) is a table with two columns. The first row defines input and output names:

name	email

The step has *name* available in the input mappings and *email* exported as output to the next step. Each defined user is in a row with the associated email address:

name	email
John Doe	john.doe@mycompany.com
Jane Doe	jane.doe@mycompany.com

At run time the step tries to match the value of the input variable *name* with values in the first column: if there is a match, the *email* output gets the second column value.

6.9.10 Example of Multi-values Mapping

Associations with multiple output values are supported:

source_path	destination_node	destination_path

In this example *source_path* has two associated outputs: *destination_node* and *destination_path*. Both outputs are available to the next step in the workflow.

6.9.11 Example of Multi-inputs Mapping

Associations with multiple input values are also supported: considering the previous table, it is possible to specify both source_path and destination_node using an input map and retrieve destination_path as output. This behavior allows implementing lookups where the primary key is defined by multiple values.

6.9.12 How to Define the Mapping Using a File

It is possible to specify the association logic using a file: this can be useful if an external process dynamically generates the logic (example an active directory server). The file has to match a simple format where the column separator is ":::" and the row separator is "\n".

Example of a mapping file:

The mapping logic used in the previous example can be defined in a file as follows:

```
source_path:::destination_node:::destination_path\n
```

incoming\provider_a:::server_london:::incoming\from_provider_a\n

incoming\provider_b:::server_nyc:::incoming\from_provider_b

The logic defined by the file is overruled by any other run time specified logic. So if in the template there is a file specified and a not empty table, the table is used.

6.9.13 How to Define the Mapping at Run Time

In a workflow a step might be responsible for creating the association logic at run time: it is possible to use the generated associations by providing the logic as input in *ParametersAssociation*. The definition syntax is the same as the one used for the mapping file.

The logic defined using *ParametersAssociation* overrules all other definitions. So if this variable is not empty both the template table and the file are ignored.

6.9.14 How to Retrieve All Possible Values for a Column

For cases with a run time defined mapping it might be useful to know all possible values of a particular column. Example: In a particular workflow one step prompts the list of all valid customers to the user for selection. The list of valid customers generates from an external accounting system that is not part of the workflow: every time the accounting system has a new customer it regenerates a file with all necessary information using the mapping syntax:

```
customer_name:::customer_email:::.....\n
```

To retrieve the list of all customer names at run time instantiate a SimpleParameterLookup step pointing to the generated file and assign the value *customer_email* to *ColumnToListValueFrom*. At run time the step produces the list of all available customers in *ColumnValues*.

Listing all values of a column works in parallel with the mapping logic: it is possible to have a single step returning both the list of all values for a column and the list of all outputs mapped to the specified input.

6.9.15 Example of Use of the KeyForTableIndexing

When this optional setting is provided, the action instead of doing a lookup, will provide the entire table of data it represents. That data is returned as a Hash indexed by the value of the column specified in the KeyForTableIndexing runtime input field. The values in this return hash each represent a table row, with row information provided as a Hash of name (the column name), value (the value specified for that column/row) pair.

If KeyForTableIndexing received the value ”node_id”, the only output provided by the action will be IndexedTable. The value will be of type Hash:

node	node_id	capacity_kb
Transcoder 1	ffmpeg1	20000
Transcoder 2	localhost	10000

IndexedTable: { 'ffmpeg1' => { 'node' => 'Transcoder 1', 'node_id' => 'ffmpeg1', 'capacity_kb' => '20000' }, 'localhost' => { 'node' => 'Transcoder 2', 'node_id' => 'localhost', 'capacity_kb' => '10000' } }

6.9.16 Use of RegEx and GLOB Matching

The plugin supports both RegEx and GLOB matching. In order to specify a RegEx as column value, the RegEx has to be specified within slashes. Example: /.*ab.*/ will match any string containing ”ab”.

For GLOB matching it is enough to specify a GLOB expression. Example: /opt/aspera/*/action will match any string starting with /opt/aspera/ and ending with /action.

6.9.17 Dependencies

- If using external mapping files, the aspweb user must have read permissions for the specified file paths.
- When using runtime-generated mappings, ensure upstream steps provide properly formatted association data.

6.10 Timecode manager



Figure 6.10: Timecode manager icon

Allows timecodes manipulations like conversions, sums or differences.

Version	Comment
0.3.1	Enhanced action template UI
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.2	No longer supporting ruby 1.8
0.2.1	Adds millisecond count as input and output.
0.2.0	Add option to only get frame count as output value. Add fix frame rate for timecode output value.
0.1.5	Fixing Ruby warnings caused by Ruby 2+
0.1.4	Refining checking of Timecode ruby lib
0.1.3	Adding runtime selection for Frame Rate. Added support for more frame rate specification and custom frame rate as input.

Version	Comment
0.1.2	Removing the gem check at the top of the plugin. If the gem is not installed, the plugin will fail but will not pollute the logs with warnings
0.1.1	Adding a rescue around the require statement to save from exceptions
0.1.0	Added NTSC support
0.0.1	Initial release

6.10.1 How It Works

- This plugin performs operations on time values expressed as timecode, frame count, or millisecond count.
- It converts between timecode, frame count, and millisecond count (optionally using an initial offset) and supports addition/subtraction operations between two time values.
- Important: By default, the plugin assumes a frame rate of 25 fps; other standard frame rates are supported.

6.10.2 Configuration Instructions

1. Create a new action step using the TimecodeManager plugin.
2. This action performs timecode-related computations as configured.
3. Fill out the following configuration fields:
    - Name: The name used to identify a saved TimecodeManager configuration.
    - Comments: Optional comments about this saved TimecodeManager configuration.
    - Operation: Select which time operation to execute (convert, add, subtract, etc.).
    - Input Value Format: Select whether inputs are provided as timecode, frame count, or millisecond count.
    - Start Timecode, Frame Count, or Millisecond Count: The first input value for the operation.
    - First Frame Offset: The frame number to treat as time zero (default: 0).
    - Second Timecode, Frame Count, or Millisecond Count: Optional second input for operations that require two values.
    - Only Get Frame Count and Millisecond Count as Output Values: When enabled, only frame count and millisecond outputs are returned (timecode output omitted).

6.10.3 Example Configuration

This is a minimal setup configuration for the TimecodeManager plugin.

Parameter	Value
Operation	Convert
Frame rate	25 fps
Input value format	Timecode
Start timecode or frame count or millisecond count	00
First frame offset	0

6.10.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved TimecodeManager configuration.
Comments	Optional comments about this saved TimecodeManager configuration.
Operation	Select which time operation to execute (convert, add, subtract, etc.).
Input Value Format	Select whether inputs are provided as timecode, frame count, or millisecond count.
Start Timecode, Frame Count, or Millisecond Count	The first input value for the operation.

Parameter	Description
First Frame Offset	The frame number to treat as time zero (default: 0).
Second Timecode, Frame Count, or Millisecond Count	Optional second input for operations that require two values.
Only Get Frame Count and Millisecond Count as Output Values	When enabled, only frame count and millisecond outputs are returned (timecode output omitted).

6.10.5 Inputs Description

Required inputs depend on how the TimecodeManager template is configured. Dynamic attributes used in the message definition will become required runtime inputs.

Parameter	Description
Start_value *	The initial time value used by the operation.
First_frame_offset	The frame value treated as time zero (default: 0).

6.10.6 Outputs Description

Parameter	Description
frame_count	Result expressed as a frame count.
timecode	Result expressed as a timecode string.
ms_count	Result expressed as a millisecond count.

6.10.7 Supported Actions

None

6.10.8 Dependencies

None

6.10.9 Operating Instructions

1. Select the operation and input format (timecode, frames, or milliseconds).
2. Provide the required input values and, if needed, the second value for binary operations (add/subtract).
3. Optionally set the first-frame offset (the frame treated as time zero).
4. If you want the frame rate configurable at runtime, set the frame rate to -Select at Runtime- in the action configuration.

Using custom Frame Rate:

The plugin supports specifying the frame rate at runtime. Provide values such as '19.00' or '19.00 fps'. To make the frame rate configurable at runtime, select -Select at Runtime- in the action configuration.



Figure 6.11: UUID Generator icon

6.11 UUID Generator

This action plug-in provides the ability to generate unique ID formatted ”39fa4e20-0f32-0132-d394-282066598c75”.

Version	Comment
0.3.1	Enhanced action template UI
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.2	Added to_i to scramble part
0.2.1	Added styling to parameter help text.
0.2.0	Added optional randomization of UUID
0.1.3	Added uuid info to help file
0.1.2	Added help file
0.1.1	Adding require statement
0.1.0	2015-02-18 PM - Adding compact flag to get rid of - in the UUID
0.0.2	2015-02-18 ML - New icon
0.0.1	initial release

6.11.1 How It Works

- A UUID is 128 bit long, and consists of a 60-bit time value, a 16-bit sequence number and a 48-bit node identifier.
- The time value is taken from the system clock, and is monotonically incrementing.
- However, since it is possible to set the system clock backward, a sequence number is added. The sequence number is incremented each time the UUID generator is started.
- The combination guarantees that identifiers created on the same machine are unique with a high degree of probability.

6.11.2 Configuration Instructions

1. Add a UUID Generator step to your workflow.
2. Fill out the following fields:
   - Compact version: If chosen, the UUID generated will not have dashes.
   - Randomize: If chosen, the UUID generated will be scrambled to avoid looking sequential.

6.11.3 Example Configuration

Parameter	Value
Compact version	Selected
Randomize	Selected

This configuration creates a shorter UUID without dashes, and consequent IDs are not sequential.

6.11.4 Saved Parameters Description

Parameter	Description
Compact version	If chosen, the UUID generated will not have dashes.
Randomize	If chosen, the UUID generated will be scrambled to avoid looking sequential.

6.11.5 Inputs Description

None

6.11.6 Outputs Description

UUID: This will give the UUID that was generated

6.11.7 Supported Actions

None

6.11.8 Dependencies

None

6.11.9 Operating Instructions

Use the plugin in a workflow when a UUID/GUID type of a string is required. If dashes are not required, check the Compact version flag.

Notes about UUID:

- UUID (universally unique identifier) is guaranteed to be unique across time and space.
- A UUID is 128 bit long, and consists of a 60-bit time value, a 16-bit sequence number and a 48-bit node identifier.
- The time value is taken from the system clock, and is monotonically incrementing. However, since it is possible to set the system clock backward, a sequence number is added. The sequence number is incremented each time the UUID generator is started. The combination guarantees that identifiers created on the same machine are unique with a high degree of probability.
- Note: Due to the structure of the UUID and the use of a sequence number, there is no guarantee that UUID values themselves are monotonically incrementing. The UUID value cannot itself be used to sort based on order of creation.
- To guarantee that UUIDs are unique across all machines in the network, the IEEE 802 MAC address of the machine's network interface card is used as the node identifier.

6.12 Watch file movement



Figure 6.12: Watch file movement icon

This action plug-in asynchronously monitors a specified file location and detects when the file is moved by periodically checking its existence.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.1	Initial implementation

6.12.1 How It Works

- Monitors a specific file to detect when it moves or is deleted from its original location.
- Operates in asynchronous mode, periodically checking for file existence.
- Initially verifies that the specified file exists at the given path.
- Polls the file location at a configurable polling frequency (default: 10 seconds).
- Workflow remains in progress as long as the file exists at the specified location.
- Completes successfully when the file is moved, deleted, or no longer accessible at the original path.
- Returns the filename when file movement is detected.

6.12.2 Configuration Instructions

1. Create a new action step using the WatchFileMovement plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved Watch File Movement configured instance.
   - Comments: Comments about this saved Watch File Movement configured instance.
   - File name: The full path to the file to monitor (e.g., /data/incoming/sample.xml or C:\temp\file.txt).
   - Polling frequency: The interval in seconds between file existence checks. Default is 10 seconds.

6.12.3 Example Configuration

This is an example configuration for the WatchFileMovement plugin monitoring a lock file.

Parameter	Value
Name	Wait for File Removal
Comments	Monitor file until it is moved or deleted
File name	/data/processing/transfer.lock
Polling frequency	5

6.12.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved Watch File Movement configured instance.
Comments	Comments about this saved Watch File Movement configured instance.
File name	The full path to the file to monitor (e.g., /data/incoming/sample.xml or C:\temp\file.txt).
Polling frequency	The interval in seconds between file existence checks. Default is 10 seconds.

6.12.5 Inputs Description

The file_name input is mandatory. If not provided or if the file does not exist at execution start, the action will fail immediately.

Parameter	Description
*File_name **	The full path to the file to monitor. This can be a local filesystem path or a path on a mounted network share. The file must exist when the action starts, or the action will fail immediately.
Polling_frequency	The interval in seconds between file existence checks. Default is 10 seconds. Lower values provide faster detection but increase system overhead. Expressed as an integer.

6.12.6 Outputs Description

Parameter	Description
File_Name	The full path of the file that was being monitored. Returned when the file is detected as moved or deleted.

6.12.7 Supported Actions

None

6.12.8 Dependencies

None

6.12.9 Operating Instructions

This plug-in monitors a specific file and completes when the file is moved or deleted from its original location.

Execution Flow

- Step 1 - Initial Validation: When the action starts, it checks if the specified file exists.
- If the file does not exist, the action exits with STATUS_FAILED and message "File not present".
- Step 2 - Begin Monitoring: If the file exists, the action enters STATUS_INPROGRESS with message "File Exists. Now looking for file movement".
- Step 3 - Polling Loop: The action periodically checks (based on polling frequency) whether the file still exists.
- While the file exists: Status remains INPROGRESS with message "File Still Exists. Now looking for file movement".
- Step 4 - Detection Complete: When the file is no longer found at the specified path, the action completes with STATUS_COMPLETE and message "File moved out of the location".
- The filename is returned in the `File_Name` output.

Common Use Cases

- Lock file pattern: Wait for a lock file to be removed before proceeding with processing. Common in file transfer workflows where a .lock file indicates ongoing transfer.
- Handoff coordination: Monitor a file until another process picks it up (moves/deletes it), ensuring proper workflow sequencing.
- Cleanup verification: Verify that temporary files or processing artifacts are properly cleaned up by other systems.
- Archive detection: Detect when files are moved to archive locations by external archival processes.
- Multi-step processing: Use as a coordination mechanism where file presence/absence signals processing stages between systems.

Configuration Guidelines

- File path format: Use absolute paths for reliability. Examples:
  - Linux/Unix: `/data/incoming/transfer.lock`
  - Windows: `C:\temp\processing.flag`
  - Network share: `\\server\share\folder\file.txt`
- Polling frequency considerations:
  - Lower values (1-5 seconds): Faster detection, higher system overhead
  - Default (10 seconds): Balanced approach for most use cases
  - Higher values (30-60 seconds): Reduced overhead for non-critical monitoring
- File must exist at start: The action requires the file to exist when it begins execution. To wait for file creation, use a different plugin or implement retry logic.
- File permissions: Ensure the aspweb user has appropriate permissions to access the file.

- Network paths: Ensure proper network connectivity and permissions when monitoring files on network shares.
- Cross-platform paths: Be mindful of path separators (/ vs \) when workflows run on different operating systems.

Best Practices

- Use descriptive filenames in the configuration to clearly indicate what is being monitored (e.g., ”transfer_complete.flag” rather than ”file.txt”).
- Set an appropriate polling frequency based on your use case - balance between detection speed and system load.
- Implement timeout logic at the workflow level if the file might never be moved (prevents indefinite waiting).
- Use this plugin for coordination and handoff scenarios rather than general file watching (file creation monitoring requires different plugins).
- Document the expected file lifecycle in the Comments field to clarify why the file is being monitored.
- Consider using lock files or flag files specifically created for coordination purposes rather than monitoring actual data files.
- Test with the actual file paths and permissions to ensure the Orchestrator process can access the monitored locations.

Chapter 7

Quality Control

7.1 ADI 3 Parser and Validator



Figure 7.1: ADI 3 Parser and Validator icon

This plugin provides ability to parse and/or validate contents of ADI 3.0 XML

Version	Comment
0.2.2	Enhanced action template UI
0.2.1	Fix regex validation issues in compare_or_match method
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.10	Don't depend on customized Rails initializers
0.1.9	Fix gatherStructValues() when allPairs is null
0.1.8	ADI Metadata updates might not contain assets info. Adjusting code to avoid exception.
0.1.7	Commenting error regarding libxml.
0.1.6	Added styling to parameter help text.
0.1.5	Made correction to a typo in the help file.
0.1.4	- 2015/05/26 - PM - Adding a new output - XSD validation error message. Also improving the XSD command line call to capture STDERR to STDOUT.
0.1.3	- 2015/05/26 - PM - Adding a new output - list of URI Ids of all classes.
0.1.2	- 2015/05/26 - PM - Removing requirement for DTD validation gem.
0.1.1	Jamuna's development release - to test at Shaw.
0.0.1	First version

7.1.1 How It Works

- This plug-in allows users to provide necessary inputs required for parsing and validating an `ADI 3.0 XML`.

7.1.2 Configuration Instructions

1. Create a new action step using the Adi3Parser plugin.

2. Fill out the following fields:
    - Name: The name used to identify a saved ADI 3 Parser and Validator configured instance.
    - Comments: Comments used to identify the ADI Parser template.
    - ADI Version: The plugin currently supports ADI v1.1 and v3.0 (as separate plugins).
    - ADI File Path: The file path to the ADI file.
    - ADI Format: Choose between XSLT and AMI formats.
    - Validate XSD?: Choose whether validation must be run against an XSD file.
    - XSD File Path: The file path to the XSD file used for validation.
    - Compare File Sizes: If checked, Orchestrator compares the file sizes of the files in ADI.XML on the filesystem with the values in ADI.XML.
    - Compare File MD5: If checked, Orchestrator compares the MD5 hashes of the files in ADI.XML on the filesystem with the values in ADI.XML.
    - Pass ADI When Missing Assets?: If checked, Orchestrator does not fail the step if assets are missing.
    - ADI Package Directory: An optional input that can be provided if the ADI package directory is different from the one where ADI.XML is located.
    - ADI Validations: Validations can be added via the widget to compare the values in AMS to the App_Data tags.

7.1.3 Example Configuration

This is an ADI Parsing and Validation template configuration example for the Adi3Parser plugin.

Parameter	Value
ADI File Path	/home/sample
Compare File Sizes	Selected
Compare File MD5	Selected
Pass ADI when missing assets?	Selected
ADI Validations - ADI Asset Class	Offer
ADI Validations - Field Name	Presentation->DisplayAsNew
ADI Validations - Field Value	P7D
Required?	Required

Note: Since ADI 3.0 supports a hierarchical field format, for validation we need to specify the tree structure when searching for inner fields. In the above example, we are searching for the 'DisplayAsNew' field.

Example:

\ <Offer uriId=”indemand.com/Offer/UNVA2001081701004000” providerVersionNum=”1”>\ <offer:Presentation>\ <offer:CategoryRef uriId=”indemand.com/Category/InDemand/Movies A-Z” />\ <offer:DisplayAsNew>P7D</offer:DisplayAsNew>\ <offer:DisplayAsLastChance>P7D</offer:DisplayAsLastChance>\ </offer:Presentation>\ </Offer>\

7.1.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved ADI 3 Parser and Validator configured instance.
Comments	Comments used to identify the ADI Parser template.
ADI Version	The plugin currently supports ADI v1.1 and v3.0 (as separate plugins).
ADI File Path	The file path to the ADI file.
ADI Format	Choose between XSLT and AMI formats.
Validate XSD?	Choose whether validation must be run against an XSD file.
XSD File Path	The file path to the XSD file used for validation.
Compare File Sizes	If checked, Orchestrator compares the file sizes of the files in ADI.XML on the filesystem with the values in ADI.XML.
Compare File MD5	If checked, Orchestrator compares the MD5 hashes of the files in ADI.XML on the filesystem with the values in ADI.XML.
Pass ADI When Missing Assets?	If checked, Orchestrator does not fail the step if assets are missing.

Parameter	Description
ADI Package Directory	An optional input that can be provided if the ADI package directory is different from the one where ADI.XML is located.
ADI Validations	Validations can be added via the widget to compare the values in AMS to the App_Data tags.

7.1.5 Inputs Description

The list of inputs depends on the configuration of the Adi3Parser action template. If dynamic attributes are used in the message definition, each attribute becomes a required input.

Parameter	Description
Adi_file_path *	Either a remote node must be selected or a remote IP address must be provided.
Adi_package_dir	Default directory is the ADI.XML's directory. Another value can be provided.
Check_Filemd5s	Default value is false, but validating the MD5 can be chosen.
Check_Filesizes	Default value is false, but validating the file sizes can be chosen.
Pass_On_Missing_Files	Default value is false, this flag provides a safe catch mechanism when files are not present during checks.

7.1.6 Outputs Description

Parameter	Description
List_of_Filenames	Names of the files in the ADI.XML
Count_of_ContentFiles	Number of files in the ADI.XML
ADI_Size_Hash	Hash with filenames as keys and their file sizes as values
ADI_MD5_Hash	Hash with filenames as keys and their MD5 hashes as values
Is_Valid_ADI	If all validations and file property checks pass, this value is true
ADI_Package_Metadata	ADI.XML metadata from the AMS tag
Filetype_list	Hash with the class name as the key and the content as the value
Filesizes_validation	This flag is true if the file sizes of the content match the file sizes on disk
FileMD5s_validation	This flag is true if the MD5 of the content matches the MD5 of the file on disk
XSD_Validation	This flag is true if the XSD validation is successful
Failed_Filesize_Assets	Array of files that failed file size validation
Failed_MD5_Assets	Array of files that failed MD5 validation
Asset_IDs_hash	Hash of Asset IDs as values for each asset class (key)
Asset_IDs_FileName_hash	Hash of Asset ID field names as values for each asset class
ADI Fields Data Hash	Hash of Asset ID field data as values for each asset class
XSD_Validation_Error_Message	Error message displayed when XSD validation fails
URI Ids List	List of all Asset URI IDs for classes

7.1.7 Supported Actions

None

7.1.8 Dependencies

None

7.1.9 Operating Instructions

This plug-in allows users to provide necessary inputs required for parsing and validating an `ADI.XML`.

Steps to populate the ADI Parsing and Validation template

- Provide a name for the action template to identify it.
- Provide a path to the ADI.XML. In a workflow, this field is available as a runtime input.
- Make sure the ADI version is 3.0.
- Check the checkboxes for file size validation and MD5 checksum validation. The MD5 checksum check is not case-sensitive.
- Check if the ADI.XML needs to be validated by an ADI.DTD file.
- Check the 'Pass on missing files' checkbox if you want the step to NOT fail when assets are missing.
- Use the Add ADI Validations widget to add a set of validations.
- To add a validation, click the Add ADI Validations widget.
- In the fields that appear, first select the appropriate asset class. The asset classes for a CableLabs ADI v3.0 XML can only be: Package, Title, Movie, Preview, Poster, Still-Image, TrickAsset, Encrypted Asset.
- Not all ADI.XML files contain all asset classes, so choose wisely what to validate.
- Next, in the Field Name text box, enter the field that needs to be validated in the chosen asset class. Familiarity with ADI.XML is needed to know the fields.
- Next, in the Field Value text box, enter a value that you want to use to validate the data.
- This value by default is a glob expression '*'. The value supports wildcards like * and ?, globbing like val*ue, and regular expressions like /.+[A-Za-z]/.
- Since ADI 3.0 supports a hierarchical field format, for validation we need to specify the tree structure when searching for inner fields.

7.2 ADI Parser and Validator



Figure 7.2: ADI Parser and Validator icon

This plugin provides ability to parse and/or validate contents of ADI XML

Version	Comment
0.5.2	Enhanced action template UI
0.5.1	Fix regex validation issues in compare_or_match method
0.5.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.4.8	Remove dependency on custom rails initializers
0.4.7	Fix gatherStructValues() when allPairs is null
0.4.6	ADI Metadata updates might not contain assets info. Adjusting code to avoid exception.

Version	Comment
0.4.5	Commenting error regarding libxml.
0.4.4	Adding new output to expose entire title class data as a Hash
0.4.3	Adding box cover to the class dropdown in the step UI
0.4.2	Added styling to parameter help text.
0.4.1	Change in the help file. Added new line tag for example templates list.
0.4.0	Adding feature to allow optional field validation. Fixed bug when deleting middle validation elements.
0.3.1	Encoding Regex pattern to escape special characters.
0.3.0	XSD validation and adding an extra output for showing the validation exception.
0.2.5	Bug Fix - validation on optional fields led to error. Fixing that.
0.2.4	Bug fix - failure of file size validation and file md5 validation, now fails the step.
0.2.3	Fixed bug where regex expressions were not evaluated correctly.
0.2.2	Fixed bug introduced in 0.2.1 where strict DTD validation was enforced. Enforce only when flag is checked.
0.2.1	2015-06-16 PM - Enforcing strict DTD validation and returning failure when DTD validation fails
0.2.0	2015-05-28 PM - Adding field for DTD file path to support file being in different directory and with a different name. Defaults to ”adi_file_dir/ADI.DTD”
0.1.1	2015-04-28 PM - Adding new output to capture more than filename asset_id combination
0.1.0	2015-02-18 ML - New icon
0.0.9	Modifications to inputs_spec and VAR_* variables to read the optional boolean inputs populated at runtime
0.0.8	Fixed bug for ADIs with multiple occurrences of same classes. Added a new output hash which contains file name and file's asset id
0.0.7	Not enforcing strict validation for optional fields.
0.0.6	Checksum is not case sensitive
0.0.5	Minor modifications to inputs_spec to allow the optional boolean inputs to be populated at runtime
0.0.4	Move to Quality Control category
0.0.3	Check definition of LibXML
0.0.2	Cleanly catches the lack of libxml gem. Updates to help file. Fixes to inputs spec to read ADI.XML path at runtime.
0.0.1	First version

7.2.1 How It Works

- This plug-in allows users to provide necessary inputs required for parsing and validating an `ADI.XML`.

7.2.2 Configuration Instructions

1. Create a new action step using the AdiParser plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved ADI Parser and Validator configured instance.
   - Comments: Comments used to identify the ADI Parser template.
   - ADI File Path: The file path to the ADI file.
   - ADI Version: Currently, the plugin supports ADI v1.1.
   - Validate Against DTD?: If checked, Orchestrator looks for the DTD file in the ADI.XML directory and uses it to validate ADI.XML.
   - Compare File Sizes: If checked, Orchestrator compares the file sizes of the files in ADI.XML on the filesystem with the values in ADI.XML.
   - Compare File MD5: If checked, Orchestrator compares the MD5 hashes of the files in ADI.XML on the filesystem with the values in ADI.XML.
   - Pass ADI When Missing Assets?: If checked, Orchestrator does not fail the step if assets are missing.
   - ADI Package Directory: Optional input that can be provided if the ADI package directory is different from the one where ADI.XML is located.
   - ADI Validations: Validations can be added via the widget to compare the values in AMS of App_Data tags.
   - DTD File Path: File path pointing to the DTD file location.
   - Validate XSD: Check this box if you want to validate against the XSD.
   - XSD File Path: Enter the path to the XSD file.
   - Strict Validation of Optional Fields: Check to fail the step when optional fields fail validation, if the value is present and does not match the expression.

7.2.3 Example Configuration

This is an ADI Parsing and Validation template configuration example for the AdiParser plugin.

Parameter	Value
ADI File Path	/home/sample
Compare File Sizes	Selected
Compare File MD5	Selected
Pass ADI when missing assets?	Selected
ADI Validations - ADI Asset Class	Offer
ADI Validations - Field Name	Presentation->DisplayAsNew
ADI Validations - Field Value	P7D
Required?	Required

7.2.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved ADI Parser and Validator configured instance.
Comments	Comments used to identify the ADI Parser template.
ADI File Path	The file path to the ADI file.
ADI Version	Currently, the plugin supports ADI v1.1.
Validate Against DTD?	If checked, Orchestrator looks for the DTD file in the ADI.XML directory and uses it to validate ADI.XML.
Compare File Sizes	If checked, Orchestrator compares the file sizes of the files in ADI.XML on the filesystem with the values in ADI.XML.
Compare File MD5	If checked, Orchestrator compares the MD5 hashes of the files in ADI.XML on the filesystem with the values in ADI.XML.
Pass ADI When Missing Assets?	If checked, Orchestrator does not fail the step if assets are missing.
ADI Package Directory	Optional input that can be provided if the ADI package directory is different from the one where ADI.XML is located.
ADI Validations	Validations can be added via the widget to compare the values in AMS of App_Data tags.
DTD File Path	File path pointing to the DTD file location.
Validate XSD	Check this box if you want to validate against the XSD.
XSD File Path	Enter the path to the XSD file.
Strict Validation of Optional Fields	Check to fail the step when optional fields fail validation, if the value is present and does not match the expression.

7.2.5 Inputs Description

The list of inputs depends on the configuration of the AdiParser action template. If dynamic attributes are used in the message definition, each attribute becomes a required input.

Parameter	Description
Adi_file_path *	Either a remote node must be selected or a remote IP address must be provided.
Adi_package_dir	Default directory is the ADI.XML's directory. Another value can be provided.
Check_Dtd_File	If checked, Orchestrator looks for the DTD file in the ADI.XML directory and uses it to validate ADI.XML.
Check_Filemd5s	Default value is false, but validating the MD5 can be chosen.
Check_Filesizes	Default value is false, but validating the file sizes can be chosen.

Parameter	Description
Dtd_file_path	File path pointing to the DTD file location.
Pass_On_Missing_Files	Default value is false, this flag provides a safe catch mechanism when files are not present during checks.
Xsd_path	Enter the path to the XSD file.

7.2.6 Outputs Description

Parameter	Description
List_of_Filenames	Names of the files in the ADI.XML
Count_of_ContentFiles	Number of files in the ADI.XML
ADI_Size_Hash	Hash with filenames as keys and their file sizes as values
ADI_MD5_Hash	Hash with filenames as keys and their MD5 hashes as values
Is_Valid_ADI	If all validations and file property checks pass, this value is true
ADI_Package_Metadata	ADI.XML metadata from the AMS tag
Filetype_list	Hash with the class name as the key and the content as the value
Filesizes_validation	This flag is true if the file sizes of the content match the file sizes on disk
FileMD5s_validation	This flag is true if the MD5 of the content matches the MD5 of the file on disk
DTD_Validation	This flag is true if the DTD validation is successful
Failed_Filesize_Assets	Array of files that failed file size validation
Failed_MD5_Assets	Array of files that failed MD5 validation
Asset_IDs_hash	Hash of Asset IDs as values for each asset class (key)
Asset_IDs_FileName_hash	Hash of Asset IDs field name as values for each asset class (key)
Class_Info_FileName_hash	Hash of the class field name as values for each asset class (key)
DTD_Validation_Error_Message	Error message if DTD validation fails
Field_Validation_Error_Message	Error message if field validations fail
XSD_Validation	Error message if XSD validation fails

7.2.7 Supported Actions

None

7.2.8 Dependencies

None

7.2.9 Operating Instructions

This plug-in allows users to provide necessary inputs required for parsing and validating an `ADI.XML`.

Steps to populate the ADI Parsing and Validation template

- Provide a name for the action template to identify it.
- Provide a path to the ADI.XML. In a workflow, this field is available as a runtime input.
- Make sure the ADI version is 1.1.
- Check the checkboxes for file size validation and MD5 checksum validation. The MD5 checksum check is not case-sensitive.
- Check if the ADI.XML needs to be validated by an ADI.DTD file.
- Check the 'Pass on missing files' checkbox if you want the step to NOT fail when assets are missing.
- Use the Add ADI Validations widget to add a set of validations.
- To add a validation, click the Add ADI Validations widget.
- In the fields that appear, first select the appropriate asset class. The asset classes for a CableLabs ADI v1.1 XML can only be: Package, Title, Movie, Preview, Poster, Still-Image, TrickAsset, Encrypted Asset.
- Not all ADI.XML files contain all asset classes, so choose wisely what to validate.
- Next, in the Field Name text box, enter the field that needs to be validated in the chosen asset class. Familiarity with ADI.XML is needed to know the fields.
- Next, in the Field Value text box, enter a value that you want to use to validate the data.
- This value by default is a glob expression '*'. The value supports wildcards like * and ?, globbing like val*ue, and regular expressions like /.+[A-Za-z]/.

7.3 Aurora file verification



Figure 7.3: Aurora file verification icon

This Action plug-ins enables the submission of a file verification request to a Tektronix Aurora Server

Version	Comment
1.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.3.1	Fixed error if ReportFolder is missing
1.3.0	Updating default port.
1.2.9	Added styling to parameter help text.
1.2.8	Returning percent_complete from check_status method to be consumed by the activities screen
1.2.7	Fixing the endpoint URL once the WSDL is loaded via the network
1.2.6	WSDL file for Windows and Linux will be fetched via the network
1.2.5	Adding RemoteNode Dependency.
1.2.4	WSDL file on Windows does not work. Accessing from Network. Report generation support broken in 1.2.0, adding it back
1.2.3	Template list failed when only 1 template present. Now shows error information instead of a blind rescue in template list method
1.2.2	Adding begin rescue in check status to catch unexpected connection errors
1.2.1	Fixed bugs after making the plugin asynchronous
1.2.0	Making the plugin asynchronous
1.1.5	Expose :queue_on_top as runtime variable
1.1.4	Made Report Folder mandatory for versions 5.2 and above
1.1.3	Add a check on server_address for VAR_NODE_OR_ADDRESS to be a required input
1.1.2	2015-02-17 ML - New icon
1.1.1	Prevents autocomplete of user field
1.1.0	Automatically move wsdl construction files

Version	Comment
1.0.0	Modified to use SharedState (requires Orchestrator 2.0)
0.0.4	Change made in the migration file to reference new plugin load directory
0.0.3	Adds functionality to mask password
0.0.2	Catch no status error
0.0.1	Initial release

7.3.1 Saved Parameters Description

- Name: The name used to identify a saved AuroraFileVerification configured instance.
- Comments: Some comments about this saved AuroraFileVerification configured instance.
- Aurora Server Node: The node where the analysis operation must be executed (i.e. the node where the Aurora Server is installed).
- Server Address: Can be used to provide a path that overwrite the address of the node. This is useful in the case where the web service can only be accessed through https as the web service address generated from the Node is http by default.
- Aurora Service Port: The port to which the connection should be made. By default this is port 1002.
- Aurora User: The user account used to execute the Aurora analysis operation.
- Aurora Password: The password for the Aurora user.
- Input source path: The source file for the analysis operation. This value can be entered using dynamic attributes
- Do not alter path?: Deactivate Unix to Windows conversion of file path. When this is checked, path are kept the way they are entered
- Test template: Select from the list of test template configured on that Aurora Server.
- Queue on top?: Submit to the top of the Aurora job queue instead of the bottom to ensure this job will be picked up next.
- Polling frequency: The progress status polling frequency in seconds (the default is 5 seconds).
- Return xml report? : If selected an additional output variable 'XML_report' containing the full report formatted in XML will be provided upon action completion.
- Return html report ? : Same but provides an additional output variable 'HTML_report' formatted in HTML.
- Return html report link? : If selected, provides an additional output variable 'HTML_report_link' containing the URL that can be used to retrieve the HTML analysis report from the Aurora server
- Return pdf report link? : Same but provides an additional output variable 'PDF_report_link' pointing to the analysis report formatted as a PDF document.
- Report folder: A folder on the Aurora server into which a sub-folder will be created for each job with all the analysis reports. Dynamic parameters can be used to compose the value of that field.
- Pass if no errors are found?: Provides the option to consider successful a job with warnings, but no errors

7.3.2 Inputs description

The list of inputs depends on the configuration of the AuroraFileVerification action template.

The list of inputs depends on the configuration of the AuroraFileVerification action template.

- *Aurora_node_or_address*: Points to the Aurora Server or service address. Can be either a node ID (i.e. 'Aurora_node'), a node address (i.e. '192.168.56.101') or a service address (i.e. 'http://192.168.56.101:1002'). This input is only requested when neither the node nor the service address is provided in the action template configuration.
- *User*: Same as 'Aurora User' but provided at run-time. Only requested if 'Aurora User' is left blank in the action template configuration.
- *Password*: Password for the Aurora user.
- *File_path*: All the variables listed in 'Input source path' field are mandatory and must be of type string. If the 'Input source path' is left blank, then 'File_path' must be provided at run-time.
- *Report_folder*: All the variables listed 'Report folder' field are mandatory and must be of type string.
- *Queue_on_top*: Same as 'Queue on top?' but provided at run-time.
- *Test_plan*: Provides the ability to select the test plan to use for the analysis at run-time. The named in this variable needs to be configured on the Aurora server where the analysis is to be performed.

7.3.3 Outputs description

- *Date_completed*: The date returned by Aurora as being the completion date for the job.
- *Error_count*: The number of error found during the analysis

- *Warning_count*: The number of warning found during the analysis
- *Aurora_job_id*: The JOB ID assigned by Aurora (an Integer)
- *Aurora_status*: The analysis status returned by Aurora
- *Template_used*: The name of the test template used for the Analysis
- *Time_to_complete*: The number of seconds it took Aurora to complete the analysis job
- *PDF_report_link*: If requested, a link to the report hosted on the Aurora server, formatted in PDF
- *HTML_report_link*: If requested, a link to the report hosted on the Aurora server, formatted in HTML
- *XML_report*: If requested the full HTML report returned as a text field
- *HTML_report*: If requested the full XML report returned as a text field

7.3.4 Supported Actions

None

7.3.5 Dependencies

None

7.3.6 Operating Instructions

This action executes the specified file analysis operation on a remote node where Aurora is available. The connection to the remote server is done using the Aurora web-service.

Note: Since Tektronix Aurora is only available on Windows platform, the path are converted by default to Windows format, so an input pass of 'c:/in/file.avi' is automatically converted to what Aurora expect which is 'c:\in\file.avi'. However, in some case, it is possible to keep the path from going through that translation step. For example, this is required if the file path is a URL 'ftp://remote_folder/file'

Important Note: this action plugins does not do any file movement. Hence, for it to run a file analysis, the specified file has to be accessible on the Aurora server where the analysis is to be performed. Either the file is transferred to that server or it must be accessible from that server either on some type of network share.

7.4 Baton file correction



Figure 7.4: Baton file correction icon

This Action plug-ins enables the submission of a file correction request to an Interra Baton Server

Version	Comment
0.5.1	Enhanced action template UI
0.5.0	Added support for Baton v9.x REST API
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.2	Failing when no job id is returned. Reduced logging
0.3.1	Correcting the plugin_name in metadata.yml.
0.3.0	Added flag to check if the QC server is different than Correction server.
0.2.1	Exposes the destination file path as a plugin output
0.2.0	Baton RPC port and Baton Rest port are different so new column for Rest port.

Version	Comment
0.1.0	Modifications based on testing at client. Only task_id is provided and testplan details need to be fetched.
0.0.1	Initial version for testing

7.4.1 How It Works

- Integrates with Baton QC/Correction server to perform automated file correction operations.
- Submits file correction jobs to a Baton server.
- Supports both Baton version < 9 (password only) and version >= 9 (password + API token).
- Can correct individual files or entire folders.
- Monitors correction job progress through configurable polling.
- Supports growing file verification for files still being written.
- Can stop on first error or continue through all corrections.
- Optionally retrieves XML correction reports for detailed results.
- Returns job details including job ID, timestamps, and output file paths.

7.4.2 Configuration Instructions

1. Create a new action step using the BatonFileCorrection plugin.
2. Fill out the following fields:
    - Name: The name used to identify a saved Baton File Correction configured instance.
    - Comments: Comments about this Baton File Correction template.
    - Baton Server Node: The remote node name of the server which hosts the Baton File Correction server. Use this OR Baton Server address.
    - Baton Server address: The remote hostname or IP address of the server hosting the Baton server. Use this OR Baton Server Node.
    - Baton RPC port: The port number for the Baton server RPC communication.
    - Baton version: Select the Baton version: < 9 (password only) or >= 9 (password + API Token required).
    - Baton user: A user configured on Baton server to make API calls.
    - Baton password: Password of the Baton user (required for all versions).
    - Baton API Token: (Version >= 9 only) The API token for Baton server access. Available in Baton GUI under SETUP > Accounts > User > admin.
    - Is Correction Server address different?: Check if the IP address for correction server is different from QC server. Makes 'Correction server IP' mandatory.
    - Correction server IP: IP address of the correction server (required if different from QC server).
    - Baton Correction port: The port number of the REST API of Baton correction server (default 6060).
    - Baton Task ID: The Task ID from Baton UI for the correction task to execute.
    - Baton QC Report as XML: XML QC report input for correction processing.
    - Test Plan XML exported from Baton: Test plan XML exported from Baton server.
    - Destination Folder: Output folder where corrected files will be placed.
    - Output Format: Format specification for output files.
    - Polling Frequency: Interval in seconds to check correction job status (default varies based on configuration).

7.4.3 Example Configuration

This is an example configuration for the BatonFileCorrection plugin using Baton version 9 with API token.

Parameter	Value
Name	Correct Media File
Baton Server address	baton.example.com
Baton RPC port	8080
Baton version	>= 9
Baton user	api_user
Baton password	********
Baton API Token	abc123token456
Baton Task ID	12345
Polling Frequency	30

7.4.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved Baton File Correction configured instance.
Comments	Comments about this Baton File Correction template.
Baton Server Node	The remote node name of the server which hosts the Baton File Correction server. Use this OR Baton Server address.
Baton Server address	The remote hostname or IP address of the server hosting the Baton server. Use this OR Baton Server Node.
Baton RPC port	The port number for the Baton server RPC communication.
Baton version	Select the Baton version: < 9 (password only) or >= 9 (password + API Token required).
Baton user	A user configured on Baton server to make API calls.
Baton password	Password of the Baton user (required for all versions).
Baton API Token	(Version >= 9 only) The API token for Baton server access. Available in Baton GUI under SETUP > Accounts > User > admin.
Is Correction Server address different?	Check if the IP address for correction server is different from QC server. Makes 'Correction server IP' mandatory.
Correction server IP	IP address of the correction server (required if different from QC server).
Baton Correction port	The port number of the REST API of Baton correction server (default 6060).
Baton Task ID	The Task ID from Baton UI for the correction task to execute.
Baton QC Report as XML	XML QC report input for correction processing.
Test Plan XML exported from Baton	Test plan XML exported from Baton server.
Destination Folder	Output folder where corrected files will be placed.
Output Format	Format specification for output files.
Polling Frequency	Interval in seconds to check correction job status (default varies based on configuration).

7.4.5 Inputs Description

All parameters are available as inputs at runtime if not specified by the template. This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

Parameter	Description
Baton_Server_Node	Remote node name for Baton server (required if Baton Server address not provided).
Baton_Server_address	Hostname or IP address of Baton server (required if Baton Server Node not provided).
baton_task_id	Task ID from Baton UI for the correction task to execute.
Baton_user	Username for Baton API authentication.
Baton_password	Password for Baton API authentication (required for all versions).
Baton_API_Token	API token for Baton version >= 9 (required for v9+).

Parameter	Description
Baton_RPC_port	RPC port number (optional, default 8080).
Baton_Correction_port	REST API port number (optional, default 6060).
Correction_server_IP	IP address of correction server if different from QC server.
destination_folder	Output folder for corrected files (optional).
output_format	Output format specification (optional).
Polling_Frequency	Status polling interval in seconds (optional).
Folder_Correction	Boolean flag for folder correction mode (default false).
Verify_growing_file	Boolean flag to verify growing files (default false).
Return_xml_report	Boolean flag to retrieve XML report post-correction (default false).
Stop_on_first_error	Boolean flag to fail step on first error (default false).
Pass_if_no_errors	Boolean flag to pass with warnings only (default false).
Non_Windows_install	Boolean flag for non-Windows Baton installation (default false).

7.4.6 Outputs Description

Parameter	Description
Result	JSON result description returned from the correction server
Job_ID	Job ID of the request sent to the Baton correction server
Start_time	Timestamp when correction server received the correction request
Completion_time	Timestamp when correction server completed the correction task
Output_file	Destination path where corrected file is placed

7.4.7 Supported Actions

None

7.4.8 Dependencies

Requires access to Baton QC/Correction server with valid credentials and network connectivity.

7.4.9 Operating Instructions

This plug-in allows users to provide necessary inputs required for a Baton server to perform file correction jobs.

Correction Modes

- Single File Correction:
  - Default mode when 'Folder Correction?' is unchecked
  - Corrects individual file specified by task
  - Best for: Single file workflows
- Folder Correction:
  - Enable by checking 'Folder Correction?' flag
  - Processes all files in specified folder

  - Best for: Batch processing multiple files
- Growing File Verification:
  - Enable by checking 'Verify growing file?' flag
  - Waits for file to finish writing before correction
  - Best for: Files being transferred or recorded
  - Prevents corruption from incomplete files

Error Handling Options

- Stop on First Error (Enabled):
  - Workflow fails immediately when first error detected
  - Remaining corrections not performed
  - Best for: Quality gates requiring zero errors
- Continue Through Errors (Disabled):
  - Continues correction even after errors
  - All corrections attempted
  - Final result includes all error details
- Pass if No Errors Found:
  - Treats warnings as acceptable
  - Step completes successfully with warnings
  - Fails only on actual errors
  - Best for: Non-critical QC scenarios

Common Use Cases

- Media file correction: Automatically correct video/audio files that fail QC checks.
- Batch processing: Correct entire folders of files using folder correction mode.
- Quality gates: Stop workflows on first correction error for critical content.
- Archive repair: Correct archived media files before restoration or reuse.
- Growing file handling: Wait for recording/transfer completion before correction.
- Multi-format correction: Apply Baton correction tasks to various media formats.
- Report generation: Retrieve detailed XML reports for audit and analysis.

Basic File Correction Example

- Step 1: Provide remote node or remote server IP and port (e.g., 10.0.103.85 or baton.example.com)
- Step 2: Provide the Baton Task ID (obtained from Baton UI for a Baton correction task)
- Step 3: For Baton version >= 9, provide API Token from Baton GUI (SETUP > Accounts > User > admin)
- Step 4: File correction job is submitted and polled until complete
- Step 5: Output file path and job results are returned

Best Practices

- For Baton >= 9, ensure both password and API token are provided and valid.
- Set appropriate polling frequency - balance between responsiveness and server load (30-60 seconds typical).
- Enable 'Stop on first error' for quality gates requiring zero defects.
- Use 'Verify growing file' when correcting files during transfer or recording.
- Enable 'Return xml report' for detailed error analysis and audit trails.
- Verify Task ID is not archived in Baton before submitting jobs.
- Ensure that the configured port is accessible from Orchestrator to Baton.
- Test credentials by logging into Baton UI before workflow deployment.
- Monitor job completion times to optimize polling frequency.

Troubleshooting

- Connection timeout: Verify that the configured port is accessible from Orchestrator to Baton server.
- Authentication failed: Verify credentials by logging into Baton UI directly with same username/password.
- API Token invalid (v9+): Obtain valid token from Baton GUI: SETUP > Accounts > User > admin. Both password AND token required.
- Task ID not found: Verify Task ID exists in Baton and is not archived. Check Baton UI for active tasks.
- Job stuck in queue: Check Baton server load and available correction slots. Verify task configuration.
- Output file not found: Check destination folder exists and Baton has write permissions.
- Growing file timeout: If 'Verify growing file' enabled, ensure file transfer completes within timeout period.
- Correction server unreachable: If using separate correction server, verify IP address and port are correct.
- XML report not returned: Verify 'Return xml report' flag is enabled and Baton has report generation enabled.

- Permission denied: Ensure Baton user has appropriate permissions for correction operations.

Important Notes

- Either Baton Server Node OR Baton Server address must be provided (not both).
- Baton version >= 9 requires THREE credentials: user, password, AND API token.
- Baton version < 9 requires TWO credentials: user and password only.
- This plugin does not support value overwrites - template values cannot be overridden at runtime.
- The Task ID must not be archived in Baton server.
- 'Folder Correction?' flag changes behavior to process entire folders instead of single files.
- 'Stop on first error' causes immediate failure - use for strict quality gates.
- 'Pass if no errors' allows warnings - only errors cause failure.
- Polling frequency affects detection speed and server load.
- Output file path may not be exposed depending on Baton configuration.

7.5 Baton file verification



Figure 7.5: Baton file verification icon

This Action plug-ins enables the submission of a file verification request to an Interra Baton Server

Version	Comment
1.8.2	Enhanced action template UI
1.8.1	Adds needed_cores field when using Baton v9.x
1.8.0	Adds support for Baton v9.x REST API
1.7.1	Adds option to use SSL for Baton server connections
1.7.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.6.3	Fixes 'Ready' status management
1.6.2	Fixes progress message
1.6.1	Proper report output handling
1.6.0	Add option to use lower load query
1.5.2	Added lower load query
1.5.1	Make logging more consistent for XMLRPC errors
1.5.0	Fix for failure before task IDs are retrieved
1.4.8	Fixes persistence of session to use shared state and adds erroring out if Baton does not return a task ID
1.4.7	modified polling frequency to have the runtime value take precedence over the configuration one.
1.4.6	added '' result with aborted status
1.4.5	added unknown result with aborted status
1.4.4	handle case of aborted status and partial result
1.4.3	Avoid empty dynamic values to Baton request
1.4.2	Allows unauthenticated connection and accept dynamic parameters in test plan
1.4.1	bumping up the version to avoid conflict ()since code was merged).
1.4.0	adding baton version to support 6.3
1.3.4	Exposed the percent complete
1.3.3	fixed polling frequency
1.3.2	Adding RemoteNode Dependency.
1.3.1	Added help file

Version	Comment
1.3.0	Fail on Aborted status
1.2.9	Allows run-time configuration of port
1.2.8	Allows run-time configuration of user and password
1.2.7	New icon
1.2.6	Adds logic to fail a work order when Baton task is canceled
1.2.5	Adds option to provide node at run-time
1.2.4	Return task ID as output
1.2.3	Return status as In-progress for XML RPC errors
1.2.2	Provides same outputs in folder mode as in file mode (but as arrays)
1.2.1	Expresses progress report in human readable numbers
1.2.0	Change polling frequency to be in seconds
1.1.0	Adds support for growing files
1.0.1	Prevents auto complete of user field
1.0.0	Made plugin asynchronous
0.7.2	Adds link to report
0.7.1	Makes dynamic test plan parameters optional
0.7.0	adds dynamic test plan parameters
0.6.0	adds option to retrieve xml report
0.5.2	adds functionality to mask password
0.5.1	provides XMLRPC error details, fix issue with runtime server var name
0.5.0	adds an input definition to the template
0.4.0	adds option to force pass the file if only warnings are found
0.3.1	adds flag for mac installs
0.3.0	adds automatic file path translation by default (not done in case of Mac install of Baton)
0.2.0	adds node selection
0.1.2	modifies the report path to be generated to match the link on the baton monitoring page
0.1.1	adds clean up of temporary files from var_run

7.5.1 How It Works

- This action plug-in submits a file verification (QC) job to a Baton server using the provided inputs.

7.5.2 Configuration Instructions

1. Create a new action step using the BatonFileVerification plug-in.
2. This action submits a QC job to a Baton server and reports the results when verification completes.
3. Fill out the following configuration fields:
    - Name: The name used to identify a saved BatonFileVerification configuration.
    - Comments: Optional comments about this BatonFileVerification configuration.
    - Baton Version: Select the Baton version to use (5.*, 6.*, 7.*, or 9.*). For version 9.*, authentication uses an API Token instead of a password.
    - Baton Server Node: The remote node name of the server hosting the Baton File Verification service.
    - Baton Server Address: The remote host name or IP address of the Baton server.
    - Baton Port: The port number of the Baton server.
    - Non-Windows Install?: Select this option if Baton is installed on a non-Windows system.
    - Baton User: The user account configured on the Baton server for API access.
    - Baton Password: (For versions < 9) The password associated with the Baton API user account.
    - API Token: (For version 9.*) The API token for authentication. Available in Baton GUI under SETUP > Accounts > User > admin.
    - Input Source Path: The full UNC path as seen by Baton. The UNC path must be registered in the Baton server.
    - Baton Test Plan: The name of the test plan as defined on the Baton server.
    - Baton Test Plan Version: The version of the specified test plan as defined on the Baton server.
    - Priority: The priority assigned to the QC job submitted to Baton.
    - Status Polling Frequency: How often progress is polled. The default is 5 seconds.
    - Checkers: Restrict the QC job to one or more specific checkers configured in Baton.
    - Needed Cores: (For version 9.* only) Specify the number of cores to use for the verification job. Default is 1.
    - Expedite?: Enables faster polling. This reduces status detail but improves responsiveness. (Folders are always expedited.)

- Folder Verification?: Select this option if the input source path refers to a folder instead of a file.
- Verify Growing File?: Select this option when verifying a file that is actively growing.
- Return XML Report as an Output?: Select this option if the XML report should be fetched after verification completes.
- Stop Verification on First Error?: Select this option to stop the verification job immediately when the first error is detected.
- Pass if No Errors Are Found?: Select this option to consider the step successful when only warnings are found.

7.5.3 Example Configuration

This is a minimal setup configuration for the BatonFileVerification plug-in.

Parameter	Value
Baton Server Address	http://test-server.baton-server.com/example1
Baton port	8080
Baton user	root
Baton password	<password>
Input source path	/source/path/to/plan
Baton Test Plan	plan-contents
Baton Test Plan Version	Version 5
Status Polling frequency	15
Priority	10
Return XML report as an output?	Selected

7.5.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved BatonFileVerification configuration.
Comments	Optional comments about this BatonFileVerification configuration.
Baton Version	Select the Baton version to use (5.*, 6.*, 7.*, or 9.*). For version 9.*, authentication uses an API Token instead of a password.
Baton Server Node	The remote node name of the server hosting the Baton File Verification service.
Baton Server Address	The remote host name or IP address of the Baton server.
Baton Port	The port number of the Baton server.
Non-Windows Install?	Select this option if Baton is installed on a non-Windows system.
Baton User	The user account configured on the Baton server for API access.
Baton Password	(For versions < 9) The password associated with the Baton API user account.
API Token	(For version 9.*) The API token for authentication. Available in Baton GUI under SETUP > Accounts > User > admin.
Input Source Path	The full UNC path as seen by Baton. The UNC path must be registered in the Baton server.
Baton Test Plan	The name of the test plan as defined on the Baton server.
Baton Test Plan Version	The version of the specified test plan as defined on the Baton server.
Priority	The priority assigned to the QC job submitted to Baton.
Status Polling Frequency	How often progress is polled. The default is 5 seconds.
Checkers	Restrict the QC job to one or more specific checkers configured in Baton.
Needed Cores	(For version 9.* only) Specify the number of cores to use for the verification job. Default is 1.
Expedite?	Enables faster polling. This reduces status detail but improves responsiveness. (Folders are always expedited.)
Folder Verification?	Select this option if the input source path refers to a folder instead of a file.

Parameter	Description
Verify Growing File?	Select this option when verifying a file that is actively growing.
Return XML Report as an Output?	Select this option if the XML report should be fetched after verification completes.
Stop Verification on First Error?	Select this option to stop the verification job immediately when the first error is detected.
Pass if No Errors Are Found?	Select this option to consider the step successful when only warnings are found.

7.5.5 Inputs Description

Required inputs depend on how the BatonFileVerification template is configured. If dynamic attributes are used, each attribute becomes a runtime input.

Parameter	Description
*Baton_server_address *	The remote host name or IP address of the Baton server.
*File_name *	The full UNC path as seen by Baton. The UNC path must be registered in Baton.
*Test_plan *	The test plan name as defined on the Baton server.
Baton_port	The port number of the Baton server.
Baton_server_password	The password of the user configured on the Baton server for API access.
Baton_server_user	The user configured on the Baton server for API access.
Dynamic_test_plarrameters	Key/value parameters required for dynamic test plans on Baton.
Polling_frequency	How often job progress is polled. Default is 5 seconds.
Priority	The priority assigned to the job submitted to the Baton server.
Test_plan_version	The version of the test plan as defined on the Baton server.

7.5.6 Outputs Description

Parameter	Description
Report_PDF_link	A link to the Baton QC report in PDF format.
Report_VIDEO_link	A link to the Baton QC report in VDR format.
Report_XML_link	A link to the Baton QC report in XML format.
Folder?	The result of the QC operation: Success, Warning, Failure, or Aborted.
Result	The output files generated as part of the QC operation.
Start_time	The start time of the QC operation.
Completion_time	The completion time of the QC operation.
Task_ID	The ID of the QC job submitted to the Baton server.
Errors	The number of errors detected during the QC operation.
Warnings	The number of warnings detected during the QC operation.

7.5.7 Supported Actions

None

7.5.8 Dependencies

None

7.5.9 Operating Instructions

1. This action plug-in submits a file verification (QC) job to a Baton server and monitors its progress.
2. Specify the Baton server address, port, and authentication details.
3. Specify the test plan name and version as defined on the Baton server.
4. Specify the full UNC path of the file or folder to verify.
5. The source path, test plan, job name, and priority are submitted to the Baton server as part of the QC job request.
6. The QC job is polled until completion, and results are returned when the step completes.
7. Ensure the configured port (e.g., 8080) is reachable from Orchestrator to the Baton server.
8. For Baton versions < 9: Ensure the login credentials (username and password) are valid by verifying them directly in the Baton UI.
9. For Baton version 9.*: Ensure the API Token is valid. The token can be obtained from the Baton GUI under SETUP > Accounts > User > admin.
10. Ensure the path submitted to Baton is a valid absolute UNC path.
11. Ensure the UNC path is registered within the Baton UI.
12. NOTE: For Baton version 6.*, dynamic test plan parameters must be valid JSON objects in the ”Value” field.

7.6 Baton xml report



Figure 7.6: Baton xml report icon

This Action plug-ins enables the retrieval of XML Report from Baton

Version	Comment
0.3.1	Enhanced action template UI
0.3.0	Added support for Baton v9.x REST API
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.1	initial release

7.6.1 How It Works

- Connects to a Baton server to retrieve XML quality control reports for specified task IDs.
- Accepts Task IDs passed in as an array.
- Supports connecting to the Baton server via a configured remote Execution Node or directly via host name or IP address.
- Supports Baton versions < 9 using username and password authentication, and Baton version >= 9 using an API token.
- Accepts an optional port number, defaulting to 8080 if not specified.
- Returns the QC report content in XML format for all provided task IDs.

7.6.2 Configuration Instructions

1. Create a new action step using the Baton XML Report plugin.
2. Fill out the following fields:
    - Name: The name used to identify a saved Baton XML Report configured instance.
    - Comments: Comments about this saved Baton XML Report configured instance.
    - Baton Server Node: Remote node name of the server hosting the Baton server.
    - Baton Server address: Remote host name or IP address of the server hosting the Baton server.
    - Baton port: Port number of the remote server hosting the Baton server. Defaults to 8080 if not specified.
    - Baton version: Select the Baton version (< 9 or >= 9). Determines the authentication method used.
    - Baton user: Username of a user configured on the Baton server to make API calls.
    - Baton password: (For versions < 9) Password of the user configured on the Baton server.
    - API Token: (For version >= 9) API token for authentication. Available in the Baton GUI under SETUP > Accounts > User > admin.
    - Baton Task IDs: The Baton Task IDs for which XML reports are to be retrieved.

7.6.3 Example Configuration

This is an example configuration for the Baton XML Report plugin retrieving XML QC reports from a Baton server using API token authentication.

Parameter	Value
Name	Get QC Report for Verified Files
Comments	Retrieve XML report for task IDs from Baton File Verification step
Baton Server Node	(blank - using IP address instead)
Baton Server address	10.0.103.85
Baton port	8080
Baton version	>= 9
Baton user	admin
API Token	*********
Baton Task IDs	(passed at runtime from Baton File Verification plugin)

7.6.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved Baton XML Report configured instance.
Comments	Comments about this saved Baton XML Report configured instance.
Baton Server Node	Remote node name of the server hosting the Baton server.
Baton Server address	Remote host name or IP address of the server hosting the Baton server.
Baton port	Port number of the remote server hosting the Baton server. Defaults to 8080 if not specified.
Baton version	Select the Baton version (< 9 or >= 9). Determines the authentication method used.
Baton user	Username of a user configured on the Baton server to make API calls.
Baton password	(For versions < 9) Password of the user configured on the Baton server.
API Token	(For version >= 9) API token for authentication. Available in the Baton GUI under SETUP > Accounts > User > admin.
Baton Task IDs	The Baton Task IDs for which XML reports are to be retrieved.

7.6.5 Inputs Description

All parameters are available as runtime inputs if not specified in the template. Note that this plugin does not support value overwrites — if a value is set in the template, it will not be available as a runtime input.

Mandatory Inputs

Parameter	Description
Baton Server Node or Baton IP Address	Either a remote Execution Node must be selected or a remote host name/IP address must be provided. One of these is required.
Baton Task IDs	Array of Task IDs for which XML reports are to be retrieved.

Optional Inputs

Parameter	Description
Name	A name to identify the configured template instance.
Comments	Additional information to help identify the template.
Baton API Port	Port number for the Baton server. Defaults to 8080 if not specified.

7.6.6 Outputs Description

The output is the full QC report content returned by the Baton server in XML format.

Parameter	Description
Xml_Report	Contents of the QC report in XML format for the provided task IDs.

7.6.7 Supported Actions

None

7.6.8 Dependencies

Requires a running and accessible Baton server. Port 8080 (or the configured Baton port) must be open from the Orchestrator server to the Baton server.

7.6.9 Operating Instructions

This plugin connects to a Baton server and retrieves XML QC reports for the provided Task IDs. Task IDs are typically passed in from the Baton File Verification plugin as part of a larger quality control workflow.

Authentication by Baton Version

- Baton versions below 9: Authentication uses a username and password. Provide the Baton user and Baton password fields.
- Baton version 9 and above: Authentication uses an API token. The token can be obtained from the Baton GUI under SETUP > Accounts > User > admin.

Best Practices

- Use the Baton File Verification plugin output to pass Task IDs directly into this plugin to ensure accurate report retrieval.
- Specify a descriptive Name and Comments in the template to make the step easily identifiable in complex workflows.
- Verify the Baton version setting matches your server version. Using the wrong authentication mode will cause login failures.
- If using API token authentication, ensure the token isn't expired to prevent mid-workflow authentication failures.
- Implement error handling workflows to manage cases where the Baton server is unreachable or task IDs are invalid.

Troubleshooting

- Connection refused: Ensure port 8080 (or the configured Baton port) is open and accessible from the Orchestrator server to the Baton server.

- Authentication failure (version < 9): Verify the username and password are valid by logging into the Baton UI directly with the same credentials.
- Authentication failure (version >= 9): Verify the API token is valid. Obtain a fresh token from the Baton GUI under SETUP > Accounts > User > admin.
- Wrong Baton version selected: Confirm the Baton version setting matches the actual server version. A mismatch will cause authentication to fail.
- Empty XML report output: Confirm the provided Task IDs are valid and that reports are available for those tasks on the Baton server.
- Server not reachable: Verify the Baton Server address or Execution Node is correctly specified and the Baton server is running.

Important Notes

- Either a Baton Server Node or a Baton Server address must be provided. (Both are not required simultaneously)
- If a value is specified in the template, it will not be available as a runtime input. This plugin does not support value overwrites at runtime.
- The Baton port defaults to 8080 if not specified.
- Task IDs must be provided as an array. These are typically passed in directly from the Baton File Verification plugin output.
- Authentication method is determined by the Baton version selected: password for versions below 9, API token for version 9 and above.
- The API token for version 9 and above is available in the Baton GUI under SETUP > Accounts > User > admin.

7.7 Cerify File Verification



Figure 7.7: Cerify File Verification icon

This Action plug-in provides the ability to submit a quality control task to a Cerify Server.

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.6	Removing soap4r dependency
0.1.5	Added styling to parameter help text.
0.1.4	Adding Remote Node Dependency
0.1.3	Bug fix - MediaSet creation fails when white space is present in the path due to the escaping library used.
0.1.2	Bug fix - Job Progress was not indicated in newer versions of Cerify, adding a REST call to fetch that status.
0.1.1	- 2015-02-11 - ADM -> Bug fix: Job name made completely flexible
0.0.8	- 2015-02-11 - ADM -> Bug fix: Job name made dynamic
0.0.7	- 2015-02-11 - PM -> Bug fix: adding a rescue for start_time fetch and check for alerts before performing operations
0.0.6	- 2014/12/18 - PM -> Bug fix: added exception handling in get_job_status and checking for nil
0.0.5	2015-02-17 ML - New icon
0.0.4	exception handling of RemoteNode.find_by_name()
0.0.3	Removes explicit dependency on xmlsimple
0.0.2	fields will use default_get instead of field value
0.0.1	initial development

7.7.1 Saved Parameters Description

- Name: The name used to identify a saved Cerify File Verification configured instance.
- Comments: The comments of the Cerify File Verification template
- Cerify Server Node: The remote node name of the server which hosts the Cerify File Verification server
- Cerify Server IP address (or hostname): The remote host name or remote IP of the server which hosts the Cerify server
- Cerify API port (optional): The port number of remote server which hosts the Cerify server
- Cerify Profiles: A dropdown of Cerify profiles available on Cerify server
- Cerify Media Locations: A dropdown of Cerify Media Locations available on Cerify server
- Media URL: The absolute path of the file on the Cerify server. Must be under the Media Location and should be prefixed by either 'file://' or 'ftp://' or 'smb://' followed by the file path
- Keep Media Set on server (optional): Check this option if the Media Set created should remain on Cerify server. If un-checked the media set will be deleted
- Force Pass on Warning (optional): Check this option if the file must be deemed passed, if there is a warning. If un-checked the file will be deemed as failed
- Job Name (optional): The name of the file verification job.
- Force Job Name (optional): Do not add punicity postfix to the name of the file verification job.
- Priority (optional): The priority that should be assigned to the job submitted to Cerify server
- Status Polling frequency: How often progress will be reported. Default is 5 seconds

7.7.2 Inputs description

The list of inputs depends on the configuration of the CerifyFileVerification action template.

- All parameters are available as inputs at runtime if not specified by the template.

  This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

These inputs are mandatory:

- *Cerify Server Node or Cerify IP Address*: Either a remote node must be selected or a remote IP address must be provided.
- *Profile Name*: A Profile Name is mandatory while submitting a job, hence a profile must be selected from dropdown.
- *Media Location*: A Media Location is mandatory while submitting a job, hence a media location must be selected from dropdown.
- *Media URL*: Absolute path to the source file on remote server must be provided. Must be under the Media Location and should be prefixed by either 'file://' or 'ftp://' or 'smb://' followed by the file path

These inputs are optional:

- *Name*: It is advisable that a name is entered to identify the template.
- *Comments*: Information about the template which will help identify the template.
- *Cerify API Port*: The port is optional as a default value (80) is used if not specified.
- *Keep Media Set on server*: If the check box is not checked, the media set created will be deleted.
- *Force Pass on Warning*: If the check box is not checked, the file will be failed even if there is only a warning.
- *Job Name*: If a Job Name is not provided a default name suffixed by timestamp is used, making it unique.
- *Priority*: A low priority is assigned to the job if nothing is provided.
- *Status Polling frequency*: Specify if progress needs to be reported within a time other than default.

7.7.3 Outputs description

- *Input_Media_Params*: The media parameters involved in the file verification operation
- *File_Verification_Result*: The output file(s) obtained as a result of the file verification operation
- *File_Stats*: The input file's name, size and path obtained from the Cerify server during the file verification operation
- *Job_Name*: The name of the job triggered by the file verification operation
- *Alerts*: Alerts reported by Cerify server during file verification operation

7.7.4 Operating Instructions

This plug-in allows users to provide necessary inputs required for a Cerify server to do a file verification/QA job.

The mandatory inputs are listed above.

The plug-in supports the latest API release. An example of a QC job is given below.

7.7.5 An example of a QC workflow below

a) Provide remote node or remote server IP and port (example: 10.0.103.85 or transcode.lala.com)

b) Click the button Get Profiles and Media Locations. This provides dropdowns with profiles and media locations obtained from the provided Cerify server

c) Select a profile from the dropdown

d) Select a media location from the dropdown and provide a media URL. The media URL should be under the media location.

e) Example of a media URL is: 'file://C:/Cerify/./cerify_demo/sample.3gp'. Media URLs should be prefixed by eithe 'file://' or 'ftp://' or 'smb://'

f) The profile name, media location, media URL are submitted along with job name and priority to Cerify server

7.8 Irt mxf analyser



Figure 7.8: Irt mxf analyser icon

This Action plug-ins enables the submission of a file verification request to a IRT MXF Analyser server.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.5	Adding Remote Node Dependency
0.0.4	fixed problem when job name set in config
0.0.3	add more outputs
0.0.2	add more outputs
0.0.1	initial release

7.8.1 Saved Parameters Description

- Name: The name used to identify a saved IrtMxfAnalyser configured instance.
- Comments: Some comments about this saved IrtMxfAnalyser configured instance.
- Job Name: The name to give to the verification job. This is optional: if specified it has to be unique or the verification will fail.
- File path: The path to the file to verify. It refers to the path on the MxfAnalyser server. See Operating instructions for more details
- User Id for the job: The user id to use for the verification job. This is optional: if not specified orchestrator is used
- Priority: The priority of the verification job. This is optional: if not specified normal is used
- Analysis profile: The profile to use for the verification job. The available profiles depend on the server configuration.
- Service node: The node where Irt Mxf Analyser server is running, if configured as an external node. If the node is not configured use the advanced parameters section to set the address and port *Advanced parameters:*
- Server address: The ip address of the node where Irt Mxf Analyser server is running. This is optional: if the service node is set this value will be ignored
- Service port: The port at which the Irt Mxf Analyser server is listening. This is optional: if not set 8080 is used

7.8.2 Inputs description

The list of inputs depends on the configuration of the IrtMxfAnalyser action template.

All template parameters can be set at run time using the corresponding input.

At runtime *File path, Analysis profile and one between Service node and Server address* are mandatory and have to be specified.

7.8.3 Outputs description

- *Traffic light*: High level result of the validation. Possible values: Red, Green
- *Analysis report*: The complete validation report in XML
- *Link to Report page*: Link to the report page in the Mxf Analyser Server

7.8.4 Supported Actions

None

7.8.5 Dependencies

None

7.8.6 Operating Instructions

The plugin operates by creating a verification job on the selected Mxf Analyser server using the given inputs.

The user has to carefully think about how the MxfAnalyser server accesses the input file: the specified file path refers to the local path to the file on the Mxf Analyser machine: since Mxf Analyser runs as a service, if a network path or a mounted network drive is used the service has to run with an authorized user to access the file. Refer to Mxf Analyser manual for more information.

7.9 Mx fixer



Figure 7.9: Mx fixer icon

Analyse an MXF file for AS11 compatibility using Metaglue MX Fixer.

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.7	Adding Remote Node Dependency
0.1.6	- 2015/03/12 - ADM - Expose execution node as runtime input
0.1.5	Added ssh errors to list of outputs
0.1.4	Changed execution script name
0.1.3	Escaped status message
0.1.2	Fix bad chars in errors detail
0.1.1	Fix bad chars in status detail

Version	Comment
0.1.0	Added basic error handling
0.0.9	Supports for scripts that don't produce an XML output
0.0.8	Using pre-built script to avoid escaping issues with windows
0.0.7	Changed internal structure for better supportability
0.0.6	Expose script file path at runtime and in config. Fixed Edit page to correctly fill the report path.
0.0.5	Fixed windows path and pointed to newest Fixerscript exe
0.0.4	Prevents autocomplete of login field
0.0.3	fixed some typos in help file
0.0.2	Fixes edge cases in which report progress happens before/after the exit
0.0.1	initial release

7.9.1 Saved Parameters Description

- Name: The name used to identify a saved MxFixer configured instance. Document all available template parameters using add_parameter_help
- parameter name as shown by the label: description

7.9.2 Inputs description

- Document all available inputs using add_input_help
- *input name (refer to the constant in the class if available*: description

7.9.3 Outputs description

- Document all available outputs using add_output_help
- *input name (refer to the constant in the class if available)*: description

7.9.4 Supported Actions

None

7.9.5 Dependencies

None

7.9.6 Operating Instructions

Document how the plugin works and make some configuration examples if required.

7.10 Orion OTT Monitor



Figure 7.10: Orion OTT Monitor icon

This is Action plug-ins provides the ability to submit file to Orion OTT Monitor

Version	Comment
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.0	Add monitoring template functionality
0.1.1	Fixing options functionality and decreasing API calls
0.1.0	Added profile field for start monitoring api call
0.0.7	support for 1.5 version
0.0.6	more logging and rescue added to get pdf api
0.0.5	Help file fix
0.0.4	outputs fix to reset outputs to make it work in 2.6.1 or older orchestrator
0.0.3	fixed some issues and added playback url api call
0.0.2	help file added
0.0.1	initial development

7.10.1 Saved Parameters Description

- Name: The name used to identify a saved Orion Ott Monitor configured instance.
- Comments: Some comments about this saved Orion Ott Monitor configured instance.
- execution_node: Select the execution node for Orion OTT server
- server_address: If no remote node is configured for Orion, use this field to enter the server IP address
- orion_port: Port to access the Orion OTT server. Defaulted to 2029
- orion_username: Username to access the server. Defaulted to admin
- orion_password: Password to access the server. Defaulted to admin
- VOD Content Name: Content name for the content to be uploaded
- VOD Content URL: Content URL for the content to be monitored
- Encryption Type: Choose between CENC and Sample-AES encryption
- Register Keys: This for for the DRM keys registering part. If more than 1 key is present, enter them as a comma separated string.
- Decode Media Segments: Check this box if you want to decode media segments
- Monitoring Profile: Optional parameter for the profile to start monitoring. If not provided, the default is used. The default is determined by Orion OTT based on the format. The profile will not be used if a *Monitoring Template* is also specified.
- Monitoring Template: Optional parameter for the template to start monitoring. If not provided, the default is used.
- Enable Quality?: While monitoring check this option if you want to enable quality
- Enable Loudness?: While monitoring check this option if you want to enable loudness
- PDF storage Path: Enter the path to storage the monitoring result PDF file
- Content Status Polling Frequency: Polling frequency to follow the status of the content until it is not 'Initializing'. Default Value is 15
- Content Monitoring Polling Frequency: Polling Frequency to monitor the status of the job. Default value is 300

7.10.2 Inputs description

The list of inputs depends on the configuration of the OrionOttMonitor action template.

Register Keys: A Key is a combination of KeyID, Initialization Vector(in case of AES), Content Encryption Key. So separate them using ”|” symbol as mentioned in the Orion OTT API guide

7.10.3 Outputs description

- *Minor_Errors_Count*: The number of errors deemed minor.
- *Major_Errors_Count*: The number of errors deemed major.
- *Critical_Errors_Count*: The number of errors deemed critical.
- *Pdf_Report_Path*: The path to the auto-generated PDF report.
- *Content_Id*: The ID of the content that was monitored.
- *HTTP_Response_Code*: The HTTP response code that was returned.
- *Raw_Response_Hash*: A hash with the contents of the response received by the client.
- *Orion_Code_Received*: The error code returned by Orion. 0 indicates success while non zero value indicates an error.
- *Message_Received*: The status message returned by Orion.
- *Content_Register_Error_Message*: Error codes returned when registering content.
- *Key_Register_Error_Message*: Error codes returned when registering keys.

- *Asset_Summary*: A hash with the contents of the response received by the client when getting VOD content summary.
- *Play_Url*: The player URL returned by Orion.

7.10.4 Operating Instructions

7.11 Pulsar content verification



Figure 7.11: Pulsar content verification icon

This Action plug-in provides the ability to submit a content verification task to a Pulsar Server.

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.1	Removing soap4r dependency
0.1.0	Added styling to parameter help text.
0.0.9	Adding percentage complete variable
0.0.8	Added Remote Node Dependency
0.0.7	Added help file
0.0.6	2015-04-01 PM - Moving helper into the controller to remove plugin load errors.
0.0.5	2015-02-17 ML - New icon, reverts to default display name for consistency
0.0.4	2015-01-07 ML - removes helper file by merging code into controller
0.0.3	exception handling of RemoteNode.find_by_name()
0.0.2	Removes explicit dependency on xmlsimple
0.0.1	initial development

7.11.1 Saved Parameters Description

- Name: The name used to identify a saved Pulsar File Verification configured instance.
- Comments: The comments of the Pulsar File Verification template
- Pulsar Server Node: The remote node name of the server which hosts the Pulsar File Verification server
- Pulsar Server IP address (or hostname): The remote host name or remote IP of the server which hosts the Pulsar server
- Pulsar API port (optional): The port number of remote server which hosts the Pulsar server
- Pulsar Profiles: A dropdown of Pulsar profiles available on Pulsar server
- Source file: The file path for Pulsar to verify.
- Status Polling frequency: How often progress will be reported. Default is 5 seconds

7.11.2 Inputs description

The list of inputs depends on the configuration of the PulsarContentVerification action template.

- All parameters are available as inputs at runtime if not specified by the template.

  This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

These inputs are mandatory:

- *Pulsar Server Node or Pulsar IP Address*: Either a remote node must be selected or a remote IP address must be provided.
- *Template*: A Template is mandatory while submitting a job, hence click on Get Templates and select a Template.
- *Source file*: A source is mandatory while submitting a job, the file must be accessile to Pulsar.

These inputs are optional:

- *Name*: It is advisable that a name is entered to identify the template.
- *Comments*: Information about the template which will help identify the template.
- *Pulsar API Port*: The port is optional as a default value (8093) is used if not specified.
- *Status Polling frequency*: Specify if progress needs to be reported within a time other than default.

7.11.3 Outputs description

- *Content_Verification_Stats*: The statistics reported by Pulsar regarding the file verification
- *Error_description*: Errors reported by Pulsar
- *Content_Verification_Result*: Verification result from Succcess, Failed or Error
- *Job_ID*: The id of the job triggered by the file verification operation
- *Input_Media_Params*: Source file submitted to Pulsar

7.11.4 Operating Instructions

This plug-in allows users to provide necessary inputs required for a Pulsar server to do a file verification/QA job.

The mandatory inputs are listed above.

The plug-in supports the latest API release. An example of a QC job is given below.

7.11.5 An example of a QC workflow below

a) Provide remote node or remote server IP and port (example: 10.0.103.85 or transcode.lala.com)

b) Click the button Get Templats. This provides dropdowns with templates obtained from the provided Pulsar server

c) Select a template from the dropdown

d) Example of a source file is //10.0.0.1/Source/Movie.mpg

e) The template, source are submitted to Pulsar server

7.12 QScan file verification



Figure 7.12: QScan file verification icon

This action plug-in provides the ability to submit and monitor a file verification job to a QScan server.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.1	Initial implementation

7.12.1 Saved Parameters Description

- Name: The name used to identify a saved QscanFileVerification configured instance.
- Comments: Some comments about this saved QscanFileVerification configured instance.
- Address: The IP address or hostname of the QScan server.
- Port: The port number of the QScan server.
- User: The username to login to the QScan server.
- Password: The password to login to the QScan server.
- Template name: A dropdown of the QScan templates.
- Template id: The ID of the selected QScan template.
- Repository name: A dropdown of the QScan repositories.
- Repository id: The ID of the selected QScan repository.
- File path: The path to the source file inside a QScan repository.
- Job name: The name of the job.
- Save pdf report: If selected, the PDF report is saved.
- PDF report file path: The path of the PDF report file.
- Delete completed job: If selected, the completed job is deleted.
- Polling frequency: The interval in seconds to check for the job status.

7.12.2 Inputs description

The list of inputs depends on the configuration of the QscanFileVerification action template.

- These inputs are mandatory:
- *address*: The IP address or hostname of the QScan server.
- *user*: The username to login to the QScan server.
- *password*: The password to login to the QScan server.
- *template_id*: The ID of the QScan template.
- *repository_id*: The ID of the QScan repository.
- *file_path*: The path to the source file inside a QScan repository.
- *pdf_report_file_path*: The path of the PDF report file if 'save PDF report' is selected in configuration. These inputs are optional:
- *port*: The port number of the QScan server (default is '8080').
- *job_name*: The name of the job (default is an UUID).
- *polling_frequency*: The interval in seconds to check for the job status (default is '10' sec).

7.12.3 Outputs description

- *events*: The QC events found.
- *job_name*: The name of the job.
- *pdf_file_size*: The size of the PDF report file if 'save PDF report' is selected in configuration.

7.12.4 Supported Actions

None

7.12.5 Dependencies

None

7.12.6 Operating Instructions

7.13 Quasar Venera QC

This Action plug-ins enables the submission of a file verification request to an Quasar Venera QC server

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	Changing the api to JobDetails to get more information

Version	Comment
0.0.1	Initial Release

7.13.1 Saved Parameters Description

- Name: The name used to identify a saved QuasarVeneraQcOperation configured instance.
- Comments: The comments of the Quasar File Verification template
- End Point URL: URL of the Quasar Server.
- Request Name: Name of the request. (Availabe option currently is only Create Job)
- Account: Account number of the QC Server
- Api Key: Api key to access the QC server
- File Path: Entire location to the file path
- Template Name: Name of the template to be executed for the job
- Template Type: Type of the template (1 - General, 2 - Adaptive)
- Job Type: Type of the job (0 - General, One frame per file, 4 - Adaptive bit rate, 7 - Package)
- Access Key: Access key to access the file on S3
- Access Key Secret: secret key to access file in S3
- User Note: Notes associated with the job
- polling frequency: Frequency at which the status has to be polled

7.13.2 Inputs description

The list of inputs depends on the configuration of the QuasarVeneraQcOperation action template.

- All parameters are available as inputs at runtime if not specified by the template.

  This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

These inputs are mandatory:

- *end point url*: Enter it along with apibroker. Example -https://apisquasar.veneratech.com/apibroker/

7.13.3 Outputs description

- *Job_Status_Raw_Response*: Job Status polling response hash
- *Create_Job_Raw_Response*: Job Creation response hash
- *Error_Message*: Any error encountered in the job submission
- *Message_Received*: If no error encountered in the job submission, this field will have the value of the message sent by quasar.
- *Error_Count*: If any error present, this field will have the count of the error messages
- *Warning_Count*: If any warning present, this field will have the count of the warning messages. (On warning, we still complete the step if there are no errors)

7.13.4 Operating Instructions

7.14 Vidchecker verification

Executes and monitors quality control verifications on a VidChecker server.

Version	Comment
0.4.2	Enhanced action template UI
0.4.1	Prevent input/output pre-processing code from being overwritten
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.0	Added alternative call for report generation
0.2.9	Fixed typo in HTML
0.2.8	Added report type selector, as there can only be PDF or XML, not PDF_and_XML evidently from the wsdl file
0.2.7	Fixing Ruby warning and removing soap4r gem reference

Version	Comment
0.2.6	Fixed check status on error to support both Rejected and Reject. Fixed generation of file report. Added report file path to outputs
0.2.5	Added styling to parameter help text.
0.2.4	Returning percent_complete from check_status method to be consumed by the activities screen.
0.2.3	Adding Remote Node Dependency
0.2.2	Fixed check status on error to support both Rejected and Reject
0.2.1	Switch to file based wsdl file
0.2.0	New icon
0.1.9	Added Vidchecker Job ID as output
0.1.8	Removes explicit dependency on xmlsimple
0.1.7	Fixes API call for alerts information
0.1.6	Retrieve alerts information and expose them as output
0.1.5	Renamed sdk class to avoid conflicts with Vantage SDK
0.1.4	Added possibility to generate a report file
0.1.3	Added all task information to outputs
0.1.2	Removed xml report and added edit screen
0.1.1	Fixed problem with fail on warning
0.1.0	Added configurable polling frequency
0.0.3	Fixed problem with persistence
0.0.2	Fixed problem during job status check
0.0.1	Initial release

7.14.1 How It Works

- This action plug-in interacts with a VidChecker server to perform media file verification. The verification operations executed depend on the configuration of the selected VidChecker template.

7.14.2 Configuration Instructions

1. Create a new action step using the VidcheckerVerification plug-in.
2. This action performs media file verification using a VidChecker server.
3. Fill out the following configuration fields:
    - Name: The name used to identify a saved VidcheckerVerification configuration.
    - Comments: Optional comments about this VidcheckerVerification configuration.
    - Template: The name of the VidChecker template to execute.
    - VidChecker server node: The remote node on which the VidChecker server is installed.
    - VidChecker IP address: The IP address of the VidChecker server.
    - VidChecker API port: The port on which the VidChecker server listens for API requests.
    - Fail on warning: When selected, the step returns a Failed status if the verification result contains warnings.
    - Generate report file: When selected, VidChecker generates a detailed verification report file.
    - Report type: The format of the generated report (for example, XML or PDF).
    - Report file path: If report generation is enabled, specifies the directory where the report is stored. This path is local to the VidChecker server. Note: VidChecker does not support specifying a report file name; only the destination directory can be configured.

7.14.3 Example Configuration

This is a minimal setup configuration for the VidcheckerVerification plug-in.

Parameter	Value
Template	HD_Quality_Check
VidChecker server node	VidChecker_Node_01
VidChecker IP address	192.168.10.25
VidChecker API port	8080
Fail on warning	Selected
Generate report file	Selected
Report type	PDF

Parameter	Value
Report file path	/var/vidchecker/reports

7.14.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved VidcheckerVerification configuration.
Comments	Optional comments about this VidcheckerVerification configuration.
Template	The name of the VidChecker template to execute.
VidChecker server node	The remote node on which the VidChecker server is installed.
VidChecker IP address	The IP address of the VidChecker server.
VidChecker API port	The port on which the VidChecker server listens for API requests.
Fail on warning	When selected, the step returns a Failed status if the verification result contains warnings.
Generate report file	When selected, VidChecker generates a detailed verification report file.
Report type	The format of the generated report (for example, XML or PDF).
Report file path	If report generation is enabled, specifies the directory where the report is stored. This path is local to the VidChecker server. Note: VidChecker does not support specifying a report file name; only the destination directory can be configured.

7.14.5 Inputs Description

Required inputs depend on how the VidcheckerVerification template is configured. If dynamic attributes are used, each attribute becomes a runtime input.

Parameter	Description
*File_path **	The full path to the media file to be verified.
*Service_node_or_address **	The remote node or IP address of the VidChecker server.
*Template_id **	The identifier of the VidChecker template to execute.
Polling_frequency	The polling interval in seconds used to check verification progress.
Report_file_path	If report generation is enabled, the directory where the report file is stored.
Server_port	The port used by the VidChecker server for API requests.

7.14.6 Outputs Description

Parameter	Description
Alerts_count	The total number of alerts detected during verification.
Verification_result	The overall verification result.
Stream_info	A hash containing detailed stream information.
Task_alerts	A list of all alerts generated during verification.
Task_info	A hash containing detailed information about the verification tasks and results.

7.14.7 Supported Actions

None



Figure 7.13: Quasar Venera QC icon



Figure 7.14: Vidchecker verification icon

7.14.8 Dependencies

None

7.14.9 Operating Instructions

1. Configure the action to connect to a VidChecker server and select the VidChecker template to execute. The template defines which verification operations are performed.
2. Specify the media file to be verified. VidChecker executes all verification tasks defined in the selected template.
3. After verification completes, the action returns the total alert count along with detailed verification results.
4. Optionally enable report generation to produce a detailed verification report containing all task results.
5. When report generation is enabled, select the report type (XML or PDF). The report file is stored locally on the VidChecker server.

7.15 VqbifAnalyzer



Figure 7.15: VqbifAnalyzer icon

This plug-in proivdes capability to execute Vqbif commands on a remote node.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	Adding Remote Node Dependency
0.0.1	Plugin release

7.15.1 Saved Parameters Description

- This action allows to execute an VQBif analysis on a remote Windows node.
- Name: The name used to identify a saved VQBifAnalyzer configured instance.
- Comments: Comments associated to this template or configuration.
- Execution node: The remote node to execute the HandBrake operation from. List of already configured Remote Nodes.
- VQBif Binary path: The path (as seen in the SSH session) of the VQBif executable.
- Input file path: The path (as seen in the Windows Terminal) of the input file to VQBif.
- Input file path two: The path (as seen in the Windows Terminal) of the optional input file to VQBif.
- Input file path three: The path (as seen in the Windows Terminal) of the optional input file to VQBif.
- Output file path: The path (as seen in the Windows Terminal) of the output file of VQBif.(Preferred format is '.txt')
- Review Mode: The review mode for the JPEG export (Comes with two options, Full Set View Mode and Quick Review Mode)
- Jpeg Folder Path: The path where you need to export the JPEGs to.
- Framewise Separation: BIF(s) Framewise Separation checked against the specified value

7.15.2 Inputs description

- - Input file path and the Execution node, VQBif binary path are mandatory parameters for the execution of this plugin.

  - Output file path if not provided, will be defaulted to the input file name appending '_PASS' or '_FAIL' depending on the analysis results.

  - Input file path two and Input file path three, are optional inputs that can be given in a single command. At the most 3 inputs are be provded in a single command.

  - Review Mode is an optional input, which always you to exprot the JPEGs file. Full review mode will export all the JPEGs while Quick review mode will export 5x5x5 JPEGs.

  - Jpeg folder path is to specify where the JPEGs files to be exported to for either of the review mode selected. If Jpeg folder path is not provided, folder name will be defaulted to input file name follwed by '_JPG'

  - framewise Separation is again an optional input. Framewise Separation is checked against the specified value.

7.15.3 Outputs description

- This action executes an VQBif Analysis on a remote node, via SSH.
- *Output_File_Path*: The path of the output file
- *Transformation_Report*: A report of the VQBif Analysis operation

7.15.4 Supported Actions

None

7.15.5 Dependencies

None

7.15.6 Operating Instructions

This action plug-in provides the ability to perform a Analysis using the VQBif command line utility.

NOTE: VQBif Executable for command line execution should be installed on the remote node.

The Output file will be stored on the specified path in the remote node.

Note:1) VQBif requires file to be specified in the windows path format. (Example C:\Documents\sample.txt)

2) Output files are preferred to be in the '.txt' format. 3) Input path two, Input path three, Review mode and Framwize Separation are optional inputs.

Example

CONFIGURATION:

VQBif binary path : ”/cygdrive/c/Users/Administrator/Desktop/VideoQ_VQBIF/VQBIF.EXE”

Input file path : ”C:\Users\Administrator\Desktop\VideoQ_VQBIF\M.bif”

Output file path : ”C:\Users\Administrator\Desktop\VideoQ_VQBIF\M_result.txt”

OUTPUTS:

OutPut Target File : ”C:\Users\Administrator\Desktop\VideoQ_VQBIF\M_result.txt”

Short guide from VideoQ (source:http://www.videoq.com/Downloads/VQBIF_ShortGuide.txt):

Usage: VQBIF -i in1.BIF [in2.BIF] [in3.BIF] [-o out.*] [-j/-q] [jpeg_folder] [-r Ref_Framewise_Separation_ms]

Optional input filenames 2 and 3 are in JPEG quality descending order

Input BIF files must be of matching durations, frame sizes, etc.

[-o out.*] option specifies output Report file name and extension

If [-o] is omitted, Report file name defaults to in1.TXT

[-j] or [-q] options: JPEG files export to Default Folder or Specified Folderv

If [jpeg_folder] specified, VQBIF creates in it sub-folders in1, in2, etc.

If [jpeg_folder] is omitted, VQBIF creates default folders in1_JPG, in2_JPG, etc.

[-j]: all available JPEGs exported (Full Set View Mode)

[-q]: for each .BIF only 5+5+5 JPEGs exported (Quick Review Mode)

[-r] option: BIF(s) Framewise Separation checked against the specified value

7.16 Watson speech to text



Figure 7.16: Watson speech to text icon

This action plug-in provides the ability to get speech transcriptions with IBM Watson Speech to Text service.

Version	Comment
0.2.1	Enhanced action template UI
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	added API key support
0.0.2	IBM Bluemix is replaced by IBM Cloud in help file
0.0.1	Initial implementation

7.16.1 How It Works

- Converts spoken audio to text using IBM Watson Speech to Text cloud service.
- Supports multiple audio formats (WAV, MP3, FLAC, OGG, etc.) and languages.
- Operates in asynchronous mode, submitting recognition jobs and polling for completion.
- Supports keyword spotting to identify specific words or phrases with confidence thresholds.
- Authenticates using either API key or username/password credentials.
- Returns detailed transcription results including text, confidence scores, timestamps, and keyword matches.
- Configurable polling frequency to balance responsiveness and API usage.
- Requires an active IBM Cloud subscription to Watson Speech to Text service.

7.16.2 Configuration Instructions

1. Create a new action step using the WatsonSpeechToText plugin.
2. Fill out the following fields:
    - Name: The name used to identify a saved Watson Speech to Text configured instance.
    - Comments: Comments about this saved Watson Speech to Text configured instance.
    - File path: The path to the source audio file to transcribe.
    - Url: The URL of IBM Watson Speech to Text service endpoint.
    - Use api key: If selected, the API key is used for authentication instead of username and password.
    - Api key: The API key of IBM Watson Speech to Text service (required if 'Use api key' is selected).
    - Username: The username of IBM Watson Speech to Text service (required if 'Use api key' is not selected).
    - Password: The password of IBM Watson Speech to Text service (required if 'Use api key' is not selected).
    - Model: The identifier of the model to be used for the recognition request (e.g., en-US_BroadbandModel, en-GB_NarrowbandModel).
    - Keywords: The list of keywords to spot in the audio. Enter one or more values comma-separated.
    - Keywords threshold: The confidence value (0.0 to 1.0) for spotting a keyword. Keywords with confidence above this threshold are reported.
    - Polling frequency: The interval in seconds to check for the job status. Lower values provide faster results but increase API calls.

7.16.3 Example Configuration

This is an example configuration for the WatsonSpeechToText plugin with keyword spotting enabled.

Parameter	Value
Name	Transcribe Customer Call
Comments	Transcribe recorded customer service calls
File path	/recordings/call_20240115_001.wav
Url	https://api.us-south.speech-to-text.watson.cloud.ibm.com
Use api key	Selected
Api key	your-api-key-here
Model	en-US_BroadbandModel
Keywords	refund, complaint, cancel, supervisor
Keywords threshold	0.7
Polling frequency	5

7.16.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved Watson Speech to Text configured instance.
Comments	Comments about this saved Watson Speech to Text configured instance.
File path	The path to the source audio file to transcribe.
Url	The URL of IBM Watson Speech to Text service endpoint.
Use api key	If selected, the API key is used for authentication instead of username and password.
Api key	The API key of IBM Watson Speech to Text service (required if 'Use api key' is selected).
Username	The username of IBM Watson Speech to Text service (required if 'Use api key' is not selected).

Parameter	Description
Password	The password of IBM Watson Speech to Text service (required if 'Use api key' is not selected).
Model	The identifier of the model to be used for the recognition request (e.g., en-US_BroadbandModel, en-GB_NarrowbandModel).
Keywords	The list of keywords to spot in the audio. Enter one or more values comma-separated.
Keywords threshold	The confidence value (0.0 to 1.0) for spotting a keyword. Keywords with confidence above this threshold are reported.
Polling frequency	The interval in seconds to check for the job status. Lower values provide faster results but increase API calls.

7.16.5 Inputs Description

The required inputs depend on the authentication mode selected. API key or username/password must be provided.

Parameter	Description
*file_path **	The path to the source audio file. Supported formats include WAV, MP3, FLAC, OGG, OPUS, WebM, and more. The file must be accessible by the Orchestrator.
*url **	The URL of IBM Watson Speech to Text service endpoint (e.g., https://api.us-south.speech-to-text.watson.cloud.ibm.com).
*api_key **	The API key of IBM Watson Speech to Text service. Required if 'Use api key' is selected in configuration.
*username **	The username of IBM Watson Speech to Text service. Required if 'Use api key' is not selected in configuration.
*password **	The password of IBM Watson Speech to Text service. Required if 'Use api key' is not selected in configuration.
model	The identifier of the model to be used for the recognition request. Default is 'en-US_BroadbandModel'. Choose based on audio quality (broadband vs narrowband) and language.
keywords	Comma-separated list of keywords to spot in the audio (e.g., 'urgent, complaint, refund'). Watson identifies these words with confidence scores.
keywords_threshold	The confidence value (0.0 to 1.0) for spotting a keyword. Default is '0.5'. Only keywords meeting or exceeding this threshold are reported.
polling_frequency	The interval in seconds to check for the job status. Default is '10' seconds. Balance between responsiveness and API usage.

7.16.6 Outputs Description

Parameter	Description
results	The complete job output containing transcribed text, confidence scores, timestamps, word alternatives, and keyword spotting results. Returned as a JSON structure with detailed recognition data.

7.16.7 Supported Actions

None

7.16.8 Dependencies

Requires an active IBM Cloud subscription to Watson Speech to Text service. You get the credentials (URL, username, password, API key) with an online subscription.

7.16.9 Operating Instructions

This plug-in converts spoken audio to text using IBM Watson Speech to Text cloud service. The service operates asynchronously, submitting recognition jobs and polling for completion.

Authentication Modes

- API Key Authentication (Recommended):
  - Select 'Use api key' checkbox in configuration
  - Provide the API key in the 'Api key' field
  - Username and password are not required
  - More secure and easier to manage
- Username/Password Authentication (Legacy):
  - Leave 'Use api key' checkbox unchecked
  - Provide username in the 'Username' field
  - Provide password in the 'Password' field
  - API key is not required
  - Used for older Watson service instances

Execution Flow

- Step 1 - Job Submission: Plugin uploads the audio file and submits a recognition job to Watson Speech to Text service.
- Step 2 - Asynchronous Processing: Watson service processes the audio file in the background. Processing time depends on file size and duration.
- Step 3 - Status Polling: Plugin polls the job status at the configured polling frequency (default: 10 seconds).
- Step 4 - Job Completion: When Watson completes processing, plugin retrieves the full recognition results.
- Step 5 - Results Return: Complete transcription results are returned in the 'results' output, including text, confidence scores, timestamps, and keyword matches.

Supported Audio Formats

- Lossless Formats (Recommended):
  - WAV (audio/wav) - Uncompressed, best quality
  - FLAC (audio/flac) - Lossless compression
- Compressed Formats:
  - MP3 (audio/mp3) - Widely supported, good quality
  - OGG (audio/ogg) - Open format with good compression
  - OPUS (audio/opus) - High-quality speech compression
  - WebM (audio/webm) - Modern web format
- Telephony Formats:
  - G.729, mu-law (8 kHz audio) - Use narrowband models
- Audio Requirements:
  - Maximum file size: Typically 100 MB (varies by service plan)
  - Sample rate: 8 kHz (narrowband) or 16+ kHz (broadband)
  - Duration: No strict limit, but longer files take more time
  - Channels: Mono preferred, stereo supported

Setup and Configuration Steps

- Step 1: Subscribe to IBM Watson Speech to Text
  - Go to IBM Cloud (https://cloud.ibm.com)
  - Create an account or sign in
  - Navigate to the Catalog and search for 'Speech to Text'
  - Create a new Speech to Text service instance
  - Select an appropriate pricing plan
- Step 2: Get Service Credentials
  - From the IBM Cloud dashboard, open your Speech to Text service instance
  - Go to 'Manage' → 'Credentials' or 'Service credentials'
  - Create new credentials if needed
  - Copy the API Key and URL (or Username/Password for legacy instances)
- Step 3: Prepare Audio File
  - Ensure audio file is in a supported format (WAV, MP3, FLAC, etc.)
  - Verify sample rate matches model type (8 kHz for narrowband, 16+ kHz for broadband)
  - Place the audio file in a location accessible by Orchestrator and note the full file path
- Step 4: Configure Orchestrator Plugin
  - Create a new Watson Speech to Text action in your workflow
  - Enter the service URL from IBM Cloud
  - Select 'Use api key' and enter your API key (or use username/password)
  - Choose appropriate model based on audio quality and language
  - Specify the audio file path

  - Configure keywords and threshold if needed
- Step 5: Test Configuration
  - Use a short test audio file to verify connectivity
  - Check the 'results' output for transcription accuracy
  - Verify keyword spotting if configured
  - Adjust model, keywords, or threshold as needed

Common Use Cases

- Call center transcription: Transcribe customer service calls for quality assurance, compliance, and analytics.
- Meeting transcription: Convert recorded meetings and interviews into searchable text documents.
- Voicemail transcription: Automatically transcribe voicemail messages for email delivery.
- Content accessibility: Create text captions for video and audio content.
- Keyword monitoring: Detect mentions of specific terms (products, competitors, issues) in audio content.
- Legal/Compliance: Transcribe depositions, hearings, and legal proceedings.
- Media monitoring: Transcribe broadcast content for searchability and analysis.
- Voice analytics: Extract data from customer interactions for sentiment and trend analysis.

Best Practices

- Choose the appropriate model based on audio quality and language (broadband for high-quality, narrowband for telephony).
- Use keyword spotting for long recordings to quickly locate important topics or concerns.
- Set an appropriate keywords threshold (0.5-0.7 recommended) to balance accuracy and coverage.
- Adjust polling frequency based on file size - shorter intervals for small files, longer for large files to reduce API calls.
- Ensure audio files are accessible from the Orchestrator server (local filesystem or mounted network share).
- Use high-quality audio when possible - clear recording with minimal background noise improves accuracy.
- Store credentials securely - prefer API key authentication over username/password.
- Monitor IBM Watson service usage and quotas to avoid service interruptions.
- Implement error handling for network failures, unsupported formats, or service unavailability.
- For large-scale operations, consider batch processing to optimize costs and throughput.

Troubleshooting

- Authentication failures: Verify API key or username/password are correct and the service is active in IBM Cloud.
- File not found errors: Ensure the audio file path is correct and accessible by Orchestrator.
- Unsupported format errors: Verify audio format is supported and properly encoded.
- Model not found errors: Confirm model identifier is correct and supported by your Watson service plan.
- Job timeout: Large files take longer to process - increase polling frequency or check Watson service status.
- Poor transcription quality: Check audio quality, sample rate, and model selection. Consider using broadband models for high-quality audio.
- Quota exceeded: Monitor your Watson service usage in IBM Cloud dashboard and upgrade plan if needed.
- Keyword not spotted: Lower keywords threshold or verify spelling. Remember matching is case-insensitive.

7.17 Watson video enrichment



Figure 7.17: Watson video enrichment icon

This action plug-in provides the ability to get structured information from video files with IBM Watson Video Enrichment service.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	Hides API key field
0.0.1	Initial implementation

7.17.1 Saved Parameters Description

- Name: The name used to identify a saved WatsonVideoEnrichment configured instance.
- Comments: Some comments about this saved WatsonVideoEnrichment configured instance.
- Video url: The URL of the video file.
- Api key: The secret API key of IBM Watson Video Enrichment service.
- Job name: The name of the job.
- Language: The language code in RFC-5646 format.
- Caption url: The URL of the caption file used as a transcript.
- Service url: The URL of IBM Watson Video Enrichment service.
- Return profanity analysis result: If selected, the profanity analysis result is returned.
- Save transcript result: If selected, the transcript result is saved.
- Transcript result file path: The path of the transcript result file.
- Return video analysis result: If selected, the video analysis result is returned.
- Polling frequency: The interval in seconds to check for the job status.

7.17.2 Inputs description

The list of inputs depends on the configuration of the WatsonVideoEnrichment action template.

- These inputs are mandatory:
- *video_url*: The URL of the video file.
- *api_key*: The secret API key of IBM Watson Video Enrichment service.
- *transcript_result_file_path*: The path of the transcript result file if 'save transcript result' is selected in configuration.
  These inputs are optional:
- *job_name*: The name of the job.
- *language*: The language code in RFC-5646 format.
- *caption_url*: The URL of the caption file used as a transcript.
- *service_url*: The URL of IBM Watson Video Enrichment service (default is 'https://api-ams.watsonmedia.ibm.com/video-enrichment/v2').
- *polling_frequency*: The interval in seconds to check for the job status (default is '10' sec).

7.17.3 Outputs description

- *profanity-analysis_url*: The URL of the profanity analysis result.
- *transcript_url*: The URL of the transcript result.
- *video-analysis_url*: The URL of the video analysis result.
- *profanity-analysis*: The profanity analysis result if 'return profanity analysis result' is selected in configuration.
- *transcript_file_size*: The size of the transcript result file if 'save transcript result' is selected in configuration.
- *video-analysis*: The video analysis result if 'return video analysis result' is selected in configuration.

7.17.4 Supported Actions

None

7.17.5 Dependencies

None

7.17.6 Operating Instructions

The secret API key of IBM Watson Video Enrichment service is received by users after registration.

Pre-signed URL can be used for video and caption files.

7.18 XSD Validation



Figure 7.18: XSD Validation icon

This action plug-in is used to validate an XML file against an XSD template.

Version	Comment
0.2.1	Enhanced action template UI
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.2	Adding rescue on file cleanup
0.1.1	Invalid migration naming fix
0.1.0	support for multiple XSDs
0.0.1	initial release

7.18.1 How It Works

- Validates XML documents against XSD schemas to ensure structural and data type compliance.
- Supports validation against single or multiple XSD schemas.
- Accepts XML input as direct text or file path.
- Accepts XSD schemas as direct text or file path(s).
- Uses xmllint command-line tool for validation (part of libxml2).
- Returns validation result (success or failure) with detailed error messages.
- On failure, provides error location (line number) and specific XSD that failed.
- Enables XML quality gates in workflows to prevent invalid data processing.

7.18.2 Configuration Instructions

1. Create a new action step using the XsdValidation plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved XSD Validation configured instance.
   - Comments: Comments about this saved XSD Validation configured instance.
   - Xml body: Enter the XML content to be validated directly in the text box.
   - Xml file path: Allows entering XML file path instead of pasting XML content in the text box.
   - Use multiple XSD: Check if the XML needs to be validated against multiple XSD schema.
   - Xsd body: Enter the XSD schema content for validation directly in the text box.
   - Xsd file path: Allows entering XSD file path instead of pasting XSD content in the text box (single XSD).
   - Xsd file paths: Allows entering multiple XSD file paths when 'Use multiple XSD' is checked.
   - Validation command: Enter only if xmllint is not installed or you need to use a custom validation tool.

7.18.3 Example Configuration

This is an example configuration for the XsdValidation plugin validating an order XML file.

Parameter	Value
Name	Validate Order XML

Parameter	Value
Comments	Validate order XML against shiporder schema
Xml file path	/data/orders/order_12345.xml
Use multiple XSD	Not selected
Xsd file path	/schemas/shiporder.xsd
Validation command	(blank - use default xmllint)

7.18.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved XSD Validation configured instance.
Comments	Comments about this saved XSD Validation configured instance.
Xml body	Enter the XML content to be validated directly in the text box.
Xml file path	Allows entering XML file path instead of pasting XML content in the text box.
Use multiple XSD	Check if the XML needs to be validated against multiple XSD schema.
Xsd body	Enter the XSD schema content for validation directly in the text box.
Xsd file path	Allows entering XSD file path instead of pasting XSD content in the text box (single XSD).
Xsd file paths	Allows entering multiple XSD file paths when 'Use multiple XSD' is checked.
Validation command	Enter only if xmllint is not installed or you need to use a custom validation tool.

7.18.5 Inputs Description

Usually this plugin does not have runtime specified parameters, unless an external file is used for the configuration, in which case the file path can be specified at runtime.

Parameter	Description
Xml_body	The XML content as a string. Used when XML is provided directly rather than via file path.
Xml_file_path	Path to the XML file to be validated. Can be specified at runtime if using external files.
Xsd_body	The XSD schema content as a string. Used when XSD is provided directly rather than via file path.
Xsd_file_path	Path to the XSD schema file (single schema validation).
Xsd_file_paths	List of XSD file paths (multiple schema validation).
Validation_command	Custom validation command if xmllint is not available or alternative tool is needed.

7.18.6 Outputs Description

The output is the validation result and a failure message in case of validation failure.

Parameter	Description
Validation_Result	Boolean indicating validation success (true) or failure (false)
Validation_Message	Detailed validation message. On success: confirmation. On failure: error description, line number, and failing XSD (if multiple)
Error_Line	Line number in XML where validation failed (available on failure)
Error_Column	Column number in XML where validation failed (available on failure)
Failing_Schema	Name of the XSD schema that failed validation (relevant when using multiple XSDs)

7.18.7 Supported Actions

None

7.18.8 Dependencies

Requires xmllint (part of libxml2 library) to be installed on the Orchestrator server.

7.18.9 Operating Instructions

This action validates a given XML against one or more given XSD schemas and returns the validation message. In case of validation failure, the message contains the failure description, the location of validation failure, and the XSD against which the validation failed (if multiple XSDs were used).

Single vs Multiple XSD Validation

- Single XSD Validation:
  - Validate XML against one XSD schema
  - Use 'Xsd file path' or 'Xsd body' parameter
  - Leave 'Use multiple XSD' unchecked
  - Best for: Simple XML documents with self-contained schema
- Multiple XSD Validation:
  - Validate XML against multiple XSD schemas
  - Check 'Use multiple XSD' option
  - Provide file list under 'Xsd file paths' parameter
  - Useful when: XML uses multiple namespaces or imported schemas
  - Example: Main schema + shared type definitions schema
  - Validation fails if ANY schema reports errors

xmllint Validation Tool

- xmllint is the default validation tool, part of the libxml2 library.
- Custom Validation Command:
  - Only specify if xmllint is not available or you need a different tool
  - Must accept XML and XSD file paths as arguments
  - Must return exit code 0 for success, non-zero for failure
  - Must output error messages to stdout or stderr

Common Use Cases

- Data quality gates: Ensure XML data meets schema requirements before processing.
- API payload validation: Validate incoming XML payloads before business logic execution.
- Configuration file validation: Verify XML configuration files are properly structured.
- Document processing: Validate XML documents before transformation or storage.
- Compliance checking: Ensure XML messages comply with industry standards (HL7, EDIFACT, etc.).
- Partner integration: Validate XML from external partners before processing.
- Workflow branching: Route to different processing paths based on validation success/failure.

Best Practices

- Use file paths rather than inline XML/XSD for large or frequently changing schemas.
- Store XSD schemas centrally for reuse across multiple workflows.
- Implement error handling workflows to process validation failures appropriately.
- Use descriptive names for XSD files when validating with multiple schemas.
- Test XSD schemas independently before integrating into workflows.
- Include line/column numbers in error notifications for easier debugging.
- Use validation as a quality gate early in workflows to fail fast on bad data.
- For multi-namespace XML, ensure all required XSD schemas are provided.
- Keep XSD schemas version controlled and synchronized with XML data formats.
- Log validation results for audit and troubleshooting purposes.

Example XML and XSD

Sample XML (shiporder.xml):

\ <?xml version=\"1.0\" encoding=\"UTF-8\"?>\ <shiporder orderid=\"889923\"\ xmlns:xsi=\"http://www.w3.org/2001/XMLSch
instance\"\ xsi:noNamespaceSchemaLocation=\"shiporder.xsd\">\ <orderperson>John Smith</orderperson>\ <shipto>\
<name>Ola Nordmann</name>\ <address>Langgt 23</address>\ <city>4000 Stavanger</city>\ <country>Nor-
way</country>\ </shipto>\ <item>\ <title>Empire Burlesque</title>\ <note>Special Edition</note>\ <quan-

tity>1</quantity>\ <price>10.90</price>\ </item>\ <item>\ <title>Hide your heart</title>\ <quantity>1</quantity>\ <price>9.90</price>\ </item>\ </shiporder>\

Corresponding XSD Schema (shiporder.xsd):

\ <?xml version=\"1.0\" encoding=\"UTF-8\"?>\ <xs:schema xmlns:xs=\"http://www.w3.org/2001/XMLSchema\">\ <xs:element name=\"shiporder\">\ <xs:complexType>\ <xs:sequence>\ <xs:element name=\"orderperson\" type=\"xs:string\"/>\ <xs:element name=\"shipto\">\ <xs:complexType>\ <xs:sequence>\ <xs:element name=\"name\" type=\"xs:string\"/>\ <xs:element name=\"address\" type=\"xs:string\"/>\ <xs:element name=\"city\" type=\"xs:string\"/>\ <xs:element name=\"country\" type=\"xs:string\"/>\ </xs:sequence>\ </xs:complexType>\ </xs:element>\ <xs:element name=\"item\" maxOccurs=\"unbounded\">\ <xs:complexType>\ <xs:sequence>\ <xs:element name=\"title\" type=\"xs:string\"/>\ <xs:element name=\"note\" type=\"xs:string\" minOccurs=\"0\"/>\ <xs:element name=\"quantity\" type=\"xs:positiveInteger\"/>\ <xs:element name=\"price\" type=\"xs:decimal\"/>\ </xs:sequence>\ </xs:complexType>\ </xs:element>\ </xs:sequence>\ <xs:attribute name=\"orderid\" type=\"xs:string\" use=\"required\"/>\ </xs:complexType>\ </xs:element>\ </xs:schema>\

Manual Testing with xmllint

You can test XML validation manually before configuring in Orchestrator:

- Validate XML against XSD: `xmllint --schema schema.xsd file.xml --noout`
- Check XML well-formedness: `xmllint --noout file.xml`
- Validate with DTD: `xmllint --dtdvalid schema.dtd file.xml --noout`
- View validation errors: Remove `--noout` to see detailed output

Troubleshooting

- xmllint not found: Install libxml2-utils (Linux) or libxml2 (macOS). Verify with `xmllint --version`.
- Schema not found: Verify XSD file path is correct and accessible from Orchestrator server.
- XML file not found: Verify XML file path is correct and Orchestrator has read permissions.
- Namespace errors: Ensure XML declares all required namespaces and XSD defines them.
- Import/include failures: When using multiple XSDs, ensure all referenced schemas are provided.
- Validation always fails: Verify XML well-formedness first (valid XML syntax). Check XSD syntax separately.
- Unclear error messages: Test XML and XSD manually with `xmllint --schema schema.xsd file.xml`.
- Permission denied: Ensure Orchestrator process has read permissions on XML and XSD files.
- Custom command not working: Verify command format matches xmllint interface and returns proper exit codes.

Important Notes

- XML and XSD can be provided either as direct text or file paths, but not both simultaneously.
- When using file paths, ensure files are accessible from Orchestrator server.
- 'Use multiple XSD' requires a list of XSD file paths under the 'Xsd file paths' parameter.
- Ensure that the aspweb user has appropriate permissions to access required files.
- The plugin uses xmllint by default. Only specify 'Validation command' if using alternative tool.
- Validation fails if XML is not well-formed (invalid XML syntax) before schema validation occurs.
- Error messages include line and column numbers for precise error location.
- When using multiple XSDs, the order may matter if schemas have dependencies.
- XSD schemas must be in XML Schema Definition format (not DTD or other schema types).
- File path inputs can be specified at runtime if not configured in template.
- Validation is synchronous - workflow waits for validation to complete.

Chapter 8

Resources

8.1 Load balancer



Figure 8.1: Load balancer icon

This action plug-in is a tool to manage resource distribution from a pool of resources using round-robin or random distribution algorithms.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.4	Leverages DB for round robin tracking instead of file system
0.0.3	New category for resource management
0.0.2	New icon
0.0.1	First version

8.1.1 How It Works

- Manages resource distribution from a pool of available resources using round-robin or random distribution algorithms.
- Provides load balancing across multiple resources to ensure even distribution of workload.
- Supports both static resource lists, defined at design time, and dynamic lists provided at runtime.
- Maintains resource state across workflows with configurable unicity scopes for shared resource pools.
- Accepts resource lists as comma- or semicolon-separated strings, or as arrays with dynamic attribute support.

8.1.2 Configuration Instructions

1. Add a LoadBalancer step to your workflow.
2. Configure the basic load balancing parameters:
   - Resource list: Enter comma or semicolon-separated list of resources (e.g., 'x,y,z').
   - Use random distribution: Leave unchecked for round-robin (default) or check to enable random selection.
   - Unicity scope: Select 'absolute' for isolated resource pools, or choose another scope for shared pools.

3. Configure optional advanced settings:
    - Resource unique name: Optional identifier for shared resource pools across workflows.

8.1.3 Example Configuration

Parameter	Example Value
Name	New LoadBalancer Step
Resource list	x,y,z
Resource unique name	optional
Unicity scope	absolute

8.1.4 Expected Test Results

With a resource list of ”x,y,z” and 3 work order executions, the round-robin distribution selects resources in the following sequence:

- First execution: Resource 'x' selected
- Second execution: Resource 'y' selected
- Third execution: Resource 'z' selected
- Demonstrates even distribution across the resource pool

8.1.5 Saved Parameters Description

Parameter	Description
Name	Display name for this load balancer step.
Comments	Notes or description for this step.
Use random distribution	When checked, uses random selection instead of default round-robin distribution.
Resource list	Pool of resources to distribute, separated by comma or semicolon (e.g., 'res1,res2,res3' or 'res1;res2;res3').
Resource unique name	Optional unique identifier for the resource pool. Useful when multiple workflows share the same load balancer. If not specified, a unique name is generated based on template/action name and ID.
Unicity scope	Defines resource pool sharing scope. Select 'absolute' for isolated pools, or choose another scope when multiple workflows need to pull from a common resource pool.

8.1.6 Inputs Description

Both 'Resource list' and 'Resource unique name' support dynamic attributes using <%=var_name%> syntax, making these variables required runtime inputs when used.

- Resource list: Mandatory input when left blank in configuration. Can be provided as array or comma/semicolon-separated string.
- Resource unique name: Optional runtime override for resource pool identifier.
- Dynamic variables: Any variables referenced using <%=var_name%> syntax in resource list or unique name fields.

8.1.7 Outputs Description

- Resource: The resource selected from the available pool based on the configured distribution algorithm.

8.1.8 Supported Actions

None

8.1.9 Dependencies

None

8.1.10 Operating Instructions

8.1.11 Distribution Algorithms

- Round-robin (default): Cycles through resources sequentially, ensuring even distribution across all resources in the pool.
- Random distribution: Selects resources randomly from the pool when 'Use random distribution' is checked.

8.1.12 Resource List Formats

- Comma-separated: `/distribution/vol1, /distribution/vol2, /distribution/vol3`
- Semicolon-separated: `/distribution/vol1;/distribution/vol2;/distribution/vol3`
- No spaces required: `/distribution/vol1,/distribution/vol2,/distribution/vol3`
- Dynamic attributes: `<%=distribution_root%>/vol1;<%=distribution_root%>/vol2;<%=distribution_root%>/vol3`

8.2 Load manager



Figure 8.2: Load manager icon

This action plugin is a tool to balance requests for a pool of resources with specific capacities. It provides a way to pause workflows until enough resources become available, and routes requests to the least taxed resources.

Version	Comment
0.5.3	Enhanced action template UI
0.5.2	Supports tags with expected queued item
0.5.1	Updated the label from Keep ongoing to Continue Monitoring
0.5.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.4.7	Fix cancel operation issue and set no timeout
0.4.6	Fixed possible error with empty tags
0.4.5	Added support for tags
0.4.4	Fixes help and added section about pool name restrictions.
0.4.3	Adds flexibility for naming resource pool.
0.4.2	Fixes allocation policy in plugin configuration view. Improves help.
0.4.1	Fix to always display and expose allocation policy.
0.4.0	Exposes allocation policy
0.3.0	Creates default label if none provided
0.2.9	Avoids double checking for resource upon initialization
0.2.8	Fixes creation of generic resources based on run-time parameters
0.2.7	Fixes ignored hold timeout
0.2.6	fixes typo in constant name (DEFAULT_MAX)
0.2.5	Typo in edit page - select pool
0.2.4	Runtime support for pool name
0.2.3	Runtime support for pool name
0.2.2	Bug fix in the UI
0.2.1	input spec bug fix after 0.2.0

Version	Comment
0.2.0	Pool Select from Resource Manager page
0.1.8	Frees the allocated resource in case a proper lock on queued item was not obtained
0.1.7	Perform a first check for resource availability ASAP
0.1.6	Selection of the RUN TIME value for queue does not persist in the dropdown although saved correctly in the database
0.1.5	New category for resource management
0.1.4	Fixed possibility to specify resource capacity at run time
0.1.3	Fix issue preventing use of RUNTIME queue
0.1.2	Use the queue_item_description as allocation label unless differently specified or not available
0.1.1	New icon
0.1.0	added keep trigger on going mode
0.0.2	fixes race condition if multiple triggers are used in conjunction with a single queue
0.0.1	first tracked version (requires ManagedResource object introduced in 2.0.0)

8.2.1 How It Works

- Manages shared resource pools across multiple workflow executions, providing advanced resource allocation and queuing capabilities beyond the basic ResourceManager plugin.
- Supports both Reserve and Release operations for controlling access to limited resources with configurable capacity and allocation policies.
- Integrates with queue systems to enable sophisticated workflow coordination, allowing elements to wait for both resource availability and queue position.
- Provides flexible allocation policies including fair distribution (round-robin) and sequential allocation for optimized resource utilization.
- Supports resource capacity management where individual resources can handle multiple concurrent allocations based on capacity limits.
- Includes timeout mechanisms for both waiting for resources and automatic resource release to prevent deadlocks and ensure resource availability.

8.2.2 Configuration Instructions

1. Add a Load Manager step to your workflow.
2. Configure the resource pool:
    - Action: Choose 'Reserve' to allocate resources or 'Release' to free resources.
    - Resource pool unique name: Enter a unique identifier for the resource pool.
    - Generic resources maximum number: Set the total number of resources available.
    - Generic resources capacity: Define the capacity of each resource (default: 1).
3. Set up allocation parameters:
    - Allocation block size: Specify the capacity needed per allocation.
    - Allocation policy: Choose fair (round-robin) or sequential allocation.
    - Wait timeout: Set maximum wait time for resource availability.
    - Polling frequency: Configure how often to check resource status.
4. Configure advanced options (optional):
    - Continue Monitoring: Enable for continuous resource allocation cycles.
    - Queue: Link with a queue system for coordinated workflow management.
    - Unicity scope: Set scope for shared resource pools across workflows.

8.2.3 Example Configuration

Parameter	Example Value
Action	Reserve
Resource pool unique name	transcoding-servers
Generic resources maximum number	4
Generic resources capacity	2
Allocation block size	1
Allocation policy	fair

Parameter	Example Value
Wait timeout	300
Hold timeout	3600
Polling frequency	5
Continue Monitoring	checked
Unicity scope	absolute

8.2.4 Saved Parameters Description

Parameter	Description
Continue Monitoring	(Only used when action is Allocation) Wait for enough available resources and then spawn a new workflow instance each time there are enough resources and (optionally) an element is available in the queue. The resource is allocated and the workflow proceeds with the next steps. A new instance of the work order will be created and wait for another available resource. If this option is not set, waits for an available resource and then proceeds with the execution without spawning a new work order instance.
Action	Specifies which action to perform in the step. It can either be a 'Reserve' (wait for an available resource), or a 'Release' of an already allocated resource.
Generic resources maximum number	The number of resources available in the pool unless resources are listed in Resource List.
Generic resources capacity	The capacity of each resource. By default it is 1.
Resource list	A comma separated list of values, each represents a resource managed. NOTE: a more complex notation allows to specify a list of resources with different capacity: { 'resource1' => capacity1, 'resource2' => capacity2 }.
Resource pool unique name	The resource pool unique name. IMPORTANT: the name must not include the termination '__a'. See operating instructions for details.
Unicity scope	If several workflows need to pull from a common pool of resources, a unicity scope can be set, otherwise, 'absolute' should be selected.
Polling frequency	The frequency in seconds at which the step will poll for resource status.
Allocation label	The label used to describe the allocation activities. It can be dynamically composed at runtime using the <%=value_to_compute_at_runtime%> notation.
Allocation policy	The policy used to allocate available resources. Possible values include fair (round-robin distribution) and sequential (fill first resource completely before moving to next).
Allocation block size	The amount of capacity needed by the allocation requester.
Wait timeout	How long a work order should wait for a resource to be available to process the specified request.
On wait timeout action	If a resource is not available before the specified timeout, the step can either fail, pass or exit with an error status.
Hold timeout	How long a resource can be held by a work order. After this timeout the resource is given back to the pool automatically. NOTE: no notification is signaled to the work order previously holding the resource.
Queue	If specified, the load manager will retrieve elements from the queue before performing the allocation (see operating instructions for detailed usage).
Use weight for allocation?	If selected, the weight of the element in the queue is used as the capacity to request from the load manager for the allocation.

8.2.5 Inputs Description

If 'Resource list' and 'Generic resource maximum number' are left blank, then 'Resource list' is a mandatory input and can be provided either as an array or as a string with the different items comma or semi-colon separated. One important runtime input is Managed_resources which corresponds to the Resource list design time configuration parameter.

- Managed_resources: Corresponds to the Resource list configuration parameter. Can be provided as an array or comma/semi-colon separated string.

- resource_pool_name: Runtime specification of the resource pool unique name if not configured in template.
- action: Runtime override for the action type (Reserve or Release).
- allocation_block_size: Runtime specification of the capacity needed for allocation.
- allocation_policy: Runtime override for the allocation policy (fair or sequential).
- wait_timeout: Runtime specification of maximum wait time for resource availability.
- queued_item: When linked with a queue, specifies the item to wait for at the top of the queue.
- allocation_label: Runtime specification of the label for allocation activities.

8.2.6 Outputs Description

- Resource_reserved: The resource selected from the pool available.
- Allocation_token: The allocation token used for tracking and releasing the resource.
- Capacity_reserved: The capacity reserved in the selected resource.
- Capacity_released: The capacity released to the specified resource.
- Queued_item: The item extracted from the queue to be processed.
- Queued_item_description: The description of the item extracted from the queue to be processed.

8.2.7 Supported Actions

- Reserve - Allocate resources from the pool
- Release - Return resources to the pool for reuse

8.2.8 Operating Instructions

- Basic Resource Management: The LoadManager is an advanced version of the ResourceManager plugin that allows management of a limited amount of resources shared between multiple workflow executions. In its simpler form, the LoadManager behaves exactly as a ResourceManager: it waits until a resource is available and when that happens the step completes returning the allocated resource. The resource will not be available until it is released to the LoadManager via a Release action.
- User-Friendliness Limitations: If used in the basic way, the LoadManager is not very user friendly, as there is no way to visualize all the elements currently waiting for a resource or reordering the elements in the waiting list.
- Queue Integration - Specific Element Waiting: When a Queue is linked to the LoadManager, a new runtime input is available: Expected_queued_item. If specified, the LoadManager will wait for two conditions: (1) There is enough free capacity on at least one of the resources, (2) The specified item is the top of the queue. IMPORTANT: This configuration can lead to starvation if an element is added to the queue without a corresponding Load Manager allocation step - the element will block any other Load Manager step from allocating the resource when the element is on top of the queue.
- Queue Integration - General Queue Processing: If the queue is specified without specifying a specific element, the LoadManager will wait for two conditions: (1) There is enough free capacity on at least one of the resources, (2) There is at least one item in the queue. IMPORTANT: Using this pattern is safer than the previous one but it is extremely important to understand how the entire mechanism works. Adding an element to the queue and getting the resource out of the Load Manager should be done in two different Work Orders. There is no guarantee that the Load Manager will retrieve from the Queue the element that the Work Order had previously added. When used in this configuration, the LoadManager will most certainly be configured as a trigger (Continue Monitoring set).
- Capacity Management: The main difference between Load Manager and Resource Manager is the possibility to specify a resource capacity, as well as a weight for the element waiting for the resource. The usual case is when a resource can process multiple items in parallel (e.g., 2). In this case a resource is considered as busy only when it has been allocated twice by the load manager. Example of use: a transcoding server that can execute multiple transcoding jobs in parallel.
- Capacity and Weight: There are cases where not all allocation tasks have the same impact on the resource. In this case it is possible to specify the capacity a given task needs. This way a resource can be allocated to multiple tasks, as long as the allocated capacity does not exceed the total capacity of the resource. A good practice when using the Load Manager together with a Queue is to use the weight of the resource as returned by the queue.
- Allocation Policy (3.1.1+ or with capsule): By default the allocation policy is fair, meaning the load manager will return the first less used available resource. In most cases this maps to a round-robin distribution of the resources. It is possible to change this behavior and have a sequential distribution: the load manager will first try to allocate all available blocks on the first resource, then the second and so on. NOTE: By default the policy will be set as '--Runtime--' in the action configuration. This allows the policy to be mapped at runtime. This mapping is optional: if it is not specified the default fair allocation policy will be used.
- Resource Pool Names: It is important to know that a suffix '__a' is added by default to the pool name. This suffix is not visible for the user, it's used internally by Orchestrator. If the user creates a new resource pool with name 'test__a', he

will see the __a stripped and not visible. This could create some confusion when trying to use the pool in the workflow. We strongly recommend to avoid using __a in the pool name.
- Tag Usage: Tags have been introduced to both Resource Pools and Queues to allow users to either document entries or to allocate a sub-set of the resource pool to specific items. When using Tags the behavior of resource allocation differs from the standard. There are two possible cases: (1) All resources in the pool are set with no Tags: Tags are ignored for the resource allocation. The load manager will allocate resources with the usual algorithm. Note this is independent of the tags associated to the element in the Queue. (2) At least one resource in the pool is associated with one or more Tags: That resource will only be reserved for entries in the Queue tagged with at least one of the Tags. If a resource is not associated with tags (it has an empty tag list) then that resource will only be allocated for Queue elements that have no tags.

8.2.9 Queue Integration Examples

- Specific Element Waiting: When a Queue is linked and a specific item is provided via runtime input, the LoadManager waits for: (1) Enough free capacity on at least one resource, (2) The specified item to be at the top of the queue. WARNING: This can lead to starvation if an element is added to the queue without a corresponding Load Manager allocation step.
- General Queue Processing: When a Queue is linked without specifying a specific element, the LoadManager waits for: (1) Enough free capacity on at least one resource, (2) At least one item in the queue. This pattern is safer and typically uses 'Continue Monitoring' to continuously process queue items.
- Capacity and Weight Usage: When 'Use weight for allocation?' is enabled, the weight of the queue element is used as the capacity requirement for the allocation, allowing dynamic resource allocation based on item complexity.

8.2.10 Tag-Based Allocation Example

Scenario: A resource pool has 3 resources: slot_1 (no tags), slot_2 (tag ”a”), slot_3 (tags ”a” and ”b”). Queue elements: elem1 (no tags), elem2 (tag ”a”), elem3 (tag ”a”), elem4 (tag ”b”), elem5 (no tags).

- At T0: slot_1 is allocated to process elem1 (no tags)
- At T1: slot_2 is allocated to process elem2 (tag 'a')
- At T2: slot_3 is allocated to process elem3 (tag 'a')
- At T0+P1: slot_1 is released and allocated to elem5 (skipping elem4 due to tag mismatch)
- At T0+P2: slot_2 is released but stays free (no matching 'a' tagged elements)
- At T0+P3: slot_3 is released and allocated to elem4 (tag 'b' match)

8.2.11 Dependencies

- Orchestrator system with sufficient memory and processing capacity to handle resource pool management and polling operations.
- Database connectivity for storing resource pool state and allocation information across workflow instances.
- Queue system properly configured if using queue integration features for coordinated workflow processing.
- Adequate timeout settings to prevent resource starvation and ensure proper resource lifecycle management.

8.2.12 Troubleshooting

- If resources are not being allocated, verify that the resource pool configuration matches between Reserve and Release operations, and check that resources are being properly released after use.
- For queue integration issues, ensure that queue elements have proper tags matching the resource pool configuration, and verify that queue items are being added and processed in the correct order.
- If allocation policies are not working as expected, confirm that the policy is set correctly ('fair' for round-robin, 'sequential' for ordered allocation) and that resource capacities are properly configured.
- For timeout-related problems, adjust wait timeout and hold timeout values based on your workflow requirements, and ensure that On wait timeout action is set appropriately (fail, pass, or error).
- If resource pool names are causing issues, verify that the pool name does not contain '__a' suffix and that the unicity scope is correctly configured for shared resource scenarios.
- For capacity allocation issues, check that allocation block size does not exceed individual resource capacity, and verify that the total allocated capacity across all users does not exceed the resource's total capacity.
- When using tags, ensure that resource tags and queue element tags are properly configured and that there are sufficient tagged resources to handle tagged queue elements to avoid resource starvation.

8.3 Queue stager



Figure 8.3: Queue stager icon

This action plugin is a tool to stage items into a managed queue. It provides a way to pause a workflow until all items queued ahead have been released from the queue. While staged in the queue, the items can be re-ordered by a user with appropriate privileges.

Version	Comment
0.9.5	Enhanced action template UI
0.9.4	Updated the label from Keep ongoing to Continue Monitoring
0.9.3	Fix for adhoc queue generation and item positioning
0.9.2	Updating migrations and position generation
0.9.1	Updating to be compatible with Ruby 3.4 and Rails 8
0.9.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.8.2	Returns the time in queue for the found item
0.8.1	Adds find method
0.8.0	Adds support for tags
0.7.2	Handle weight and priority run time list on add multiple operation
0.7.1	Fixes daemonization on get-multiple operation
0.7.0	Makes description field optional
0.6.0	Adds option to daemonize (to be used as a trigger)
0.5.0	Adds operation to retrieve multiple item at the time (Orchestrator 2.6.1 or above required, or a patch would be required)
0.4.6	Provides the queued item ID as an output
0.4.5	Provides the queued item ID as an output
0.4.4	Avoids accessing the queues via transactions
0.4.3	New category for Resource Management
0.4.2	Fixing priority in the view code(there were 2 references) and adding help file
0.4.1	Making Queue creation a default option for ad-hoc queues irrespective to the action chosen
0.4.0	Support auto-purging when originating work order is not in progress any longer (requires 2.3.6)
0.3.4	New icon
0.3.3	Fixes another issue in weight_get when weight is not defined
0.3.2	Fixes an issue in weight_get when weight is not defined
0.3.1	Fixes a race condition in get_next in high concurrency situations.
0.3.0	Adds option to add multiple
0.2.8	Return weight as outputs for get_next and retrieve
0.2.7	Make weight optional if set to 0
0.2.6	Captures work_order ID to support auto-cleanup
0.2.5	rescue crash if weight column not found
0.2.4	Fixes polling frequency on get_next
0.2.3	forces weight to 0 if null
0.2.2	fixed exposing list of all entries in the queue
0.2.1	fixed exposing priority for filtering when doing a list all
0.2.0	fixed not null expectation for the weight column. It was not possible to save a configuration without a weight
0.1.0	added support for item weight and correctly pass the priority when adding items

Version	Comment
0.0.2	provides option to use a queue specified at run-time, return the queue as output. UI refinements
0.0.1	first tracked version

8.3.1 How It Works

- Creates and manages priority-based queues for coordinating workflow processing and resource allocation across multiple work orders.
- Supports comprehensive queue operations including adding items, removing items, waiting for specific elements, and retrieving queue status information.
- Enables priority and weight-based item management where items can be ordered by priority (0 highest, 100 lowest) and assigned weights (0 lowest, 100 highest).
- Provides queue access control through user and group permissions, allowing fine-grained control over who can interact with specific queues.
- Includes queue lifecycle management with options for automatic purging, pausing/resuming processing, and clearing all queue contents.
- Integrates with LoadManager and other plugins for sophisticated workflow coordination using queue-based triggers and resource allocation.

8.3.2 Configuration Instructions

1. Add a Queue Stager step to your workflow.
2. Configure the queue setup:
   - Queue id: Enter a unique identifier for the queue.
   - Create queue ad hoc?: Check to automatically create the queue if it doesn't exist.
   - Action: Select the queue operation (Add, Remove, List All, etc.).
3. Set up access permissions:
   - Users: Select specific users who can access this queue.
   - Group: Choose a user group for queue access permissions.
   - Auto-purge: Set purge policy (Never or On work order stop).
4. Configure item parameters (for Add operations):
   - Queued Item: Specify the element to be queued.
   - Queued Item Priority: Set priority (0=highest, 100=lowest).
   - Queued Item Weight: Assign weight (0=lowest, 100=highest).

8.3.3 Example Configuration

Parameter	Example Value
Queue id	file-processing-queue
Create queue ad hoc?	checked
Action	Add
Users	admin, operator
Group	processing-team
Auto-purge	On work order stop
Queued Item	image-file-001.jpg
Queued Item Description	High priority image file
Queued Item Priority	10
Queued Item Weight	75

8.3.4 Saved Parameters Description

Parameter	Description
Create queue ad hoc?	When checked, Orchestrator will automatically create the queue with the given name if not already existing, irrespective of the chosen action.
Queue id	The identifier for the queue to be created or managed.

Parameter	Description
Users	Choose the users who can have access to this queue.
Group	Choose a group of users who can have access to this queue.
Auto-purge	Choose from the 2 provided options: Never or On work order stop.
Action	This option lists the different operations the queue can perform: Add, Add Multiple, Wait for, List All, Clear, Pause, Resume, Remove, Get Next, Find.
Queued Item	Specify the element to be queued.
Queued Item Description	Specify a description for the element to be queued.
Queued Item Priority	Specify a priority to assign to a queued item. 100 is the lowest priority, 0 the highest.
Queued Item Weight	Specify a weight to assign to a queued item. 0 is the lowest weight, 100 the highest.

8.3.5 Inputs Description

- Add: This action is used to add an element into the queue. Queued Item is required and Queued Item Description, Weight and Priority are optional.
- Add Multiple: This action is used to add multiple elements into the queue. Queued Item is required and Queued Item Description, Weight and Priority are optional.
- Wait for: This action will make the step wait for a given queue id. Queued Item is required.
- List All: This action fetches all the elements present in the queue as output.
- Item Count: This action returns the number of elements present in the queue.
- Clear: This action clears the elements present in the queue.
- Pause: This action pauses the queue processing.
- Resume: This action resumes the queue processing.
- Peek: This action retrieves the element at the top of the queue without removing it.
- Remove: This action removes an item. Queued Item is required.
- Get Next: This action gets the next element in the queue.
- Get Multiple: This action gets a specified number of elements from the top of the queue.
- Find: This action searches for an item in the queue. If the item is present, then it returns its details, otherwise it fails. It supports RegEx.

8.3.6 Outputs Description

- Queued_items: This is available as output for List All action to show the list of items in the queue.
- Total_weight: An Integer showing cumulative weight of the items in the queue. Available for the List All action.
- Queued_item: The item that was queued/removed. Available for Add and Remove actions.
- Queued_item_description: The description provided for the item. Available for the Add, Remove, Wait for, Get Next actions.
- Weight: The weight of the queued item. Available for the Wait For and Get Next actions.
- Queue_status: Shows the status for the Pause, Resume and Clear operations.
- Items added: List of all the added items. Available for the Add Multiple operation.
- Rank: When finding an item, this value is the rank of the item in the queue.
- Time in queue: When finding an item, this value indicates the time in seconds the element has been in the queue.

8.3.7 Supported Actions

- Add - Add a single item to the queue
- Add Multiple - Add multiple items to the queue in batch
- Wait for - Wait for a specific item to be available in the queue
- List All - Retrieve all items currently in the queue
- Item Count - Return number of items in the queue
- Clear - Remove all items from the queue
- Pause - Suspend queue processing
- Resume - Restart queue processing after pause
- Peek - Retrieve item at the top of the queue without removing it
- Remove - Remove a specific item from the queue

- Get Next - Retrieve the next item from the queue
- Get Multiple - Retrieve a specified number of items from the top of the queue
- Find - Search for a specific item in the queue (supports RegEx)

8.3.8 Operating Instructions

- Queue Management: This plugin is used to create and manage a queue for coordinating workflow processing. Queues are particularly useful when you need to process items with different priorities or coordinate multiple workflows accessing shared resources.
- Priority-Based Processing: Use case example: If you have multiple files in a watched directory, you may need to transfer the '.JPG' files as soon as possible while other files come with lower priority. Configure priorities where 0 is highest priority and 100 is lowest priority.
- Batch Operations: If you select the Add Multiple operation, you can pass the priorities within the plugin configuration as a single integer (e.g., 1). The same integer will then be applied to all chosen queued items. You can also pass different priorities as a runtime input with an array like [1, 2, 3]. In this case, the number of chosen queued items must match the number of priorities (e.g., ['item1', 'item2', 'item3']. The same policy applies to the weights.
- Variable Limitations: The <%= variable_name %> form cannot be applied to the priority and weight fields due to processing constraints.
- Queue Administration: The elements in a queue can be cleared manually from the Queues page in the Orchestrator GUI or in a queue dashboard for administrative management and monitoring.
- Access Control: Configure user and group permissions to control who can add, remove, or modify items in specific queues, ensuring proper workflow security and access management.
- Integration with LoadManager: Queues work seamlessly with the LoadManager plugin to provide sophisticated resource allocation and workflow coordination capabilities.

8.3.9 Priority and Weight Guidelines

- Priority Values: Priority ranges from 0 (highest priority) to 100 (lowest priority). Items with lower priority numbers are processed first.
- Weight Values: Weight ranges from 0 (lowest weight) to 100 (highest weight). Weights can be used by LoadManager for capacity-based resource allocation.
- Default Behavior: If no priority is specified, items are processed in first-in-first-out (FIFO) order. If no weight is specified, items are treated with equal resource requirements.
- Combined Usage: Priority determines processing order, while weight determines resource consumption when used with LoadManager for capacity-based allocation.

8.3.10 Dependencies

- Orchestrator queue management system must be properly configured and accessible.
- Database connectivity for storing queue state and item information across workflow instances.
- Appropriate user permissions and group memberships for queue access control.
- Sufficient system resources for queue monitoring and item processing operations.

8.3.11 Troubleshooting

- If queue creation fails, verify that 'Create queue ad hoc?' is checked and that the Queue id is unique and doesn't conflict with existing queues.
- For permission-related issues, ensure that the specified users and groups have appropriate access rights and that the current user has queue management permissions.
- If items are not being processed in expected priority order, verify that priority values are correctly set (0=highest, 100=lowest) and that the queue is not paused.
- When using Add Multiple operations, ensure that the number of items matches the number of priorities and weights if arrays are provided as runtime inputs.
- For Wait for operations that hang indefinitely, check that the specified item exists in the queue and verify the exact item name matches what was added.
- If Find operations with RegEx fail unexpectedly, test the regular expression pattern separately to ensure it's valid and matches the expected item names.
- When queues appear to be stuck, use the List All action to inspect current queue contents and consider using Pause/Resume operations to reset processing state.

- For auto-purge issues, verify that the purge setting matches your workflow requirements (Never vs On work order stop) and that queue cleanup occurs as expected.

8.4 Resource manager



Figure 8.4: Resource manager icon

This action plug-in is a tool to manage a limited pool of resources. It provides a way to pause a workflow until the resource becomes available.

Version	Comment
2.3.3	Enhanced action template UI
2.3.2	Updated the label from Keep ongoing to Continue Monitoring
2.3.1	Removing legacy support
2.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
2.2.4	Fixing issue when specifying resource pool at run time
2.2.3	Fixing issue when using legacy mode and DB Persistence
2.2.2	bug fix for max resource as runtime input and blank resource name
2.2.1	fix for queue id
2.2.0	adds an operation switch for clarity sake, fix release by resource ID
2.1.1	release using token without giving resource pool
2.1.0	migration bug fix
2.0.2	generic resource bug fix
2.0.1	New resource manager to release resource using resource_id (not just token)
2.0.0	Use managed_resource object as back-end
1.1.5	Fixes bug causing double token creation when persistence to DB is used
1.1.4	Makes dynamic binding to queue an explicit option
1.1.3	Removes need for transaction
1.1.2	Fixes inputs_spec issue preventing the release by token path functionality (problem introduced in 1.0.8)
1.1.1	New category
1.1.0	Added help file.
1.0.9	Removed the extra DEFAULT_QUEUE_ID added in v 1.0.8 as it was already present.
1.0.8	Made the queue_id as optional input.
1.0.7	New icon
1.0.6	Fixes possible rise condition when multiple items with the same id are in the queue
1.0.4	Changes default polling frequency to 5 to avoid choking the system in highly concurrent situation
1.0.3	Avoids error message during clean up token in highly concurrent situation
1.0.2	Fix a possible failure when trying to delete a file in highly concurrent situation (trac 24417)
1.0.1	Adds ability to release using a token path from a generic template
1.0.0	Uses shared state instead of files, to remove needs for shared disk
0.2.4	Adds placeholder in UI
0.2.3	Exposes queue_id as input
0.2.2	Strengthen resource identification to avoid defaulting to 0
0.2.1	Fixes when used without a specified queued item to wait for
0.2.0	Relaxes matching rules for release of token
0.1.9	Avoids timeouts by adding timestamps to queue status message
0.1.8	Avoids deadlock when 2 identical resources are queued

Version	Comment
0.1.7	Removes surrounding quotes from retrieved queue item returned
0.1.6	Modified to support monitoring portlet
0.1.5	Fixes issue with use without queue
0.1.4	Fixes issue with group token
0.1.3	Fixes typo
0.1.2	Fixes race condition in queue
0.1.1	Bug fix for queue-name
0.1.0	Adds queue constraining for release
0.0.2	Fixes resource release
0.0.1	first tracked version

8.4.1 How It Works

- Manages and allocates resources from a defined pool to workflow instances requesting them.
- Supports both generic resource pools (numbered resources) and named resource lists (specific resource identifiers).
- Provides resource reservation with configurable wait and hold timeouts to prevent resource starvation.
- Integrates with Managed Queues to coordinate resource allocation with queued workflow items.
- Offers multiple unicity scopes (absolute, workflow, workorder group, step) for flexible resource sharing across workflows.

8.4.2 Configuration Instructions

1. Add a ResourceManager step to your workflow.
2. Configure the resource pool parameters:
    - Name: Enter a descriptive name for the resource manager.
    - Comments: Add notes or description for this step.
    - Operation: Select the operation (Reserve, Release, or let the plugin decide based on inputs).
    - Adhoc resource pool: Choose whether to use existing resource pools or create a new one.
    - Resource unique name: Provide a unique ID to identify this resource pool.
    - Generic resources maximum number: Specify the number of resources in the pool.
    - Resource list: Provide a comma-separated list of specific resource names (e.g., r1,r2,r3).
3. Set up resource sharing scope:
    - Unicity scope: Choose how resources are shared (absolute, workflow, workorder group, or step).
4. Configure timeout and polling settings:
    - Wait timeout: Maximum time a workflow will wait for a resource (0 = infinite).
    - Hold timeout: Maximum time a resource can be held before auto-release (0 = infinite).
    - Polling frequency: Interval in seconds for checking resource availability.
    - Continue Monitoring: Enable to use as a trigger step with continuous monitoring.
5. Configure queue integration and outputs:
    - Bind to queue: Select a queue to coordinate resource allocation with queued items.
    - Return token as output?: Check to receive a token file path for resource release.

8.4.3 Example Configuration

Parameter	Example Value
Name	Media Processing Resource Pool
Comments	Manages resources for video transcoding
Operation	Reserve
Adhoc resource pool	unchecked
Generic resources maximum number	2
Resource unique name	test_resource
Unicity scope	absolute
Polling frequency	15
Wait timeout	0
Hold timeout	1200
Return token as output?	checked
Bind to queue	QUEUE_name

8.4.4 Saved Parameters Description

Parameter	Description
Name	Display name for this resource manager instance.
Comments	Notes or description for this step.
Operation	Specifies what the resource manager should do (Reserve, Release, or auto-detect from inputs).
Adhoc resource pool	Choose whether to use list of available resource pools or to create a new one.
Generic resources maximum number	The number of resources in the pool to be provided.
Resource list	List of resources that will be provided to the items in the queue (e.g., r1,r2,r3).
Resource unique name	A unique ID to identify the resource pool of the current template.
Unicity scope	Decides at what level a given pool of resources are shared globally (absolute, workflow, workorder group, or step).
Polling frequency	Polling interval in seconds. If not set, the default value of 5 seconds will be used.
Continue Monitoring	To use this plugin as a trigger, set a non-null polling frequency in the text field.
Wait timeout	Maximum time the item in queue will wait for the resource. Defaults to 0 value, meaning infinite wait time.
Hold timeout	Maximum time the resource will be kept on hold state. After the hold timeout, the resource is released. Defaults to 0 value, meaning infinite hold time.
Return token as output?	Token file path is returned if this check box is checked.
Bind to queue	When a queue is selected, a resource will only be attempted to be reserved if an entry is present in the queue. When -composite- is selected, the queue name can include dynamic variables. If -dynamic- is used, the queue and queued item are provided through run-time variables.
Queue	ID of the linked queue (Advanced parameter).

8.4.5 Inputs Description

Changes in this list of inputs will override any values set in the action template.

- Expected_queued_item: The specific queued item to wait for when using dynamic queue binding.
- Maximum_resources: The number of resources in the pool to be provided (for generic resources).
- Resource_list: Comma-separated list of specific resource names (e.g., r1,r2,r3).
- Resource_name: A unique ID to identify the resource pool of the current template.
- Resource_to_release: The resource ID to release when using Release by Resource ID method.
- Resource_token_path_to_release: The token file path to release when using Release by Token method (recommended).

8.4.6 Outputs Description

- Resource_id: Resource id of the allocated resource.
- Resource_token_path: If 'Return token as output?' is checked, this output field will have the token file path populated.
- Queued_item: Current queued item of the given queue id.
- Queued_item_description: Description of the current queued item of the given queue id.

8.4.7 Supported Actions

None

8.4.8 Dependencies

- Resource Manager works in conjunction with Queue Manager plugin.
- Database access for storing resource allocation state.

8.4.9 Operating Instructions

8.4.10 Overview

The Resource Manager plugin controls access to a limited pool of resources across workflow instances. It ensures that only a specified number of workflows can access particular resources simultaneously, preventing resource contention and enabling orderly processing. The plugin works hand-in-hand with Managed Queue to coordinate resource allocation with queued workflow items.

8.4.11 Understanding the Example Configuration

Consider the following Resource Manager template configuration:

- Generic resources maximum number: 2
- Resource unique name: test_resource
- Unicity scope: absolute
- Polling frequency: 15
- Hold timeout: 1200
- Return token as output?: checked
- Queue: QUEUE_name

What this configuration means:

- Maximum of 2 resources are available in the pool.
- Unicity scope is absolute, so resources can be shared amongst multiple workflows.
- Wait timeout is not given, so queue items will wait forever for a resource to become available.
- Hold timeout is 1200 seconds (20 minutes). Resources will be automatically released after 1200 seconds of idle time.
- 'Return token as output?' is checked, meaning the token file path will be returned as output and populated in the Resource_token_path output field.
- The Queue name comes from a runtime input (usually from a previous step, in this case named 'QUEUE_name').

8.4.12 Resource Pool Types

- Generic resources (numbered): Specify only the maximum number. Resources are automatically numbered (e.g., 1, 2, 3). Use when the specific identity of the resource doesn't matter.
- Named resource list: Provide specific resource identifiers (e.g., r1,r2,r3 or server_a,server_b). Use when workflows need to know which specific resource they received.

8.4.13 Unicity Scope Options

- absolute: Resources are shared globally across all workflows using this resource pool. Use when you want a single shared pool accessible by any workflow.
- workflow: Each workflow gets its own independent pool. Two workflows using the same template with ”r1,r2” will each have separate r1 and r2 resources (4 total resources in use).
- workorder group: Each workorder group gets its own pool. Useful with triggers where each trigger event creates a new workorder group with independent access to resources.
- step: Each step instance gets its own pool. Useful for parallel processing within the same workflow.

8.4.14 Timeout Configuration

- Wait timeout: How long a workflow will wait for an available resource before giving up. Set to 0 for infinite waiting (workflow will wait indefinitely). Set to a specific number of seconds to fail after that duration if no resource becomes available.
- Hold timeout: How long a reserved resource can remain idle before automatic release. Set to 0 to require explicit release. Set to a specific number of seconds to automatically release after that idle period (useful to prevent resource leaks if release step fails).

8.4.15 Queue Integration

- When bound to a queue, the Resource Manager will only attempt to reserve resources if there are items in the specified queue.

- Use the -composite- option to provide a dynamic queue name that can include variables.
- Use the -dynamic- option to provide both queue name and specific queued item through runtime variables.
- The plugin outputs information about the current queued item being processed.

8.4.16 Releasing Resources

There are two methods to release a resource:

8.4.17 Method 1: Release by Token (Recommended)

- Enable ”Return token as output?” in the Reserve step.
- The token file path will be available in the Resource_token_path output.
- Pass this token to the Release step by mapping it to the 'Resource_token_path_to_release' input.
- Why this is recommended: The token ensures unicity and provides additional safeguards that resource ID alone cannot guarantee.

8.4.18 Method 2: Release by Resource ID

- The Reserve step outputs the Resource_id (the name/identifier of the resource).
- Map this output to the Release step's resource ID input.
- You must also provide the Resource unique name (the pool identifier) to the Release step.
- Caution: This method does not guarantee the same level of unicity as token-based release.

8.4.19 Important Notes:

- Resource unique name is a mandatory field.
- Either a Generic resources maximum number OR Resource list must be provided.
- The same Resource Manager template can be re-used among multiple workflows to globally share a given pool of resources. In this case, use the absolute Unicity scope.
- Use the workflow unicity scope to share the resources at the workflow level. The use case is 2 different workflows (separate workflow ids) using the same pool of resources (e.g. a unique resource name with r1,r2 resource list) but gaining independent access to (r1,r2). A total of 4 resources may be used concurrently.
- Use the workorder group unicity scope to share the resources within a workorder group. The use case is a workflow with a trigger (e.g local file watcher). All triggered workorders gain access to a pool of resources (e.g. a unique resource name with r1,r2 resource list). Another workorder group will gain its own independent access to (r1,r2). A workorder group is created manually when creating a new workorder or via a Workflow Launcher call from another workflow.
- Use the step unicity scope to share the resources at the workflow step level.

8.4.20 Best Practices

- Always use token-based release when possible for better reliability.
- Set appropriate hold timeouts to prevent resource leaks if workflows fail or hang.
- Choose the correct unicity scope based on whether you want resources shared globally or isolated per workflow/workorder.
- Consider using wait timeouts in production to prevent indefinite workflow blocking.
- Use the Resource Manager page in Orchestrator for centralized resource pool management.

Chapter 9

System

9.1 Array fan-out



Figure 9.1: Array fan-out icon

This plugin executes the following workflow step once for each element in an input array, enabling parallel or serial processing of array data with customizable variable exposure and step labeling for efficient batch operations.

Version	Comment
1.2.2	Enhanced action template UI
1.2.1	Fix bug on handling error status prerequisites in a serialized array
1.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.1.1	If OR connector after fanout, start next steps in the loop even on failure/error
1.1.0	Adds the possibility to specify the label of the fanned steps. IMPORTANT a new version of FunnelIn is needed.
1.0.9	Adds case to add shared state data for updated step IDs in vantage plugin.
1.0.8	Fixes backward compatibility issue with pre 3.1 versions
1.0.7	Fixes clean up of fanned out step in case of retry
1.0.6	Modified to support internalized workstep inputs/outputs (required for version 3.1)
1.0.5	Removes infinite timeout and added progress reporting
1.0.4	Optimizes modification of fan-out to reflect new episteps
1.0.3	Fixes re-start or fanout step
1.0.2	Fixes issues in funnel-in when fanned-out step has error and failure branches.
1.0.1	Modifies the name of fanned-out step to be suffixed with (index)
1.0.0	Modified to work with Orchestrator 2.6 (requires Funnelin 1.0.0), does not work with pre-2.6 version
0.4.1	Fixes assignment of fanout_variable
0.4.1	Fixes assignment of fanout_variable
0.4.0	Modified to support integrated prerequisites
0.3.4	Populating the FanOut size output variable value
0.3.3	New icon
0.3.2	Cleans-up extra branch upon execution (to be reset friendly)
0.3.1	Exits as failed if the input array is empty

Version	Comment
0.3.0	Adds option to serialize fanned out step
0.2.7	Added help file
0.2.6	Standardizes comment field in UI
0.2.5	Avoid timeouts in case of a lot of entries
0.2.4	Supports or branching to funnel in
0.2.3	fixes UI field names to be human readable
0.2.2	Avoids cloning of fanned-out steps in recurring workorders
0.2.1	Ensures status is set to error in case of error
0.2.0	Adds comments
0.1.1	Adds display name
0.1.0	First versioned code

9.1.1 How It Works

- Executes the following workflow step once per element of a source array, effectively fanning out parallel or serial executions based on configuration.
- Exposes each current array element to the next step under a user-defined variable name (Fanned as) while also reporting the original array size.
- Supports parallel execution by default, with an option to run serially to preserve order or manage resource constraints.
- Allows custom step labels that can include the current item value for easier tracking of fanned executions (e.g., Work for <%= item %>).
- Works in synergy with the Array Funnel in step to enable parallel processing of array elements.

9.1.2 Configuration Instructions

1. Add an ArrayFanouts step immediately before the action to repeat.
2. Configure the essential parameters:
    - Fan-out variable: Name of the input variable holding the array to iterate (e.g., files_list).
    - Fanned as: Name of the variable that will carry each item to the next step (e.g., current_file).
    - Variable Type: Expected type of each item (e.g., string or hash) for validation.
    - Execute serial: Check to process items one after another; leave unchecked for parallel runs.
3. Optionally configure advanced settings:
    - Step label: Custom label template; include the current item with <%= item %> (e.g., Work for <%= item %>).

9.1.3 Example Configuration

Parameter	Example Value
Fan-out variable	files_list
Fanned as	current_file
Variable Type	string
Execute serial	unchecked
Step label	Work for <%= item %>

9.1.4 Saved Parameters Description

Parameter	Description
Fan-out variable	The name of the variable to expose as the array to fan out from.
Fanned as	The name of the variable produced by the step for each fanned execution; holds the current array item.
Variable Type	Specifies the expected type of the fanned-as variable (e.g., string, hash); used for basic validation.
Execute serial	By default all fanned out steps are executed in parallel (all at the same time). If checked they will be executed serially (one after another).

Parameter	Description
Step label	By default all fanned out steps will be named as the step name in the workflow definition followed by the index. This field allows you to specify a different label, including the item name by specifying <%= item %>. Make sure you are using a compatible version of the Funnel in action.

9.1.5 Inputs Description

- Fan-out variable: Runtime array to iterate over. The step will fail if this array is empty.

9.1.6 Outputs Description

- The string specified in Fanned as: For each execution, the selected value is exposed with the variable named as specified in the Fanned as field.
- Fanout_Size: Size of the array that was submitted to the plugin.

9.1.7 Operating Instructions

- This action works in synergy with the Array Funnel in. When there are multiple values in an array, these two steps allow executing the same action over each value in the array, in parallel.
- The action included between a Fan out and a Funnel in will be executed as many times as values in the input array.
- Note: only one step is supported between a fan out and the corresponding funnel in.
- ATTENTION: By default, if one one fanned execution fails, all others will stop at the same time. In order to guarantee that all parallel execution completes, make sure to link the complete, fail and error outputs to a logical OR and then to the corresponding Funnel in step.
- Using the step label field it's possible to have better naming of the fanned steps. As example if there is a list of files in input of the step, it's possible to define the step label as 'Work for <%= item %>'.

9.1.8 Dependencies

- Requires a compatible Array Funnel in step to collect results from the fanned executions.
- The input array must be populated and non-empty, or the step will fail.
- Sufficient system resources are recommended for large parallel fan-outs; consider enabling serial mode if resource constrained.

9.1.9 Troubleshooting

- Step fails immediately: Confirm the Fan-out variable points to a non-empty array at runtime.
- Unexpected variable value or type: Verify the Variable Type matches the actual array contents and that the upstream step populates the array as expected.
- Parallel runs stop early: Review failure wiring and use a logical OR to aggregate complete/fail/error outputs before connecting to the Funnel in step.
- Step label not displaying correctly: Ensure you're using a compatible version of the Funnel in action and that the <%= item %> syntax is properly formatted.
- Memory or resource issues: For large arrays, consider enabling 'Execute serial' to reduce concurrent resource usage.

9.2 Federated workflow

This plug-in is used to launch workflows on a remote orchestrator instance and get the ID and output(s) or just get the ID and not wait for the output(s) of the remote workflow

Version	Comment
0.6.3	Enhanced action template UI
0.6.2	Added optional JWT Implementation for API
0.6.1	Remove duplicate declaration of web root
0.6.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.5.0	Adds option to report status details to top level work order

Version	Comment
0.4.4	Added retry when checking for status
0.4.3	Fixes outputs lost when step configuration is saved
0.4.2	Adds error status for workflow launcher operation and does not gather execution outputs if no output is specified
0.4.1	Fixes issue with input spec fetch with portable id
0.4.0	Added Workflow Portable ID for remote machines
0.3.8	Bug fix-setting workflow_id in run time
0.3.7	Adding help file
0.3.6	Adds missing constants (OUT_EXECUTION_ADDRESS and OUT_EXECUTION_NODE)
0.3.5	Added step name to detailed status report
0.3.4	Fixed detailed status report to only consider in progress steps
0.3.3	Adds execution node and address as outputs
0.3.2	Removes display name (for consistency), Adds new icon
0.3.1	Fixed condition to display status details option
0.3.0	Exposes status details option
0.2.2	Fixes issue in reporting when no variables were found.
0.2.1	Fixes support for non-string parameters
0.2.0	Adds support for no-webroot
0.1.3	Adds support for array and hash outputs, Fixes edge case of outputs added, then all removed (value is ”{null}” instead of {} or nil)
0.1.2	Fix an issue in remote_inputs_spec() for xml_body[”parameter”]
0.1.1	Removes explicit dependency on xmlsimple
0.1.0	provides ability to identify workflow by name
0.0.3	prevents autocomplete of user field
0.0.2	deleted reference to modal_dialog in edit.html
0.0.1	first version of the federated plug-in which can launch workflows on a remote orchestrator instance

9.2.1 How It Works

- This action plug-in launches a workflow on a remote Orchestrator instance.
- The remote workflow can be identified and launched using its workflow ID, workflow name, or portable workflow ID, depending on the configuration.
- The action can either wait for the remote workflow to complete and return its outputs, or return only the remote work order ID without waiting for completion.

9.2.2 Configuration Instructions

1. Create a new action step using the FederatedWorkflow plug-in.
2. This action launches and monitors a workflow on a remote Orchestrator instance.
3. Fill out the following configuration fields:
    - Name: The name used to identify a saved FederatedWorkflow configuration.
    - Comments: Optional comments about this FederatedWorkflow configuration.
    - Remote Node: The Aspera Orchestrator node on which the remote workflow is executed.
    - Remote Address: The IP address or hostname of the remote Orchestrator instance, used when a Remote Node configuration is not defined.
    - Remote Port: The port used to access Orchestrator on the remote system when a Remote Node configuration is not defined.
    - Use SSL?: When selected, the connection to the remote Orchestrator instance uses SSL.
    - No web-root?: When selected, indicates that the remote Orchestrator instance does not have a web-root configured.
    - Remote Web-root: The web-root path of the remote Orchestrator instance. This is typically set to '/aspera/orchestrator/'.
    - Remote user: The login name of the Orchestrator user on the remote instance.
    - Remote Password: The password of the Orchestrator user on the remote instance.
    - Identify Workflow by Name?: By default, the workflow is identified by its ID. When selected, the workflow is identified by name instead.
    - Identify Workflow by Portable ID?: By default, the workflow is identified by its ID. When selected, the workflow is identified by Portable ID instead.



Figure 9.2: Federated workflow icon

- Remote Workflow ID: The ID of the remote workflow to launch.
- Remote Workflow Name: The name of the remote workflow to launch.
- Remote Workflow Portable ID: The Portable ID of the remote workflow to launch.
- Remote Work Order Name: Optional custom name for the remote work order. If not specified, it is displayed as '(remote)'.
- Operation: Select whether the remote execution runs as a Sub-Workflow or as a Workflow Launcher.
- Priority: The priority assigned to the remote workflow execution.
- Launch Dedicated Worker: When selected, a dedicated worker is launched to execute the remote workflow.
- Preemptive Level: The preemptive execution level assigned to the workflow.
- Reporting Frequency: The interval at which execution status is retrieved from the remote workflow.
- Report Running Status details: When selected, detailed running status updates are retrieved during execution.
- Report Running Status details to top level work order: When selected, running status details are propagated to the top-level work order.
- Add Outputs: Defines variables used to store outputs returned by the remote workflow execution.

9.2.3 Example Configuration

This is a minimal setup configuration for the FederatedWorkflow plug-in.

Parameter	Value
Remote Node	Remote_Orchestrator_Node
Use SSL?	Selected
Remote Web-root	/aspera/orchestrator/
Remote user	orchestrator_admin
Remote Password	********
Identify Workflow by Name?	Selected
Remote Workflow Name	Media_Processing_Workflow
Remote Work Order Name	Federated Media Processing
Operation	Workflow Launcher
Priority	Normal
Launch Dedicated Worker	Selected
Preemptive Level	Standard
Reporting Frequency	30 seconds
Report Running Status details	Selected
Report Running Status details to top level work order	Selected

9.2.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved FederatedWorkflow configuration.
Comments	Optional comments about this FederatedWorkflow configuration.
Remote Node	The Aspera Orchestrator node on which the remote workflow is executed.

Parameter	Description
Remote Address	The IP address or hostname of the remote Orchestrator instance, used when a Remote Node configuration is not defined.
Remote Port	The port used to access Orchestrator on the remote system when a Remote Node configuration is not defined.
Use SSL?	When selected, the connection to the remote Orchestrator instance uses SSL.
No web-root?	When selected, indicates that the remote Orchestrator instance does not have a web-root configured.
Remote Web-root	The web-root path of the remote Orchestrator instance. This is typically set to '/aspera/orchestrator/'.
Remote user	The login name of the Orchestrator user on the remote instance.
Remote Password	The password of the Orchestrator user on the remote instance.
Identify Workflow by Name?	By default, the workflow is identified by its ID. When selected, the workflow is identified by name instead.
Identify Workflow by Portable ID?	By default, the workflow is identified by its ID. When selected, the workflow is identified by Portable ID instead.
Remote Workflow ID	The ID of the remote workflow to launch.
Remote Workflow Name	The name of the remote workflow to launch.
Remote Workflow Portable ID	The Portable ID of the remote workflow to launch.
Remote Work Order Name	Optional custom name for the remote work order. If not specified, it is displayed as '(remote)'.
Operation	Select whether the remote execution runs as a Sub-Workflow or as a Workflow Launcher.
Priority	The priority assigned to the remote workflow execution.
Launch Dedicated Worker	When selected, a dedicated worker is launched to execute the remote workflow.
Preemptive Level	The preemptive execution level assigned to the workflow.
Reporting Frequency	The interval at which execution status is retrieved from the remote workflow.
Report Running Status details	When selected, detailed running status updates are retrieved during execution.
Report Running Status details to top level work order	When selected, running status details are propagated to the top-level work order.
Add Outputs	Defines variables used to store outputs returned by the remote workflow execution.

9.2.5 Inputs Description

Required inputs depend on how the FederatedWorkflow template is configured. If dynamic attributes are used, each attribute becomes a runtime input.

Parameter	Description
Remote_address *	The IP address or hostname of the remote Orchestrator instance.
Remote_workflow_id *	The ID of the workflow to launch on the remote Orchestrator instance.

Parameter	Description
Work_order_parameters *	A hash of input parameters passed to the remote workflow when creating the work order.
Remote_port	The port used to connect to the remote Orchestrator instance (optional).
Web_root	The web-root path of the remote Orchestrator instance (optional).
Work_order_name	An optional custom name for the remote work order.

9.2.6 Outputs Description

Parameter	Description
Execution_node	Information about the node on which the remote workflow is executed.
Execution_address	The IP address or hostname of the node executing the remote workflow.
WorkOrder_ID	The ID of the remote work order created for the workflow execution.
WorkOrder_Status	The current execution status of the remote work order.

9.2.7 Supported Actions

None

9.2.8 Dependencies

None

9.2.9 Operating Instructions

1. This action plug-in initiates a workflow execution on a remote Orchestrator instance and optionally monitors its progress until completion.

9.3 Filter



Figure 9.3: Filter icon

This action plug-in allows conditional execution of a step by evaluating a custom Ruby expression at run-time. The result determines whether the step completes successfully or fails.

Version	Comment
0.3.1	Enhanced action template UI

Version	Comment
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.8	Added styling to parameter help text.
0.2.7	2015-02-16 ML - New icon
0.2.6	improved help file
0.2.5	allows status details to be set for error also
0.2.4	allows status details to be set.
0.2.3	adds help file
0.2.2	adds payload support in command, creates usable default
0.2.1	enables use of return inside filter command field
0.2.0	adds optional outputs
0.1.0	first tracked version

9.3.1 How It Works

- Executes a Ruby command at run-time to determine whether the step should proceed or fail.
- The command must return a boolean value (`true` or `false`).
- Inputs are accessed via the `inputs` hash; outputs can optionally be set via the `outputs` hash.
- If the result is `true`, the step completes. If `false`, the step fails.

9.3.2 Configuration Instructions

1. Create a new action step using the Filter plug-in.
2. Fill out the following fields:
   - Name: A label to identify the filter step.
   - Command: Ruby code to evaluate. Must return `true` or `false`.
   - Required inputs: Declare mandatory inputs (with data types).
   - Optional inputs: Declare optional values used by the command.
   - Filter outputs: Optionally declare any values to expose from the command execution.
3. Click Update to save the step.

9.3.3 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved Filter configuration instance.
Command	The body of the Ruby code to execute. The result of the code must be a boolean. Inputs are accessed through the `inputs` hash.
Required inputs	Used to declare the mandatory inputs for the command. Inputs must be typed.
Optional inputs	Used to declare the optional inputs for the command.
Filter outputs	Optional outputs that can be set using the `outputs` hash during command execution.

9.3.4 Inputs Description

Inputs are defined in the Required inputs and Optional inputs fields of the step.

The inputs will be the ones declared in those sections and can be accessed in the Ruby command using the `inputs` hash.

9.3.5 Outputs Description

Output	Description
FilterResult	The boolean result returned by the execution of the filter command.

Output	Description
Other declared outputs	Any additional outputs defined in the Filter outputs section, with their names and types.

9.3.6 Supported Actions

None

9.3.7 Operating Instructions

This plugin allows conditional branching using custom Ruby code. The command has access to the inputs (both required and optional) via the `inputs` hash and may set outputs via the `outputs` hash.

The command must return a boolean value:

- `true`: Step completes successfully.
- `false`: Step fails with Failed status.

9.3.8 Example Configuration

Validate that a variable `file_size` (Integer) is not null or zero:

\ (inputs['file_size'] > 0)\

Alternate multi-line version:

\ if inputs['file_size'] > 0\ return true\ else\ return false\ end\

Check that file at path input `file_path` (String) exists and is non-empty:

\ filePath = inputs['file_path']\ return File.exist?(filePath) && (File.size(filePath) > 0)\

Same as above, but also output the file size as `file_size` (Integer):

\ filePath = inputs['file_path']\ if File.exist?(filePath)\ fileSize = File.size(filePath)\ outputs['file_size'] = fileSize\ return (fileSize > 0)\ else\ return false\ end\

9.4 Funnel-in

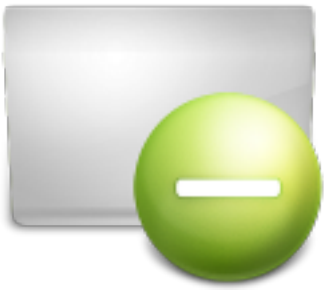


Figure 9.4: Funnel-in icon

This plugin collects and aggregates outputs from parallel executions initiated by Array Fanout, converging multiple parallel processing threads back into a unified workflow.

Version	Comment
1.1.3	Enhanced action template UI
1.1.2	Handle configurations without funneled inputs present
1.1.1	Updating to be compatible with Ruby 3.4 and Rails 8
1.0.5	Added support for step renaming in fan out (ArrayFanout >= 1.1.0)

Version	Comment
1.0.4	Modified to support internalized workstep inputs/outputs (required for version 3.1)
1.0.3	Restores compatibility with pre 1.0.0 version for the structure of the ”Hash_by_state_id” output
1.0.2	Changes to avoid crash when variables are not present on some steps
1.0.1	Fixes status exit when fail_if_failures is set to true
1.0.0	Modified to work with Orchestrator 2.6 (requires Fan-out 1.0.0), does not work with pre-2.6 version
0.4.0	Extends inputs column
0.3.7	Added styling to help text.
0.3.6	Fixed logic to fail if failures flag when result statuses are not requested
0.3.5	Added help file
0.3.4	Implements recover and removes dependency on output list as a system input
0.3.3	Adds support for pre-processing in funneled in inputs, remove auto-casting of funneled variables into strings
0.3.2	Fixed logic to fail if failures flag
0.3.1	New icon
0.3.0	Adds fail if failures flag
0.2.0	Adds hash of statuses as output
0.1.2	Use widget for funnels variables
0.1.1	Adds funnel by state_id
0.1.0	Adds name
0.0.3	Adds display name
0.0.2	First versioned code

9.4.1 How It Works

- Collects and aggregates outputs from parallel executions created by the Array Fanout step, closing the loop on fanned operations.
- Captures the output from each step that executed after the Array Fanout, organizing results by work step ID.
- Provides status tracking for each parallel execution, indicating whether steps completed, failed, or errored.
- Supports variable type validation and customizable failure handling for robust parallel processing workflows.
- Works exclusively in conjunction with Array Fanout to enable complete parallel processing workflows.

9.4.2 Configuration Instructions

1. Add a Funnel-In step to your workflow after the parallel execution section.
2. Configure the essential parameters:
   - Funnel variable: Name of the variable to capture output from each fanned step (e.g., result_data).
   - Type of the funnel variable: Expected data type of the collected variables (e.g., string, hash).
   - Funneled inputs: Check to capture output from steps between Array fanout and Funnel-in.
   - Return statuses: Enable to get execution status information for each parallel step.
3. Optionally configure failure handling:
   - Fail if any failures: Check to fail the Funnel-in step if any parallel execution fails.

9.4.3 Example Configuration

Parameter	Example Value
Funnel variable	processed_result
Type of the funnel variable	hash
Funneled inputs	checked
Return statuses	checked
Fail if any failures	unchecked

9.4.4 Saved Parameters Description

Parameter	Description
Funnel variable	The name of the variable to expose to capture the output for each fanned out step.
Type of the funnel variable	The type of the variable produced by the step for each fanned out step.
Funneled inputs	Capture the output of all the steps between Array fanout and Funnel-in step.
Return statuses	Returns an extra output when checked which helps in determining the status of each step.
Fail if any failures	If any of the steps between Array fanout and Funnel-in step fail, then fails the Funnel-in step also.

9.4.5 Inputs Description

- Funnel variable inputs: Captures input from the Array fanout step to close the loop on the number of steps that executed after the Array Fanout.

9.4.6 Outputs Description

- Hash_by_state_id: An output, as a Hash, that is a map of the work step id that executed after Array Fanout step, and Funnel variable.
- Status_by_state_id: An output, as a Hash, that is a map of the work step id that executed after Array Fanout step, and whether that step completed, failed or error.

9.4.7 Operating Instructions

- This action works in synergy with the Array Fanout. When there are multiple values in an array, these two steps allow executing the same action over each value in the array, in parallel.
- The action included between a Fan out and a Funnel in will be executed as many times as values in the input array.
- Note: only one step is supported between a fan out and the corresponding funnel in.
- ATTENTION: by default, if one of the action step (the step between a fan out and the funnel in) execution fails, all others will stop at the same time. In order to guarantee that all parallel execution completes, make sure to link the complete, fail and error outputs to a logical OR and then to the corresponding Funnel in step.

9.4.8 Dependencies

- Requires a corresponding Array Fanout step earlier in the workflow to provide the parallel executions to funnel.
- The step between Array Fanout and Funnel-In must produce outputs compatible with the specified funnel variable type.
- Proper workflow connection is required between the parallel execution step and this funnel-in step.

9.4.9 Troubleshooting

- Missing outputs: Verify that the funnel variable name matches the output variable from the step between Array Fanout and Funnel-In.
- Type mismatches: Ensure the 'Type of the funnel variable' setting matches the actual data type produced by the parallel steps.
- Incomplete parallel execution: If some parallel steps don't complete, check that complete, fail, and error outputs are properly wired to a logical OR before the Funnel-In step.
- Unexpected failures: If 'Fail if any failures' is enabled, review the status outputs to identify which parallel executions failed and why.
- Empty results: Confirm that the Array Fanout step is properly configured and that the intermediate step is producing the expected outputs.



Figure 9.5: Merge point icon

9.5 Merge point

This plugin can be used to reconcile inputs coming from different branches of logical statements. It provides the ability to define pairs of inputs that get merged. Such steps can be used without inputs to clarify branching in complex workflows.

Version	Comment
0.5.2	Enhanced action template UI
0.5.1	Fix issues when passing Array-typed inputs
0.5.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.4.1	Fixes problem when using multi inputs without specifying the number. Assuming 2 by default.
0.4.0	Add option to consider empty objects (array or hashes) has nil. Fixed escaping problem when using dynamic inputs.
0.3.1	Fix gatherStructValues() when allPairs is null
0.3.0	Adds support for multi-inputs
0.2.3	Adds support for password fields
0.2.2	New icon
0.2.1	Fixes bug in the typing of hash Inputs in configurator
0.2.0	Adds inputs merging behavior
0.1.0	First tracked version (did nothing)

9.5.1 How It Works

- Merges multiple inputs from alternate workflow branches into single consolidated outputs for streamlined workflow reconciliation.
- Supports both merged inputs (primary/alternate pairs) and multi inputs (multiple candidate sources) with configurable priority handling.
- Provides flexible input resolution using dynamic attributes and custom expressions for complex workflow scenarios.
- Handles empty object validation by treating empty arrays and hashes as null objects when configured to do so.
- Enables complex logical operations like (A || B) && (C || D) by serving as an intermediate merge point between logical operators.

9.5.2 Configuration Instructions

1. Add a Merge Point step to your workflow.
2. Configure the basic settings:
   - Consider empty objects as null: Check to treat empty arrays and hashes as null values.
   - Merged Inputs: Define primary/alternate input pairs for simple merge scenarios.
   - Multi Inputs: Define multiple candidate inputs for complex selection logic.
3. Set up input definitions:
   - For each merged input, specify the name and data type.
   - Configure whether the input is required or optional.
   - Use dynamic attributes for custom expressions if needed.

9.5.3 Example Configuration

Parameter	Example Value
Consider empty objects as null	checked
Merged Input Name	file_path
Merged Input Type	string
Merged Input Required	checked
Multi Input Name	processing_result
Multi Input Type	hash
Multi Input Required	unchecked
Number of Alternates	3

9.5.4 Saved Parameters Description

Parameter	Description
Comments	The comments for the Merge Point template.
Consider empty objects as null	If checked it instructs the plugin to consider empty arrays and hashes as null objects.
Merged Inputs	Defines a new Merged input. For each merged input, the output will be the primary input if it is found, the alternate input otherwise. In the case of required merged inputs, if neither is found, the action will exit in failure. Dynamic attributes can be used to define expressions for both the primary and the alternate input source.
Multi Inputs	Defines a new Multi input. For each multi input, the output will be the first encountered matching input candidate for which a value is present. The number of input alternates that can be provided and the type of those inputs can be specified. If required is selected, at least one alternate input needs to have a value or the execution will be declared failed.

9.5.5 Inputs Description

- Primary inputs: If a custom expression is not provided for the primary and alternate values, then a runtime input with the name of the merged input will be requested as optional runtime input.
- Alternate inputs: Runtime inputs with the name suffixed with '_alternate' will be requested as optional runtime inputs for each merged input.
- Dynamic attributes: If dynamic attributes are used, then they are also requested as optional inputs of type 'string'.
- Multi input candidates: For multi inputs, all defined candidate inputs are requested as optional runtime inputs with the specified data types.

Note: The merged inputs are typed. By default (unless an expression is defined for the primary or alternate value), the requested variable will be of the same type as the merged input type. If an expression uses dynamic attributes, those variables are of type 'string'.

9.5.6 Outputs Description

- Merged outputs: One output will be returned for each defined merged input. The output will have the type defined in the definition of the merged input and will have the same name.
- Multi outputs: One output will be returned for each defined multi input, containing the first matching input candidate that has a value present.

9.5.7 Operating Instructions

- This plugin allows users to merge multiple inputs into a single output. This is useful when merging alternate branches from a workflow and reconciling some inputs.
- Example use case: If there is an optional move step, a logical 'OR' will be required connecting back into the workflow the branch in which the move is done and the one in which it is not done. If a step after the OR needs to know where the file is, it needs to use either the location before the move or the one after the move, depending on which branch was followed.
- A Merge Point can be used for that scenario. In this case, only one merged input would be defined, calling it for example: 'file_path'. The step will then have 2 inputs (both marked optional) 'file_path' and 'file_path_alternate'.

- If 'file_path' is found it will be used, otherwise the value of 'file_path_alternate' will be used. If neither are present and the merged input is marked as required, the execution will fail.
- Another use case would be when you need to do something like (A || B) && (C || D). Since you cannot AND logical operators directly, you need a merge step in the middle that is just there to allow you to link the logical operators, but it does not do anything.
- For multi inputs, the plugin evaluates candidate inputs in the order they are defined and selects the first one that has a value present.
- Dynamic attributes allow for custom expressions to determine input sources, providing flexibility for complex workflow scenarios.

9.5.8 Dependencies

- No external dependencies required - this plugin operates purely on workflow data and logical operations.
- Proper workflow branch connections must be established for effective input merging from alternate paths.

9.6 Snapshot



Figure 9.6: Snapshot icon

Takes a snapshot of the Orchestrator configuration.

Version	Comment
0.2.1	Enhanced action template UI
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	Correcting the plugin_name in metadata.yml
0.0.6	Correcting the plugin_name in metadata.yml
0.0.5	Edited help file
0.0.4	Adding icon
0.0.3	Moving to System category and fixing glitch in metadata
0.0.2	Backward compatibility ensured - added logic to obtain constants
0.0.1	initial release

9.6.1 How It Works

- Creates a MySQL dump of the Orchestrator database configuration.
- Includes all standard Orchestrator tables by default.
- Optionally includes custom database tables specified by the user.
- Stores the snapshot file in the specified directory or uses the default archive location.
- Returns the full file path of the created snapshot for use in subsequent workflow steps.
- Enables automated backup and recovery of Orchestrator configuration data.

9.6.2 Configuration Instructions

1. Create a new action step using the OrchestratorSnapshot plugin.
2. Fill out the following fields:
    - Name: The name used to identify this saved Orchestrator Snapshot configured instance.

- Comments: Comments about this saved Orchestrator Snapshot configured instance to help identify its purpose.
- Snapshot name: The name of the snapshot file to be created. If not specified, a default name with timestamp will be used.
- Snapshot directory: The directory path where the snapshot file will be stored. If not specified, the default archive directory will be used.
- Custom tables: Comma-separated list of any custom database tables that need to be included in the MySQL dump along with the standard Orchestrator tables.
- Export full db: Select this option to include a full database dump in the export.
- Db export directory: The directory path where the database dump file will be stored.

9.6.3 Example Configuration

This is a minimum setup configuration for the OrchestratorSnapshot plugin.

Parameter	Value
Name	Daily Orchestrator Backup
Comments	Daily snapshot of Orchestrator configuration
Snapshot name	orchestrator_daily_backup
Snapshot directory	/backups/orchestrator
Custom tables	custom_workflows,custom_plugins
Export full db	Checked
Db export directory	/tmp/db_backups

9.6.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify this saved Orchestrator Snapshot configured instance.
Comments	Comments about this saved Orchestrator Snapshot configured instance to help identify its purpose.
Snapshot name	The name of the snapshot file to be created. If not specified, a default name with timestamp will be used.
Snapshot directory	The directory path where the snapshot file will be stored. If not specified, the default archive directory will be used.
Custom tables	Comma-separated list of any custom database tables that need to be included in the MySQL dump along with the standard Orchestrator tables.
Export full db	Select this option to include a full database dump in the export.
Db export directory	The directory path where the database dump file will be stored.

9.6.5 Inputs Description

The list of inputs depends on the configuration of the OrchestratorSnapshot action template. All parameters are available as inputs at runtime if not specified by the template.

Parameter	Description
Name	The name used to identify this template. It is advisable that a descriptive name is entered for easy identification.
Comments	Information about the template which will help identify its purpose and usage context.
Snapshot_name	The name of the snapshot file to be created (e.g., 'orchestrator_backup_2024'). If not provided, a default timestamped name will be used.
Snapshot_directory	Full path to the directory where the snapshot should be stored (e.g., '/backup/orchestrator'). If not provided, the default archive directory will be used.
Custom_tables	Comma-separated list of custom table names to include in the MySQL dump (e.g., 'custom_workflows,custom_plugins').
Export full db	Select this option to include a full database dump in the export.
Db export directory	Full path to the directory where the database dump file will be stored.

9.6.6 Outputs Description

Parameter	Description
Snapshot_path	The full file path of the created snapshot file

9.6.7 Supported Actions

None

9.6.8 Dependencies

None

9.6.9 Operating Instructions

This plug-in allows users to create a backup snapshot of the Orchestrator configuration and database data.

Example Workflow Using This Plugin:

A typical backup workflow consists of the following steps:

Step 1: Configure a Scheduled Trigger that triggers every 24 hours (e.g., at 2:00 AM daily).

Step 2: Configure the Orchestrator Snapshot plugin to create a backup of the Orchestrator configuration data.

Step 3: Optionally add a File Transfer or Archive step to move the snapshot to long-term storage or a remote backup location.

With these steps, an automated periodic backup of Orchestrator can be achieved, ensuring business continuity and disaster recovery capabilities.

Best Practices

- Use descriptive snapshot names that include date/time information for easy identification.
- Schedule snapshots during low-usage periods to minimize performance impact on the database.
- Regularly test snapshot restoration to ensure backups are valid and complete.
- Implement a retention policy to manage disk space (e.g., keep daily snapshots for 7 days, weekly for 30 days).
- Store snapshots in a separate location from the Orchestrator server for disaster recovery purposes.
- Monitor disk space on the backup directory to prevent failures due to insufficient storage.

9.7 Sub-workflow



Figure 9.7: Sub-workflow icon

This action plug-in allows you to embed an entire, separately defined workflow as a single step within another workflow. This enables you to break down complex processes into manageable sub-workflows, making them easier to build, maintain, and reuse.

Version	Comment
1.5.3	Ensure before_save resolves by portable ID first and falls back to workflow ID
1.5.2	Enhanced action template UI
1.5.1	Fix migration issues caused by nil references to deleted workflows
1.5.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.4.4	Fix possible problem when stopping the subworkflow step and the inner steps are not stopped.
1.4.3	Fix possible duplication of inputs and outputs definition
1.4.2	Lists published workflows only
1.4.1	Removed warning for use of deprecated index method
1.4.0	Adds possibility to specify priority of sub-workflow execution (default was to use parent priority)
1.3.3	Handles a pontential race conditions.
1.3.2	Fixes the use of master_id to represent the ID of the top level work order in all sub-workflows.
1.3.1	Bumping up the version for the ::Context fix
1.3.0	Reverts 1.2.7(Fixes the bug that creeps up on save). Fixes what 1.2.7 was supposed to fix.
1.2.9	Fixes backward compatibility of 1.2.8 with pre 2.7.0
1.2.8	Avoids polling right after execute on 2.7.0
1.2.7	Fix for output variable not showing up in editor
1.2.6	Combobox full match and empty value bug fix.
1.2.5	Combobox full match and empty value bug fix.
1.2.4	Added Combobox for selecting workflows.
1.2.3	Added link to open selected workflow in designer.
1.2.2	Fix for the before_save function.
1.2.1	Added styling to help text.
1.2.0	Portable id added to the sub-workflows
1.1.3	Adds progress reporting during the initialization of the sub-workflow (to avoid timeout)
1.1.2	Adds inheritance of caller tags to the sub-workflow
1.1.1	Reducing the default timeout from 0 (infinite) to 300
1.1.0	Adds support for push status update from sub-workflow WO to caller step
1.0.0	Proceed to actual launch of sub-workflow (bypassing prior engine heartbeat requirement)
0.5.0	Added new field to display work order status vs. last work step status”
0.4.2	New icon
0.4.1	Adds sub-workflow work order ID as an output
0.4.0	There is a requirement from a client to not append data after work order name, going to do this with a new flag
0.3.0	Fix return status in case of SubWorkflow error
0.2.6	Truncates the name field automatically to size to avoid problems if WO or step names are long
0.2.5	Limits to authorized workflows, fixes glitch in name when no workflow is selected
0.2.4	Adds dependency on sub-workflow definition (for export)
0.2.3	Fixes issue with update of mappings after creation
0.2.2	More avoidance of crash in case of ghost outputs
0.2.1	Avoids crash in case of ghost outputs
0.2.0	Allow custom names
0.1.9	Uses work order calculated status details instead of default
0.1.8	Adds a forced clean-up to support reset and restart
0.1.7	Changes the icon
0.1.6	Fixes bug with Failed exit status
0.1.5	Adds display name
0.1.4	fixes crash when mapped output is a parameter
0.1.3	fixes the output returned, considers canceled sub-wf as failed
0.1.2	added support for multi-tenancy
0.1.1	adds ability to select sub-workflow to launch at runtime
0.1.0	first tracked version

9.7.1 How It Works

- Executes a published workflow as a sub-step within a parent workflow.
- The parent workflow waits until the sub-workflow completes before proceeding.
- Inputs can be mapped from the parent workflow to the sub-workflow.
- Outputs from the sub-workflow can be passed back to the parent workflow if explicitly named.

9.7.2 Configuration Instructions

1. Create a new action step using the SubWorkflow plug-in.
2. Fill out the following fields:
   - Name: A label for this step (e.g., `Run Child Workflow`).
   - Associated with Workflow *: The published workflow to execute. Leave blank to select at run time.
   - Map Error state to: Exit status to set if the sub-workflow terminates in error (default: `Failed`).
   - Use true name: If checked, the launched work order name will not be appended with the user name or timestamp.
   - Execution priority: Overrides the calling work order's priority. Use numbers (`1` = low, `2` = medium, `3` = high) or negative numbers for preemptive priority (`-100` higher than `-1000`).
   - End with workorder status: If checked, the parent work order ends with the sub-workflow's status instead of the last step's status.
3. Map inputs from the parent workflow to the sub-workflow inputs as required.
4. Optionally map outputs from the sub-workflow back to the parent workflow by naming them.

9.7.3 Example Configuration

Field	Value
Associated with Workflow	`Data_Processing_Workflow`
Map Error state to	`Failed` (default)
Use true name	Unchecked
Execution priority	`2` (medium)
End with workorder status	Checked

Behavior: This configuration will run the `Data_Processing_Workflow` as a sub-step. If the sub-workflow fails, the step will fail. The parent work order will end with the sub-workflow's status.

9.7.4 Saved Parameters Description

Parameter	Description
Name	Label to identify this SubWorkflow step
Associated with Workflow	The published workflow to execute
Map Error state to	Exit status to use when the sub-workflow errors
Use true name	If checked, retains the original work order name without suffixes
Execution priority	Overrides the parent work order priority
End with workorder status	Adopts the sub-workflow's final status for the parent work order

9.7.5 Inputs Description

When a workflow is associated, all its inputs will be displayed for mapping. These names can be adjusted if necessary.

9.7.6 Outputs Description

Outputs are only exposed to the parent workflow if they are explicitly named in the "Output Name" field. By default, no outputs are returned.

9.7.7 Supported Actions

None

9.7.8 Dependencies

None

9.7.9 Operating Instructions

This plug-in executes another workflow as a black box step within a parent workflow. It allows passing inputs to the sub-workflow and retrieving named outputs. The parent workflow will wait until the sub-workflow finishes before proceeding.

9.7.10 Troubleshooting

- Ensure the associated workflow is published and available.
- Verify that input mappings match the sub-workflow's expected inputs.
- Check priority settings if execution order is important.
- Enable "End with workorder status" if the parent should adopt the sub-workflow's result.

9.8 User lookup

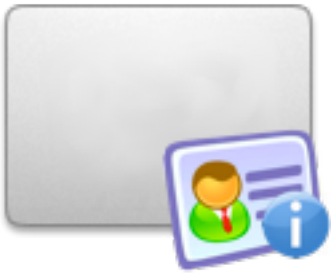


Figure 9.8: User lookup icon

This action plug-in provides the ability to lookup the information about orchestrator users identified by ID, name, login or email address.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	2015-02-18 ML - New icon
0.0.1	initial release

9.8.1 How It Works

- Searches for Orchestrator users based on ID, login, last name, or email address.
- Supports exact matching or GLOB pattern matching with wildcards (* and ?).
- Can return single match (first found) or all matching users.
- Option to include or exclude inactive users from search results.
- Returns comprehensive user information including ID, login, email, groups, phone, and timestamps.
- Action exits as 'Failed' if no matching users are found.
- Output format varies based on 'Return all matches?' setting.

9.8.2 Configuration Instructions

1. Create a new action step using the UserLookup plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved User Lookup configured instance.
   - Comments: Comments about this saved User Lookup configured instance.
   - Lookup by: Select the key to be used for the search (Id, lastname, login or email address).

- Include inactive users?: By default users marked active are considered, checking this option extends the search to all users.
- Enable glob pattern search?: (only if the lookup is not done by Id) Allows the use of GLOB wildcard in the user search ('*' for any combination of characters and '?' for any single character).
- Return all matches?: (only if the lookup is not done by Id) Returns all matches instead of the first one found.

9.8.3 Example Configuration

This is an example configuration for the UserLookup plugin searching for a user by email address.

Parameter	Value
Name	Find User by Email
Comments	Search for user by email address
Lookup by	email address
Include inactive users?	Not selected
Enable glob pattern search?	Selected
Return all matches?	Not selected

9.8.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved User Lookup configured instance.
Comments	Comments about this saved User Lookup configured instance.
Lookup by	Select the key to be used for the search (Id, lastname, login or email address).
Include inactive users?	By default users marked active are considered, checking this option extends the search to all users.
Enable glob pattern search?	(only if the lookup is not done by Id) Allows the use of GLOB wildcard in the user search ('*' for any combination of characters and '?' for any single character).
Return all matches?	(only if the lookup is not done by Id) Returns all matches instead of the first one found.

9.8.5 Inputs Description

The specific input required depends on the 'Lookup by' parameter selected. Only one input is used per lookup operation.

Parameter	Description
id	The Orchestrator user ID provided as a numeric value. Used when 'Lookup by' is set to 'Id'.
login	The login ID string used to authenticate into Orchestrator. Used when 'Lookup by' is set to 'login'.
last_name	The last name string to search for. Used when 'Lookup by' is set to 'lastname'.
email	The email address string to search for. Used when 'Lookup by' is set to 'email address'.

9.8.6 Outputs Description

Important:The output format varies depending on whether 'Return all matches?' is enabled. See the Operating Instructions section for detailed examples of single vs. multiple match output formats.

Parameter	Description
Orchestrator user ID	The unique numerical user ID used internally by Orchestrator to identify the user
Orchestrator login	The login ID used to authenticate into Orchestrator

Parameter	Description
Group memberships	An array of the group names the user is member of
Email address	The email on file for that user if provided
Alternate email addresses	An array of the alternate email addresses on file if provided
Phone number	The phone number on file for that user if provided
Last name	The last name on file for that user if provided
First name	The first name on file for that user if provided
User type	The user type (OrchestratorUser vs. AD)
Active	A boolean indicating if the user is active or has been deactivated
Created	The creation date for the user
Last updated	The last time the user record was modified. Note: changing group membership would not be considered an update

9.8.7 Supported Actions

None

9.8.8 Dependencies

None

9.8.9 Operating Instructions

This plug-in searches for Orchestrator users based on various lookup criteria and returns comprehensive user information.

Lookup Options

- Lookup by Id: Searches by unique numerical user ID. Always returns exact match. GLOB patterns and 'Return all matches?' are not applicable.
- Lookup by login: Searches by login username. Supports GLOB patterns (e.g., `john*` finds all logins starting with 'john').
- Lookup by lastname: Searches by user's last name. Supports GLOB patterns (e.g., `Smith` or `Sm*`).
- Lookup by email address: Searches by email address. Supports GLOB patterns (e.g., `*@example.com` finds all users from example.com domain).

GLOB Pattern Matching

When 'Enable glob pattern search?' is selected (not applicable for ID lookup), you can use wildcards in your search:

- Asterisk (*) - Matches any combination of characters (zero or more):
  - `john*` matches 'john', 'johnson', 'johnny'
  - `*smith` matches 'smith', 'blacksmith', 'johnsmith'
  - `*@gmail.com` matches all Gmail addresses
- Question mark (?) - Matches any single character:
  - `jo?n` matches 'john', 'joan' but not 'jon'
  - `user??` matches 'user01', 'user99' but not 'user1'
- Combination patterns:
  - `j*son` matches 'json', 'jackson', 'jameson'
  - `test_???` matches 'test_001', 'test_abc' but not 'test_1'

Output Format: Single Match vs. Multiple Matches

- Single Match Mode (Return all matches? = Not selected):
  - Returns the first matching user found
  - Output values are direct values
  - Example: `"Orchestrator user ID" => 11`
  - Example: `"Email address" => "user@example.com"`
  - Example: `"Group memberships" => ["Contributor", "Scheduler"]`

- Multiple Match Mode (Return all matches? = Selected):
  - Returns all matching users
  - `"Orchestrator user ID"` is returned as an array of IDs
  - All other outputs are hashes indexed by user ID
  - Example: `"Orchestrator user ID" => [11, 9, 3]`
  - Example: `"Email address" => {11 => "user1@example.com", 9 => "user2@example.com", 3 => "user3@example.com"}`
  - Example: `"Group memberships" => {11 => [], 3 => ["Contributor", "Scheduler"], 9 => ["Contributor"]}`

Multiple Match Output Example

When 'Return all matches?' is selected, here is an example of the output structure:

\ ”Orchestrator user ID” => [11, 9, 3]\ ”Phone number” => {11 => ””, 3 => ””, 9 => ””}\ ”Alternate email addresses” => {11 => [], 3 => [], 9 => []}\ ”Group memberships” => {11 => [], 3 => [”Contributor”, ”Scheduler”], 9 => [”Contributor”]}\ ”Created” => {11 => Tue, 13 May 2014 00:06:13 UTC +00:00, 3 => Wed, 01 May 2013 00:58:44 UTC +00:00, 9 => Thu, 19 Dec 20 ”User type” => {11 => ”OrchestratorUser”, 3 => ”OrchestratorUser”, 9 => ”OrchestratorUser”}\ ”First name” => {11 => ”baba”, 3 => ”b”, ”Orchestrator login” => {11 => ”b”, 3 => ”bob”, 9 => ”baba”}\ ”Active” => {11 => false, 3 => true, 9 => true}\ ”Last updated” => {11 => Tue, 13 May 2014 00:06:13 UTC +00:00, 3 => Wed, 01 May 2013 17:42:11 UTC +00:00, 9 => Thu, 24 Jul 2014 21:12:4 ”Last name” => {11 => ””, 3 => ”ob”, 9 => ””}\ ”Email address” => {11 => ””, 3 => ””, 9 => ””}\

Common Use Cases

- Find specific user by ID: Set 'Lookup by' to 'Id', provide user ID as input. Returns exact user details.
- Find all users with Gmail: Set 'Lookup by' to 'email address', enable GLOB patterns, enable 'Return all matches?', search for `*@gmail.com`.
- Find inactive users: Enable 'Include inactive users?', use appropriate lookup key to find deactivated accounts.
- Search by partial name: Set 'Lookup by' to 'lastname', enable GLOB patterns, search for `Smith*` to find all Smiths, Smithsons, etc.
- Verify user exists: Use in workflow with error handling to check if a user account exists before processing.

Best Practices

- Use ID lookup when you have the exact user ID for fastest, most reliable results.
- Enable 'Include inactive users?' only when specifically searching for deactivated accounts to reduce result set.
- Use GLOB patterns carefully - overly broad patterns (e.g., `*`) may return too many results.
- When using 'Return all matches?', ensure downstream steps can handle hash-based outputs indexed by user ID.
- Implement error handling for workflows since the action fails when no users are found.
- Test GLOB patterns with a small dataset first to validate the pattern matches expected users.
- Note that 'Last updated' timestamp does not change when group memberships are modified.

Important Notes

- This action exits as 'Failed' if no matching users are found.
- GLOB pattern search and 'Return all matches?' options are not available when 'Lookup by' is set to 'Id'.
- When 'Return all matches?' is selected, the 'Orchestrator user ID' output is an array, and all other outputs are hashes indexed on those user IDs.
- Group membership changes do not update the 'Last updated' timestamp.

9.9 Work Order Metadata

This action plug-in provides work order information and enables customization of work order labels and tags. It supports dynamic labeling, tag management, master ID override, and recursive label updates for better work order identification and management.

Version	Comment
0.7.1	Enhanced action template UI
0.7.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.6.2	Add workflow name and id in outputs
0.6.1	Returns initiated_by value
0.6.0	Add a way to assign tags rather than append tags

Version	Comment
0.5.1	Add previous step information output
0.5.0	Adding a flag to recursively change label to the master work order
0.4.0	Adding option to set the master_id
0.3.0	Adding a flag to show work_order_id with label
0.2.0	Adding a flag to ignore step related activity during labeling
0.1.1	Added exception handling
0.1.0	Added work orders tags edit field
0.0.5	Added work orders tags
0.0.4	Removed the include of module ssh tools.
0.0.3	Added new outputs like step info, previous step etc to the plugin.
0.0.2	New icon
0.0.1	Initial release.

9.9.1 How It Works

- Assigns custom labels and tags to work orders for easier identification and reporting.
- Allows dynamic and step-specific labeling using workflow attributes or runtime expressions.
- Supports strict labeling, recursive label updates, and master ID override for flexible workflows.

9.9.2 Configuration Instructions

1. Create a new action step using the WorkOrderMetaData plugin.
2. Fill out the following fields:
   - Name: Label for the step (e.g., `File Labeler`).
   - Comments: Notes for the template or configuration.
   - Label: Custom display name for the work order (optional).
   - Tags: Comma-separated tags to append to the work order (supports dynamic values).
   - Master ID: Optional override for the work order Master ID; may use dynamic expressions.
   - Append work order id to label?: If checked, will append the system work order ID to the label.
   - Strict labeling: If enabled, ignores step-related updates for labeling.
   - Change label of top level work order: If checked, applies label recursively to sub-workflows.
   - Append to tags?: If checked (default), appends new tag(s) rather than replacing them.

9.9.3 Example Configuration

Field	Value
Name	`New WorkOrderMetaData Step`
Label	`label`
Tags	`tag,tags`
Master ID	(empty)
Append work order id to label?	Unchecked
Strict label and tags	Unchecked
Change label of top level work order	Unchecked
Append to tags?	Checked (default)

9.9.4 Saved Parameters Description

Parameter	Description
Comments	Notes and comments for this step or template configuration.
Label	Custom display name for the work order (optional, for dynamic titling).
Tags	Comma-separated tags for this work order instance.
Master ID	Override master ID assigned to the work order.
Strict labeling	Ignores step updates for labeling if enabled.

Parameter	Description
Append work order id to label?	Appends the system work order ID to the label.
Change label of top level work order	Recursively updates title for sub-workflows.
Append to tags?	Appends new tag(s) if checked, replaces if unchecked.

9.9.5 Inputs Description

- Label: Optional, lets you customize the work order title.
- Most fields can accept dynamic expressions or runtime attributes.

9.9.6 Outputs Description

Output	Description
Work Order Label Name	Name assigned to the work order instance.
Step Information	Names and statuses of all workflow steps.
Work Order Status URL	URL for checking work order status (update credentials as needed).
Longest Step Names	Step(s) with maximum elapsed execution time.
Smallest Step Names	Step(s) with the shortest execution time.
Work Order ID	ID of the executing work order.
Previous Work Step	Name(s) of the step(s) that triggered this step.
Work Step Count	Total steps in the work order.
Tags	Tags assigned to the work order.
Master ID	Master ID of the work order instance.
Previous Work Step Information	Details about the previous triggering step(s).
Initiated By	User ID of the initiator.
Workflow Name	Name of the workflow.
Workflow ID	ID of the workflow.

9.9.7 Supported Actions

None

9.9.8 Dependencies

None

9.9.9 Operating Instructions

This plugin enables custom titling and tagging of work order instances for easier identification and reporting.

- Name workflow instances using file names or job attributes.



Figure 9.9: Work Order Metadata icon

- Tag work orders for searching, filtering, and reporting.
- Override master ID, or apply custom labeling recursively to sub-workflows.

9.9.10 Troubleshooting

- Ensure label and tags are set appropriately and support dynamic assignment.
- Verify master ID override validity.
- Check that the tag and label updates are visible in the workflow reports.
- Use the recursive label option as needed for sub-workflows.

9.10 Workflow launcher



Figure 9.10: Workflow launcher icon

This action plug-in is used to launch one workflow from within another. It is similar to, but not the same as, the SubWorkflow plug-in. The main difference is that WorkflowLauncher launches another workflow and then immediately allows the current workflow to proceed. This behavior is asynchronous, whereas in SubWorkflow, the current workflow waits for the launched workflow to complete before continuing. A WorkflowLauncher step is considered Complete once it has successfully launched the configured workflow, while a SubWorkflow step is considered Complete only after the sub-workflow it launched has finished execution.

Version	Comment
1.6.1	Enhanced action template UI
1.6.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.5.0	Adds possibility to use the portable id with run-time selection
1.4.3	Fix possible duplication of inputs and outputs definition
1.4.2	Lists published workflows only
1.4.1	Removed warning for use of deprecated index method
1.4.0	Added option to fail the step if the launched workflow fails.
1.3.3	Fix for help file not opening.
1.3.2	Backward compatibility for default priority.
1.3.1	Combobox full match and empty value bug fix.

Version	Comment
1.3.0	Makes master_id configurable
1.2.3	Adds support for run-time provided priority
1.2.2	Added Combobox for selecting workflows.
1.2.1	Returns to synchronocity when possible, removes some minor DB calls when un-necessary
1.2.0	Added option to wait until launched work order completes
1.1.1	Added link to open selected workflow in designer.
1.1.0	Offers up comments field for dynamic inputs
1.0.2	Adds progress reporting during the initialization of the sub-workflow (to avoid timeout)
1.0.1	Adding a default step timeout of 300
1.0.0	Enable direct launch optimization (avoids engine HB wait)
0.4.0	Adding support for tagging
0.3.1	New icon
0.3.0	Added possibility to specify the user when starting the workflow.
0.2.4	Supports dynamic attribute for work order name in case of dynamic workflow
0.2.3	Added the initializing and initialized states to the list of complete status
0.2.2	Added the create state to the list of complete status
0.2.1	Fix problem migration in 2.0
0.2.0	There is a requirement from a client to not append data after work order name, going to do this with a new flag
0.1.4	Defaults mapped input name to launched workflow inputs name
0.1.3	Truncates launched work order name to avoid issues if provided name is too long
0.1.2	Enforces workflow entitlements in selector
0.1.1	Adds dependency on sub-workflow definition (for export)
0.1.0	add work_order_name field
0.0.6	changes the icon
0.0.5	adds help file
0.0.4	adds support for multi-tenancy
0.0.3	adds runt-time selection of workflow
0.0.2	adds default name and naming run-time override

9.10.1 How It Works

- Launches another workflow from the current workflow.
- By default, it launches the workflow asynchronously — the step completes immediately without waiting for the launched workflow to finish.
- If Wait for Completion is checked, the step waits until the launched workflow finishes before completing.
- Returns the launched work order ID so it can be tracked via the UI or API.

9.10.2 Configuration Instructions

1. Add a WorkflowLauncher step to your workflow.
2. Configure the workflow selection and identification:
    - Name: A label for the step (e.g., Launch Sub-Workflow).
    - Associated with Workflow (ID) *: Select the published workflow to launch. This cannot be changed after saving.
    - Comments: Notes about this step configuration.
    - Work Order Description: Description for the launched work order.
    - Work Order Comments: Additional notes stored with the work order.
3. Set up execution parameters:
    - Running as User: User account to run the workflow (default: system).
    - Execution Priority: Priority for the launched workflow (default: 2 = medium; 1 = low; 3 = high; negative values for pre-emptive priorities).
    - Master ID (optional): Work orders launched with the same Master ID are grouped together.
    - Use True Name: If checked, uses the work order description exactly as entered (without timestamp).
    - Use Portable ID: Allows specifying a portable workflow ID instead of a numeric ID (runtime selection only).
4. Configure completion and monitoring behavior:
    - Fail if Launched Work Order Fails: Marks this step as failed if the launched workflow fails.
    - Wait for Completion: Waits until the launched workflow completes. *(Mandatory if you want this step to return a success/failure status of the launched workflow.)*

- Polling Frequency: Interval (in seconds) to check the launched workflow's status (default: 1).
- Run with Dedicated Execution Resource: Launches the workflow in a dedicated worker thread, which processes only steps from that work order.

5. Map inputs to the launched workflow:
   - Sub-workflow inputs mapping: These will match the inputs required by the workflow associated in the drop-down box above.
   - Saved inputs: Provides the ability to save in the template the values for some inputs requested by the workflow specified.

9.10.3 Example Configuration

Parameter	Example Value
Name	Launch Exit Status Check
Associated with Workflow (ID)	exitstatus (5)
Running as User	system (default)
Execution Priority	2 (default)
Wait for Completion	checked
Polling Frequency	1 (default)

Behavior: This configuration launches the workflow `exitstatus`, waits for it to finish, and then returns the final status.

9.10.4 Saved Parameters Description

Parameter	Description
Name	Label to identify this WorkflowLauncher step.
Comments	Notes about this step configuration.
Associated with Workflow (ID)	Selected published workflow to launch. This cannot be changed after saving.
Work Order Description	Description for the launched work order.
Work Order Comments	Additional notes stored with the work order.
Running as User	User account to run the workflow (default: system).
Execution Priority	Priority level for the launched work order (default: 2 = medium; 1 = low; 3 = high; negative values for pre-emptive priorities).
Master ID	Optional - Work orders launched with the same Master ID are grouped together.
Use True Name	If checked, uses the work order description exactly as entered (without timestamp).
Use Portable ID	Allows specifying a portable workflow ID instead of a numeric ID (runtime selection only).
Fail if Launched Work Order Fails	Marks this step as failed if the launched workflow fails.
Wait for Completion	Waits for the launched workflow to finish before completing this step.
Polling Frequency	Time interval (in seconds) to check launched workflow status (default: 1).
Run with Dedicated Execution Resource	Launches the workflow in a dedicated worker thread, which processes only steps from that work order.

9.10.5 Inputs Description

Changes in this list of inputs will override any values set in the action template.

- workflow_id: ID of the workflow to launch.
- workflow_name: Description for the launched work order.
- workflow_params: Hash of the input parameters for the launched workflow.
- running_as_user: User account for execution (default: system).

9.10.6 Outputs Description

- work_order_id: ID assigned to the launched workflow (can be used to track execution).
- status: Status of the launched workflow at step completion (usually 'In Progress' unless Wait for Completion is checked).

9.10.7 Supported Actions

None

9.10.8 Dependencies

- The workflow to be launched must be published and available in the Orchestrator system.
- User account specified in 'Running as User' must have appropriate permissions to execute the target workflow.

9.10.9 Operating Instructions

9.10.10 Overview

This action launches another workflow from the current workflow. The behavior depends on whether you want synchronous or asynchronous execution.

9.10.11 Execution Modes

- Asynchronous (default): If Wait for Completion is unchecked, the step completes immediately after launching the workflow. The launched workflow continues running independently. This is useful when you want to trigger a workflow without waiting for its completion.
- Synchronous: If Wait for Completion is checked, the step polls the launched workflow until it completes, then returns its status. This is mandatory if you want this step to return a success/failure status of the launched workflow.

9.10.12 Tracking Launched Workflows

The launched workflow's ID (work_order_id) can be tracked in multiple ways:

9.10.13 Via UI

`http://orchestrator_url/work_orders/<work_order_id>`

9.10.14 Via API

`http://orchestrator_url/api/work_order_status/<work_order_id>?login=admin&password=<YOUR_PASSWORD>`

The result will be presented in an XML file detailing step-by-step status for that work order.

9.10.15 Priority Levels

- 1: Low priority
- 2: Medium priority (default)
- 3: High priority
- Negative values: Pre-emptive priorities - these work orders will be processed before all others

9.10.16 Work Order Grouping

Use the Master ID parameter to group related work orders together. All work orders launched with the same Master ID will be associated, making it easier to track and manage related workflows.

9.10.17 Dedicated Execution Resources

When Run with Dedicated Execution Resource is enabled, the launched workflow executes in a dedicated worker thread. This thread processes only steps from that specific work order, ensuring isolated execution and preventing interference from other workflows. This is useful for workflows that require guaranteed resources or isolation.

9.10.18 Important Notes

- Use Wait for Completion when the parent workflow needs to know the outcome of the launched workflow before proceeding.
- Set appropriate Execution Priority based on the urgency of the launched workflow.
- Use Master ID to group related work orders for easier tracking and management.
- When launching multiple workflows in parallel, consider using asynchronous mode (unchecked Wait for Completion) for better performance.
- Monitor the work_order_id output to track launched workflows via UI or API.
- Use Fail if Launched Work Order Fails when the parent workflow should fail if the child workflow fails.
- Note that Wait for Completion must be used with Fail if Launched Work Order Fails to ensure that the parent workflow fails/succeeds based on the launched workflow's status.
- Consider using Dedicated Execution Resource for critical workflows that require guaranteed execution resources.

Chapter 10

Transcoding

10.1 Akamai transcoding

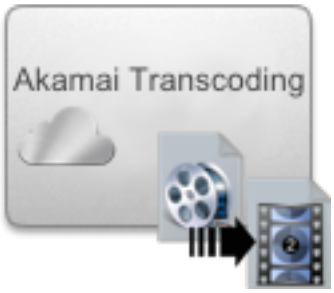


Figure 10.1: Akamai transcoding icon

This plugin starts and monitors a Transcoding job using the Akamai Transcoding platform.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.8	2015-04-12 ADM - Fixed outputs format for first execution
0.0.7	2015-04-11 ADM - Changed output format
0.0.6	2015-04-11 ADM - Changed token validity period
0.0.5	2015-04-10 ADM - Removed token validity
0.0.4	2015-04-03 ADM - Added additional inputs
0.0.3	2015-04-03 ADM - Added protection in case of RestClient errors when checking for status
0.0.2	2015-04-03 ADM - Fixed transcoding submission specification
0.0.1	2015-02-16 ADM - Initial release

10.1.1 Saved Parameters Description

- Name: The name used to identify a saved AkamaiTranscoding configured instance.
- User name: The akami user name to use for transcoding.
- Active password: The active password generated by Akamai for API cased access.
- Workspace ID: The ID of the akamai workspace to use.
- Origin ID: The ID of the origin where the source object is stored.
- Library ID: The ID of the library where the transcoding collection is.
- Collection ID: The transcoding profile to use.
- Source object: The path, relative to the Origin, of the source file to transcode.
- Target object name: Optional. A base name for the transcoded files. If not specified the source object naked name will be used.
- Target Path: The target path of the transcoding operation.

- Media Library Id: The Id of the media library where the transcoded files will be stored.
- Media Library Origin Id: Optional. The Origin Id where to store the transcoded files.
- Remove source object on success: If selected the source object will be deleted once the transcoding completes with success.
- Status Polling frequency: The frequency for polling status changes.

10.1.2 Inputs description

All inputs can be specified both at design time via the configuration panel, or at run time via the Map Inputs panel.

10.1.3 Outputs description

- *Output object*: The transcoded file.
- *Status message*: The final status message.
- *Result code*: The result code. Check the Akamai Transcoding documentation for more informations

10.1.4 Supported Actions

None

10.1.5 Dependencies

None

10.1.6 Operating Instructions

This plugin allows to trigger and monitor a transcoding job using the Akamai Content Preparation service.

This plugin supports transcoding of source object already available in an Akamai Library.

How to activate API access

By default API access to Akamai Content Preparation platform is disabled. In order to enable it you need to log in with you accound to Akamai Luna Control Center and go to Configure->Content Preparation. Now click on API Access on the top right part of the page.

In the Content Preparation API Access page make sure that API Access is set to ON.

From this page you need to retrieve the Workspace,User name and Active Password that are needed in the plugin configuration.

How to retrieve Library ID and Collection ID Using the Akamai Luna Control Center, go to the Collection you want to use.

Click on the setting icons and select Edit.

The browser will point to a url similar to this:

https://control.akamai.com/media-workflow/call/collection/edit?collectionId=_COLLECTION_UUID_&libraryId=_LIBRARY_UUID_&workspaceId=.....

Both Collection ID and Library ID can be retrieved from this URL. They are both UUID of the form:

0.....2b-4..9-4..6-b..d-c.....b (where . replaces standard characters for security reasons)

10.2 Alchemist Snell OD operations

This action plug-in provides the ability to control Alchemist Snell OD systems.

Version	Comment
0.2.2	Enhanced action template UI
0.2.1	Fix operation payload issues & cleanup code
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	added configurable polling frequency

Version	Comment
0.0.3	removing polling frequency reference from the edit screen
0.0.2	set default polling frequency. Polling frequency not defined by the model
0.0.1	first tracked version

10.2.1 How It Works

- Integrates with the Alchemist Snell OD (On-Demand) service via REST API calls to submit and monitor transcoding or processing jobs.
- Supports three operations: Submit Job, Query Job Status by ID, and Query Job Status by Filter.
- Accepts a pre-populated or custom XML/JSON payload for each operation, giving flexibility to use defaults or supply fully custom request bodies.
- The Submit Job operation is asynchronous. After submitting, the plugin polls the Snell OD service periodically for job status at the configured polling frequency.
- The Query Job Status operations execute synchronously and return the current status of one or more jobs immediately.
- Returns the Resource ID of a submitted job, the full response hash, the submit response XML, and any error details for downstream processing.

10.2.2 Configuration Instructions

1. Create a new action step using the AlchemistSnellOdOperation plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved AlchemistSnellOdOperation configured instance.
   - Comments: Comments about this saved AlchemistSnellOdOperation configured instance.
   - Url: The full URL including port and endpoint required to reach the Snell OD service (e.g. `http://snell-server.example.com:8080/api/jobs`).
   - Operation: The operation to perform. Supported values: Submit Job, Query Job Status by Id, Query Job Status by Filter.
   - Payload: The request payload for the selected operation. A default payload is pre-populated for each operation and can be used as-is or replaced with a custom payload.
   - Polling Frequency: The number of seconds between polls when waiting for an asynchronous Submit Job operation to complete.

10.2.3 Example Configuration

This is an example configuration for the AlchemistSnellOdOperation plugin submitting a new on-demand processing job to the Snell OD service.

Parameter	Value
Name	Submit Alchemist Snell OD Job
Comments	Submit a new on-demand processing job to the Snell OD service
Url	http://snell-server.example.com:8080/api/jobs
Operation	Submit Job
Payload	(pre-populated default payload or custom payload)
Polling Frequency	10

10.2.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved AlchemistSnellOdOperation configured instance.
Comments	Comments about this saved AlchemistSnellOdOperation configured instance.
Url	The full URL including port and endpoint required to reach the Snell OD service (e.g. `http://snell-server.example.com:8080/api/jobs`).

Parameter	Description
Operation	The operation to perform. Supported values: Submit Job, Query Job Status by Id, Query Job Status by Filter.
Payload	The request payload for the selected operation. A default payload is pre-populated for each operation and can be used as-is or replaced with a custom payload.
Polling Frequency	The number of seconds between polls when waiting for an asynchronous Submit Job operation to complete.

10.2.5 Inputs Description

The Payload parameter is available as a runtime input, allowing the request body to be specified or overridden dynamically within the workflow.

Parameter	Description
Payload	The request payload for the selected operation. A corresponding default payload is pre-populated for each operation and can be used as-is or replaced entirely with a custom payload at runtime.

10.2.6 Outputs Description

Outputs include the submitted job's Resource ID, the full response hash, the raw XML response for Submit Job, and any error details encountered during execution.

Parameter	Description
ResourceId	The Job ID assigned to the submitted job. Returned by the Submit Job operation. Use this ID in subsequent Query Job Status by ID operations to track the job.
Output_Hash	A hash of the entire response received from the Snell OD service for the operation performed.
Responce_XML_Fcmit	The full XML response returned by the Snell OD service for a Submit Job operation.
Error_Message	Contains error details if the operation fails or the service returns an error. Empty on success.

10.2.7 Supported Actions

None

10.2.8 Dependencies

Requires a running and accessible Alchemist Snell OD service. The service endpoint must be reachable from the Orchestrator server on the configured port.

10.2.9 Operating Instructions

This action integrates with the Alchemist Snell OD service to submit and monitor on-demand processing jobs via REST API. The Submit Job operation is asynchronous and polls for completion; query operations execute synchronously.

Supported Operations

- Submit Job: Submits a new on-demand processing job to the Snell OD service using the provided payload. This is an asynchronous operation. After submission, the plugin polls the service at the configured polling frequency until the job completes. Returns the Resource ID and full XML response for the submitted job.
- Query Job Status by ID: Retrieves the current status of a specific job using its Resource ID. Executes synchronously and returns immediately with the job status in the Output Hash.
- Query Job Status by Filter: Retrieves the status of one or more jobs matching criteria defined in the payload. Executes synchronously and returns matching job statuses in the Output Hash.

Working with Payloads



Figure 10.2: Alchemist Snell OD operations icon

- Each operation has a pre-populated default payload that can be used directly without modification for standard use cases.
- The payload can be replaced entirely with a custom payload at the template level or at runtime via the Payload input to support advanced or non-standard job configurations.
- For Submit Job, the payload typically defines the job specification including source, processing parameters, and output destination.
- For Query Job Status by Filter, the payload defines the filter criteria used to match one or more jobs on the server.
- Ensure the payload is correctly formatted for the Snell OD API, malformed payloads will result in an error from the service.

Best Practices

- Store the Resource ID output from a Submit Job step as a workflow variable to use in subsequent Query Job Status by ID steps for tracking.
- Set an appropriate Polling Frequency for Submit Job operations, too frequent polling increases load on the Snell OD server; too infrequent delays downstream workflow steps.
- Use Query Job Status by ID for targeted status checks on a known job, and Query Job Status by Filter when monitoring multiple jobs or querying by job attributes.
- Validate custom payloads against the Snell OD API specification before using them in a production workflow to avoid runtime errors.
- Implement error handling workflows that check the Error_Message output to manage submission failures or service unavailability gracefully.
- Use the pre-populated default payloads as a starting point and modify only the fields that need to differ from the defaults to reduce configuration effort and errors.

Troubleshooting

- Connection refused: Verify the URL is correct and includes the full endpoint with the correct port. Confirm the Snell OD service is running and reachable from the Orchestrator server.
- Job submission fails: Check the Error_Message output for details. Verify the payload is correctly formatted and conforms to the Snell OD API specification.
- Resource ID not returned: Confirm the Submit Job operation completed without errors. Check the Responce_XML_For_Job_Submit output for the raw server response.
- Query returns no results: For Query by ID, confirm the Resource ID is correct. For Query by Filter, verify the filter criteria in the payload match existing jobs on the server.
- Job status not updating: Verify the polling frequency is not too long and that the Snell OD service is actively processing the submitted job.
- Malformed payload error: Validate the custom payload structure against the Snell OD API documentation. Use the pre-populated default payload as a reference.

Important Notes

- The Submit Job operation is asynchronous, the plugin polls the Snell OD service until the job completes. All other operations execute synchronously.
- A default payload is pre-populated for each operation. It can be used as-is or overridden with a custom payload at the template or runtime level.
- The URL must include the full endpoint path and port required to reach the Snell OD service. No path segments are appended automatically by the plugin.

- The Resource ID returned by Submit Job is required for subsequent Query Job Status by ID operations, store it as a workflow variable when chaining steps.
- The Responce_XML_For_Job_Submit output is only populated for Submit Job operations.
- The Error_Message output is populated on failure. Always evaluate this output in error handling logic to diagnose issues.

10.3 Amazon Elastic Transcoder



Figure 10.3: Amazon Elastic Transcoder icon

This action plug-in provides the ability to use Amazon Elastic Transcoder

Version	Comment
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.0	Add assume role
0.2.1	Fix for secret masking
0.2.0	Add http proxy support
0.1.2	Avoid autofill of key and secret
0.1.1	Masking of AWS Secret
0.1.0	Allows for SSL to not be used
0.0.1	Initial release.

10.3.1 Saved Parameters Description

- Name: The name used to identify a saved AmazonElasticTranscoding configured instance.
- Comments: Some comments about this saved AmazonElasticTranscoding configured instance.
- Amazon access Key: The access key credentials provided by amazon
- Amazon secret key: The secret key credentials provided by amazon
- Region: The region under which transcoder is hosted
- Verify SSL?: Check this box to use SSL.
- External id: External id when assuming a role (optional).
- Policy: Identity and Access Management (IAM) policy in JSON format when assuming a role (optional).
- Role arn: Amazon Resource Name (ARN) of the role. Required to assume a role.
- Token duration: Duration of the generated token. When assuming a role. In seconds. Default is 3600s (1 hour).
- Inputs: Inputs required for the transcoder
- Outputs: Outputs required for the transcoding job
- pipeline_id: The pipeline that has to be used to launch the job
- output_key_prefix: The value to prepend to every outputs
- user_metadata: the metadata that you can associate with a job
- playlists: If a preset Id is selected, playlist should contain the information about the master
- Proxy URL: The URL of the proxy server through which AWS will be accessed. Leave this field blank if you do not wish to use a proxy server.

10.3.2 Inputs description

The list of inputs depends on the configuration of the AmazonElasticTranscoding action template.

Changes in this list of inputs will override any values set in the action template.

- *Aws_key*: The Amazon access key id.
- *Aws_secret*: The Amazon secret access key.
- *Aws_session_token*: The Amazon session token (optional).
- *Aws_region*: The Amazon region.
- *External_id*: External id when assuming a role (optional).
- *Policy*: IAM policy in JSON format when assuming a role (optional).
- *Role_arn*: Role arn to assume.
- *Token_duration*: Generated token duration in seconds, when assuming a role.
- *Proxy_url*: The HTTPS Proxy URL.

10.3.3 Outputs description

- *Response*: Response received from Amazon
- *Error_Message*: Error message if any received from Amazon
- *Temporary_access_key*: Temporary access key generated if assume role is called
- *Temporary_secret*: Temporary secret generated if assume role is called
- *Temporary_token*: Temporary token generated if assume role is called

10.3.4 Supported Actions

None

10.3.5 Dependencies

None

10.3.6 Operating Instructions

This action plugin is used to perform queuing action to Amazon Elastic Transcoder.

Assume role: for an IAM user to assume a role.

Pre-requisites:

The assume role operation is called only if there is a specified role arn.

The IAM user must be allowed as a principal in the role trust relationship (sts:AssumeRole action).

Instructions:

The IAM user must first be identified with its access key, secret and region.

Specify the external id if any defined in the role trust relationship.

Specify the IAM policy in JSON format. Don't put backslashes. For example, use something like { ”Version”: ”2012-10-17”, ”Statement”: [{ ”Sid”: ”Stmt1”, ”Effect”: ”Allow”, ”Action”: ”elastictranscoder:*”, ”Resource”: ”*” }] }. This policy will be effective for all operations realized with the generated temporary credentials.

Specify the role arn. For example arn:aws:iam::921583127875:role/elt_access

Temporary credentials (access key, secret, token) are generated, used and returned if a role is specified.

Specify the token duration in seconds. The value must be more than 900 seconds (15 minutes) and be less than the role Maximum session duration (check in the AWS Console). The Maximum session duration can be set to 12 hours maximum.

10.4 Amberfin transcoding

This Action plug-in provides the ability to submit file transcoding job to a Amberfin server.

Version	Comment
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.2	Remove dependency on custom Rails initializers
0.2.1	Fixes javascript error when selecting port

Version	Comment
0.2.0	Making plugin asynchronous
0.1.7	removing soap4r dependency
0.1.6	Supporting new API port 21012 for Amberfin connection
0.1.5	2015-02-24 ML - New icon, change display name
0.1.4	makes server address and node runtime inputs
0.1.3	adds dynamic inputs from XML payload to inputs
0.1.2	deleted reference to modal_dialog in edit.html and also corrected the plugin version
0.1.1	fields will use default_get instead of field value
0.1.0	added support for repurpose as well as transcode
0.0.1	initial development

10.4.1 Saved Parameters Description

- Name: The name used to identify a saved Amberfin Transcoding configured instance.
- Comments: The comments of the Amberfin Transcoding template
- Amberfin Server Node: The remote node name of the server which hosts the Amberfin Transcoding server
- Amberfin Server IP address (or hostname): The remote host name or remote IP of the server which hosts the Amberfin Transcoding server
- Amberfin API port (optional): The port number of remote server which hosts the Amberfin Transcoding server
- Operation: Select from dropdown if the job is a repurpose or transcode
- Amberfin Templates: A dropdown of Amberfin templates available on Amberfin server to repurpose and transcode files
- Template Id (hidden): Id of the Amberfin template selected
- Source file: The absolute path to the file on the server's file system
- Target directory: The absolute path to the directory on the server's file system.
- Transcode XML: The fully constructed XML with job details. This will stop Orchestrator from constructing its own XML
- Status Polling frequency: How often progress will be reported. Default is 5 seconds

10.4.2 Inputs description

The list of inputs depends on the configuration of the AmberfinTranscoding action template.

- All required parameters are available as inputs at runtime if not specified by the template.

  This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

These inputs are mandatory for both repurpose and transcode operations:

- *Amberfin Server Node or Amberfin IP Address*: Either a remote node must be selected or a remote IP address must be provided.
- *Template Id*: A template ID is mandatory while submitting a job, hence a template must be selected from dropdown.
- *Source file*: Absolute path to the source file on remote server must be provided Mandatory Fields if transcode operation is selected
- *Target directory*: Absolute path to the directory on remote server must be provided for transcode operation

These inputs are optional:

- *Name*: It is advisable that a name is entered to identify the template.
- *Comments*: Information about the template which will help identify the template.
- *Operation*: Selecting an operation from dropdown tells the plugin what to do. Defaults to repurpose when not chosen.
- *Amberfin API Port*: The port is optional as a default value is used if not specified.
- *Transcode XML*: The fully constructed XML with job details. It will overwrite the default transcode specification that the plugin builds.
- *Status Polling frequency*: Specify if progress needs to be reported within a time other than default.

10.4.3 Outputs description

- *Input_file*: The input file path involved in the transcode operation
- *Output_file*: The output fil obtained as a result of the transcode operation
- *Job_Id*: The ID of the job triggered by the transcode operation
- *Errors*: Errors reported by transcode server during transcode operation



Figure 10.4: Amberfin transcoding icon

10.4.4 Operating Instructions

This plug-in allows users to provide necessary inputs required for a Amberfin server to do a transcode job.

The mandatory inputs are listed above.

An example of a transcode job is given below. Currently only a single source file can be specified. Target if specified is used by the template setup on the Amberfin server to place the output file

10.4.5 An example of a transcode

a) Provide remote node or remote server IP and port (example: 10.0.103.85 or transcode.lala.com)

b) Decide what operation is needed - Repurpose or Transcode

c) Click the button Get Amberfin Templates. This provides a dropdown with templates from the provided Amberfin server

d) Select a template from the dropdown

e) The template ID is submitted along with source file path to Amberfin server

f) Provide source file name (Ex: ”/data/input/expendables.avi”) and a target directory ”/data/output/”

g) Amberfin server takes these inputs and performs the job

10.5 Ateme transcoding



Figure 10.5: Ateme transcoding icon

This action plugin provides the ability to transcode video files via Ateme Titan File. It implements the Ateme Titan 3.7 REST API 1.7.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	Adding RemoteNode Dependency.
0.0.1	initial release

Version	Comment

10.5.1 How It Works

- Integrates with Ateme transcoding servers to perform video and audio transcoding operations through web service APIs.
- Supports preset-based transcoding using predefined configurations available on the Ateme server for streamlined workflow setup.
- Provides flexible transcoding options through either preset selection or custom XML payload configuration for advanced use cases.
- Handles Windows-based file paths on the Ateme server with support for both forward slash and backslash path formats.
- Includes automatic preset retrieval functionality to dynamically populate available transcoding profiles from the connected server.
- Monitors transcoding job status and provides detailed output including XML results, status codes, and error messaging.

10.5.2 Configuration Instructions

1. Add an Ateme Transcoding step to your workflow.
2. Configure the server connection:
    - Ateme Node: Select a remote node pointing to your Ateme server.
    - Ateme server IP address: Enter the IP address or domain name of the Ateme server.
3. Set up transcoding parameters:
    - Get Presets: Click this button to retrieve available presets from the server.
    - Ateme Presets: Select a preset from the populated dropdown list.
    - Source: Specify the source file path on the Ateme server.
    - Destination: Specify the destination file path on the Ateme server.
4. Optional advanced configuration:
    - Transcoding payload: Provide custom XML payload for advanced transcoding capabilities.

10.5.3 Example Configuration

Parameter	Example Value
Ateme Node	ateme-server-01
Ateme server IP address	192.168.1.50
Ateme Presets	HD_H264_Standard
Source	d:\media\input\HCN00080.mxf
Destination	d:\media\output\HCN00080_transcoded.mp4
Transcoding payload	(optional custom XML)

10.5.4 Saved Parameters Description

Parameter	Description
Ateme Node	The Ateme node name from Orchestrator defined Remote Nodes.
Ateme server IP address	Ateme node IP address or domain name for direct connection to the transcoding server.
Ateme Presets	Ateme list of defined presets retrieved from the server. Select a preset after clicking 'Get Presets'.
Source	The file path on the Ateme server for the source file. Must be a complete file path, not a directory.
Destination	The file path on the Ateme server for the destination file. Must be a complete file path, not a directory.
Transcoding payload	The XML payload for the create transcode job. Optional custom configuration for advanced transcoding requirements.

10.5.5 Inputs Description

Changes in this list of inputs will override any values set in the action template.

- server address: Remote Node name or remote server IP address/domain name.
- preset_name: The Ateme preset name (optional) without the kpreset extension.
- source: The file path on the Ateme server for the source file.
- destination: The file path on the Ateme server for the destination file.
- transcoding_payload: The XML payload for the create transcode job (optional).

10.5.6 Outputs Description

- XML_result: The Ateme transcode job XML output (in parsed form like name:value).
- Ateme_status_code: The Ateme status code indicating the result of the transcoding operation.
- Ateme_error_msg: The Ateme error message if an error condition is met during processing.

10.5.7 Operating Instructions

- Server Configuration: Provide the Remote Node or remote IP address/domain name of the Ateme server. Ensure network connectivity is established before proceeding.
- Preset Retrieval: Get the presets from the Ateme system by clicking on 'Get Presets'. A list of presets will appear in the 'Ateme presets' dropdown. Select a preset from this list for your transcoding job.
- Preset Persistence: When the plugin configuration is re-opened, the selected preset is preserved. It can be changed by selecting a new preset from the presets list.
- Runtime Input Considerations: Note that 'Get Presets' cannot be clicked if the Remote Node or server address is not filled in the configuration. If connection details come as runtime input, define the preset name as runtime input as well.
- File Path Configuration: Specify the source and destination file paths. Both source and destination are paths on the Ateme server using Windows path format. They can be expressed like `d:\media\HCN00080.mxf` or `d:/media/HCN00080.mxf`. Note that destination must be a file path, not a folder path.
- Custom XML Payload Alternative: Alternatively to preset+source+destination, a custom XML payload can be specified. This may allow more transcoding capabilities than the preset-based combination. When using custom XML payload, source and destination must also be specified.
- Runtime Variables in XML: When using custom XML payload, you can reference source and destination as runtime inputs with this notation: `<%=source%>` and `<%=destination%>`. In this case, leave the Source and Destination fields empty and map them through inputs instead.

10.5.8 Dependencies

- Network connectivity to the Ateme transcoding server on the specified IP address or domain name.
- Valid Ateme SDK license required to use the web services functionality.
- Proper file system access permissions on the Ateme server for both source and destination paths.
- Available transcoding presets configured on the Ateme server.
- Sufficient storage space on the Ateme server for both input and output files.

10.5.9 Troubleshooting

- If the Ateme server is unreachable, an 'Error Occurred' message will appear in 'Ateme presets' after a timeout of 1 minute. Verify network connectivity and server availability.
- If you receive 'RestClient::Conflict : 409 Conflict' error, check that the Ateme SDK license is properly installed and valid (required to use the web services).
- If you get 'RestClient::Conflict : 409 Conflict' during job execution, this may indicate the Ateme server Engine was restarted while transcode jobs are running. Wait for the engine to fully restart before retrying.
- If a preset passed as runtime input is not found, you'll receive 'RestClient::Conflict : 409 Conflict'. Verify the preset name exists on the server and is spelled correctly (without the .kpreset extension).
- If you encounter 'RestClient::ResourceNotFound : 404 Resource Not Found', this indicates a transcode job was removed (e.g., manually through the Ateme supervision GUI) while in progress. Check the Ateme server console for job status.
- For file path issues, ensure you're using the correct Windows path format and that both source and destination are complete file paths, not directory paths.
- If custom XML payload is not working, verify the XML syntax and ensure that runtime variables like <%=source%> and <%=destination%> are properly referenced.



Figure 10.6: Bitmovin encoding icon

10.6 Bitmovin encoding

This plug-in provides the ability to start and monitor a Bitmovin encoding.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	enhance help file
0.0.1	initial release

10.6.1 Saved Parameters Description

- Name: The name used to identify a saved BitmovinEncoding configured instance.
- Comments: Some comments about this saved BitmovinEncoding configured instance.
- Bitmovin domain name: The Bitmovin domain name, by default api.bitmovin.com.
- Bitmovin port : The Bitmovin port, by default 443.
- Bitmovin api key: Your Bitmovin API key to authenticate with.
- Inputs: List of available Bitmovin inputs.
- Outputs: List of available Bitmovin outputs.
- Cloud region: The Bitmovin cloud region, by default automatically adjusted.
- Infrastructure id: Your infrastructure id if you have servers in premises. Optional.
- Source filepath: The source file path.
- Output path root: The output folder root path.
- Overwrite output?: To allow overwriting existing files.
- Output format: The output format within MPEG-DASH, HLS, MP4.
- Streams: The streams to create, one per codec.
- Segment length: The muxing segment length, default is 4.
- Polling frequency: Interval in seconds to check for the job submission status.

10.6.2 Inputs description

The list of inputs depends on the configuration of the BitmovinEncoding action template.

Changes in this list of inputs will override any value set in the action configuration/template.

- *Bitmovin API key*: Your Bitmovin API key to authenticate with.
- *Cloud region*: The Bitmovin cloud region.
- *Infrastructure id*: Your infrastructure id.
- *Source file path*: The source file path
- *Output path root*: The output folder root path
- *Segment length*: The muxing segment length

10.6.3 Outputs description

- *result*: The job submission output in text format
- *result_json*: The job submission output in JSON format
- *manifest_names*: The names of the manifests being generated for DASH or HLS

10.6.4 Supported Actions

None

10.6.5 Dependencies

None

10.6.6 Operating Instructions

The Bitmovin domain name is by default api.bitmovin.com

The Bitmovin addessable port is by default 443 (HTTPS protocol).

Specify your Bitmovin API key for authentication. Note that the Get Inputs, Get Outputs and Add stream actions will return a data list out of Bitmovin only if a valid API key is specified.

Click on 'Get Inputs' to obtain a list of available inputs from Bitmovin. Select one of them. After you save and re-open the plugin, the selected input is visible under the Inputs field name

Click on 'Get Outputs' to obtain a list of available outputs from Bitmovin. Select one of them. After you save and re-open the plugin, the selected input is visible under the Inputs field name

When changing the API key, close and re-open the plugin to refresh the inputs, outputs and codecs lists.

The cloud region is by default automatically computed. If necessary, you can specify it (e.g. GOOGLE_EUROPE_WEST_1).

Add an infrastructure id if necessary.

Specify the source file path. We support only one source file path (e.g. /pub/graphics/blender/demo/movies/Sintel.2010.1080p.mkv)

Specify the output folder root path (e.g. output/bitmovin-ruby). This is used to build the muxing output path as explained below.

Click on Overwrite output? if you want to avoid generating variable output path names and manifest names. As explained below an hex number is added automatically to the generated names. This can be disabled with this check box, allowing files to go into the same target folder or manifest name over multiple invocations of the plugin.

Specify the output format(s) within the MPEG-DASH, HLS, MP4 list. Multiple of them can be selected. Note that you can type on the label (e.g. MPEG-DASH) to select or unselect a format.

Add a stream per codec. Specify the stream name, select a codec from the list of available codecs and select audio or video based on your type of codec.

A muxing is created per stream and per output format. A fmp4 muxing is created for the DASH output format. A ts muxing is created for the HLS output format. A mp4 muxing is created for the MP4 output format. The generated muxing output path is the combination of the output root path, the stream name plus a variable 4 bytes hexa number and the muxing type (fmp4, ts, mp4).

If no stream is configured, an error is returned by Bitmovin. Check the output result for the error cause.

Specify the muxing segment length, default is 4.

A DASH manifest is created if the output format is MPEG-DASH. The .mpd file is created under the output root path. The .mpd file name is based on the source file name plus a variable 4 bytes hexa number. A fmp4 representation is created per muxing.

An HLS manifest is created if the output format is HLS. The .m3ua and rendition.m3ua files are created under the output root path. The .m3ua file names are based on the source file name plus a variable 4 bytes hexa number. A manifest video or audio media is created per muxing.

In outputs, you'll get the content returned from the encoding submission in json and text format.

The filenames of the manifests being generated are also in outputs. Extension .m3ua for HLS and extension .mpd for DASH.

If the manifest startup fails (always executed after the encoding startup), the outputs are the one from the manifest operation, otherwise it's the outputs from the encoding operation.

10.7 Cambria ftc transcoder



Figure 10.7: Cambria ftc transcoder icon

Add description here

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.4	Correctly initialize outputs
0.0.3	Fixed api port and errors if Task info not returned on check status
0.0.2	Avoid errors in cleaning the Api layer
0.0.1	Initial implementation

10.7.1 Saved Parameters Description

- Name: The name used to identify a saved CambriaFtcTranscoder configured instance. Document all available template parameters using add_parameter_help
- parameter name as shown by the label: description

10.7.2 Inputs description

- Document all available inputs using add_input_help
- *input name (refer to the constant in the class if available*: description

10.7.3 Outputs description

- Document all available outputs using add_output_help
- *input name (refer to the constant in the class if available)*: description

10.7.4 Supported Actions

None

10.7.5 Dependencies

None

10.7.6 Operating Instructions

Document how the plugin works and make some configuration examples if required.

10.8 Carbon/WFS transcoding

This is Action plug-ins provides the ability to submit file transcoding job to a Harmonic Carbon Coder/WFS server.



Figure 10.8: Carbon/WFS transcoding icon

Version	Comment
1.4.1	Updating to be compatible with Ruby 3.4 and Rails 8
1.4.0	Update variables to standard ruby coding style (snake case)
1.3.3	Bug fix in queuejob_by_workflow
1.3.2	Fix error conditions
1.3.1	Incorporating new API call to fetch more than 50 templates
1.3.0	Removing soap4r dependency
1.2.9	Adding percent complete for progress bar support
1.2.8	Adding RemoteNode Dependency.
1.2.7	Exposes the JobID in status message of execute function to allow external publication via custom progress notifier.
1.2.6	Fix odd issue (sometimes the outputs files were returned as array of array)
1.2.5	Reorganizing variable instantiation in queuejob_by_workflow and queuejob_by_xml to avoid nil errors
1.2.4	New icon, new name
1.2.3	Bug fix in output errors array for check status
1.2.2	Bug fix in output file fetch and add check for job progress calculation
1.2.1	Corrected logic to accept Workflow GUID as a runtime parameter
1.2.0	Added logic to support multiple tasks within same transcode workflow and calculate the task statuses correctly
1.1.9	Fixed logic to calculate port in controller
1.1.8	exception handling of RemoteNode.find_by_name()
1.1.7	Removes explicit dependency on xmlsimple
1.1.6	Added generic error output in case a more specific one cannot be reported
1.1.5	Fixed server address runtime input
1.1.4	Fixed port runtime input when server address is specified at runtime
1.1.3	Fixed serialisation problem when in asynchronous mode
1.1.2	Fix null reference when polling status
1.1.1	Improved error handling - Exposes errors to outputs in case of connection errors
1.1.0	Adding a field for JobName to be custom when submitted to Carbon
1.0.0	Making it asynchronous - Tested at Turner and Disney
0.5.7	Fix PCP file being mandatory when using Rhozet API 1.4.6 and above
0.5.6	Adds support to submit PCP files using Rhozet API 1.4.6 and above
0.5.5	Bug fix: Fixed typo in change made at version 0.5.4 (ARRAT to ARRAY)
0.5.4	Bug fix: Files output was declared a string and returned an array
0.5.3	Bug fix: Previous commit, did not include version bump-up. Also in the controller, the port to carbon will be defaulted to 1120 or 8731 depending on API version selected, instead of leaving it blank
0.5.2	Bug fix: In the controller, for Carbon API calls (v1.2) were initializing the WSDL driver for connection parameters instead of making a direct call.
0.5.1	Bug fix: the @connection_parameters object is for use in controller only not in action file. Replaced @connection_params[”remote_addr”] with server_get
0.5.0	Adds new column for WFS API port to differentiate Carbon API port and WFS API port and corresponding code changes. Deletes modal_dialog reference in edit.html
0.4.3	Fixes api_version defaulting
0.4.2	Fixes inputs spec for old carbon mode
0.4.1	Updates made to help file

Version	Comment
0.4.0	Adds functionality to interact with a WFS server (API v1.4.6) while preserving the legacy connectivity functionality.
0.3.0	Adds functionality to choose preset while submitting a request instead of default module xmls
0.2.1	added more states to the polling reports
0.2.0	split config and runtime info
0.1.2	bug fix in runtime port
0.1.1	Adds status polling frequency to configurator
0.1.0	Adds node selector
0.0.2	Adds support for rich textediting and xml parsing for the payload
0.0.1	initial development

10.8.1 Saved Parameters Description

- Name: The name used to identify a saved Carbon Coder Transcoding configured instance.
- Name: The name for the Carbon Coder Transcoding template
- Comments: The comments of the Carbon Coder Transcoding template
- Carbon/WFS Server Node: The remote node name of the server which hosts the Carbon Coder Transcoding server
- Carbon/WFS Server IP address (or hostname): The remote host name or remote IP of the server which hosts the Carbon Coder Transcoding server
- Carbon/WFS API port (optional): The port number of remote server which hosts the Carbon Coder Transcoding server
- API version: Used to identify which version or Carbon coder is deployed on the target server. \ - Carbon API Version 1.2: This version must be selected when the remote server hosts an Harmonic Carbon Coder server (pre WFS). When users select this API Version, they can connect to a Carbon Coder server on port 1120 and send transcoding jobs and receive updates on the submitted job.\ - Carbon API Version 1.4.6 legacy: This version must be selected when the remote server hosts on a Rhozet WFS server but the jobs are being submitted using the legacy format. Selecting this option would allow to maintain continuity after an upgrade to WFS by still using plugin configuration that were set-up for the Carbon API 1.2. What the plugin does is submit the job to Carbon Coder using the legacy API (as on 1.2), as the legacy submit is still supported by WFS, however, polling for status is done using the WFS manager (on port 8731). That complexity however is abstracted from the end-user by Orchestrator and to them the behavior will be exactly the same as the one offered by the previous version of the plugin for the Carbon API 1.2.\ - Carbon API Version 1.4.6: This version must be selected when the remote server hosts a Rhozet WFS server and the job sent use new features only supported by WFS. When users select this API Version, both submission and status polling are done using the new SOAP based Carbon API (on port 8731).\
- Status Polling frequency: How often progress will be reported. Default is 5 seconds Fields for WFS API Version 1.4.6
- WFS Workflows: A dropdown of remote WFS workflows
- WFS Workflows GUID: GUID of the WFS workflow selected from above dropdown of remote WFS workflows
- Source: The absolute path to the file on the server's file system
- Target: The absolute path to the file or directory on the server's file system Fields for WFS API Version 1.2 and 1.4.6 legacy
- Preset Payload File: A dropdown of templates available on Orchestrator server to transcode files
- File Name: The absolute path to the file on the server's file system
- Carbon Job Name: An optional input when provided identifies
- Module GUID: The absolute path to the file on the server's file system
- Target dir: The absolute path to the directory on the server's file system
- Transcoding Payload: An XML which represents the template selected above in dropdown

10.8.2 Inputs description

The list of inputs depends on the configuration of the CarbonCoderTranscoding action template.

- All parameters are available as inputs at runtime if not specified by the template.

  This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

These inputs are mandatory:

- *Carbon/WFS Server Node or Carbon/WFS IP Address and Carbon/WFS API Port*: Either a remote node must be selected or a remote IP address and its port must be provided. Fields for WFS API Version 1.4.6
- *WFS Workflows GUID*: A workflow ID must be provided

- *Source*: Absolute path to the source file on remote server must be provided Fields for WFS API Version 1.2 and 1.4.6 legacy
- *Preset Payload File*: A preset payload must be provided
- *File Name*: Absolute path to the source file on remote server must be provided
- *Module GUID*: A module GUID must be provided

These inputs are optional:

- *Name*: It is advisable that a name is entered to identify the template.
- *Comments*: Information about the template which will help identify the template.
- *Target*: Target directory on remote server is identified automatically if not provided during operation
- *Status Polling frequency*: Specify if progress needs to be reported within a time other than default.
- *Carbon Job Name*: If specified, the Carbon Job Name can be used to identify the transcode job.

10.8.3 Outputs description

- *input_file*: The input file path involved in the transcode operation
- *output_file(s)*: The output file obtained as a result of the transcode operation *(not available in API v1.2)*
- *Job_GUID*: The GUID of the job triggered by the transcode operation
- *Warnings*: Warnings reported by transcode server during transcode operation
- *Errors*: Errors reported by transcode server during transcode operation

10.8.4 Operating Instructions

This plug-in allows users to provide necessary inputs required for a Carbon/WFS server to do a transcode job.

The mandatory inputs are listed above.

The plug-in supports Carbon API version 1.2 and WFS API version 1.4.6 and also an intermediate mode in which a job can be submitted which uses Carbon API 1.2 to submit a job and WFS API 1.4.6 to poll status of the job (termed 1.4.6 legacy)

When using WFS API 1.4.6 the workflow template must be

10.8.5 An example of a transcode

API Version 1.2 and API Version 1.4.6 Legacy

a) Provide remote node or remote server IP and port (example: 10.0.103.85 or transcode.lala.com)

b) Provide transcode payload (either custom or select from payload drop down)

c) Provide source file name (Ex: ”C:/Data/Production/Transcode/Input/test.avi”)

If ”DEFAULT” preset payload was selected, select Module from drop down

d) The Module's GUID is submitted along with source file path to Carbon server

e) Carbon server takes these inputs and assigns a priority to the job and performs the transcode

API Version 1.4.6

a) Provide remote node or remote server IP and port (example: 10.0.103.85 or transcode.lala.com)

b) Select workflow id from the dropdown

c) Provide source file name (Ex: ”C:/Data/Production/Transcode/Input/test.avi”)

d) The workflow's GUID is submitted along with source file path to Carbon server

e) WFS server takes these inputs and assigns a priority to the job and performs the transcode

API Version 1.4.6 but using Transcoding Payload

a) Provide remote node or remote server IP and port (example: 10.0.103.85 or transcode.lala.com)

b) Provide a Transcoding payload that needs to be submitted to Rhozet. The Transcoding payload are usually the PCP files obtained from Rhozet/Carbon. c) WFS server takes these inputs and assigns a priority to the job and performs the transcode

10.9 Imagine StreamZ transcoding



Figure 10.9: Imagine StreamZ transcoding icon

This is Action plug-ins provides the ability to submit file transcoding job to a DigitalRapids Stream server. This has been obsoleted by the DigitalRapids TM plugin.

Version	Comment
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.5	2016-02-17 - Adding Remote Node Dependency
0.3.4	2016-02-17 - Added help file
0.3.3	2015-02-24 ML - New icon, change display name
0.3.2	add a note about plugin obsolescence
0.3.1	exception handling of RemoteNode.find_by_name()
0.3.0	Extend size of payload field
0.2.3	Adds port variable
0.2.2	Localize converts with right number of \ of use through payload.
0.2.1	fine-tuned running engine detection
0.2.0	rewriten to use imported project as templated payload
0.1.0	adds template for destination
0.0.4	adds dynamic check for available stream engines
0.0.3	adds intermediary status updates, removes obsolete dependencies
0.0.2	added generated project output, filters out generated proj from selector
0.0.1	initial development

10.9.1 Saved Parameters Description

- Name: The name used to identify a saved imagine StreamZ transcoding configured instance.
- Comments: The comments of the imagine streamz Transcoding template
- Stream Server Node: The remote node name of the server which hosts the imagine streamz transcoding server
- Server ip address (or hostname): The remote host name or remote IP of the server which hosts the streamz Transcoding server
- Stream api port (optional): The port number of remote server which hosts the streamz Transcoding server
- Stream wsdl uri (optional): Default is :/Stream?wsdl
- Status Polling frequency: How often progress will be reported. Default is 5 seconds
- Encoder wait timeout (sec): The wait timeout value for the encoder
- Encoder stall timeout (sec): The stall timeout value for the encoder
- Project Name: The name of the project to be transcoded. Can be selected from a list of projects on the stream server.
- Transcoding Payload: An XML which represents the template selected above in dropdown

10.9.2 Inputs description

The list of inputs depends on the configuration of the imagine StreamZ transcoding action template.

- All parameters are available as inputs at runtime if not specified by the template.

  This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

These inputs are mandatory:

- *Stream Server Node or server IP Address and server API Port*: Either a remote node must be selected or a remote IP address and its port must be provided.
- *Project*: A project must be selected from the list of projects in the stream server.
- *Transcoding payload*: A preset payload must be provided if not present in the project

These inputs are optional:

- *Name*: It is advisable that a name is entered to identify the template.
- *Comments*: Information about the template which will help identify the template.
- *Status Polling frequency*: Specify if progress needs to be reported within a time other than default.

10.9.3 Outputs description

- *Output_files*: The output file obtained as a result of the transcode operation
- *Orchestrator_project*: The output project obtained as a result of the transcode operation

10.9.4 Operating Instructions

This plug-in allows users to provide necessary inputs required for a Streamz server to do a transcode job.

The mandatory inputs are listed above.

10.9.5 An example of a transcode

a) Provide remote node or remote server IP and port (example: 10.0.103.85 or transcode.lala.com)

b) Provide transcode payload (either custom or select from project drop down)

e) streamz server takes these inputs and assigns a priority to the job and performs the transcode

10.10 Imagine TM transcoding



Figure 10.10: Imagine TM transcoding icon

This is Action plug-in provides the ability to submit a file transcoding job to a DigitalRapids Transcode Manager server.

Version	Comment
0.4.1	Updating to be compatible with Ruby 3.4 and Rails 8
0.4.0	updating to standard ruby code standards (snake case variables)
0.3.3	removing soap4r dependency
0.3.2	Renaming a class Timecode which conflicts with Timecode gem
0.3.1	Supporting project type zenium. Adding protection to fetch projects, even when one project has incomplete information
0.3.0	Making the plugin asynchronous
0.2.4	Retry when receiving HTTP errors during status check
0.2.3	Adding Remote Node Dependency
0.2.2	- 2015/05/07 - ML -> Fixes support for use of external project file

Version	Comment
0.2.1	- 2015/04/08 - CT -> Fixes issue with destination dir not passed from inputs
0.2.0	- 2015/01/21 - PM -> Adding functionality to expose a new field - audio source file. This audio file will be submitted to DRTM if separate_audio_file box is checked.
0.1.3	2015-02-24 ML - New icon, change display name
0.1.2	Add support for API 2.4
0.1.1	exception handling of RemoteNode.find_by_name()
0.1.0	Added more fields - priority and tagValues. Added more rescue statements for better execution of plugin
0.0.5	Exposed more fields (required and optional) as Map inputs
0.0.4	Fixed bug in controller which was stopping projects from being displayed
0.0.3	deleted reference to modal_dialog in edit.html
0.0.2	Fixed bug in fetching group info; differentiating projects and kayak files; fixed bug crashing builds
0.0.1	initial development

10.10.1 Saved Parameters Description

- Name: The name used to identify a saved Digital Rapids TM Transcoding configured instance.
- Comments: The comments of the Digital Rapids TM template
- Transcode manager server node: The remote node name of the server which hosts the Digital Rapids Transcoding server
- Server ip address (or hostname): The remote host name or remote IP of the server which hosts the Digital Rapids Transcoding server
- Transcode manager api port (optional): The port number for the transcoding service on the remote server which hosts the Digital Rapids Transcoding server
- Role: Role of the transcoding server. Either ROLE_TRANSCODE or ROLE_KAYAK_TRANSCODE
- Source file: The absolute path to the source file to transcode on the Digital Rapids server file system
- Separate Audio File: If checked, the value provided in Audio File Path will be submitted to DRTM
- Audio File Path: The absolute path to the audio file to transcode on the Digital Rapids server file system
- Destination directory: The absolute folder path for the output files on the Digital Rapids server file system
- Project list: The transcode project path selected within the list of available projects
- Group list: The transcode group name selected within the list of available groups
- Priority: The transcode job priority. Within LOW, MEDIUM, HIGH
- Tag values: Metadata name/value pairs
- Status Polling frequency: How often progress will be reported. Default is 5 seconds

10.10.2 Inputs description

The list of inputs depends on the configuration of the DigitalrapidsTmTranscoding action template.

- All parameters are available as inputs at runtime if not specified by the template.

  This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

These inputs are mandatory:

- *Transcode_manager_server*: Either a remote node must be selected or a remote IP address
- *Source_file*: Absolute path to the source file on the remote server
- *Project_path*: The full project path
- *Project_id*: The project id obtained from the Transcode Manager
- *Group_name*: The resource group name
- *Tm_group_id*: The resource group id obtained from the Transcode Manager
- *Tag_values*: Metadata name/value pairs

These inputs are optional:

- *Destination_dir*: The absolute folder path for the output files on the Digital Rapids server file system
- *Audio File Path*: The absolute file path for audio file to transcode on the Digital Rapids server file system

10.10.3 Outputs description

- *Output_files*: The output file(s) obtained as a result of the transcode operation

- *Job_id*: The ID of the job triggered by the transcode operation
- *Errors_during_transcode*: Errors reported by transcode server during transcode operation

10.10.4 Operating Instructions

This plug-in allows users to provide the inputs required by an Digital Rapids TM (Transcode Manager) server to perform a transcode job.

The plugin has been tested with the Digital Rapids TM SOAP API 2.0 up to 2.5

Each transcode task uses either a Digital Rapids Stream project (choose role ROLE_TRANSCODE) or a Digital Rapids Kayak blueprint (choose role ROLE_KAYAK_TRANSCODE) for processing. Stream transcoding projects are created with Digital Rapids Stream software, while the Digital Rapids Kayak Workflow Designer or Kayak SDK can be used to create Kayak Transcode blueprints.

Currently only a single source file can be specified.

If you want to pass project and group as inputs, you need to pass Project_path + Project_id and Group_name + Tm_group_id. Passing Project_path and Group_name is not sufficient.

10.10.5 An example of a transcode

a) Provide remote node or remote server IP and port (example: 10.0.103.85 or transcode.lala.com)

b) Click the button ”Get Projects and Groups”. This provides 2 dropdowns with projects and groups from the Digital Rapids server

c) Select a project and a group from the dropdowns

d) Provide the source file name (Ex: ”\\drcuk_colo1\\AsperaShare\\JS_Scary2.mxf”)

e) Provide the audio source file path if different from the video file (Ex: ”\\drcuk_colo1\\AsperaShare\\JS_Scary2.wav”)

f) The Digital Rapids server takes these inputs and performs the transcode

10.11 Dolby VM600/DP600



Figure 10.11: Dolby VM600/DP600 icon

This action plugin provides the ability to invoke Dolby VM600/DP600 API operations for audio processing and workflow management.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.6	New icon
0.0.5	Improving test compatibility
0.0.4	Deal with incorrect parameters
0.0.3	Make plugin compatible with Orchestrator 2.5.1
0.0.2	More error handling and force payload encoding
0.0.1	Initial Release

Version	Comment

10.11.1 How It Works

- Integrates with Dolby VM600/DP600 services to perform audio processing operations through REST API calls.
- Supports two primary operations: listing available workflow profiles and submitting workorders for audio processing tasks.
- Handles various workorder types including SWO, MULTI_FILE_SWO, PINNACLE_SWO, OMNEON_SWO, HFWO, and others for different audio processing scenarios.
- Provides flexible URI support for source and destination files across multiple protocols (SMB, FTP, SFTP, local file shares).
- Includes workorder monitoring capabilities with configurable polling intervals and automatic status tracking until completion.

10.11.2 Configuration Instructions

1. Add a Dolby Operation step to your workflow.
2. Configure the basic connection settings:
    - URL: Enter the complete URL with port and endpoint for the Dolby service.
    - API Operation: Choose between 'Listing Workflow Profiles' or 'Submitting a Work Order'.
3. For workorder submission, configure additional parameters:
    - Work Order Name: Specify a name for the work order.
    - Work Order Type: Select the appropriate type (SWO, MULTI_FILE_SWO, etc.).
    - Source/Destination Filenames: Provide URIs for input and output files.
    - Workflow Profile: Choose the processing profile to use.
    - Polling Frequency: Set the interval for status checking.

10.11.3 Example Configuration

Parameter	Example Value
URL	http://dolby-server.domain.com:8080/api/v1
API Operation	Submitting a Work Order
Work Order Name	AudioProcessing_001
Work Order Type	SWO
Source Filename	smb://user:pass@fileserver.com/audio/input.wav
Destination Filename	smb://user:pass@fileserver.com/audio/output.wav
Workflow Profile	Standard_Audio_Processing
Work Order Priority	5
Delete on Cancel	checked
Polling Frequency	30

10.11.4 Saved Parameters Description

Parameter	Description
URL	Enter the entire URL along with the port and the endpoint to reach the Dolby VM600/DP600 service.
API Operation	Operation to be performed. Currently supports: Listing Workflow Profiles and Submitting a Work Order.
Work Order Name	The name of the workorder to be submitted. Required for workorder submission operations.
Work Order Type	The type of the workorder to be submitted. Accepted types are: SWO, MULTI_FILE_SWO, PINNACLE_SWO, OMNEON_SWO, HFWO, PINNACLE_HFWO, GVHFWO, OMNEON_HFWO, MONO_HFWO, and MONO_WO.
Source Filename	The URI of the file to be used as the source. Supports SMB, FTP, SFTP, and local file protocols with optional authentication credentials.
Destination Filename	The URI of the file to be used as the destination. Supports SMB, FTP, SFTP, and local file protocols with optional authentication credentials.

Parameter	Description
Workflow Profile	The profile of the workflow to be used for processing the audio content.
Work Order Priority	A unique number identifying the priority of the workorder with 1 being the highest priority and 10 being the lowest.
Delete on Cancel	If checked, the Dolby workorder will be deleted if the plugin step is canceled. If unchecked, the workorder will remain as is.
Polling Frequency	The time interval (in seconds) for how often the server needs to be polled for status updates.

10.11.5 Outputs Description

The outputs differ between the API operations:

10.11.6 Listing Workflow Profiles:

- Workflows List: Hash of workflows retrieved with keys being the names and values being the descriptions of the workflows.
- Workflows List Count: Number of profiles that were retrieved from the Dolby service.

10.11.7 Submitting a Work Order:

- Refresh Time: Minimum number of minutes until next checkResponse if status was not 'FINISHED'.
- Correlation Id: Correlation ID of the workorder that was submitted for tracking purposes.
- Status: Current status of the workorder that was submitted to the Dolby service.

10.11.8 Supported URI Formats

- SMB: smb://[username[:password]@]hostname[:port][absolute-path] or smb://[[domain;]username[:password]@]hostname[:port][absolute-path]
- FTP: ftp://[username[:password]@]hostname[:port][absolute-path] or ftp://[[domain;]username[:password]@]hostname[:port][absolute-path]
- SFTP: sftp://[username[:password]@]hostname[:port][absolute-path] or sftp://[[domain;]username[:password]@]hostname[:port][absolute-path]
- Local File: file://username:password@/mount/someshare/somedir (for local shares)

Note: Username and password can be optional if remote shares are configured for anonymous access.

10.11.9 Supported Work Order Types

- SWO (Standard Work Order)
- MULTI_FILE_SWO (Multi-file Standard Work Order)
- PINNACLE_SWO (Pinnacle Standard Work Order)
- OMNEON_SWO (Omneon Standard Work Order)
- HFWO (High Frequency Work Order)
- PINNACLE_HFWO (Pinnacle High Frequency Work Order)
- GVHFWO (GV High Frequency Work Order)
- OMNEON_HFWO (Omneon High Frequency Work Order)
- MONO_HFWO (Mono High Frequency Work Order)
- MONO_WO (Mono Work Order)

10.11.10 Operating Instructions

- Operation Selection: Choose between 'Listing Workflow Profiles' to retrieve available processing profiles, or 'Submitting a Work Order' to process audio files through the Dolby service.
- Profile Listing: When listing workflow profiles, only the URL and API Operation parameters are required. The operation will return all available profiles with their descriptions.

- Work Order Submission: For workorder submission, ensure all required parameters are configured including source/destination URIs, workorder type, and workflow profile.
- File URI Configuration: Use the appropriate URI format for your file storage system. Include authentication credentials in the URI or configure shares for anonymous access.
- Priority Management: Set workorder priority between 1 (highest) and 10 (lowest) to manage processing queue order on the Dolby service.
- Status Monitoring: The plugin will automatically poll the Dolby service at the specified interval until the workorder status reaches 'FINISHED'.
- Cancellation Behavior: Configure the 'Delete on Cancel' option to determine whether workorders should be removed from the Dolby service if the plugin step is canceled.

10.11.11 Dependencies

- Network connectivity to the Dolby VM600/DP600 service on the specified URL and port.
- Valid authentication credentials for accessing remote file shares (SMB, FTP, SFTP) if required.
- Proper file system permissions for local file share access when using file:// protocol.
- Available workflow profiles configured on the Dolby service for workorder processing.

10.11.12 Troubleshooting

- If connection to the Dolby service fails, verify the URL format includes the correct protocol, hostname, port, and endpoint path.
- For file access issues, check that the URI format is correct and authentication credentials are valid for the specified file shares.
- If workorder submission fails, ensure the selected workorder type is supported and the workflow profile exists on the Dolby service.
- For polling issues, verify the polling frequency is appropriate for your workorder processing time and service load.
- Check that source files exist and are accessible from the Dolby service before submitting workorders.
- Ensure destination paths are writable and have sufficient space for the processed output files.

10.12 Elemental Media Convert



Figure 10.12: Elemental Media Convert icon

This Action plugin provides the ability to submit a file transcoding job to the AWS Elemental MediaConvert service.

Version	Comment
0.4.2	Enhanced action template UI
0.4.1	Fix issues when fetching job templates & using AWS secrets with certain special characters
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.0	Add operations start job and check job status
0.2.1	Add endpoint as a configurable parameter
0.2.0	Add more output group types and the destinations structure
0.1.0	Add reload job templates option
0.0.3	Remove wrong mention if help about max job templates
0.0.2	Make job template id optional when a custom json payload is filled in

Version	Comment
0.0.1	initial development

10.12.1 How It Works

- Integrates with AWS Elemental MediaConvert to perform cloud-based video and audio transcoding operations using predefined job templates.
- Supports three distinct operation modes: convert (full workflow), start job (initiate only), and check job status (monitor existing jobs).
- Utilizes AWS IAM authentication with access keys, secret keys, and role-based permissions for secure service access.
- Dynamically retrieves and manages job templates from MediaConvert, allowing selection from available transcoding configurations.
- Handles multiple output group types including FILE_GROUP_SETTINGS, MS_SMOOTH_GROUP_SETTINGS, DASH_ISO_GROUP_SETTINGS, and HLS_GROUP_SETTINGS.
- Provides flexible configuration options through custom JSON payloads for advanced transcoding scenarios beyond standard templates.

10.12.2 Configuration Instructions

1. Add an Elemental Media Convert step to your workflow.
2. Configure AWS authentication and region:
    - Access key: Enter your AWS access key for authentication.
    - Secret key: Provide your AWS secret key for secure access.
    - Region: Select the AWS region where MediaConvert will process jobs.
3. Set up MediaConvert job configuration:
    - Operation: Choose operation type (convert, start job, or check job status).
    - Get Job Templates: Click to retrieve available templates from MediaConvert.
    - Job templates: Select a template from the populated dropdown list.
    - Role: Specify the IAM role ARN for MediaConvert execution.
4. Configure source and destination paths:
    - Input file path: Provide S3 path to the source media file.
    - Output path: Specify S3 destination folder for transcoded output.

10.12.3 Example Configuration

Parameter	Example Value
Operation	convert
Access key	AKIA...
Secret key	**********
Region	EU (Ireland)
Job templates	Standard_HD_Transcode
Endpoint	po21umhx.mediaconvert.eu-west-1.amazonaws.com
Role	arn:aws:iam::123456789931:role/MediaConvert3
Input file path	s3://aspmediaconvert/test.mxf
Output path	s3://aspmediaconvert/outputs/
Status Polling frequency	5

10.12.4 Saved Parameters Description

Parameter	Description
Comments	The comments of the Elemental Media Convert template for documentation and identification purposes.
Operation	The operation to perform. Choose from: convert (full transcode with status monitoring), start job (initiate transcode only), or check job status (monitor existing job).
Access key	The AWS access key used for authentication to access MediaConvert services.
Secret key	The AWS secret key used for authentication to access MediaConvert services.

Parameter	Description
Region	The AWS region where MediaConvert jobs will be processed (e.g., EU Ireland, US East).
Job templates	The list of defined MediaConvert job templates retrieved from the AWS account.
Endpoint	The AWS account-specific endpoint where API requests are sent (e.g., po21umhx.mediaconvert.eu-west-1.amazonaws.com).
Role	The IAM role ARN that MediaConvert will assume to access S3 resources and perform transcoding operations.
Input file path	The S3 path to the source file that will be transcoded.
Output path	The S3 path to the destination folder where transcoded files will be stored.
Custom json payload	Custom JSON data to send in the transcode request body for advanced configurations beyond standard templates.
Reload job templates?	If checked, reload the job templates data structure from Elemental MediaConvert before each transcoding job execution.
Status Polling frequency	How often the job status is checked during transcoding. Default is 5 seconds.

10.12.5 Inputs Description

All parameters can be available at runtime if not specified in the plugin configuration.

10.12.6 Mandatory Inputs:

- Job_id: The job ID, only required for the check job status operation.
- Access_key_value: The AWS access key for authentication.
- Secret_key_value: The AWS secret key for authentication.
- Region: The AWS region for MediaConvert processing.
- Job template id: The job template ARN. Visible only if a job template is not selected in the plugin and a JSON payload is not entered.
- Endpoint: The AWS account-specific endpoint for API requests.
- Role: The AWS IAM role for MediaConvert execution.
- Input_file_path: The S3 path to the source file for transcoding.
- Output_folder_path: The S3 path to the destination folder for output files.

10.12.7 Optional Inputs:

- Custom JSON payload: Custom JSON data to send in the transcode request body for advanced configurations.
- Destinations: The destinations JSON structure for cases where multiple output paths must be passed. See operating instructions for format examples.

10.12.8 Outputs Description

- result: The result from the transcode operation expressed as a Ruby hash structure containing job details and status information.
- result_json: The result from the transcode operation expressed as a JSON structure for easy integration with other systems.
- result_job_template_json: The chosen job template JSON data structure containing the complete template configuration used for the transcode operation.

10.12.9 Supported Output Group Types

- FILE_GROUP_SETTINGS - File-based outputs
- MS_SMOOTH_GROUP_SETTINGS - Microsoft Smooth Streaming
- DASH_ISO_GROUP_SETTINGS - DASH streaming format
- HLS_GROUP_SETTINGS - HTTP Live Streaming

10.12.10 Operating Instructions

- Prerequisites: The plugin works with Orchestrator 3.1 release and upward. Earlier versions may experience SSL connectivity issues. Ensure the AWS domain mediaconvert.<region>.amazonaws.com (e.g., mediaconvert.eu-west-1.amazonaws.com for EU Ireland) is reachable on port 443 from your Orchestrator system.
- Basic Transcode Setup: Specify the operation to perform: 'convert' to start a job and monitor status, 'start job' to initiate without monitoring, or 'check job status' to monitor existing jobs. The convert operation is equivalent to start job + check job status. In some cases, splitting operations helps avoid restarting jobs when status checks return failures.
- Authentication Configuration: Provide AWS access key and secret key for authentication, select a region from the available list (e.g., EU Ireland), and specify the IAM role ARN (e.g., arn:aws:iam::123456789931:role/MediaConvert3).
- Job Template Management: Click 'Get Job Templates' to retrieve available templates from MediaConvert. Select a template from the dropdown, which will be preserved when you save and re-open the plugin. The AWS account endpoint is automatically fetched and displayed when job templates are retrieved.
- File Path Configuration: Enter S3 paths for input file (e.g., s3://aspmediaconvert/test.mxf) and output folder (s3://aspmediaconvert/outputs/). Currently, only single source files and single output paths are supported through the standard interface.
- Runtime Job Template Specification: When no custom JSON payload is used, job templates are fetched from a shared_states database entry. The job template ID (ARN) passed as runtime input must match stored templates. Use 'Get Job Templates' button or enable 'Reload job templates?' option to refresh template data.
- Multiple Destinations: Pass multiple destinations as runtime input using JSON format: { ”destinations”: [{ ”type”: ”FILE_GROUP_SETTINGS”, ”destination”: ”s3://xxx/” }, { ”type”: ”DASH_ISO_GROUP_SETTINGS”, ”destination”: ”s3://yyy/” }] }
- Custom JSON Payload: For advanced configurations, provide custom JSON payload matching the MediaConvert API request body specification at https://docs.aws.amazon.com/mediaconvert/latest/apireference/jobs.html. Use <%=custom_payload%> syntax for runtime input variables.

10.12.11 Dependencies

- Orchestrator version 3.1 or higher for proper SSL connectivity with AWS services.
- Network connectivity to AWS MediaConvert endpoints on port 443 for the specified region.
- Valid AWS account with MediaConvert service access and appropriate IAM permissions.
- Configured IAM role with permissions to access S3 buckets and execute MediaConvert operations.
- S3 buckets and appropriate permissions for both source file access and output file storage.
- MediaConvert job templates configured in the AWS account for template-based operations.

10.12.12 Troubleshooting

- If SSL connectivity issues occur, verify you're using Orchestrator 3.1 or higher and that the MediaConvert endpoint is accessible on port 443 from your system.
- For authentication failures, confirm AWS access keys are correct, have appropriate permissions, and that the specified IAM role exists and has MediaConvert and S3 access.
- If job templates don't load, verify AWS credentials, check network connectivity to MediaConvert, and ensure the region is correctly specified. Try clicking 'Get Job Templates' again after verifying connectivity.
- For job submission failures, verify S3 paths are correct, accessible, and that the IAM role has permissions to read source files and write to destination locations.
- If the endpoint value appears incorrect or empty, clear the endpoint field and re-open the plugin after clicking 'Get Job Templates' to fetch the current endpoint automatically.
- When using runtime job template IDs, ensure the template exists in the shared_states database by clicking 'Get Job Templates' at least once, or enable 'Reload job templates?' option for automatic refresh.
- For custom JSON payload errors, validate the JSON syntax and ensure it matches the MediaConvert API specification. Test the payload structure with AWS CLI or SDK first.

10.13 Elemental transcoding

This action plugin provides the ability to submit file transcoding jobs to an Elemental server.

Version	Comment
0.7.1	Enhanced action template UI

Version	Comment
0.7.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.6.3	Show completion percentage in status details
0.6.2	Fixes profiles list
0.6.1	List all available profiles (not just the first 20)
0.6.0	Added option to return ERROR instead of STATUS_UNDEFINED for backward compatibility
0.5.3	Return an error instead of STATUS_UNDEFINED when an exception is raised
0.5.2	Modifying reporting to reduce chances of timeout
0.5.1	Adding certificate_file nil for s3 authentication support
0.5.0	Add S3 Authentication params support
0.4.0	Make the API port accept other than 443 and 80
0.2.8	Bug fix - fetch more profiles than the default 20
0.2.7	Adding percent complete for progress bar support
0.2.6	adding remote node dependency
0.2.5	adding missing statements
0.2.4	2016-01-07 JG - Adding server_address as a runtime parameter to allow mapping
0.2.3	2015-02-24 ML - New icon, new name
0.2.2	add custom payload example
0.2.1	add proxy data
0.2.0	moves to asynchronous execution mode
0.1.7	exception handling of RemoteNode.find_by_name()
0.1.6	Improved exception handling when response is non 200
0.1.5	Removes explicit dependency on xmlsimple
0.1.4	Fix for edges cases where a) percent complete is not returned when job is submitted b) output folder is not returned during status check
0.1.3	Considers the pre and post processing polling statuses
0.1.2	Fixes issue with username and port fields when the info is provided within a remote node
0.1.1	Fixes issues with Elemental server version 2.1.2 where request header has to be explicitly set and returns hash with error when job profile listing fails
0.1.0	Adds support for custom XML payloads
0.0.4	deleted reference to modal_dialog in edit.html
0.0.3	support for non-SSL requests (users have to chose port as 80 or 443)
0.0.2	fields will use default_get instead of field value
0.0.1	initial development

10.13.1 How It Works

- Integrates with Elemental Transcoding servers to perform video and audio transcoding operations using predefined profiles.
- Supports flexible authentication options including username/API key authentication and S3 credential management for secure access.
- Provides dynamic profile retrieval from Elemental servers, allowing selection from available transcoding configurations.
- Handles SSL/HTTPS connections with configurable port settings and optional proxy server support for network flexibility.
- Supports custom XML payload configuration for advanced transcoding scenarios beyond standard profile-based operations.
- Monitors job progress with configurable polling intervals and provides comprehensive error reporting and status tracking.

10.13.2 Configuration Instructions

1. Add an Elemental Transcoding step to your workflow.
2. Configure server connection:
   - Elemental Server Node: Select a remote node pointing to your Elemental server.
   - Elemental Server IP address: Enter the IP address or hostname of the Elemental server.
   - Elemental API port: Specify port number (default 443 for HTTPS).
   - Use ssl?: Check to enable HTTPS (automatically enabled for port 443).
3. Set up authentication (if required):
   - Enable Authentication: Check this box if the server requires authentication.
   - User name: Provide the Elemental server username.



Figure 10.13: Elemental transcoding icon

- API Key: Enter the API key from your Elemental user profile.

4. Configure transcoding parameters:
   - Get Elemental Profiles: Click to retrieve available profiles from the server.
   - Elemental Profiles: Select a profile from the populated dropdown.
   - Source: Specify the absolute path to the source file on the server.

10.13.3 Example Configuration

Parameter	Example Value
Elemental Server Node	elemental-server-01
Elemental Server IP address	10.0.103.85
Elemental API port	443
Use ssl?	checked
Enable Authentication	checked
User name	elemental_user
API Key	**********
Elemental Profiles	HD_H264_Profile
Profile Id	12345
Source	/data/input/expendables.avi
Status Polling frequency	5

10.13.4 Saved Parameters Description

Parameter	Description
Comments	The comments of the Elemental Transcoding template for documentation and identification purposes.
Elemental Server Node	The remote node name of the server which hosts the Elemental Transcoding server.
Elemental Server IP address (or hostname)	The remote host name or remote IP of the server which hosts the Elemental Transcoding server.
Elemental API port (optional)	The port number of remote server which hosts the Elemental Transcoding server (by default 443).
Use ssl?	To specify if HTTPS must be used instead of HTTP. If API port 443 is specified, no need to check this option as HTTPS will be used automatically.
Proxy ip address or hostname (optional)	The proxy host name or IP address used to reach the Elemental Transcoding server.
Proxy port (optional)	The port number of the proxy server for network connectivity.

Parameter	Description
Enable Authentication	If the Elemental server requires authentication, check this box to enable authentication parameters.
User name	The user name to be used to login/authenticate with the Elemental server.
API Key	API Key of the user used to authenticate with the Elemental server (found under Elemental User profile).
Elemental Profiles	A dropdown of Elemental profiles available on Elemental server to transcode files.
Profile Id	ID of the Elemental profile selected from the dropdown list.
Source	The absolute path to the file on the Elemental server file system.
Use S3 Authentication	Check this box if you want to enter S3 authentication credentials for accessing S3 resources.
S3 authentication type	Enter the authentication type for S3 access. Defaulted to 'basic'.
S3 Shared Key	Shared key for S3 that will be used as user name for S3 authentication.
S3 Secret Key	Secret Key for S3 that will be used as password for S3 authentication.
Return error?	Return an ERROR status rather than an UNDEFINED status (default) upon exception in the check status method.
Custom xml payload	Custom XML data to send to Elemental server for advanced transcoding configurations. See operating instructions for format examples.
Status Polling frequency	How often progress will be reported during transcoding. Default is 5 seconds.

10.13.5 Inputs Description

All parameters are available as inputs at runtime if not specified by the template. This plugin does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

10.13.6 Mandatory Inputs:

- Elemental Server Node or Elemental IP Address: Either a remote node must be selected or a remote IP address must be provided.
- Profile Id: A profile ID is mandatory while submitting a job, hence a profile must be selected from dropdown.
- Source: Absolute path to the source file on the remote server must be provided.

10.13.7 Mandatory Fields if Authentication is Needed:

- User name: Username must be provided when authentication is enabled.
- API Key: API key must be provided when authentication is enabled.

10.13.8 Optional Inputs:

- Name: It is advisable that a name is entered to identify the template.
- Comments: Information about the template which will help identify the template.
- Elemental API Port: The port is optional as a default value is used if not specified.
- Proxy_server_address: Proxy server IP address or host name.
- Proxy_port: Proxy port number for network connectivity.
- Enable Authentication: If the check box is not checked, the requests to the server do not include authentication parameters.
- Status Polling frequency: Specify if progress needs to be reported within a time other than default.

10.13.9 Outputs Description

- Input_file: The input file path involved in the transcode operation.
- Output_file(s): The output file(s) obtained as a result of the transcode operation.
- Job_Id: The ID of the job triggered by the transcode operation.
- Errors: Errors reported by transcode server during transcode operation.

10.13.10 Operating Instructions

- Plugin Purpose: This plugin allows users to provide the inputs required by an Elemental server to perform a transcode job. The plugin supports authentication as requested by the Elemental Server in the latest API release. Currently only a single source file can be specified.
- Basic Transcode Setup: Provide remote node or remote server IP and port (e.g., 10.0.103.85 or transcode.lala.com). Decide if authentication should be enabled - if yes, check the Enable Authentication box, otherwise leave it unchecked.
- Profile Configuration: Click 'Get Elemental Profiles' button to retrieve available profiles from the Elemental server. Select a profile from the dropdown - the profile ID will be submitted along with the source file path to the Elemental server.
- File Processing: Provide the source file name (e.g., '/data/input/expendables.avi'). The Elemental server takes these inputs, assigns a priority (default is 50) to the job, and performs the transcode operation.
- Custom XML Payload: A custom XML payload can be sent to Elemental for advanced configurations. When using custom payloads, source, username, password, and profile ID are typically defined as runtime inputs. However, if source and profile ID are also defined in the configuration, they take precedence.
- Variable Precedence: If names other than 'Source' and 'Profile_Id' (or 'Profile_ID' or 'Profile ID') are used in the custom payload, only runtime inputs are considered. Existing configuration values for these fields are not used in such cases.

10.13.11 Custom XML Payload Example

A custom XML payload can be sent to Elemental, including for example this information:

\ <?xml version=”1.0” encoding=”UTF-8”?>\ <job>\ <input>\ <file_input>\ <uri><%=Source%></uri>\ <username>aspera</username>\ <password>toto</password>\ </file_input>\ </input>\ <profile><%=Profile_Id%></profile>\ </job>\

With such a payload, source, username, password, and profile ID are typically defined as runtime inputs. But if source and profile ID are also defined in the configuration, they take precedence. Actually, no runtime inputs are available for Source and Profile_Id in this case.

10.13.12 Dependencies

- Network connectivity to the Elemental Transcoding server on the specified IP address and port.
- Valid Elemental server user account with appropriate permissions for transcoding operations (if authentication is enabled).
- Proper file system access permissions on the Elemental server for source file locations.
- Available transcoding profiles configured on the Elemental server.
- Sufficient storage space on the Elemental server for both input and output files.
- For S3 integration: Valid AWS S3 credentials and appropriate bucket permissions.

10.13.13 Troubleshooting

- If connection to Elemental server fails, verify the IP address and port are correct, and check network connectivity. Ensure SSL settings match server configuration.
- For authentication failures, confirm the username and API key are correct and that the user has appropriate permissions for transcoding operations.
- If profiles don't appear in the dropdown, verify server connectivity and authentication credentials. Try clicking 'Get Elemental Profiles' again after confirming access.
- For source file errors, ensure the file path is correct and accessible from the Elemental server, and that the file format is supported by the selected profile.
- If proxy configuration is causing issues, verify proxy server details are correct and test connectivity through the proxy to the Elemental server.
- For S3 authentication issues, confirm the S3 shared key and secret key are valid and have appropriate permissions for the specified S3 resources.
- If custom XML payload fails, validate the XML syntax and ensure runtime variables like <%=Source%> and <%=Profile_Id%> are properly referenced.

10.14 Emotion operation

This action plug-in provides the ability to invoke Emotion API operation.



Figure 10.14: Emotion operation icon

Version	Comment
1.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.1.8	Encapsulating the job and workflow id field withing curly braces {} if not already done.
1.1.7	Making the Job id field dynamic for Initiate a Job operation.
1.1.6	Fixes source and destination runtime value conversion
1.1.5	Fixed documentation
1.1.4	Adding percent_complete variable.
1.1.3	Supporting Windows File Path
1.1.2	Making Source Field Mandatory
1.1.1	Added percentage complete to inprogress message
1.1.0	2015/03/12 - JG -> JG Added Asynchronous nature to trigger a job
1.0.0	2015/20/08 - JG -> Bug Fix and Feature Enhancement- Added get job status, list workflows, payload execution
0.0.1	2015/20/08 - JG -> Initial Commit

10.14.1 Saved Parameters Description

- Name: The name used to identify a saved EmotionOperation configured instance.
- Comments: Some comments about this saved EmotionOperation configured instance.
- Service URI: The URL of the server where the Emotion service is hosted.(Example: http://localhost:44544/jobs)
- API Operation: Operation to be performed. Currently supports Triggering a Job (Asynch), Submitting a Job through Payload (Asynch), Listing Workflows(Synch), Polling Job Status (Asynch).
- Source File: The file(only 1) on which the operation is to be performed. It uses a hash format. Example : { ”sourceUUID” => ”/tmp/source/first.wav” }. Note that the source UUID must already be present in the Emotion system.
- Destination File: The file into which the output has to be stored. This uses a hash format.Example : { ”destinationUUID” => ”/tmp/source/first.wav” }. Note that the destinationUUID UUID must already be present in the Emotion system.
- Workflow ID : The UUID of the workflow that describes the operation on the source file must be entered here. We manually add the curly brackets for workflow id when they are not there for both the parameters and inputs.
- Job UUID : Optional field. If not assigned, the server would assign a unique id to every job. We manually add the curly brackets for job uuid when they are not there for both the parameters and inputs.
- Inputs : Some workflows require extra inputs for its execution. That could be entered here. It again follows a hash format(Example given below)
- Outputs : Some workflows require extra outputs after its execution. That could be entered here. It again follows a hash format(Example given below)
- Polling Frequency: The frequency of polling for the asynchronous operations.

10.14.2 Inputs Description

The Mandatory inputs are service uri, source, workflow id. Some workflows doesnt require destination. So set it up according to the workflow that you are triggering. For all the Asynchronous job submissions, after the job is submitted the process keeps polling for a status till it gets a valid stable status (”Success”, ”Error”). The polling continues for the ”Queued” and ”InProgress” return status.

10.14.3 Outputs Description

- The output fields obtained are the Job UUID, Job Raw Response, State Description and the message received. In case of an exception, we have a error msg field which displays the error associated with it.

10.14.4 Operating Instructions

Note: - The path for triggering job is http://host:port/jobs. We have added code to handle /jobs. So enter just the IP(or host) and the port.

- Source, Destination, Inputs, Outputs are all entered as a hash. Follow the same.

- *IMPORTANT* - The file path entered should follow different naming to not fail on Windows Machines.(Follow one of the below methods)

1. Change the Windows path to Unix Path.

”C:\\Users\Administrator\Documents\wav-files\audio-file.wav” Should be changed to ”C://Users/Administrator/Documents/wav-files/audio-file.wav”

2. Add extra ”\” for every ”\” present to escape it right. ”\\127.0.0.1\tfs\effengine\inputs\abc.mov” should be changed to ”\\\\10.20.104.40\\tfs\\effengine\\inputs\\abc.mov”

Example:

service URI : *http://localhost:44544/*

workflow Id : *{ be8ce50d-62f9-4e9a-8a8f-9966d63804d1 }*

Source : *{ '7621eb1b-bc75-45c5-9bf4-a90aabae1afb' => 'C:/Users/Administrator/Documents/wav-files/audio-sample.wav' }*

Destination: *{ 'a09751ae-3ec1-4cef-80d7-0a06f89c84a4' => 'C:/Users/Administrator/Desktop/orchestraddtor.wav' }*

Inputs : *{ 'a09751ae-3ec1-4cef-80d7-0a06f89c84a4' => 'C:/Users/Administrator/Desktop/i1.wav', 'a09751ae-3ec1-4cef-80d7-0a06f89c84a4' => 'C:/Users/Administrator/Desktop/i2.wav' }*

10.15 Encoding.com transcoding



Figure 10.15: Encoding.com transcoding icon

This Action plug-in provides the ability to submit file encoding job to a Encoding.com server.

Version	Comment
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.2	remove dependency on custom Rails initializers
0.2.1	Adding percent complete for progress bar support
0.2.0	2016-03-10 RT - Fixed custom payload, added JSON payload support
0.1.3	2015-02-24 ML - New icon, fixes name for consistency
0.1.2	Removes explicit dependency on xmlsimple
0.1.1	prevents autocomplete of user field
0.1.0	added support for custom payload to be submitted to Encoding.com server

Version	Comment
0.0.3	deleted reference to modal_dialog in edit.html
0.0.2	added a default_get method for all plugin fields
0.0.1	initial development

10.15.1 Saved Parameters Description

- Name: The name used to identify a saved Encoding.com Transcoding configured instance.
- Comments: The comments of the Encoding.com Transcoding template
- User ID: The user id to be used to login/authenticate with the Encoding.com server (found under Encoding.com User profile)
- API Key: API Key of the user used to authenticate with the Encoding.com server (found under Encoding.com User profile)
- Use SSL: If SSL is desired, check this box
- Operation: Select one of the four operations. See below for description.
- Source Files: The absolute path to the files. Use comma separated for multiple files. See below for instructions.
- Media IDs: A media id is an unique identifier for Media available in the queue of Encoding.com server.
- Output format: An output format specifies to which format the source file must be encoded to.
- Delete file after encoding: If files are to be deleted from queue after encoding, check this box.
- Notify URL: Encoding.com server sends a notification to this URL (can be an HTTP server or a simple mailto:)
- Status Polling frequency: How often progress will be reported. Default is 5 seconds
- Use json transcoding payload: Check if the custom payload is in JSON format
- Transcoding Payload: If the operation selected is Submit Custom Payload, the plugin submits the xml/json payload given here to encoding.com

10.15.2 Inputs description

The list of inputs depends on the configuration of the EncodingdotcomTranscoding action template.

- All parameters are available as inputs at runtime if not specified by the template.

  This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

These inputs are mandatory:

- *User ID*: A User ID must be provided.
- *API Key*: An API Key must be provided.
- *Operation*: A task must be specified for the plugin to act on. Additional Mandatory Fields if Adding to Queue or if Adding and Encoding
- *Source Files*: The absolute path to the files. Use comma separated for multiple files. See below for instructions.
- *Output format*: An output format specifies to which format the source file must be encoded to. Additional Mandatory Fields if Selecting Media to Encode
- *Media IDs*: A Media must be selected from the dropdown or an array of multiple ids can be specified from Map Inputs.

These inputs are optional:

- *Name*: It is advisable that a name is entered to identify the template.
- *Comments*: Information about the template which will help identify the template.
- *Use SSL*: If SSL is desired, check this box.
- *Delete file after encoding*: If files are to be deleted from queue after encoding, check this box.
- *Notify URL*:
- *Status Polling frequency*: Specify if progress needs to be reported within a time other than default.

10.15.3 Outputs description

- *Input_and_Output_media*: Contains both the input file(s) path involved in the encoding operation and the output file obtained as a result of the encoding
- *Errors*: Errors reported by Encoding.com server during the encoding operation for that specific media

10.15.4 Operating Instructions

This plug-in allows users to provide necessary inputs required for a Encoding.com server to do an encoding job.

Different operations require different set of input parameters. User ID, API Key are needed for all operations. The below table provides information about each operation and what inputs are needed.

Operation	Description	Input(s) and Output
Add to Queue or Add and Encode	The ”Add to Queue” operation adds a source file to the queue of Encoding.com server. The ”Add and Encode” operation adds the file to the queue and immediately encodes it.	Source file(s) and Output format are needed. For each media, the input file url, media_id and output file url are provided as output
Selective Encoding	Once Media ID(s) is/are provided, those media are encoded to requested format.	Media Id must be selected or provided via Map Inputs. For each media, the input file url, media_id and output file url are provided as output
Encode all	All media in Encoding.com server's queue are encoded to requested format.	No more inputs required. For each media, the input file url, media_id and output file url are provided as output

Source file URL formats: A source file URL can be of any of the following protocols ASPERA(FASP), FTP, HTTP, AMAZON S3, RACKSPACE, CLOUDFILES, MICROSOFT AZURE or DROPBOX

The plug-in supports authentication as requested by Encoding.com Server in the latest API release. An example of a encode job is given below. Both single source file and multiple source files (comma separated) can be specified. Encoding.com server uploads the result file to one of its Amazon servers and is available for download.

Important Note1: An output format can be assigned only during Add to Queue or Add and Encode operations.

10.15.5 An example of a transcode

a) Provide User ID and API Key

b) Decide if SSL is necessary. If yes, check the box else leave it un-checked

c) Select an operation that needs to be performed.

d) If the operation is add to queue or add and encode provide source file(s) and output format. Save.

e) When run, the source file(s) and output format is submitted to Encoding.com server and a Media ID (unique identifier) is generated

f) If the operation is selective encoding. Click the button Get Encoding.com Media. This provides a drop-down with media from the Encoding.com server's queue

g) Select a media from the dropdown. Save.

h) When run, the Media ID is submitted to Encoding.com server for encoding

i) If the operation is encode all just Save.

j) When run, all the media in Encoding.com server's queue is encoded media by media

10.16 Envivio transcoding

This is Action plug-ins provides the ability to submit file encoding jobs to an Envivio VOD Encoder thru a 4Balancer manager.

Version	Comment
1.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.0.6	Adding more debug information on failures
1.0.5	Fixing URL to fetch 500 presets and increasing timeout to 10 seconds
1.0.4	Adding Remote Node Dependency

Version	Comment
1.0.3	2015-02-24 ML - New icon, change display name
1.0.2	exception handling of RemoteNode.find_by_name()
1.0.1	Fixes job submission
1.0.0	Making asynchronous
0.1.0	Adds support for additional parameters
0.0.1	initial development

10.16.1 Saved Parameters Description

- Name: The name used to identify a saved EnvivioVodEncoding configured instance.
- Comments: Some comments about this saved EnvivioVodEncoding configured instance.
- 4balancer server node: The node where the 4Balancer controlling the 4Caster VOD is located.
- 4balancer server ip address (or hostname): Can be used to provide a path that overwrite the address of the node.
- Input file name : The source file for the encoding operation. This value can be entered using dynamic attributes
- Output file name : The output file for the encoded media. This value can be entered using dynamic attributes
- Preset ID: Select from the list of encoding presets configured on that 4Balancer.
- Priority: Optional priority for the job - the default is 5.
- Polling frequency: The progress status polling frequency in seconds (the default is 5 seconds).
- Job name : Optional - if not provided, one is generated by Orchestrator. Dynamic attributes are supported.
- Content ID: Optional - used to submit an encoding request with encryption turned on. Dynamic attributes are supported.
- Input start-time: Optional - by default 0 is used. If a value is provided, the Encoding will start at the specified timecode. Dynamic attributes are supported.
- Input end-time: Optional - if a value is provided the encoding will stop at the specified timecode. Dynamic attributes are supported.

10.16.2 Inputs description

The list of inputs depends on the configuration of the EnvivioVodEncoding action template.

The list of inputs depends on the configuration of the AuroraFileVerification action template.

- *4Balancer_server*: Points to the 4Balancer Server or service address. Can be either a node ID (i.e. 'Envivio_node') or a node address (i.e. '192.168.56.101'). This input is only requested when neither the node nor the server address is provided in the action template configuration.
- *Input_filename*: All the variables listed in 'Input file name' field are mandatory and must be of type string. If the 'Input file name' is left blank, then 'Input_filename' must be provided at run-time.
- *Output_filename*: Same as above for the path to the output file name.
- *Priority*: A number between 1 and 10. Default is 5.
- *Preset_ID*: Provides the ability to select the Preset to use for the encoding at run-time. The named in this variable needs to be configured on the 4Balancer server where the execution is routed.
- *Content_ID*: Provides the ability to set a value for 'Content ID' at run-time, if it is not set in the configuration template used.
- *Job_name*: Provides the ability to set a value for 'Job name' at run-time, if it is not set in the configuration template used.
- *Input_start_time* : Provides the ability to set a value for 'Input start-time' at run-time, if it is not set in the configuration template used.
- *Input_end_time* : Provides the ability to set a value for 'Input end-time' at run-time, if it is not set in the configuration template used.

10.16.3 Outputs description

- *Output_file_name*: The file path for the output of the encoding operation.
- *Job_name*: The specified job name for the encoding operation
- *Job_ID*: The job id assigned by 4Balancer for the encoding operation

10.16.4 Supported Actions

None



Figure 10.16: Envivio transcoding icon

10.16.5 Dependencies

None

10.16.6 Operating Instructions

This action executes the file encoding operation configured in the specified Preset on the 4Caster VOD instance managed by the specified 4Balancer server. The connection to the remote server is done using the 4Balancer web-service.

Note: The file path used by Envivio 4Balancer uses the format file://host/path in which host is a server where the network file share is located and path is the mounting path. if a path is provided without a 'file://' prefix, the prefix will be automatically added.

Important Note: this action plugins does not do any file movement. Hence, for it to run a file encoding operation, the specified file has to be accessible on the 4Balancer server to which the job is sent. Either the file is transferred to that server or it must be accessible from that server either on some type of network share.

10.17 EolementheWorkflowManager



Figure 10.17: EolementheWorkflowManager icon

This plug-in provides the ability to run an Eolementhe workflow.

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	Support of the new API with apikey
0.0.2	Correcting the name of metadata.yml for the Help tab to load. It should be same as class name.
0.0.1	initial release

10.17.1 Saved Parameters Description

- Name: The name used to identify a saved EolementheWorkflowManager configured instance.
- Comments: Some comments about this saved EolementheWorkflowManager configured instance.

- Remote Node: The Eolementhe server node name from the list of Remote Nodes.
- Remote server ip address: The Eolementhe server node IP address or domain name.
- Remote service port : The Eolementhe service port (443 by default).
- API key: The API key to authenticate against the Eolementhe server.
- Eolementhe workflows: List of available workflows.
- Workflow id: The id of the workflow to execute. This is populated automatically upon choosing a workflow name in the workflow list.
- Source filename: A source file name.
- Polling frequency: Interval in seconds to check for the job submission status.

10.17.2 Inputs description

The list of inputs depends on the configuration of the EolementheWorkflowManager action template.

Changes in this list of inputs will override any value set in the action configuration/template.

- *remote node or server address*: Eolementhe remote node name or node IP address or domain name
- *API key*: API key to authenticate against the Eolementhe server
- *source*: source filename
- *workflow id*: ID of the workflow to execute. Careful: it has to be the workflow id, not the workflow name.

10.17.3 Outputs description

- *result*: The job submission JSON output

10.17.4 Supported Actions

None

10.17.5 Dependencies

None

10.17.6 Operating Instructions

Specify a remote node or an IP address/domain name and a port for the Eolementhe server.

The IP address/ domain name takes precedence over the remote node if both are defined (which is typically not a good practise).

Specify an API key for authentication.

Click on 'Get Eolementhe Workflows' to obtain a list of available workflows from the Eolementhe server. Select a workflow from the drop down list. The workdlow id is automatically displayed.

Add the source filename (not the full path). Multiple file names can be specified one after the other with a ; separator

In output, you'll get the JSON content returned from the job submission.

Each time you open the plugin, if a workflow has already been selected, a new request is made to the Eolementhe server

to refresh the list of available workflows.

The port can be passed optionaly in input. This input value is not taken into account when clicking on

'Get Eolementhe Workflows', so it may return an error. In this case, specify the workflow id in input and

don't click on 'Get Eolementhe Workflows'.

10.18 Episode transcoding

This is Action plug-ins provides the ability to submit file transformation job to a Telestream Episode engine.



Figure 10.18: Episode transcoding icon

Version	Comment
1.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.1.7	Corrected the variable name and removed colon from :episode_transcoding_epitask_payload for rich text editor helper in edit.html.erb.
1.1.6	Returning percent_complete from check_status method to be consumed by the activities screen.
1.1.5	Adding remote node dependency
1.1.4	Added a parameter in check_staus method to get % complete for all the in-progress tasks.
1.1.3	2015-02-24 ML - New icon
0.1.2	Sometimes output file path is not an array, accounting for this while assigning the output variable
0.1.1	Bug fix in server get method
0.1.0	Added field to allow share-io to be on/off
0.0.3	Submits destination dir, priority, naming with API call. Fetches output file paths correctly. Provides workflow ids as output. Added help file. Tested at Turner 3/12/2014
0.0.2	Exposes epitask filepath
0.0.1	initial development

10.18.1 Saved Parameters Description

- Name: The name used to identify a saved Episode Transcoding configured instance.
- Comments: The comments of the Episode Transcoding template
- Episode Engine Node: The remote node name of the server which hosts the Episode Transcoding engine
- Episode RPC port(optional): The XML RPC port of remote server which hosts the Episode Transcoding engine
- Input filepath: The absolute filepath of the input file which needs to be transcoded
- Output filepath: The directory in which the output file needs to be written into
- Naming: The name which needs to be assigned to the output file
- Epitask Filepath: The absolute path to the file on the server's file system
- Jobname (optional): The absolute filepath to the Epitask file.
- Priority (optional): The priority that needs to be assigned to the Episode task
- Submit in demo mode: If checked, the job will be submitted in demo mode.
- Epitask Payload: The fully constructed XML with job details. This will stop Orchestrator from constructing its own XML
- Polling frequency: Determines how often progress will be reported. Default is 5 seconds

10.18.2 Inputs description

The list of inputs depends on the configuration of the EpisodeTranscoding action template.

- All required parameters are available as inputs at runtime if not specified by the template.

  This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

These inputs are mandatory for transcode operations:

- *Episode Server Node or Episode IP Address*: Either a remote node must be selected or a remote IP address must be provided.
- *Epitask Filepath*: Absolute path to the Epitask file accessible to the Episode engine must be provided

- *Input filepath*: Absolute path to the source file accessible to the Episode engine must be provided

These inputs are optional:

- *Name*: It is advisable that a name is entered to identify the template.
- *Comments*: Information about the template which will help identify the template.
- *Episode XML RPC Port*: The port is optional as a default value is used if not specified.
- *Output filepath*: The output directory will be chosen by Episode using the Epitask file, if not provided.
- *Naming*: The output filename will be chosen by Episode using the Epitask file, if not provided.
- *Jobname*: The jobname will be chosen by Episode using the Epitask file, if not provided.
- *Priority*: The priority will be chosen by Episode using the Epitask file, if not provided.
- *Submit in demo mode*: Default value is false. Check the box to submit in demo mode.
- *Epitask Payload*: Users can chose to not specify a Epitask file but submit a Epitask payload.
- *Polling frequency*: Specify if progress needs to be reported within a time other than default.

10.18.3 Outputs description

- *Output_file_paths*: The output file path as a result in the transcode operation
- *Workflow_IDs*: The task ID of the tasks triggered by job
- *Job_ID*: The ID of the job triggered by the transcode operation

10.18.4 Operating Instructions

This plug-in allows users to provide necessary inputs required for a Episode server to do a transcode job.

The mandatory inputs are listed above.

An example of a transcode job is given below. Currently only a single source file can be specified. Output filepath and Naming if specified is used by the template setup on the Episode server to place the output file

10.18.5 An example of a transcode

a) Provide remote node port (example: 10.0.103.85 or transcode.lala.com)

b) Provide input file name (Ex: ”/data/input/expendables.avi”) and a output file path ”/data/output/”

c) Provide Epitask file path (Ex: ”/data/input/epitask.tsk”)

d) Provide optional inputs such as Naming, Priority and Job Name

e) Orchestrator submits these parameters to the Episode server takes these inputs and performs the transcoding job.

10.19 FFMPEG transcoding



Figure 10.19: FFMPEG transcoding icon

This plugin provides the ability to execute file transcoding operations using the FFMPEG toolset on remote nodes.

Version	Comment
0.6.1	Enhanced action template UI

Version	Comment
0.6.0	Removing Windows support code, Updating to be compatible with Ruby 3.4 and Rails 8
0.5.0	Add execution on Windows flag
0.4.3	Fixes possible timeout when transcoding audio only files
0.4.2	Fixes hide/show in configuration screen
0.4.1	Parse variables in binary_path
0.4.0	Manual Command
0.3.0	Extending option column to text
0.2.7	Adding Remote Node Dependency
0.2.6	New icon, fixes name for consistency
0.2.5	-> Fixed constants in help file (Matched it to their purpose)
0.2.4	-> Fixed constants in help file
0.2.3	-> Corrects description of options in the UI ”Command options (after target filename)”
0.2.2	-> Exposes overwrite option as an optional runtime input
0.2.1	-> fixes typo in post_path_parameters_get
0.2.0	-> Adds possibility to specify ”after file path” parameters. Added help file.
0.1.1	-> Fixed execution node passed as run time parameter
0.1.0	-> Adds overwrite mode to UI
0.0.5	-> Ensure non-interactive exection in Unix
0.0.4	-> suppress error in ffmpeg return
0.0.3	-> Error out if no status is provided
0.0.2	-> Fix bug in default output var name and correct the command line option position
0.0.1	-> Initial release. Does not support add hoc node definition

10.19.1 How It Works

- Executes FFMPEG transcoding operations on remote nodes via SSH connections for distributed media processing.
- Constructs FFMPEG command lines with configurable input/output paths, binary locations, and custom options.
- Supports overwrite protection to prevent accidental file replacement and Windows-specific execution handling.
- Allows flexible command option placement before and after target filenames for advanced FFMPEG parameter control.
- Provides detailed operation reports and output file path information for workflow integration.

10.19.2 Configuration Instructions

1. Add an FFmpeg Transcoding step to your workflow.
2. Configure the essential parameters:
   - Execution node: Select the remote node where FFMPEG is installed.
   - FFMPEG Binary path: Full path to the FFMPEG executable on the remote node.
   - Input file path: Path to the source media file as seen in the SSH session.
   - Output file path: Path where the transcoded file will be created.
3. Optionally configure advanced settings:
   - Command options: Add FFMPEG parameters before or after the target filename.
   - Overwrite output if present?: Enable to allow overwriting existing files.
   - Execution on windows?: Check if the remote node is a Windows system.

10.19.3 Example Configuration

Parameter	Example Value
Execution node	media-processing-server
FFMPEG Binary path	/usr/bin/ffmpeg
Input file path	/media/input/source.mp4
Output file path	/media/output/transcoded.mp4
Overwrite output if present?	checked
Execution on windows?	unchecked
Command options (before target)	-c:v libx264 -preset medium
Command options (after target)	-c:a aac -b:a 128k

10.19.4 Saved Parameters Description

Parameter	Description
Comments	Comments associated to this template or configuration for documentation purposes.
Execution node	The remote node to execute the FFMPEG operation from. List of already configured Remote Nodes.
FFMPEG Binary path	The path (as seen in the SSH session) of the ffmpeg executable on the remote node.
Input file path	The path (as seen in the SSH session) of the input file to FFMPEG for transcoding.
Output file path	The path (as seen in the SSH session) of the output file of FFMPEG transcoding operation.
Overwrite output if present?	Allows to overwrite an existing output file. If not selected, FFMPEG will fail if the output file already exists.
Execution on windows?	To avoid the >/dev/null at the end of the command line in case the execution node is Windows.
Command options (before target filename)	Any FFMPEG option or parameter. This string will be added to the FFMPEG command BEFORE the input file path.
Command options (after target filename)	Any FFMPEG option or parameter. This string will be added to the FFMPEG command AFTER the input file path.

10.19.5 Inputs Description

- Runtime variables: The runtime inputs of this action depend on how the configuration is done. Variables exposed in the command line will be exposed as mandatory inputs.

10.19.6 Outputs Description

- Result_file_path: The path of the output file created by the FFMPEG transcoding operation.
- transformation_report: A detailed report of the FFMPEG operation including processing information and any error messages.

10.19.7 Command Construction

- The FFMPEG command line is constructed in the following order:
- <FFMPEG Binary path> -i <Input file path> [-y (if overwrite flag is set)] <Command options (before target filename)> <Output file path> <Command options (after target filename)>
- Example: `/usr/bin/ffmpeg -i /input/video.mp4 -y -c:v libx264 -preset medium /output/transcoded.mp4 -c:a aac -b:a 128k`

10.19.8 Operating Instructions

- This action plugin provides the ability to submit and control a file transcoding operation using the FFMPEG toolset.
- FFMPEG is executed on a remote node via an SSH connection, allowing for distributed media processing across multiple servers.
- The plugin constructs the complete FFMPEG command line based on your configuration parameters and executes it remotely.
- NOTE: FFMPEG needs to be installed on the remote node. Ensure the binary path is correct and the FFMPEG executable is accessible.
- Variables in command options are exposed as runtime inputs, allowing dynamic parameter configuration during workflow execution.
- The overwrite flag adds the -y parameter to FFMPEG, forcing overwrite of existing output files without prompting.

10.19.9 Dependencies

- FFMPEG must be installed and accessible on the target execution node.
- SSH access must be enabled on the remote node for command execution.
- The remote user account must have read permissions for input files and write permissions for output directories.
- Network connectivity between Orchestrator and the execution node is required for SSH operations.

10.19.10 Troubleshooting

- FFMPEG not found errors: Verify the FFMPEG Binary path is correct and the executable exists at that location on the remote node.
- File already exists errors: Enable 'Overwrite output if present?' option or ensure output paths are unique for each execution.
- SSH connection failures: Check network connectivity and SSH credentials for the remote execution node.

10.20 Flics transcoding



Figure 10.20: Flics transcoding icon

This action plug-in provides the ability to submit and monitor a file transcoding job to a Flics server.

Version	Comment
0.1.1	Enhanced action template UI
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	Returns message in case of failure
0.0.1	Initial implementation

10.20.1 How It Works

- This action plug-in submits a transcoding task to a Flics server.
- It supports both standard and advanced task creation modes.
- The plug-in monitors the transcoding task until completion using polling.
- Generated destination files are returned as outputs once the task completes.

10.20.2 Configuration Instructions

1. Create a new action step using the FlicsTranscoding plug-in.
2. Complete the following configuration fields:
   - Name: The name used to identify a saved FlicsTranscoding configuration.
   - Comments: Optional comments explaining the purpose of this transcoding step.
   - Address: The IP address or hostname of the Flics server.
   - Use ssl?: If selected, SSL is used to connect to the Flics server.
   - Port: The port number of the Flics server.
   - Username: The username used to authenticate with the Flics server.
   - Password: The password used to authenticate with the Flics server.
   - Use advanced mode: If selected, a custom transcoding task definition is used.

- Custom task: JSON body used to create a custom transcoding task when advanced mode is enabled.
- Source file name: Path to the source file inside a Flics source storage.
- Source storage name: Dropdown list of available Flics source storages.
- Source storage id: ID of the selected Flics source storage.
- Template name: Dropdown list of available Flics templates.
- Template id: ID of the selected Flics template.
- Title: Title assigned to the transcoding task.
- Tags: Tags associated with the transcoding task.
- Polling frequency: Interval in seconds used to check the transcoding task status.

10.20.3 Example Configuration

Below is a minimal configuration example for the FlicsTranscoding plug-in.

Parameter	Value
Address	flics.example.com
Use ssl?	Selected
Username	flics_user
Source file name	/incoming/video.mov
Template name	HD_Transcode
Polling frequency	10

10.20.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved FlicsTranscoding configuration.
Comments	Optional comments explaining the purpose of this transcoding step.
Address	The IP address or hostname of the Flics server.
Use ssl?	If selected, SSL is used to connect to the Flics server.
Port	The port number of the Flics server.
Username	The username used to authenticate with the Flics server.
Password	The password used to authenticate with the Flics server.
Use advanced mode	If selected, a custom transcoding task definition is used.
Custom task	JSON body used to create a custom transcoding task when advanced mode is enabled.
Source file name	Path to the source file inside a Flics source storage.
Source storage name	Dropdown list of available Flics source storages.
Source storage id	ID of the selected Flics source storage.
Template name	Dropdown list of available Flics templates.
Template id	ID of the selected Flics template.
Title	Title assigned to the transcoding task.
Tags	Tags associated with the transcoding task.
Polling frequency	Interval in seconds used to check the transcoding task status.

10.20.5 Inputs Description

Mandatory and optional inputs depend on the selected configuration mode.

Parameter	Description
address *	IP address or hostname of the Flics server.

Parameter	Description
*username **	Username to log in to the Flics server.
*password **	Password to log in to the Flics server.
*custom_task **	JSON body defining a custom transcoding task when advanced mode is enabled.
*source_file_nam **	Path to the source file inside a Flics source storage.
*source_stor-age_id_or_name **	ID or name of the Flics source storage (must be an input storage).
*tem-plate_id_or_nam **	ID or name of the Flics template.
port	Port number of the Flics server. Defaults to 80 (non-SSL) or 443 (SSL).
title	Title of the transcoding task.
tags	Tags associated with the transcoding task. Default is 'local'. Comma-separated values are supported.
polling_fre-quency	Polling interval in seconds. Default is '10' seconds.

10.20.6 Outputs Description

Parameter	Description
destination_files	Files generated by the transcoding task.

10.20.7 Supported Actions

None

10.20.8 Dependencies

Network connectivity to the Flics server and valid Flics credentials.

10.20.9 Operating Instructions

1. The plug-in connects to the configured Flics server using the provided credentials.
2. A transcoding task is created using either a template-based or custom task definition.
3. If advanced mode is enabled, the provided JSON body is used directly.
4. The task status is polled periodically until completion or failure.
5. Once completed, generated destination files are returned as outputs.

10.21 FlipFactory transcoding



Figure 10.21: FlipFactory transcoding icon

This is Action plug-ins provides the ability to submit file transformation job to a Telestream FlipFactory server.

Version	Comment
1.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.1.2	Added remote Node dependency
1.1.1	Added help file
1.1.0	2015-02-24 ML - New icon, change display name
1.0.9	Exposing server node as a runtime input
1.0.8	exception handling of RemoteNode.find_by_name()
1.0.7	Bug fix in default payload which was submitting user password without substituting provided value.
1.0.6	Expose password as optional input
1.0.5	detect network errors when communicating with Flip server and handle failures
1.0.4	exposes failures as outputs
1.0.3	fix possible uuid without curly
1.0.2	fix accessing the URL variable
1.0.1	fix typo in polling frequency
1.0.0	Move to asynchronous
0.2.0	Move FLIP_DIR creation out of clean-up function
0.1.5	Bug fix for over-written message
0.1.4	Automated clean up of state files
0.1.3	adds rich text editing for XML message
0.1.2	fix bug in output of files
0.1.1	adds password and mediatype to default message
0.1.0	adds support for node selection
0.0.3	remove dependency on ROXML
0.0.2	expose flip factory server as parameter
0.0.1	initial development

10.21.1 Saved Parameters Description

- Name: The name used to identify a saved FlipFactoryTransformation configured instance.
- Comments: Some comments about this saved FlipFactoryTransformation configured instance.
- Name: Name used to identify the Plugin Step
- Comments: The description field where you can enter any comments to be stored for future reference
- Flipfactory Server Node: The remote node on which the TlipFactory Transcoding server is installed
- Server url (or address/hostname): If the node is not provided, you can enter the server url here
- Polling Frequency: The frequency at which the status has to be polled
- Message: This field is to store the external payload. All the Ruby inputs given here will come on the Preset Parameters on refresh
- user name: Default field for empty message. This is to collect the user name of the transcoding user
- user_password: Default field for empty message.This field is used to collect the password of the trancoding user
- factory name: Default field for empty message. This field is used to collect the factory name
- Media type: Default field for empty message.This field is used to store the media type of the field being transcoded
- file_paath: Default field for empty message.This field stores the path to the file to be transcoded

10.21.2 Inputs description

The list of inputs depends on the configuration of the FlipFactoryTransformation action template.

- *Remote Node/Service URL,* : Is the only mandatory field needed
- *user_name, user_password, media_type, factory_name, file_path*: These fields are mandatory if the default payload is chosen.

10.21.3 Outputs description

- *output_directory*: The directory in which the output file is stored
- *output_files_array*: This stores the list of files that has been transcoded
- *total_processed*: Total number of files processed
- *first_error_msg*: Stores the first error message if any
- *first_error_type*: Stores the type of the first error message if any
- *error_messages*: This stores all the error messages received from the transcoder

- *error_types*: This stores the type of all the error messages

10.21.4 Supported Actions

None

10.21.5 Dependencies

None

10.21.6 Operating Instructions

This plugin helps to perform the flip factory transcoding operation on a given file.

NOTE: From the service provider -> FlipFactory and all products in the FlipFactory family (including Ad Manager, Traffic-Manager and Graphics Factory) are no longer available for purchase.

10.22 HandBrake transcoding



Figure 10.22: HandBrake transcoding icon

This action plug-in provides the ability to submit and control a file transcoding operation using the HandBrake toolset.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.5	Removing Windows support code
0.0.4	Adding Remote Node Dependency
0.0.3	2015-02-24 ML - New icon, fixes display name for consistency
0.0.2	- 2015/02/09 - JG -> Parse Code Commented since its not used.
0.0.1	-> Initial release.

10.22.1 Saved Parameters Description

- This action allows to execute an HandBrake task on a remote node.
- Name: The name used to identify a saved HandbrakeTranscoding configured instance.
- Comments: Comments associated to this template or configuration.
- Execution node: The remote node to execute the HandBrake operation from. List of already configured Remote Nodes.
- HandBrake Binary path: The path (as seen in the SSH session) of the HandBrakeCLI executable.
- Input file path: The path (as seen in the SSH session) of the input file to HandBrake.
- Output file path: The path (as seen in the SSH session) of the output file of HandBrake.
- Command options (before source filename): Any HandBrake option or parameter. This string will be added to the HandBrake command BEFORE the input file path

10.22.2 Inputs description

- Input file path, Output file path and the Execution node, Handbrake binary path are mandatory parameters for the execution of this plugin.

  All of them support both design time and runtime specification. They also support dynamic entry (Example shown below)

  The run time inputs of this action depend on how the configuration is done. Variables exposed in the command line will be exposed as mandatory inputs.

10.22.3 Outputs description

- This action executes an HandBrake transformation action on a remote node, via SSH.
- *Result_file_path*: The path of the output file
- *transformation_report*: A report of the HandBrake operation

10.22.4 Supported Actions

None

10.22.5 Dependencies

None

10.22.6 Operating Instructions

This action plug-in provides the ability to submit and control a file transcoding operation using the HandBrake toolset.

HandBrake will be executed on a remote node via an SSH connection.

NOTE: HandBrakeCLI needs to be installed in the remote node.

The transformation result file will be stored on the specified path in the remote node.

Below are some examples showing different ways the plugin can be used.

Note: Execution node is not included below since the examples are all run on a SSH client.

Example 1 (Inputs at Design time):

In this example all the inputs required are provided at design phase.

DESIGN TIME CONFIGURATION:

Input file path : ”/usr/bin/input/test.flv”

Output file path : ”/usr/bin/output/test.mp4”

Handbrake binary path : ”/usr/bin/./HandBrakeCLI”

OUTPUTS:

Shows the progress report along with the status of the transcoding

Example 2 (Dynamic entry: Inputs at Design and Run time):

In this scenario, we are providing partial inputs at design name and the filename value is collected at runtime.

DESIGN TIME CONFIGURATION:

Input file path : ”/usr/bin/input/<%=filename %>.flv”

Output file path : ”/usr/bin/output/<%=filename %>.mp4”

Handbrake binary path : ”/usr/bin/./HandBrakeCLI”

RUNTIME CONFIGURATION:

filename : ”test”

OUTPUTS:

Shows the progress report along with the status of the transcoding

Example 3 (Dynamic entry : Inputs at Design and Run time using inbuilt functions):

In this example, we collect the input_file_path at runtime and use inbuilt function to create the output_file_path from it.

DESIGN TIME CONFIGURATION:

Output file path : ”/usr/bin/output/<%=File.nakedname(input_file_path) %>.mp4”

Handbrake binary path : ”/usr/bin/./HandBrakeCLI”

RUNTIME CONFIGURATION:

input_file_path : ”usr/bin/input/test.flv”

OUTPUTS:

Shows the progress report along with the status of the transcoding

10.23 Hybrik Transcoder



Figure 10.23: Hybrik Transcoder icon

This Action plug-in provides the ability to submit a transcoding job to the Hybrik transcoder

Version	Comment
0.1.2	Enhanced action template UI
0.1.1	Remove beta flag, cleanup code
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	output error message in case of error
0.0.1	initial development

10.23.1 How It Works

- Integrates with Hybrik cloud transcoding service via REST API.
- Authenticates using API key/secret or username/password.
- Submits transcode jobs via JSON payload matching Hybrik API specifications.
- Supports all Hybrik transcode task parameters and codecs.
- Monitors transcode job progress through configurable polling.
- Returns job ID, results, and error messages as outputs.
- Results available in both Ruby structure and JSON format.
- Asynchronous job processing with automatic status polling.

10.23.2 Configuration Instructions

1. Create a new action step using the HybrikTranscoder plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved Hybrik Transcoder configured instance.
   - Comments: Comments about this Hybrik Transcoder template.

- Endpoint: The URL endpoint for API requests to Hybrik (e.g. https://api-demo.hybrik.com/v1). Must be reachable on port 443.
- Oapi key: The API key used for authentication.
- Oapi secret: The API secret used for authentication.
- Username: The Hybrik account username for authentication.
- Password: The Hybrik account password for authentication.
- Json payload: The JSON payload defining the transcode job. Must match Hybrik transcode request body specification.
- Status Polling frequency: Interval in seconds to check job status (default: 5 seconds).

10.23.3 Example Configuration

This is an example configuration for the HybrikTranscoder plugin using API key authentication.

Parameter	Value
Name	Transcode to ProRes
Endpoint	https://api-demo.hybrik.com/v1
Oapi key	your_api_key_here
Oapi secret	your_api_secret_here
Username	your_username
Password	your_password
Status Polling frequency	10
Json payload	{”transcode_task”: {...}}

10.23.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved Hybrik Transcoder configured instance.
Comments	Comments about this Hybrik Transcoder template.
Endpoint	The URL endpoint for API requests to Hybrik (e.g. https://api-demo.hybrik.com/v1). Must be reachable on port 443.
Oapi key	The API key used for authentication.
Oapi secret	The API secret used for authentication.
Username	The Hybrik account username for authentication.
Password	The Hybrik account password for authentication.
Json payload	The JSON payload defining the transcode job. Must match Hybrik transcode request body specification.
Status Polling frequency	Interval in seconds to check job status (default: 5 seconds).

10.23.5 Inputs Description

All parameters can be available at runtime if not specified in the plugin configuration. Runtime inputs override configured template values.

Parameter	Description
*Endpoint **	URL endpoint for Hybrik API (e.g. https://api-demo.hybrik.com/v1). Port 443 must be reachable.
*Oapi_key **	API key from Hybrik Account > Info > API details.
*Oapi_secret **	API secret from Hybrik Account > Info > API details.
*Username **	Hybrik account username for authentication.
*Password **	Hybrik account password for authentication.
*Json_payload **	JSON payload defining transcode job. Must match Hybrik API specification.
Status_Polling_frequency	Polling interval in seconds (default: 5 seconds).

10.23.6 Outputs Description

Parameter	Description
result	The result from the transcode operation expressed as a Ruby structure (hash).
result_json	The result from the transcode operation expressed as a JSON structure (string).
job_id	The unique Hybrik transcode job ID for tracking and status queries.
error_message	The transcode error message if the job fails.

10.23.7 Supported Actions

Transcode operations only (all Hybrik transcode task types supported via JSON payload).

10.23.8 Dependencies

Requires network access to Hybrik API endpoint on port 443 (HTTPS). Requires valid Hybrik account with API credentials or username/password.

10.23.9 Operating Instructions

This plugin integrates with the Hybrik cloud transcoding service to submit and monitor transcode jobs via REST API.

Pre-requisites

- The Hybrik endpoint must be reachable on port 443 from your Orchestrator system.
- Requires valid Hybrik account with either API credentials or username/password.
- API credentials recommended for production use - found in Hybrik GUI: `Account > Info > API details`.
- HTTPS connectivity required (port 443). HTTP not supported.

Authentication Methods

- API Key/Secret Authentication:
  - Recommended for production use
  - More secure than username/password
  - Found in Hybrik GUI: `Account > Info > API details`
  - Both Oapi key and Oapi secret must be provided
- Username/Password Authentication:
  - Account-level authentication
  - Less secure than API credentials
  - Provide your Hybrik account username and password
  - Both fields must be provided

JSON Payload Specification

- The JSON payload must conform to Hybrik API v1 specification.
- Reference: `https://docs.hybrik.com/api/v1/HybrikAPI.html?#transcode-task`
- Minimum required payload structure:
  - `{"transcode_task": {...}}`
  - Includes source/destination configuration
  - Specifies video codec and encoding parameters
  - Defines audio encoding if needed
- Payload can include:
  - Input source (file path, URL, or S3)
  - Output destination (local path or S3)
  - Video codec, bitrate, resolution, frame rate
  - Audio codec, bitrate, sample rate
  - Container format (MP4, MXF, MOV, etc.)
  - Watermarking and overlay options

  - DRM settings if applicable

- Example minimal payload:

```
{"transcode_task": {"source": {"path": "/input/video.mov"}, "destination": {"path": "/output/video.mp4"}, "outputs": [{"video_codec": "h264", "bitrate": "5000k"}]}}
```

Common Use Cases

- H.264 video transcoding: Convert video to H.264 codec for web delivery.
- Multiple output formats: Create multiple outputs (proxy, archive, delivery) in single job.
- Codec conversion: Transcode from ProRes to DNxHD or other professional codecs.
- Resolution scaling: Resize video to multiple resolutions (4K, 1080p, 720p, etc.).
- Watermarking: Apply watermarks or graphics overlays during transcoding.
- Frame rate conversion: Convert between 24fps, 25fps, 29.97fps, 50fps, 59.94fps, etc.
- Archive creation: Create long-term archive versions with high bitrate and quality.
- Proxy generation: Generate low-resolution proxies for editing workflows.
- Live streaming preparation: Create HLS/DASH streaming files with multiple bitrate tiers.
- DRM protection: Apply DRM/encryption to output files when supported.

Best Practices

- Use API key/secret authentication instead of username/password for production.
- Verify port 443 accessibility from Orchestrator to Hybrik endpoint before deployment.
- Use valid Hybrik endpoint URL - include protocol (https://) and API version (e.g. /v1).
- Test JSON payload locally against Hybrik API documentation before using in workflow.
- Set appropriate polling frequency - 5-10 seconds typical (balance responsiveness vs server load).
- Use meaningful job names/tags in payload for tracking and debugging.
- Enable detailed logging to capture API requests/responses for troubleshooting.
- Monitor Hybrik account quota to ensure sufficient transcoding resources.
- Use S3 paths for input/output when possible for better performance and scalability.
- Handle error messages in workflow to retry or notify on transcode failures.
- Validate output file existence after job completion before proceeding downstream.
- Keep API credentials secure - use environment variables or secure storage, never hardcode.

Troubleshooting

- Connection timeout/refused: Verify Hybrik endpoint URL is correct and port 443 is accessible from Orchestrator.
- Authentication failed: Verify API key/secret or username/password are correct. Check Hybrik GUI for current credentials.
- Invalid JSON payload: Validate JSON syntax and structure against Hybrik API documentation (https://docs.hybrik.com/api/v1).
- Job submission fails: Check payload matches Hybrik transcode task specification. Review error message in output.
- Job stuck/timeout: Monitor Hybrik account for available transcoding resources. Check if job is queued behind other jobs.
- Output file not created: Verify output path in JSON payload is writable and accessible. Check Hybrik job logs.
- Codec not supported: Verify codec specified in payload is supported by Hybrik. Refer to API documentation.
- Polling never completes: Increase polling frequency or check Hybrik service status. Verify job ID is being returned.
- Memory/resource errors: Reduce video resolution, bitrate, or complexity in payload. Use proxy quality for testing.
- S3 access denied: If using S3 paths, verify AWS credentials and bucket permissions in Hybrik configuration.

Important Notes

- All authentication fields required: Must provide EITHER (API key + API secret) OR (Username + Password).
- Endpoint must be HTTPS: Protocol https:// is required, not http.
- Port 443 required: Hybrik API only accessible via HTTPS port 443. Not configurable.
- JSON payload validation: Payload must be valid JSON and match Hybrik API specification exactly.
- Default polling is 5 seconds: Can be overridden with Status Polling frequency parameter.
- Asynchronous execution: Plugin returns immediately with job ID, then polls until completion.
- Job ID critical: Job ID is required for status tracking and cancellation. Always capture in output.
- Results available in two formats: Use 'result' for Ruby hash or 'result_json' for JSON string depending on needs.
- Error message provided on failure: Check 'error_message' output for detailed failure reason.
- Parameters override template values: Runtime inputs override configured values for flexibility.

10.24 Marquis Mews Operation



Figure 10.24: Marquis Mews Operation icon

This plug-in provides the ability to use the API's listed under the Medway Web Service solution of Marquis.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	fixing a commit bug.Variable missing
0.0.1	initial release

10.24.1 Saved Parameters Description

- Name: The name used to identify a saved Marquis Mews Operation configured instance.
- Comments: The comments of the Vantage Transcoding template
- Service URL: The remote url along with its port to access the Medway server
- Operation: This Marquis plugin supports Simple MEWS Transfer, EDL Transfer to Interplay and Get Job Status Operations currently.
- Payload: The parameter shows the payload based on the operation chosen.

10.24.2 Inputs description

The list of inputs depends on the configuration of the MarquisMewsOperation action template.

- All parameters are available as inputs at runtime if not specified by the template.

  This plugin loads the payload based on the Operation chosen. So the inputs vary according to the payload. Users, can create addition fields by appending <%= FieldName %> to the payload.

  This plug-in does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

These inputs are mandatory if you are submitting with the Simple Transfer Operation

- *Service URL*: The url with the port to access the Medway Service
- *Payload*: Payload for the operation chosen. This payload is submitted as is after filling in the required fields. Either a remote node must be selected or a remote IP address must be provided.
- *SourceFile*: The path to the source file
- *DestinationFile*: The path to the destination file
- *ResourceId*: The unique resource id that is need for each job

These inputs are mandatory if you are submitting with Get the Job Status Operation:

- *Service URL*: The url with the port to access the Medway Service
- *Payload*: Payload for the operation chosen. This payload is submitted as is after filling in the required fields. Either a remote node must be selected or a remote IP address must be provided.
- *ResourceId*: The resource id of the job that you want the status for

These inputs are mandatory if you are submitting with the EDL Transfer to Interplay Operation

- *Service URL*: The url with the port to access the Medway Service

- *Payload*: Payload for the operation chosen. This payload is submitted as is after filling in the required fields. Either a remote node must be selected or a remote IP address must be provided.
- *AvidInterplayLocationURL* : The path to the AVID location URL
- *DestinationFile*: The path to the destination file

These inputs are optional:

- *Name*: It is advisable that a name is entered to identify the template.
- *Comments*: Information about the template which will help identify the template.

10.24.3 Outputs description

- *Response Received*: The XML received as is from the Medway Server
- *Error Message*: In case of error this holds the cause of the exeption

10.24.4 Operating Instructions

This plug-in allows users to provide necessary inputs required for a Medway server to do a transcode job. The mandatory inputs are listed above.

Any input can be added to the payload by using ”<%= FieldName %>” format.

10.25 Minnetonka AudioTools Server



Figure 10.25: Minnetonka AudioTools Server icon

This plug-in provides the ability to process, convert, encode, and decode audio files with the Minnetonka AudioTools Server. It implements the AudioTools SOAP based WEB service API.

Version	Comment
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.2	Updating to standard coding style (snake case variables)
0.2.1	Removing Windows support code
0.2.0	Adding a way to disable escaping HTML
0.1.1	Parsing ATS response XML to provide more info on execution from ATS log
0.1.0	Adding new dropdown to support new WSDL file
0.0.5	Fix to load the right WSDL path for Windows
0.0.4	percent complete for progress bar support
0.0.3	Added Remote Node Dependency
0.0.2	Added metadata.yml file
0.0.1	initial release

10.25.1 Saved Parameters Description

- Name: The name used to identify a saved MinnetonkaAudiotoolserver configured instance.
- Comments: Some comments about this saved MinnetonkaAudiotoolserver configured instance.
- Remote Node: The Audiotools server node name from the list of Remote Nodes.

- Remote server ip address: The Audiotools server node IP address or domain name.
- Remote web service port : To change the Audiotools SOAP service port (9090 by default).
- Workflow XML: A file path or a memory buffer to the workflow XML to execute.
- Init from type: The method used to load the workflow XML: from a file path or from a memory buffer.
- Priority: The submitted job priority: 0 for low, 1 for medium, 2 for high.
- Disable escaping?: To disable escaping the workflow XML content. By default this content is sent to Minnetonka with escaping.
- Polling frequency: Interval in seconds to check for the job submission status.

10.25.2 Inputs description

The list of inputs depends on the configuration of the MinnetonkaAudiotoolserver action template.

Changes in this list of inputs will override any value set in the action configuration/template.

- *remote node or server address*: Audiotools remote node name or node IP address or domain name
- *workflow xml*: A file path or a memory buffer to the workflow XML to execute
- *priority*: The submitted job priority

10.25.3 Outputs description

- *XML_result*: The workflow XML output
- *ATS_status_message*: The job status code

10.25.4 Supported Actions

None

10.25.5 Dependencies

None

10.25.6 Operating Instructions

Specify a remote node or an IP address/domain name for the Audiotools server.

Specify the workflow XML content in clear (non escaped).

The referenced workflow XML content can be in a file. In this case, specify 'file' in 'Init from type' and the full filepath to the file in 'Workflow XML'. The filepath must be visible from Orchestrator.

Or it can be in a memory buffer. In this case, specify 'buffer' in 'Init from type', don't put anything in 'Workflow XML' and map the XML content as 'Workflow XML' input. For example, the XML content can be built with a File Generator plugin in an earlier step.

Specify a priority for the job to submit.

In output, you'll get the Audiotools status code and the workflow XML output.

The completed workflow jobs are removed from the Audiotools server. The failed workflow jobs are left on the server.

10.26 MOG MXF Speedrail

his plug-in provides the ability to run a MOG workflow. It implements MOG mxfSPEEDRAIL v2.22 SOAP API.

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.2	Fix progress report display issue
0.1.1	Adding new flow name mog.speedrail.flows.MetadataCheckin
0.1.0	Adding new column, job name to help identify the jobs submitted uniquely
0.0.3	Adding percentage complete variable

Version	Comment
0.0.2	Adding Remote Node Dependency
0.0.1	initial release

10.26.1 Saved Parameters Description

- Name: The name used to identify a saved MogMxfspeedrail configured instance.
- Comments: Some comments about this saved MogMxfspeedrail configured instance.
- Remote Node: The MOG server node name from the list of Remote Nodes.
- Remote server ip address: The MOG server node IP address or domain name.
- Remote soap service port : The MOG SOAP API service port (8731 by default).
- Username: The login name to authenticate against the MOG server.
- Password: The password for the login to the MOG server.
- Flow name: The type of workflow to execute.
- XML payload: The XML payload for the job submission request.
- Polling frequency: Interval in seconds to check for the job submission status.

10.26.2 Inputs description

The list of inputs depends on the configuration of the MogMxfspeedrail action template.

Changes in this list of inputs will override any value set in the action configuration/template.

- *remote node or server address*: MOG remote node name or node IP address or domain name
- *user name*: login name to the MOG server
- *password*: password for the login to the MOG server
- *flow name*: type of workflow to execute
- *xml payload*: XML payload for the job submission request

10.26.3 Outputs description

- *XML_result*: The job submission XML output
- *MOG_error_message*: MOG error message in case of error

10.26.4 Supported Actions

None

10.26.5 Dependencies

None

10.26.6 Operating Instructions

Specify a remote node or an IP address/domain name for the MOG server.

Specify a username and password for authentication.

Select the type of workflow to execute in the flow name field.

Specify the XML payload for your specific type of workflow. This is the section within the CDATA delimiters, not including the delimiters. For example, if we consider a mog.speedrail.flows.transferplainmxf flow name, the XML starts with <pf:TransferPlainMXF> and ends with </pf:TransferPlainMXF>.

In output, you'll get the XML content returned from the job submission and the MOG error message in case of error.

10.27 Unified packager

This action plug-in provides the ability to create a server manifest file with mp4split tool.



Figure 10.26: MOG MXF Speedrail icon



Figure 10.27: Unified packager icon

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.1	Initial implementation

10.27.1 Saved Parameters Description

- Name: The name used to identify a saved UnifiedPackager configured instance.
- Comments: Some comments about this saved UnifiedPackager configured instance.
- Execution node: The remote node where mp4split is installed.
- Output file path: The path of the output file.
- Binary path: The path of the mp4split executable.
- License path: The path of the license file.
- Option (before output file path): The options added BEFORE the output file path.
- Option (after output file path): The options added AFTER the output file path.
- Input file path: The path of an input file.
- Track options: The related track options to a specific input file.

10.27.2 Inputs description

The list of inputs depends on the configuration of the UnifiedPackager action template.

- These inputs are mandatory:
- *execution_node*: The remote node where mp4split is installed.
- *output_file_path*: The path of the output file. These inputs are optional:
- *binary_path*: The path of the mp4split executable (default is 'mp4split').
- *license_path*: The path of the license file.
- *options_before_output_file_path*: The options added BEFORE the output file path.
- *options_after_output_file_path*: The options added AFTER the output file path.
- *input_files_data*: The list of input file paths and related track options expressed as a string with content such as [[”/tmp/test1.mp4”,”--track_id=1”],[”/tmp/test2.mp4”,”--track_id=2”]]

10.27.3 Outputs description

- *report*: The execution report.
- *status_code*: The mp4split status code.

10.27.4 Supported Actions

None

10.27.5 Dependencies

None

10.27.6 Operating Instructions

mp4split will be executed on a remote node via an SSH connection.

File path can be relative or absolute.

License file can be set in a mp4split alias on remote node.

mp4split command line used is constructed as follow : <binary_path> --license-key=<license_path> <options_before_output_file_path> -o <output_file_path> <options_after_output_file_path> <input_file_path> <track_options> <input_file_path> <track_options>

10.28 Vantage Transcoding



Figure 10.28: Vantage Transcoding icon

This action plugin provides the ability to submit file transcoding jobs to a Vantage server.

Version	Comment
1.0.1	Enhanced action template UI
1.0.0	Migrated from soap/wsdlDriver to Savon gem for SOAP operations
0.9.0	Remove dependency on custom Rails initializers, Updating to be compatible with Ruby 3.4 and Rails 8
0.8.0	Added option to fail if restart of Vantage job fails.
0.7.3	If restart job selected, and restarting fails on vantage, fails the action.
0.7.2	Forcing to use DB persistency in shared state to reuse job id
0.7.1	Added job tracking logs
0.7.0	Added possibility to restart a job id
0.6.8	Added debug information on sessions stats returned by Vantage
0.6.7	Protect against empty values in Vantage response
0.6.6	Fixed problem with progress report not being populated
0.6.5	Alphabetizes Vantage workflow list
0.6.4	Verify ssl false
0.6.3	Adding raw for hepler method to render tht HTML as is.
0.6.2	Removing soap4r dependency
0.6.1	Comparing the full path instead of the basename while removing input file from output list

Version	Comment
0.6.0	New output - Labels
0.5.9	New output(output files as hash) includes label
0.5.8	New output(output files as hash) includes attachment and pruned detailed error message
0.5.7	New output(output files as hash) and detailed error message for vantage 7
0.5.6	Impose runtime workflow specification to have precedence over design time configuration
0.5.5	Make sure job id is always returned as output when available.
0.5.4	Run time support for polling frequency.
0.5.3	Returning percent_complete from check_status method to be consumed by the activities screen
0.5.2	Make sure runtime workflow specification has precedence over design time configuration
0.5.1	Adding Remote Node Dependency
0.5.0	File paths that are pointing at ftp, sftp location should not be Windozed.
0.4.9	Remove input files from list of output files when using UNC paths.
0.4.8	Adds support for dynamic category reporting.
0.4.7	Beta release for a performance enhacement. Not making two calls for variables fetch and not making a workflow list call when workflow id is present.
0.4.6	Fixing bug with Vantage variables and items when they are booleans.
0.4.5	Moved helper code into controller and made necessary changes for the helper to load. Added new method to only output vantage workflows
0.4.4	Tested at Turner 3-26-2015. Added help file content. Cleaned up logging. Added new method to only output variables and items.
0.4.3	Adding a new output for Vantage sessions (steps) info which will contain step/session details
0.4.2	Made wf_variables and wf_items fields available at runtime as JSON Strings
0.4.1	Made server node available for runtime configuration
0.4.0	Added the default polling frequency to configuration UI
0.3.9	Fixes bug introduced in wf_items_get in prior versions. When items(s) are hardcoded the value did not make it to Vantage. Tested at Turner 2014/11/14
0.3.8	Correctly fixes single variable issue tackled in above versions. Adds additional check if object exists. Requested by Turner 2014/11/12
0.3.7	Fixes variable names in case of run time specification
0.3.6	Fixes syntax error in case of a single variable wf
0.3.5	exception handling of RemoteNode.find_by_name()
0.3.4	Fixes problem when a workflow has a single variable
0.3.3	Fixes problem with output item file paths
0.3.2	Fixes problem when the ”run time selected workflow” has less inputs/variables than the one used for design time configuration.
0.3.1	Filtering items and variables if not requested by the workflow at runtime
0.3.0	Added WaitingToRetry as one of the Inprogress statuses - requested by Showtime 8/8/14
0.2.9	Silences the warnings and allows to overwrite the workflow id or name at runtime
0.2.8	Fixed getting correct input specs when server is specified at runtime input.
0.2.7	Fixed type in wf_items_get
0.2.6	Fixed issue with wf_items and wf_variables methods
0.2.5	Made server address available as a runtime input. Made workflow_id and workflow_name as optional runtime inputs. Workflow_id will be derived from workflow_name if it is provided.
0.2.4	Added exception handling for vantage variables' and items' fetch
0.2.3	Submitting variables to Vantage with same object id instead of a new random object id. Bug fix to read runtime inputs for variables and items correctly.
0.2.2	made variables and items runtime optional inputs
0.2.1	made job name a runtime input
0.2.0	added support to apply values to variables requested in Vantage workflow
0.1.0	turning into asynchronous #not tested
0.0.4	deleted reference to modal_dialog in edit.html
0.0.3	fixed call to workflow_id_get
0.0.2	added a default_get method for all plugin fields and moved dependent files into plugin folder
0.0.1	initial development. Supports API version 4.0.3

10.28.1 How It Works

- Integrates with Vantage Transcoding servers to perform video and audio transcoding operations through API calls.

- Supports four API methods: SubmitFile for single-file transcoding, SubmitItems for multi-file processing with variables and items, Fetch Variables & Items to retrieve workflow parameters, and Fetch Vantage Workflows to get available workflows.
- Provides dynamic workflow retrieval from Vantage servers, allowing users to select from available transcoding profiles and configurations.
- Handles workflow variables and items submission, enabling customized transcoding parameters for different processing requirements.
- Includes job monitoring capabilities with configurable polling intervals and automatic status tracking until transcoding completion.
- Supports job restart functionality when Orchestrator steps are re-executed, providing workflow resilience and recovery options.

10.28.2 Configuration Instructions

1. Add a Vantage Transcoding step to your workflow.
2. Configure the basic connection settings:
    - Vantage Server Node: Select a remote node pointing to your Vantage server.
    - Vantage Server IP address: Enter the IP address or hostname of the Vantage server.
    - Vantage API port: Specify the port number (default is 8676).
3. Set up transcoding parameters:
    - API Method: Choose between SubmitFile or SubmitItems based on your requirements.
    - Source file: Provide the absolute path to the input file.
    - Vantage workflows: Click 'Get Vantage Workflows' and select from available workflows.
    - Job name: Optionally specify a custom name for the transcoding job.
4. Configure advanced options:
    - Vantage Variables/Items: Provide values for workflow-specific parameters.
    - Status Polling frequency: Set the interval for job status checking.
    - Restart options: Configure job restart behavior for step re-execution.

10.28.3 Example Configuration

Parameter	Example Value
Vantage Server Node	vantage-server-01
Vantage Server IP address	10.0.103.85
Vantage API port	8676
Source file	C:\InputFiles\Wildlife.wmv
API Method	SubmitFile
Vantage workflows	Standard_HD_Transcode
Job name	Orchestrator_Transcode_001
Status Polling frequency	5
Restart Vantage Job if step restart?	checked
Fail if Vantage job cannot restart	unchecked

10.28.4 Saved Parameters Description

Parameter	Description
Comments	The comments of the Vantage Transcoding template. Note that the comments field can be used to provide instruction on which category the action belongs to as an override.
Vantage Server Node	The remote node name of the server which hosts the Vantage Transcoding server.
Vantage Server IP address (or hostname)	The remote host name or remote IP of the server which hosts the Vantage Transcoding server.
Vantage API port	The port number of remote server which hosts the Vantage Transcoding server. Default is 8676.

Parameter	Description
Source file	The absolute path to the file on the server's file system.
API Method	This Vantage plugin supports four API methods: SubmitFile, SubmitItems, Fetch Variables & Items, and Fetch Vantage Workflows.
Vantage workflows	A dropdown of Vantage workflows available on Vantage server to transcode files.
Workflow Id (hidden)	ID of the Vantage workflow selected, populated when a workflow is selected from dropdown.
Workflow name (hidden)	Name of the Vantage workflow selected, populated when a workflow is selected from dropdown.
Vantage Variables	Using the plugin, users can submit values to variables used in Vantage workflow.
Vantage Items	Using the plugin, users can submit values to items used in Vantage workflow.
Job name	The name for the transcode job. Defaults to orchestrator generated string.
Restart Vantage Job if step restart?	Usually if the orchestrator step is re-executed (because of a retry on failure or manually) a new job is created in Vantage. If this option is selected, Orchestrator will try to restart the previously defined job in Vantage.
Fail if Vantage job cannot restart	If the Restart Vantage Job option is selected, Vantage might still fail to restart a job. By default in this case the plugin will try to create a new job. If this option is selected, the step will fail if restarting does not work.
Status Polling frequency	How often progress will be reported. Default is 5 seconds.

10.28.5 Inputs Description

All parameters are available as inputs at runtime if not specified by the template. This plugin does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

10.28.6 Mandatory Inputs for SubmitFile Method:

- Vantage Server Node or Vantage IP Address: Either a remote node must be selected or a remote IP address must be provided.
- Workflow Id: A workflow ID is mandatory while submitting a job, hence a workflow must be selected from dropdown.
- Source file: Absolute path to the source file on remote server must be provided.

10.28.7 Mandatory Inputs for SubmitItems Method:

- Vantage Server Node or Vantage IP Address: Either a remote node must be selected or a remote IP address must be provided.
- Workflow Id: A workflow ID is mandatory while submitting a job, hence a workflow must be selected from dropdown.
- Vantage Variables: Provide values to the Vantage variables and Orchestrator will submit these to Vantage.
- Vantage Items: Provide values to the Vantage items and Orchestrator will submit these to Vantage. The file to transcode can be one of the items.

10.28.8 Optional Inputs:

- Name: It is advisable that a name is entered to identify the template.
- Comments: Information about the template which will help identify the template.
- Vantage API Port: The port is optional as a default value is used if not specified.
- Job Name: The name for the transcode job. Defaults to orchestrator generated string.
- Status Polling frequency: Specify if progress needs to be reported within a time other than default.

10.28.9 Outputs Description

- Input_file: The input file path involved in the transcode operation.
- Output_file(s): The output file(s) obtained as a result of the transcode operation.
- Job_Id: The ID of the job triggered by the transcode operation.
- Job_Name: The name of the job triggered by the transcode operation.

- Errors: Errors reported by transcode server during transcode operation.
- Vantage_Steps_Info: Details about the Vantage sessions/steps that were executed in Vantage workflow.
- Vantage_Items: Items part of the Vantage workflow.
- Vantage_Variables: Variables part of the Vantage workflow.

10.28.10 Supported API Methods

- SubmitFile - For single file transcoding operations
- SubmitItems - For multi-file processing with variables and items
- Fetch Variables & Items - To retrieve workflow-specific parameters
- Fetch Vantage Workflows - To retrieve available workflows from the Vantage server

10.28.11 Operating Instructions

- Basic Transcoding with SubmitFile: Provide remote node or server IP (e.g., 10.0.103.85), click 'Get Vantage Workflows' to retrieve available workflows, select a workflow, enter source file path (e.g., 'C:\InputFiles\Wildlife.wmv'), and submit. The workflow ID and source file path are sent to Vantage server for processing.
- Advanced Transcoding with SubmitItems: Follow the same initial setup, but instead of specifying a single source file, provide values for workflow variables and items. Multiple items and variables can be submitted, and the file to transcode can be one of the items.
- Runtime Configuration: For maximum flexibility, you can provide Vantage server details, workflow ID/name, variables, and items as runtime parameters. Variables should be in JSON format (e.g., { ”Network(String)”: ”Aspera”, ”Video Height(Int32)”: ”480” }), and items should follow similar JSON structure.
- Variable and Item Format: When providing variables and items in JSON format, include both the variable name and class type as shown in the examples. This ensures proper parameter mapping in the Vantage workflow.
- Job Management: The plugin supports job restart functionality when Orchestrator steps are re-executed. Configure the restart options based on your workflow requirements - you can either restart existing jobs or create new ones.
- Category Override: For reporting purposes, you can instruct the plugin to report itself as being from a different category by adding 'category: [The custom category]' to the comments field (e.g., 'category: Quality Control').
- Monitoring: Configure the status polling frequency to balance between responsiveness and system load. The default 5-second interval works well for most scenarios.

10.28.12 Step-by-Step Examples

10.28.13 Example 1: Transcoding with SubmitFile API Method

A single source file can be specified. Target can be specified during the workflow setup on the Vantage server.

1. Provide remote node or remote server IP and port (example: 10.0.103.85 or transcode.lala.com)
2. Click the button Get Vantage Workflows. This provides a dropdown with workflows from the provided Vantage server
3. Select a workflow from the dropdown
4. Beside each Variable and Item a text box is provided to enter values. Ignore these and enter source file
5. Provide source file name (Ex: ”C:\InputFiles\Wildlife.wmv”)
6. The workflow ID is submitted along with source file path to Vantage server
7. Vantage server takes these inputs and performs the transcode and generates an output file

10.28.14 Example 2: Transcoding with SubmitItems API Method

Multiple items and variables can be submitted. Target can be specified during the workflow setup on the Vantage server.

1. Provide remote node or remote server IP and port (example: 10.0.103.85 or transcode.lala.com)
2. Click the button Get Vantage Workflows. This provides a dropdown with workflows from the provided Vantage server
3. Select a workflow from the dropdown
4. Upon each selection of a Vantage workflow, its variables and items will be displayed if they exist
5. Beside each Variable and Item a text box is provided to enter values
6. Provide valid values in the text boxes for variables and items
7. The workflow ID is submitted along with items and variables to Vantage server
8. Vantage server takes these inputs and performs the transcode and generates an output file

10.28.15 Example 3: Transcoding with Runtime Parameters (SubmitItems API Method)

All inputs provided at runtime for maximum flexibility.

1. Provide Vantage server name or Vantage server node as a runtime parameter
2. Provide workflow ID or workflow name as a runtime parameter
3. Provide Vantage variables in JSON format as a runtime parameter
4. Example Vantage variables in JSON string: { ”Network(String)”: ”Aspera”, ”Video Height(Int32)”: ”480”, ”Video Width(Int32)”: ”720” }
5. Provide Vantage items in JSON format as a runtime parameter
6. Example Vantage items in JSON string: { ”Metadata(Store)”: ”//server-name/foldername/filename.xml”, ”Original(Media)”: ”//server-name/foldername/filename.mov” }
7. Note that the variable name and the variable class are needed in the format shown
8. Save and publish the workflow
9. When a work order is created, the workflow ID is submitted along with items and variables to Vantage server
10. Vantage server takes these inputs and performs the transcode and generates an output file

10.28.16 JSON Format Examples

10.28.17 Vantage Variables Example:

\{\ ”Network(String)”: ”Aspera”,\ ”Video Height(Int32)”: ”480”,\ ”Video Width(Int32)”: ”720”\ }\

10.28.18 Vantage Items Example:

\{\ ”Metadata(Store)”: ”//server-name/foldername/filename.xml”,\ ”Original(Media)”: ”//server-name/foldername/filename.mov”\ }\

10.28.19 Dependencies

- Network connectivity to the Vantage Transcoding server on the specified IP address and port.
- Valid authentication and permissions to access the Vantage server and submit transcoding jobs.
- Sufficient file system permissions to access source files on the Vantage server.
- Available Vantage workflows configured on the server for the intended transcoding operations.
- Adequate storage space on the Vantage server for both input and output files.

10.28.20 Troubleshooting

- If connection to Vantage server fails, verify the IP address, port number, and network connectivity. The default port is 8676.
- If workflows don't appear in the dropdown, ensure the Vantage server is running and accessible, and click 'Get Vantage Workflows' to refresh the list.
- For job submission failures, check that the source file path is correct and accessible from the Vantage server, and that the selected workflow is compatible with the input file format.
- If job restart functionality isn't working, verify that the original job still exists in Vantage and hasn't been manually removed or expired.
- For JSON format errors with variables/items, ensure proper syntax with variable names and class types exactly as they appear in the Vantage workflow configuration.
- If transcoding jobs fail, check the Vantage server logs and ensure sufficient disk space is available for both input processing and output generation.

10.28.21 Important Note - Custom Category Override

- In some cases, a Vantage workflow can be used to do something other than just transcoding. If for reporting reasons it is important to reflect what the step truly does, it is possible to instruct the plugin to report itself as being from a different category.
- This is done by adding to the comments field of the action configuration: ”category: [The custom category]”
- For example: category: Quality Control

10.29 VqmaAnalyzer



Figure 10.29: VqmaAnalyzer icon

This action plug-in provides the ability to submit and control a file transcoding operation using the HandBrake toolset.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.3	Adding Remote Node Dependency
0.0.2	Fixing the description
0.0.1	initial release

10.29.1 Saved Parameters Description

- This action allows to execute an VQMA task on a remote Windows node.
- Name: The name used to identify a saved VQMAAnalyzer configured instance.
- Comments: Comments associated to this template or configuration.
- Execution node: The remote node to execute the HandBrake operation from. List of already configured Remote Nodes.
- VQMA Binary path: The path (as seen in the SSH session) of the VQMA executable.
- Input file path: The path (as seen in the Windows Terminal) of the input file to VQMA.
- Output file path: The path (as seen in the Windows Terminal) of the output file of VQMA.(Preferred txt or csv files)
- Horizonal Frame Size: The hsize of the input file supplied
- Vertical Frame Size: The vsize of the input file supplied
- File Format: The format of the file(uyvy or yuv2)
- Command options (after destination filename): Any VQMA CLI option or parameter. This string will be added to the VQMA command at the end of the output file path

10.29.2 Inputs description

- Input file path and the Execution node, VQMA binary path are mandatory parameters for the execution of this plugin.

  Output file path if not provided, orchestrator would default it to the input filepath and name appended with a unique identifier(state_id).

  hSize and vSize though not mandatory, VQMA advices to provide them for better results.

  All of them support both design time and runtime specification.

  The run time inputs of this action depend on how the configuration is done. Variables exposed in the command line will be exposed as mandatory inputs.

10.29.3 Outputs description

- This action executes an VQMA Analysis on a remote node, via SSH.
- *out_target_file*: The path of the output file
- *out_report*: A report of the VQMA Analysis operation

10.29.4 Supported Actions

None

10.29.5 Dependencies

None

10.29.6 Operating Instructions

This action plug-in provides the ability to perform a Analysis using the VQMA toolset.

VQMA will be executed on a remote Windows node via an SSH connection.

NOTE: VQMA Executable for command line execution should be installed on the remote node.

The transformation result file will be stored on the specified path in the remote node.

Note: VQMA requires file to be specified in the windows path format. (Example C:\Documents\sample.txt). Follow that while giving the file path.

Note: Output file is not mandatory. If not provided, orchestrator creates a file in the input_path with the same as input_file_name appended with a uniqueId with a _PASS or _FAIL at end depending on the execution status.

Note: Even if the Output File is provided, VQMA modifies the file name with the execution status.(Example If output path is ”results.txt” it would give the output back as results_PASS.txt or results_FAIL.txt based on the execution

Note: Execution node is not included below since the examples are all run on a SSH client.

Note: .INI file should be present in the input path. If not, it would consider the default .INI present in the VQMA Binary Path.

Example

CONFIGURATION:

Input file path : ”C:\Users\Administrator\Desktop\VCMQ\current.yuv”

Output file path : ”C:\Users\Administrator\Desktop\VCMQ\current_execution.txt”

VQMA binary path : ”/cygdrive/c/Users/Administrator/Desktop/VCMQ/videoexec/VQMA.EXE”

OUTPUTS:

Shows the progress report along with the status of the transcoding

OutPut Target File : ”C:\Users\Administrator\Desktop\VCMQ\current_execution_PASS.txt”

10.30 Xf converter



Figure 10.30: Xf converter icon

This plug-in provides the ability to convert (rewrap) audio and video wrapper files (AVI, GXF, MOV, MP4 and MXF) with OpenCube XFConverter. It implements the XFConverter SOAP based WEB service API 2.0

Version	Comment
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.4	Added styling to parameter help text.
0.2.3	Adding Remote Node Dependency
0.2.2	allow profile to point to a memory content rather than a file path if 'init from type' is buffer
0.2.1	fix server address get
0.2.0	add overwrite flag
0.1.0	change category to transcoding and add init_from_type
0.0.1	initial release

10.30.1 Saved Parameters Description

- Name: The name used to identify a saved XfConverter configured instance.
- Comments: Some comments about this saved XfConverter configured instance.
- Remote Node: The XfConverter node name from the list of Remote Nodes.
- Remote server ip address: The XfConverter node IP address or domain name.
- Remote web service port : To change the XfConverter WEB service port (1111 by default).
- Profile: A file path or a memory buffer to an XFConverter project.
- Init from type: The API method used to load the profile: init_from_file or init_from_buffer.
- Source: The file path to the audio/video wrapper source.
- Destination : The file path to the converted destination.
- Overwrite?: To overwrite the destination file if already existing.
- Polling frequency: Interval in seconds to check for the conversion task status.

10.30.2 Inputs description

The list of inputs depends on the configuration of the XfConverter action template.

Changes in this list of inputs will override any value set in the action configuration/template.

- *remote node or server address*: XfConverter node name or node IP address or domain name
- *profile*: A file path or a buffer to an XFConverter project
- *source*: The file path to the audio/video wrapper source.
- *destination*: The file path to the converted destination.

10.30.3 Outputs description

- *XML_result*: The XfConverter parsed XML output
- *Xf_status_message*: The XfConverter status code

10.30.4 Supported Actions

None

10.30.5 Dependencies

None

10.30.6 Operating Instructions

Specify a remote node or an IP address/domain name for the XfConverter server. This server must run the SOAP WEB service (typically xfc_gsoap_stl_server.exe).

Specify a conversion profile (full file path including the file name with .xfg extension or memory buffer with same content as the .xfg file).

Specify if the API method to load the profile is init_from_file (in this case, provide in ”Profile” the file path to the xfg profile on the XfConverter server) or init_from_buffer (in this case, provide in ”Profile” a memory buffer to the same profile content).

Specify a source wrapper file (full file path including the file name).

Specify the destination file (full file path including the file name).

In output, you'll get the XfConverter status code and the parsed XML output of the get task status request upon success or the output of the first failed request upon error (e.g. the XML output of the conversion request).

Most common errors:

When a profile or source file path is not found, the error is XFRESULT_FILE_DOES_NOT_EXIST (code 20).

When a destination directory path (under the filename) is not found, the error is XFRESULT_ERROR (code 1).

When the destination file name is already existing, the error is XFRESULT_FILE_ALREADY_EXISTS (code 21).

The full list of error codes is exposed on the XfConverter server at:

C:\Program Files (x86)\EVS Broadcast Equipment\OpenCube XFConverter\inc\xfconverter_errorcodes.h

Orchestrator can dynamically build a profile from an XML template. The template file has to be in the /opt/aspera/orchestrator/config/orchestrator/actions_config/xf_converters/ folder. A workflow has been implemented for this purpose, generating a profile based on an XML template and a number of parameters (e.g. number of audio tracks). The workflow is converting the source file based on the generated profile.

10.31 Zencoder transcoding



Figure 10.31: Zencoder transcoding icon

This action plugin provides the ability to submit file transformation jobs to the Zencoder cloud service.

Version	Comment
0.3.2	Enhanced action template UI
0.3.1	Fix issue in saving preset parameters
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.4	Added styling to parameter help text.
0.2.3	Added Help file
0.2.2	2015-02-24 ML - New icon
0.2.1	avoids issue with MultiJson initialization
0.2.0	adds region setting
0.0.2	fixed progress reported with very long precision 50.7439824798374 %
0.0.1	initial development, works if outputs is left blank. Not tested with custom outputs

10.31.1 How It Works

- Integrates with Zencoder cloud-based video encoding service to perform video and audio transcoding operations.
- Supports multiple output profiles and preset configurations for different transcoding requirements and quality settings.
- Provides flexible output configuration through array-based hash structures for multiple simultaneous transcoding jobs.

10.31.2 Configuration Instructions

1. Add a Zencoder Transcoding step to your workflow.
2. Configure Zencoder service connection:
    - API key: Enter your Zencoder service access key for authentication.
    - Input uri: Provide the URL to the source media file for transcoding.
    - Region: Specify region if required for service access optimization.
3. Set up transcoding parameters:
    - OutputProfile: Select the appropriate profile from the populated list.
    - Outputs: Configure output specifications as array of hash structures.
    - Preset Parameters: Set profile-specific parameters for transcoding operation.

10.31.3 Example Configuration

Parameter	Example Value
API key	your-zencoder-api-key
Input uri	https://example.com/input-video.mp4
Region	us-east-1
OutputProfile	HD_Standard_Profile
Outputs	[{ label: 'HD Output', url: 'https://example.com/output-hd.mp4' }]
Preset Parameters	Profile-specific parameters

10.31.4 Saved Parameters Description

Parameter	Description
API key	The access key to reach the Zencoder service for authentication and authorization.
Input uri	The URL to the Zencoder server or source media file location for transcoding input.
Region	If a region needs to be specified to access the service, specify it here for optimal performance and compliance.
OutputProfile	Choose from the list of profiles populated the appropriate profile for calling the Zencoder service.
Outputs	Outputs are described as an array of hash structures. The default format is: [{ label: 'Default transformation', url: '<%= Output_URI>' }].
Preset Parameters	Based on the Output Profile chosen, this lists the parameters needed to execute the transcoding operation.

10.31.5 Inputs Description

Mandatory Fields:

- API Key: Required access key for authenticating with the Zencoder service.
- Input URI: Required URL pointing to the source media file for transcoding.
- Output_URI: Required destination URL or path for the transcoded output files.

Optional Runtime Inputs:

- region: Geographic region specification for service optimization.
- output_profile: Selected transcoding profile for processing parameters.
- preset_parameters: Custom parameters specific to the chosen output profile.

10.31.6 Outputs Description

- Output URIs: Stores the list of output URIs pointing to the transcoded media files generated by the Zencoder service.

10.31.7 Supported Actions

- Transcode media files using Zencoder cloud service
- Monitor transcoding job status and progress
- Retrieve transcoded media file URLs and metadata

10.31.8 Operating Instructions

- Service Setup: Obtain a valid API key from Zencoder and ensure you have access to the transcoding service in your desired region.
- Input Configuration: Specify the input URI pointing to your source media file. The file should be accessible via HTTP/HTTPS or stored in cloud storage that Zencoder can access.
- Profile Selection: Choose an appropriate output profile based on your transcoding requirements. Different profiles offer various quality settings, formats, and encoding parameters.
- Output Configuration: Configure the outputs array with hash structures specifying labels and destination URLs for each transcoded version you want to generate.
- Parameter Customization: Set preset parameters based on the selected output profile. These parameters control specific aspects of the transcoding process such as bitrate, resolution, and codec settings.

10.31.9 Dependencies

- Valid Zencoder account with active API access and sufficient transcoding credits.
- Appropriate permissions and access rights for both source and destination media locations.

10.31.10 Troubleshooting

- If authentication fails, verify that your API key is correct and that your Zencoder account has active transcoding credits.
- For input file access issues, ensure the source media file is publicly accessible via the provided URI and that the file format is supported by Zencoder.
- If transcoding jobs fail, check the selected output profile compatibility with your input media format and verify all preset parameters are valid.
- If output URIs are not generated, verify that the destination storage locations are properly configured and accessible for writing transcoded files.
- For parameter configuration errors, review the output profile documentation to ensure all required preset parameters are correctly specified.

Chapter 11

Triggers

11.1 Amazon SQS trigger



Figure 11.1: Amazon SQS trigger icon

This action plug-in provides the ability to watch for messages out of an Amazon SQS queue.

Version	Comment
0.4.4	Enhanced action template UI
0.4.3	Fix issue when loading help page
0.4.2	Updated the label from Keep ongoing to Continue Monitoring
0.4.1	Add multi-threaded message retrieval
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.0	Add assume role
0.2.2	Fix for secret masking
0.2.1	Expose polling frequency as run time input
0.2.0	Add http proxy support
0.1.1	Avoid autofill of key and secret
0.1.0	Adding WaitTimeSeconds Parameter.
0.0.3	added cancel/pause/resume at step level
0.0.2	added date/time reporting
0.0.1	first tracked version

11.1.1 How It Works

- Watches messages on a specified Amazon SQS queue.
- Messages are deleted from the queue once fetched.
- Requires:
  - AWS access key and secret
  - AWS region
  - Queue URL

- Click Continue Monitoring to continuously watch for new messages.
- Set polling frequency for checking the queue (default: 5 seconds).
- Maximum number of messages returned per workflow instance:
  - Default: 1
  - Valid range: 1 to 10
- If a get message** or delete operation fails:**
  - The watcher stops.
  - Prevents repeated triggering on the same message.

11.1.2 Configuration Instructions

1. Create a new action step using the AmazonSqsTrigger plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved Amazon SQS Trigger configured instance.
   - Comments: Comments about this saved Amazon SQS Trigger configured instance.
   - Amazon Access Key: The access key ID credential provided by Amazon.
   - Amazon Secret Key: The secret access key credential provided by Amazon.
   - Region: The region where SQS is hosted (e.g., us-east-2 for US East Ohio).
   - Verify SSL?: Check this box to validate the server SSL certificate.
   - External ID: External ID when assuming a role (optional).
   - Policy: Identity and Access Management (IAM) policy in JSON format when assuming a role (optional).
   - Role ARN: Amazon Resource Name (ARN) of the role. Required to assume a role.
   - Token Duration: Duration of the generated token when assuming a role, in seconds. Default is 3600s (1 hour).
   - Queue URL: The queue URL (e.g., https://sqs.us-east-2.amazonaws.com/921583127875/orders).
   - Continue Monitoring: If checked, the containing workflow will be cloned after each trigger event, ensuring that the workflow executes every time messages are placed in the queue.
   - Polling Frequency: By default, the queue is checked every 5 seconds for new messages. This polling frequency can be changed to any number of seconds.
   - Max Number of Messages: The maximum number of messages to fetch from the queue (default is 1).
   - Proxy URL: The URL of the proxy server through which AWS will be accessed. Leave this field blank if you do not wish to use a proxy server.

11.1.3 Example Configuration

This is a minimum setup configuration for the AmazonSqsTrigger plugin.

Parameter	Value
Amazon Access Key	access-key
Amazon Secret Key	secret-key
Region	us-east-2
Queue URL	https://sqs.us-east-2.amazonaws.com/921583127875/orders
Token Duration	7200
Continue Monitoring	Selected
Max Number of Messages	10

11.1.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved Amazon SQS Trigger configured instance.
Comments	Comments about this saved Amazon SQS Trigger configured instance.
Amazon Access Key	The access key ID credential provided by Amazon.
Amazon Secret Key	The secret access key credential provided by Amazon.
Region	The region where SQS is hosted (e.g., us-east-2 for US East Ohio).
Verify SSL?	Check this box to validate the server SSL certificate.
External ID	External ID when assuming a role (optional).

Parameter	Description
Policy	Identity and Access Management (IAM) policy in JSON format when assuming a role (optional).
Role ARN	Amazon Resource Name (ARN) of the role. Required to assume a role.
Token Duration	Duration of the generated token when assuming a role, in seconds. Default is 3600s (1 hour).
Queue URL	The queue URL (e.g., https://sqs.us-east-2.amazonaws.com/921583127875/orders).
Continue Monitoring	If checked, the containing workflow will be cloned after each trigger event, ensuring that the workflow executes every time messages are placed in the queue.
Polling Frequency	By default, the queue is checked every 5 seconds for new messages. This polling frequency can be changed to any number of seconds.
Max Number of Messages	The maximum number of messages to fetch from the queue (default is 1).
Proxy URL	The URL of the proxy server through which AWS will be accessed. Leave this field blank if you do not wish to use a proxy server.

11.1.5 Inputs Description

The list of inputs depends on the configuration of the AmazonSqsTrigger action template. If dynamic attributes are used in the message definition, each attribute becomes a required input.

Parameter	Description
*Aws_key **	The access key ID credential provided by Amazon.
*Aws_secret **	The secret access key credential provided by Amazon.
*Aws_region **	The region where SQS is hosted (e.g., us-east-2 for US East Ohio).
*Queue_url **	The queue URL (e.g., https://sqs.us-east-2.amazonaws.com/921583127875/orders).
Aws_session_token	The access session token credential provided by Amazon.
External_id	External ID when assuming a role (optional).
Max_number_of_messages	The maximum number of messages to fetch from the queue (default is 1).
Policy	Identity and Access Management (IAM) policy in JSON format when assuming a role (optional).
Polling_frequency	By default, the queue is checked every 5 seconds for new messages. This polling frequency can be changed to any number of seconds.
Proxy_url	The URL of the proxy server through which AWS will be accessed. Leave this field blank if you do not wish to use a proxy server.
Role_arn	Amazon Resource Name (ARN) of the role. Required to assume a role.
Token_duration	Duration of the generated token when assuming a role, in seconds. Default is 3600s (1 hour).

11.1.6 Outputs Description

Parameter	Description
Response	Response received from Amazon
Messages_body	The array of message bodies
Error_Message	The error message received
Temporary_access_key	Temporary access key generated if assume role is called
Temporary_secret	Temporary secret generated if assume role is called
Temporary_token	Temporary token generated if assume role is called

11.1.7 Supported Actions

None

11.1.8 Dependencies

None

11.1.9 Operating Instructions

This plugin provides the ability to watch messages on a specified Amazon SQS queue.

Template Configuration

1. Navigate to the plugin configuration page.
2. Enter your AWS access key, secret key, and select the correct AWS region.
3. Provide the SQS queue URL you want to monitor.
4. Set the polling frequency (default is 5 seconds).
5. Specify the maximum number of messages to fetch per cycle (valid range: 1-10).
6. Enable Continue Monitoring if you want continuous monitoring.
7. Save the configuration and start the watcher.
8. Monitor logs for any get message or delete operation failures, as these will stop the watcher.

For an IAM user to assume a role:

Pre-requisites:

- The assume role operation is called only if a role ARN is specified.
- The IAM user must be allowed as a principal in the role trust relationship (`sts:AssumeRole` action).

Instructions:

- The IAM user must first be identified with its access key, secret, and region.
- Specify the external ID if defined in the role trust relationship.
- Provide the IAM policy in JSON format without backslashes. For example:

  {\ ”Version”: ”2012-10-17”,\ ”Statement”: [\ {\ ”Sid”: ”Stmt1”,\ ”Effect”: ”Allow”,\ ”Action”: ”SQS:*”,\ ”Resource”: ”*”\ }\ ]\ }

  This policy will apply to all operations performed using the generated temporary credentials.

- Specify the role ARN. For example:

  `arn:aws:iam::921583127875:role/sqs_access`

- Temporary credentials (access key, secret, token) are generated, used, and returned if a role is specified.
- Specify the token duration in seconds:
  - Must be greater than 900 seconds (15 minutes).
  - Must be less than the role's Maximum session duration (check in AWS Console).
  - The maximum session duration can be set to 12 hours.

11.2 AMQP trigger

This action plug-in provides the ability to wait for messages in an AMQP queue.

Version	Comment
0.9.2	Enhanced action template UI
0.9.1	Updated the label from Keep ongoing to Continue Monitoring
0.9.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.8.7	Improves new plugin version format.
0.8.6	Replaces 'Message_body' with 'Message_headers' and 'Messages_bodies' with 'Messages_headers' as outputs names.

Version	Comment
0.8.5	Fix when several items are returned
0.8.4	Exposing the entire messages as outputs
0.8.3	Fix when to send Ack in >8.0 Amqp protocol
0.8.2	Exposes tls options used ONLY for RabbitMQ > 3
0.8.1	Changes in connection attributes
0.8.0	Changed routing key to be string
0.7.1	Additional UI changes in Not Durable??
0.7.0	Adding extra connection attributes
0.6.4	Fix for frozen string error
0.6.3	Patch to support RabbitMQ version 3.7.3
0.6.2	Patches carrot gem
0.6.1	Catches unresponsive server error as it seems to be transient
0.6.0	Adds parameterization of connection timeout
0.5.4	Changed default timeout
0.5.3	Ensures connection is stopped
0.5.2	Fixes possible variable allocation issue
0.5.1	Adding RemoteNode Dependency.
0.5.0	Added support for AMQP versions higher than 0.8.0
0.4.2	Fixed lost of message from the queue when queue has more elements than what requested
0.4.1	Fixed queue ack when retrieving multiple messages
0.4.0	Added Maximum number of items to return per cycle
0.3.0	Added support for Routing Key
0.2.9	Added vhost to dynamic inputs
0.2.8	New icon
0.2.7	Fixes reference to vhost variable
0.2.6	exception handling of RemoteNode.identify()
0.2.5	Prevents autocomplete of user
0.2.4	Fixes issue when exchange are not available
0.2.3	Fixes typo in dynamic node retrieval
0.2.2	fix - edit.html file has a typo which make username and password fields default to 0
0.2.1	adds help file
0.2.0	adds ack vs no_ack configuration switch
0.1.0	adds support for vhost
0.0.2	renamed output - breaks compatibility with 0.0.1
0.0.1	first tracked version

11.2.1 How It Works

- The plugin waits until a message is found in a specified AMQP queue.
- An AMQP server (such as RabbitMQ or Apache Qpid) must be set up on a server that Orchestrator can reach on the specified port.
- Since messages are text-based, structured messages (XML, JSON, or custom formats) can be processed as long as the parsing logic is defined in the Outputs formatting code field.
- When parsing a message:
  - The raw message is available in a variable named `message`>.
  - The parsing result must be assigned to the output variables defined in the Formatted outputs field.

11.2.2 Configuration Instructions

1. Create a new action step using the AmqpTrigger plugin.
2. The plugin action waits until a message is found in a specified AMQP queue.
3. Fill out the following fields:
   - Name: The name used to identify a saved AmqpTrigger configured instance.
   - AMQP queue name: The name identifying the queue on the AMQP server. If the queue does not exist, it will be created when the first message is inserted.
   - AMQP Exchange name: The name defining the exchange on the AMQP server.
   - AMQP Version > 0.8.0 ?: Checks whether the AMQP server version is greater than 0.8.0.



Figure 11.2: AMQP trigger icon

- AMQP Exchange type: The type of the exchange on the AMQP server. Defaults to Direct, but you may choose from Direct, Topic, Fanout, or Headers.
- Queue connection arguments - AMQP server node: If the server where AMQP runs was defined as a remote node, it can be selected here. Alternatively, its address can be provided using the fields below.
- Queue connection arguments - Queue host: The host name, if a node was not provided.
- Queue connection arguments - Queue vhost: Specify a virtual host if one is used for that queue on the AMQP server.
- Queue connection arguments - Queue port: A specific port can be specified. If not specified, the default port 15672 will be used.
- Queue connection arguments - Queue user: A specific user can be specified if the default guest account is not used.
- Queue connection arguments - Queue password: The password for the user, if the default is not used.
- Queue connection arguments - Use SSL?: Indicates whether SSL is to be used to communicate with the AMQP server.
- Consume without ACK?: By default, an ACK is required when the message is unqueued. Checking this option will remove that requirement.
- Polling frequency: By default, the queue is probed every 5 seconds for new messages. This polling frequency can be changed to any number of seconds.
- Routing key: (Optional) Used to support direct message exchange.
- Not Durable?: Used to indicate whether durability (i.e., whether the queue will survive a broker restart) is required. By default, it is set to false.
- X-max-priority: Supports specifying the maximum number of priorities.
- X-message-ttl: Supports per-queue Message Time-to-Live (TTL).
- X-expires: Specifies how long the queue may remain unused, in milliseconds.
- Maximum number of items to return when triggering: The number of items in the queue to return during each cycle (if available). The default is 1.
- Advanced outputs - Formatted outputs: A list of additional outputs can be specified. Note that these outputs will be set to `nil` unless code is provided in the Outputs formatting code text area to set their values.
- Advanced outputs - Outputs formatting code: Ruby code that is executed to extract values from the `message` variable and assign them to the outputs specified in Formatted outputs.

11.2.3 Example Configuration

This is a minimum setup configuration for the AmqpTrigger plugin.

Parameter	Value
AMQP queue name	amqp_home
Continue Monitoring	Selected
AMQP Version > 0.8.0 ?	Selected
Queue connection arguments - AMQP server node	AMQP node
Queue connection arguments - Queue host	localhost
Queue connection arguments - Queue port	5672
Queue connection arguments - Queue user	root
Queue connection arguments - Queue password	password
Polling frequency	5

Parameter	Value

11.2.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved AmqpTrigger configured instance.
AMQP queue name	The name identifying the queue on the AMQP server. If the queue does not exist, it will be created when the first message is inserted.
AMQP Exchange name	The name defining the exchange on the AMQP server.
AMQP Version > 0.8.0 ?	Checks whether the AMQP server version is greater than 0.8.0.
AMQP Exchange type	The type of the exchange on the AMQP server. Defaults to Direct, but you may choose from Direct, Topic, Fanout, or Headers.
Queue connection arguments - AMQP server node	If the server where AMQP runs was defined as a remote node, it can be selected here. Alternatively, its address can be provided using the fields below.
Queue connection arguments - Queue host	The host name, if a node was not provided.
Queue connection arguments - Queue vhost	Specify a virtual host if one is used for that queue on the AMQP server.
Queue connection arguments - Queue port	A specific port can be specified. If not specified, the default port 15672 will be used.
Queue connection arguments - Queue user	A specific user can be specified if the default guest account is not used.
Queue connection arguments - Queue password	The password for the user, if the default is not used.
Queue connection arguments - Use SSL?	Indicates whether SSL is to be used to communicate with the AMQP server.
Consume without ACK?	By default, an ACK is required when the message is unqueued. Checking this option will remove that requirement.
Polling frequency	By default, the queue is probed every 5 seconds for new messages. This polling frequency can be changed to any number of seconds.
Routing key	(Optional) Used to support direct message exchange.
Not Durable?	Used to indicate whether durability (i.e., whether the queue will survive a broker restart) is required. By default, it is set to false.
X-max-priority	Supports specifying the maximum number of priorities.
X-message-ttl	Supports per-queue Message Time-to-Live (TTL).
X-expires	Specifies how long the queue may remain unused, in milliseconds.

Parameter	Description
Maximum number of items to return when triggering	The number of items in the queue to return during each cycle (if available). The default is 1.
Advanced outputs - Formatted outputs	A list of additional outputs can be specified. Note that these outputs will be set to `nil` unless code is provided in the Outputs formatting code text area to set their values.
Advanced outputs - Outputs formatting code	Ruby code that is executed to extract values from the `message` variable and assign them to the outputs specified in Formatted outputs.

11.2.5 Inputs Description

The list of inputs depends on the configuration of the AmqpTrigger action template. If dynamic attributes are used in the message definition, each attribute becomes a required input.

Parameter	Description
*Queue_name **	Mandatory - the name of the queue into which the message will be inserted.
*Message **	Mandatory - the message to insert into the specified queue.
Exchange_name	Optional - Orchestrator will attempt to send the message to a queue.
Queue_host	Optional - the address of the node where the queue is hosted.
Queue_node	Optional - the node where the queue is hosted.
Queue_password	Optional - the password for the user posting the message, if different from the default guest account.
Queue_port	Optional - the port used to access the queue.
Queue_user	Optional - the username used for posting the message, if different from the default guest account.
Queue_vhost	Optional - the virtual host for the queue, if one is used.
Routing_key	Used to support direct message exchange.
additional_queue_message_parameters	Enables configuration of additional queue, exchange, and message headers.
app_id	Optional application ID.
auto_delete	Indicates whether the exchange should be automatically deleted when it is no longer in use.
content_encoding	The encoding used for the message content (e.g., utf-8).
content_type	The message content type (e.g., application/json).
correlation_id	The identifier linking this message to another, such as the request it is replying to.
exclusive	Indicates whether this queue is exclusive (usable only by this connection and removed when the connection closes).
expiration	The time (in milliseconds) after which the message will be deleted.
headers	A Ruby hash of additional attributes required by the application. Nested hashes are supported.
mandatory	Indicates whether the message should be returned if it cannot be routed to any queue.
message_id	An identifier for the message.
priority	The priority level of the message.
reply_to	The queue name to which other applications should send the response.
timestamp	A time value represented as an integer.
type	The type of the exchange on the AMQP server. Defaults to Direct, but you may choose from Direct, Topic, Fanout, or Headers.
user_id	Optional user ID, verified by RabbitMQ against the connection username.
x_expires	Specifies how long the queue may remain unused, in milliseconds.

Parameter	Description
x_max_priority	Supports specifying the maximum number of priorities.
x_message_ttl	Supports per-queue Message Time-to-Live (TTL).

11.2.6 Outputs Description

Parameter	Description
Message_headers	The headers of the message pulled from the queue.
Message_payload	The message pulled from the queue.
Message_response	The response to the message pulled from the queue.
Messages_left_coun	The number of messages left in the queue at the time of probing (excluding the current one).
Queue_name	The actual queue name from which the message was pulled.

11.2.7 Supported Actions

None

11.2.8 Dependencies

None

11.2.9 Operating Instructions

This plugin allows Orchestrator to wait until a message is detected in a specified AMQP queue. To function properly, it requires an AMQP server—such as RabbitMQ or Apache Qpid—to be installed on a server that Orchestrator can access using the configured port.

Because all messages are text-based, the plugin can also handle structured formats such as XML, JSON, or any custom structure. To do this, the logic for interpreting and extracting data from the message must be provided in the Outputs formatting code field.

When a message is received, its raw content is made available in a variable named message. The parsing logic uses this variable to extract the desired values, which must then be assigned to the output variables defined in the Formatted outputs field.

For example, let's assume XML messages are received with a format like:

\<message>\ <asset>\ <id>1234H</id>\ <path>/DAM/HD/tron.mxf</path>\ </asset>\ <asset>\ <id>1234S</id>\ <path>/DAM/SD/tron.mpg</path>\ </asset>\ </message>\

If an formatted output 'assets' of type 'Hash' is selected (ID->path), the code in Outputs formatting code will look like:

\<message>\ <asset>\ <id>1234H</id>\ <path>/DAM/HD/tron.mxf</path>\ </asset>\ <asset>\ <id>1234S</id>\ <path>/DAM/SD/tron.mpg</path>\ </asset>\ </message>\

Here the resulting hash found in the output 'assets' would be:

{ '1234H' => '/DAM/HD/tron.mxf', '1234S' => '/DAM/SD/tron.mpg' }

The pulling of the messages can be done either with ACK (default) or without.

11.3 Aspera central watcher

This action plugin provides the ability to sense the initiation or completion of transfers on an Aspera server.



Figure 11.3: Aspera central watcher icon

Version	Comment
2.8.2	Enhanced action template UI
2.8.1	Updated the label from Keep ongoing to Continue Monitoring
2.8.0	Updating to be compatible with Ruby 3.4 and Rails 8
2.7.3	Returns files status from a triggered transfer session
2.7.2	Add node user and node password as optional inputs
2.7.1	Use central address if a remote monitored node is not defined
2.7.0	Add cookie filter
2.6.9	Avoids daemonization by setting to error instead of failed when central is unreachable
2.6.8	Added md5 and user to files_information
2.6.7	Fixing sessionID to submit as type array instead of string
2.6.6	Fixing wsdl file
2.6.5	Fix to support new rails converting string to array for soap calls.
2.6.4	Fix for new rails support
2.6.3	Session File Filter input accepts comma and semicolon delimeters
2.6.2	Added File sizes list output param for session based triggers
2.6.1	Added separate status constants for file status
2.6.0	Adding new field for file transfer status
2.5.7	Adding status support for files
2.5.6	Added styling to help text.
2.5.5	Adding RemoteNode Dependency.
2.5.4	Making filter params runtime.
2.5.3	Changing the order of filter session and file filter pattern.
2.5.2	Filter Out will now filter if the file doesn't match the pattern provided
2.5.1	Fixes race condition causing files to be ignored when reported as 0 file transfered too early
2.5.0	Added filtering based on the entire session instead of file based. If a single file in the session matches the given pattern, it will rule out the session entirely with this change
2.4.2	Added additional filters part to file based transfers and changed user filter logic for session based
2.4.1	Added MaxResults to getSessionFilesList
2.4.0	Adds support for authenticated request via Node
2.3.1	New icon
2.3.0	Added new fields for triggering on empty files and zero byte transfers. Changed code accordingly.
2.2.4	API parameter changed to TransferStatus for File Based Trigger
2.2.3	Layout Fix for Trigger Operation
2.2.2	Backward Compatibility fixed for output spec.
2.2.1	Backward Compatibility scenario updated. Reversed the check to default to session based.
2.2.0	Single file Transfer and multi file based transfer triggers added. (UNSTABLE VERSION)
2.1.0	Adds expected file count
2.0.9	Adds error message to outputs
2.0.8	Do not revert to old token for subsequent requests, fixes pause.
2.0.7	Adds timestamped status message when in progress
2.0.6	Fixes filtering by file name
2.0.5	Adds support for any (except running and paused)
2.0.4	Returns session start and end dates
2.0.3	Exposes Cookie as an output

Version	Comment
2.0.2	Changes default polling frequency to 5 and exposed in configurator with placedholder
2.0.1	Expose exception message in case the list of session cannot be retrieved
2.0.0	Simplified to use generic trigger persistence (requires Orchestrator 2.0)
1.5.4	Adds Server user list as an optional input as requested by Penn Boon (HBO) on 06-27-2013
1.5.3	Avoids looping if initialization fails
1.5.2	Fixed central node resolution when central node provided as standard input
1.5.1	Added MD5 checksum to output
1.5.0	Expose the central node as standard input and make service_uri always optional input
1.5.0	Added trigger type
1.4.3	Provides the option to specify the node at runtime
1.4.2	Change made to reference new plugin load directory
1.4.1	Bug fix in central node execution address retrieval
1.4.0	Use local caching of generated wsdl by default
1.3.2	Refined file name filtering rules to exclude only non-compliant files
1.3.1	Expose file filter as run-time parameter
1.3.0	Add node manager
1.2.0	Add filtering options (by receipient user, by regexp)
1.1.4	Fix bug preventing the remote_address to be exposed
1.1.3	Adds automatic clean up of temporary files from var run
1.1.2	Corrects missed in_progress sessions issue
1.1.1	Add the source path list as an output
1.1.0	Add option to ignore 0 bytes sessions
1.0.0	Add support for 'running' sessions

11.3.1 How It Works

- Monitors Aspera transfer sessions by polling Aspera Central web services on Enterprise Server, Connect Server, and Point-to-Point configurations.
- Supports multiple trigger types including session-based, file-based, and single-file triggering with configurable persistence scopes.
- Provides comprehensive filtering capabilities for transfer direction, server/client addresses, file paths, cookies, and session status.
- Integrates with Node API for authentication or connects directly to Aspera Central service using WSDL-based communication.
- Offers flexible trigger persistence management across workorders, workflows, templates, and steps to prevent duplicate processing.
- Supports both one-time execution and continuous monitoring modes with configurable polling frequencies for real-time transfer tracking.

11.3.2 Configuration Instructions

1. Add an Aspera Central Watcher step to your workflow.
2. Configure the monitored Aspera server:
   - Monitored Aspera node: Select from defined remote nodes, or
   - Aspera Central Address: Provide IP address and optionally port for direct connection.
   - Node user/password: Enter credentials if using Node API authentication.
3. Set up trigger behavior:
   - Trigger Types: Choose between Session, Files, or Single File triggering.
   - Session Status: Select which session events trigger the watcher (e.g., running, completed).
   - Continue Monitoring: Enable for continuous monitoring or disable for one-time execution.
4. Configure filtering options:
   - Transfer direction: Filter for incoming or outgoing transfers.
   - Target filepath pattern: Use regex to match specific file paths.
   - Server user list: Filter by recipient account names.

11.3.3 Example Configuration

Parameter	Example Value
Monitored Aspera node	aspera-server-01
Aspera Central Address	192.168.1.100:40001
Node user	aspera_user
Node password	**********
Trigger Types	Trigger On Session
Session Status	running
Transfer direction	incoming
Target filepath pattern	^/home/ingest_drop/.*
Continue Monitoring	checked
Polling frequency	0
Trigger persistence scope	By workorder group

11.3.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved AsperaCentralWatcher configured instance.
Comments	Some comments about this saved AsperaCentralWatcher configured instance.
Monitored Aspera node	Provides a selector to pick from defined remote nodes. If only a node is picked, the default central port (40001) will be used.
Aspera Central Address	Provides the IP address at which the Enterprise Server is located and optionally the port if different from the default (40001). This field is used to have Orchestrator generate the WSDL file for the specified central address and populate the 'Aspera Central Service WSDL Address' field. The central address field is optional, derived by default from the monitored node execution address if a node is specified.
Node user	Node API username for authentication when connecting through Node API.
Node password	Node API password for authentication when connecting through Node API.
Trigger On Empty Files	Check this option if you want to trigger on empty files (files with 0 bytes). Available only for file-based transfer monitoring.
Trigger On Zero Byte Transfers	Check this option if you want any zero byte transfers recorded (if you retransmit a file that is already present, check this option if you want to trigger on them). Available only for file-based transfer monitoring.
Aspera Central service wsdl address	Provides the location for the WSDL file for the Aspera Central Webservice being queried. As mentioned above in Central Address, it's by default generated automatically.
Continue Monitoring	States whether the watcher is run once only, or every time the event occurs.
Transfer server address	Enables filtering out transfers that do not use the specified address as the server. Note: this address must be provided the same way it appears in a transfer. For example if the transfer uses localhost, a filter on 127.0.0.1 would not work.
Transfer client address	Same as above but filters on the client address.
Server user list	Filters out all transfers sent to recipient accounts not in the list provided. Use comma, space or semi-colon as separators. e.g., 'root, admin' or 'bob bill doug'.
Target filepath pattern	Use regexp matching to only trigger when the pattern is matched by one of the files in the session. e.g., '^/home/ingest_drop/.*'.
Cookie pattern	Use regexp matching to only trigger when the pattern is matched by the session cookie. e.g., 'aspera.shares2.*'.
Transfer direction	Enables filtering out outgoing or incoming transfers.
Session Status	Specifies on which session event the watcher is triggered. For example: if 'running' is used the watcher triggers when a transfer is detected as being initiated.
Report transfer sessions with no files?	By default, transfers with no files are ignored, but checking this flag enables triggers even when no files are sent.

Parameter	Description
Session File filter	Applies the given filter(s) to all the files in the session. If at least one file matches the given filter, it will reject the entire session. (Available only for session-based trigger). Comma and semicolon can be used as delimiters in case of more than one filter.
Polling frequency	Frequency (in seconds) with which the AsperaCentral Web service is called to check for transfer session statuses. By default, a value of 0 is used (i.e., poll as often as possible).
Trigger persistence scope	Sets the scope/availability of the trigger output. The default value is 'By workorder group'. The definitions are: 'By workorder group', 'By action template', 'absolute', 'By step', 'By workflow', 'None'.
Trigger Types	Sets the event for Trigger Initiation. The default instantiation is on Session.

11.3.5 Trigger Persistence Scope Details

- 'By workorder group' - Trigger folders are tracked while workorder is executing and will only be trapped once. If the workorder is restarted, all past triggers will be detected again if they are still present.
- 'By workflow' - Same as 'By workorder group' except no triggering will occur upon a 'restart' action as the workflow is still the same.
- 'By action template' - Allows the tracking and thus non-triggering in the second workflow for previously processed trigger files shared across workflows that contain the same template.
- 'absolute' - One trigger list is tracked regardless of where it occurs.
- 'By step' - Similar to 'By workflow', but allows for multiple independent trigger steps within the workflow.
- 'None' - No persistence at all. If a past trigger file is still present, it will re-trigger if 'Continue Monitoring' is selected. This option is recommended when 'Continue Monitoring' is not selected.

11.3.6 Trigger Types Details

- 'Trigger On Session' - Trigger is initiated for every new session.
- 'Trigger On Files' - Triggers on all the new files from the previous delta. New trigger is initiated on the new files created after initial trigger.
- 'Trigger On Single File' - New trigger is created on every single file.

11.3.7 Inputs Description

In most cases, all the inputs are optional. The exception being when the node to be monitored is not defined (no nodes selected, no central address provided). In that case the input *Central_node* can be used to specify the name of the remote node. Alternatively a central address can be provided in input if you use the <%= variable_name %> form in the central address field. Another possibility is to select ”Runtime Specified” as a central node in the template and then set the node input.

- Central_node: Optional runtime specification of the remote node name when not defined in template.
- transfer_direction: Optional override for the template parameter. Valid values are 'send' and 'receive' to be provided as string.
- central_address: Runtime central address specification when using <%= central_address %> notation in the Central address field. The central or node API port can be specified with colon notation (e.g., 172.12.45.67:9092).

11.3.8 Outputs Description

- Session Information: Depending on the session filter specified, multiple session-related information are exposed. NOTE: there might be outputs with empty or nil values depending on the monitored session (e.g., MD5).

11.3.9 Supported Actions

- Cancel

11.3.10 Operating Instructions

- Compatibility: This plugin works against Enterprise Server, Connect Server, and Point to Point as they all use Aspera Central to expose the transfer sessions.

- Server Configuration: The reliable API must be enabled on the Aspera Server being polled, and the IP address polling needs to be enabled with the following configuration in aspera.conf: <central_server> <address>0.0.0.0</address> <port>40001</port> <persistent_store>enable</persistent_store> </central_server>
- Node API Authentication: If a node user and password are specified, there is no need to change the address to 0.0.0.0 since the requests will not go directly to central (via port 40001) but to NodeAPI.
- Runtime Address Input: If you want to enter a central address in runtime input rather than a remote node, use the <%= central_address %> notation within the Central address field. The central or node API port can be specified in this input with the colon notation after the address such as 172.12.45.67:9092.
- Async Activity Logging: If you are using async, in the default section of the aspera.conf file, the tag <async_activity_logging>true</async_activity_logging> must be specified for async activity to be reported to Aspera Central.
- Filter Examples: If the central watcher needs to find files only from the folder '/tmp/server1/' then the File Filter can be set as '^/tmp/server1/.*'. Similarly, if even 1 file from the session contains 'hdfs//', then apply the session filter as '(^hdfs//.*)' to filter out the entire session.
- Filter Updates: After adding/updating/removing a filter, restart the workorder. It's not sufficient to reset the workorder as you may get a Filter Changed Error.
- WSDL Configuration: The WSDL found at asperacentral:40001/console/services/soap/FASPSession.wsdl cannot be used as is. It assumes web service calls to be local and thus would not work if orchestrator is running on a different machine than the enterprise server. Use the Orchestrator WSDL generator (don't put anything in the central service WSDL address) or save a local copy of the WSDL and edit the IP address to reflect the address at which the Enterprise Server is actually found.
- WSDL Generator: The central service WSDL address is by default built out of /opt/aspera/orchestrator/config/orchestrator/actions/aspera_central_watchers/wsdl/FASPSession200911.wsdl.erb, shipped with Orchestrator. The IP address (127.0.0.1) is replaced at runtime by the actual central address. The built WSDL address path looks like file:///opt/aspera/var/run/orchestrator/aspera_central_watchers/wsdl/37.58.115.131_40001__gen_FASPSession200911.wsdl.

11.3.11 Dependencies

- Aspera Central service must be enabled and configured on the monitored server.
- Network connectivity to the Aspera server on the specified central port (default 40001) or Node API port.
- Proper authentication credentials when using Node API authentication method.
- WSDL file accessibility either through automatic generation or manual configuration.
- The aspweb user must have proper permissions to read the WSDL file or to generate a WSDL file using the Orchestrator WSDL generator.

11.3.12 Troubleshooting

- If the watcher is not triggering, verify that Aspera Central is enabled and the persistent store is configured in aspera.conf on the monitored server.
- For authentication issues, confirm that Node API credentials are correct and that the user has appropriate permissions for transfer monitoring.
- If filters are not working as expected, restart the workorder after making filter changes. Resetting the workorder may result in Filter Changed Errors.
- For WSDL-related errors, ensure the central service WSDL address is properly configured or use the automatic WSDL generator by leaving the field empty.
- If transfers are not being detected, check that the transfer server/client addresses in filters match exactly how they appear in actual transfers (e.g., localhost vs 127.0.0.1).
- For async transfers not appearing, verify that async_activity_logging is enabled in the aspera.conf file on the monitored server.
- If runtime node specification is not working, ensure you're using the correct variable notation (<%= central_address %>) in the configuration fields.

11.4 AoC Package Watcher

This plugin can be used to trigger workflows on new packages uploaded to Aspera On Cloud.

Version	Comment
0.6.3	Enhanced action template UI
0.6.2	Handle issues when no package has been sent for some time

Version	Comment
0.6.1	Updated the label from Keep ongoing to Continue Monitoring
0.6.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.5.4	Fix issue when the Package_File_Paths_On_Node output duplicates part of the path
0.5.3	Fix issue when receiver is only in Bbc-receiver and not standard receiver
0.5.2	Removed dependency to Amqp plugins
0.5.1	Correctly fills sender information in case of external senders
0.5.0	Adding support for Http proxy
0.4.0	Major release including all the new outputs and optimisation.
0.3.9	Better exposure of outputs
0.3.8	Made token a local variable. Added code to make node credentials optional.
0.3.7	Fix filtering on dropbox_name
0.3.6	Fix for JSON parsing
0.3.5	Fix for Node API details of single package
0.3.4	Fix for sent_at_timestamp
0.3.3	Fix for blank responses
0.3.2	Fix for plugin failing before baseline is set
0.3.1	Updating branding from Aspera Files to AoC
0.3.0	architecture change from last_timestamp
0.2.0	adding fields for page and token storage
0.1.3	bug fix for packages with same timestamp and limit token api call
0.1.2	object reload for retrieving the current last_timestamp
0.1.1	start watching from Time.now if ignore existing package is set
0.1.0	pull packages one at a time by keeping track of sent_at from previous packages
0.0.4	new output metahash
0.0.3	Verify ssl false
0.0.2	Fixes a warning in Ruby code
0.0.1	Initial implementation

11.4.1 How It Works

- Monitors and retrieves package information from Aspera On Cloud (AoC) using OAuth authentication and API calls.
- Supports two main operations: fetching single package details by ID or continuous polling for new packages.
- Provides comprehensive package filtering based on name, sender, recipient, dropbox, and custom conditions.
- Integrates with Node API to extract individual file names and paths within packages when enabled.
- Offers configurable trigger persistence scopes and polling frequencies for flexible workflow automation.

11.4.2 Configuration Instructions

1. Add an AsperaOnCloudPackageWatcher step to your workflow.
2. Configure AoC authentication parameters:
    - Name: Enter a descriptive name for the watcher instance.
    - Package Operation: Choose 'Fetch Single Package Details' or 'Keep Polling New Packages'.
    - AoC Client: API client ID from AoC Organization → Integrations.
    - AoC Client Secret: API client secret for OAuth authentication.
    - AoC Organization Name: Name of the AoC organization.
    - AoC User Name: User email address for AoC access.
    - Private key filepath: Path to SSH private key file on Orchestrator.
3. Set up operation-specific parameters:
    - AoC Package ID: Specific package ID (for single package operation).
    - AoC Packages Folder: Package storage path (for polling operation, e.g., /workspaces/[workspace_id]/packages/).
    - Filter Conditions: Package filtering criteria (for polling operation).
    - Keep Ongoing: Enable continuous monitoring and workflow cloning.
    - Polling Frequency: Interval in seconds between AoC API calls (default: 10).

11.4.3 Example Configuration



Figure 11.4: AoC Package Watcher icon

Parameter	Example Value
Name	AoC Package Monitor
Comments	Monitors new packages in shared inbox
Package Operation	Keep Polling New Packages
AoC Client	12345678-abcd-1234-5678-123456789abc
AoC Client Secret	********
AoC Organization Name	mycompany
Scope	user:all
AoC User Name	monitor@company.com
Private key filepath	/opt/aspera/keys/aoc_private_key
AoC Packages Folder	/workspaces/ws123/packages/
Keep Ongoing	checked
Ignore Existing Packages	checked
Extract File Names Using Node API	checked
Polling Frequency	10

11.4.4 Saved Parameters Description

Parameter	Description
Name	Display name for this package watcher instance.
Comments	Notes or description for this step.
Package Operation	Operation type: 'Fetch Single Package Details' for specific package retrieval or 'Keep Polling New Packages' for continuous monitoring.
AoC Client	API client ID used to access Aspera On Cloud.
AoC Client Secret	Secret key of the API client for OAuth authentication.
AoC Organization Name	Name of the AoC organization associated with the user.
Scope	Authorization scope for package watching operations.
AoC User Name	AoC username (email) for accessing Aspera On Cloud.
Private key filepath	Path to file containing the AoC user's private key information.
Http proxy	HTTP proxy URL for AoC API access (requires Orchestrator 4.0.1+ or capsule).
AoC Package ID	Package ID for 'Fetch Single Package Details' operation only.
Ignore Existing Packages	For 'Keep Polling New Packages': check to monitor only new incoming packages, uncheck to include existing packages in inbox.
AoC Packages Folder	For 'Keep Polling New Packages': path where AoC packages are stored on the node (format: /workspaces/[workspace_id]/packages/).

Parameter	Description
Filter Conditions	For 'Keep Polling New Packages': conditions to filter triggered packages. Available fields: package_name, dropbox_name, title, sender_email, sender_name, recipient_user_email, recipient_user_name. Example: ”(package_name == '<%= package_name %>') and (dropbox_name == '<%= dropbox_name %>')”.
Keep Ongoing	For 'Keep Polling New Packages': enable to clone workflow after each trigger event for continuous monitoring.
Extract File Names Using Node API	Enable to retrieve individual file names through Node API and populate Package_File_Paths_On_Node output.
Node API Access Key	Access key for Node API communication (required only when 'Extract File Names Using Node API' is enabled).
Node API Secret	Secret key for Node API authentication (required only when 'Extract File Names Using Node API' is enabled).
Polling Frequency	Interval in seconds between AoC API polls (default: 10 seconds if left blank).
Trigger Persistence Scope	Scope of trigger output availability: 'By workorder group' (default), 'By action template', 'absolute', 'By step', 'By workflow', 'None'.

11.4.5 Inputs Description

All inputs support dynamic variables using the <%= variable_name %> syntax for runtime configuration.

- Dynamic authentication variables: AoC client credentials, organization name, user name, and private key path when configured with runtime variables.
- Package filtering variables: Variables referenced in Filter Conditions for dynamic package filtering.
- Node API credentials: Access key and secret for Node API when Extract File Names Using Node API is enabled.
- Runtime package ID: Package ID for single package operations when configured as runtime input.

11.4.6 Outputs Description

- HTTP_Return_Code_Token_Call: HTTP response code for the OAuth token request.
- Files_OAuth_Token: AoC OAuth token generated from the API for authentication.
- Files_Package_Name: Name of the triggered package.
- Files_Package_Note: Note/description of the triggered package.
- Files_Package_ID: Unique identifier of the triggered package.
- Contents_File_ID: ID of the package contents folder.
- File_ID: ID of the package file.
- Package_Metadata: Array format of package metadata information.
- Metadata_Hash: Hash format of package metadata (same information as Package_Metadata).
- Sender_Email: Email address of the package sender.
- Sender_Name: Name of the package sender.
- External_Sender: Boolean indicating if sender is external user (true/false).
- Recipient_Email: Email address of the package recipient.
- Recipient_Name: Name of the package recipient.
- Recipient_Type: Type of package recipient (user, dropbox, etc.).
- Bearer_Token: Bearer token for Node API access (available only when Extract File Names Using Node API is enabled).
- Node_API_Header: Header information for Node API communication.
- Package_File_Paths_On_Node: Array of file paths on the node (available only when Extract File Names Using Node API is enabled).

11.4.7 Supported Actions

- Fetch Single Package Details
- Keep Polling New Packages

11.4.8 Dependencies

- Valid Aspera On Cloud organization with appropriate API client configuration.

- SSH key pair configured (public key in AoC user profile, private key on Orchestrator).
- The aspweb user must have proper permissions to read the private key file.
- Orchestrator must have internet connectivity to reach AoC APIs.
- For versions 0.3.1+: Capsule '2018_11_06_modified_aspera_files_module_1' required for Orchestrator 3.1.1 and earlier releases.

11.4.9 Operating Instructions

11.4.10 Package Operations

- Fetch Single Package Details: Retrieves information for a specific package using the provided Package ID.
- Keep Polling New Packages: Continuously monitors for new packages in the specified folder, applying filters and triggering workflows as configured.

11.4.11 Filter Configuration

- Filter Conditions support dynamic variables and logical operators for flexible package selection.
- Available filter fields: package_name, dropbox_name, title, sender_email, sender_name, recipient_user_email, recipient_user_name.
- Example filter: `(package_name == '<%= package_name %>') and (dropbox_name == '<%= dropbox_name %>')`
- For shared inbox filtering, use: `(dropbox_name == '<%= shared_inbox_name %>')` in Filter Conditions.

11.4.12 Node API Integration

- Enable 'Extract File Names Using Node API' to populate Package_File_Paths_On_Node output with individual file paths.
- Requires Node API Access Key and Secret for authentication with the transfer node.
- Bearer Token and Node API Header outputs are generated automatically when Node API extraction is enabled.

11.4.13 Trigger Persistence Scopes

- By workorder group (default): Tracks triggers during workorder execution, prevents duplicate triggering within the same workorder group.
- By workflow: Similar to workorder group but maintains persistence across workflow restarts.
- By action template: Shares trigger tracking across workflows using the same template.
- Absolute: Global trigger tracking regardless of workflow context.
- By step: Independent trigger tracking per step within workflows.
- None: No trigger persistence, allows re-triggering on existing packages.

11.5 Aspera Node API Transfer Monitor



Figure 11.5: Aspera Node API Transfer Monitor icon

This action plug-in provides the ability to monitor and detect file transfer events on remote Aspera servers via the Aspera Node API.

Version	Comment
0.3.3	Enhanced action template UI
0.3.2	Removed empty option from trigger persistence scope dropdown
0.3.1	Updated the label name of Keep ongoing to Continuous Monitoring
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.2	Add handling of filter_file-path
0.2.1	Add destination_path and remote_user in returned transfer spec
0.2.0	Add tag filter
0.1.2	verify ssl false
0.1.0	Changing the field name and fixing some format issues.
0.0.3	Declaring output variable(Undefined before).
0.0.2	Corrected plugin name and display name in metadata.yml
0.0.1	initial release

11.5.1 How It Works

- Monitors Aspera transfer events through Node API database queries with configurable filters and criteria.
- Supports filtering by transfer direction, event types, file paths, tags, and transfer IDs for targeted monitoring.
- Provides ongoing trigger capability to spawn new workflow instances when matching transfers are detected.
- Offers authentication options including Basic and Bearer token methods for secure API access.
- Returns comprehensive transfer information including file lists, detailed metadata, and transfer IDs.

11.5.2 Configuration Instructions

1. Add an AsperaNodeApiTransferMonitor step to your workflow.
2. Configure basic connection settings:
    - Remote Node: Select configured node or enter manual IP address and credentials.
    - Use ssl?: Check for HTTPS (port 9092), uncheck for HTTP (port 9091).
    - Auth Type: Choose Basic or Bearer authentication.
3. Set up monitoring filters:
    - Filter Direction: Specify 'send', 'receive', or leave blank for all.
    - Type: Select event type or use patterns (e.g., download.*).
    - Keep Ongoing?: Check to enable continuous monitoring.
4. Save, publish, and deploy the workflow. On launch, it remains active until matching transfers are detected.

11.5.3 Advanced Configuration Options

11.5.4 Connection Parameters

- Node API port: Custom port number (defaults: 9091 for HTTP, 9092 for HTTPS).
- Node api user login: API username or access key for authentication.
- Node api user password: API password or secret for authentication.

11.5.5 Filtering Options

- Type Glob: Pattern matching for multiple event types (e.g., download.*, file.*).
- Filter file path: Regular expression to filter based on file paths.
- Tag: Filter by transfer tags (use %3D for = symbol).
- Transfer ID: Filter by specific transfer ID for targeted monitoring.
- Count: Maximum events returned per polling cycle (default: 1).

11.5.6 Monitoring Behavior

- Ignore zero byte transfers?: Exclude transfers with no bytes transferred.
- Ignore empty transfers?: Exclude transfers with no completed files.
- Return single match?: Return single transfer instead of array.
- Polling Frequency: Database polling interval in seconds (default: 10).

11.5.7 Example Configuration

Parameter	Example Value
Name	New AsperaNodeApiTransferMonitor Step
Use SSL?	checked
Remote node address	[HSTS IP address]
Node api port	9092
Node api user login	[Node API username]
Node api user password	********
Auth Type	Basic
Count	1
Keep Ongoing?	checked
Polling Frequency	10

11.5.8 Saved Parameters Description

Parameter	Description
Name	Display name for this transfer monitoring step.
Use ssl?	Enable HTTPS for Node API (port 9092) instead of HTTP (port 9091).
Remote Node	Configured node with IP address, port, username, and password for Aspera Node API.
Remote node address	IP address or domain name of the node (when Remote Node is not provided).
Node api port	Node API port number (when Remote Node is not provided).
Node api user login	Node API username or access key (when Remote Node is not provided).
Node api user password	Node API password or secret (when Remote Node is not provided).
Auth Type	Authentication method: Basic or Bearer token authentication.
Filter Direction	Transfer direction filter: 'send' or 'receive'.
Filter file path	Regular expression to filter transfers based on file path.
Count	Maximum number of returned events from Node API database (default: 1).
Type	Event type to monitor: transfer.started, transfer.ended, download.started, download.ended, upload.started, upload.ended, file.created, file.modified, file.deleted, file.moved.
Type Glob	Pattern matching expression for Type field (e.g., download.*, file.*).
Tag	Filter by transfer tag (e.g., aspera.faspex.dropbox%3Ddb111 for Faspex transfers, %3D represents =).
Transfer ID	Filter by specific transfer ID for targeted monitoring.
Ignore zero byte transfers?	Exclude transfers with no bytes transferred from results.
Ignore empty transfers?	Exclude transfers with no completed files from results.
Keep Ongoing?	Enable continuous monitoring to spawn new workflow instances when matching transfers are detected.
Return single match?	Return single matching transfer instead of array of matching transfers.
Polling Frequency	Interval in seconds for polling Node API database (default: 10 seconds).

11.5.9 Inputs Description

- Remote Node information: Required parameters for Node API connection including address, credentials, and authentication settings.

11.5.10 Outputs Description

- File_list: Array of file lists for matching transfers. Each entry contains the file list for a matching transfer. With 'Return

single match', this becomes a single transfer file list.
- File_List_With_Details: Array of transfer structures containing detailed hash structures for matching transfers. With 'Return single match', this becomes a single transfer hash.
- Transfer_id_list: Array of distinct transfer IDs for matching transfers. With 'Return single match', this becomes a single transfer ID string.
- Error: Error message details when issues occur during monitoring or API communication.

11.5.11 Supported Actions

None

11.5.12 Dependencies

- Aspera Transfer Server or Transfer Endpoint with valid license.
- Asperanoded service running for Node API functionality.
- Node API database containing transfer event data (typically last 24 hours).
- Valid Node API user credentials with appropriate permissions.
- If the Aspera HSTS version is 4.4.5 or 4.4.6, users will need to use an access key and secret instead of a Node API User and password.

11.5.13 Operating Instructions

11.5.14 Important Parameters

- Type vs Type Glob: Use Type for single patterns, Type Glob for multiple patterns. Never use both simultaneously.
- Count optimization: Use count=1 for slow processing (one event per cycle), count>1 for faster batch processing.
- Tag filtering: Use URL encoding (e.g., %3D for =) in tag filters like 'aspera.faspex.dropbox%3Ddb111'.

11.6 Aspera node file watcher



Figure 11.6: Aspera node file watcher icon

This action plug-in provides the ability to detect and monitor files and folders matching specified patterns on remote Aspera servers via the Aspera Node API.

Version	Comment
0.9.5	Enhanced action template UI
0.9.4	Fix crash when re-scanning with random selection and trigger_on_partial
0.9.3	Updated the label from Keep ongoing to Continue Monitoring
0.9.2	Updated the label from Keep ongoing to Continue Monitoring
0.9.1	Fixed count field UI update issue and changed certain fields from text_field to number_field
0.9.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.8.0	Add random file selection for file search
0.7.2	Allow to change search order by descending date of modification (newest first)
0.7.1	Get count elements for files/page
0.7.0	Support files/page for searching

Version	Comment
0.6.0	Allow to ignore the BadGateway errors
0.5.2	Support the form hostname:port in node address
0.5.1	Added Filename exclusion help
0.5.0	Added return only folders option
0.4.0	Added aggressive pruning with beacon file for stability check
0.3.0	Adding logic to exclude zero bytes files
0.2.0	Adding logic to exclude candidate file based on a regex
0.1.0	Add column to ask Rest Timeout value. Default is 60.
0.0.9	Added styling to help text.
0.0.8	RemoteNode Dependency
0.0.7	Avoid error when no files are found and data['items'] is nil
0.0.6	Allow remote node in server address input
0.0.5	use path for watch dir and paths for file pattern
0.0.4	update of help file regarding root user
0.0.3	update of help file regarding depth max
0.0.2	exception handling of RemoteNode.find_by_name()
0.0.1	initial release - basic with no import, no recurring

11.6.1 How It Works

- Monitors directories on remote Aspera servers for new files and folders matching specified patterns using Node API.
- Supports both file search (with directory recursion) and file pagination (flat directory scanning) modes.
- Provides flexible pattern matching with GLOB expressions, regular expression exclusions, and depth limitations.
- Offers continuous monitoring (hot folder) or single-check operations with configurable polling frequencies.
- Includes advanced features like stability checking, aggressive pruning, and configurable trigger persistence scopes.

11.6.2 Configuration Instructions

1. Add an AsperaNodeFileWatcher step to your workflow.
2. Configure the Node API connection parameters:
   - Name: Enter a descriptive name for the file watcher.
   - Aspera Node: Select from defined remote nodes or leave empty for manual configuration.
   - Aspera node IP address: IP address, domain name, or address:port format.
   - Use ssl?: Check for HTTPS (port 9092), uncheck for HTTP (port 9091).
   - Node user name: Node API username for authentication.
   - Node user password: Node API password for authentication.
3. Set up file monitoring parameters:
   - Watch directory: The directory path to monitor for files (e.g., /incoming).
   - File pattern: GLOB pattern for file matching (e.g., *.txt, *.pdf).
   - Continue Monitoring: Check for continuous monitoring, uncheck for single check.
   - Polling frequency: Interval in seconds between directory scans (e.g., 30).
   - Return single match?: Check to return one file at a time, uncheck for arrays.

11.6.3 Example Configuration

Parameter	Example Value
Name	File Drop Monitor
Comments	Monitors incoming directory for new files
Aspera Node	transfer-server-01
Node user name	api_user
Node user password	********
Use ssl?	checked
Watch directory	/incoming
File pattern	*.txt,*.pdf
Continue Monitoring	checked
Polling frequency	30

Parameter	Example Value
Return single match?	unchecked
Count	1000
Trigger persistence scope	by workorder group

11.6.4 Saved Parameters Description

Parameter	Description
Name	Display name for this file watcher instance.
Comments	Notes or description for this step.
Aspera Node	Remote node name from defined Remote Nodes list.
Aspera node IP address	Remote node IP address, domain name, or address:port format.
Use ssl?	Enable HTTPS on port 9092, disable for HTTP on port 9091.
Node api port	Custom Node API port number (overrides default ports).
Node user name	Node API username for authentication.
Node user password	Node API password for authentication.
Watch directory	The directory search path for files or folders on the remote server.
File pattern	File name or pattern using GLOB expressions (e.g., *.txt, a*,b*).
Depth max	Maximum search depth for file search operation (infinite if not set, not applicable to file pagination).
Count	Maximum number of files returned per request (default: 1000).
Filename exclusions	Regular expression to exclude specific filenames from search results.
Ignore folders?	Exclude folders from results, return only matching files.
Polling frequency	Interval in seconds for scanning the watched directory.
Check only once	Single check for file presence and return (no waiting).
Continue Monitoring	Enable continuous scanning for matching files (hot folder mode).
Return single match?	Return one file at a time instead of arrays of matching files.
Trigger on partial?	Detect files even if not stable (still being written).
Ignore BadGateway error?	Ignore RestClient::BadGateway and HTTP 500 internal server errors.
Aggressive pruning	Remove files from past triggers list if no longer found in watched directory (file search only).
Beacon file for stability check	Beacon file path to verify file system accessibility for aggressive pruning.
Use file pagination rather than file search?	Use file pagination to overcome 1000 file limitation of file search.
Order by newest date of modification first for file search?	Sort by newest modification date instead of oldest (file search only).
Select files randomly for file search?	Randomly select count files from up to 1000 retrieved files (file search only).

Parameter	Description
Trigger persistence scope	Scope of trigger tracking: 'by workorder group' (default), 'by workflow', 'by action template', 'absolute', 'by step', 'none'.

11.6.5 Inputs Description

Changes in this list of inputs will override any values set in the action template.

- Watch_directory: The directory search path for files or folders.
- File_pattern: Detection file name or pattern using GLOB expressions.
- node_user_name: Node API username for authentication.
- node_password: Node API password for authentication.
- server_address: Remote node name, IP address, domain name, or address:port format.

11.6.6 Outputs Description

- File_list: Array of matching files (when Return single match is unchecked).
- File_path: Single matching file as string (when Return single match is checked).

11.6.7 Supported Actions

None

11.6.8 Dependencies

- Aspera Transfer Server or Transfer Endpoint with valid license and asperanoded service running.
- Node API must be enabled and accessible on the remote Aspera server.
- Network connectivity to Node API ports (9091 for HTTP, 9092 for HTTPS).
- Valid Node API user account with appropriate permissions.

11.6.9 Operating Instructions

11.6.10 Plugin Operation

This plugin detects new files or folders matching patterns in watched directories on remote servers. It can perform single-time checks or continuous monitoring (hot folder behavior) with configurable polling intervals.

11.6.11 Configuration Guidelines

11.6.12 Common Settings (File Search and File Pagination)

- Populate watch directory field with the directory to scan (wildcards not allowed in directory path).
- Use file pattern field for GLOB expressions: *.txt for all text files, *.txt,*.mxf for multiple extensions.
- Pattern matching is case sensitive.
- The number of returned files may vary based on matching files found during scans and stability checks.
- Select 'Continue Monitoring' for periodic scanning (hot folder), leave unchecked for single check.
- 'Check only once' and 'Continue Monitoring' are incompatible options.

11.6.13 File Search Specific (Default Mode)

- Searches all subdirectories below watched directory by default.
- Use 'Depth max' to restrict search depth (e.g., depth max 1 for files directly under watched directory).
- Advanced patterns: foo*/*.txt searches txt files in directories starting with 'foo'.
- Pattern matching: fo? matches 'foo' or 'fo1', but not 'foo1'. fo[1-9] matches 'fo1' or 'fo2'.
- Default sorting by oldest modification date first, configurable to newest first.
- 'Select files randomly' option provides wider retrieval window among existing files.
- May block after Count files returned if same files remain in directory.

11.6.14 File Pagination Specific

- Overcomes 1000 file limitation of file search by using paging.
- Returns matching files by default (no folders unless specifically configured).
- Sorting done by filename in ascending alphabetical order.
- Cannot retrieve files in subdirectories of watched directory.
- Processes pages containing the specified count of files until candidates and stable files are found.

11.6.15 Node Configuration Requirements

11.6.16 Node API Account Setup

Node username is a Node API account (not an OS account). Find the associated transfer user:

```
\ # /opt/aspera/bin/asnodeadmin -l
\ List of node user(s):
\          user      system/transfer user              acls
\ ====================  ======================  ====================
\       node_user                xfer      []\
```

11.6.17 Path Configuration

Watch directory paths are relative to transfer user's docroot in `/opt/aspera/etc/aspera.conf`:

```
\ <user>
\    <name>xfer</name>
\    <file_system>
\       <access>
\          <paths>
\             <path>
\                <absolute>/home/xfer/data</absolute>
\             </path>
\          </paths>
\       </access>
\    </file_system>
\ </user>\
```

Example: Watch directory ”/root” refers to ”/home/xfer/data/root” on the remote server.

11.6.18 Port Configuration

Enable both HTTP and HTTPS ports in `/opt/aspera/etc/aspera.conf`:

```
\ <server>
\    <server_name>node.example.com</server_name>
\    <http_port>9091</http_port>
\    <https_port>9092</https_port>
\    <enable_http>true</enable_http>
\    <enable_https>true</enable_https>
\ </server>\
```

Restart asperanoded service to apply changes:

\ systemctl restart asperanoded\

11.6.19 Trigger Persistence Scope Details

- By workorder group (default): Tracks triggers during workorder execution, prevents duplicate processing within workorder group.
- By workflow: Same as workorder group but no triggering on restart actions for same workflow.
- By action template: Prevents re-triggering across workflows sharing the same template.
- Absolute: Global trigger tracking regardless of context.
- By step: Independent trigger tracking per step within workflows.
- None: No persistence, allows re-triggering on existing files when 'Continue Monitoring' is selected.

11.6.20 Important Notes

- Select 'Trigger on partial' to allow triggering before files are stable (still being written).
- When changing 'Return single match', save and re-publish workflow to register output type changes.
- File search may block if the same files remain in directory - consider moving/deleting processed files.
- For file pagination, ensure fast Node API response times to avoid long delays during page processing.

11.7 Azure Service Bus Trigger

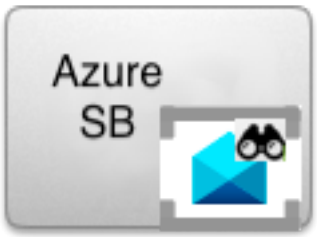


Figure 11.7: Azure Service Bus Trigger icon

This action plug-in provides the ability to watch for messages out of an Azure Service Bus queue

Version	Comment
0.1.3	Enhanced action template UI
0.1.2	Fix issue when loading help page
0.1.1	Updated the label from Keep ongoing to Continue Monitoring
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.1	Initial release.

11.7.1 How It Works

- This action plug-in monitors an Azure Service Bus queue or topic for new messages.

11.7.2 Configuration Instructions

1. Create a new action step using the AzureServiceBusTrigger plug-in.
2. This action monitors an Azure Service Bus queue or topic and triggers when new messages are available.
3. Fill out the following configuration fields:
   - Name: The name used to identify a saved AzureServiceBusTrigger configuration.
   - Comments: Optional comments about this AzureServiceBusTrigger configuration.
   - Azure namespace: The Azure Service Bus namespace.
   - Azure key name: The Azure Service Bus access key name.

- Azure key: The Azure Service Bus access key.
- Queue or Topic name: The Azure Service Bus queue or topic name.
- Subscription_name: For the Message_get_from_topic operation, enter the Azure Service Bus subscription name.
- Proxy url: The proxy URL used to access Azure Service Bus. Leave this field blank if a proxy is not required.
- Operation: The operation to execute.
- Continue Monitoring: When selected, the workflow is cloned after each trigger event so that new messages are processed continuously.
- Polling frequency: The interval in seconds at which the queue or topic is polled for new messages. The default is 10 seconds.

11.7.3 Example Configuration

This is a minimal configuration example for the AzureServiceBusTrigger plug-in.

Parameter	Value
Azure namespace	<sample_namespace>
Azure key name	<sample_key_name>
Azure key	<sample_key>
Queue or Topic name	topic1
Proxy url	https://test-source.azure-bus.com/bus1
Operation	Message_get_from_queue

11.7.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved AzureServiceBusTrigger configuration.
Comments	Optional comments about this AzureServiceBusTrigger configuration.
Azure namespace	The Azure Service Bus namespace.
Azure key name	The Azure Service Bus access key name.
Azure key	The Azure Service Bus access key.
Queue or Topic name	The Azure Service Bus queue or topic name.
Subscription_name	For the Message_get_from_topic operation, enter the Azure Service Bus subscription name.
Proxy url	The proxy URL used to access Azure Service Bus. Leave this field blank if a proxy is not required.
Operation	The operation to execute.
Continue Monitoring	When selected, the workflow is cloned after each trigger event so that new messages are processed continuously.
Polling frequency	The interval in seconds at which the queue or topic is polled for new messages. The default is 10 seconds.

11.7.5 Inputs Description

Required inputs depend on how the AzureServiceBusTrigger template is configured. If dynamic attributes are used, each attribute becomes a runtime input.

Parameter	Description
*Azure_namespace *	The Azure Service Bus namespace.
*Azure_key_name *	The Azure Service Bus access key name.
*Azure_key *	The Azure Service Bus access key.
*Queue or Topic name *	The queue or topic name.

Parameter	Description
Proxy_url	The proxy URL (optional).
Polling_frequency	The polling frequency in seconds (default is 10 seconds).

11.7.6 Outputs Description

Parameter	Description
Message_body	For Message_get operations, the retrieved message body.
Error_message	The error message returned by Azure Service Bus, if an error occurs.

11.7.7 Supported Actions

None

11.7.8 Dependencies

None

11.7.9 Operating Instructions

1. This action plug-in monitors an Azure Service Bus queue or topic for new messages.
2. Specify the Azure Service Bus namespace, access key name, and access key.
3. Specify the queue or topic name.
4. Specify the subscription name only when retrieving messages from a topic.
5. Specify a proxy URL (e.g., https://myproxy.com:8443) if direct Internet access is not available. Without a proxy, Orchestrator must be able to reach .servicebus.windows.net on port 443.
6. Enter the operation to execute (Message_get_from_queue or Message_get_from_topic). Only one message can be retrieved with either operation.
7. Message_get_from_queue and Message_get_from_topic are get-and-delete operations.
8. Select Continue Monitoring to continue monitoring for new messages indefinitely.
9. Specify the polling frequency for monitoring the queue or topic (default is 10 seconds).

11.8 Custom trigger

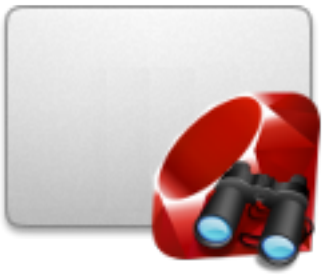


Figure 11.8: Custom trigger icon

The Custom Trigger Plugin allows workflows to be initiated based on custom Ruby logic that runs periodically until a defined condition is met.

Version	Comment
0.4.2	Enhanced action template UI
0.4.1	Updated the label from Keep ongoing to Continue Monitoring
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.0	Adding cancel option to cancel the underlying process
0.2.1	Adds support for cancel
0.2.0	Adds support for basic pause
0.1.4	New icon
0.1.3	Adds help file
0.1.2	Adds placeholder for polling frequency
0.1.1	Added DEFAULT_POLLING_FREQUENCY constant as it was missing and the plug-in failed when executed
0.1.0	Adds UI helper widget for i/o specifications
0.0.4	clean up. Even in error, check_status should return a polling delay
0.0.3	Adds tolerance to inputs_spec function
0.0.2	adds rich text editing
0.0.1	initial release

11.8.1 How It Works

- The Custom Trigger plugin allows you to define your own trigger logic using Ruby scripts.
- It is designed for scenarios where built-in triggers (like file watchers or schedule triggers) are not sufficient.
- The plugin continuously runs the custom Ruby script until a defined condition is met.
- Once the condition is satisfied, the plugin triggers the associated workflow.
- This is useful for integrating with external systems, APIs, or custom monitoring logic.
- The script can include loops, condition checks, and external calls as needed.
- It gives full control over when and how a workflow should be triggered.

11.8.2 Configuration Instructions

1. Add a Custom Trigger step to your workflow.
2. Configure the trigger environment:
   - Name: A label to identify this Custom Trigger step.
   - Required inputs: Define any inputs your Ruby script needs.
   - Outputs: Define the expected outputs from the script.
   - Execute code: Write or paste the Ruby logic that defines the trigger condition.
   - Check status code: The body of the Ruby code that will be executed periodically at each polling interval to evaluate the trigger condition.

11.8.3 Example Configuration

11.8.4 Example: Sample code to check that a file transfer is complete based on the aspx file extension.

Field	Parameter	Type
Required inputs	transferred_file	String
Outputs	file_size	Integer

In the 'Execute code' section:

@file = inputs[”transferred_file”] + ”.aspx”

In the 'Check status code' section:

\ if File.exist?(@file)

\ @status_details = ”Transfer of file #{inputs[”transferred_file”]} on-going at #{DateTime.now}”

\ @status = STATUS_INPROGRESS

\ else

\ @status_details = ”Transfer of file #{inputs[”transferred_file”]} is complete”

\ @status = STATUS_COMPLETE

\ end

The status_details variable takes the data to display in the step status when the workflow is executed.

11.8.5 Saved Parameters Description

Parameter	Description
Name	A label to identify the saved Custom Trigger instance.
Comments	Notes or remarks about this Custom Trigger instance.
Required inputs	Define any inputs your Ruby script needs.
Optional inputs	Add any additional inputs that may or may not be used.
Outputs	Define the expected outputs from the script.
Execute code	Ruby code that runs once at the beginning of the action.
Check status code:	Ruby code that runs periodically at each polling interval to check the trigger condition.
Support direct pause?	If enabled, a Pause button appears on the Work Orders page. This allows pausing the action, which stops polling without executing any additional pause logic.
Support direct cancel?	If enabled, a Cancel button appears on the Work Orders page. This allows canceling the action along with its underlying process.
Polling frequency	Time interval for polling. Default is 5 seconds.
Continue Monitoring	If enabled, the plugin acts as a trigger and starts a new work order each time the code completes.

11.8.6 Inputs Description

- The list of inputs depends on how the Custom Trigger action template is configured.
- Inputs are defined in the 'Required inputs' and 'Optional inputs' sections.

11.8.7 Outputs Description

- Outputs are defined in the 'Outputs' section.
- These outputs are returned with the names and types specified during configuration.

11.8.8 Supported Actions

None

11.8.9 Operating Instructions

- This plugin runs Ruby code repeatedly to check for trigger conditions.
- Inputs are accessed using the inputs hash.
- Inputs must be declared under Required or Optional inputs.
- Outputs are accessed using the outputs hash.
- Use the Execute code section to write Ruby code that runs once at the beginning.
- Use the Check status code section to write Ruby code that runs periodically at each polling interval.
- To keep the action running, return *Action::STATUS_INPROGRESS*.
- To stop the action, return one of the following: *Action::STATUS_COMPLETE* - for successful completion, *Action::STATUS_FAILED* - for failure and *Action::STATUS_ERROR* - for execution errors.

11.8.10 Dependencies

- Ruby must be installed in the same environment as the Orchestrator server.

11.9 Database trigger



Figure 11.9: Database trigger icon

This plugin provides the ability to periodically execute SQL queries on external databases (MySQL, Oracle, ODBC compatible) and trigger workflows when new rows are found.

Version	Comment
1.1.6	Enhanced action template UI
1.1.5	Handle non-SELECT query results
1.1.4	Ensure results with key-value pairs are not reversed
1.1.3	Updated the label from Keep ongoing to Continue Monitoring
1.1.2	Update help docs with revised steps to use OracleDB and ODBC
1.1.1	Updating to be compatible with Ruby 3.4 and Rails 8
1.1.0	Remove hardcoded aspera directory reference
1.0.7	Changed default polling frequency from 0 to 10.
1.0.6	Added styling to parameter help text.
1.0.5	Help file
1.0.4	New icon
1.0.3	Do not persist triggers if scope is none
1.0.2	Prevents auto-complete of user field
1.0.1	Freed result set after select query to optimize performance
1.0.0	Simplified to use generic trigger persistence (requires Orchestrator 2.0)
0.1.1	Removes debug trace for past triggers
0.1.0	Allows different persistence scope, enforce cancel
0.0.7	Enforces polling frequency
0.0.6	Adds functionality to mask password
0.0.5	Adds support for Oracle and use database module
0.0.4	Exposes keep-ongoing Fixes typo in inputs_spec
0.0.3	Adds support for ODBC
0.0.2	Adds rich-text editing
0.0.1	Initial release, only supports mysql

11.9.1 How It Works

- Connects to external databases (MySQL, Oracle, ODBC) to execute SQL queries with dynamic attribute support.
- Supports stored procedures and parameterized queries using template variables for runtime substitution.
- Provides configurable polling frequency and trigger persistence scopes to control when and how often queries execute.
- Offers row and result formatting capabilities to transform query results into desired output formats.
- Returns formatted trigger results with configurable data types and maximum row limits.

11.9.2 Configuration Instructions

1. Add a Database Trigger step to your workflow.
2. Configure the essential database connection fields:
   - Database Type: Select MySQL, Oracle, or ODBC.
   - Database host: Hostname or inline connection string (see connection modes).

- Database port: Connection port for MySQL and ODBC.
- Database name (or DSN): Database name for MySQL; DSN for ODBC; TNS alias for Oracle.
- Database user: Username for database authentication.
- Database password: Password for the database user.
- Database query: SQL query with optional dynamic variables.

3. Set polling and trigger behavior:
   - Polling Frequency: Frequency in seconds (0 = poll as often as possible).
   - Trigger persistence scope: Choose scope for trigger tracking.

11.9.3 Connection Modes

Database connections support two modes:

11.9.4 Mode A — Configuration File / DSN Mode

- Oracle: Use TNS alias via `tnsnames.ora`.
- ODBC: Use DSN via `/home/aspweb/.odbc.ini`.
- MySQL: Use Database host + Database port + Database name.

11.9.5 Mode B — Inline Connection String Mode

Instead of using configuration files, full connection strings may be provided directly via the Database host field.

Oracle EZCONNECT Example:

hostname:1521/ORCLPDB1

ODBC-Style Example (Canonical):

Driver={ODBC Driver 17 for SQL Server};Server=10.10.1.10,1433;Database=mydb;Uid=user;Pwd=;

TinyTDS Alternative Syntax:

server=10.10.1.10;port=1433;database=mydb;username=user;password=;

11.9.6 Connection Precedence

1. Inline connection string (Database host)
2. TNS / DSN (`tnsnames.ora` / `/home/aspweb/.odbc.ini`)
3. Defaults (host + port + database)

11.9.7 Example Configuration

Parameter	Example Value
Database Type	MySQL
Database host	localhost
Database port	3306
Database name (or DSN)	orchestrator_db
Database user	db_user
Database password	********
Database query	SELECT name, login FROM users WHERE status = 'active'
Polling Frequency	30
Trigger persistence scope	By workorder group

Parameter	Example Value
Maximum number of rows returned	1000
Result Type	Array

11.9.8 Query and Formatting Examples

- Dynamic Query: `select name, login from users where login = "<%=user_login%>"` uses the input variable 'user_login'.
- Row Formatting: `row = "#{row[:login]} (#{row[:name]})"` converts rows to formatted strings.
- Results Formatting: `results = results.join(", ")` produces a single formatted string.
- Default Output: Returns array of hashes like `[{:name=>'Maurice', :login=>'momo'}]` when no formatting is applied.

11.9.9 Trigger Persistence Scope Details

- By workorder group: Trigger folders are tracked while workorder is executing and will only be triggered once.
- By workflow: Same as 'By workorder group' except no triggering on restart action.
- By action template: Tracks processing across workflows sharing same template.
- absolute: One trigger list is tracked for all workflows.
- By step: Allows independent trigger tracking for multiple trigger steps.
- None: No persistence; past trigger rows re-trigger if still present.

11.9.10 Saved Parameters Description

Parameter	Description
Name	Display name for this database trigger step.
Comments	Notes or description for this step.
Database Type	MySQL, Oracle, or ODBC.
Database host	Hostname or inline connection string (Mode B).
Database port	Port for MySQL or ODBC.
Database name (or DSN)	Database name for MySQL; DSN for ODBC; TNS alias for Oracle.
Database user	Username for external database authentication.
Database password	Password for the database user account.
Database query	SQL query to execute; supports template variables and stored procedures.
Polling Frequency	Polling interval in seconds (0 = poll continuously).
Trigger persistence scope	Defines tracking behavior for trigger results.
Maximum number of rows returned	Maximum number of rows processed per polling cycle.
Row formatting code	Ruby code executed per row; row is a hash indexed by symbols.
Results formatting code	Ruby code executed after all rows processed; operates on `results` array.
Result Type	Output type; default is array of hashes; must match formatting logic.

11.9.11 Inputs Description

- Dynamic query variables: Variables referenced using `<%=variable_name%>`.
- Formatting variables: Variables used in row or results formatting code.

11.9.12 Outputs Description

- Trigger_results: Formatted results of the SQL query.

11.9.13 Supported Actions

None

11.9.14 Dependencies

- Oracle: Oracle Instant Client; optional `tnsnames.ora` for TNS alias mode.
- ODBC: Requires FreeTDS + unixODBC (or TinyTDS); optional `/home/aspweb/.odbc.ini` for DSN mode.
- MySQL: No additional gems required.

11.9.15 Operating Instructions

11.9.16 Driver Installation (Choose One)

Run installation commands from:

cd /opt/aspera/orchestrator

Oracle Database:

\ bundle add ruby-oci8\ orchestrator restart\

ODBC (TinyTDS) / SQL Server:

\ bundle add tiny_tds\ orchestrator restart\

Note: `bundle add` modifies the Gemfile and runs `bundle install`.

11.9.17 Oracle Configuration

- `tnsnames.ora` may be placed in `/home/aspweb` or `/etc`.
- Important: The aspweb user requires read permissions to access the tnsnames.ora file if using TNS alias mode.
- Inline EZCONNECT strings override TNS-based resolution.
- Test connectivity using `sqlplus` before use.

11.9.18 ODBC Configuration

- `/home/aspweb/.odbc.ini` optional; inline connection strings override DSN.
- Important: The aspweb user requires read permissions to access the DSN file if using DSN mode.
- Test connectivity using `isql` or `tsql`.

11.9.19 MySQL Configuration

No additional drivers required.

11.10 Email inbox watcher

This action plug-in provides a way to trigger workflows based on the arrival of incoming packages in a IMAP inbox.

Version	Comment
0.4.2	Enhanced action template UI

Version	Comment
0.4.1	Updated the label from Keep ongoing to Continue Monitoring
0.4.0	Remove check on ruby version (only supporting one ruby on machine from now on), Updating to be compatible with Ruby 3.4 and Rails 8
0.3.2	Fix for occasional wrong hex characters in output
0.3.1	Fix for occasional missing output.
0.3.0	Add compatibility for special characters.
0.2.1	Add compatibility for multi-part emails.
0.2.0	Correct archiving functionality.
0.1.9	Improve use of download attachments functionality.
0.1.8	Update configuration panel and help file.
0.1.7	Use Mail gem instead of TMail (for 2+ compatibility).
0.1.6	Added styling to parameter help text.
0.1.5	Added help file.
0.1.4	Fixes for SSL configuration on Windows. Works on Linux too.
0.1.3	Fix ensure syntax on check_status
0.1.2	New icon
0.1.1	disconnect from IMAP server after a poll
0.1.0	added possibility to download attachments
0.0.8	exception handling of RemoteNode.find_by_name()
0.0.7	prevents autocomplete of user field
0.0.6	expose default port in UI as placeholder
0.0.5	monkey patching IMAP to avoid crash on empty box
0.0.4	flushes out deleted emails
0.0.3	fixes bug when using pre-defined nodes
0.0.2	adds functionality to mask password
0.0.1	Work in Progress - IMAP support only

11.10.1 How It Works

- Establishes a connection to the configured IMAP server (with optional SSL/TLS) using the provided credentials and mailbox folder.
- Periodically polls the specified mailbox at the configured polling interval to detect new or unread emails.
- Applies runtime filter conditions (for example, subject, sender, or date constraints) to determine which emails should trigger the workflow.
- When a matching email is detected, triggers the associated workflow and exposes email metadata (subject, body, sender, recipients, date, etc.) as outputs.
- Optionally downloads email attachments to a local directory on the Orchestrator server, automatically creating the directory if it does not exist.
- After processing, removes the email from the watched mailbox and either deletes it or moves it to the configured archive folder.

11.10.2 Configuration Instructions

1. Create a new action step using the EmailInboxWatcher plugin.
2. The plugin action waits until a message is found in a specified IMAP inbox.
3. Fill out the following fields:
    - Name: The name used to identify a saved EmailInboxWatcher configured instance.
    - Comments: Some comments about this saved EmailInboxWatcher configured instance.
    - Email Server Node: The node where the IMAP server is listening. The node should be of type IP. If no remote node is chosen, use the Advanced Parameters section.
    - IMAP Server Address: Advanced - The hostname or IP address of the IMAP server.
    - IMAP Server Port: Advanced - The port on the IMAP server.
    - IMAP User: The IMAP user account used to access emails.
    - IMAP Password: The password for the IMAP user account.
    - Use SSL?: Check this box to have the plugin use SSL (now TLS) to connect to the server.
    - Mailbox: The folder on the email client to be watched for new emails. Use INBOX or SENT to watch for all incoming or outgoing emails, respectively.
    - Filter Conditions: Conditions executed at runtime to specify which emails to trigger on.



Figure 11.10: Email inbox watcher icon

- Continue Monitoring: If checked, the work order will be cloned to continue looking for new emails.
- Download Attachments?: If checked, attachments will be downloaded. Downloaded files will have the same name as the email attachment. If a file with that name already exists in the directory, the new download will have a timestamp appended to it.
- Attachments Destination Folder: Location on the Orchestrator server where attachments are downloaded. The directory will be created if it does not already exist. Attachments are only downloaded if 'Download Attachments?' is checked. If this field is not specified, attachments will be downloaded to /opt/aspera/var/run/orchestrator/email_inbox_watchers/attachments.
- Archive Folder: If not filled in, triggered emails are deleted. If an email folder is specified here, any triggered emails will be moved to that folder. Regardless of whether this field is given a value, the email will be removed from the folder being watched (Mailbox).
- Encoding: Select the appropriate encoding. ISO-8859-2/Windows-1250 is recommended for Office 365 or Outlook when emails contain characters from Central or Eastern European languages. The 'None' option is recommended in all other cases.
- Polling Frequency: The time interval (in seconds) for how often the IMAP server will be polled.

11.10.3 Example Configuration

Example plugin configuration for a Gmail user.

This configuration will start an IMAP watcher to the email account 'user@gmail.com' and fetch all emails.

Parameter	Value
Name	Gmail IMAP
Comments	Sample configuration for Gmail IMAP Watcher
IMAP Server Address	imap.gmail.com
IMAP Server Port	993
Use ssl?:	Selected
IMAP User	user@gmail.com
IMAP Password	password
Mailbox	INBOX
Continue Monitoring	Selected

Example plugin configuration for a Outlook user.This configuration will start an IMAP watcher to the email account 'user@outlook.com' and fetch all emails.

Parameter	Value
Name	Outlook IMAP
Comments	Sample configuration for Outlook IMAP Watcher
IMAP Server Address	imap-mail.outlook.com
IMAP Server Port	993
Use ssl?:	Selected
IMAP User	user@outlook.com
IMAP Password	password

Parameter	Value
Mailbox	INBOX
Continue Monitoring	Selected

11.10.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved EmailInboxWatcher configured instance.
Comments	Some comments about this saved EmailInboxWatcher configured instance.
Email Server Node	The node where the IMAP server is listening. The node should be of type IP. If no remote node is chosen, use the Advanced Parameters section.
IMAP Server Address	Advanced - The hostname or IP address of the IMAP server.
IMAP Server Port	Advanced - The port on the IMAP server.
IMAP User	The IMAP user account used to access emails.
IMAP Password	The password for the IMAP user account.
Use SSL?	Check this box to have the plugin use SSL (now TLS) to connect to the server.
Mailbox	The folder on the email client to be watched for new emails. Use INBOX or SENT to watch for all incoming or outgoing emails, respectively.
Filter Conditions	Conditions executed at runtime to specify which emails to trigger on.
Continue Monitoring	If checked, the work order will be cloned to continue looking for new emails.
Download Attachments?	If checked, attachments will be downloaded. Downloaded files will have the same name as the email attachment. If a file with that name already exists in the directory, the new download will have a timestamp appended to it.
Attachments Destination Folder	Location on the Orchestrator server where attachments are downloaded. The directory will be created if it does not already exist. Attachments are only downloaded if 'Download Attachments?' is checked. If this field is not specified, attachments will be downloaded to /opt/aspera/var/run/orchestrator/email_inbox_watchers/attachments.
Archive Folder	If not filled in, triggered emails are deleted. If an email folder is specified here, any triggered emails will be moved to that folder. Regardless of whether this field is given a value, the email will be removed from the folder being watched (Mailbox).
Encoding	Select the appropriate encoding. ISO-8859-2/Windows-1250 is recommended for Office 365 or Outlook when emails contain characters from Central or Eastern European languages. The 'None' option is recommended in all other cases.
Polling Frequency	The time interval (in seconds) for how often the IMAP server will be polled.

11.10.5 Inputs Description

The list of inputs depends on the configuration of the EmailInboxWatcher action template. If dynamic attributes are used in the message definition, each attribute becomes a required input.

Parameter	Description
Email_server_address *	The hostname or IP address of the IMAP server
Email_server_po	The port on the IMAP server.
Filters_override	Conditions executed at runtime to specify which emails to trigger on.
Polling_frequency	The time interval (in seconds) for how often the IMAP server will be polled.

11.10.6 Outputs Description

Parameter	Description
To	Recipient's email address.
Cc	Any email addresses in the CC field.
Body	Body of the email.
Date	Date and time when email was sent.
Subject	Subject of email.
From	Sender's email address.
HTML_Body	HTML body of the email. If email does not use multi-part, this output will be the same as the Body output.
Id	Unique ID of the email.
Message_id	Unique ID of the email.
Reply_to	Email address to reply to.
Attachments	Names of the attachments of the email. Note that if attachments were downloaded, these names may differ from the names of the downloaded files. This will occur when a file with the same name as the attachment being downloaded already exists in the download folder. In that case, the newly downloaded file will have a timestamp appended to the name.

11.10.7 Supported Actions

None

11.10.8 Dependencies

None

11.10.9 Operating Instructions

How to Configure an IMAP Inbox Watcher?

To watch an IMAP server for emails, the following is required:

- Provide the IP address or hostname of the IMAP server.
- Ensure firewall access on the port mandated by the IMAP server, from Orchestrator to it.
- Use a valid IMAP user login and password.

Is POP Supported?

POP-based email servers are not supported at this time.

Deletion of Emails

If no archive folder is specified:

- Gmail will remove the email from the watched folder but keep it on the server.
- Other email clients, such as Outlook, will delete the email completely.

Encoding

Parsing some emails may fail if the email uses special encoding. In this case, select the appropriate encoding for the *Encoding* parameter. Note: Handling different encodings and the Encoding parameter is not available for Orchestrator running Ruby 1.8.7.

Troubleshooting Tips

Not using SSL when it is mandated (or vice versa) usually results in errors for the plugin. Contact the system administrator for IMAP configuration.

Some supported filter expressions:

- FROM alex@ibm.com
- SUBJECT ”hello”



Figure 11.11: Faspex5 inbox watcher icon

11.11 Faspex5 inbox watcher

This plugin monitors Faspex5 server mailboxes for incoming packages matching specified criteria and triggers workflows, providing comprehensive package information including metadata, file lists, and transfer specifications.

Version	Comment
0.8.6	Enhanced action template UI
0.8.5	Improved error handling during reauthentication and API operations
0.8.4	Add configurable transfer_type field to support both 'connect' and 'transfer_client' transfer types
0.8.3	Updated the label from Keep ongoing to Continue Monitoring
0.8.2	Add optional checkbox to control transfer_spec_per_package execution
0.8.1	Fix issue when there are multiple recipients and the faspex user is a non admin user
0.8.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.7.1	Remove upload_completion_time field from persistance logic
0.7.0	Add package id as input
0.6.0	Supports package download links.
0.5.3	Watching globally for packages.
0.5.2	Fix issue when the refresh token has expired or the access_token expires among calls to faspex
0.5.1	Fix for shared inbox name as run time input
0.5.0	Added destination root and fixed missing code for token renewal
0.4.5	Improve Authentication logic
0.4.4	Improved package outputs and fixed issue with filters. Added workaround for token renewal
0.4.3	Improve Authentication logic and fix case baseline when there are not packages to retrieve
0.4.2	Improve package_location details
0.4.1	Improve plugins architecture
0.4.0	Adds support for filters key
0.3.1	Adds support for private key
0.3.0	Adds support for public key
0.2.1	Fixes with the Shared Inboxes Support
0.2.0	Shared Inboxes Support
0.1.0	This is based of Faspex inbox watcher version 1.5.2

11.11.1 How It Works

- Monitors Faspex 5 server inboxes for new packages matching specified criteria and triggers workflow instances.
- Connects to Faspex 5 using SSH private keys and API client credentials for secure authentication.
- Supports various mailbox types including Global All, My Packages, All Packages, or specific shared inboxes.
- Provides comprehensive package information including metadata, file lists, transfer specs, and download links.
- Includes flexible filtering capabilities with general search terms and custom filter conditions.
- Offers trigger persistence options to prevent duplicate processing across workflow instances.

11.11.2 Configuration Instructions

1. Add a Faspex5InboxWatcher step to your workflow.
2. Configure the Faspex 5 connection settings:

- Faspex Node: Select the remote node where the Faspex server is installed.
- Faspex Service Address: The full URL or hostname if not using a remote node.
- Private key: SSH private key in PEM format for authentication.
- Faspex User: Username for the Faspex account to monitor.
- API Client ID: The client ID from the Faspex server API configuration.

3. Set up the monitoring parameters:
   - Mailbox: Choose Global All, My Packages, All Packages, or Shared Inbox.
   - Polling Frequency: Interval for checking new packages.
   - Limit: Maximum number of packages to return (default 10).
   - Continue Monitoring: Enable for continuous monitoring.

11.11.3 Example Configuration

Parameter	Example Value
Faspex Node	faspex5-server
Faspex Service Address	https://faspex5.company.com
Private key	
Packages root	/opt/aspera/var/packages
Faspex User	orchestrator@company.com
API Client ID	orchestrator-client-12345
General Search	urgent OR priority
Limit	20
Polling Frequency	60
Mailbox	My Packages
Continue Monitoring	checked
Trigger persistence scope	by workorder group
Get Package Download Link	checked
Get Transfer Spec	checked

11.11.4 Saved Parameters Description

Parameter	Description
Faspex Node	The node where the Aspera Faspex server is installed. If a remote node is not configured, use the Advanced parameters section.
Faspex Service Address	Advanced - The full URL or the hostname of the Faspex server, if not configured as Remote Node.
Private key	Private key in PEM format for authenticating with the Faspex server.
Packages root	The mount point on the Faspex server or the Enterprise server where Faspex packages are stored. This value can be found in aspera.conf on the Enterprise server linked to Faspex.
Faspex User	The Faspex user account to monitor for packages.
API Client ID	The API Client ID configured in the Faspex server.
General Search	String applied to the title, recipients, senders, and note of the packages for filtering.
Filter conditions	Filter conditions executed at run-time on incoming packages for advanced filtering.
Limit	The maximum number of resources to return. Default is 10.
Polling Frequency	Determines how often Faspex should be polled for new packages.
Mailbox	Choose from Global All, My Packages, All Packages or a Shared Inbox to monitor.
Trigger persistence scope	Sets the scope/availability of the trigger output. Options: 'by workorder group', 'by action template', 'absolute', 'by step', 'by workflow', 'none'.
Continue Monitoring	If set, the work order will be cloned to continue looking for new Faspex packages.

Parameter	Description
Trigger before package is fully uploaded?	If set, Orchestrator will trigger on in progress transfers.
Ignore packages already in inbox on start-up?	If set, Orchestrator will ignore the current Faspex packages in the mailbox.
Ignore faspex failures?	If set, Orchestrator will continue to poll Faspex when Faspex is unavailable instead of failing the step.
Ignore empty metadata values?	If set, Orchestrator will ignore empty metadata and not populate the metadata hash with those keys.
Get Package Download Link	Specifies whether to return the package download link. The link is available if the package's recipient definition public_download is true.
Get Transfer Spec	Specifies whether to retrieve the transfer specification for the package. (Enabled by default) The transfer_spec output will contain the package's transfer specification in JSON format.
Transfer Type	Specifies the transfer type to use when retrieving transfer specifications. Options: 'Connect' (default) or 'Transfer Client'. Both options work with Faspex 5.0.16+. 'Connect' will be deprecated in future Faspex versions. 'Transfer Client' is recommended for new configurations.

11.11.5 Inputs Description

- Faspex_server_address: Faspex server address or service address or remote node name. Overrides configuration value.
- Faspex_user: Faspex user's login username. Required if a node is not present. Overrides configuration value.
- Faspex_api_client_id: Faspex API client ID. Required if a node is not present. Overrides configuration value.
- Filter conditions variables: Filter conditions variables executed at run-time on incoming packages. Overrides configuration value.

11.11.6 Outputs Description

- title: Title of the Faspex package.
- id: UUID of the Faspex package.
- released: Date and time when the Faspex package was sent.
- updated: Date and time when the Faspex package was updated.
- status: Status of the Faspex package whether it is open, unopened, completed or failed.
- author_name: Faspex sender's login username.
- author_email: Faspex sender's email address.
- metadata: The Faspex tags for this package. They include information on the sender like first and last names, email address, etc.
- recipient_name: Faspex recipient's login username.
- recipient_email: Faspex recipient's email address.
- location: Faspex package's absolute file path, if package root was provided.
- file_list: List of file names included in the Faspex package.
- transfer_spec: Transfer specification for the package.
- message: Faspex package message content.
- ear_enabled: True if the package has Encryption At Rest enabled.
- package_uuid: Faspex package UUID identifier.
- completed_download_count: Faspex package complete download count.
- completed_partial_download_count: Faspex package partial download count.
- active_download_count: Faspex package active download count.
- expiration_policy: Faspex package expiration policy settings.
- package_download_link: Faspex package download link if available.

11.11.7 Trigger Persistence Scope Details

- 'by workorder group' - Trigger packages are tracked while the workorder is executing, and it will only be trapped once for a workorder group. If the workorder is restarted (new workorder group), all past triggers will be detected again if still matching the search criteria.
- 'by workflow' - Same as 'by workorder group' except no triggering will occur upon a 'restart' action for the same workflow.
- 'by action template' - Allows the tracking and thus non-triggering in another workflow for previously processed trigger files shared across workflows via the same template.
- 'absolute' - The trigger list is tracked regardless of where it occurs.
- 'by step' - Similar to 'by workflow', but allows for multiple independent trigger steps within the workflow.
- 'none' - No persistence at all. If a past trigger package is still present, it will re-trigger if 'Continue Monitoring' is selected. This option is recommended when 'Continue Monitoring' is not selected.

11.11.8 Operating Instructions

11.11.9 Orchestrator Instance Setup

- Generate a new SSH key pair in PEM format exclusively for Faspex interaction:

  `ssh-keygen -t rsa -b 4096 -m PEM -C "your_email@example.com"` (save as key_for_faspex.pub)

- Generate the PEM format version for the public key:

  `ssh-keygen -f key_for_faspex.pub -m 'PEM' -e > key_for_faspex.pub.pem`

- In HA environments, share the new private key among all Orchestrator instances.
- For Faspex 5.0.4+ versions, enable the parameter 'allow_global_key': 'true' or 'Allow signing with global key'.

Note: Newer versions of OpenSSH default to generating ED25519 keys rather than RSA. If you encounter the error *do_convert_to_pem: unsupported key type ED25519* when exporting the public key to PEM format, it means an ED25519 key was generated, which is not supported for PEM conversion. Make sure to use the `-t rsa` flag as shown above to explicitly generate an RSA key.

11.11.10 Faspex Server Configuration

API Client Configuration:

- Go to `/admin/configurations/api-clients` page and create a new API Client
- Enable JWT grant type
- Enable 'Allow refresh token' toggle (Faspex 5.0.6 and above versions)
- Configure 'Access token expiration (hours) (optional)' (Faspex 5.0.6 and above versions)
- Configure 'Refresh token expiration (hours)' (Faspex 5.0.6 and above versions)
- In the 'Authentication' section enable 'Global key' toggle and paste the orchestrator public key in PEM format
- In the 'Access' section enable 'All users' in case you want all the F5 users to use this client to access the API

11.11.11 Required Configuration Parameters

- The faspex service address or a Faspex Node
- SSH private key generated exclusively to interact with Faspex
- Faspex user - the user to impersonate
- Faspex API client ID - the client ID generated from Faspex
- Mailbox - If Shared Inbox is selected, a valid shared inbox name must be provided

11.11.12 Testing and Validation

- Test connectivity: Create an Orchestrator remote node (Type as IP) to the Faspex server with the IP address or hostname and TCP port as 443. Test TCP connectivity.
- Test configuration: After logging into the Faspex server, if packages are present in the user's mailbox then Orchestrator should be able to trigger on the package.

11.11.13 Dependencies

- The Faspex 5 server must be accessible from the Orchestrator system on TCP port 443.
- A valid SSH private key in PEM format is required for authentication.
- The API client must be properly configured in Faspex 5 with JWT grant type enabled.
- The aspweb user must have read permissions for the SSH private key file used for Faspex 5 authentication.
- If using packages root parameter, the aspweb user must have read permissions for the package storage location.

11.11.14 Troubleshooting

- Packages not triggered: Without valid credentials (user, client_id), packages are not detected. Verify all authentication parameters are correct.
- Permission issues: Without download permissions on the package, packages are not detected. Ensure the Faspex user has appropriate access rights.
- Connectivity failures: If TCP connectivity test fails, contact the server administrator to verify network access and firewall settings.
- Authentication errors: Verify that the SSH private key is in correct PEM format and matches the public key configured in the Faspex API client.
- API client configuration: Ensure JWT grant type is enabled and the global key is properly configured with the correct public key.
- Shared inbox access: If monitoring a shared inbox, verify that the specified shared inbox name exists and the user has access permissions.
- Package filtering: If expected packages are not being detected, review the General Search and Filter conditions settings for accuracy.
- Transfer type compatibility: If using Faspex 5.0.15 or earlier, use 'Connect' transfer type. For Faspex 5.0.16+, both 'Connect' and 'Transfer Client' are supported. 'Transfer Client' is recommended for future compatibility.
- For Faspex 5.0.4 and later, confirm that 'allow_global_key' is set to true in the server settings.

11.12 Faspex5 package monitor



Figure 11.12: Faspex5 package monitor icon

Provides the capability to monitor Faspex 5 package uploads, downloads, and completion status.

Version	Comment
0.2.2	Enhanced action template UI
0.2.1	Improved error handling during reauthentication after access token expiry
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.1	Adds Package completed operation.
0.1.0	Supports package download links.
0.0.3	Fixed internal input mapping
0.0.2	Improve authentication service logic
0.0.1	Based on faspex_package_monitor 0.0.4

11.12.1 How It Works

- Monitors Aspera Faspex 5 packages for various completion states including upload completion and download events.

- Connects to Faspex servers using JWT-based API authentication with SSH private key signing.
- Supports multiple monitoring operations: Upload done, All users download, Any user download, and Package completed.
- Provides configurable polling frequency for real-time package status monitoring.
- Returns detailed package information including downloader lists, download details, and optional download links.

11.12.2 Configuration Instructions

1. Add a Faspex5PackageMonitor step to your workflow.
2. Configure the Faspex server connection:
   - Name: Enter a descriptive name for the package monitor.
   - Faspex Node: Select configured Faspex node or leave empty for manual configuration.
   - Faspex Server Address: Full URL or hostname of Faspex service (if not using Faspex Node).
   - SSH Private key: Private key in PEM format for JWT authentication.
   - Faspex User: Username to impersonate for package monitoring.
   - Faspex API Client ID: API Client ID from Faspex Admin → Configurations → API clients.
3. Set up monitoring parameters:
   - Operation: Choose monitoring condition (Upload done, All users download, Any user download, Package completed).
   - Delivery ID: Package ID to monitor or leave empty to use Delivery URL.
   - Delivery URL: Package URL to extract delivery ID from (e.g., https://server/aspera/faspex/sent/1293).
   - Polling frequency: Interval in seconds for server polling.
   - Get Package Download Link: Check to return package download link (if public_download is enabled).

11.12.3 Example Configuration

Parameter	Example Value
Name	Package Upload Monitor
Comments	Monitors package upload completion
Faspex Node	faspex-server-01
SSH Private key	
Faspex User	monitor_user
Faspex API Client ID	orchestrator-client-id
Polling frequency	30
Delivery ID	1293
Operation	Upload done
Get Package Download Link	checked

11.12.4 Saved Parameters Description

Parameter	Description
Name	Display name for this package monitor step.
Comments	Notes or description for this step.
Faspex Node	Configured node where Aspera Faspex server is installed. Leave empty to use Advanced parameters.
Faspex Server Address	Full URL or hostname of Faspex service when not using configured Remote Node.
SSH Private key	Private key in PEM format for JWT-based API authentication.
Faspex User	Faspex username to impersonate for package monitoring operations.
Faspex API Client ID	API Client ID obtained from Faspex Admin → Configurations → API clients.
Polling frequency	Interval in seconds for polling server to check package status.
Delivery ID	Specific package delivery ID to monitor.
Delivery URL	Package URL from which delivery ID will be extracted (e.g., https://10.0.98.3/aspera/faspex/sent/1293 extracts ID 1293).

Parameter	Description
Operation	Monitoring condition: Upload done (wait for upload completion), All users download (wait for all recipients to download), Any user download (wait for at least one download), Package completed (wait for package completion).
Get Package Download Link	Enable to return package download link in outputs (available only if package recipient definition has public_download enabled).

11.12.5 Inputs Description

- Faspex_server_address: Address or hostname of the Faspex server.
- Faspex_user: Faspex username for authentication and user impersonation.
- Faspex_api_client_id: API Client ID from Faspex Admin → Configurations → API clients (locate Orchestrator client and find Client ID value).
- delivery_id: Package delivery ID to monitor. Can be obtained from Faspex5InboxWatcher plugin output or extracted from package URL (e.g., 480 from https://10.0.98.3/aspera/faspex/packages/all/480).

11.12.6 Outputs Description

- downloader_list: Array of usernames of individuals who have downloaded the package.
- download_details: Hash containing package download details including start time, stop time, status, and completion percentage.
- step_information: Hash containing comprehensive package information including start time, stop time, status, and completion percentage.
- Package Download Link: Package download link (available only when Get Package Download Link is enabled and package has public_download enabled).

11.12.7 Supported Actions

- Upload done
- All users download
- Any user download
- Package completed

11.12.8 Dependencies

- Aspera Faspex 5 server with API client properly configured.
- SSH key pair in PEM format for JWT authentication.
- The aspweb user must have proper permissions to read the SSH private key file.
- Network connectivity to Faspex server for API communication.
- Faspex 5.0.4+ requires 'allow_global_key' parameter enabled or 'Allow signing with global key' setting.

11.12.9 Operating Instructions

11.12.10 Prerequisites Setup

11.12.11 Orchestrator Instance Requirements

Generate a new SSH key pair in PEM format exclusively for Faspex interaction:

- Create SSH key: `ssh-keygen -t rsa -b 4096 -m PEM -C "your_email@example.com"` (save as key_for_faspex.pub)
- Generate PEM format public key: `ssh-keygen -f key_for_faspex.pub -m 'PEM' -e > key_for_faspex.pub.pem`
- HA environment: Share the new private key among all Orchestrator instances
- Faspex 5.0.4+: Enable 'allow_global_key': 'true' or 'Allow signing with global key' parameter

Note: Newer versions of OpenSSH default to generating ED25519 keys rather than RSA. If you encounter the error *do_convert_to_pem: unsupported key type ED25519* when exporting the public key to PEM format, it means an ED25519 key was generated, which is not supported for PEM conversion. Make sure to use the `-t rsa` flag as shown above to explicitly generate an RSA key.

11.12.12 Faspex Server Configuration

Configure API client in Faspex server at `/admin/configurations/api-clients`:

- Create new API Client and enable JWT grant type
- Enable 'Allow refresh token' toggle (Faspex 5.0.6+)
- Configure 'Access token expiration (hours)' (Faspex 5.0.6+)
- Configure 'Refresh token expiration (hours)' (Faspex 5.0.6+)
- In Authentication section: enable 'Global key' toggle and paste Orchestrator public key in PEM format
- In Access section: enable 'All users' to allow all F5 users to use this client for API access

11.12.13 Required Configuration Parameters

- Faspex server address or Faspex Node: Server connection information
- SSH private key: Private key generated exclusively for Faspex interaction
- Faspex user: Username to impersonate for package monitoring
- Faspex API client ID: Client ID generated from Faspex 5 configuration
- Operation: Monitoring condition to watch for

11.12.14 Operation Behavior

- Upload done: Monitor waits until searched package is fully uploaded. Upload progress shown as percentage in step status during upload.
- All users download: Monitor waits until all destination users have downloaded the searched package.
- Any user download: Monitor waits until at least one destination user has downloaded the searched package.
- Package completed: Monitor waits for overall package completion status.

11.12.15 Important Notes

- Outputs downloader_list and download_details will be empty if no destination user downloaded the package.
- Plugin returns 404 Not Found if input package delivery ID does not match an existing package.
- Downloads are counted only if fully completed. Failed downloads are not counted; monitor continues waiting until successful download.
- Package delivery ID can be obtained from Faspex5InboxWatcher plugin output or extracted from package URL.

11.13 FTP Folder Trigger



Figure 11.13: FTP Folder Trigger icon

This action provides the ability to detect folders on a remote node via the FTP protocol.

Version	Comment
0.1.2	Enhanced action template UI
0.1.1	Updated the label from Keep ongoing to Continue Monitoring
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.2	Adding Remote Node Dependency
0.0.1	Initial Commit

Version	Comment

11.13.1 How It Works

- Monitors a remote FTP server directory for folders matching a specified pattern.
- Triggers workflow execution when matching folders are detected.
- Supports regular expression patterns for flexible folder matching.
- Can scan subdirectories recursively or only the specified directory.
- Configurable maximum number of folders returned per trigger (default: 1, 0 = unlimited).
- Operates in continuous monitoring mode with 'Continue Monitoring' or one-time check with 'Check once'.
- Supports both passive and active FTP modes.
- Provides trigger persistence scopes to track previously processed folders and prevent re-triggering.
- Optional reverse DNS lookup disable for improved performance in certain network configurations.
- Returns triggered folder name(s) as output for use in subsequent workflow steps.

11.13.2 Configuration Instructions

1. Create a new action step using the FtpFolderTrigger plugin.
2. Fill out the following fields:
    - Name: The name used to identify a saved FTP Folder Trigger configured instance.
    - Comments: Comments about this saved FTP Folder Trigger configured instance.
    - Remote Node: Selector for the remote system containing the FTP server. Choose from configured Remote Nodes.
    - Remote Server: Remote server hostname or IP address containing the FTP server, if not using the 'Remote node' selector.
    - Remote Server Port: The FTP command port, normally 21 on the FTP server system.
    - Remote User: Username required on the FTP server for authentication.
    - Remote Password: FTP server password associated with the username.
    - Remote Directory: Directory on the FTP server searched for the existence of folder(s) matching the pattern.
    - Trigger foldername pattern (regex): The regular expression pattern used for folder triggering.
    - Scan subdirectories recursively?: If checked, subdirectories will be scanned recursively for the provided pattern.
    - Maximum number of folders returned: Default is 1. Set to 0 for unlimited (returns all matching folders).
    - Polling Frequency: Frequency in seconds to poll the remote FTP server for new folders.
    - Disable Reverse DNS lookup: Disable Reverse DNS lookup, which can slow down FTP listing in certain network configurations.
    - Transfer in passive mode: Determines if the transfer should be in passive or active mode.
    - Check once?: If checked, the plugin checks for existence of folder(s) matching the pattern and exits (does not continue monitoring).
    - Continue Monitoring: If checked, the plugin re-triggers after a successful trigger, continuously monitoring for new folders.
    - Trigger persistence scope: Sets the scope/availability of the trigger output. Options: 'By workorder group', 'By action template', 'absolute', 'By step', 'By workflow', 'None'.

11.13.3 Example Configuration

This is an example configuration for the FtpFolderTrigger plugin watching for daily batch folders.

Parameter	Value
Name	Watch for Incoming Batch Folders
Remote Node	IncomingFTP
Remote Directory	incoming/batches
Trigger foldername pattern (regex)	batch_\d{8}
Scan subdirectories recursively?	Not selected
Maximum number of folders returned	1
Polling Frequency	60
Transfer in passive mode	Selected
Continue Monitoring	Selected
Trigger persistence scope	By workorder group

11.13.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved FTP Folder Trigger configured instance.
Comments	Comments about this saved FTP Folder Trigger configured instance.
Remote Node	Selector for the remote system containing the FTP server. Choose from configured Remote Nodes.
Remote Server	Remote server hostname or IP address containing the FTP server, if not using the 'Remote node' selector.
Remote Server Port	The FTP command port, normally 21 on the FTP server system.
Remote User	Username required on the FTP server for authentication.
Remote Password	FTP server password associated with the username.
Remote Directory	Directory on the FTP server searched for the existence of folder(s) matching the pattern.
Trigger foldername pattern (regex)	The regular expression pattern used for folder triggering.
Scan subdirectories recursively?	If checked, subdirectories will be scanned recursively for the provided pattern.
Maximum number of folders returned	Default is 1. Set to 0 for unlimited (returns all matching folders).
Polling Frequency	Frequency in seconds to poll the remote FTP server for new folders.
Disable Reverse DNS lookup	Disable Reverse DNS lookup, which can slow down FTP listing in certain network configurations.
Transfer in passive mode	Determines if the transfer should be in passive or active mode.
Check once?	If checked, the plugin checks for existence of folder(s) matching the pattern and exits (does not continue monitoring).
Continue Monitoring	If checked, the plugin re-triggers after a successful trigger, continuously monitoring for new folders.
Trigger persistence scope	Sets the scope/availability of the trigger output. Options: 'By workorder group', 'By action template', 'absolute', 'By step', 'By workflow', 'None'.

11.13.5 Inputs Description

Changes in this list of inputs will override any values set in the action template.

Parameter	Description
remote_directory	Directory on the FTP server below the default FTP directory. Relative to the user's root directory.
remote_password	FTP password on the remote server (overrides template value if provided).
remote_user	FTP username on the remote server (overrides template value if provided).
remove_after_transfer	Remove the source file upon successful completion of the transfer (not typically used for folder triggers).
check_folders_recursively	Boolean flag to scan subdirectories recursively for the provided pattern.

11.13.6 Outputs Description

Parameter	Description
trigger_folder	The folder name which matches the pattern provided, on which the plugin triggered (returned when maximum folders = 1)
trigger_folders	Array of folder names which match the pattern provided, on which the plugin triggered (returned when maximum folders > 1)

11.13.7 Supported Actions

None

11.13.8 Dependencies

None

11.13.9 Operating Instructions

This plugin provides a configurable interface to watch a remote folder on an FTP server. The plugin triggers only on folders present that match the specified pattern. Passive/active modes are supported. Username and password authentication, if required at the server, is also supported.

Prerequisites

- FTP server must have port 21 accessible from Orchestrator.
- The folders in the searched directory must have read permissions for the FTP user.
- For passive mode, ensure passive data port range is accessible through firewalls.
- FTP user account must have sufficient permissions to list directory contents.

Basic Workflow

- Identify the remote FTP server and configure a Remote Node, or place the server details in the advanced parameters section.
- If username and password are required, place them in the Remote Node or advanced parameters settings.
- The FTP server provides a root directory for each connection. Subdirectories should be entered in the 'Remote Directory' field.
- Important path note: If the FTP user root directory is `/root/` and you want to search under `/root/dir1/`, specify `dir1/` in Remote Directory (not `/dir1/`).
- Select a regular expression pattern that matches your target folders.
- Select the appropriate persistence scope for your use case.
- Check 'Continue Monitoring' if the trigger needs to continuously monitor the remote FTP folder.
- Do not check 'Continue Monitoring' when 'Check once' is selected.

Passive vs Active Mode

- Passive Mode (Recommended):
  - FTP server opens data connection to client
  - Works better with firewalls and NAT
  - Client doesn't need inbound firewall rules
  - Most modern FTP servers support passive mode
  - Use when: Orchestrator is behind firewall or NAT
- Active Mode:
  - FTP client opens data connection from server
  - Requires inbound firewall rules on client side
  - Traditional FTP mode
  - May have issues with NAT/firewalls
  - Use when: Direct connection without firewall restrictions

Common Use Cases

- Batch processing: Trigger when new batch folders arrive on FTP server for processing.
- Partner integration: Monitor FTP drop folders from external partners or vendors.

- Archive processing: Detect archive folders ready for restoration or processing.
- Multi-tenant workflows: Trigger separate workflows for different client folders.
- Date-based processing: Trigger on daily folders (e.g., 'YYYYMMDD' pattern) for time-series processing.
- Workflow staging: Detect when staging folders are ready for promotion to production.
- Conditional processing: Use folder naming conventions to route to different workflows.

Best Practices

- Use 'By workorder group' persistence for most use cases to prevent duplicate processing.
- Use 'None' persistence only with 'Check once' mode or when folders are removed after processing.
- Set appropriate polling frequency - balance between responsiveness and server load (e.g., 60-300 seconds).
- Use passive mode unless you have specific reasons to use active mode.
- Create specific regex patterns - overly broad patterns may trigger on unexpected folders.
- Test regex patterns on sample folder names before deployment.
- Use 'Maximum number of folders' = 1 for sequential processing, or higher values for batch processing.
- Enable 'Disable Reverse DNS lookup' if FTP directory listings are slow.
- Use Remote Node configuration for credentials rather than direct parameters.
- Implement downstream folder removal or renaming to prevent re-triggering.

Troubleshooting

- Connection timeout: Verify FTP server is reachable, port 21 is accessible, and firewall rules allow connections.
- Authentication failed: Verify username and password are correct. Check if account is locked or expired.
- Directory not found: Verify Remote Directory path is correct relative to FTP root. Don't include leading slash for relative paths.
- No folders detected: Verify folders exist, regex pattern is correct, and FTP user has read permissions.
- Passive mode failure: Ensure passive data port range is accessible. Try active mode if passive fails.
- Slow directory listing: Enable 'Disable Reverse DNS lookup' option to improve performance.
- Folders re-triggering: Check persistence scope setting. Use 'By workorder group' or stricter scope.
- Regex not matching: Test pattern with online regex tester. Remember to escape special characters.
- Trigger not continuous: Ensure 'Continue Monitoring' is checked and 'Check once' is NOT checked.

Important Notes

- The Remote Directory path is relative to the FTP user's root directory, not the server filesystem root.
- If FTP root is `/ftproot/` and you want `/ftproot/incoming/`, specify `incoming/` (not `/incoming/`).
- 'Continue Monitoring' and 'Check once' are mutually exclusive - do not enable both.
- Maximum number of folders 0 means unlimited - all matching folders will be returned.
- Maximum number of folders 1 returns 'trigger_folder' (string), >1 returns 'trigger_folders' (array).
- Persistence scope 'None' with 'Continue Monitoring' will continuously re-trigger on same folders - only use if folders are removed.
- Passive mode typically uses port 21 for control and high-numbered ports for data.
- Recursive scanning can be resource-intensive on directories with many subdirectories.
- Polling frequency affects detection latency and server load - find appropriate balance.

11.14 FTP Trigger



Figure 11.14: FTP Trigger icon

This action enables the detection of files on a remote node via the FTP protocol.

Version	Comment
2.2.2	Enhanced action template UI
2.2.1	Updated the label from Keep ongoing to Continue Monitoring
2.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
2.1.4	Fixes a potential infinite loop in case of recursive scan
2.1.3	Fixes trigger_filename output when multiple remote folders are scanned
2.1.2	Improves regex of line formats
2.1.1	Fixed space usage when persistency scope set to NONE
2.1.0	Added the option to scan multiple remote folders
2.0.9	Added the option to specify a custom persistence scope
2.0.8	Default get for remote directory
2.0.7	Added support for files with leading or trailing spaces
2.0.6	Check pattern matching before checking file size 0
2.0.5	Adding remote Node dependency
2.0.4	Added persistency call in case nothing has been detected. This is needed otherwise at the next loop after a detection the history is not updated correctly
2.0.3	Fixed problem with moved files path in outputs
2.0.2	Bug Fix - Incorrect variable was referenced.
2.0.1	Fixed problem when using recursion and including folders
2.0.0	Added option to disable reverse dns lookup
1.9.3	Create directory when importing
1.9.2	Ignore base directory if folder is not moved and if searching recursively
1.9.1	Fix for nil object in find_is_dir_and_name
1.9.0	Modifications to recursion, fix a bug where ftp.list(*) would cause scan to break
1.8.4	Added comment in help about regex pattern matching and remote dir path
1.8.3	Fixed a bug where FtpTransfer was being called instead of FtpTrigger
1.8.2	2015-06-26 CT - Added file sizes outputs
1.8.1	2015-04-29 ADM - Fixed regression introduced in 1.8.0 for ignore_folder when no recursion.
1.8.0	2015-04-29 ADM - Added possibility to specify a time based suffix when moving the source file. Fixed support for MS-FTP servers.
1.7.1	2015-04-29 ADM - Added additional output in case of move after trigger option
1.7.0	2015-04-28 ADM - Added possibility to add a suffix when moving the source file
1.6.1	2015-04-28 ADM - Fix output path when no recurse is selected.
1.6.0	2015-04-28 ADM - Add option to ignore folder from result list when no recursion is selected
1.5.4	2015-04-28 ADM - Fix problem in case of simple search (no recursion) and the triggering expression matches a folder name
1.5.3	2015-02-27 ML - New icon
1.5.2	01/27/2015 - ADM - Fix move to folder after processing behavior
1.5.1	Expose timeout configuration in editor
1.5.0	Added ignore zero bytes files
1.4.1	Reduced debug log
1.4.0	Made timeout configurable
1.3.4	Added timeout for listing files in the ftp server. Work around blocking ftp servers
1.3.3	Close the ftp session in case of errors to avoid running out of resources
1.3.2	Added debug lines for freezing connections
1.3.1	Fix issue when importing a file
1.3.0	Bumps version to ensure check_once migration is run
1.2.1	Prevents autocomplete of user field
1.2.0	FTP Scan checks sub-dirs recursively if the check_folders_recursively flag is checked
1.1.2	FTP Scan continues when polling fails but simply returning an UNDEFINED status. Added basic help file.
1.1.1	Adds K2 News format (files don't always have size)
1.1.0	Adding functionality to move the trigger file to another remote FTP directory after successful trigger (and after download)
1.0.0	Simplified to use generic trigger persistence (requires Orchestrator 2.0), Adds check for port at start and default to 21
0.2.7	Removing reference to an non-existent calculateSize method
0.2.6	Ignore timestamps when pruning trigger list when trigger on partial is enabled
0.2.5	Modifies standard stability check to include timestamp
0.2.4	Adds display name
0.2.3	Hides password characters while entering in edit GUI.

Version	Comment
0.2.2	adds alternate line formats, fixes glitch in persistence by action template
0.2.1	writes past triggers to temp file to avoid risk of overwriting with incomplete file
0.2.0	adds trigger persistency scope
0.1.8	fixes issue when no limits is set for the files returned
0.1.7	excludes standard dir like . and ..
0.1.6	import reporting frequency is now timebased
0.1.5	fixes bug with self contained remote server
0.1.4	cleans up debugging traces
0.1.3	removes duplicate triggering when files are left on server
0.1.2	catches permission error 550 when no files match the pattern on some servers
0.1.1	uses node user if not explicitly provided
0.1.0	implements trigger on partial
0.0.2	fixes outputs when returning arrays of file
0.0.1	functional release with no proxy -supports get, put, delete. Use default port

11.14.1 How It Works

- This plugin provides a configurable interface to monitor a remote folder on an FTP server.
- It supports triggering on partial, single, or multiple files. It also imports the trigger file.
- Both passive and active modes, as well as binary and text modes are supported.
- If required by the server, username and password authentication is also supported.

11.14.2 Configuration Instructions

1. Create a new action step using the FtpTrigger plugin.
2. Fill out the following fields:
    - Name: The identifier used for a saved FtpTrigger configuration instance.
    - Comments: Optional notes about the saved FtpTrigger configuration.
    - Remote Node *: Selector for the remote system hosting the FTP server.
    - Remote Server: Name of the remote server with the FTP service. Used if ”Remote Node” is not selected.
    - Remote Server Port: FTP command port, typically ”21”, on the server system.
    - Remote User: Username required for authentication on the FTP server.
    - Remote Password: Password for the specified FTP user; displayed in plain text.
    - Remote Directory: Directory on the FTP server to search for files matching the pattern.
    - Trigger filename pattern (regex): Regex pattern used to detect trigger files.
    - Scan subdirectories recursively?: If enabled, subdirectories are scanned recursively for matching files.
    - Ignore folders: If enabled, subdirectories are excluded from results. Applies only when recursive scanning is disabled.
    - Ignore zero bytes files: If enabled, files with 0 bytes will not trigger notifications.
    - Maximum number of files returned: Default is 1; setting to 0 means no limit.
    - Polling Frequency: Interval (in seconds) for polling the remote FTP server.
    - Disable Reverse DNS lookup: Disables reverse DNS lookup, which may improve FTP listing speed.
    - Trigger on partial files: If enabled, triggers will occur even for incomplete files.
    - Import directory: If left blank, files remain on the FTP server; otherwise, files are downloaded.
    - Transfer in passive mode: Specifies whether to use passive or active FTP mode.
    - Transfer in binary mode: Determines whether files are transferred in ASCII or binary mode. Use ASCII for text, binary for non-text files.
    - Remove source file after transfer: Deletes the source file after a successful transfer.
    - Move source file after trigger: Moves triggered files to a specified directory relative to the watch folder.
    - Moved file suffix: Optional suffix added to moved files; valid only if file move is enabled.
    - Use automatic suffix: If enabled, adds a timestamp as a suffix when moving files.
    - Check once?: If enabled, checks for matching files once and exits.
    - Continue Monitoring: If enabled, the plugin continues triggering after each successful detection.
    - Trigger persistence scope: Defines the scope of trigger output. Default is 'By workorder group'. Options include: 'By workorder group', 'By action template', 'Absolute', 'By step', 'By workflow', 'None'.
    - Transfer block size: Size of data chunks transferred from application to kernel memory during FTP operations.
    - Reporting increment: Interval (in seconds) for reporting progress during the step.

11.14.3 Example Configuration

Example 1: Within the remote server track text files:

- Track example directory `ftpuser`
- Use the following Trigger filename pattern (regex) value `.*\.txt`

If a file with a `.txt` extention gets created at `ftpuser` in the remote server, the plugin will complete

- Ensure the details provided are correct and accurate
- Select the desired type of action to trigger the response

11.14.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved FtpTrigger configured instance.
Comments	Some comments about this saved FtpTrigger configured instance.
Remote Node	Selector for the remote system containing the FTP server.
Remote Server	Remote server name containing the FTP server, if not using the ”Remote node” selector.
Remote Server Port	The ftp command port normally ”21” on the FTP server system.
Remote User	User name required on the FTP server for authentication.
Remote Password	FTP server password associated with the fore mentioned user name. Note this is shown in clear text.
Remote Directory	Directory on the FTP server searched for the existence of file(s) matching the pattern.
Trigger filename pattern (regex)	The pattern used for file triggering.
Scan subdirectories recursively?	If checked, sub-dirs will be scanned recursively for provided pattern.
Ignore folders	If checked, sub-dirs will not be included in the result. Only apply when Scan sub directories recursively is not selected.
Ignore zero bytes files	If set, the trigger will not notify on files that are 0 bytes.
Maximum number of files returned	Default is 1. 0 indicates no maximum.
Polling Frequency	Frequency (in seconds) to poll the remote FTP server.
Disable Reverse DNS lookup	Disable Reverse DNS lookup, which can slow down FTP listing in certain cases.
Trigger on partial files	Check the box to trigger on partial files.
Import directory	If left blank, files are left on the files on the FTP server. Else, files will be downloaded.
Transfer in passive mode	Determines if the transfer should be in passive or active mode.
Transfer in binary mode	Determines if the transfer should be sent via ASCII or binary mode. General rule is ASCII for text, binary for non text files.
Remove source file after transfer	Remove the source file upon successful completion of the FTP transfer.
Move source file after trigger	Provide a directory relative to the watch folder to move the triggered files into.

Parameter	Description
Moved file suffix	An optional suffix that will be added to the file when moved. Only valid if Move source file after trigger is selected.
Use automatic suffix	If checked, when the file is moved, the current timestamp is added as suffix to the file name.
Check once?	If checked, the plugin checks for existence of file(s) matching the pattern and exits.
Continue Monitoring	If checked, the plugin re-triggers after a successful trigger.
Trigger persistence scope	Sets the scope/availability of the trigger output. The default value is 'By workorder group'. The definitions are: 'By workorder group', 'By action template', 'absolute', 'By step', 'By workflow', 'None'.
Transfer block size	Amount of data copied from application memory space to kernel memory space at one time during FTP transfers.
Reporting increment	Frequency (in seconds) in which progress is reported in step.

11.14.5 Inputs Description

The list of inputs depends on the configuration of the FtpTrigger action template. Changes in this list of inputs will override any values set in the action template.

Input	Description
binary	Transfer flag, binary or ASCII.
remote_directory	Directory on the FTP server below the default FTP directory.
remote_password	FTP password on the remote server.
remote_user	FTP username on the remote server.
remove_after_transfer	Remove the source file upon successful completion of the transfer.
partial_file	Check it if the plugin needs to trigger on partial files.
import_directory	Provide a local directory if the trigger file needs to be downloaded.
move_after_trigger	Provide a remote directory, relative to trigger folder, if the trigger file needs to be moved.
check_folders_recursively	Check it if sub-dirs need to be scanned recursively for provided pattern.
remove_after_trigger	Check it if the trigger file needs to be deleted after triggering.

11.14.6 Outputs Description

Output	Description
trigger_file	The name of the file that matched the specified pattern and triggered the plugin.
trigger_files	The name of the files that matched the specified pattern and triggered the plugin, when max files is not 1.
archived_filename	The archived path of the file on the server if move after trigger is selected.
archived_files_array	The list of archived paths of the files on the server if move after trigger is selected.
trigger_files_array_size	Detected files size, if max files is not 1.

11.14.7 Supported Actions

None

11.14.8 Dependencies

None

11.14.9 Operating Instructions

NOTE: FTP servers require port 21 to be open and accessible from Orchestrator. Additionally, the files in the target directory must have read permissions to be detected.

- Identify the remote FTP server and configure a remote node for it, or place the details of the FTP server in the advanced parameters section. If a particular user name and password is required to access the FTP server, you can place them in the remote node or within the advanced parameters settings.
- The FTP server will normally provide a directory that will be the ”root” directory for a particular connection. If there are directories that will be below this location, they should be entered in the ”remote directory” field. If the ftp user root directory is /root/ and you want to search elements under /root/dir1/, specify dir1/ in remote directory instead of /dir1/
- Select a regular expression pattern which the trigger files have in common.
- Insert the location that the files need to be imported to on local system, leave blank if it is not required. If the file being transferred is a binary file, select the ”Transfer in binary mode” check box.
- Check options for partial files, subdirectory scan, passive mode to if they are required for your workflow.
- Once triggered, option to remove or move the trigger file are provided. In this case both the original path and the path of the moved files are available as outputs.
- – Select the persistence scope for the trigger. Details:
  – 'By workorder group' - Trigger folders are tracked while workorder is executing and will only be trapped once. If the workorder is restarted, all past triggers will be detected again if they are still present.
  – 'By workflow' - Same as 'By workorder group' except no triggering will occur upon a 'restart' action as the workflow is still the same.
  – 'By action template' - Tracks previously processed trigger files and prevents them from re-triggering in another workflow that uses the same template.
  – 'absolute' - One trigger list is tracked regardless of where it happens.
  – 'By step' - Similar to 'By workflow', but allows for multiple independent trigger steps within the workflow.
  – 'None' – No persistence at all. If a past trigger file is still present, it will re-trigger if 'Continue Monitoring' is selected. This option is recommended when 'Continue Monitoring' is not selected.
- Select Continue Monitoring if the trigger needs to continuously monitor the remote FTP folder.
- Don't select Continue Monitoring when Check once is selected.

  NOTE ON FILEZILLA SERVERS In some cases a Filezilla Server will show files while they are not yet complete. Those file will be detected by the plugin even if they are still growing on disk. To avoid an early triggering, select ”Ignore zero bytes files”. These files are marked as 0 bytes by Filezilla while they are opened for writing.

11.15 Local file watcher

This action plug-in watches a local directory for files matching defined patterns and enables the creation of triggers upon detection.

Version	Comment
1.9.3	Enhanced action template UI
1.9.2	Handle files that are deleted during candidate processing
1.9.1	Updated the label from Keep ongoing to Continue Monitoring
1.9.0	Remove hardcoded aspera directory reference, Updating to be compatible with Ruby 3.4 and Rails 8
1.8.2	Optimized detection reactivity in case of multiple candidates
1.8.1	Avoid failure when a candidate gets deleted before triggering.

Version	Comment
1.8.0	Make optional the update of timestamps when moving file to archive.
1.7.9	Fix for persistence issue when maxEntries is set
1.7.8	Adds a flag to include folders in detected elements
1.7.7	Modify logic to allow picking up new candidates by checking the watched directory even if entries are found in the candidate list
1.7.5	Lowered timeout to 300 (from infinite) and added timestamp to the polling status messages when no files are found.
1.7.4	Fixes a display glitch for comments affecting orchestrator 3.0.0
1.7.3	Adds check to create runtime directory, if not present, at step start
1.7.2	Added styling to help text.
1.7.1	Exceptions in processTriggers that are currently caught and not escalated, will be escalated and will affect the work step.
1.7.0	Adds option to always prune past-triggers (1.6.0 skipped to avoid conflicts)
1.5.1	Exposes timeout as an input
1.5.0	Adds support for separate cooling off period from scan frequency
1.4.3	Fix type of directory input in case of multi-folders search
1.4.2	Remove duplicates in case of multiple folder expressions that would return the same file twice
1.4.1	New icon
1.4.0	Added a flag to let users submit an array of file paths to watch.
1.3.5	Minor Layout change
1.3.4	change help file to specify behavior when ** is used in watch directory
1.3.3	change edit view to make return all field consistent with other checkboxes
1.3.2	Add examples in help files for wildcard manipulation
1.3.1	Avoids error in comments generation
1.3.0	Adds option to ignore files that are locked (useful to avoid false positive on stability check for copied files in windows)
1.2.2	Avoids exiting with 0 files, if trigger files can not be moved
1.2.1	Fixes bug in 1.1.9 exception handling, and flushes obsolete past candidates
1.1.9	Fixes bug in 1.1.8 exception handling
1.1.8	Ensures timestamps are ignored in past trigger comparison when triggering on partial files
1.1.7	Fixes some minor inconsistencies from 1.1.6 and added specifications
1.1.6	Improves the handling of disappearing files in the watchfolder.
1.1.5	Changed the timestamping to conserve file extension name.extension -> name_timespamp.extension
1.1.4	Avoids migration issues when continuing execution of step started with a version pre 0.5.0
1.1.3	Avoids crash if files disappear after being identified as candidate
1.1.2	Fixes pause/resume functionality.
1.1.1	Adds support for runtime-defined triggers
1.1.0	Adds support for exclusions and remove duplicate zero byte check box
1.0.0	Simplified to use generic trigger persistence (requires Orchestrator 2.0)
0.5.0	Adds option to ignore 0 bytes file
0.4.2	Fix backward compatibility of inputs_spec
0.4.1	Adds reporting on candidates found, adds support for dynamic attributes
0.4.0	Changes comments to be user configurable instead of generated
0.3.5	fix problems with synchronous watchers and multiple candidates
0.3.4	special handling for empty trigger and candidates files.
0.3.3	closes the file after reading content (solve windows issue)
0.3.2	adds no-persistence option
0.3.1	fixes glitch in persistence by action template, optimize glob searches
0.3.0	use polling frequency in seconds
0.2.1	adds cool-off enforcement to stability check
0.2.0	drops the synchronous option from GUI, supports different modes of trigger persistence, moves creation of temp dir
0.1.3	adds clean up of var run files (candidates and past triggers)
0.1.2	persists the list of matching entries to avoid the re-scanning at each iteration.
0.1.1	temporary fix for ATT to add an optional max_entry parameter



Figure 11.15: Local file watcher icon

11.15.1 How It Works

- Monitors local directories for new files matching specified patterns or filenames.
- Supports single-check operations or continuous monitoring (hot folder) with configurable polling frequencies.
- Provides flexible pattern matching with GLOB expressions, regular expressions, and filename/path exclusions.
- Includes advanced features like stability checking, file archiving, trigger file removal, and configurable trigger persistence scopes.
- Allows processing of trigger file contents and supports both single file and multiple file list outputs.

11.15.2 Configuration Instructions

1. Add a LocalFileWatcher step to your workflow.
2. Configure the directory monitoring parameters:
   - Name: Enter a descriptive name for the file watcher.
   - Comments: Add notes or description for this step.
   - Watch directory: The directory path to monitor for trigger files (e.g., /foo/bar/).
   - Multi folder search?: Check if multiple folders are provided in the watch directory field.
   - Trigger filename or pattern: The file name or pattern to trigger on (e.g., *.txt).
   - Use regex pattern matching: Check to enable regular expression pattern matching.
3. Set up monitoring behavior:
   - Continue Monitoring: Check for continuous monitoring, uncheck for single check.
   - Polling frequency: Interval in seconds between directory scans.
   - Triggers on partial file: Check to detect files being modified or incomplete.
   - Stability cool-off time: Minimum time for a file to remain unchanged to be considered stable.
4. Configure file handling options:
   - Remove trigger file: Check to delete trigger file after detection.
   - Trigger files archive directory: Path where trigger file copies will be placed.
   - Add timestamp to archived copy: Appends '_YYYYMMDD_HHMMSS' to archived file names.
   - Keep timestamps when archiving?: Preserve original timestamps when archiving files.
5. Set output and persistence options:
   - Return all matching files as a list: Output all matching triggers as a list.
   - Provide content of trigger file as additional output: Parse and output trigger file contents.
   - Trigger persistence scope: Set scope for trigger tracking (default: 'by workorder group').

11.15.3 Example Configuration

Parameter	Example Value
Name	Incoming Files Monitor
Comments	Monitors incoming directory for text files
Watch directory	/foo/bar/*
Multi folder search?	unchecked
Trigger filename or pattern	*.txt
Use regex pattern matching	unchecked
Continue Monitoring	checked
Polling frequency	60

Parameter	Example Value
Triggers on partial file	unchecked
Stability cool-off time	30
Ignore 0 bytes files?	checked
Return all matching files as a list	unchecked
Remove trigger file	unchecked
Trigger files archive directory	/archive/processed
Add timestamp to archived copy	checked
Trigger persistence scope	by workorder group

11.15.4 Saved Parameters Description

Parameter	Description
Name	Display name for this file watcher instance.
Comments	Notes or description for this step.
Continue Monitoring	Wait for matching file(s), spawn a new workflow instance each time matching file(s) are detected and proceed with the next steps. The new instance will wait for new matching files. If this option is not set, wait for matching files once and proceed with the next steps.
Watch directory	The directory path searched for the existence of a trigger filename or pattern.
Multi folder search?	Check this box if multiple folders are provided in the watch directory field.
Trigger filename or pattern	The file name or pattern utilized as a trigger.
Trigger filename base	Portion of trigger file that is stripped from the name, altering the root directory for the trigger file.
Use regex pattern matching	Permits the use of regular expression in the trigger file name or pattern text box.
Filename exclusions	All files whose basename match the provided regex, are filtered out of the result.
Full path exclusions	All files whose full name (full path including basename) match the provided regex, are filtered out of the result.
Polling frequency	Interval in seconds searching occurs in the watch directory for trigger files.
Triggers on partial file	A trigger file will be detected whether presently being modified or complete.
Stability cool-off time	Optional - The minimum time it takes for a file to be seen unchanged to be considered stable. If not provided, the polling frequency is used.
Ignore 0 bytes files?	Empty files are filtered out of the result if this option is selected.
Ignore read-locked files?	In windows file being copied can not be detected as growing, which creates a risk of false positive for the file stability check whenever they are locked. So ignoring locked files avoids those false positive.
Return all matching files as a list	List of all matching trigger are made available as output.
Provide content of trigger file as additional output	Parse the trigger file and make its content available as output.
Remove trigger file	Remove the trigger file from the search directory after detection has been made.

Parameter	Description
Trigger files archive directory	Full path to a location where trigger file copies will be placed.
Add timestamp to archived copy	Appends '_YYYYMMDD_HHMMSS' to the end of the trigger file name when it is archived.
Keep timestamps when archiving?	By default archived files are touched before being moved, if this option is checked, then the timestamps are left alone instead.
Trigger persistence scope	Sets the scope/availability of the trigger output. The default value is 'by workorder group'. The definitions are: 'by workorder group', 'by action template', 'absolute', 'by step', 'by workflow', 'none'.
Enable aggressive persistence list pruning?	When checked, the persistence list is updated at every scan, not just when a new candidate is found. Checking this option is dangerous as if a network drive becomes unaccessible during a scan, all the files on it will be considered not present any longer (unless a pruning beacon file is provided) and hence will re-trigger when the network drive becomes available again.
Pruning-beacon filepath	Optional - The path to a file whose presence indicates the scan folder is accessible.

11.15.5 Inputs Description

Changes in this list of inputs will override any values set in the action template.

- allowMultiple: Scanning for trigger files is continuous and processing will be started whenever they are detected.
- archiveDirectory: Full path to a location where trigger file copies will be placed.
- archiveWithTimestamp: Appends '_YYYYMMDD_HHMMSS' to the end of the trigger file name when it is archived.
- directory: The directory path searched for the existence of a trigger filename or pattern.
- fileName: The file name or pattern utilized as a trigger.
- maxEntries: Has impact only when 'Return all matching files as a list' is selected. At that time, this will limit the size of that listing by the values set in maxEntries.
- readFile: Parse the trigger file and make its content available as output.
- removeFile: Remove the trigger file from the search directory after detection has been made.
- tempo: Interval in seconds searching occurs in the watch directory for trigger files.

11.15.6 Outputs Description

- FileContent: The contents of the trigger file.
- FileName: Based on state of 'Return all matching files as a list' will contain a list of all trigger files or name of oldest matching file.

11.15.7 Supported Actions

None

11.15.8 Dependencies

- Local file system access with appropriate read/write permissions for the aspweb user.
- Directory structure must be accessible to the Orchestrator process.
- For network drives, ensure stable connectivity and proper mounting.

11.15.9 Operating Instructions

11.15.10 Overview

This plugin detects new files matching a certain pattern and located below a directory visible locally. Both the pattern matching and the directory path must be provided by the user. The directory may be searched a single time as part of

a workflow or can be treated similarly to a hot folder where it is continually scanned at set time intervals looking for a pre-defined trigger file. Once a trigger file has been detected it may either be parsed to obtain its contents, or have the names in the case of multiple triggers, be collected and output. Trigger files may be archived and/or removed once detected.

11.15.11 Watch Directory Configuration

Populate the Watch directory field with the directory to be scanned for trigger files:

- Root directory only: Use the path without wildcards (e.g., /foo/bar/) for files that can only appear in the root directory. This provides better performance.
- First-level subdirectories: Use a single wildcard (e.g., /foo/bar/*) to search the first directory level below the root.
- All subdirectories: Use double wildcards (e.g., /foo/bar/**) to search ALL directories below the watch directory.
- Hidden folders: Folders starting with a DOT (.) are not searched by default as GLOB ignores them. For example, if the directory is /foo/bar/* and /foo/bar/.tmp exists, the watcher will not look inside .tmp.
- If watch directory = `/foo/bar/` and pattern = `*.txt`

  Detected: /foo/bar/titi.txt

  Not detected: /foo/bar/bar1/toto.txt or /foo/bar/bar1/bar2/tutu.txt
- If watch directory = `/foo/bar/*` and pattern = `*.txt`

  Detected: /foo/bar/bar1/toto.txt

  Not detected: /foo/bar/titi.txt or /foo/bar/bar1/bar2/tutu.txt
- If watch directory = `/foo/bar/**` and pattern = `*.txt`

  Detected: /foo/bar/titi.txt, /foo/bar/bar1/toto.txt, and /foo/bar/bar1/bar2/tutu.txt

11.15.12 Multi-Folder Search

To monitor multiple directories simultaneously, check the ”Multi folder search?” box and provide directories in one of these formats:

- Array format: `["/tmp/test", "<%= Rails.root %>/tmp/"]`
- Variable array format: `["<%= f1%>", "<%= f2%>"]` and provide values to f1 and f2 in the map inputs screen
- Semicolon-separated format: `<%= f1%>;<%= f2%>` and provide values to f1 and f2 in the map inputs screen

11.15.13 File Pattern Configuration

Once configured, the plugin will loop through all folders looking for files matching the pattern provided.

- Enter only the filename or pattern, not the full path.
- Use wildcards for pattern matching (e.g., `*.txt` matches all files with .txt extension).
- Select ”Use regex pattern matching” for more sophisticated pattern matching using regular expressions.

11.15.14 Monitoring Behavior

- Continue Monitoring: Select this option to periodically scan for files arriving into the Watch directory and act upon their arrival (hot folder behavior).
- Triggers on partial file: Select if it's acceptable to trigger on files still being written. This allows processing to begin before the trigger file has been completely updated.
- Stability cool-off time: Specify the minimum time a file must remain unchanged to be considered stable before triggering.

11.15.15 File Content and Output Options

- Provide Content of Trigger File as additional output: Select if the file contents are required for later processing (e.g., JSON or XML files that need parsing).
- Return all matching files as a list: Enable to output all matching trigger files as a list instead of processing them individually.

11.15.16 Archiving and Cleanup

- Trigger files archive directory: Specify the full path where trigger file copies will be placed. Note that ownerships and permissions will be set based on the user executing Orchestrator.
- Trigger file name base: Specify a portion of the trigger file path to strip from the name when archiving. Useful when trigger files exist in subdirectories but you don't want to preserve that directory structure in the archive location.
- Add timestamp to archived copy: Select to append a timestamp (_YYYYMMDD_HHMMSS) to archived trigger files.
- Remove trigger file: Select if the trigger file should be deleted after processing.

11.15.17 Trigger Persistence Scope Details

The trigger persistence scope determines how the action tracks and prevents duplicate processing of trigger files:

- by workorder group (default): Tracks triggers during workorder execution, prevents duplicate processing within workorder group.
- by workflow: Same as workorder group but no triggering on restart actions for same workflow.
- by action template: Prevents re-triggering across workflows sharing the same template.
- by step: Independent trigger tracking per step within workflows.
- none: No persistence, allows re-triggering on existing files when 'Continue Monitoring' is selected.

11.16 Local folder watcher



Figure 11.16: Local folder watcher icon

This action plug-in provides the ability to wait for the arrival of folder matching a certain pattern and optionally wait for the stability of all files in that folder.

Version	Comment
1.6.3	Enhanced action template UI
1.6.2	Removed empty option from trigger persistence scope dropdown
1.6.1	Updated the label from Keep ongoing to Continue Monitoring
1.6.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.5.1	Fixed support for dynamic expression in rejection regex
1.5.0	Adding support for aggressive pruning
1.4.0	Adding support to omit empty folders
1.3.0	Adding support for multi folder option
1.2.3	Made polling frequency and ignore pattern optional runtime inputs.
1.2.2	Added styling to help text.
1.2.1	Clean up metadata. Fix condition to search basename OR full path depending on flag
1.2.0	Current ignore checks for basename of a folder. so added a logic to check for the entire file.
1.1.7	Modify the pruning logic, to be applied on every pulse instead of just upon triggering
1.1.6	Fixes behavior of blank persistence scope
1.1.5	New icon
1.1.4	Fixes problem when base folder is empty, triggered entities are not persisted
1.1.3	Fixes typo in VAR_TRIGGER_TYPE introduced in 1.1.2. Tested at CBS 2014/11/18
1.1.2	Making persistenceScope runtime variable
1.1.1	fixes regex ignore, fixes processing when stability check is not requested

Version	Comment
1.1.0	adds ignore folder name pattern, restrict regex matching to basename only
1.0.2	r24946 s8349 - Local Folder Watcher fails if path does not exist
1.0.1	Only update persistence list when it triggers
1.0.0	Simplified to use generic trigger persistence (requires Orchestrator 2.0)
0.1.0	Adds stability filter regex
0.0.3	Forces re-scanning of folder files in case a new file is detected between 2 check_status
0.0.2	Adds help file, fix trigger on partial option
0.0.1	initial release

11.16.1 How It Works

- This plugin monitors the contents of a specified local folder and identifies the subfolders within it.

11.16.2 Configuration Instructions

1. Create a new action step using the LocalFolderWatcher plugin.
2. Fill out the following fields:
   - Watch directory: The search root. GLOB wildcards are supported (`*`, `**` and `?`). Dynamic attributes are also supported.
   - Multi folder search?: Check this box if multiple folders are provided in the *Watch directory* field.
   - Folder pattern: (optional) The pattern for the names of the folders to be detected. The pattern can be expressed using GLOB or REGEX (depending on the option selected below). Dynamic attributes are supported. If no pattern is provided, any new folder will trigger.
   - Use regex pattern matching?: By default, GLOB matching is used, but if checked, REGEX will be used.
   - Ignore folder name pattern: (optional) The pattern (expressed as REGEX) for directory names to be excluded.
   - Ignore folder name pattern applied to full path?: By default, the exclusion pattern above is applied only to the folder's basename. If checked, the pattern will be applied to the full path.
   - Polling frequency: (optional) The wait time between each status check. This value is also used as the minimum time a folder must remain stable if *Trigger on partial* is not checked.
   - Return folder content?: If checked, additional inputs containing a list of the contents of the triggered folders will be provided.
   - Continue Monitoring: If checked, the containing workflow will be cloned after each trigger event.
   - Omit Empty Folders?: If checked, the workflow will not pick empty folders. (For this purpose, only one level down is checked; subfolders are not scanned recursively.)
   - Return all matching entries?: By default, only the oldest matching entry is returned. If checked, all matching entries will be returned.
   - Trigger on partial?: By default, only stable folders are returned. If checked, the stability check is bypassed.
   - Trigger persistence scope: Sets the scope/availability of the trigger output. The default value is *By workorder group*. The available options are: *By workorder group*, *By action template*, *absolute*, *By step*, *By workflow*, *None*.

11.16.3 Example Configuration

This is a minimal setup configuration for the plugin.

Parameter	Value
Watch directory	/home
Folder pattern	.*\s$
Polling frequency	10
Use regex pattern matching?	Selected
Trigger on partial?	Selected

11.16.4 Saved Parameters Description

Parameter	Description
Watch directory	The search root. GLOB wildcards are supported (`*`, `**` and `?`). Dynamic attributes are also supported.
Multi folder search?	Check this box if multiple folders are provided in the *Watch directory* field.
Folder pattern	(optional) The pattern for the names of the folders to be detected. The pattern can be expressed using GLOB or REGEX (depending on the option selected below). Dynamic attributes are supported. If no pattern is provided, any new folder will trigger.
Use regex pattern matching?	By default, GLOB matching is used, but if checked, REGEX will be used.
Ignore folder name pattern	(optional) The pattern (expressed as REGEX) for directory names to be excluded.
Ignore folder name pattern applied to full path?	By default, the exclusion pattern above is applied only to the folder's basename. If checked, the pattern will be applied to the full path.
Polling frequency	(optional) The wait time between each status check. This value is also used as the minimum time a folder must remain stable if *Trigger on partial* is not checked.
Return folder content?	If checked, additional inputs containing a list of the contents of the triggered folders will be provided.
Continue Monitoring	If checked, the containing workflow will be cloned after each trigger event.
Omit Empty Folders?	If checked, the workflow will not pick empty folders. (For this purpose, only one level down is checked; subfolders are not scanned recursively.)
Return all matching entries?	By default, only the oldest matching entry is returned. If checked, all matching entries will be returned.
Trigger on partial?	By default, only stable folders are returned. If checked, the stability check is bypassed.
Trigger persistence scope	Sets the scope/availability of the trigger output. The default value is *By workorder group*. The available options are: *By workorder group*, *By action template*, *absolute*, *By step*, *By workflow*, *None*.

11.16.5 Inputs Description

Details:

- By workorder group: Trigger folders are tracked while the workorder is executing and will be triggered only once. If the workorder is restarted, previously detected triggers will be triggered again if still present.
- By workflow: Same as *By workorder group*, except no triggering will occur upon a *restart* action because the workflow remains the same.
- By action template: Enables tracking across workflows that share the same template, preventing duplicate triggering for previously processed folders.
- absolute: A single trigger list is tracked globally, regardless of where it occurs.
- By step: Similar to *By workflow*, but allows multiple independent trigger steps within the same workflow.
- None: No persistence at all. If a previously triggered folder is still present, it will trigger again if *Continue Monitoring* is selected. Recommended when *Continue Monitoring* is not selected.

11.16.6 Outputs Description

Parameter	Description
Folder_path	Returned only if *Return all matching entries?* is not checked. Provides the full path to the detected folder matching the search criteria.

11.16.7 Supported Actions

None

11.16.8 Dependencies

None

11.16.9 Operating Instructions

This plugin does not support value overwrites: if a value is specified in the template, it will not be available as an input to the step at runtime.

11.17 Remote file watcher



Figure 11.17: Remote file watcher icon

This action plug-in provides the ability to wait for the arrival of files matching a certain pattern on a remote aspera server.

Version	Comment
2.4.3	Enhanced action template UI
2.4.2	Removed empty option from trigger persistence scope dropdown
2.4.1	Updated the label from Keep Ongoing to Continue Monitoring
2.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
2.3.2	Fix Rails compatibility issue
2.3.1	Fixes a problem when using GLOB expression where the plugin would browse folders that do not match
2.3.0	Added aggressive pruning with beacon file for stability check
2.2.2	Avoids error-ing out if a folder without permissions is encountered.
2.2.1	Fixing watch_directory_get array to string conversion
2.2.0	Added option to ignore ctime
2.1.1	Fixing code to support 1.8.7 and 2+
2.1.0	Adding logic to exclude candidate file based on a regex
2.0.2	Added Remote Node Dependency
2.0.1	Clean up metadata, hide the debug_extensive
2.0.0	Added possibility to limit the maximum number of files returned per scan round
1.9.2	Fix detect connection exception when scanning folders
1.9.1	Fix folder matching where GLOBING in the middle of folder path
1.9.0	Split cool off from polling frequency
1.8.1	Fix wrong variable name in check one case
1.8.0	Add force exact matching option. This disable regex matching of file name. Fixed candidate counts in case of multi folders.
1.7.0	Add extensive debug option for acmd as_ls outputs
1.6.0	Removes possible duplicates from the notification list (in case of multi folders)
1.5.9	Fixes problem in case of intermittent SSH disconnections while scanning the remote folders
1.5.8	Adds ability to set a custom persistence scope
1.5.7	Adds ability to provide polling frequency at run-time
1.5.6	New icon
1.5.5	Avoid persistency issues with Remote Node with multiple addresses
1.5.4	Fixes selection of execution address in case of multiple addresses. Fixes search algorithm in case of subfolder reg ex (/dir1/dir2*/*.txt)
1.5.3	Avoids error if one scanned folders is not found in case of multi-folder search

Version	Comment
1.5.2	Exposed the scan time out field in edit.html
1.5.1	Added protection in case of permission errors when listing folders
1.5.0	Added recursive search option (avoid need for globing at directory level)
1.4.3	Fixed problem with files in the root directory of the search not detected.
1.4.2	Fixed ignore instant file checkbock in the editor
1.4.1	exception handling of RemoteNode.find_by_name()
1.4.0	Expands size of watch_directory field to accommodate path list longer than 255 characters
1.3.0	Adds multi-folder support
1.2.0	Adds filter on 0-byte files
1.1.4	Corrected the instantiation of a class variable
1.1.3	Removed excessive debugging logs.
1.1.2	Added generic error protection for SSH errors. Retry 10 times before giving up.
1.1.1	Added protection for SSH mac address errors. Retry.
1.1.0	Adding code for avoid detection of files where ctime == mtime when instant files are ignored. Added SSH timeout protection
1.0.2	Debugging drag and drop behavior
1.0.2	Authentication errors were masked during node list due to a rescue []. Removing it as the plugin handles
1.0.1	Force trigger on partial behavior if check once is selected
1.0.0	Simplified to use generic trigger persistence (requires Orchestrator 2.0), does not persist empty list to avoid overwritting the file when connection is lost
0.2.9	fix method names introduced in 0.2.8
0.2.8	fix for REMOTE_PASSWORD not correctly set at runtime
0.2.7	Added protections in case the persistent file for past triggers cannot be read
0.2.6	fixes error when encountering folder with bad permissions
0.2.5	fixes no-persistence glitch when used with stability check
0.2.3	adds no-persistence option
0.2.2	fixes glitch when using multiple wildcard in folder expression
0.2.1	fixes glitch in persistence by action template
0.2.0	adds keep ongoing and persistence types
0.1.0	adds option to check only once, fails if no match is found
0.0.1	initial release - basic with no import, no recurring

11.17.1 How It Works

- This action detects new files or folders that match a specified pattern within a watch directory on a remote server.
- You must provide both:
  - The directory path to be monitored.
  - The pattern used to identify files or folders.
- The directory can be scanned in two ways:
  - Single scan — Searches the directory once for matching files. (Select 'Check only once' or leave 'Continue Monitoring' unselected.)
  - Continuous monitoring — Treats the directory like a hot folder, continuously scanning for matching files. (Select 'Continue Monitoring'.)
- If 'Check only once' is selected and no matching file is found, the action will follow the failure path.

11.17.2 Configuration Instructions

1. Create a new action step using the RemoteFileWatchers plugin.
2. Fill out the following fields:
   - Remote Node: The remote node name from defined Remote Nodes.
   - Watch directory: The directory search path for files or folders.
   - Multi folder search?: If selected, indicates that a list of folders will be provided. If the folders are set at design time, they can be entered either using a semi-colon for separator, or alternatively the ruby array notation can be used (ie.: ['folder1', 'folder2'])
   - Descend in sub directories?: If selected, indicates that the watchfolder will look for file on the base path and in all subfolders at all levels.
   - Folder pattern: The detection file name or pattern expressed as a regular expression.
   - Ignore folders?: Folders will be ignored. Only matching files are returned.

- Ignore file attribute changes?: Choose this option if you don't want files to be trigged on metadata changes like permissions. This will ignore the ctime of the file while triggering on it again.
- Ignore instant files?: Ignore files for which the creation date equals the last modification date (condition true for some Windows files that are still growing).
- Ignore zero-byte files?: Ignore empty files (files whose size is 0 bytes).
- Polling frequency: Interval in seconds to search for matching files in the watch directory.
- Check only once: Check for matching file presence (do not wait) and stop processing.
- Continue Monitoring: Wait for matching file(s), spawn a new workflow instance each time matching file(s) are detected and proceed with the next steps. The new instance will wait for new matching files. If this option is not set, wait for matching files once and proceed with the next steps.
- Return single match?: Return only one matching file at a time. Otherwise return matching files as an array.
- Maximum number of files detected: Limits the maximum number of files detected in a single round. Use 0 to specify no limit.
- Trigger on partial: A matching file will be detected even if not stable.
- Stability cool-off time: Optional - The minimum time it takes for a file to be seen unchanged to be considered stable. If not provided, the polling frequency is used.
- Scan time out: Increase the scan time out value for folders with large number of files.
- Enable extensive debug: If set it provides extensive debug information when listing files on remote node. Must be used only in debugging situations.
- Force exact matching: This option disable regex matching for the file name. The watcher will only detect files that match exactly the File pattern.
- Trigger persistence scope: Sets the scope/availability of the trigger output. The default value is 'by workorder group'. The definitions are: 'by workorder group', 'by action template', 'absolute', 'by step', 'by workflow', 'none'.

11.17.3 Example Configuration

This is a minimum setup configuration for the plugin.

Parameter	Value
Watch directory	/home
Folder pattern	.*\s$
Polling frequency	10
Use regex pattern matching?	Selected
Trigger on partial?	Selected

11.17.4 Saved Parameters Description

Parameter	Description
Remote Node	The remote node name from defined Remote Nodes.
Watch directory	The directory search path for files or folders.
Multi folder search?	If selected, indicates that a list of folders will be provided. If the folders are set at design time, they can be entered either using a semi-colon for separator, or alternatively the ruby array notation can be used (ie.: ['folder1', 'folder2'])
Descend in sub directories?	If selected, indicates that the watchfolder will look for file on the base path and in all subfolders at all levels.
Folder pattern	The detection file name or pattern expressed as a regular expression.
Ignore folders?	Folders will be ignored. Only matching files are returned.
Ignore file attribute changes?	Choose this option if you don't want files to be trigged on metadata changes like permissions. This will ignore the ctime of the file while triggering on it again.
Ignore instant files?	Ignore files for which the creation date equals the last modification date (condition true for some Windows files that are still growing).

Parameter	Description
Ignore zero-byte files?	Ignore empty files (files whose size is 0 bytes).
Polling frequency	Interval in seconds to search for matching files in the watch directory.
Check only once	Check for matching file presence (do not wait) and stop processing.
Continue Monitoring	Wait for matching file(s), spawn a new workflow instance each time matching file(s) are detected and proceed with the next steps. The new instance will wait for new matching files. If this option is not set, wait for matching files once and proceed with the next steps.
Return single match?	Return only one matching file at a time. Otherwise return matching files as an array.
Maximum number of files detected	Limits the maximum number of files detected in a single round. Use 0 to specify no limit.
Trigger on partial	A matching file will be detected even if not stable.
Stability cool-off time	Optional - The minimum time it takes for a file to be seen unchanged to be considered stable. If not provided, the polling frequency is used.
Scan time out	Increase the scan time out value for folders with large number of files.
Enable extensive debug	If set it provides extensive debug information when listing files on remote node. Must be used only in debugging situations.
Force exact matching	This option disable regex matching for the file name. The watcher will only detect files that match exactly the File pattern.
Trigger persistence scope	Sets the scope/availability of the trigger output. The default value is 'by workorder group'. The definitions are: 'by workorder group', 'by action template', 'absolute', 'by step', 'by workflow', 'none'.

11.17.5 Inputs Description

Parameter	Description
Watch directory	The directory search path for files or folders.
Folder pattern	The detection file name or pattern.
Remote node	Remote node IP address or domain name.
remote_node_user	Remote node user name.
remote_node_password	Remote node password.
Exclusions	File names to exclude
persistence_scope	The scope to trigger the persistance
polling_frequency	Interval in seconds to search for matching files in the watch directory.

Details:

- By workorder group: Trigger folders are tracked while workorder is executing and will only be trapped once. If the workorder is restarted, all past triggers will be detected again if they are still present.
- By workflow: Same as 'By workorder group' except no triggering will occur upon a 'restart' action as the workflow is still the same.
- By action template: Allows the tracking and thus non-triggering in the second workflow for previously processed trigger files shared across workflows that contain the same template,
- absolute: One trigger list is tracked regardless of the where it occurs,

- By step: Similar to 'By workflow', but allows for multiple independent trigger steps within the workflow.
- None: No persistence at all. If a past trigger file is still present, it will re-trigger if 'Continue Monitoring' is selected. This option is recommended when 'Continue Monitoring' is not selected.

11.17.6 Outputs Description

Parameter	Description
File_list	The matching files as an array.

11.17.7 Supported Actions

None

11.17.8 Dependencies

None

11.17.9 Operating Instructions

Instructions:

Configure the settings below to control how the system searches for trigger files:

Watch directory:

- Populate the Watch directory with the directory to be scanned for trigger files.
- Globbing pattern matching is used.
- If trigger files may appear one level below the root directory, include a wildcard in the path (e.g. `/foo/bar/*`).
- Use the *Descend in sub directories?* flag to scan deeper folder levels.
- If files appear only in the root directory, avoid enabling descent for performance reasons.
- Pattern example: `/fo?` matches `/foo` and `/fo1`, but not `/foo1`.
- The search is case sensitive.

File Pattern:

- Enter the file name or regular expression used as the trigger pattern.
- Example: `.*\.txt` matches all `.txt` files.
- Multiple patterns example: `(.*\.mxf|.*\.mpg)`.

Remote Node Requirements:

- Works with remote nodes running an Aspera transfer server (uses the `ascmd` executable).
- Remote node must run an SSH server for authentication.
- The remote username must be an OS account defined in `/opt/aspera/etc/aspera.conf`.
- The connection to the remote node must be made by a user with `root`-level access.
- The Watch directory path is relative to the remote user's docroot, as declared in `aspera.conf`.
- Files and directories must have correct permissions for the configured user.

```
\                <user>\                <name>xfer</name>\                <file_system>\                <access>\
<paths>\                <path>\                <absolute>/home/xfer/data</absolute>\                </path>\
</paths>\                </access>\                </file_system>\                </user>\
```

For example, a watch directory `/foo` corresponds to `/home/xfer/data/foo` on the remote server.

Trigger Behavior:

- Select *Continue Monitoring* to periodically scan for new files or folders.
- *Check only once* and *Continue Monitoring* are incompatible — do not select both.
- Select *Triggers on partial file* to allow processing before a file is completely written.



Figure 11.18: Remote folder watcher icon

11.18 Remote folder watcher

This action plug-in provides the ability to wait for the arrival of a folder matching a certain pattern on a remote aspera server.

Version	Comment
0.2.4	Enhanced action template UI
0.2.3	Updated the label from Keep ongoing to Continue Monitoring
0.2.2	Updated the label from Keep ongoing to Continue Monitoring
0.2.1	Updating plugin migrations to Rails 8 format
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	Ignore file addition
0.0.4	Multiple execution bug fix
0.0.3	bug fix
0.0.2	Adding Remote Node Dependency
0.0.1	initial release

11.18.1 How It Works

- The action monitors a remote server for new folders that match a user-specified pattern located under a watch directory.
- You must provide both the watch directory path and the pattern used to identify matching folders.
- Choose how the directory is scanned:
  - Single scan: Search the directory one time for a matching folder (do not select *Continue Monitoring*).
  - Continuous scan (hot folder): Continuously scan the directory for new matching folders (select *Continue Monitoring*).
- If no matching folder is found, the action follows the failure path.

11.18.2 Configuration Instructions

1. Create a new action step using the RemoteFolderWatcher plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved RemoteFolderWatcher configured instance.
   - Comments: Some comments about this saved RemoteFolderWatcher configured instance.
   - Remote Node: The remote node name from the defined Remote Nodes.
   - Watch directory: The directory search path for folders.
   - Multi-directory search?: If selected, indicates that a list of folders will be provided. If the folders are set at design time, they can be entered using a semicolon as a separator, or the Ruby array notation can be used (e.g., ['folder1', 'folder2']).
   - Polling frequency: Interval in seconds to search for matching folders in the watch directory.
   - Check only once: Check for matching folder presence (do not wait) and stop processing.
   - Continue Monitoring: Waits for matching folder(s), spawns a new workflow instance each time matching folder(s) are detected, and proceeds with the next steps.
   - Return single match?: Return only one matching folder at a time; otherwise, return matching folders as an array.
   - Maximum number of folders detected: Limits the maximum number of folders detected in a single round. Use 0 to specify no limit.
   - Trigger on partial: A matching folder will be detected even if it is not stable.

- Stability cool-off time: Optional - The minimum time it takes for a folder to be seen unchanged to be considered stable. If not provided, the polling frequency is used.
- Scan timeout: Increase the scan timeout value for folders with a large number of files.
- Enable extensive debug: If enabled, provides detailed debug information when listing folders on the remote node.
- Force exact matching: This option disables regex matching for the folder name. The watcher only detects folders that match the folder pattern exactly.
- Ignore file addition?: Do not trigger again when a file is added or removed under a folder already returned by the watcher. This prevents double processing.
- Trigger persistence scope: Sets the scope/availability of the trigger output. The default value is 'by workorder group'. The definitions are: 'by workorder group', 'by action template', 'absolute', 'by step', 'by workflow', 'none'.

11.18.3 Example Configuration

This is a minimum setup configuration for the plugin.

Parameter	Value
Watch directory	/home
Folder pattern	.*\s$
Polling frequency	10
Multi-directory search?	Selected
Trigger on partial?	Selected

11.18.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved RemoteFolderWatcher configured instance.
Comments	Some comments about this saved RemoteFolderWatcher configured instance.
Remote Node	The remote node name from the defined Remote Nodes.
Watch directory	The directory search path for folders.
Multi-directory search?	If selected, indicates that a list of folders will be provided. If the folders are set at design time, they can be entered using a semicolon as a separator, or the Ruby array notation can be used (e.g., ['folder1', 'folder2']).
Polling frequency	Interval in seconds to search for matching folders in the watch directory.
Check only once	Check for matching folder presence (do not wait) and stop processing.
Continue Monitoring	Waits for matching folder(s), spawns a new workflow instance each time matching folder(s) are detected, and proceeds with the next steps.
Return single match?	Return only one matching folder at a time; otherwise, return matching folders as an array.
Maximum number of folders detected	Limits the maximum number of folders detected in a single round. Use 0 to specify no limit.
Trigger on partial	A matching folder will be detected even if it is not stable.
Stability cool-off time	Optional - The minimum time it takes for a folder to be seen unchanged to be considered stable. If not provided, the polling frequency is used.
Scan timeout	Increase the scan timeout value for folders with a large number of files.
Enable extensive debug	If enabled, provides detailed debug information when listing folders on the remote node.
Force exact matching	This option disables regex matching for the folder name. The watcher only detects folders that match the folder pattern exactly.
Ignore file addition?	Do not trigger again when a file is added or removed under a folder already returned by the watcher. This prevents double processing.

Parameter	Description
Trigger persistence scope	Sets the scope/availability of the trigger output. The default value is 'by workorder group'. The definitions are: 'by workorder group', 'by action template', 'absolute', 'by step', 'by workflow', 'none'.

11.18.5 Inputs Description

The list of inputs depends on the configuration of the RemoteFolderWatcher action template. If dynamic attributes are used in the message definition, each attribute becomes a required input.

Parameter	Description
Watch directory	The directory search path for folders.
Folder pattern	The detection folder name or pattern.
Remote node	Remote node IP address or domain name.
remote_node_user	Remote node username.
remote_node_password	Remote node password.
Result_limit	Maximum number of results to return.
persistence_scope	The scope used to track trigger persistence.
polling_frequency	Interval in seconds to search for matching folders in the watch directory.

Persistence Scope Details:

- By workorder group: Trigger folders are tracked while workorder is executing and will only be trapped once. If the workorder is restarted, all past triggers will be detected again if they are still present.
- By workflow: Same as 'By workorder group' except no triggering will occur upon a 'restart' action as the workflow is still the same.
- By action template: Allows the tracking and thus non-triggering in the second workflow for previously processed trigger files shared across workflows that contain the same template,
- absolute: One trigger list is tracked regardless of where it occurs,
- By step: Similar to 'By workflow', but allows for multiple independent trigger steps within the workflow.
- None: No persistence at all. If a past trigger file is still present, it will re-trigger if 'Continue Monitoring' is selected. This option is recommended when 'Continue Monitoring' is not selected.

11.18.6 Outputs Description

Parameter	Description
Folder_list	The matching folders as an array.

11.18.7 Supported Actions

None

11.18.8 Dependencies

None

11.18.9 Operating Instructions

Instructions:

Configure the settings below to control how the system searches for trigger files:

Watch directory:

- Populate the watch directory field with the directory to be scanned for trigger folders.
- Globbing pattern matching is used for the watch directory.
- The search is case sensitive.

File Pattern:

- Populate the folder pattern field with the folder name or pattern used for the trigger.
- Regular expression matching is used for the folder pattern.
- Example: `.*\.txt` matches all `.txt` files.

Remote Node Requirements:

- Works with remote nodes running an Aspera transfer server (e.g., Point-to-Point), as it uses the Aspera `ascmd` executable to browse files.
- The remote node must run an SSH server for authentication purposes.
- The remote username must be defined as an operating system account in `/opt/aspera/etc/aspera.conf`.
- The connection to the remote node must be made by a user with root-level access.
- The watch directory path is relative to the remote user's docroot as defined in `/opt/aspera/etc/aspera.conf`.
- Files and folders must have the correct permissions for the configured remote user in order to be detected.

```
\                <user>\                <name>xfer</name>\                <file_system>\                <access>\
<paths>\                <path>\                <absolute>/home/xfer/data</absolute>\                </path>\
</paths>\                </access>\                </file_system>\                </user>\
```

For example, a watch directory such as `/foo` corresponds to `/home/xfer/data/foo` on the remote server.

Trigger Behavior:

- Select *Continue Monitoring* to periodically scan for arriving folders and act upon their detection..
- *Check only once* and *Continue Monitoring* are incompatible options — do not select both.
- If it is acceptable to trigger on a partial folder, select *Triggers on partial folder* to allow processing before the folder is fully updated.

11.19 Schedule trigger



Figure 11.19: Schedule trigger icon

This action plug-ins provide the ability schedule the triggering of the following steps in a Workflow.

Version	Comment
1.2.2	Enhanced action template UI
1.2.1	Updated the label from Keep ongoing to Continue Monitoring
1.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
1.1.8	Fix backward compatibility issue with rule value with delimiter versus no delimiter

Version	Comment
1.1.7	Fix Rules Evaluation and UI Issue
1.1.6	Use start time when specified
1.1.5	Fix rules input when spaces are present
1.1.4	Fixes timezone issue with HoD rule
1.1.3	Uses database time instead of system time to calculate number of seconds to wait until trigger
1.1.2	Removed debug statements.
1.1.1	Added styling to parameter help text.
1.1.0	Adds support for splitting calculation from processing of the event
1.0.5	Fixes comparison of times to avoid multiple triggering for HoD and MoH
1.0.4	Adds buffer wait to avoids multiple triggering on the same second for a given event.
1.0.3	Added operating instructions in help file for run-time input
1.0.2	Fixes categorization of day rule to allow n days + ToD combination
1.0.1	Adding help file
1.0.0	Re-work the logic to fix multi-rule issues
0.2.4	Use Time instead of DateTime to allow localtime
0.2.3	Correctly expose trigger time output
0.2.2	New icon
0.2.1	fix runtime stop condition
0.2.0	correctly report the next polling time
0.1.9	adds support for dynamic attributes
0.1.8	direct check for single rule
0.1.7	instant cancel
0.1.6	fix erroneous next check time displayed
0.1.5	fix calculation glitch for ToD, HoD
0.1.4	fix resume
0.1.3	fix issue with blank schedules introduced in 0.1.2
0.1.2	improve UI - adds default rule (if rule is a number only, consider it a number of sec wait)
0.1.1	made VAR_ALLOW_MULTIPLE always optional and fix GUI glitch causing loss of configuration

11.19.1 How It Works

- This action plug-in allows you to schedule when subsequent steps in a workflow are triggered.
- Example 1
  - Define a new action template and click the Add Parameter button.
  - Enter the value 300 and select Second(s) from now.
  - When the workflow reaches this step, it will wait 300 seconds before moving to the next step.
- Example 2
  - If a polling step is configured in the workflow, you can place a Schedule Trigger action before it.
  - Use Trigger Start and Trigger End to define the time window during which polling should start and stop.
  - Set the polling frequency by adding the appropriate parameters.

11.19.2 Configuration Instructions

1. Create a new action step using the ScheduleTrigger plugin.
2. Fill out the following fields:
   - Name: The name used to identify a saved ScheduleTrigger configured instance.
   - Comments: Some comments about this saved ScheduleTrigger configured instance.
   - Trigger start: Start time of the trigger.
   - Trigger end: End time of the trigger.
   - Only calculates?: The action will not wait until the scheduled time, but instead just return that time as an output. When this mode is chosen, Continue Monitoring is not possible.
   - Continue Monitoring: States whether the trigger is ran once only, or every time the rule(s) conditions are satisfied.
   - Rules: Specifies the trigger rules. Trigger will run based on the rules provided. E.g. 30 Second(s) from now will have the Workflow execution to move to next step after 30 seconds. Use the 'add rule' button to specify a scheduling rule.

11.19.3 Example Configuration

This is a minimum setup configuration for the ScheduleTrigger plugin.

Parameter	Value
Continue Monitoring	Selected
Rules	300 Second(s) from now

11.19.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved ScheduleTrigger configured instance.
Comments	Some comments about this saved ScheduleTrigger configured instance.
Trigger start	Start time of the trigger.
Trigger end	End time of the trigger.
Only calculates?	The action will not wait until the scheduled time, but instead just return that time as an output. When this mode is chosen, Continue Monitoring is not possible.
Continue Monitoring	States whether the trigger is ran once only, or every time the rule(s) conditions are satisfied.
Rules	Specifies the trigger rules. Trigger will run based on the rules provided. E.g. 30 Second(s) from now will have the Workflow execution to move to next step after 30 seconds. Use the 'add rule' button to specify a scheduling rule.

11.19.5 Inputs Description

The list of inputs depends on the configuration of the ScheduleTrigger action template. If dynamic attributes are used in the message definition, each attribute becomes a required input.

Parameter	Description
schedule_rules *	Specifies the trigger rules. Trigger will run based on the rules provided. E.g. 30 Second(s) from now will have the Workflow execution to move to next step after 30 seconds. Use the 'add rule' button to specify a scheduling rule.
Continue Monitoring	States whether the trigger is ran once only, or every time the rule(s) conditions are satisfied.
Trigger start	Start time of the trigger.
Trigger end	End time of the trigger.

11.19.6 Outputs Description

Parameter	Description
trigger_time	If Continue Monitoring is checked, this output shows the time when the trigger is run.
scheduled_trigger_time	This output indicates the scheduled time at which the action is triggered and the execution proceeds to the next step.

11.19.7 Supported Actions

None

11.19.8 Dependencies

None

11.19.9 Operating Instructions

Design-Time Configuration

- The schedule trigger can be defined either at design time or at run time.
- To configure a schedule at design time:
  - Define a new action template and click the Add Parameter button.
  - At least one rule must be defined.
  - Multiple values can be added using the parameters listed below.
  - The value format must match the format shown in parentheses.

Available Value Formats

- Available Value Formats
  - Second(s) from now
  - Minute(s) from now
  - Hour(s) from now
  - Day(s) from now
  - Month(s) from now
  - Year(s) from now
  - Date/Time — `YYYY-MM-DD hh:mm:ss ZZZ`
  - Time of the Day — `hh:mm:ss`
  - Day of the Week — `0-6` (0 = Sun, 6 = Sat)
  - Day of the Month — `01-31`
  - Day of the Year — `001-366` or `DD MMM`
  - Month of the Year — `01-12`
  - Minute of the Hour — `00-59`
  - Second of the Minute — `00-59`

Run-Time Configuration

- If no rules are defined at design time, rules must be supplied at run time using the schedule_rules input.
- schedule_rules is a string and may contain any combination of the rule formats listed above.
- The simplest form is to provide the number of seconds (as a string or integer).
  - In this case, the Second(s) from now rule is applied automatically.
- If schedule_rules is provided as a Date, the Date/Time rule is applied.
- Multiple rules can be combined by appending the rule definitions together.
  - Each rule uses the format: `'value rule_type'`.

Rule Type Keywords

- Rule Type Keywords
  - sec → Second(s) from now
  - `min` → Minute(s) from now
  - `hour` → Hour(s) from now
  - `day` → Day(s) from now
  - `month` → Month(s) from now
  - `year` → Year(s) from now
  - `date` → Date/Time (YYYY-MM-DD hh:mm:ss ZZZ)
  - `ToD` → Time of the Day (hh:mm:ss)
  - `DoW` → Day of the Week (0-6)
  - `DoM` → Day of the Month (01-31)
  - `DoY` → Day of the Year (001-366 or DD MMM)
  - `MoY` → Month of the Year (01-12)
  - `MoH` → Minute of the Hour (00-59)
  - `SoM` → Second of the Minute (00-59)

Example

```
04:00:00,23:00:00 ToD 0,3 DoW
```

This configuration triggers at 4:00 AM and 11:00 PM on Sundays and Wednesdays, assuming the Continue Monitoring

checkbox is enabled.

11.20 SFTP trigger



Figure 11.20: SFTP trigger icon

Enables hot folder functionality on a remote server using SFTP for automated file transfers.

Version	Comment
0.4.3	Enhanced action template UI
0.4.2	Fixes depth issue and adds recursion_depth to optional inputs
0.4.1	Updated the label from Keep ongoing to Continue Monitoring
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.7	Avoid passwords printed in the logs
0.3.6	Adds pause/resume/cancel support
0.3.5	Fix for serialization issue and overall refactoring
0.3.4	Fix for polling falling into an infinite loop
0.3.3	Fix for current_triggers being nil when shared state does not contain value
0.3.2	Fix for files not detected when re-appearing with the same name on an empty folder
0.3.1	Added Remote Node Dependency
0.3.0	Made passphrase possible as mapped input
0.2.9	Fix for file path being output correctly and to have correct filenames persisted in database
0.2.8	Fix scan once and return, extra slash in output
0.2.7	Restore singular file return output
0.2.6	Use recursive function to do both recursive and non-recursive globbing
0.2.5	Fixing logic when no recursive value is set
0.2.4	Fixed problem with recursive scan not including folders, and using the file selection to select which folder to recursively scan
0.2.3	Corrections to changes made in 0.2.2 - fix version, remove double constant. Added help file
0.2.2	Implement recursion depth
0.2.1	Version not incremented on previous release. Fixing and releasing new minor version
0.2.0	2015-07-29 JY - Add recursive folder search
0.1.0	2015-04-09 ML - Provides the option to scan once and add separate parameter for stability detection
0.0.1	2015-03-11 ML - initial release

11.20.1 How It Works

- Monitors an SFTP server for files.
- Files can be filtered using a specified pattern.
- Supports both password and key-based authentication methods.
- Credentials can be provided either through the Remote Node configuration or directly as plugin inputs/parameters.

11.20.2 Configuration Instructions

1. Create a new action step using the SftpTrigger plugin.
2. Fill out the following fields:

- Name: The name used to identify a saved SftpTrigger configuration instance.
- Comments: Comments related to the SftpTrigger template.
- Remote Node*: The remote node to connect to.
- Remote Address*: IP address or hostname of the SFTP server (e.g., lala.com or 10.0.200.202).
- Remote Port*: The port on which the SFTP server listens for requests.
- Remote User*: The login ID required for authentication on the SFTP server.
- Remote Password*: The password associated with the user for authentication.
- Watch Folder: Directory to monitor for files.
- File Pattern: Pattern used to identify files to monitor.
- Recursion Depth: Specifies how deep to search if the watch folder contains subdirectories.
- Return All Matching Files?: If enabled, returns all matching file names as an array.
- Trigger on Partial Files?: If enabled, stability checks are skipped and partial files will trigger actions.
- Scan Once and Return?: Checks the remote folder only once.
- Polling Frequency: How often the remote server is polled for files; default is 30 seconds.
- Stability Cool-Off Time: Time to wait for a file to stabilize before triggering; default is 30 seconds.
- Keep Ongoing?: Continues monitoring and triggers new work orders for new files.

11.20.3 Example Configuration

NOTE: In this example, the required remote server is configured separately and selected as a `Remote Node`.

Parameter	Value
Name	The name used to identify a saved SftpTrigger configuration instance.
Remote Node	The previously configured and saved remote server.
Watch Folder	/home/sftpuser/sftp_test
File Pattern	*.txt
Recursion Depth	1
Return All Matching Files?	enabled
Scan Once and Return?	enabled

This configuration checks for the creation of files with a .txt extension in the specified sftp_test folder.

11.20.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved SftpTrigger configuration instance.
Comments	Comments related to the SftpTrigger template.
Remote Node	The remote node to connect to.
Remote Address	IP address or hostname of the SFTP server (e.g., lala.com or 10.0.200.202).
Remote Port	The port on which the SFTP server listens for requests.
Remote User	The login ID required for authentication on the SFTP server.
Remote Password	The password associated with the user for authentication.
Proxy Host (Optional)	If an HTTP proxy is used to access the SFTP server, provide the hostname or IP address (e.g., proxy.lala.com or 10.0.200.202).
Proxy Port (Optional)	The port on which the proxy server listens for requests.
Proxy User (Optional)	The login ID for authentication if the proxy server requires it.
Proxy Password (Optional)	The password for the proxy user if authentication is required.

Parameter	Description
Watch Folder	Directory to monitor for files.
File Pattern	Pattern used to identify files to monitor.
Recursion Depth	Specifies how deep to search if the watch folder contains subdirectories.
Return All Matching Files?	If enabled, returns all matching file names as an array.
Trigger on Partial Files?	If enabled, stability checks are skipped and partial files will trigger actions.
Scan Once and Return?	Checks the remote folder only once.
Polling Frequency	How often the remote server is polled for files; default is 30 seconds.
Stability Cool-Off Time	Time to wait for a file to stabilize before triggering; default is 30 seconds.
Keep Ongoing?	Continues monitoring and triggers new work orders for new files.

11.20.5 Inputs Description

The set of available inputs is determined by the configuration of the SftpTrigger action template. All parameters will be accessible as inputs during runtime unless they are explicitly defined within the template. Note that this plugin does not support the overwriting of values; therefore, if a value is specified within the template, it will not be available as an input for the step at runtime.

11.20.6 Outputs Description

Output	Description
Files_found	The list of files found on the remote SFTP server matching the pattern

11.20.7 Supported Actions

None

11.20.8 Dependencies

None

11.20.9 Operating Instructions

This plugin allows monitoring of an SFTP server for files. The files can be filtered using a specified pattern. Both password and key-based authentication methods are supported, and credentials can be provided either through the Remote Node configuration or directly as plugin inputs/parameters.

11.21 Soap request listener

This action plug-ins provides the ability wait for a SOAP message and extract its content.

Version	Comment
0.2.4	Enhanced action template UI
0.2.3	Updated the label from Keep ongoing to Continue Monitoring
0.2.2	Updated help documentation
0.2.1	Fix issue on loading plugin icon

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.1	Making it compatible with the latest version of Orchestrator.
0.1.0	Adding exception handling in controller when action_web_service is required.
0.0.9	Added styling to parameter help text.
0.0.8	Fixes collision when separate instances are used
0.0.7	Adds central-like route to controller, fix issue with output processing of nil
0.0.6	Fix issue with displaying new one
0.0.5	adds support for high-concurrency
0.0.4	adds wsdl generator utility
0.0.3	fixes end point
0.0.2	Adds endpoint to the configurator, remove special characters for step names when used as callback ID
0.0.1	Initial release

11.21.1 How It Works

- This action plug-in waits for inbound SOAP messages sent to the Orchestrator server.
- Incoming SOAP calls are routed to the correct workflow and step using a callback identifier.
- Multiple callback identification strategies are supported, including workflow, step, action template, runtime, and preset identifiers.
- The listener can run continuously and optionally clone the workflow after each trigger event.
- SOAP header, body, raw XML payload, and sender IP address are exposed for custom processing.
- Additional outputs can be dynamically generated using Ruby code.

11.21.2 Configuration Instructions

1. Create a new action step using the SoapRequestListener plug-in.
2. Complete the following configuration fields:
   - Name: The name used to identify a saved SoapRequestListener configuration.
   - Comments: Optional comments explaining the purpose of this listener.
   - Continue Monitoring: If selected, the workflow is cloned after each received SOAP message.
   - Polling frequency: Interval in seconds used to check for incoming SOAP messages (default is 5 seconds).
   - Callback ID mode: Defines how incoming SOAP calls are matched to the listener. Available options include:
     - workflow - Identify the listener using the workflow ID.
     - step - Identify the listener using the workflow step name.
     - action_id - Identify the listener using the action template ID.
     - run-time - Identifier is provided dynamically at run-time.
     - preset - Identifier is predefined or dynamically generated using variables.
   - Preset ID value: Callback identifier used when preset mode is selected. Dynamic attributes such as <%= file_name %> are supported.
   - Processed outputs: List of additional outputs to be generated by this listener.
   - Processed outputs code: Ruby code used to assign values to processed outputs using the @outputs hash.

11.21.3 Example Configuration

Below is a minimal configuration example for the SoapRequestListener plug-in.

Parameter	Value
Name	Wait for External SOAP Call
Continue Monitoring	Selected
Polling frequency	5
Callback ID mode	preset
Preset ID value	encoding_<%= File.nakedname(video_file) %>
Processed outputs	target_path, genre



Figure 11.21: Soap request listener icon

11.21.4 Saved Parameters Description

11.21.5 Inputs Description

Inputs depend on the selected callback identification strategy. If dynamic attributes are used, each attribute becomes a runtime input.

Parameter	Description
*Run-time call back ID **	Required when Callback ID mode is set to 'runtime'. Used to match incoming SOAP calls.

11.21.6 Outputs Description

Parameter	Description
Remote_address	IP address of the SOAP call sender.
XML_payload	Raw SOAP XML payload as a string.
Header	SOAP header parsed as a Hash.
Body	SOAP body parsed as a Hash.

Additional outputs defined in Processed outputs are returned with values assigned by the processed outputs code.

11.21.7 Supported Actions

Trigger

11.21.8 Dependencies

Network connectivity to the Orchestrator SOAP listener endpoint and an external system capable of sending SOAP messages.

11.21.9 Operating Instructions

1. The listener waits until a SOAP message is received by the Orchestrator server.
2. SOAP messages must be sent to an endpoint containing the callback identifier.
3. Endpoint format: http://<orchestrator_address>/aspera/orchestrator/soap_listener/<callback_id>
4. The SOAP action must be SOAPAction: ”/soap_request_listener/api/Trigger”.
5. A WSDL can be generated dynamically at http://<orchestrator_address>/aspera/orchestrator/soap_request_listener/wsdl.
6. Callback ID format depends on the selected Callback ID mode.
7. Processed outputs can be extracted from the SOAP body using Ruby code.

11.22 SonyCi file watcher

This plugin monitors Sony Ci folders for new files matching specified patterns and returns file information when matches are detected.

Version Comment



Figure 11.22: SonyCi file watcher icon

11.22.1 How It Works

- Monitors Sony Ci folders for new files that match specified patterns using regular expressions.
- Connects to Sony Ci using API credentials (username, password, client id, and client secret).
- Supports both one-time file detection and continuous monitoring with configurable polling intervals.
- Returns file information for files with 'Complete' status, either as single matches or arrays.
- Includes trigger persistence options to prevent duplicate processing across workflow instances.

11.22.2 Configuration Instructions

1. Add a SonyCiFileWatcher step to your workflow.
2. Configure the Sony Ci authentication:
   - Username: Your Sony Ci account username.
   - Password: Your Sony Ci account password.
   - Client id: Sony API client id for authentication.
   - Client secret: Sony API client secret for authentication.
3. Set up the file monitoring parameters:
   - Folder id: The specific folder id to monitor in Sony Ci.
   - File pattern: Regular expression pattern for file matching (e.g., .*\.mxf for MXF files).
   - Polling frequency: Interval in seconds between file checks.
   - Return single match?: Check to return only one file per instance.

11.22.3 Example Configuration

Parameter	Example Value
Username	user@company.com
Password	********
Client id	abc123def456
Client secret	********
Folder id	321e7522a7a74c9aadcb631d3bc0b778
File pattern	.*\.mxf
Polling frequency	30
Check only once	unchecked
Continue Monitoring	checked
Return single match?	unchecked
Trigger persistence scope	by workorder group

11.22.4 Saved Parameters Description

Parameter	Description
Username	The Sony Ci account username for authentication.
Password	The Sony Ci account password for authentication.

Parameter	Description
Client id	The Sony API client id used for authentication.
Client secret	The Sony API client secret used for authentication.
Folder id	The folder id to search into. Files are not searchable in subfolders.
File pattern	The detection file name pattern expressed as a regular expression.
Polling frequency	Interval in seconds to search for matching files.
Check only once	Check for matching file presence (do not wait) and stop processing.
Continue Monitoring	Wait for matching file(s), spawn a new workflow instance each time matching file(s) are detected and proceed with the next steps. The new instance will wait for new matching files.
Return single match?	Return only one matching file per workflow instance. Otherwise return matching files as an array.
Trigger persistence scope	Sets the scope/availability of the trigger output. Options: 'by workorder group', 'by action template', 'absolute', 'by step', 'by workflow', 'none'.

11.22.5 Inputs Description

- Username: The Sony Ci account username. Overrides configuration value.
- Password: The Sony Ci account password. Overrides configuration value.
- Client_id: The Sony API client id used for authentication. Overrides configuration value.
- Client_secret: The Sony API client secret used for authentication. Overrides configuration value.
- Folder_id: The folder id to search into. Overrides configuration value.
- File_pattern: The detection file pattern as regular expression. Overrides configuration value.

11.22.6 Outputs Description

- File_info_list: The matching file information list as an array of hash structures.
- File_info: A single matching file information as a Ruby hash structure containing id, name, size, createdOn, status, and kind fields.

Sample hash structure returned by the plugin:

```
{ "id" => "321e7522a7a74c9aadcb631d3bc0b778", "name" => "RAW football pkg.mxf", "size" => 1150732785, "createdOn" => "2020-10-06T13:38:00.75Z", "status" => "Complete", "kind" => "Asset" }
```

11.22.7 Trigger Persistence Scope Details

- 'by workorder group' - Trigger files are tracked while the workorder is executing and will only be captured once for a given workorder group. If the workorder is restarted (new workorder group), all past triggers will be detected again if still matching the search criteria.
- 'by workflow' - Same as 'by workorder group' except no triggering will occur upon a restart workorder action for the same workflow.
- 'by action template' - Allows the tracking and thus non-triggering in another workflow for previously processed trigger files shared across workflows via the same template.
- 'absolute' - The trigger list is tracked regardless of where it occurs.
- 'by step' - Similar to 'by workflow', but allows for multiple independent trigger steps within the workflow.
- 'none' - No persistence at all. If a past trigger file is still present, it will re-trigger if 'Continue Monitoring' is selected. This option is recommended when 'Continue Monitoring' is not selected.

11.22.8 Operating Instructions

- This plugin detects new files matching a certain pattern, located in a folder within Sony Ci. Both the pattern matching and the folder id must be provided.
- The folder may be searched a single time for matching files (do not select 'Continue Monitoring') or can be treated similarly to a watchfolder where it is continually scanned for matching files (select 'Continue Monitoring').

- Matching file presence can be checked once (select 'Check only once'), with a failure being returned if no matching file is found.
- 'Check only once' and 'Continue Monitoring' are incompatible options - do not select both of them.
- Specify the file pattern with a regular expression for the files to search for. Examples: `.*` for any file, `.*\.json` for files with json extension, `(.*\.mxf|.*\.mpg)` for multiple patterns.
- Select 'Return single match?' if you want to retrieve only one matching file per workflow instance. If not selected, all matching files are returned in an array.
- Only files with a 'Complete' status are returned by the plugin.
- Files are not searchable in subfolders of the specified folder - only direct contents are monitored.

11.22.9 Dependencies

- The api.cimediacloud.com endpoint must be reachable on TCP port 443 from your Orchestrator system.
- Valid Sony Ci account credentials and API client credentials are required.
- The specified folder id must exist and be accessible with the provided credentials.

11.22.10 Troubleshooting

- If authentication fails, verify that the username, password, client id, and client secret are correct and that the API credentials are active.
- If no files are found, check that the folder id exists and contains files with 'Complete' status that match your regular expression pattern.
- Network connectivity issues: Ensure that api.cimediacloud.com is reachable on port 443 from your Orchestrator system. Check firewall settings if needed.
- If the plugin returns duplicate files when 'Continue Monitoring' is enabled, review the 'Trigger persistence scope' setting to ensure proper deduplication.
- Regular expression patterns are case-sensitive. Use appropriate regex syntax for your file matching requirements.
- If polling frequency is too low, you may experience performance issues. Consider increasing the interval for large folders or high-frequency monitoring.

Chapter 12

User Interactions

12.1 Console notification



Figure 12.1: Console notification icon

This plugin provides the ability to insert entries in the Aspera Console database representing Orchestrator workflow events, enabling integration between Orchestrator and Console for comprehensive monitoring and tracking of automated transfer activities.

Version	Comment
0.5.1	Enhanced action template UI
0.5.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.4.3	Added styling to input help text.
0.4.2	2016-02-18 RT - Added help file
0.4.1	2015-05-19 ML - Adds support for transaction with un-managed nodes
0.4.0	2015-02-20 ML - Adds full control over cookie formatting. New icon
0.3.3	adds implicit conversion of int to string for the numeric fields
0.3.2	adding missing constants and merging Marc's code
0.3.1	more fixes for console 2.3.1, disable auto-complete for login and adds ability to provide console info as parameter
0.3.0	fixes for console 2.3.1
0.2.0	adds nodes support
0.1.4	adds functionality to mask password
0.1.3	bypass console connection if DB connection can not be established
0.1.2	explicitly closes open DB connections
0.1.1	bugfix in journaling status
0.1.0	Adding journaling
0.0.1	Initial release

12.1.1 How It Works

- Sends notifications to the console database upon specific workflow events, providing real-time status updates and monitoring capabilities.
- Integrates with console servers to record transfer session information and event data in the fasp_sessions table.
- Supports flexible cookie pattern generation with dynamic content including workstep IDs and event payloads for customizable notifications.
- Provides optional node credential authentication for secure database connections to remote console servers.
- Enables journaling of transactions with configurable status and step information for comprehensive workflow tracking.
- Supports runtime variable expansion in event payloads for dynamic content generation based on workflow context.

12.1.2 Configuration Instructions

1. Add a Console Notification step to your workflow.
2. Configure console server connection:
    - Console node name or host address: Enter the console server IP or hostname (e.g., 192.168.1.100).
    - Use node credentials?: Check to provide custom database credentials. Leave unchecked to use default console database credential values.
    - Console db port: Specify the database port (default: 4406).
    - Console db user: Enter the database username (e.g., root).
3. Set up notification parameters:
    - Aspera cookie pattern: Configure the cookie format for notifications.
    - Event payload: Define the content for event notifications.
    - Journal the transaction?: Enable transaction journaling if required.

12.1.3 Example Configuration

Parameter	Example Value
Name	New ConsoleNotification Step
Console node name or host address	192.168.1.100
Use node credentials?	checked
Console db port	4406
Console db user	root
Console DB password	<previous password>
Aspera cookie pattern	aspera.orchestrator:<%=workstep_id%>:<%=event%>
Event payload	test
Journal the transaction?	unchecked

12.1.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify this Console Notification step configuration.
Comments	Optional comments to describe the purpose and configuration details of this notification step.
Console node name or host address	The remote node name/address of the console server where notifications will be sent.
Use node credentials?	Check this if credentials need to be provided for the console server. When checked, the console port, user and passwords should be provided.

Parameter	Description
Console db port	The port number of the console database server.
Console db user	The username to login to the console database server.
Console DB password	The password to login to the console database server.
Aspera cookie pattern	By default, the Cookie is generated using formatted 'aspera.orchestrator:[workstep id]:[expanded event payload]'. The string used can contain static text and refer to the executing workstep ID as and the expanded event payload as .
Event payload	The event_payload is used to generate the content of the event field included as the [expanded event payload] section of the cookie. The string used can contain static text and variables expanded at run-time.
Saved Inputs	The default values for the runtime variables.
Journal the transaction?	Check this if the transaction needs to be journaled in the console database.
Journal status	The status of the file that would be journaled.
Journal step	The step name to be journaled.

12.1.5 Inputs Description

All parameters are available as inputs at runtime if not specified by the template. This plugin does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

12.1.6 Mandatory Inputs:

- Console node name or host address: The remote node name/address of the console server (required if not specified in template).
- event_name: This is a mandatory runtime variable that must be provided for the notification.

12.1.7 Optional Runtime Inputs:

- Bytes_transferred: Number of bytes transferred in the operation (string format).
- Bytes_written: Number of bytes written during the operation (string format).
- Console_DB_password: Console database password if not specified in template.
- Console_DB_user: Console database username if not specified in template.
- destination_path: Destination path for the transfer operation.
- error_message: Error message to include in the notification if applicable.
- files_complete: Array of successfully completed files.
- files_failed: Array of files that failed during the operation.

12.1.8 Outputs Description

- Constructed Message: The only output provided is the constructed notification message that was sent to the console database.

12.1.9 Setup Instructions

- Whitelist the orchestrator IP from console by running: `asctl mysql:grant_remote_access <orch-ip> <user>`
- In the action template, in Map inputs enter `node_from` and `node_to`, which are console managed node names.
- Enter the remaining details in the Edit tab including console connection parameters.
- On launch, the workorder should add an entry into the `fasp_sessions` table of the console database.

12.1.10 Operating Instructions

- Database Connection: Ensure the console database is accessible from the Orchestrator server and that proper network connectivity exists on the specified port.
- Cookie Pattern Customization: The cookie pattern supports dynamic variables including <workstep_id> for the current workflow step ID and <event> for the expanded event payload content.

- Event Payload Configuration: Configure the event payload to include relevant workflow information that will be expanded at runtime and included in the notification cookie.
- Node Credentials: When 'Use node credentials?' is checked, provide the console database port, username, and password for secure authentication.
- Transaction Journaling: Enable transaction journaling to record detailed workflow step information in the console database for audit and monitoring purposes.
- Runtime Variables: Use saved inputs to provide default values for runtime variables that can be overridden when the workflow executes.

12.1.11 Dependencies

- Console server with accessible database (typically MySQL on port 4406).
- Orchestrator server needs to be whitelisted on the Console database. This can be done by running the following command on the Console server: `asctl mysql:grant_remote_access <orchestrator host> <orchestrator account>`
- Valid database credentials with appropriate permissions for the fasp_sessions table.
- Network connectivity between Orchestrator and console server on the specified database port.
- Console managed node names need to be configured for node_from and node_to parameters.
- Orchestrator and Console should be configured on the same version of MySQL.

12.1.12 Troubleshooting

- If database connection fails, verify that the console server IP and port are correct and that the database service is running.
- For authentication errors, confirm that the database username and password are correct and that the user has appropriate permissions.
- If the orchestrator IP is blocked, ensure that `asctl mysql:grant_remote_access` has been run with the correct IP and user parameters.
- For cookie pattern issues, verify that the <workstep_id> and <event> variables are properly formatted and that the event payload contains valid content.
- If entries are not appearing in the fasp_sessions table, check that the console database schema is correct and that the user has INSERT permissions.
- When using node credentials, ensure that 'Use node credentials?' is checked and all required credential fields are populated.
- For journaling issues, verify that the journal status and step parameters are correctly configured and that the database supports the journaling schema.

12.2 Email notification



Figure 12.2: Email notification icon

This action plugin provides the ability to send email as part of the execution of a workflow. It can be configured with recipients, subject, message body, and optional sender details to enable workflow-driven email notifications.

Version	Comment
0.7.4	Enhanced action template UI

Version	Comment
0.7.3	Fix issues when using legacy mode
0.7.2	Fix escaping issues in HTML message template
0.7.1	Removing Legacy support
0.7.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.6.0	Adds a flag to force attachments encoding in base64
0.5.6	Handle deactivated Mailer Configurations
0.5.5	Make new email address separator comaptible with old one
0.5.4	Handle issue with email address containing _- characters
0.5.3	Reinitializing the rich text editor after everytime the checkbox for legacy mode is checked/unchecked.
0.5.2	Undoing the expand of Payload when destinations are fetched for UI.
0.5.1	Added address variable which was not defined in find_destinations method.
0.5.0	Added option to exclude not compliant email address
0.4.6	Added styling to parameter help text.
0.4.5	JG Fixing code to consider map input params
0.4.4	2015-03-19 ML - Modify JS function name to avoid collisions
0.4.3	2015-02-24 ML - New icon
0.4.2	Prevents crash and wrong matcher result when runtime email addresses contain a newline chars
0.4.1	Prevents issue when of crash when importing previous versions.
0.4.0	Add explicit check for backward compatibility to pre 0.3.0
0.3.1	Default template to blank to ensure backward compatibility
0.3.0	Replaces payload with form
0.2.2	Uses mailer default (requires 1.9.8 00010)
0.2.1	Adds Specific message when no mailers are configured
0.2.0	Adds support for attachments
0.1.3	Supports its methods for payload expansion
0.1.2	adds support for implicitly named mail servers
0.1.1	adds support for rich textarea editor for xml and html fields
0.1.0	adds support for multiple mail server configurations
0.0.1	initial release

12.2.1 How It Works

- Sends an email using a configured SMTP Mailer (set up under *Engine > Mailer Configurations*).
- The email's core parts (Destinations, Subject, Message) are defined by the Message Core parameters template.
- An optional HTML Message Template can define the HTML body; the plain *Message* is used for non-HTML clients.
- Both templates support runtime variables using `<%= variable_name %>`, which become inputs at run time.
- You can attach local files by passing absolute paths in `attachment_list` (comma-separated).
- The full XML of the constructed email is returned in the FullMessage output, even if sending fails.

12.2.2 Configuration Instructions

1. Ensure a valid Mailer configuration exists in Engine > Mailer Configurations.
2. Create a new action step using the EmailNotification plug-in.
3. Fill out the following fields:
   - Mailer: Pick the SMTP configuration to use (e.g., `test`).
   - Message Core parameters: Leave blank to use the default template (inputs: `To`, `Subject`, `Message`), or provide a custom XML template (see examples below).
   - HTML Message Template: Optional HTML body; supports `<%= %>` variables.
   - Force attachments encoding in base64: If checked, all attachments are base64-encoded for compatibility.
4. Map inputs (Default template example):
   - To: `user@example.com`
   - Subject: `Hi! This is a test email.`
   - Message: `Email generated by Aspera Orchestrator`
5. Optional: Set `attachment_list` to absolute file paths (comma-separated), e.g., `/tmp/report.pdf,/var/log/app.log`.

12.2.3 Example Configuration

Field	Value
Mailer	`test`
Message Core parameters	Default (leave blank)
Destinations	`To` → `user@example.com`
Subject	`Hi! This is a test email.`
HTML Message Template	Empty (plain message used for non-HTML clients)
Force attachments encoding in base64	Unchecked

Behavior: With the default template and example values above, the step sends an email to `user@example.com`. The FullMessage output contains the XML that was sent.

12.2.4 Saved Parameters Description

Parameter	Description
Name	Label to identify this EmailNotification step.
Comments	Optional notes about this step.
Mailer	SMTP server configuration to use (configured under *Engine > Mailer Configurations*).
Message Core parameters	XML template describing Destinations, Subject, and Message; supports `<%= %>` variables.
HTML Message Template	Optional HTML body template for HTML-capable clients; supports `<%= %>` variables.
Force attachments encoding in base64	When enabled, encodes all attachments in base64.

12.2.5 Inputs Description

The available inputs depend on the templates you define:

- Default template (when *Message Core parameters* is left blank)
  - To: Destination address.
  - Subject: Email subject text.
  - Message: Plain-text message body (used if no HTML body or client doesn't support HTML).
- Custom templates — Any `<%= variable %>` appearing in your templates becomes an input at run time.
- Always available (optional)
  - MessageDefinition_Override: Raw body text that overrides both *Message Core parameters* and the *HTML Message Template*.
  - attachment_list: Comma-separated list of absolute file paths to attach.

12.2.6 Outputs Description

- FullMessage: The entire email in XML form (same schema as defined by *Message Core parameters*). This output can be present even if sending failed.

12.2.7 Operating Instructions

12.2.8 *Defining the Message Core parameters*

Provide an XML template describing how to build the email. Minimal fields are *Destinations*, *Subject*, and *Message*.

\ <MessageDefinition>\ <Destinations><To>orchestrator@asperasoft.com</To></Destinations>\ <Subject>New notification</Subject>\ <Message>You have received a new notification.</Message>\ </MessageDefinition>\

- *Destinations*: list of recipients (one or more `<To>` elements).
- *Subject*: email subject line.
- *Message*: plain-text body used when no HTML body is present or the client cannot render HTML.

Any field can contain runtime variables using `<%= variable_name %>`. Those variables become step inputs.

Example — destination resolved at run time

<MessageDefinition>\ <Destinations><To><%= user_address %></To></Destinations>\ <Subject>New notification</Subject>\ <Message>You have received a new notification.</Message>\ </MessageDefinition>\

Default template (when left blank)

<MessageDefinition>\ <Destinations><To><%= To %></To></Destinations>\ <Subject><%= Subject %></Subject>\ <Message><%= Message %></Message>\ </MessageDefinition>\

12.2.9 *Defining the HTML Message Template*

Provide standard HTML for the email body. Runtime variables with `<%= %>` are supported and exposed as inputs.

12.2.10 *Attaching files*

Supply a comma-separated list of absolute file paths via `attachment_list`. Files must be locally accessible to the Orchestrator machine. Enable base64 encoding if required by your mail system.

12.2.11 Supported Actions

None

12.2.12 Dependencies

A valid Mailer configuration is required under Engine > Mailer Configurations.

12.2.13 Troubleshooting

- If the step uses a deactivated Mailer configuration, the step completes but NO email is sent; FullMessage still contains the constructed XML.
- Verify SMTP credentials and connectivity in the Mailer configuration.
- Ensure all `<%= %>` variables used in templates are provided as inputs.
- Confirm attachment paths are absolute and readable by the Orchestrator host.
- If HTML is blank, only the plain *Message* appears for non-HTML clients.

12.3 RSS Feed Reader

This action plug-in provides the ability to read an RSS feed and can be used as a trigger.

Version	Comment
0.5.2	Enhanced action template UI
0.5.1	Updated the label from Keep ongoing to Continue Monitoring
0.5.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.4.2	Adds compatibility with open-uri 0.1.0
0.4.1	Deletes temporary files
0.4.0	Added option to disable SSL Verification

Version	Comment
0.3.1	Adding detailed information in case of error reading the feed
0.3.0	Adding a flag to cherry pick which steps want to use simple-rss vs rss
0.2.2	Fixed correct detection of link. Added output for a list of item keys
0.2.1	Fixed detection of missing items
0.2.0	Added option to only trigger on new posts
0.1.7	Removed excessive logs
0.1.6	Add error message in case of internal failure
0.1.5	Defer RSS item deletion from persistent store to avoid false triggering because of RSS feed not always showing the entire list
0.1.4	Adding protection in case the RSS feed cannot be read
0.1.3	Enhancing functionality added in 0.1.2 to extract attributes from non standard tags
0.1.2	Adding support for non-standard tags to be retrieved
0.1.1	Fixed problem when selecting to return all entries
0.1.0	Added option to do (or don't do) validation
0.0.4	2015-03-11 ML - New icon
0.0.3	2015-01-15 ML - Fixes issue with polling frequency
0.0.2	2014-11-14 ML - Fixes issue with parsed Time outputs, adds help file
0.0.1	2014-11-11 ML - initial release

12.3.1 How It Works

- This action reads an RSS feed and can be used as a trigger to produce outputs for new posts.

12.3.2 Configuration Instructions

1. Create a new action step using the RssFeedReader plugin.
2. This action polls the configured RSS feed and emits outputs for new posts according to the configuration.
3. Fill out the following configuration fields:
    - Name: The name used to identify a saved RssFeedReader configuration.
    - Comments: Optional comments about this saved RssFeedReader configuration.
    - RSS URL: The HTTP URL of the RSS feed, e.g., http://rss.cnn.com/rss/cnn_topstories.rss.
    - Post ID Field: Optional. The field used to uniquely identify a post (e.g., 'id' or 'guid'). If omitted, the entire post is used as the identifier.
    - Polling Frequency: Polling interval in seconds (default: 60).
    - Continue Monitoring: To use as a trigger, set a non-null polling frequency.
    - Return All Posts?: By default posts are returned one by one (oldest first). If enabled, all new posts found are returned as an array.
    - Ignore Posts Already in Feed on Start-up?: If enabled, only newly published posts will trigger; otherwise existing items will also be returned on startup.
    - Validate RSS Feed?: If enabled, the feed is validated before items are analyzed.
    - Ignore SSL Validation Errors?: If enabled, SSL certificate validation errors are ignored.
    - Return Entire Post as XML?: If enabled, an additional output contains the full XML of the post as text.
    - Additional Outputs: Configure extraction of specific fields (e.g., 'guid' or 'pubDate'). Output names are produced by capitalizing the field name (e.g., 'pubDate' → 'Pubdate').
    - Additional Output Extraction Code: Optional Ruby code to transform extracted values. The raw post is available as 'raw_post' (String) and as a Hash 'post' keyed by the capitalized output names (e.g., 'Pubdate').

12.3.3 Example Configuration

This is a minimal setup configuration for the RssFeedReader plugin.

Parameter	Value
RSS URL	http://rss.cnn.com/rss/cnn_topstories.rss
Post ID Field	guid
Validate RSS Feed?	Selected
Ignore Posts Already in Feed on Start-up?	Selected
Return entire post as XML?	Selected

Parameter	Value
Continue Monitoring	Selected
Additional outputs -	guid -> String/Text
Additional outputs -	pubDate -> Date/Time

The resulting outputs will be as follows each time a new post is found:

Guid (String) -> http://example.com/article\ \ Post_XML (String) ->\ <item>...</item>\ \ Title (String) -> Example title\ \ Description (String) -> Example description\ \ Pubdate (Date) -> 2018-10-03 10:15:35 EDT\ \ Link (String) -> http://example.com/article\

12.3.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved RssFeedReader configuration.
Comments	Optional comments about this saved RssFeedReader configuration.
RSS URL	The HTTP URL of the RSS feed, e.g., http://rss.cnn.com/rss/cnn_topstories.rss.
Post ID Field	Optional. The field used to uniquely identify a post (e.g., 'id' or 'guid'). If omitted, the entire post is used as the identifier.
Polling Frequency	Polling interval in seconds (default: 60).
Continue Monitoring	To use as a trigger, set a non-null polling frequency.
Return All Posts?	By default posts are returned one by one (oldest first). If enabled, all new posts found are returned as an array.
Ignore Posts Already in Feed on Start-up?	If enabled, only newly published posts will trigger; otherwise existing items will also be returned on startup.
Validate RSS Feed?	If enabled, the feed is validated before items are analyzed.
Ignore SSL Validation Errors?	If enabled, SSL certificate validation errors are ignored.
Return Entire Post as XML?	If enabled, an additional output contains the full XML of the post as text.
Additional Outputs	Configure extraction of specific fields (e.g., 'guid' or 'pubDate'). Output names are produced by capitalizing the field name (e.g., 'pubDate' → 'Pubdate').
Additional Output Extraction Code	Optional Ruby code to transform extracted values. The raw post is available as 'raw_post' (String) and as a Hash 'post' keyed by the capitalized output names (e.g., 'Pubdate').

12.3.5 Inputs Description

Required inputs depend on how the RssFeedReader template is configured. If dynamic attributes are used, each attribute becomes a runtime input.

Parameter	Description
rss_url *	The HTTP URL of the RSS feed to poll.
post_id_field	Optional field used to uniquely identify posts; if omitted the entire post is used.

12.3.6 Outputs Description



Figure 12.3: RSS Feed Reader icon

Parameter	Description
Description	The content of the description field.
Link	The content of the link field.
Title	The content of the title field.
Post_XML	The entire post XML as text (only present if 'Return entire post as XML?' is enabled).
Item_keys	List of keys available in the feed item; one can be used as the post identifier.

Additional outputs are created for each configured extraction. Output names are capitalized versions of the extracted fields (e.g., 'pubDate' -> 'Pubdate').

12.3.7 Supported Actions

None

12.3.8 Dependencies

None

12.3.9 Operating Instructions

1. Set the RSS URL and polling frequency; to use the plugin as a trigger, choose a non-null polling frequency.
2. Optionally set the Post ID field to uniquely identify posts; otherwise the entire post is used.
3. Select whether to return posts individually or as an array, and whether to ignore existing items at startup.
4. Optionally enable feed validation or ignore SSL validation errors as needed.
5. If 'Return entire post as XML?' is enabled, use the provided XML output for downstream processing.

12.4 SCOM notification



Figure 12.4: SCOM notification icon

This action plugin provide the ability to submit log entry to the Windows Event Log. The execution can only occur on a windows host.

Version	Comment
1.2.0	Removing Windows support code, Updating to be compatible with Ruby 3.4 and Rails 8



Figure 12.5: Slack notification icon

12.5 Slack notification

This action plug-in provides the ability to interact with Slack

Version	Comment
0.2.2	Enhanced action template UI
0.2.1	Update to be compatible with latest Slack API (tested with Slack v4.47)
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.1.0	added attachments
0.0.3	regategorized
0.0.2	changed token type to password
0.0.1	Initial release.

12.5.1 How It Works

- Sends messages to Slack channels using the Slack API.
- Supports both user tokens and bot tokens for authentication.
- Allows posting messages as the authenticated user or as a bot.
- Supports rich message formatting with attachments, colors, titles, and text.
- Returns message metadata including timestamp and team information for tracking.

12.5.2 Configuration Instructions

1. Add a SlackNotification step to your workflow.
2. Configure the basic parameters:
   - Name: Enter a descriptive name for the Slack notification.
   - Comments: Add notes or description for this step.
3. Set up Slack API connection:
   - token: Token used to authenticate Slack API (user token or bot token).
   - slack url: Provide the Slack URL.
   - operation: Currently only 'send message to a channel' operation is supported.
4. Configure message parameters:
   - channel: Enter the channel name here (e.g., #general, #notifications).
   - text_message: Text message to be posted in the channel.
   - as_user: Pass true to post the message as the authenticated user, instead of as a bot. Defaults to false.
5. Configure message attachments (optional):
   - color: The border color along the left side of a message attachment.
   - title: The title displayed as larger, bold text near the top of a message attachment.
   - text_attachment: The main text in a message attachment.

12.5.3 Example Configuration

Parameter	Example Value
Name	Workflow Completion Notification
Comments	Sends notification when workflow completes
token	
slack url	https://slack.com/
operation	Send a Message
channel	#my-channel
text_message	Workflow completed successfully
as_user	true
color	#3AA3E3
title	Workflow Status
text_attachment	All processing steps completed without errors

12.5.4 Saved Parameters Description

Parameter	Description
Name	Display name for this Slack notification instance.
Comments	Notes or description for this step.
token	Token used to authenticate Slack API. Can be a user token or bot token.
slack url	The slack URL (e.g., https://slack.com/).
operation	The operation to perform. Currently only 'send message to a channel' is supported.
channel	The target channel name where the message will be posted (e.g., #general, #notifications).
text_message	The main text message to be posted in the channel.
as_user	Pass true to post the message as the authenticated user, instead of as a bot. Defaults to false.
color	The border color along the left side of a message attachment. Can use preset colors (good, warning, danger) or hex codes.
title	The title displayed as larger, bold text near the top of a message attachment.
text_attachment	The main text content in a message attachment. Supports markdown formatting.

12.5.5 Inputs Description

Changes in this list of inputs will override any values set in the action template.

Mandatory Inputs:

- token: Token used to authenticate Slack API (user token or bot token).
- slack_url: Provide the Slack workspace URL.
- channel: Enter the channel name here (e.g., #general, #notifications).
- text_message: Text message to be posted in the channel.

Optional Inputs:

- attachments: List of message attachments expressed as JSON-based array. Example: [{ ”color”:”warning”,”title”:”Message1”,”text”:”Details1” }, { ”color”:”#FF0000”,”title”:”Message2”,”text”:”Details2”,”footer”:”Signature” }]

12.5.6 Outputs Description

- Team_Name: Team name associated with the authentication token.
- Team_Id: Team ID for the authentication token.
- Company_Url: URL of the company/workspace the token belongs to.
- Error_Message: In case of error, this field stores the error cause.
- Auth_Response_Received: Raw response from authentication API call.
- TS_Value: Timestamp value of the sent message (used for message threading and updates).
- Send_Text_Response: Raw response from the send message API call.

12.5.7 Supported Actions

None

12.5.8 Dependencies

- Valid Slack workspace with appropriate permissions.
- Slack API token (User Token or Bot Token) with chat:write scope.
- Network connectivity to Slack API endpoints.
- Channel must exist and the bot/user must be a member of the target channel.

12.5.9 Operating Instructions

12.5.10 Overview

The SlackNotification plugin enables workflows to send messages to Slack channels using the Slack API. Messages can be posted as the authenticated user or as a bot, and support rich formatting with attachments, colors, and titles.

12.5.11 Authentication Setup

- Token Types: The plugin supports both User Tokens (xoxp-) and Bot Tokens (xoxb-).
- User Token: Messages posted with user tokens appear as if sent by that user. Requires user-level permissions.
- Bot Token: Messages posted with bot tokens appear as sent by the bot. More suitable for automated notifications.
- Required Scopes: The token must have the `chat:write` scope at minimum. Additional scopes may be needed for advanced features.
- Obtaining Tokens: Create tokens in your Slack workspace settings under 'Apps' > 'OAuth & Permissions'.

12.5.12 Channel Configuration

- Channel Format: Prefix public channels with # (e.g., #general, #notifications).
- Private Channels: The bot or user must be invited to private channels before posting messages.
- Direct Messages: Use the user ID format (e.g., @U1234567890) to send direct messages.
- Channel Validation: Ensure the channel exists and the bot/user is a member to avoid errors.

12.5.13 Message Formatting

12.5.14 Basic Text Messages

- text_message: The primary message content. Supports Slack's mrkdwn formatting.
- Markdown Support: Use *bold*, _italic_, ~strikethrough~, and ‘code‘ formatting.
- Links: Format as <https://example.com|Link Text> for hyperlinks.
- Mentions: Use <@U1234567890> to mention users or <!channel> to notify all channel members.

12.5.15 Message Attachments

Attachments provide rich formatting with color-coded borders, titles, and structured content.

- JSON Format: Attachments must be provided as a JSON array.
- Color Options: Use preset values (good, warning, danger) or hex codes (#FF0000).
- Structure: Each attachment can include color, title, text, and optional footer.
- Multiple Attachments: Include multiple attachment objects in the array for complex messages.

Attachment Example:

\[\ {\ ”color”: ”good”,\ ”title”: ”Success”,\ ”text”: ”Workflow completed successfully”\ },\ {\ ”color”: ”#FF0000”,\ ”title”: ”Error Details”,\ ”text”: ”No errors encountered”,\ ”footer”: ”Orchestrator Bot”\ }\ ]\

12.5.16 Posting As User vs Bot

- as_user = true: Messages appear as sent by the authenticated user. (Required if using a user token)
- as_user = false (default): Messages appear as sent by a bot.
- Bot Customization: When posting as bot, you can modify the display name in Slack app settings (App Home > Display Information). The username parameter (available in < v0.2.0 of this plugin) is deprecated in modern Slack API.
- Use Case: Use bot mode for system notifications, user mode for messages that should appear as from a specific person.

12.5.17 Sample Request

Here's a complete example configuration:

\ channel: #plugin_test\ text_message: Workflow execution completed successfully\ username: Orchestrator Bot\ attachments: [\ {\ ”color”: ”good”,\ ”title”: ”Execution Summary”,\ ”text”: ”All steps completed without errors”\ }\ ]\

12.5.18 Output Processing

- Error_Message: Check this output first to determine if the message was sent successfully. Empty value indicates success.
- TS_Value: The timestamp of the sent message. Save this if you need to update or thread messages later.
- Team_Name and Team_Id: Useful for verifying which workspace the message was sent to.
- Send_Text_Response: Contains the full API response for debugging purposes.
- Auth_Response_Received: Contains authentication details, useful for troubleshooting token issues.

12.5.19 Common Color Values

- good: Green border (use for success messages).
- warning: Orange/yellow border (use for warnings).
- danger: Red border (use for errors or critical issues).
- Hex codes: Use any hex color code for custom colors (e.g., #0000FF for blue).

12.5.20 Error Handling

- Invalid Token: Check Error_Message output for 'invalid_auth' or 'token_revoked' errors.
- Channel Not Found: Ensure the channel name is correct and includes the # prefix for public channels.
- Not in Channel: The bot/user must be invited to the channel before posting. Check for 'not_in_channel' error.
- Missing Scopes: Verify the token has the required chat:write scope in Slack app settings.
- Rate Limits: Slack enforces rate limits. Space out notifications if sending many messages rapidly.

12.5.21 Best Practices

- Use bot tokens for automated notifications to keep user tokens secure.
- Always check the Error_Message output to verify successful message delivery.
- Save the TS_Value output if you need to reference or thread messages later.
- Use attachments for structured, visually distinct messages rather than plain text.
- Choose appropriate colors for attachments to convey message severity (good, warning, danger).
- Test messages in a development channel before implementing in production workflows.
- Configure bot display names in Slack app settings (App Home > Display Information). The username parameter (available in < v0.2.0 of this plugin) is deprecated in modern Slack API.
- Avoid sending too many messages in rapid succession to prevent rate limiting.
- Ensure the bot is invited to all target channels before deploying workflows.
- Store Slack tokens securely and rotate them periodically for security.

12.6 User input

This plugin provides an interactive way to present information and collect data from users through web forms within the Orchestrator control application.

Version	Comment
0.8.2	Enhanced action template UI
0.8.1	Fix migration issues caused by nil references to deleted users & groups
0.8.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.7.0	Added option to output all information about the user completing the task
0.6.4	Fixed wrong name when detecting exception in listing user assignments
0.6.3	Added styling to parameter help text.
0.6.2	Fixes backward compatibility with pre-0.6.0 version
0.6.1	Bumping up the version since I fixed the UI code

Version	Comment
0.6.0	Add The user names into the db for export import purpose.
0.5.1	Added help file
0.5.0	Adds support for update push (instead of polling)
0.4.8	Fix email notifications when no users are selected
0.4.7	New icon
0.4.6	Fix for additional email fields (to, cc, bcc) to be considered
0.4.4	Fix UI glitch (removing one input, removed all)
0.4.3	Making event description optional runtime input - doing it right
0.4.2	Making event description optional runtime input
0.4.1	Sets polling frequency to a default, switching UI to localtime
0.4.0	Adds support for required user input fields
0.3.3	Uses default mailer if field is left blank (requires 1.9.8 00010)
0.3.2	Isolates notification failures
0.3.1	Fixed problem in exposing the event description and correctly resolve the task description
0.3.1	Prevents failure of all notifications if one address is corrupted
0.3.0	Adds email notification
0.2.9	Fixes glitch in 0.2.8 preventing the outputs from being set
0.2.8	Only prepare actual inputs (ignore passed-thru variables)
0.2.7	Deghost active assignment before generating new ones
0.2.6	Clean {} null assignment thru migration
0.2.5	supports for legacy {} null assignment
0.2.4	improves backward compatibility
0.2.3	adds user and role selectors
0.2.2	adds support for file attachments
0.2.1	moved the templates directory out of install path
0.2.0	provides option to use as trigger
0.1.2	added input provider output
0.1.1	added event_description field (payload)
0.1.0	stable version used in IBC demo

12.6.1 How It Works

- Creates interactive HTML-based forms that appear in the Orchestrator Tasks tab for human users to complete and provide input.
- Supports configurable mandatory and optional inputs with various data types, allowing flexible data collection from users.
- Enables workflow pause functionality where execution waits for user completion of requested data before proceeding to the next step.
- Provides comprehensive email notification system with customizable recipients, subjects, and message content for task assignments.
- Offers user and group assignment capabilities with complete user information output options including names and email addresses.

12.6.2 Configuration Instructions

1. Add a User Input step to your workflow.
2. Configure the basic task settings:
    - Event description: Provide a clear description for the user task (mandatory).
    - View template: Select custom form template from available options.
    - On-going trigger: Check to keep plugin execution ongoing if needed.
3. Set up data collection requirements:
    - Mandatory requested data: Define required outputs users must provide.
    - Optional requested data: Define optional outputs for additional information.
    - Mandatory inputs: Configure required runtime inputs with data types.
    - Optional inputs: Configure optional runtime inputs with data types.
4. Configure user assignment and notifications:
    - Users: Assign specific users to the task.
    - Groups: Assign user groups to the task.



Figure 12.6: User input icon

- Send notification email: Enable email notifications with server and content settings.

12.6.3 Example Configuration

Parameter	Example Value
Event description	Review and approve transcoded media file
View template	_media_review.html.erb
On-going trigger	unchecked
Provide all user informations in outputs?	checked
Send notification email	checked
Mail server	smtp.company.com
Subject	Media Review Required - Workflow Task
Email body	Review the transcoded media file and provide approval.
Users	john.doe, jane.smith
Groups	media-reviewers, supervisors

12.6.4 Saved Parameters Description

12.6.5 Basic Parameters

Parameter	Description
Event description	The description of the user task that gets created on the tasks tab.
On-going trigger	The On-going trigger checkbox is to be checked if the plugin execution is to be kept ongoing.
View template	The template of the form that should be displayed when user clicks on the task. Template files should be located in '/opt/aspera/orchestrator/config/orchestrator/actions_config/user_inputs' and need to have their name start with '_'.
Provide all user informations in outputs?	If checked the action will return complete information about the user completing the tasks, including name, email address etc.
Mandatory inputs	Multiple runtime mandatory inputs can be created by adding Mandatory inputs. Input name and the data type should be selected for each input that is added.
Optional inputs	Multiple runtime optional inputs can be created by adding Optional inputs. Input name and the data type should be selected for each input that is added.

Parameter	Description
Mandatory requested data	Multiple mandatory inputs/data can be created by adding Mandatory requested data. Input name and the data type should be selected for each input that is added. These inputs act as plugin outputs and execution will not move to next step until these fields are provided with data.
Optional requested data	Multiple optional inputs/data can be created by adding Optional requested data. Requested data name and the data type should be selected for each input that is added. These inputs act as plugin outputs.

12.6.6 Email Notification Parameters

Parameter	Description
Send notification email	This checkbox is to be checked if email notification is to be sent upon execution of this plugin.
Mail server	Mail server that is to be used to send email notification.
Subject	Subject line of the email that will be sent.
Email body	Email body of the email that will be sent.
Additional to destinations	Additional 'to' email address apart from the users/groups (discussed below) that are selected.
Additional cc destinations	Additional 'cc' email address apart from the users/groups (discussed below) that are selected.
Additional bcc destinations	Additional 'bcc' email address apart from the users/groups (discussed below) that are selected.

12.6.7 User Assignment Parameters

Parameter	Description
Users	All the users that are supposed to be assigned with the user task that will be created by the execution of this plugin.
Groups	All the groups that are supposed to be assigned with the user task that will be created by the execution of this plugin.

12.6.8 Inputs Description

Event description is a mandatory field. The inputs created by the Mandatory inputs field are mandatory. If email notification is selected, Email subject and Email body are mandatory fields and should be populated.

- Event description: Required description of the user task (mandatory).
- Mandatory inputs: Runtime inputs marked as required with specified data types.
- Email subject: Subject line for notification emails (mandatory if email notifications enabled).
- Email body: Content for notification emails (mandatory if email notifications enabled).

12.6.9 Outputs Description

- Mandatory requested data: Mandatory requested data will appear as outputs and will serve as outputs to next step.
- Optional requested data: Optional requested data will appear as outputs and will serve as outputs to next step.

12.6.10 Operating Instructions

- The User Input plugin is designed to display and request information from human users through HTML-based forms that appear in the Orchestrator GUI's ”Tasks” tab.
- Custom forms must be located in the folder `/opt/aspera/orchestrator/config/orchestrator/actions_config/user_inputs/` with filenames that begin with ”_” and end with ”.html.erb” (example: `_myform.html.erb`).
- These forms will appear in the dropdown for 'View template' selection during plugin configuration.

- Example use case: A workflow that takes an input file and performs transcoding can include a UserInput plugin step to enable users to view the transcoded media and validate it. Upon validation, the workflow continues (with validation parameters as part of Mandatory requested data).
- Form customization is unlimited - server-side processing can be implemented with embedded Ruby (e.g., database access, file parsing), client-side processing with JavaScript (e.g., REST requests, HTML form generation), and GUI customization using standard web technologies like HTML5 and CSS3.

12.6.11 Dependencies

- Custom form templates must be properly placed in the designated directory with correct naming conventions.
- Email notification requires a configured and accessible mail server for sending task notifications.
- Assigned users and groups must exist in the Orchestrator system for proper task assignment.
- The aspweb user must have read/write permissions for the user_inputs configuration directory.

Chapter 13

Virus Scan

13.1 ClamAV check



Figure 13.1: ClamAV check icon

This action plug-in provides the ability to check a file, folder or set of files for viruses using the clamAV utility.

Version	Comment
0.3.1	Enhanced action template UI
0.3.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.2.1	Fix bug preventing to run concurrent clamav scans
0.2.0	Bug fix - typo times method
0.1.9	Fixing the process_output to match clamdscan output
0.1.8	Attemps file moves 3 times and errors out in case of failure to move
0.1.7	Changed the Category to Virus Scan
0.1.6	Fixes file-base option for move to clean and quarantine directories
0.1.5	Provides system path for service command and only performs validation on error
0.1.3	Adapted status check for calmv-daemon service
0.1.2	Adapted status check for non US OS
0.1.1	2015-05-23 PM - Correcting status check for CentOS 7
0.1.0	2015-03-06 PM - Better distinguish the failures that occur on a ClamAV scan
0.0.9	2015-03-06 PM - Check for CLAMAV status before scan
0.0.8	2015-02-17 ML - Fix display name for consistency
0.0.7	2015-02-17 ML - New icon
0.0.6	-> Fixed the command_line_get method so that MAX File Size to be entered into the command line call
0.0.5	-> Fixed the missing method for truncate_get
0.0.4	-> Corrected the clean file move, quarantine move and truncate functionality to use remote calls instead of assuming local
0.0.3	-> Adds help file
0.0.2	-> Fixes support for dynamic node
0.0.1	-> Initial release. Does not support add hoc node definition

13.1.1 How It Works

- Performs virus and malware scanning using ClamAV antivirus software on remote nodes.
- Executes ClamAV command-line tools (clamscan or clamdscan) via SSH connection.
- Scans individual files or entire directories for viruses, malware, and threats.
- Automatically moves clean files to staging folder if configured.
- Automatically quarantines infected files to isolation folder if configured.
- Supports file size limits to skip scanning of very large files.
- Can truncate infected files to zero-byte placeholders to obliterate infections.
- Maintains directory structure when moving files using configurable file base path.
- Returns detailed reports including clean files, infected files, and infection descriptions.

13.1.2 Configuration Instructions

1. Create a new action step using the ClamavCheck plugin.
2. Fill out the following fields:
    - Name: The name used to identify a saved ClamAV Check configured instance.
    - Comments: Comments about this saved ClamAV Check configured instance.
    - Execution Node: The node where the ClamAV command line operation must be executed (i.e., the node where the ClamAV application is installed).
    - Clamav binary path: The path to locate ClamAV command line executable, typically clamscan or clamdscan. Not needed if the binary folder is in the system path for the account used to remote connect on the specified server.
    - To be scanned file or directory path: The path to the file or directory to be scanned. Can use dynamic attributes (e.g., '/ingest/<%=file_basename%>').
    - Clean file staging folder: If provided, files that pass the AV scan will be moved to this folder. The file base is used to maintain tree structure if provided.
    - Quarantine folder: If provided, infected files will be moved to this folder. The file base is used to maintain tree structure if provided.
    - File base: If provided, it is applied to clean file staging and quarantine folders when files are moved. Represents the part of the full path that is not kept (e.g., file base '/ingest' removes this prefix when moving files).
    - Fail on infection?: Indicates whether the step should exit in failure or complete if an infection is found.
    - Truncate infected files?: Specifies whether infected files should be reduced to zero-byte files, obliterating the infection but keeping a file placeholder.
    - Scanned maximum file size: Specifies a maximum size over which files won't be submitted to the scanner (e.g., '100M', '1G').
    - Action if above max size: Selects whether to block or pass files that are skipped because they are above the maximum size set.
    - Additional Command line arguments: Any optional parameters supported by the virus scanner command line binary. These options will be passed on the command line as first arguments, after the command line executable.

13.1.3 Example Configuration

This is an example configuration for the ClamavCheck plugin to scan incoming media files with automatic quarantine.

Parameter	Value
Name	Scan Incoming Media Files
Comments	Virus scan all incoming media files before processing
Execution Node	SecurityServer01
Clamav binary path	/usr/bin/clamdscan
To be scanned file or directory path	/ingest/<%=filename%>
Clean file staging folder	/clean
Quarantine folder	/quarantine
File base	/ingest
Fail on infection?	Selected
Truncate infected files?	Not selected
Scanned maximum file size	500M
Action if above max size	Pass
Additional Command line arguments	--verbose

13.1.4 Saved Parameters Description

Parameter	Description
Name	The name used to identify a saved ClamAV Check configured instance.
Comments	Comments about this saved ClamAV Check configured instance.
Execution Node	The node where the ClamAV command line operation must be executed (i.e., the node where the ClamAV application is installed).
Clamav binary path	The path to locate ClamAV command line executable, typically clamscan or clamdscan. Not needed if the binary folder is in the system path for the account used to remote connect on the specified server.
To be scanned file or directory path	The path to the file or directory to be scanned. Can use dynamic attributes (e.g., '/ingest/<%=file_basename%>').
Clean file staging folder	If provided, files that pass the AV scan will be moved to this folder. The file base is used to maintain tree structure if provided.
Quarantine folder	If provided, infected files will be moved to this folder. The file base is used to maintain tree structure if provided.
File base	If provided, it is applied to clean file staging and quarantine folders when files are moved. Represents the part of the full path that is not kept (e.g., file base '/ingest' removes this prefix when moving files).
Fail on infection?	Indicates whether the step should exit in failure or complete if an infection is found.
Truncate infected files?	Specifies whether infected files should be reduced to zero-byte files, obliterating the infection but keeping a file placeholder.
Scanned maximum file size	Specifies a maximum size over which files won't be submitted to the scanner (e.g., '100M', '1G').
Action if above max size	Selects whether to block or pass files that are skipped because they are above the maximum size set.
Additional Command line arguments	Any optional parameters supported by the virus scanner command line binary. These options will be passed on the command line as first arguments, after the command line executable.

13.1.5 Inputs Description

The list of inputs depends on the configuration of the ClamAV Check action template as the fields for the paths and options all support dynamic inputs.

Parameter	Description
Input_file_path	By default, a file name must be provided as an input. However, if the 'To be scanned file or directory path' field is populated, then instead all the dynamic variables listed in it are mandatory and must be of type string.
Options	If 'Additional Command line arguments' is left blank, an optional 'Options' input is provided. However, if it is populated, all the dynamic variables listed in it will be mandatory and must be of type string.

13.1.6 Outputs Description

Parameter	Description
Clean_files	An array containing the path to the files that passed scanning
Infected_files	An array containing the path to the files that failed scanning
Infection_report	The text description of the infections found reported by the scanner, if any
Output_file_path	The path to the file submitted for scanning where it lands after scanning (can be moved if clean staging or quarantine directories are specified)

13.1.7 Supported Actions

None

13.1.8 Dependencies

ClamAV must be installed on the execution node. The connection to the remote server is done over SSH, so ensure the target server has SSH enabled.

13.1.9 Operating Instructions

This action executes the specified file ClamAV scanning operation on a remote node where a ClamAV command line is available. The connection to the remote server is done over SSH.

clamscan vs. clamdscan

There are two different command line executables available:

- clamscan (Standalone Scanner):
  - Stateless executable - fully loaded every time it runs
  - Results in a few seconds delay for each file scan
  - No daemon required - simpler setup
  - Suitable for occasional scanning or batch operations
  - Command: `/usr/bin/clamscan`
- clamdscan (Daemon Client):
  - Client that communicates with clamd daemon running in background
  - No load time delay - much faster for frequent scans
  - Requires clamd daemon to be running
  - Suitable for high-volume, real-time scanning
  - Command: `/usr/bin/clamdscan`

File Movement and Quarantine Logic

- File Base Path Concept:
- The file base represents the part of the full path that is not kept when moving files.
- Example:
  - Full file path: `/ingest/sub1/sub2/file.ext`
  - File base: `/ingest`
  - Quarantine folder: `/quarantine`
  - Infected file moved to: `/quarantine/sub1/sub2/file.ext`
  - The tree structure `sub1/sub2` is preserved
- Clean File Movement:
  - If 'Clean file staging folder' is specified, files that pass scanning are moved there
  - Original directory structure is maintained using file base
  - Example: Clean to `/clean/sub1/sub2/file.ext`
- Infected File Quarantine:
  - If 'Quarantine folder' is specified, infected files are isolated there
  - Original directory structure is maintained using file base
  - Prevents accidental access to infected files
  - Example: Quarantine to `/quarantine/sub1/sub2/file.ext`

Common ClamAV Command Line Options

- Scanning Options:
  - `--recursive` or `-r` - Scan directories recursively
  - `--infected` or `-i` - Only print infected files
  - `--remove` - Remove infected files
  - `--move=DIRECTORY` - Move infected files to specified directory
  - `--exclude=REGEX` - Exclude files matching regular expression
  - `--exclude-dir=REGEX` - Exclude directories matching regular expression
- Output Options:
  - `--verbose` or `-v` - Be verbose
  - `--quiet` - Only output error messages
  - `--no-summary` - Disable summary at end of scanning
- Performance Options:
  - `--max-filesize=SIZE` - Files larger than SIZE won't be scanned

- `--max-scansize=SIZE` - Max amount of data to scan for each file
- `--max-recursion=NUMBER` - Max directory recursion level

Common Use Cases

- Incoming file quarantine: Scan files as they arrive and quarantine any infections before processing.
- Media file validation: Ensure video/audio files are virus-free before transcoding or distribution.
- Archive scanning: Scan archived content before retrieval or restoration.
- Upload validation: Scan user-uploaded files before storing in production systems.
- Periodic scanning: Regular scans of file repositories to detect new threats.
- Pre-delivery scanning: Scan files before delivery to external parties or clients.
- Compliance scanning: Meet security compliance requirements with automated AV scanning.

Best Practices

- Use clamdscan with daemon for high-volume scanning to avoid load-time delays.
- Keep virus definitions up to date with regular freshclam updates (automate with cron/scheduled task).
- Configure clean file staging and quarantine folders for automatic file segregation.
- Set appropriate maximum file size limits to avoid scanning extremely large files that tie up resources.
- Use file base paths to maintain directory structure when moving files.
- Enable 'Fail on infection?' for critical workflows where infected files should halt processing.
- Implement quarantine monitoring to review and analyze infected files.
- Ensure files are accessible from the scanning node (local or network share).
- Test ClamAV commands manually via SSH before configuring in Orchestrator.
- Monitor clamd daemon status if using clamdscan to ensure continuous availability.

Important Notes

- File Accessibility: This action plugin does not perform any file movement before the scan. The specified file must be accessible from the execution node (local path or network share).
- Virus Database Updates: Keep ClamAV virus definitions current using the `freshclam` command (typically automated via a cron job).
- Daemon Requirement: If using `clamdscan`, ensure the `clamd` daemon is running.
- Path Formats: Provide explicit paths to ClamAV binaries to avoid configuration and PATH-related issues.
- Windows Paths: Use Unix-style paths when executing on Windows nodes (forward slashes, optional Cygwin POSIX format).
- SSH Shell: On Windows, configure the SSH user with a `/bin/sh` shell, not `/bin/switch`.
- File Base Requirement: File base is applied only when clean staging or quarantine folders are configured.
- Truncation Behavior: Truncating infected files removes the infection while retaining a placeholder, which is useful for audit and tracking.

Troubleshooting

- ClamAV not found: Verify ClamAV is installed and provide an explicit binary path. Use `which clamscan` on Linux.
- clamd connection refused: Ensure the `clamd` daemon is running when using `clamdscan`: `sudo systemctl start clamav-daemon`.
- File not accessible: Verify the file exists and is accessible from the execution node. Check file permissions.
- Outdated virus definitions: Update the virus database using `sudo freshclam`.
- Permission denied on quarantine: Ensure the SSH user has write permissions for the quarantine and clean staging folders.
- SSH connection failure: Verify the Remote Node configuration and confirm the SSH service is running.
- Windows path issues: Use Unix-style paths with forward slashes, for example `C:/path/file.ext`.
- Large file timeouts: Configure an appropriate maximum file size or increase SSH timeouts for very large files.
- False positives: Review infection reports and configure ClamAV exclusions if required.

Understanding ClamAV Output

ClamAV provides detailed output about scan results:

\ /path/to/file.txt: OK\ /path/to/infected.exe: Trojan.Generic FOUND\ /path/to/suspicious.doc: Heuristics.Phishing.Email FOUND\ \ ----------- SCAN SUMMARY -----------\ Infected files: 2\ Time: 5.234 sec (0 m 5 s)\

Output indicators:

- OK – File is clean
- FOUND – Virus or malware detected

- Infection names – Indicate the type of threat (Trojan, Heuristic, Worm, etc.)
- Scan summary – Overall scan statistics

The plugin parses this output to populate the `Clean_files` and `Infected_files` arrays and the `Infection_report` text.

13.2 ICAP virus scan



Figure 13.2: ICAP virus scan icon

This action plug-in communicates with an ICAP enabled virus scanner engine to quarantine infected files.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.9	Making plugin compatible with 3.0.x
0.0.8	Added styling to parameter help text.
0.0.7	Changed category to Virus Scan
0.0.6	Adding help file
0.0.5	0.0.5 Adds display name
0.0.4	0.0.4 adds generation of test virus file
0.0.3	0.0.3 modifies process loop to avoid lock-up
0.0.2	0.0.2 genericized to work with symantec icap client ssecls and ssecls.jar
0.0.1	0.0.1 initial creation works with LINUX icap-client command

13.2.1 Saved Parameters Description

- Name: The name used to identify a saved IcapVirusScan configured instance.
- Comments: Some comments about this saved IcapVirusScan configured instance.
- ICAP server address: The address of the server hosting the virus scan
- Client Command: The command to be executed can be entered here
- Result Parsing Command: The result is parsed usually with the return code 0/1. If a custom parsing needs to be done, thats entered here
- List infection command: Infection found is identified by matching the regex ”X-Infection-Found”. If custom identification code is needed that can be entered here
- Keep trigger on going: This field is to keep looking for multiple files as soon as 1 is done
- Scan Directory: The directory from where the files has to be scanned
- To-be-scanned filename pattern: The pattern of the files to look out for in the given directory
- Filename base (leading part to remove from full name): The base name of the file to be scanned
- Use Regex Pattern matching?: By default, GLOB matching is used. By checking this REGEX can be used instead for more complex patterns (i.e. 'movie_.*.(avi|mpg)')
- Max file size (in bytes): The maximum size of the file that should be considered
- Action for files above max size: Choose beterrn Pass/Fail/Leave/Error defaultly without called the virus_scan at all
- Scan frequency (seconds): The frequency for looking at the incoming files
- Triggers on partial file? : If checked the scan will also be triggered on partial file
- Clean directory: The directory to which the clean files have to be moved after the scan
- Quarantine directory : The directory to which the infected files have to be moved after the scan

- Scan suffix: The suffix to be appended to the file while scanning(example .scanning)
- Truncate infected file to 0 bytes?: Truncates the infected files to 0 bytes if checked
- Infection manifest directory: The directory into which the infection manifest file is written to

13.2.2 Inputs description

The list of inputs depends on the configuration of the IcapVirusScan action template.

13.2.3 Outputs description

- *Clean file list*: List of files that are found clean
- *Quarantined_file_list*: List of files that are found infected
- *Infection_manifest*: This gives the location to the manifest file if asked for

13.2.4 Supported Actions

None

13.2.5 Dependencies

None

13.2.6 Operating Instructions

This Plugin helps to stream data to a server using the ICAP protocol for virus scanning. (Verify if the anti-virus software being used supports ICAP before using this plugin)

Basic Example:

Server Address: 10.0.142.20

File_list : *.txt

Scan Directory: /tmp

Keep trigger on-going? : checked

this will scan all the files under the /tmp folder that has the ”txt” extension.

13.3 McAfee virus check



Figure 13.3: McAfee virus check icon

This action plug-in provides the ability to check a file, folder or set of files for viruses using the McAfee anti-virus application.

Version	Comment
0.2.0	Updating to be compatible with Ruby 3.4 and Rails 8

Version	Comment
0.1.9	Fixes UTF-8 encoding in response
0.1.8	Changed Category
0.1.7	Added styling to help text.
0.1.6	Add Remote Node Dependency
0.1.5	For windows, ensure that file is recognized as infected even when exit code is 0, loosen matching of file pattern
0.1.4	Adds display name
0.1.3	New icon
0.1.2	-> Formatted lines in help file
0.1.1	-> Adding double quotes for all filenames in the command line calls to escape whitespace. Corrects commands for Windows and Unix
0.1.0	-> Extended support for file size limit, check for node type, help file, move to clean folder, move to quarantine folder
0.0.1	-> Initial release. Only single file support for McAfee in single file mode

13.3.1 Saved Parameters Description

- Name: The name used to identify a saved McAfeeAntiVirusScan configured instance.
- Name: Name of the template
- Comments: (optional) Provide comments to define the template
- Execution Node: Select node from the dropdown which has the McAfee antivirus installed on. Visit Remote Nodes page for node creation.
- Node Type: Check the box if the target server is Unix. Else it will get considered or Windows.
- Binary Path: Path to the McAfee scan utility
- Input File Path: Full path to the file which has to be scanned
- Quarantine Folder: If provided, McAfee will move the provided file to this folder, if it fails virus check.
- Delete Infected File: If checked, McAfee will delete the provided file, if it fails virus check.
- Clean Folder: If provided, Orchestrator will move the provided file to this folder, if it passes virus check.
- File base: If provided, the part of the file name will be removed and the rest of it will be created under target folder
- Fail on infection: Fail the step if the virus check fails. By default, step will compelte
- Truncate Infected File:
- Options: Refer to the list of options below and provide extra options.
- Max Scan Size: Ignore files above the specified size (in MB)

13.3.2 Inputs description

The list of inputs depends on the configuration of the McAfeeAntiVirusScan action template.

- *Execution Node*: If a 'Execution Node' is not provided in the configuration, it can be provided as an runtime input.
- *Input file path*: If a 'Input file path' is not provided in the configuration, it can also be provided as an runtime input. Also, if dynamic attributes are used in the 'Watch directory' or 'Folder pattern', they will be required inputs typed as strings.

13.3.3 Outputs description

- *Clean_files*: Returns and array with full path of clean files
- *Infected_files*: Returns and array with full path of infected files
- *Scan_report*: Returns a report of the McAfee virus scan execution on provided file
- *Scan_Summary*: Returns a summary of the McAfee virus scan execution on provided file
- *Output_file_path*: Returns and array with full path of clean files

13.3.4 Supported Actions

None

13.3.5 Operating Instructions

This plugin does not support value overwrites: if a value is specified in the template it will not be available as input of the step at runtime.

McAfee VirusScan Windows Command Line options:

/? : Display this help screen.

/AD : Scan all drives (not removable media).

/ADL : Scan all local drives (not removable media).

/ADN : Scan all network drives.

/AFC=<ache size> : Set the Size(in MB) of the Internal Cache Used When Decompressing Archive Files.

/ALL : Scan all files regardless of filename extension.

/ALLOLE : Treat all files as compound/OLE regardless of extension.

/ANALYZE : Turn on heuristic analysis for programs and macros.

/APPEND : Append to report file rather than overwriting.

/APPENDBAD : Append to bad file rather than overwriting.

/ASCII : Display filenames as ASCII text.

/BADLIST=<filename> : Filename and path for bad list log file.

/BOOT : Scan boot sector and master boot record Only.

/CHECKLIST=<filename> : Scan list of files contained in <filename>.

/CLEAN : Attempt to clean infected files.

/CONTACTFILE=<filename> : Display contents of <filename> when a virus is found.

/DAM : Remove all macros from infected MS Office files.

/DEL : Delete infected files except archive files.

/DOHSM : Scan migrated files(hierarchical storage management).

/DRIVER=<dir> : Directory specifying location of DAT files.

/EXCLUDE=<filename> : Do not scan files/directories listed in <filename>.

/EXTENSIONS : Scan defaults & user extension list.

/EXTLIST : List file-extensions scanned by default.

/EXTRA=<filename> : Specify the full path and file name of any extra.dat file.

/FAM : Find all macros - not just infected macros. Used with /DAM will remove all macros.

/FDC : Force digital signature check.

/FREQUENCY=<hours> : Do not scan <hours> after the previous scan.

/HELP : Displays this help.

/HTML=<filename> : Create and specify a HTML report file.

/LOAD=<filename> : Load options from <filename>.

/LOUD : Include all scanned files in the /REPORT file.

/MAILBOX : Scan inside plain text mailboxes.

/MANALYZE : Turn on macro heuristics.

/MANY : Scan many floppy diskettes.

/MAXFILESIZE=<size> : Examine Only those files smaller than the specified size(in MB).

/MEMSIZE=<size> : File size(in KB) to load into memory for scanning limited by a maximum file size defaulting to 1MB.

/MIME : Scan inside MIME, UUE, XXE and BinHex files.

/MOVE=<dir> : Move infected file into directory <dir>, preserving path.

/NOBKSEM : Prevent scanning of files that are normally protected.

/NOBOOT : Do not scan boot sectors.

/NOBREAK : Disable Ctrl-C / Ctrl-Break during scanning.

/NOCOMP : Do not scan self extracting executables by default.

/NOD : Don't switch into /ALL mode when repairing.

/NODDA : Do not scan boot sectors.

/NODECRYPT : Don't scan password-protected MS Office documents.

/NODOC : Do not scan MS Office files.

/NOEXPIRE : Disable data files expiration date notice.

/NOJOKES : Do not alert on joke files.

/NOMEM : Do not scan memory for viruses.

/NORECALL : Do not move files from remote storage into local storage after scanning.

/NORENAME : Do not rename infected files that cannot be cleaned.

/NOSCRIPT : Do not scan files that contain HTML, JavaScript, Visual Basic, or Script Component Type Libraries.

/PANALYZE : Turn on program heuristics.

/PAUSE : Pause at end of each screen page.

/PLAD : Preserve the last-accessed time and date for files that are scanned.

/PROGRAM : Scan for potentially unwanted applications.

/RECURSIVE : Examine any subdirectories in addition to the specified target directory.

/REPORT=<filename> : Report names of viruses found into <filename>.

/RPTALL : Include all scanned files in the /REPORT file.

/RPTCOR : Include corrupted files in /REPORT file.

/RPTERR : Include errors in /REPORT file.

/RPTOBJECTS : Reports number of objects at all levels scanned in summary.

/SECURE : Equivalent to Analyse, doall, unzip.

/SHOWCOMP : Report any files that are packaged.

/SILENT : Disable all screen output.

/STREAMS : Scan inside NTFS streams (NT & DATAPOL Only).

/SUB : Examine any subdirectories in addition to the specified target directory.

/THREADS=<nn> : Set scan thread count.

/TIMEOUT=<seconds> : Set the maximum time to spend scanning any one file.

/UNZIP : Scan inside archive files, such as those saved in ZIP, LHA, PKarc, ARJ, TAR, CHM, and RAR.

/VERSION : Display the scanner's version number.

/VIRLIST : Display virus list.

/WINMEM[=<pid>] : If pid given scans the Windows Process with Process ID <pid> otherwise scans all Windows Processes.

/XMLPATH=<filename> : Filename and path for XML log file.

McAfee VirusScan Unix Command Line options:

--? : Display this help screen.

--AFC=<ache size> : Set the Size(in MB) of the Internal Cache Used When Decompressing Archive Files.

--ALLOLE : Treat all files as compound/OLE regardless of extension.

--ANALYZE : Turn on heuristic analysis for programs and macros.

--APPEND : Append to report file rather than overwriting.

--APPENDBAD : Append to bad file rather than overwriting.

--ASCII : Display filenames as ASCII text.

--ATIME-PRESERVE : Preserve the last-accessed time and date for files that are scanned.

--BADLIST=<filename> : Filename and path for bad list log file.

--BOOT : Scan boot sector and master boot record Only.

--CHECKLIST=<filename> : Scan list of files contained in <filename>.

--CLEAN : Attempt to clean infected files.

--CLEANDOCALL : Remove all macros from infected MS Office files.

--CONFIG=<filename> : Load options from <filename>.

--CONTACTFILE=<filename> : Display contents of <filename> when a virus is found.

--DAM : Remove all macros from infected MS Office files.

--DAT=<dir> : Directory specifying location of DAT files.

--DATA-DIRECTORY=<dir> : Directory specifying location of DAT files.

--DEL : Delete infected files except archive files.

--DRIVER=<dir> : Directory specifying location of DAT files.

--EXCLUDE=<filename> : Do not scan files/directories listed in <filename>.

--EXIT-ON-ERROR : Exit on error.

--EXTENSIONS : Scan defaults & user extension list.

--EXTLIST : List file-extensions scanned by default.

--EXTRA=<filename> : Specify the full path and file name of any extra.dat file.

--FAM : Find all macros - not just infected macros. Used with /DAM will remove all macros.

--FILE=<filename> : Scan list of files contained in <filename>.

--FLOPPYA : Scan the boot sector of the disk in drive A.

--FLOPPYB : Scan the boot sector of the disk in drive B.

--FREQUENCY=<hours> : Do not scan <hours> after the previous scan.

--HELP : Displays this help.

--HTML=<filename> : Create and specify a HTML report file.

--IGNORE-COMPRESSED : Do not scan self extracting executables by default.

--IGNORE-LINKS : Do not resolve any symbolic links and do not scan the link targets.

--LOAD=<filename> : Load options from <filename>.

--LOUD : Include all scanned files in the /REPORT file.

--MACRO-HEURISTICS : Turn on macro heuristics.

--MAILBOX : Scan inside plain text mailboxes.

--MANALYZE : Turn on macro heuristics.

--MAXFILESIZE=<size> : Examine Only those files smaller than the specified size(in MB).

--MEMSIZE=<size> : File size(in KB) to load into memory for scanning limited by a maximum file size defaulting to 1MB.

--MIME : Scan inside MIME, UUE, XXE and BinHex files.

--MOVE=<dir> : Move infected file into directory <dir>, preserving path.

--NOBOOT : Do not scan boot sectors.

--NOCOMP : Do not scan self extracting executables by default.

--NOD : Don't switch into /ALL mode when repairing.

--NODECRYPT : Don't scan password-protected MS Office documents.

--NODOC : Do not scan MS Office files.

--NOEXPIRE : Disable data files expiration date notice.

--NOJOKES : Do not alert on joke files.

--NORENAME : Do not rename infected files that cannot be cleaned.

--NOSCRIPT : Do not scan files that contain HTML, JavaScript, Visual Basic, or Script Component Type Libraries.

--ONE-FILE-SYSTEM : Scan an entire directory tree without scanning mounted file systems.

--PANALYZE : Turn on program heuristics.

--PAUSE : Pause at end of each screen page.

--PLAD : Preserve the last-accessed time and date for files that are scanned.

--PROGRAM : Scan for potentially unwanted applications.

--RECURSIVE : Examine any subdirectories in addition to the specified target directory.

--REPORT=<filename> : Report names of viruses found into <filename>.

--RPTALL : Include all scanned files in the /REPORT file.

--RPTCOR : Include corrupted files in /REPORT file.

--RPTERR : Include errors in /REPORT file.

--RPTOBJECTS : Reports number of objects at all levels scanned in summary.

--SECURE : Equivalent to Analyse, doall, unzip.

--SELECTED : Look for infections in any file that has execute permissions.

--SHOWCOMP : Report any files that are packaged.

--SILENT : Disable all screen output.

--SUB : Examine any subdirectories in addition to the specified target directory.

--SUMMARY : Produce a summary of the scan.

--THREADS=<nn> : Set scan thread count.

--TIMEOUT=<seconds> : Set the maximum time to spend scanning any one file.

--UNZIP : Scan inside archive files, such as those saved in ZIP, LHA, PKarc, ARJ, TAR, CHM, and RAR.

--VERBOSE : Display a progress summary during the scan.

--VERSION : Display the scanner's version number.

--VIRLIST : Display virus list.

--VIRUS-LIST : Display virus list.

--XMLPATH=<filename> : Filename and path for XML log file.

13.4 Opswat metadefender

Integration with Opswat Metadefender

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.1	Initial implementation



Figure 13.4: Opswat metadefender icon

13.4.1 Saved Parameters Description

- Name: The name used to identify a saved OpswatMetadefender configured instance.
- OPSWat Server Address: OPSWAT Metadefender server address
- Port: OPSWAT Metadefender server port, default is 8008
- apikey: Optional, APIKEY to use with scan requests. Default is empty API ey
- username: Optional, username to login to Metadefender. The resulting session will be used for scan requests. Default is empty username
- password: Optiona, password to login to Metadefender. Default is empty password
- Fail if infected file found: Causes the step to fail if an infected file was flaged by Metadefender
- files: List of files, separated by the separator below, to scan
- separator: Optional, by default is is a comma
- user_agent: Optional, user_agent value will be used in all requests headers to Metadefender. Default is Aspera Orchestrator
- rule: Optional, Metadefender rule to request. By default it is File scan
- archivepwd: Optional, if you are scanning a password protected archive, provide the password. Default is empty password
- Poll status every: Optional. How often to check for scan job status. By default it is 5 seconds

13.4.2 Inputs description

- *Files*: MandatoryList of files to scan.
- *opswat_address*: Mandatory. OPSWAT Metadefender server address

13.4.3 Outputs description

- *Scan result*: Hash. Provides the scan result of each scanned file
- *Clean Files*: Array. List of clean files
- *Infected Files*: Array. List of infected files
- *Sanitised Data IDs*: Array. List of data_id to files that have been sanitisation and need to be downloaded

13.4.4 Supported Actions

None

13.4.5 Dependencies

None

13.4.6 Operating Instructions

Provide a list of files separated by the specified separator. The result is 2 arrays, one containing list of clean files and the other one contains list of infected files. IF you selected ”Fail if infected file found”, then the step will fail, otherwise, it will succeed. The hash outpput is structured as following:

- *Key:* file path. *Value:* A hash:

- *Key:* data_id. *Value:* data_id, as it comes back from Metadefeneder
- *Key:* result. *Value:* Result returned by Metadefender REST call for scanning the file

To download sanitised files, use CURL plugin with a custom command like this:

cd DOWNLOAD_PATH ; curl -X GET http://OPSWAT_ADDRESS:OPSWAT_PORT/file/converted/DATA_ID?apikey=APIKEY -O -J

replace the upper case names to variables and map them as input

You need to get an API_KEY. You can either assigned an APIKEY to the user inside Metadefender dashboard (accounts) or login and use the session_id as apikey

13.5 Symantec Icap DLP Operations



Figure 13.5: Symantec Icap DLP Operations icon

This plugin is used to make a ICAP call to Symantec DLP Engine.

Version	Comment
0.1.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.0.1	initial release

13.5.1 Saved Parameters Description

- Name: The name used to identify a saved SymantecIcapDlpOperation configured instance.
- Service Ip: Provide the ip of the Symantec server.
- Port: Provide the port of the CMM server.
- Service Name: Provide the of the anti-virus scan utility.
- File Name: Name of the file to be scanned.
- Sender Email: Email id of the sender
- Receiver Email: Email id of the receiver.
- Timeout Value: Timeout value for the streaming operation

13.5.2 Inputs description

- Document all available inputs using add_input_help
- *All Inputs*: These inputs are made mandatory for the ICAP call to be executed.

13.5.3 Outputs description

- *Raw_Execution_Output*: Shows the raw execution output of the call
- *Log_File_Path*: Shows the path to the log file where the outputs are saved.

13.5.4 Supported Actions

None

13.5.5 Dependencies

None

13.5.6 Operating Instructions

This plugin is used to make a call to the Symantec DLP utility.

13.6 Symantec Icap Virus Scan Operations



Figure 13.6: Symantec Icap Virus Scan Operations icon

This plugin is used to make a ICAP call to Symantec Antivirus Engine.

Version	Comment
0.4.0	Updating to be compatible with Ruby 3.4 and Rails 8
0.3.0	Add database migration
0.2.2	Fixing bug in the command line option code
0.2.1	fixing readline bug
0.2.0	adding command line execution
0.1.4	Fixed code for remote execute after testing on SCB.
0.1.3	Fixed process_output to return hash instead of string
0.1.2	Fixed bug in the service_ip get method
0.1.1	Fixed variable name, removed the encrypted part of the request and added an extra parameter to see raw log file content
0.1.0	Adding Remote Node execution option
0.0.1	initial release

13.6.1 Saved Parameters Description

- Name: The name used to identify a saved SymantecIcapVirusScanOperation configured instance.
- Operation Type: We support two types. Command line Symantec Utility execution or to execute our jar file
- is Remote Execution: Check this if the execution is on a remote node
- Remote Node: Choose the remote node for the execution
- Remote Java jar path: Provide the path along with the file name for the Symantec virus scan Java jar file
- Service IP: Provide the IP of the Symantec server.
- Port: Provide the port of the CMM server.
- Service Name: Provide the of the anti-virus scan utility.
- File Name: Name of the file to be scanned.
- File Size: Size of the file to be scanned
- Send Size: The size of the date to be streamed. Default(8192)
- Local Jar Path: For command line utility enter the jar path along with the file name
- action: Choose from the list of actions the command line utility supports
- clobber: select this if you want to always overwrite the scanned file with server response
- verbose: select this option if you want to print the scan result on standard output
- api_version: Select the appropriate API version

13.6.2 Inputs description

- Document all available inputs using add_input_help
- *Service IP, Port, Service Name and File Name*: These inputs are made mandatory for the ICAP call to be executed.

13.6.3 Outputs description

- *Raw_Execution_Output*: Shows the raw execution output of the call
- *Log_File_Path*: Shows the path to the log file where the outputs are saved.

13.6.4 Supported Actions

None

13.6.5 Dependencies

None

13.6.6 Operating Instructions

This plugin is used to make a call to the Symantec Anti-Virus utility.

End of document